

Practical Biostatistics in 30 Days

↺ Reset Interpreter 304 code blocks • state persists between runs • use ▶▶ to auto-run dependencies

From p-values to pipelines — a structured journey through the statistics every biologist actually needs.

You have the data. Thousands of gene expression measurements. Hundreds of patient outcomes. Millions of variants. You know the biology. You understand the experiment. But when it comes time to choose a statistical test, set a significance threshold, or interpret a confidence interval, the ground shifts under your feet.

This book fixes that. In 30 days, you will go from statistical anxiety to statistical fluency — not by memorizing formulas, but by solving real biological problems with real data. Every test you learn has a reason. Every formula has a story. Every p-value has a context.

And you will do it all in BioLang, whose current runtime provides hundreds of builtins across statistics, data handling, biological formats, and visualization. That lets you express an analysis — from data loading to hypothesis testing to publication-quality visualization — in readable, pipe-chained steps.

Who This Book Is For

This book is for anyone who works with biological data and needs to make sound statistical decisions. You might be:

- **A biologist who dreads the statistics section.** You can design elegant experiments, but when the reviewer asks why you used a t-test instead of a Mann-Whitney U, you panic. You have tried statistics textbooks, but they are full of coin flips and card games when you need differential

expression and survival curves. This book teaches statistics through biology, using datasets and questions you actually care about.

- **A developer entering biotech.** You can write production code and build data pipelines, but you do not know the difference between a parametric and a non-parametric test. You have heard that bioinformatics requires “statistical thinking,” but nobody has explained what that means in practice. This book gives you the statistical intuition alongside the implementation, so you understand *why* you are computing a fold change, not just *how*.
- **A graduate student facing qualifying exams.** Your program expects fluency in biostatistics, but your coursework is a blur of Greek letters and proof sketches. You need a practical guide that connects the math to the biology and shows you how to actually run these tests on real data. This book builds that bridge in 30 structured days.
- **A clinical researcher designing or analyzing studies.** You work with patient cohorts, treatment outcomes, and survival data. You need to choose the right test, compute adequate sample sizes, and report results that satisfy both statisticians and regulatory reviewers. This book covers clinical biostatistics end to end — from power analysis through Cox proportional hazards.

No matter which category you fall into, you share one thing: you want statistical skills that solve real problems, not abstract exercises. Every day in this book produces an analysis you can adapt to your own data.

Your Path Through the Book

The first week builds foundations for every reader, but your starting point may differ. Here is which days to prioritize based on your background:

Your background	Focus on	Skim or review
Biologist, limited stats training	Days 1-3 (distributions, central tendency, variability)	Day 4 if you already know probability basics
Statistician, new to biology	Days 5-7 (biological context for common tests)	Days 1-3 (you know the math already)
New to both stats and biology	Every day — they are written for you	Nothing — read it all
Some stats background	Skim Week 1 for BioLang syntax, start deeply at Week 2	Days 1-4 for review only

Complete beginner? That is completely fine. Day 1 starts with a single concept — what is a distribution? — and builds from there. No calculus. No linear algebra. No prior programming experience beyond basic BioLang familiarity. If you can type `bl run script.bl`, you are ready.

What You Will Learn

Over 30 days, you will go from statistical uncertainty to being able to:

- Describe and summarize any biological dataset (distributions, central tendency, spread, outliers)
- Choose the correct statistical test for any experimental design
- Perform and interpret t-tests, ANOVA, chi-square tests, and their non-parametric alternatives
- Run linear and logistic regression on biological data
- Analyze time-to-event data with Kaplan-Meier curves and Cox proportional hazards models
- Reduce high-dimensional data with PCA and interpret biplots
- Cluster samples and genes using hierarchical and k-means methods

- Correct for multiple testing with Bonferroni, Benjamini-Hochberg, and permutation approaches
- Compute effect sizes, confidence intervals, and statistical power
- Design experiments with proper sample size calculations
- Build volcano plots, Manhattan plots, Q-Q plots, and forest plots
- Apply Bayesian reasoning to biological problems
- Complete three capstone analyses that mirror real research publications

You will learn all of this in BioLang, which provides dedicated builtins for every test and method. But you will not be locked in. Every day includes comparison examples in Python (scipy/statsmodels) and R (base stats/survival/ggplot2), so you can translate your skills to any environment.

How This Book Is Structured

The book is organized into four weeks plus capstone projects:

Week	Days	Theme	What You Build
Week 1	1-5	Foundations	Understand distributions, probability, and descriptive statistics
Week 2	6-12	Core Methods	Master hypothesis testing, t-tests, ANOVA, chi-square, non-parametric tests
Week 3	13-20	Modeling	Correlation, regression, survival, design, effect size, and batch effects
Week 4	21-27	Advanced Topics	PCA, clustering, resampling, Bayesian methods, visualization, and reproducibility
Capstone	28-30	Projects	Clinical trial, differential expression, and GWAS analyses

Each day follows the same structure:

1. **The Problem** — a vivid scenario that shows *why* you need today's method. A researcher staring at ambiguous results. A clinician choosing between treatments. A graduate student defending a finding.
2. **What Is [Topic]?** — a plain-language explanation of the statistical concept, free of jargon. If your collaborator asked “what is a p-value?” at a coffee shop, this is how you would explain it.
3. **Core Concepts** — the ideas, assumptions, and mechanics, presented with tables, diagrams, and worked examples. Formulas appear when they clarify; they are never the point.
4. **[Topic] in BioLang** — working code that applies the concept to biological data. Pipe-chained, readable, annotated.
5. **Python and R Comparison** — the same analysis in scipy/statsmodels and R, so you can see how the languages compare.
6. **Exercises** — practice problems at three difficulty levels (Foundations, Applied, Challenge).
7. **Key Takeaways** — the essential points to remember, in bold-and-explanation format.

Days are designed to take 1-3 hours each. Concept-heavy days (like Day 1 on distributions) are shorter. Method-heavy days (like Day 14 on logistic regression) are longer. Work at your own pace — there is no penalty for spending two days on one topic.

Prerequisites

You need:

- **A computer** running Windows, macOS, or Linux
- **BioLang installed** — see the setup section below or [Appendix A](#)
- **Basic BioLang familiarity** — you can write variables, use pipes, and call functions. If you have completed *Practical Bioinformatics in 30 Days* or the BioLang tutorials, you are ready.
- **High school math** — you understand addition, multiplication, fractions, and basic algebra. That is all.

You do *not* need:

- A statistics course (this book *is* the course)
- Calculus or linear algebra (we explain everything from scratch)
- Prior experience with R, Python, or any statistics software
- A powerful machine (a laptop with 4 GB of RAM handles every exercise)

If you can run `bl --version` and get a version number, you are ready.

The Companion Files

Every day has a companion directory with a BioLang setup script and Python and R comparison scripts. The runnable BioLang analyses are the code blocks in the chapter itself. The current companion structure is:

```
biostatistics/  
  days/  
    day-01/  
      init.bl           # Setup script – run this first  
      scripts/  
        analysis.py     # Python equivalent  
        analysis.R      # R equivalent  
      expected/  
        .gitkeep        # Reserved for chapter-specific baselines  
    day-02/  
    ...
```

To use the companion files:

1. **Run `init.bl` first.** Each day's init script generates sample datasets, downloads reference data, or creates whatever that day's exercises need. Run it with `bl run init.bl`.
2. **Run the BioLang blocks in the chapter.** Blocks on the same page may build on variables defined earlier. File-backed and network examples are marked as CLI-only.

3. **Validate your script.** Run `bl check analysis.bl`, then compare important estimates with the Python and R scripts. Statistical results should agree within the stated rounding tolerance and method assumptions.
4. **Use `scripts/analysis.py` and `scripts/analysis.R` for independent validation.** Check test variants, missing-value behavior, correction methods, and model families before treating small numerical differences as errors.

To get the companion files:

```
git clone https://github.com/bioras/practical-biostatistics.git
cd practical-biostatistics
```

Or download the ZIP from the book's website and extract it.

Setting Up Your Environment

Full installation instructions are in [Appendix A](#), but here is the short version:

```
# Install BioLang
curl -sSf https://biolang.org/install.sh | sh

# Verify it works
bl --version

# Launch the REPL to test
bl repl
```

On Windows, use the PowerShell installer:

```
irm https://biolang.org/install.ps1 | iex
```

If you want to run the Python comparison scripts (optional but recommended):

```
pip install scipy numpy pandas matplotlib statsmodels lifelines
scikit-learn
```

If you want to run the R comparison scripts (optional but recommended):

```
install.packages(c("stats", "survival", "ggplot2", "dplyr", "pwr",
"lme4", "boot"))
```

A Quick Taste

Here is what statistical analysis looks like in BioLang. This script loads a differential-expression result table, adjusts p-values, filters significant genes, and generates a volcano plot:

```
# Load a compact differential-expression result table
let expression = read_csv("data/expression.csv")
let adjusted = p_adjust(col(expression, "pvalue"), "BH")
let results = zip(to_records(expression), adjusted)
  |> map(|pair| {...pair[0], padj: pair[1]})
  |> to_table()

# How many genes are differentially expressed?
let significant = results
  |> filter(|row| row.padj < 0.05 && abs(row.log2fc) > 1.0)
println("Differentially expressed genes: " + str(len(significant)))

# Volcano plot
volcano(results, {fc: "log2fc", p: "padj"})
```

The pipe operator makes the analytical logic visible: load, correct, combine, filter, and plot. You will understand every line by the end of Week 2.

Here is another example — survival analysis in three lines:

```
let patients = read_csv("data/clinical.csv")
let survival = patients |> map(|row| {
  time: row.survival_months,
  event: if row.status == "deceased" { 1 } else { 0 },
  group: row.treatment
}) |> to_table()
kaplan_meier(survival, {title: "Overall Survival by Treatment"})
```

And power analysis for planning your next experiment:

```
# Power calculation: how many samples per group?
let result = power_t_test(0.5, 0.05, 0.8)
println("Required sample size per group: {result.n}")
println("Effect size: {result.effect_size}, alpha: {result.alpha},
power: {result.power}")
```

BioLang's statistical, tabular, and visualization builtins keep the analysis close to the biological question.

Week-by-Week Overview

Week 1: Foundations (Days 1-5)

You start where every statistical analysis starts — with the data itself. Day 1 explains why statistics matters; Day 2 covers descriptive statistics; Day 3 examines distributions; Day 4 introduces probability; and Day 5 covers sampling, bias, and sample size.

Week 2: Core Methods (Days 6-12)

Day 6 introduces confidence intervals. Day 7 covers hypothesis tests and p-values; Day 8 covers t-tests; Day 9 handles non-parametric alternatives; Day 10 covers ANOVA; Day 11 tackles categorical tests; and Day 12 addresses multiple-testing correction.

Week 3: Modeling (Days 13-20)

You move from association to modeling. Day 13 covers correlation; Day 14 introduces linear regression; Day 15 extends it to multiple predictors; Day 16 covers logistic regression; Day 17 covers survival analysis; Day 18 covers experimental design and power; Day 19 focuses on effect sizes; and Day 20 handles batch effects and confounding.

Week 4: Advanced Topics and Capstones (Days 21-27)

Day 21 covers PCA and dimensionality reduction. Day 22 covers clustering. Day 23 introduces bootstrap and permutation methods; Day 24 introduces Bayesian thinking; Day 25 develops statistical visualization; Day 26 covers meta-analysis; and Day 27 turns the methods into a reproducible workflow.

Capstone Projects (Days 28-30)

Three full projects integrate the book. Day 28 analyzes a clinical trial with survival and subgroup outcomes. Day 29 conducts a differential-expression study. Day 30 analyzes a genome-wide association study with Manhattan and Q-Q plots and genomic-inflation checks.

Conventions Used in This Book

Throughout this book, you will see several recurring elements:

Code Blocks

BioLang code appears in fenced code blocks:

```
let data = [2.3, 4.1, 3.7, 5.2, 4.8]
mean(data)      # 4.02
stdev(data)     # 1.082
```

When a code block shows REPL interaction, lines starting with `bl>` are what you type:

```
bl> ttest([23.1, 25.4, 22.8], [19.2, 20.1, 18.7])
TTestResult { t: 4.12, df: 4, p: 0.0146 }
```

Shell commands use `bash` syntax:

```
bl run day07_ttest.bl
```

Python and R Comparisons

Multi-language comparisons appear with labeled blocks:

BioLang:

```
let data = read_csv("data/expression.csv")
let control = data |> filter(|row| row.condition == "control") |>
map(|row| row.sample1)
let treated = data |> filter(|row| row.condition == "treatment") |>
map(|row| row.sample1)
println(ttest(control, treated))
```

Python:

```
import pandas as pd
from scipy.stats import ttest_ind
data = pd.read_csv("expression.csv")
stat, p = ttest_ind(data["ctrl"], data["treated"])
print(f"t={stat:.4f}, p={p:.4f}")
```

R:

```
data <- read.csv("expression.csv")
t.test(data$ctrl, data$treated)
```

Callout Boxes

Important notes, insights, and warnings appear as blockquotes throughout:

Key insight: A statistically significant result is not necessarily biologically meaningful. Always report effect sizes alongside p-values.

Clinical relevance: In oncology trials, a hazard ratio below 0.7 is typically considered clinically meaningful, regardless of the p-value.

Common pitfall: Running 20 t-tests on the same dataset without multiple testing correction gives you a 64% chance of at least one false positive at $\alpha = 0.05$.

Exercises

Each day ends with exercises labeled by difficulty:

- **Foundations** — reinforce the core concept with guided problems
- **Applied** — use the method on a new biological dataset
- **Challenge** — extend the method or combine it with previous days

Key Takeaways

Each day concludes with a bulleted list of the most important points:

- **The p-value is not the probability that your hypothesis is wrong.** It is the probability of observing data this extreme if the null hypothesis were true. This distinction matters enormously.

A Note on the Multi-Language Approach

This book uses BioLang as its primary language because its statistical builtins let you focus on the concepts rather than the plumbing. A t-test is one function call, not a chain of imports and data manipulations. A volcano plot is one line, not thirty.

But the real world uses Python and R for most biostatistics. We include comparisons for two reasons:

1. **Translation.** If you already know scipy or R's stats package, seeing the BioLang equivalent helps you learn faster. If you learn BioLang first, seeing the Python and R equivalents prepares you for collaborative work.
2. **Verification.** Running the same analysis in three languages and getting the same answer builds confidence. When your BioLang t-test gives $p = 0.014$ and your R t-test gives $p = 0.014$, you know you have done it right.

The `compare.md` file in each day's companion directory provides a detailed side-by-side comparison. The `analysis.py` and `analysis.R` scripts are runnable equivalents you can execute and compare.

Let's Begin

You have everything you need. The next 30 days will transform how you think about biological data — not just how to analyze it, but how to reason about uncertainty, variability, and evidence.

Day 1 starts with the most fundamental question in statistics: what does your data look like?

Turn the page. Your journey starts now.

Day 1: Why Statistics? The Story Your Data Is Trying to Tell

Day 1 of 30

No Prerequisites

~45 min reading

Motivation & Context

The Problem

In 2006, a pharmaceutical company invested over \$800 million developing a promising cancer therapy. Phase II clinical trials had shown a statistically significant survival benefit in 87 patients with advanced colorectal cancer. The data looked compelling: a 38% improvement in median progression-free survival. Investors were jubilant. The company fast-tracked Phase III.

Then came the reckoning. The Phase III trial enrolled 1,200 patients across 120 medical centers. The drug performed no better than placebo. The stock price collapsed overnight. Nearly a billion dollars and eight years of research, gone — not because the science was wrong, but because the statistics were misunderstood. The Phase II “signal” was noise, amplified by a sample too small to tell the difference.

This story is not unusual. It plays out every year in laboratories, hospitals, and boardrooms around the world. Similar scenarios have unfolded with Alzheimer’s drugs, cardiovascular therapies, and anti-inflammatory agents. The difference between a breakthrough and a blunder often comes down to a few fundamental statistical concepts — concepts that any biologist, clinician, or bioinformatician can learn.

That is what this book is about. In 30 days, you will build the statistical intuition and practical skills to avoid these mistakes — whether you are designing a clinical trial, analyzing RNA-seq data, or evaluating a paper over morning coffee.

What Is Statistics?

Statistics is the science of learning from data in the presence of uncertainty. Think of it as a translator between the messy, noisy world of observations and the clean, confident conclusions you want to draw.

Imagine you are standing in a dark room, trying to understand the shape of an object by touching it with gloves on. You can feel something — ridges, curves, a rough texture — but every touch is imprecise. You might mistake a bump for an edge, or miss a hole entirely. Statistics gives you a flashlight. Not a perfect one — the beam flickers and the lens is smudged — but it is incomparably better than groping in the dark.

In biology, the “dark room” is enormous. A single human genome contains 3.2 billion base pairs. A transcriptomics experiment measures expression levels for 20,000 genes simultaneously. A clinical trial tracks hundreds of variables across thousands of patients over years. No human can intuit patterns in data this vast. Statistics provides the framework to ask precise questions and get defensible answers.

The Reproducibility Crisis

In 2012, a team at Amgen, one of the world’s largest biotechnology companies, attempted to reproduce 53 landmark studies in cancer biology — papers published in top-tier journals by respected labs. These were not obscure findings; they were studies that had shaped drug development programs and clinical practice.

They could reproduce only 6. That is an 89% failure rate.

Around the same time, Bayer HealthCare reported a similar effort. Of 67 preclinical studies they attempted to validate, roughly two-thirds could not be reproduced. The results that had guided millions of dollars in investment simply vanished when subjected to rigorous replication.

How does published, peer-reviewed science fail at this rate? The reasons are many, but they share a common root: **insufficient statistical reasoning**.

P-Hacking: Torturing Data Until It Confesses

One of the most insidious contributors is “p-hacking” — the practice of trying multiple analyses until one produces a statistically significant result. A researcher might:

- Test 15 different subgroups and report only the one with $p < 0.05$
- Remove outliers selectively until the result becomes significant
- Try multiple statistical tests and report whichever gives the smallest p-value
- Add or remove covariates until the “right” answer appears
- Decide when to stop collecting data based on whether the current result is significant

None of these practices involves fabricating data. Each individual decision might even seem reasonable in isolation. But collectively, they dramatically inflate the false positive rate. If you flip through enough combinations, you will find “significance” by pure chance — it is a mathematical certainty.

Underpowered Studies

Many published studies use sample sizes far too small to reliably detect the effects they claim to find. A study with only 12 mice per group has roughly a 20% chance of detecting a moderate treatment effect. That means 80% of real effects go undetected. But the 20% that are detected appear larger than they truly are (because only the noisiest, most extreme results cross the significance threshold), creating a distorted picture of biology.

The Garden of Forking Paths

Researchers make dozens of analytical decisions: how to clean the data, which variables to include, how to handle missing values, which test to use, whether

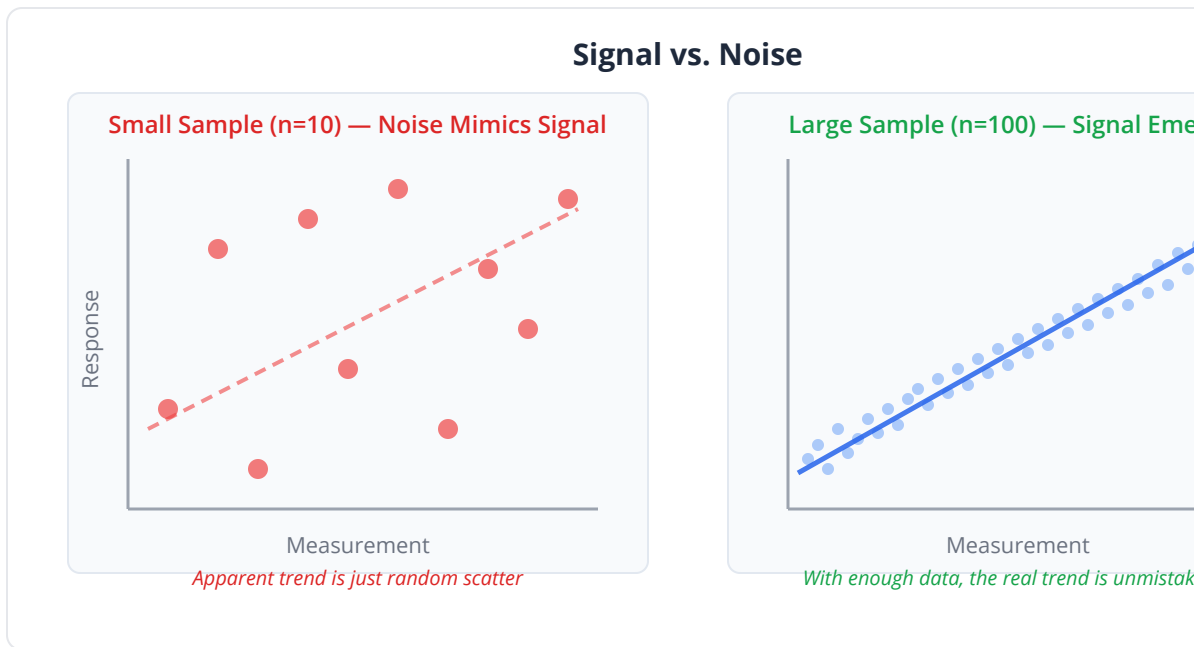
to transform the data, how to define the outcome. Each decision is a fork in the path, and different choices lead to different results. When these choices are made after seeing the data (rather than pre-specified in an analysis plan), the researcher unconsciously navigates toward significance.

This is not an indictment of individual scientists. Most researchers receive minimal formal training in statistics. A typical biology PhD might include one semester-long course, crammed between lab rotations and qualifying exams. The result is a generation of brilliant experimentalists who treat statistical tests as black boxes — input data, output significance. Reviewers, equally uncertain about statistics, wave the paper through. The system rewards novelty over rigor.

Key insight: The reproducibility crisis is not primarily a crisis of fraud or incompetence. It is a crisis of statistical literacy. Understanding the concepts in this book is one of the most impactful things you can do for the quality of your science.

Signal vs. Noise

Here is the most fundamental question in statistics: **Is the pattern I see real, or could it have happened by chance?**



Consider a simple experiment. You flip a coin 10 times and get 8 heads. Is the coin biased? Your intuition says maybe — 8 out of 10 is a lot of heads. But if you do the math, a fair coin produces 8 or more heads about 5.5% of the time. That is unlikely, but not astronomically so. You might just be unlucky.

Now flip the coin 100 times and get 80 heads. Is the coin biased? Almost certainly yes. A fair coin producing 80 or more heads in 100 flips has a probability of about 0.000000000000006. You would need to flip coins continuously for the age of the universe to expect this by chance.

The pattern (80% heads) is the same in both cases. What changed is the **sample size**. With 10 flips, 80% heads is plausible noise. With 100 flips, 80% heads is an unmistakable signal.

This is exactly what happened with the cancer drug. In 87 patients, a 38% improvement could easily arise from random variation — which patients happened to be enrolled, how they responded to the placebo, what comorbidities they had. In 1,200 patients, the noise averages out, and the true effect (or lack thereof) becomes visible.

Common pitfall: Small studies frequently produce dramatic-looking results. This is not because small studies discover larger effects — it is

because small samples are inherently noisy, and noise occasionally looks like a big signal. This phenomenon is called the “winner’s curse” and it haunts biomedical research.

The Cost of Being Wrong

In statistics, there are exactly two ways to be wrong, and they have very different consequences.

Type I Error: The False Alarm

A Type I error occurs when you conclude there is an effect when there is none. You declare the coin biased when it is actually fair. You approve a drug that does not work.

The most devastating Type I error in pharmaceutical history may be thalidomide in the 1950s. Marketed as a safe sedative for pregnant women, the drug was approved based on inadequate evidence. It caused severe birth defects in over 10,000 children worldwide. While this tragedy involved failures far beyond statistics — regulatory, ethical, and scientific — the core issue was concluding safety from data that could not support that conclusion.

A more modern example: in 2004, Merck withdrew Vioxx (rofecoxib), a blockbuster anti-inflammatory drug, after it became clear that it significantly increased heart attack risk. The drug had been on the market for five years. Post-withdrawal analysis suggested that the cardiovascular risk had been detectable in the original trial data, but was either missed or downplayed. The cost: an estimated 88,000-140,000 excess cases of heart disease in the United States alone.

Type II Error: The Missed Discovery

A Type II error occurs when you fail to detect a real effect. You declare the coin fair when it is actually biased. You reject a drug that actually works.

The canonical example is *Helicobacter pylori*. In 1982, Barry Marshall and Robin Warren proposed that stomach ulcers were caused by a bacterium, not by stress or spicy food as the medical establishment believed. Their initial data was compelling but their sample sizes were small. The medical community dismissed their findings for over a decade, costing millions of patients effective treatment. Marshall eventually infected himself with *H. pylori*, developed gastritis, and cured it with antibiotics to prove his point. He and Warren won the Nobel Prize in 2005.

Every year, real treatments are abandoned because clinical trials were too small to detect their effect. Every year, genuine biological mechanisms are dismissed because the experiment lacked statistical power. Type II errors are the silent killers of science — you never know what you missed, because the missed discovery never makes it into a journal.

How many effective cancer therapies have been shelved because the Phase II trial enrolled 40 patients instead of 400? We will never know. But the statistical tools to prevent this — power analysis and sample size calculation — are straightforward. You will learn them on Day 26.

Error Type	What Happens	Consequence	Biology Example
Type I (False Positive)	Conclude effect exists when it does not	Wasted resources, patient harm	Approving ineffective drug
Type II (False Negative)	Miss a real effect	Lost discoveries, delayed treatments	Rejecting H. pylori hypothesis
Correct rejection	Correctly conclude no effect	Good science	Debunking a false supplement claim
Correct detection	Correctly detect real effect	Discovery!	Identifying BRCA1 as cancer gene

Decision Outcomes: The 2x2 Reality		
	Reality: No Effect (H0 true)	Reality: Effect Exists
Decision: "Significant"	TYPE I ERROR False Positive (alpha) Approve drug that doesn't work Rate controlled at 5%	CORRECT True Positive (Power) Detect real treatment Goal: 80%+
Decision: "Not Significant"	CORRECT True Negative Correctly reject bad drug Rate = 1 - alpha = 95%	TYPE II ERROR False Negative (beta) Miss a real treatment Rate = 1 - Power

Clinical relevance: In diagnostic testing, Type I errors produce false positives (telling a healthy person they have cancer) and Type II errors produce false negatives (telling a cancer patient they are healthy). Both

are harmful, but in different ways. The balance between them is one of the central tensions in medicine.

Why Biology Needs Statistics More Than Most Fields

A physicist measuring the speed of light will get the same answer (within measurement precision) whether the experiment is run in Tokyo or Toronto, on Monday or Friday, in summer or winter. The speed of light does not have a bad day.

Biology is fundamentally different, for three reasons.

1. Biological Variability

Every organism is unique. Two genetically identical mice raised in the same cage, fed the same diet, will still differ in gene expression, tumor growth rate, immune response, and lifespan. This is not experimental error — it is the intrinsic variability of living systems. Evolution has built variability into every level of biology, from stochastic gene expression to somatic mutation to behavioral differences.

This variability means that a single measurement tells you almost nothing. If one mouse responds to a drug, you cannot conclude the drug works. If one patient's tumor shrinks, you cannot attribute it to the treatment. You need replicates, and you need statistics to make sense of them.

2. Measurement Noise

Biological measurement is imprecise. A sequencing run introduces base-calling errors at a rate of roughly 0.1-1% per base. RNA-seq quantification depends on library preparation, read depth, alignment parameters, and normalization

method. Mass spectrometry measurements fluctuate with instrument calibration, sample preparation, and ionization efficiency.

Every measurement in biology is the true signal plus some unknown amount of noise. Statistics provides the tools to separate one from the other.

3. Massive Parallel Testing

Modern biology is high-dimensional. A genome-wide association study (GWAS) tests millions of genetic variants. A differential expression analysis tests 20,000 genes. A proteomics experiment quantifies thousands of proteins. A drug screen tests hundreds of compounds.

When you test 20,000 hypotheses simultaneously, you expect 1,000 false positives by chance alone (at the conventional 5% threshold). Without proper statistical correction for multiple testing, you would drown in spurious results. This is not a theoretical concern — it is the daily reality of genomics, and getting it wrong has real consequences.

To make this concrete, consider a differential expression analysis. You measure expression of 20,000 genes in treatment versus control. Even if the treatment does absolutely nothing — affects zero genes — testing each gene at $\alpha = 0.05$ will flag approximately 1,000 genes as “significant.” If you published a paper claiming these 1,000 genes are treatment-responsive, every single one would be a false positive.

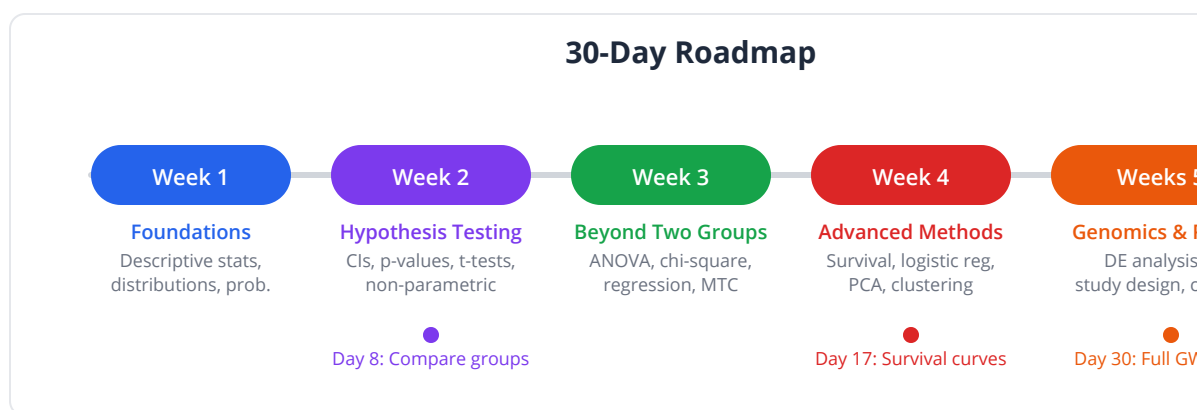
The solution (multiple testing correction, which we cover on Day 15) reduces the significance threshold to account for the number of tests. In a GWAS with 1 million variants, the genome-wide significance threshold is $p < 5 \times 10^{-8}$ — one thousand times more stringent than the usual 0.05. Understanding why this correction is necessary, and how to apply it properly, is one of the core skills of a computational biologist.

Key insight: Biology sits at the intersection of high variability, high noise, and high dimensionality. This makes it arguably the field most in need of

statistical sophistication, yet it has historically been one of the least statistically trained.

A Tour of What Lies Ahead

This book will take you from zero to practicing biostatistician in 30 days. Here is a preview of the journey:



Week 1 (Days 1-5): Foundations. You will learn to summarize data, understand distributions, reason about probability, and appreciate why sample size matters. These are the tools you need before you can test any hypothesis.

Week 2 (Days 6-10): Hypothesis Testing. You will learn confidence intervals, p-values, t-tests, and non-parametric alternatives. By Day 8, you will be able to rigorously determine whether two groups differ. By Day 10, you will know when to use (and when to avoid) parametric tests.

Week 3 (Days 11-15): Beyond Two Groups. ANOVA, chi-square tests, correlation, regression, and multiple testing correction. You will analyze multi-group experiments, categorical data, and learn why “correlation does not imply causation” is more nuanced than it sounds.

Week 4 (Days 16-20): Advanced Methods. Survival analysis, logistic regression, principal component analysis, and clustering. You will build Kaplan-

Meier curves, classify patients, reduce high-dimensional data, and find natural groupings in gene expression datasets.

Week 5 (Days 21-25): Genomics Applications. Differential expression analysis, enrichment testing, multiple testing correction in practice, and Bayesian thinking. The methods that power modern computational biology.

Week 6 (Days 26-30): Real-World Practice. Power analysis and study design, meta-analysis, machine learning basics, reproducible research practices, and a capstone project that ties everything together.

By Day 8, you will know if two groups truly differ. By Day 17, you will build survival curves that predict patient outcomes. By Day 22, you will analyze differential gene expression. By Day 30, you will design a complete statistical analysis plan for a GWAS.

Each day follows the same pattern: a real-world problem that motivates the method, the conceptual framework, hands-on BioLang code, comparisons with Python and R, and exercises to cement your understanding. The emphasis throughout is on **understanding** — not memorizing formulas, but developing the intuition to know which method to use and why.

The Statistician's Mindset

Before we dive into formulas and code, internalize these four questions. Ask them every time you look at data, read a paper, or plan an experiment:

1. How variable is it?

A mean without a measure of spread is almost meaningless. “Average tumor size decreased by 2 cm” sounds impressive until you learn that the standard deviation was 4 cm. Always ask: what is the spread?

2. Could chance explain this?

The human brain is wired to see patterns, even in random noise. We see faces in clouds, constellations in random stars, and trends in stock prices. Before accepting any pattern as real, quantify the probability that it arose by chance. This is the essence of hypothesis testing.

3. How big is the effect?

Statistical significance and practical significance are not the same thing. With a large enough sample, you can detect a difference of 0.001 grams in tumor weight with $p < 0.001$. But is a one-milligram difference clinically meaningful? Always report effect sizes alongside p-values.

4. Is my sample representative?

If you study the genetics of heart disease using only patients from a single hospital in Boston, your results may not generalize to patients in rural India. If you select only the “best” cell lines for your experiment, your conclusions may not extend to primary cells. Sampling bias is the silent assassin of biomedical research.

Putting the Mindset into Practice

These four questions are not abstract philosophy. They are a practical checklist:

Question	When Reading a Paper	When Designing an Experiment
How variable?	Check SD, IQR, range	Plan enough replicates
Could chance explain it?	Scrutinize p-values, CI	Pre-register analysis plan
How big is the effect?	Look for effect sizes, not just significance	Define minimum meaningful difference
Representative sample?	Check inclusion criteria, demographics	Match your sample to target population

You will encounter these questions again and again throughout this book. By Day 30, they will be second nature — the automatic mental checklist of a statistically literate scientist.

Key insight: Statistics is not a set of tests to run after the experiment. It is a way of thinking that should inform every stage — from study design to data collection to analysis to interpretation. The best time to consult a statistician is before you collect a single data point.

The Burden of Proof

In everyday life, we make decisions based on intuition, anecdote, and authority. “My grandmother smoked until 95, so smoking cannot be that bad.” “This supplement worked for my friend, so it must be effective.” “The famous professor says this treatment works, so it must.”

Science demands a higher standard. The burden of proof rests on the claimant. If you claim a drug works, you must demonstrate it with evidence strong enough to withstand scrutiny. If you claim a gene is associated with a disease, you must show that the association is unlikely to be a coincidence.

Statistics provides the machinery for this burden of proof. It forces you to be explicit about your assumptions, quantify your uncertainty, and acknowledge the limits of your data. It is, in essence, formalized humility.

Consider the claim “Vitamin D supplements reduce cancer risk.” An anecdote is worthless: your uncle took vitamin D and did not get cancer. A small observational study is weak: 50 people who took vitamin D had fewer cancers than 50 who did not — but maybe the vitamin D group was healthier to begin with (confounding). A large randomized controlled trial with 25,000 participants, pre-registered outcomes, and proper statistical analysis is strong evidence. Each step up the ladder requires more statistical sophistication.

The hierarchy of evidence is, fundamentally, a hierarchy of statistical rigor:

Evidence Level	Design	Statistical Rigor
Weakest	Case report / anecdote	None
Weak	Case series	Descriptive only
Moderate	Observational study	Potential confounding
Strong	Randomized controlled trial	Causal inference possible
Strongest	Meta-analysis of multiple RCTs	Pooled estimates, high power

This book will equip you to evaluate and produce evidence at every level of this hierarchy.

The Numbers Tell a Story

To bring this all together, let us look at a real-world scenario that illustrates every concept from today.

A research group publishes a paper claiming that a new biomarker predicts response to immunotherapy. Their study: 24 patients (12 responders, 12 non-

responders). They measure the biomarker level in each patient and find a “statistically significant” difference ($p = 0.03$).

Here is what a statistical thinker would ask:

How variable is it? The biomarker levels range from 2 to 200 ng/mL. The standard deviation within each group is enormous — nearly as large as the difference between groups. The signal is weak relative to the noise.

Could chance explain it? With only 12 per group and high variability, the p -value of 0.03 is fragile. If you removed two extreme patients, it becomes 0.12. The result is not robust.

How big is the effect? The difference in medians is 15 ng/mL, but the overlap between groups is substantial. Many responders have lower biomarker levels than many non-responders. The effect size (Cohen’s d) is only 0.4 — a “small to medium” effect.

Is the sample representative? All patients came from a single institution, were predominantly male, and had a specific tumor subtype. Whether the biomarker works in a broader population is unknown.

A naive reader sees “ $p < 0.05$, significant.” A statistically literate reader sees a fragile, underpowered result from a non-representative sample with a modest effect size. These are different conclusions from the same data.

A Preview of the Tools

Throughout this book, you will use BioLang to perform statistical analyses. Here is a tiny glimpse of what Day 2 will look like — just to whet your appetite:

```
# Tomorrow, you'll summarize 10,000 quality scores in one line:
# let stats = summary(quality_scores)
#
# And visualize them instantly:
# histogram(quality_scores, {bins: 50, title: "Sequencing Quality
# Distribution"})
```

But today is about the **why**, not the **how**. The tools are only as good as the thinking behind them. A researcher who understands why a t-test exists will use it correctly even with imperfect software. A researcher who merely knows how to call a t-test function will misuse it regularly, regardless of how elegant the software is.

Exercises

Exercise 1: The Newspaper Test

Find a news article reporting a scientific or medical finding (e.g., “Coffee reduces cancer risk by 15%”). Write down your answers to these four questions:

- (a) What was the sample size? If the article does not mention it, what does that tell you?
- (b) Could the result be due to chance? What would you need to know to answer this?
- (c) Is the effect size meaningful in practice? A 2% reduction in cancer risk sounds different from a 50% reduction.
- (d) Is the sample representative of the population you care about? Who was studied, and who was not?

If the article does not provide enough information to answer these questions, that itself is informative. Most science journalism omits sample sizes, effect sizes, and confidence intervals — precisely the information you need to evaluate the claim.

Exercise 2: Coin Flip Thought Experiment

Without doing any math, estimate the following:

- If you flip a fair coin 20 times, what is the probability of getting exactly 10 heads?
- What about 15 or more heads out of 20?

- What about 20 heads in a row?

Write down your guesses. We will revisit this on Day 4 with the tools to compute exact answers, and you can see how well your intuition calibrated.

Exercise 3: Reproducibility Reflection

Think about a result from your own work (or a paper you have read) that you found surprising or striking. List three reasons why the result might fail to reproduce if someone repeated the experiment. For each reason, identify whether it is:

- Statistical (sample size, random variation, multiple testing)
- Methodological (different protocols, reagent lots, equipment)
- Biological (different cell lines, patient populations, environmental conditions)

Exercise 4: Type I vs Type II in Your Field

Identify one example each of a Type I error (false positive) and a Type II error (false negative) that would be particularly damaging in your area of biology. For each:

- Describe the scenario concretely
- Estimate the consequences (financial, clinical, scientific)
- State which error type you consider more dangerous in your context, and why

Exercise 5: Spotting P-Hacking

A paper reports testing a drug on patients across 8 different cancer subtypes. Only one subtype shows a significant result ($p = 0.04$). The paper's title highlights this positive finding. What statistical concerns should this raise? How many false positives would you expect by chance when testing 8 subtypes at $\alpha = 0.05$?

Key Takeaways

- Statistics is the science of learning from data in the presence of uncertainty — it is essential, not optional, for biological research.
- The reproducibility crisis is largely a statistical literacy crisis: most landmark findings fail replication due to underpowered studies, p-hacking, and misunderstood tests.
- Signal vs. noise is the fundamental statistical question: the same percentage difference can be meaningful or meaningless depending on sample size.
- Type I errors (false positives) waste resources and can cause harm; Type II errors (false negatives) cause missed discoveries. Neither can be eliminated — only managed.
- Biology is uniquely challenging for statistics due to inherent biological variability, measurement noise, and massive parallel testing.
- The statistician's mindset asks four questions: How variable? Could chance explain it? How big is the effect? Is the sample representative?
- Statistics should inform every stage of research, from design through interpretation — not just the analysis phase.

What's Next

Tomorrow, we roll up our sleeves and meet real data. You will learn to summarize 10,000 numbers into a handful of meaningful statistics — means, medians, standard deviations, and more. You will discover why the mean is a liar when outliers are present, why box plots reveal truths that histograms hide, and how a single command in BioLang can tell you whether a sequencing run is worth analyzing or should be thrown away. Day 2 is where the hands-on work begins.

Day 2: Your Data at a Glance — Descriptive Statistics

Day 2 of 30

Prerequisites: Day 1

~50 min

Hands-on

The Problem

Dr. Sarah Chen stares at her screen. The Illumina NovaSeq 6000 finished its run overnight, and now she has 10,247 quality scores — one for each tile on the flow cell. Her PI needs a decision by the morning meeting: is this data usable, or do they need to re-run the library, burning another \$4,000 and two days?

She cannot read 10,247 numbers. She cannot scroll through them and develop an intuition. She has five minutes before the meeting starts. What she needs is a way to compress 10,247 numbers into a handful of meaningful summaries that answer three questions: What is the typical quality? How much does it vary? Are there any red flags?

This is the job of descriptive statistics. They are the first thing you compute, every time, before you run a single test. Get them wrong — or skip them — and everything downstream is built on sand.

What Are Descriptive Statistics?

Descriptive statistics are summaries. Think of them as a movie trailer for your data. The full movie (the raw dataset) might be two hours long, but the trailer gives you the genre, the tone, and the key plot points in two minutes. A good trailer does not lie about the movie. A good set of descriptive statistics does not lie about the data.

There are three things you need to know about any dataset:

1. **Center** — Where is the “middle” of the data?
2. **Spread** — How far do values range from that center?
3. **Shape** — Is the data symmetric? Skewed? Are there outliers?

Why Do These Three Matter?

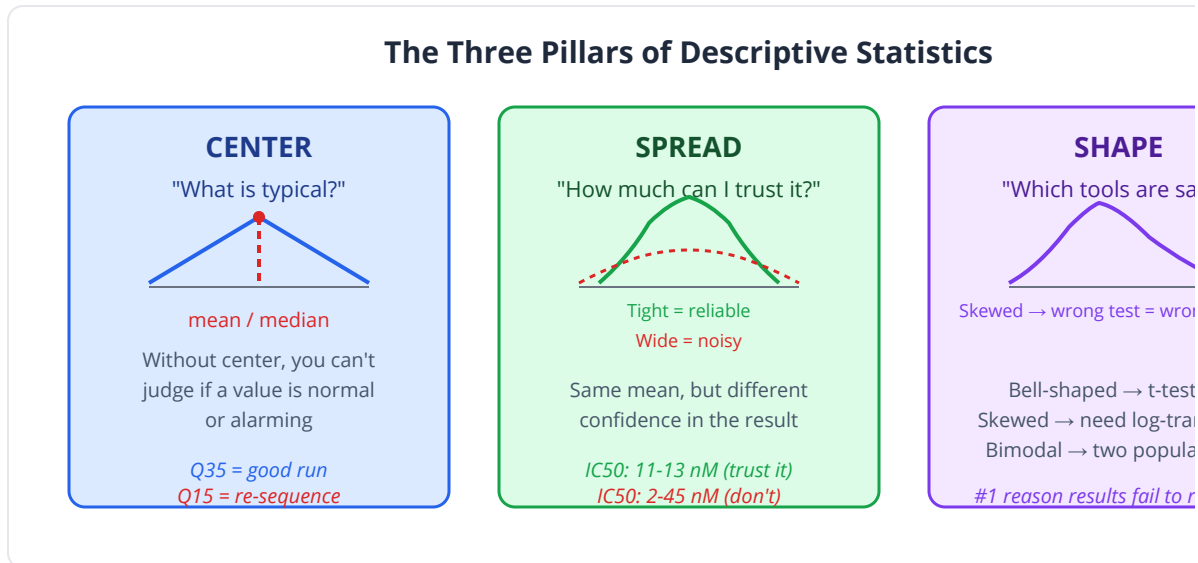
Center tells you what is “normal.” If the mean quality score of your sequencing run is Q35, you know tiles are performing well. If it is Q15, something went wrong. Without center, you have no reference point — you cannot say whether a single observation is typical or alarming. In clinical genomics, center defines the baseline: what is the typical variant allele frequency in this cohort? What is the average coverage across your target regions?

Spread tells you how much you can trust the center. Two experiments might both report a mean IC50 of 12 nM, but one has values ranging from 11 to 13 (tight, reproducible) while the other ranges from 2 to 45 (noisy, unreliable). The center is the same — the spread tells you completely different stories. High spread means your next measurement could land anywhere; low spread means you can make confident predictions. In RNA-seq, high within-group variance buries real differential expression in noise.

Shape tells you which statistical tools are safe to use. Almost every standard statistical test (t-test, ANOVA, linear regression) assumes the data is approximately bell-shaped (normally distributed). If your data is heavily right-skewed — which gene expression nearly always is — those tests give wrong answers. Shape also reveals hidden structure: a bimodal distribution might mean you have two distinct populations mixed together (e.g., responders and non-responders to a drug). Ignoring shape is the single most common reason published biomedical results fail to replicate.

Key insight: Center without spread is meaningless (“the average temperature is 72°F” tells you nothing if the range is -20 to 160). Spread

without shape is dangerous (a standard deviation assumes symmetry, but skewed data violates this). You always need all three.



Every statistical analysis starts here. If you skip descriptive statistics and jump straight to hypothesis testing, you are performing surgery without an examination.

Measures of Center

Mean (Arithmetic Average)

The mean is the balance point. If your data values were weights placed along a ruler, the mean is the spot where the ruler would balance perfectly.

Formula: $\bar{x} = (1/n) \sum x_i$

The mean uses every data point, which is both its strength and its weakness. It is the most efficient estimator of center when data is symmetric with no outliers. But it is exquisitely sensitive to extreme values.

Example: Five gene expression values (FPKM): 12, 15, 14, 13, 16. Mean = 14.0. Reasonable.

Now add one highly expressed gene: 12, 15, 14, 13, 16, 5000. Mean = 845.0. The mean has been dragged from 14 to 845 by a single outlier. It no longer represents “typical” expression.

Common pitfall: In genomics, gene expression distributions are heavily right-skewed. Reporting mean FPKM/TPM values without acknowledging this skew is misleading. The median is almost always a better summary for expression data.

Median (Middle Value)

The median is the value that splits the data in half: 50% of observations fall below it, 50% above. Sort the data and pick the middle number (or average the two middle numbers if n is even).

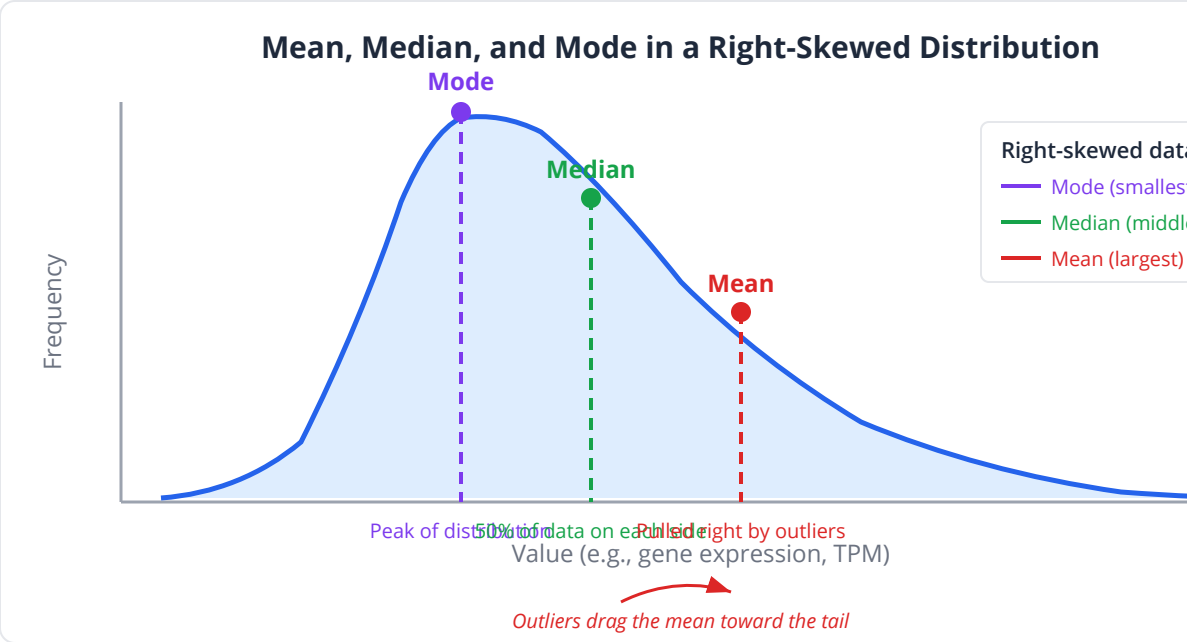
For our outlier-contaminated expression data: sorted = 12, 13, 14, 15, 16, 5000. Median = $(14 + 15) / 2 = 14.5$. The outlier barely matters.

The median is **robust** — it resists the pull of extreme values. This makes it the preferred measure of center for skewed distributions, which are the norm in biology.

Mode (Most Frequent Value)

The mode is the most common value. It is most useful for categorical or discrete data: the most common blood type, the most frequent variant allele, the peak of a histogram.

For continuous data, the mode is the peak of the density curve. Bimodal distributions (two peaks) arise in biology more often than you might expect — for instance, CpG methylation levels often cluster near 0% and 100%, reflecting fully unmethylated and fully methylated states.



When to Use Each

Measure	Best For	Sensitive to Outliers?	Biological Example
Mean	Symmetric, well-behaved data	Yes, very	Measurement error in technical replicates
Median	Skewed data, outliers present	No	Gene expression (FPKM/TPM)
Mode	Categorical data, multimodal	No	Variant allele frequency peaks

Key insight: Always report both mean and median. If they differ substantially, your data is skewed, and the median is the more honest summary.

Measures of Spread

Knowing the center is not enough. Two datasets can have identical means and wildly different behaviors. Consider drug response in two patient cohorts — both might have a mean survival of 12 months, but in one cohort everyone lives 11-13 months, while in the other, half die in 2 months and half live 22 months. The clinical implications are completely different.

Range

The simplest measure: maximum minus minimum. It tells you the total extent of the data but is completely determined by the two most extreme points.

Variance and Standard Deviation

Variance measures the average squared distance from the mean:

Variance: $s^2 = (1 / (n-1)) \sum (x_i - \bar{x})^2$

Standard Deviation: $s = \sqrt{s^2}$

We divide by (n-1) rather than n (Bessel's correction) because a sample underestimates the true population variance. The standard deviation is in the same units as the data, making it more interpretable than variance.

Rule of thumb: For normally distributed data, about 68% of values fall within 1 SD of the mean, 95% within 2 SDs, and 99.7% within 3 SDs. If a quality score is more than 3 SDs below the mean, something is wrong with that tile.

Interquartile Range (IQR)

The IQR is the range of the middle 50% of the data: $Q3 - Q1$, where $Q1$ is the 25th percentile and $Q3$ is the 75th percentile.

Like the median, the IQR is robust to outliers. It is the foundation of the box plot and a standard measure of spread for skewed data.

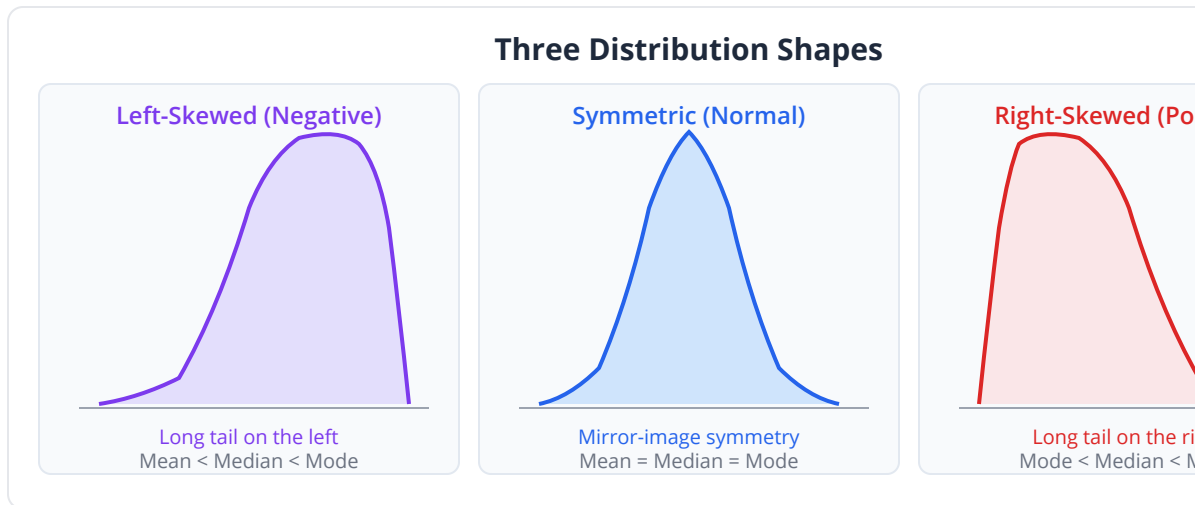
Coefficient of Variation (CV)

$CV = (SD / \text{Mean}) \times 100\%$. The CV expresses variability relative to the mean, making it useful for comparing spread across measurements with different scales.

Example: If gene A has mean expression 1000 TPM with SD 100, and gene B has mean 10 TPM with SD 5, which is more variable? Gene A has higher SD, but gene B has a higher CV (50% vs 10%). Gene B's expression is relatively more variable.

Measure	Formula	Robust to Outliers?	Use Case
Range	max - min	No	Quick overview
Variance	s^2	No	Mathematical convenience
Standard Deviation	s	No	Symmetric data
IQR	Q3 - Q1	Yes	Skewed data, box plots
CV	$(SD / \text{Mean}) \times 100\%$	No	Comparing variability across scales

Shape: Skewness and Kurtosis



Skewness

Skewness measures asymmetry. A skewness of 0 means perfectly symmetric (like the normal distribution). Positive skewness means a right tail (most values cluster low, with a few very high values). Negative skewness means a left tail.

Biological reality: Most biological measurements are positively skewed. Gene expression, protein abundance, cell counts, read depths — all tend to have many low values and a few very high ones. This is because multiplicative processes (gene regulation cascades, exponential growth) naturally produce right-skewed distributions.

Kurtosis

Kurtosis measures the “tailedness” of the distribution — how likely extreme values are compared to a normal distribution. High kurtosis means heavy tails (more outliers than expected). Low kurtosis means light tails.

In genomics, variant allele frequency distributions often have high kurtosis, reflecting the mixture of common variants (clustered near 50%) and rare variants (near 0%).

Key insight: If skewness is far from 0 or kurtosis is far from 3, your data is not normally distributed, and parametric tests that assume normality may give misleading results. We will explore this deeply on Day 3.

Descriptive Statistics in BioLang

Let us return to Dr. Chen's problem. She has 10,247 quality scores and five minutes.

Loading and Summarizing Data

```
set_seed(42)
# Simulate sequencing quality scores (Phred scale, typically 0-40)
let quality_scores = rnorm(10247, 32.5, 3.8)

# One-line comprehensive summary
let stats = summary(quality_scores)
print(stats)
# Output:
#   n:      10247
#   mean:    32.48
#   median:  32.51
#   sd:      3.81
#   min:     17.23
#   max:     47.12
#   q1:      29.93
#   q3:      35.07
#   iqr:      5.14
#   skewness: -0.01
#   kurtosis: 2.99
```

That single call answers Dr. Chen's three questions immediately. Mean quality is 32.5 (good — above the Phred 30 threshold). The SD is 3.8 (moderate spread). Skewness is near zero (symmetric). No red flags. The run is usable.

Computing Individual Statistics

```
# Individual measures of center
let avg = mean(quality_scores)      # 32.48
let med = median(quality_scores)    # 32.51
let mod = mode(quality_scores)      # 32 (rounded to nearest
integer)

# Individual measures of spread
let sd = stdev(quality_scores)      # 3.81
let v = variance(quality_scores)    # 14.52
let r = range_stat(quality_scores)  # [17.23, 47.12]
let q = quantile(quality_scores, [0.25, 0.5, 0.75]) # [29.93,
32.51, 35.07]
let i = iqr(quality_scores)          # 5.14

# Shape
let sk = skewness(quality_scores)   # -0.01
let ku = kurtosis(quality_scores)   # 2.99

print("Mean: {avg}, Median: {med}")
print("SD: {sd}, IQR: {i}")
print("Skewness: {sk}, Kurtosis: {ku}")
```

The summary() Function

For a more detailed report:

```
let report = summary(quality_scores)
print(report)
# Output:
#   Variable:    quality_scores
#   Count:       10247
#   Mean:        32.48
#   Std Dev:     3.81
#   Min:         17.23
#   25%:         29.93
#   50%:         32.51
#   75%:         35.07
#   Max:         47.12
#   Skewness:    -0.01
#   Kurtosis:    2.99
#   CV:          11.73%
#   SE(mean):    0.038
```

Working with Real Sequencing Data

```
set_seed(42)
# In practice, you'd load from a file:
# let scores = read_csv("data/expression.csv") |>
  column("phred_score")

# Simulate a problematic run with bimodal quality
let good_tiles = rnorm(8000, 33.0, 2.5)
let bad_tiles = rnorm(2247, 18.0, 4.0)
let mixed_scores = good_tiles + bad_tiles

let mixed_stats = summary(mixed_scores)
print(mixed_stats)
# Mean: 29.7 – looks okay at first glance
# Median: 32.1 – the median reveals the truth: most tiles are fine
# Skewness: -1.4 – strongly left-skewed, warning sign!

# The mean alone would have hidden the problem.
# Always look at the full distribution.
```

Common pitfall: Relying on the mean alone can hide bimodal distributions. The “bad tile” problem above is common in sequencing — a

mean of 29.7 looks passable, but 22% of tiles are failing. Always visualize.

Visualization: Always Plot Before You Test

Numbers tell part of the story. Plots tell the rest. Anscombe's Quartet — four datasets with identical means, variances, and correlations but wildly different structures — demonstrates why you must always look at your data.

Histograms

A histogram bins your data and counts frequencies. It reveals the distribution's shape at a glance.

```
# Basic histogram (uses quality_scores from block above – click "Run All Above + This")
histogram(quality_scores, {bins: 50, title: "Sequencing Quality Distribution"})
```

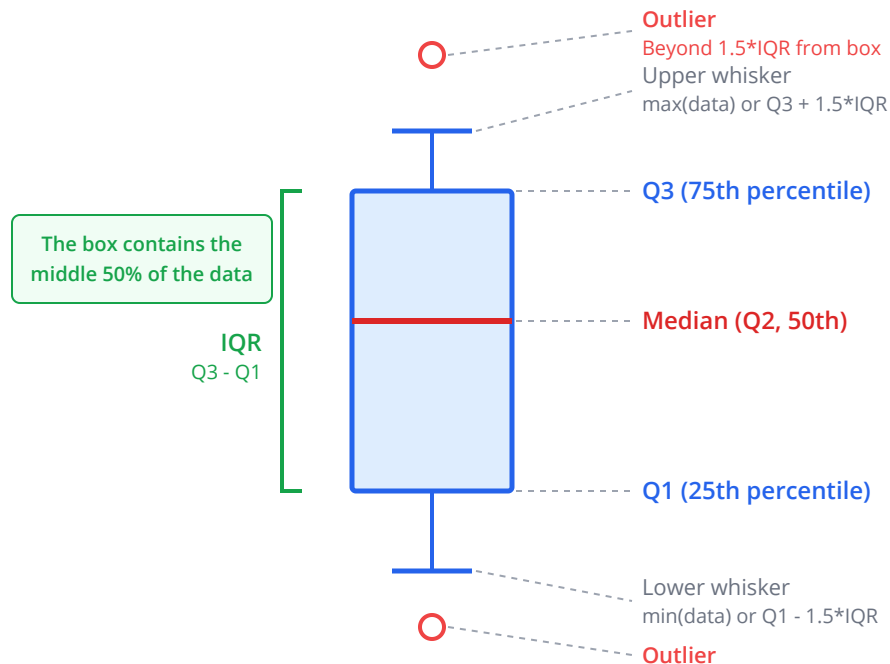
```
# Histogram of the problematic run (uses mixed_scores from earlier block)
histogram(mixed_scores, {bins: 50, title: "Bimodal Quality – Problem Run"})
# This immediately reveals the two peaks that the mean masked.
```

The good run produces a single symmetric peak around 32-33. The problem run shows two distinct peaks — one centered at 18 and one at 33. No summary statistic captures this as effectively as the histogram.

Box Plots

A box plot displays the median (center line), IQR (box), and whiskers (typically $1.5 \times \text{IQR}$). Points beyond the whiskers are marked as individual outliers.

Anatomy of a Box Plot



```
set_seed(42)
# Compare quality across lanes
let lane1 = rnorm(2000, 33.0, 2.0)
let lane2 = rnorm(2000, 31.5, 3.5)
let lane3 = rnorm(2000, 28.0, 5.0)

let bp_table = table({"Lane 1": lane1, "Lane 2": lane2, "Lane 3":
lane3})
boxplot(bp_table, {title: "Quality Score Distribution by Lane"})
# Lane 3 immediately stands out: lower median, wider spread, more
outliers.
```

Box plots shine when comparing groups. You can see at a glance which lane is problematic, without computing a single number.

Combining Plots and Stats

```
set_seed(42)
# Full QC report in a few lines
let scores = rnorm(10000, 32.5, 3.8)

# Side-by-side: histogram + box plot
histogram(scores, {bins: 40, title: "Quality Score Distribution"})
boxplot(scores, {title: "Quality Score Box Plot"})

# Summary table
let stats = summary(scores)
print("QC Decision: {if stats.mean > 30 then 'PASS' else 'FAIL'}")
print("Mean Q-score: {stats.mean:.1}")
print("Tiles below Q20: {scores |> filter(|s| s < 20) |> len()}")
```

Python and R Equivalents

For those coming from Python or R, here are the equivalent operations.

Python:

```
import numpy as np
from scipy import stats

scores = np.random.normal(32.5, 3.8, 10247)

# Measures of center
np.mean(scores)      # 32.48
np.median(scores)    # 32.51
stats.mode(scores)    # most frequent

# Measures of spread
np.std(scores, ddof=1) # 3.81 (ddof=1 for sample SD)
np.var(scores, ddof=1) # 14.52
np.percentile(scores, [25, 50, 75])
stats.iqr(scores)     # 5.14

# Shape
stats.skew(scores)    # -0.01
stats.kurtosis(scores) # -0.01 (scipy uses excess kurtosis)

# Comprehensive summary
import pandas as pd
pd.Series(scores).describe()
```

R:

```
scores <- rnorm(10247, mean = 32.5, sd = 3.8)

# Measures of center
mean(scores)      # 32.48
median(scores)    # 32.51

# Measures of spread
sd(scores)        # 3.81
var(scores)       # 14.52
quantile(scores, c(0.25, 0.5, 0.75))
IQR(scores)       # 5.14

# Shape (requires moments package)
library(moments)
skewness(scores)  # -0.01
kurtosis(scores)  # 2.99

# Comprehensive summary
summary(scores)

# Visualization
hist(scores, breaks = 50, main = "Quality Distribution")
boxplot(scores, main = "Quality Box Plot")
```

Worked Example: The QC Decision

Let us walk through Dr. Chen's complete analysis.

```

set_seed(42)
# Step 1: Load the data
let scores = rnorm(10247, 32.5, 3.8)

# Step 2: Quick summary
let stats = summary(scores)

# Step 3: QC criteria
let pass_threshold = 30.0      # Minimum acceptable mean quality
let fail_fraction_limit = 0.10 # Max 10% tiles below Q20

# Step 4: Compute QC metrics
let tiles_below_q20 = scores |> filter(|s| s < 20) |> len()
let fraction_below_q20 = tiles_below_q20 / len(scores)

# Step 5: Decision
let qc_pass = stats.mean >= pass_threshold and fraction_below_q20 <=
fail_fraction_limit

print("=== Sequencing Run QC Report ===")
print("Total tiles:           {len(scores)}")
print("Mean quality:          {stats.mean:.2}")
print("Median quality:         {stats.median:.2}")
print("Std deviation:          {stats.sd:.2}")
print("Tiles below Q20:         {tiles_below_q20} ({fraction_below_q20
* 100:.1}%)")
print("QC Result:               {if qc_pass then 'PASS' else 'FAIL'}")
print("=====")

# Step 6: Visualize
histogram(scores, {bins: 50, title: "Run QC: Quality Score
Distribution"})

```

This takes about 10 seconds to run. Dr. Chen has her answer well before the meeting.

Exercises

Exercise 1: Gene Expression Summary

Compute descriptive statistics for a set of gene expression values and identify the best measure of center.

```
# Gene expression values (TPM) for 20 genes
let expression = [0.5, 1.2, 3.4, 5.1, 8.7, 12.3, 15.0, 22.4, 45.6,
78.9,
                  120.5, 250.3, 0.1, 2.8, 6.5, 0.3, 1100.0, 33.2,
0.8, 18.5]

# TODO: Compute mean, median, stdev, skewness
# TODO: Which measure of center best represents "typical"
expression? Why?
# TODO: Create a histogram. What shape do you see?
```

Exercise 2: Comparing Sequencing Runs

Two sequencing runs produced quality scores. Determine which run is better.

```
set_seed(42)
let run_a = rnorm(5000, 31.0, 2.5)
let run_b = rnorm(5000, 33.0, 6.0)

# TODO: Compute summary() for both runs
# TODO: Which has higher mean? Which has lower variability?
# TODO: Compute the CV for each. Which is more consistent?
# TODO: Create side-by-side box plots
# TODO: If you could only pick one run, which would you choose and
why?
```

Exercise 3: Outlier Detection

Identify outliers using the IQR method (values below $Q1 - 1.5IQR$ or above $Q3 + 1.5IQR$).

```
# Protein abundance measurements with some suspicious values
let protein = [45.2, 48.1, 50.3, 47.8, 49.5, 46.7, 51.2, 48.9,
              150.0, 47.3, 49.8, 46.1, 50.5, 48.4, 3.0, 49.1]

# TODO: Compute Q1, Q3, IQR
# TODO: Calculate lower and upper fences
# TODO: Identify which values are outliers
# TODO: Compute mean with and without outliers – how much does it
change?
```

Exercise 4: Multi-Sample QC Dashboard

Build a QC summary for multiple samples.

```
set_seed(42)
let samples = {
  "Sample_A": rnorm(1000, 34.0, 2.0),
  "Sample_B": rnorm(1000, 29.0, 4.0),
  "Sample_C": rnorm(1000, 32.0, 3.0),
  "Sample_D": rnorm(1000, 20.0, 5.0)
}

# TODO: Loop through samples, compute summary() for each
# TODO: Flag any sample with mean < 25 or CV > 20%
# TODO: Create a box plot comparing all four samples
```

Key Takeaways

- Descriptive statistics compress large datasets into interpretable summaries of center, spread, and shape.
- The **mean** is efficient but outlier-sensitive; the **median** is robust. For skewed biological data, prefer the median.
- **Standard deviation** measures absolute spread; **IQR** is robust to outliers; **CV** enables comparison across different scales.
- **Skewness** and **kurtosis** reveal whether your data's shape matches the assumptions of common statistical tests.

- **Always visualize** your data with histograms and box plots before computing any test. Summary statistics can hide multimodal distributions, outliers, and other structural features.
- `summary()` in BioLang provides comprehensive descriptive statistics in a single function call.
- Descriptive statistics are not optional preliminaries — they are the foundation of every analysis.

What's Next

You now know how to summarize data, but you may have noticed something: we keep saying “normally distributed” and “skewed” without precisely defining what a distribution is. Tomorrow, on Day 3, we dive into the mathematical shapes that biological data follows — the normal distribution, the log-normal, the Poisson, and the binomial. You will learn why gene expression data refuses to be normal, why read counts follow a Poisson process (sort of), and how to test whether your data fits the distribution you think it does. Understanding distributions is the key to choosing the right statistical test — and avoiding the wrong one.

Day 3: Distributions — The Shape of Biological Variation

Day 3 of 30

Prerequisites: Days 1-2

~55 min

Hands-on

The Problem

Dr. James Park has RNA-seq data from 12 tumor samples. He needs to identify genes that are differentially expressed between treatment and control groups. He knows the standard approach: run a t-test on each gene. The t-test assumes data is normally distributed, so he checks his data.

Gene FPKM values range from 0 to 50,000. Most genes sit near zero, with a handful expressed at astronomical levels. The histogram looks nothing like a bell curve — it is a massive spike at zero with a long right tail stretching to infinity. He runs the t-test anyway. Out of 20,000 genes, 4,700 come back “significant” at $p < 0.05$. That is 23.5% of all genes, far more than seems biologically plausible for this modest treatment.

Something is deeply wrong. The t-test assumed his data was symmetric and bell-shaped. It was neither. The test produced garbage because the assumption was violated. If Dr. Park had understood distributions — the subject of today’s chapter — he would have known to transform his data or use a different test entirely.

What Is a Distribution?

A distribution is a recipe that tells you how likely each possible value is. Think of it as a map of a city, where the height at each point represents how many

people live there. Downtown is a tall spike (many people). The suburbs are a gentle slope. The surrounding farmland is nearly flat.

Every dataset has an underlying distribution — the theoretical shape that generated the data you observe. When you draw a histogram of your data, you are estimating this shape from a finite sample. With 10 data points, the histogram is choppy and unreliable. With 10,000, it smooths out and begins to reveal the true underlying curve.

Why does this matter? Because **every statistical test makes assumptions about the distribution of your data**. The t-test assumes normality. The chi-square test assumes expected counts are large enough. The Poisson regression assumes counts follow a Poisson process. Violate the assumption, and the test's guarantees evaporate.

Key insight: A statistical test is a contract. It says: "If your data follows distribution X, then I guarantee my conclusions are reliable with probability Y." Break the contract, and you get no guarantees.

The Normal Distribution

The normal distribution — the bell curve — is the most famous distribution in statistics, and for good reason. It arises naturally whenever many small, independent effects add together.

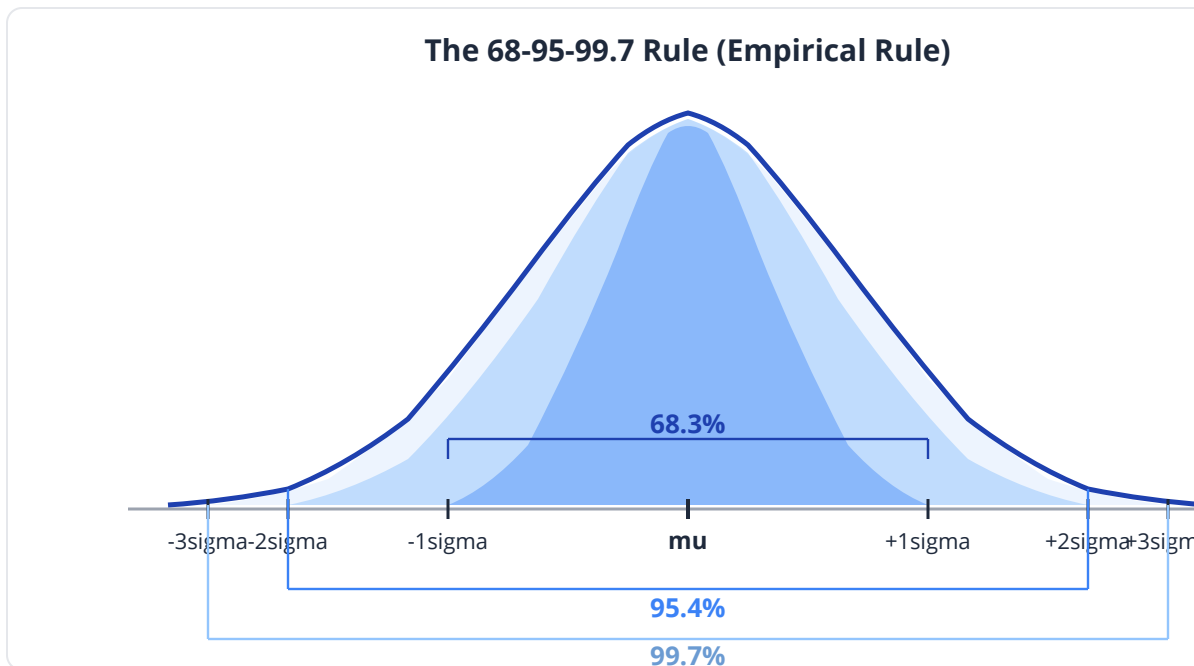
Properties

The normal distribution is defined by two parameters:

- **μ (mu):** the mean, which determines the center
- **σ (sigma):** the standard deviation, which determines the width

The curve is perfectly symmetric around μ . It extends infinitely in both directions, though values far from the mean are exceedingly unlikely.

The 68-95-99.7 Rule



Range	Probability	Meaning
$\mu \pm 1\sigma$	68.3%	About two-thirds of data
$\mu \pm 2\sigma$	95.4%	Nearly all data
$\mu \pm 3\sigma$	99.7%	Almost everything
Beyond 3σ	0.3%	Extreme outliers

If a measurement falls more than 3 standard deviations from the mean, it is either a genuine outlier or something went wrong.

Biological Examples of Normality

The normal distribution is a good model for:

- **Measurement error:** Technical replicates of the same sample tend to be normally distributed.
- **Height and weight** in a homogeneous population (though mixtures of populations are not normal).

- **Blood pressure** readings in healthy adults.
- **Quantitative traits** influenced by many genes of small effect (additive genetic model).
- **Log-transformed gene expression** (more on this below).

When Data Is NOT Normal

The normal distribution is a terrible model for:

- Raw gene expression (FPKM, TPM, counts) — heavily right-skewed
- Read counts — discrete, non-negative, often zero-inflated
- Allele frequencies — bounded between 0 and 1
- Survival times — always positive, typically right-skewed
- Any data with a hard boundary (concentrations cannot be negative)

```
set_seed(42)
# Generate and visualize normal data
let heights = rnorm(5000, 170, 8)

histogram(heights, {bins: 50, title: "Adult Heights (cm) – Normal
Distribution"})
let stats = summary(heights)
print("Mean: {stats.mean:.1}, Median: {stats.median:.1}, Skewness:
{stats.skewness:.3}")
# Mean and median nearly identical; skewness near zero – hallmarks
of normality
```

The Log-Normal Distribution

If gene expression is not normal, what is it? In most cases, it is **log-normal**: the data itself is skewed, but the logarithm of the data is normally distributed.

Why Gene Expression Is Log-Normal

Gene regulation is a cascade of multiplicative processes. A transcription factor binds (or doesn't), an enhancer activates (fold-change), mRNA is stabilized (half-life multiplied), ribosomes translate at varying rates (multiplied efficiency). When effects multiply rather than add, the result is log-normal, not normal.

This is a mathematical fact: if $X = Y_1 \times Y_2 \times \dots \times Y_n$ and the Y values are independent, then $\log(X) = \log(Y_1) + \log(Y_2) + \dots + \log(Y_n)$. Sums of independent variables tend toward normal (by the Central Limit Theorem), so $\log(X)$ is approximately normal, meaning X is log-normal.

The Log-Transform Trick

This is why bioinformaticians routinely log-transform expression data before analysis:

```
set_seed(42)
# Simulate gene expression (log-normal)
let log_expr = rnorm(5000, 3.0, 2.0)
let expression = log_expr |> map(|x| 2.0 ** x) # 2^x to simulate FPKM

# Raw expression: heavily skewed
histogram(expression, {bins: 50, title: "Raw FPKM – Right Skewed"})
let raw_stats = summary(expression)
print("Raw – Mean: {raw_stats.mean:.1}, Median: {raw_stats.median:.1}, Skew: {raw_stats.skewness:.2}")

# Log2-transformed: approximately normal
let log2_expr = expression |> map(|x| log2(x + 1)) # +1 to handle zeros
histogram(log2_expr, {bins: 50, title: "log2(FPKM+1) – Approximately Normal"})
let log_stats = summary(log2_expr)
print("Log2 – Mean: {log_stats.mean:.1}, Median: {log_stats.median:.1}, Skew: {log_stats.skewness:.2}")
```

After log-transformation, the mean and median converge, skewness drops toward zero, and the histogram looks bell-shaped. Now parametric tests are

appropriate.

Clinical relevance: Differential expression tools like DESeq2 and edgeR work with counts and model them with the negative binomial distribution, but many downstream analyses (clustering, PCA, visualization) require log-transformed data. Understanding why is essential for correct analysis.

The Poisson Distribution

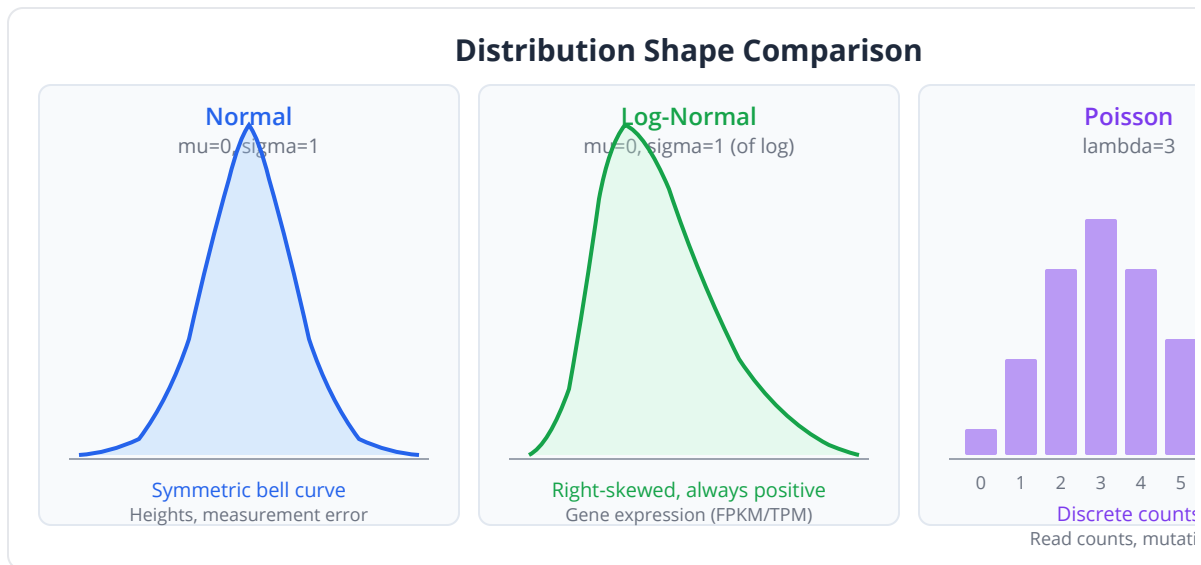
The Poisson distribution models the number of events that occur in a fixed interval, when events happen independently at a constant average rate.

Properties

- **Parameter:** λ (lambda) — the average rate
- **Support:** 0, 1, 2, 3, ... (non-negative integers)
- **Mean = Variance = λ** (this is the key property)

Biological Examples

Application	What is the “event”?	What is the “interval”?
RNA-seq read counts	One read mapping to a gene	One gene in one sample
Mutations	One mutation	Per megabase of genome
Rare diseases	One case	Per 100,000 population per year
Sequencing errors	One error	Per 1000 bases



The Overdispersion Problem

In theory, RNA-seq counts should be Poisson. In practice, biological replicates show more variability than Poisson predicts — the variance exceeds the mean. This is **overdispersion**, caused by biological variability between samples.

The solution is the **negative binomial** distribution, which adds a dispersion parameter to allow variance > mean. This is why DESeq2 uses negative binomial, not Poisson.

```
set_seed(42)
# Poisson distribution for mutation counts

# Average 3.5 mutations per megabase in a tumor
let mutation_counts = rpois(1000, 3.5)

histogram(mutation_counts, {bins: 15, title: "Mutations per Megabase
(Poisson, lambda=3.5)"})

# Verify mean ~ variance (Poisson property)
print("Mean: {mean(mutation_counts):.2}")
print("Variance: {variance(mutation_counts):.2}")
# Both should be close to 3.5

# Probability of seeing 10+ mutations in a region (hypermutation?)
let p_hyper = 1.0 - ppois(9, 3.5)
print("P(10+ mutations): {p_hyper:.4}")
# Very low – a region with 10+ mutations is genuinely unusual
```

The Binomial Distribution

The binomial distribution models the number of successes in a fixed number of independent trials, each with the same probability of success.

Properties

- **Parameters:** n (number of trials), p (probability of success)
- **Support:** $0, 1, 2, \dots, n$
- **Mean = np , Variance = $np(1-p)$**

Biological Examples

Application	What is a "trial"?	What is "success"?	What is p?
Heterozygous genotype	One offspring	Inherits variant allele	0.5
SNP calling	One read at a position	Carries alt allele	VAF
Drug response	One patient	Responds	Response rate
Hardy-Weinberg	One individual	Has genotype AA	p^2

Hardy-Weinberg Equilibrium

For a biallelic locus with allele frequencies p and $q = 1-p$, Hardy-Weinberg predicts genotype frequencies:

- AA: p^2
- Aa: $2pq$
- aa: q^2

Deviations from HWE can indicate selection, population structure, or genotyping error.

```
set_seed(42)
# Genotype frequencies under Hardy-Weinberg
let p = 0.3 # frequency of allele A
let q = 1.0 - p

let freq_AA = p * p          # 0.09
let freq_Aa = 2.0 * p * q    # 0.42
let freq_aa = q * q          # 0.49

print("Expected genotype frequencies:")
print("  AA: {freq_AA:.3}")
print("  Aa: {freq_Aa:.3}")
print("  aa: {freq_aa:.3}")

# Simulate genotyping 500 individuals
let n_individuals = 500
let AA_count = rbinom(1, n_individuals, freq_AA) |> sum()

# Probability of observing exactly k heterozygotes
let k = 200
let p_exact = dbinom(k, n_individuals, freq_Aa)
print("P(exactly {k} heterozygotes in {n_individuals}):
{p_exact:.6}")

# Probability of 230+ heterozygotes (possible HWE violation?)
let p_excess = 1.0 - pbinom(229, n_individuals, freq_Aa)
print("P(230+ heterozygotes): {p_excess:.4}")
```

Checking Your Distribution: Diagnostic Tools

Before running any parametric test, verify that your data matches its assumptions. Here are the essential tools.

Q-Q Plots

A Q-Q (quantile-quantile) plot compares your data's quantiles against the theoretical quantiles of a reference distribution (usually normal). If data follows the reference distribution, points fall on a straight diagonal line. Deviations reveal departures from the assumed shape.

```

set_seed(42)
# Q-Q plot for normal data (should be a straight line)
let normal_data = rnorm(500, 0, 1)
qq_plot(normal_data, {title: "Q-Q Plot: Normal Data"})

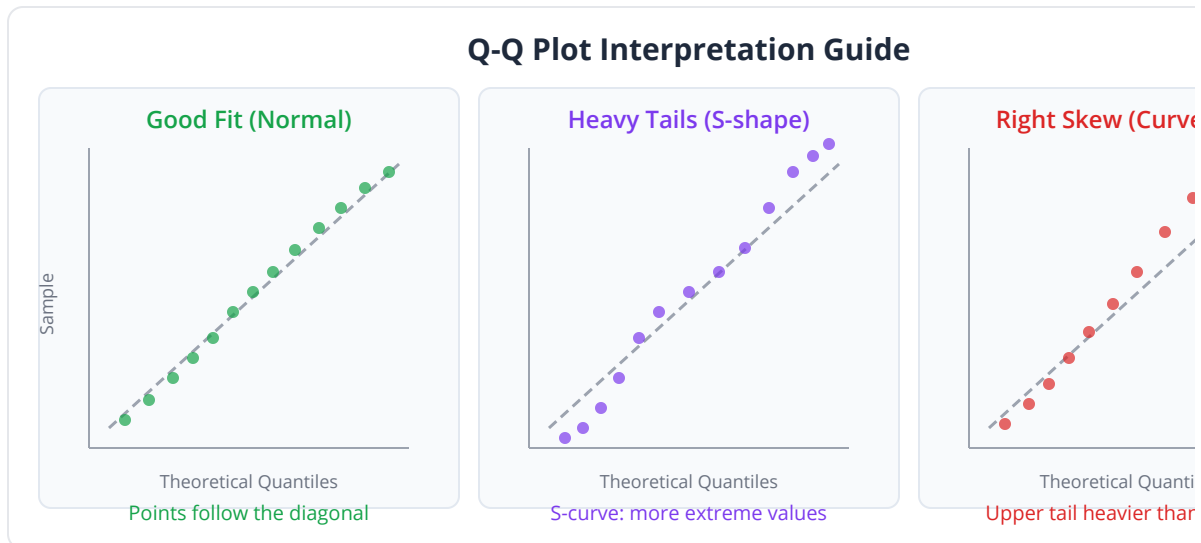
# Q-Q plot for log-normal data (curved – not normal!)
let lognormal_data = normal_data |> map(|x| exp(x))
qq_plot(lognormal_data, {title: "Q-Q Plot: Log-Normal Data (Raw)"})

# Q-Q plot after log-transform (straight again)
let transformed = lognormal_data |> map(|x| log(x))
qq_plot(transformed, {title: "Q-Q Plot: Log-Normal Data (After Log Transform)"})

```

Reading a Q-Q plot:

- Points on the line: data matches the assumed distribution
- Upward curve at the right end: right skew (heavy right tail)
- Downward curve at the left end: left skew (heavy left tail)
- S-shape: heavy tails on both sides (high kurtosis)



Shapiro-Wilk Test

The Shapiro-Wilk test formally tests whether data is normally distributed. A small p-value (< 0.05) means the data is significantly non-normal.

```
set_seed(42)
# Test normality visually with Q-Q plots and histograms
let normal_data = rnorm(200, 50, 10)
let skewed_data = rnorm(200, 3, 1) |> map(|x| exp(x))

# For normal data: Q-Q plot should show points on the diagonal
qq_plot(normal_data, {title: "Q-Q: Normal Data"})
let stats_normal = summary(normal_data)
print("Normal data – Skewness: {stats_normal.skewness:.4}")
# Skewness near 0: consistent with normality

# For skewed data: Q-Q plot will curve away from the diagonal
qq_plot(skewed_data, {title: "Q-Q: Skewed Data"})
let stats_skewed = summary(skewed_data)
print("Skewed data – Skewness: {stats_skewed.skewness:.4}")
# High skewness: definitely not normal
```

Common pitfall: The Shapiro-Wilk test is very sensitive with large samples. With $n = 10,000$, it will reject normality for data that is “close enough” to normal for practical purposes. For large samples, rely more on Q-Q plots and skewness/kurtosis values than on the Shapiro-Wilk p-value.

Histogram with Density Overlay

Overlay the theoretical density curve on your histogram to visually assess fit:

```
set_seed(42)
let data = rnorm(2000, 100, 15)

# Histogram with normal density overlay
histogram(data, {bins: 40, title: "Data with Normal Fit Overlay"})
density(data, {title: "Kernel Density Estimate"})
```

Distribution Summary Table

Distribution	Shape	Parameters	Mean	Variance
Normal	Symmetric bell	μ, σ	μ	σ^2
Log-Normal	Right-skewed	μ, σ (of log)	$\exp(\mu + \sigma^2/2)$	Complex
Poisson	Right-skewed (low λ), symmetric (high λ)	λ	λ	λ
Binomial	Varies	n, p	np	$np(1-p)$
Negative Binomial	Right-skewed	r, p	$r(1-p)/p$	$r(1-p)/p^2$
Beta	Flexible (0,1)	α, β	$\alpha/(\alpha+\beta)$	Complex



Python and R Equivalents

Python:

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

# Normal distribution
x = np.linspace(-4, 4, 1000)
plt.plot(x, stats.norm.pdf(x, 0, 1))

# Poisson
counts = np.random.poisson(lam=3.5, size=1000)

# Binomial
genotypes = np.random.binomial(n=500, p=0.42, size=1000)

# Q-Q plot
stats.probplot(data, dist="norm", plot=plt)

# Shapiro-Wilk test
stat, p = stats.shapiro(data)

# Distribution fitting
params = stats.norm.fit(data) # MLE fit
```

R:

```
# Normal distribution
x <- seq(-4, 4, length.out = 1000)
plot(x, dnorm(x, 0, 1), type = "l")

# Poisson
counts <- rpois(1000, lambda = 3.5)

# Binomial
genotypes <- rbinom(1000, size = 500, prob = 0.42)

# Q-Q plot
qqnorm(data)
qqline(data)

# Shapiro-Wilk test
shapiro.test(data)

# Density overlay
hist(data, freq = FALSE)
curve(dnorm(x, mean(data), sd(data)), add = TRUE, col = "red")
```

Why This Matters for Testing

Here is the critical connection between today's material and the rest of the book:

If your data is...	Then you can use...	But NOT...
Normal	t-test, ANOVA, Pearson correlation	—
Log-normal	t-test after log-transform	t-test on raw values
Poisson counts	Poisson regression, exact tests	t-test
Overdispersed counts	Negative binomial (DESeq2, edgeR)	Poisson, t-test
Non-normal, unknown	Mann-Whitney, Kruskal-Wallis	t-test, ANOVA
Bounded (0,1)	Beta regression, logit transform	Linear regression

Choosing the wrong test because you assumed the wrong distribution is one of the most common errors in computational biology. Dr. Park's mistake from our opening scenario — running a t-test on raw FPKM values — is committed daily in bioinformatics labs around the world.

Key insight: The distribution is not a detail. It is the foundation. Get it right, and your downstream analysis is trustworthy. Get it wrong, and no amount of sophisticated testing can rescue your conclusions.

Exercises

Exercise 1: Identify the Distribution

For each dataset, determine the most appropriate distribution and justify your choice.

```
set_seed(42)
# Dataset A: Sequencing read counts per gene
let dataset_a = rpois(1000, 25)

# Dataset B: Patient blood pressure
let dataset_b = rnorm(500, 120, 15)

# Dataset C: Gene expression (raw TPM)
let log_vals = rnorm(2000, 2.0, 1.5)
let dataset_c = log_vals |> map(|x| 10.0 ** x)

# TODO: For each dataset, create a histogram and Q-Q plot
# TODO: Check normality visually with qq_plot() and histogram()
# TODO: For dataset C, try log-transforming and re-check
# TODO: State which distribution best describes each and why
```

Exercise 2: The Poisson Check

Verify whether mutation count data follows a Poisson distribution by checking the mean-variance relationship.

```
set_seed(42)
# Scenario 1: Pure Poisson (technical replicates)
let technical = rpois(500, 8.0)

# Scenario 2: Overdispersed (biological replicates)
# Simulate by mixing Poisson with varying lambda
let lambdas = rnorm(500, 8.0, 3.0) |> map(|x| max(x, 0.1))

# TODO: Compute mean and variance for technical replicates
# TODO: Are they approximately equal? (Poisson property)
# TODO: For overdispersed data, compute the dispersion ratio
# (variance/mean)
# TODO: What does a ratio >> 1 tell you about the data?
```

Exercise 3: Transform and Test

Take a skewed dataset, find the right transformation, and verify normality.

```
set_seed(42)
let protein_abundance = rnorm(300, 4, 1.2) |> map(|x| exp(x))

# TODO: Plot histogram of raw data
# TODO: Check normality with qq_plot() – is it normal?
# TODO: Apply log transform
# TODO: Plot histogram of transformed data
# TODO: Check normality of transformed data with qq_plot()
# TODO: Compare skewness before and after
```

Exercise 4: Distribution Detective

A collaborator gives you mystery data. Identify its distribution using all tools from today.

```
set_seed(42)
# Mystery data – what distribution is this?
let mystery = rbinom(1000, 50, 0.15)

# TODO: Compute summary()
# TODO: Create histogram
# TODO: Try Q-Q plot against normal
# TODO: Note: the data is discrete. What distributions produce
discrete data?
# TODO: Estimate the parameters and identify the distribution
```

Key Takeaways

- A **distribution** is the theoretical shape describing how likely each value is. Every dataset has one, and every statistical test assumes one.
- The **normal distribution** arises from additive effects and is defined by mean and standard deviation. It is appropriate for measurement error and many physiological traits.
- **Gene expression is NOT normal** — it is log-normal because gene regulation is multiplicative. Always log-transform before using parametric tests.
- The **Poisson distribution** models count data (reads, mutations) with the key property that mean equals variance. When variance exceeds the mean (overdispersion), use the negative binomial instead.
- The **binomial distribution** models fixed trials with a success probability — relevant for genotype frequencies and allele sampling.
- **Q-Q plots** are the most informative visual diagnostic for distribution checking. The **Shapiro-Wilk test** provides a formal hypothesis test for normality.
- Choosing the right distribution is not optional — it determines which statistical tests are valid and which will produce misleading results.

What's Next

You now understand the shapes that biological data takes. But distributions describe what values are likely — which is just another way of saying they describe probabilities. Tomorrow, on Day 4, we formalize probability itself. You will learn to compute the chance that a child inherits a BRCA1 mutation, understand why a positive genetic test might mean less than you think (Bayes' theorem will surprise you), and discover why the prosecutor's fallacy has sent innocent people to prison. Probability is the language of uncertainty, and uncertainty is the native tongue of biology.

Day 4: Probability — Quantifying Uncertainty

Day 4 of 30

Prerequisites: Days 1-3

~50 min

Hands-on

The Problem

Maria and David sit in a genetic counselor's office. The air is still. Maria has just learned she carries a pathogenic BRCA1 mutation — a variant that dramatically increases lifetime risk of breast and ovarian cancer. They are planning to start a family and they need answers.

What is the probability their child inherits the mutation? If the child inherits it, what is the probability she develops breast cancer by age 70? They are considering preimplantation genetic testing — if the test says the embryo is mutation-free, how confident can they be? The counselor pulls out a notepad and begins writing probabilities.

This is not an abstract exercise. These numbers will determine whether Maria and David proceed with natural conception, pursue IVF with genetic screening, or consider adoption. The difference between a 50% risk and a 5% risk changes lives. Understanding how to compute, combine, and interpret probabilities is not just statistics — in genetics, it is clinical care.

What Is Probability?

Probability is a number between 0 and 1 that quantifies how likely an event is to occur. Zero means impossible. One means certain. Everything interesting happens in between.

Think of probability as a weather forecast. When the forecaster says “70% chance of rain,” she means: in historical situations with similar atmospheric conditions, it rained about 70% of the time. She is not saying it will rain exactly 70% of the total rainfall. She is quantifying uncertainty about a future event using past data and models.

In biology, we use probability constantly:

- The probability a child inherits a specific allele from a heterozygous parent: 0.5
- The probability a random human carries at least one pathogenic BRCA1 variant: roughly 1/400
- The probability that a drug produces a response in a given cancer type: varies, but measured in clinical trials
- The probability that a sequencing read is mapped incorrectly: the mapping quality score encodes this directly

Probability	Meaning	Example
0	Impossible	Rolling a 7 on a standard die
0.001	Very unlikely	Rare disease prevalence
0.05	Unlikely	Conventional significance threshold
0.25	Possible	Child inheriting recessive allele from two carriers
0.50	Even odds	Coin flip, heterozygous allele transmission
0.95	Very likely	Diagnostic test sensitivity
1.0	Certain	Sum of all possible outcomes

Basic Rules of Probability

The Addition Rule

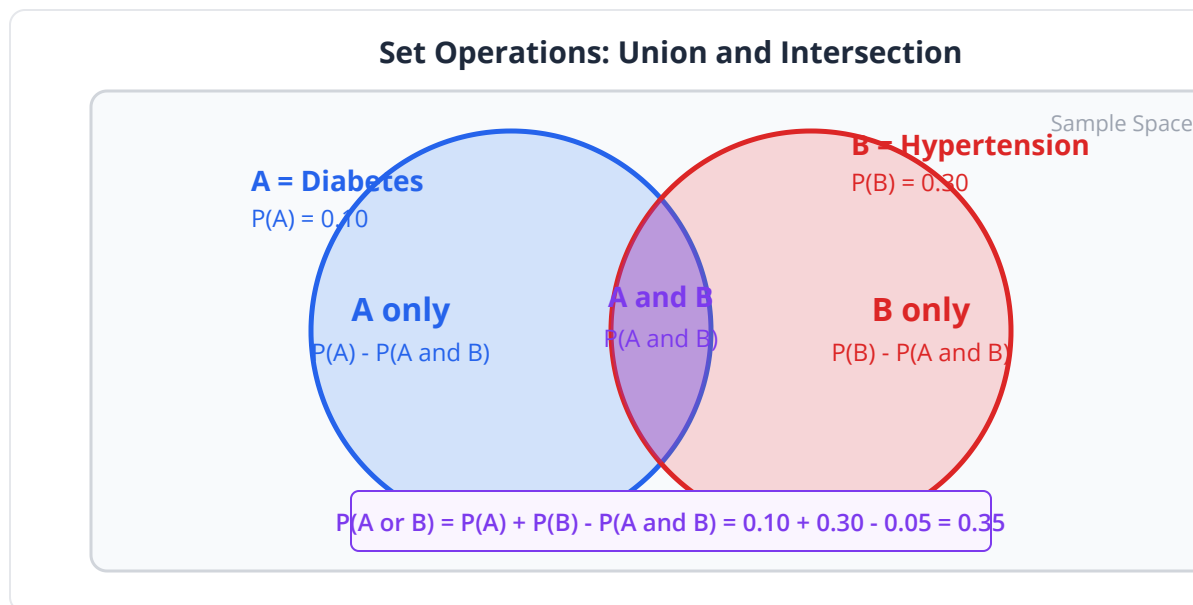
The probability of event A **or** event B occurring depends on whether they can both happen at the same time.

Mutually exclusive events (cannot co-occur): $P(A \text{ or } B) = P(A) + P(B)$

A person's blood type is A, B, AB, or O. These are mutually exclusive — you cannot be both type A and type B simultaneously. So $P(A \text{ or } B) = P(A) + P(B) = 0.40 + 0.11 = 0.51$.

Non-mutually exclusive events (can co-occur): $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$

A patient might have diabetes, hypertension, or both. If $P(\text{diabetes}) = 0.10$, $P(\text{hypertension}) = 0.30$, and $P(\text{both}) = 0.05$, then $P(\text{either}) = 0.10 + 0.30 - 0.05 = 0.35$. You subtract the overlap to avoid double-counting.



The Multiplication Rule

The probability of event A **and** event B both occurring depends on whether they are independent.

Independent events (one does not affect the other): $P(A \text{ and } B) = P(A) \times P(B)$

The probability that two unrelated people both carry a BRCA1 mutation:
 $P(\text{carrier}) \times P(\text{carrier}) = (1/400) \times (1/400) = 1/160,000$.

Dependent events (one affects the other): $P(A \text{ and } B) = P(A) \times P(B | A)$

where $P(B | A)$ is the probability of B **given that** A has occurred. If a mother carries BRCA1 (event A), the probability her daughter both inherits it (B) and develops cancer by 70 (C) is: $P(B \text{ and } C) = P(B | A) \times P(C | B) = 0.5 \times 0.72 = 0.36$.

The Complement Rule

$$P(\text{not } A) = 1 - P(A)$$

The probability of **not** inheriting the mutation is $1 - 0.5 = 0.5$. The probability a diagnostic test does **not** give a false positive is $1 - (\text{false positive rate})$. Simple but powerful — often easier to compute the complement and subtract.

Key insight: Most probability errors in biology come from confusing independent and dependent events, or from forgetting to subtract the overlap in non-mutually exclusive events. Write out the formula before plugging in numbers.

Conditional Probability

Conditional probability is the probability of an event **given that** another event has occurred. It is written $P(A | B)$ and read “the probability of A given B.”

This is where most people's intuition breaks down, because **$P(A|B)$ is not the same as $P(B|A)$** .

The Critical Distinction

- $P(\text{cancer} \mid \text{BRCA1 mutation}) \approx 0.72$ — If you carry BRCA1, your lifetime breast cancer risk is about 72%.
- $P(\text{BRCA1 mutation} \mid \text{cancer}) \approx 0.05$ — If you have breast cancer, the probability it is due to BRCA1 is only about 5%.

These are completely different numbers answering completely different questions. Confusing them is called the **inverse probability fallacy**, and it has real consequences in medicine, law, and genetics.

The Prosecutor's Fallacy

In forensic genetics, the prosecutor's fallacy works like this: "The probability of this DNA match occurring by chance is 1 in 10 million. Therefore, the probability the defendant is innocent is 1 in 10 million."

This is logically wrong. $P(\text{match} \mid \text{innocent})$ is not $P(\text{innocent} \mid \text{match})$. In a city of 8 million people, you would expect roughly one other person to match by chance. If the only evidence is the DNA match, the probability of innocence might be closer to 50%, not 1 in 10 million.

The same fallacy appears in genetic testing: "The test is 99% accurate" does not mean a positive result is 99% likely to be correct. The answer depends on how common the condition is — which brings us to Bayes' theorem.

Common pitfall: Never interpret $P(\text{result} \mid \text{hypothesis})$ as $P(\text{hypothesis} \mid \text{result})$. This mistake is ubiquitous in clinical genetics, drug development, and forensics. Bayes' theorem is the correction.

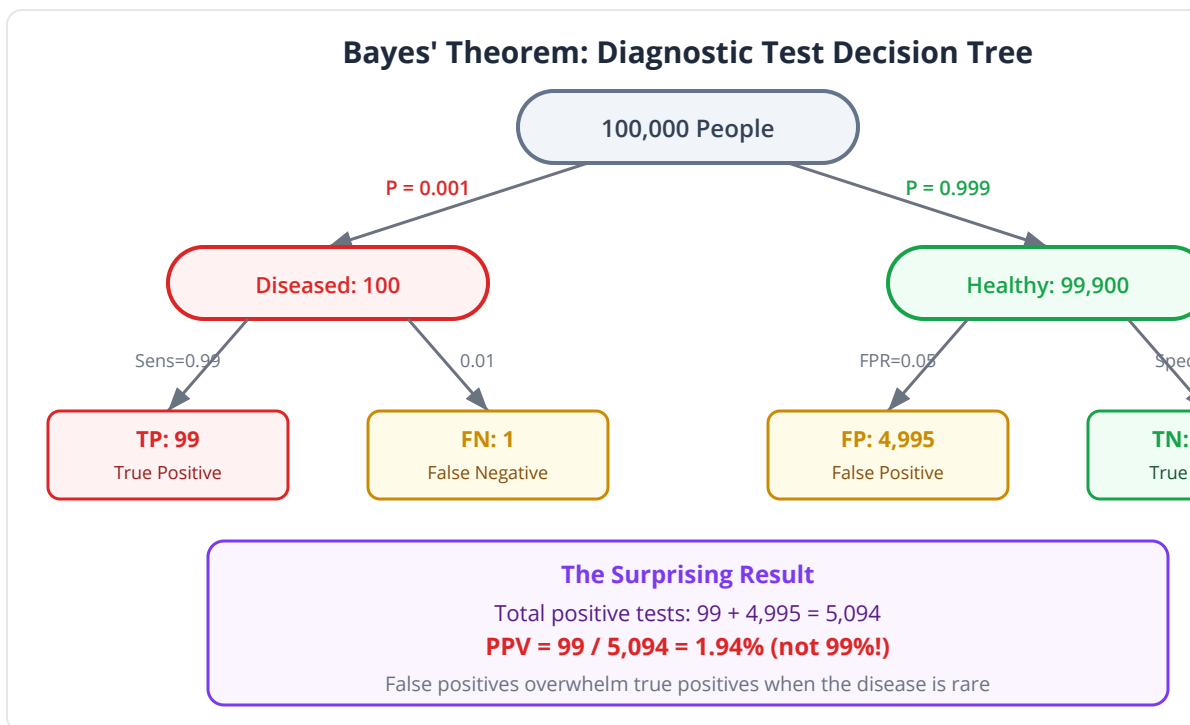
Bayes' Theorem

Bayes' theorem provides the mathematical machinery to flip conditional probabilities. Given $P(B|A)$, it computes $P(A|B)$:

$$P(A|B) = P(B|A) \times P(A) / P(B)$$

In words: the probability of A given B equals the probability of B given A, times the prior probability of A, divided by the total probability of B.

The Diagnostic Test Example



This is the single most important application of Bayes' theorem in biomedical science. Work through it carefully.

Setup:

- A genetic disease has a prevalence of 1 in 1,000 ($P(\text{disease}) = 0.001$)
- A diagnostic test has 99% sensitivity: $P(\text{positive} | \text{disease}) = 0.99$

- The test has 95% specificity: $P(\text{negative} \mid \text{no disease}) = 0.95$, so $P(\text{positive} \mid \text{no disease}) = 0.05$

Question: A patient tests positive. What is the probability they actually have the disease?

Intuition says: The test is 99% sensitive and 95% specific — surely a positive result means ~95-99% chance of disease?

Bayes says:

$$P(\text{disease} \mid \text{positive}) = P(\text{positive} \mid \text{disease}) \times P(\text{disease}) / P(\text{positive})$$

$$P(\text{positive}) = P(\text{positive} \mid \text{disease}) \times P(\text{disease}) + P(\text{positive} \mid \text{no disease}) \times P(\text{no disease})$$

$$P(\text{positive}) = 0.99 \times 0.001 + 0.05 \times 0.999$$

$$P(\text{positive}) = 0.00099 + 0.04995 = 0.05094$$

$$P(\text{disease} \mid \text{positive}) = 0.00099 / 0.05094 = \mathbf{0.0194}$$

A positive test result means only a **1.94% chance** of actually having the disease.

How can a “99% accurate” test give such a low positive predictive value?

Because the disease is rare. In 100,000 people, 100 have the disease (99 test positive) and 99,900 are healthy (4,995 test positive by mistake). Of the 5,094 total positives, only 99 are true positives.

Group	Population	Test Positive	Test Negative
Diseased	100	99 (TP)	1 (FN)
Healthy	99,900	4,995 (FP)	94,905 (TN)
Total	100,000	5,094	94,906

$PPV = 99 / 5,094 = 1.94\%$. The false positives overwhelm the true positives.

Clinical relevance: This is why population-wide genetic screening for rare conditions produces mostly false positives. It is also why a confirmatory test with a different methodology is always required. Understanding PPV is essential for anyone interpreting genetic test results.

Bayes in BioLang

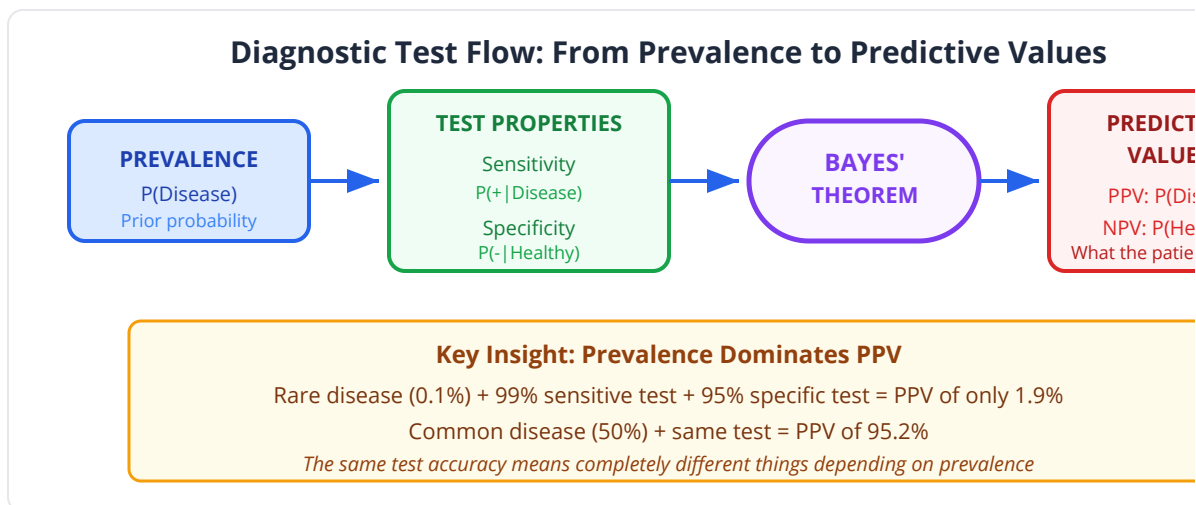
```
# Diagnostic test calculator using Bayes' theorem
let prevalence = 0.001      # P(disease) = 1 in 1,000
let sensitivity = 0.99      # P(positive | disease)
let specificity = 0.95      # P(negative | no disease)
let fpr = 1.0 - specificity # P(positive | no disease) = 0.05

# Total probability of testing positive
let p_positive = sensitivity * prevalence + fpr * (1.0 - prevalence)

# Positive Predictive Value (PPV)
let ppv = (sensitivity * prevalence) / p_positive

# Negative Predictive Value (NPV)
let p_negative = (1.0 - sensitivity) * prevalence + specificity *
(1.0 - prevalence)
let npv = (specificity * (1.0 - prevalence)) / p_negative

print("=== Diagnostic Test Analysis ===")
print("Prevalence:      {prevalence}")
print("Sensitivity:      {sensitivity}")
print("Specificity:       {specificity}")
print("P(positive):       {p_positive:.4}")
print("PPV:                {ppv:.4} ({ppv * 100:.1}%)")
print("NPV:                {npv:.6} ({npv * 100:.4}%)")
print("")
print("Interpretation: A positive result means only a {ppv *
100:.1}% chance of disease.")
print("A negative result means a {npv * 100:.4}% chance of being
disease-free.")
```



How Prevalence Changes Everything

```

# Show how PPV changes with disease prevalence
let sensitivity = 0.99
let specificity = 0.95

let prevalences = [0.0001, 0.001, 0.01, 0.05, 0.10, 0.20, 0.50]

print("Prevalence | PPV")
print("-----|-----")
for prev in prevalences {
  let fpr = 1.0 - specificity
  let p_pos = sensitivity * prev + fpr * (1.0 - prev)
  let ppv = (sensitivity * prev) / p_pos
  print(" {prev:.4} | {ppv * 100:.1}%")
}
# At 0.01% prevalence: PPV = 0.2% (nearly all positives are false)
# At 50% prevalence: PPV = 95.2% (most positives are true)
  
```

This table is one of the most important results in clinical statistics. It explains why screening tests work well for common conditions but poorly for rare ones.

Probability Distributions Revisited

On Day 3, we met distributions as shapes. Now we can understand them as probability functions.

Discrete Distributions

For discrete random variables (counts, genotypes), the **probability mass function** (PMF) gives $P(X = k)$ — the probability of observing exactly the value k .

```
# Binomial PMF: probability of exactly k successes in n trials
# Scenario: 4 children, mother is BRCA1 carrier (p = 0.5)
let n_children = 4
let p_inherit = 0.5

print("Number of children inheriting BRCA1:")
for k in 0..5 {
  let prob = dbinom(k, n_children, p_inherit)
  print("  {k} children: {prob:.4} ({prob * 100:.1}%)")
}
# 0: 6.25%, 1: 25.0%, 2: 37.5%, 3: 25.0%, 4: 6.25%
```

Cumulative Distribution

The **cumulative distribution function** (CDF) gives $P(X \leq k)$ — the probability of observing k or fewer.

```
# What's the probability that at most 1 of 4 children inherits the
mutation?
let p_at_most_1 = pbinom(1, 4, 0.5)
print("P(0 or 1 child inherits): {p_at_most_1:.4}") # 0.3125

# What's the probability at least 1 inherits?
let p_at_least_1 = 1.0 - pbinom(0, 4, 0.5)
print("P(at least 1 inherits): {p_at_least_1:.4}") # 0.9375
```

Continuous Distributions

For continuous random variables (gene expression, blood pressure), the **probability density function** (PDF) does not give $P(X = k)$ (which is always zero for continuous variables). Instead, probabilities are computed over intervals using the CDF.

```
# Blood pressure: normal with mean 120, SD 15
let mu = 120.0
let sigma = 15.0

# P(BP > 140) – hypertension threshold
let p_hypertension = 1.0 - pnorm(140, mu, sigma)
print("P(BP > 140): {p_hypertension:.4}") # ~0.0912

# P(100 < BP < 130)
let p_normal_range = pnorm(130, mu, sigma) - pnorm(100, mu, sigma)
print("P(100 < BP < 130): {p_normal_range:.4}")

# What BP value has only 5% of people above it?
let bp_95th = qnorm(0.95, mu, sigma)
print("95th percentile BP: {bp_95th:.1}") # ~144.7
```

Hardy-Weinberg as a Probability Model

Hardy-Weinberg equilibrium (HWE) is one of the most elegant applications of probability in genetics. For a biallelic locus with allele frequencies p (allele A) and $q = 1 - p$ (allele a), random mating produces genotypes with probabilities:

Genotype	Frequency	If $p = 0.3$
AA	p^2	0.09
Aa	$2pq$	0.42
aa	q^2	0.49

This model treats each allele transmission as an independent random event — like drawing two alleles from a bag with replacement.

```

# Test for Hardy-Weinberg equilibrium
# Observed genotype counts from a population study
let observed_AA = 45
let observed_Aa = 210
let observed_aa = 245
let total = observed_AA + observed_Aa + observed_aa # 500

# Estimate allele frequencies
let p_A = (2.0 * observed_AA + observed_Aa) / (2.0 * total)
let p_a = 1.0 - p_A
print("Estimated allele frequencies: p(A) = {p_A:.3}, p(a) = {p_a:.3}")

# Expected counts under HWE
let expected_AA = p_A * p_A * total
let expected_Aa = 2.0 * p_A * p_a * total
let expected_aa = p_a * p_a * total
print("Expected: AA={expected_AA:.1}, Aa={expected_Aa:.1}, aa={expected_aa:.1}")
print("Observed: AA={observed_AA}, Aa={observed_Aa}, aa={observed_aa}")

# Chi-square test for HWE (we'll cover this formally on Day 13)
let chi2 = (observed_AA - expected_AA) ** 2 / expected_AA +
           (observed_Aa - expected_Aa) ** 2 / expected_Aa +
           (observed_aa - expected_aa) ** 2 / expected_aa
print("Chi-square statistic: {chi2:.3}")
print("Deviation from HWE: {if chi2 > 3.84 then 'Significant' else 'Not significant'}")

```

Carrier Probability Calculations

Returning to Maria and David's consultation:

```
# Genetic counseling probability calculator

# Maria is a BRCA1 carrier (heterozygous)
# David is not a carrier (assumed)

# Autosomal dominant: 50% chance each child inherits
let p_inherit = 0.5

# They want 3 children
let n_children = 3

print("=== Genetic Counseling: BRCA1 Inheritance ===")
print("")

# Probability none of 3 children inherit
let p_none = dbinom(0, n_children, p_inherit)
print("P(no children inherit): {p_none:.4} ({p_none * 100:.1}%)")

# Probability exactly 1 inherits
let p_one = dbinom(1, n_children, p_inherit)
print("P(exactly 1 inherits): {p_one:.4} ({p_one * 100:.1}%)")

# Probability at least 1 inherits
let p_at_least_one = 1.0 - p_none
print("P(at least 1 inherits): {p_at_least_one:.4} ({p_at_least_one * 100:.1}%)")

print("")
print("=== Conditional Cancer Risk ===")
# If a daughter inherits BRCA1, lifetime breast cancer risk ~ 72%
let p_cancer_given_brca = 0.72

# P(inherits AND develops cancer)
let p_inherit_and_cancer = p_inherit * p_cancer_given_brca
print("P(daughter inherits AND gets cancer):
{p_inherit_and_cancer:.3} ({p_inherit_and_cancer * 100:.1}%)")

# P(daughter gets cancer by 70 | she is female)
# Must account for 50% chance of being female
let p_affected_daughter = 0.5 * p_inherit * p_cancer_given_brca
print("P(random child is affected daughter):
{p_affected_daughter:.3} ({p_affected_daughter * 100:.1}%)")
```

Python and R Equivalents

Python:

```
from scipy import stats
import numpy as np

# Binomial probabilities
stats.binom.pmf(k=2, n=4, p=0.5)      # P(X = 2)
stats.binom.cdf(k=1, n=4, p=0.5)      # P(X <= 1)

# Normal probabilities
stats.norm.cdf(140, loc=120, scale=15) # P(X <= 140)
stats.norm.ppf(0.95, loc=120, scale=15) # 95th percentile

# Poisson
stats.poisson.pmf(k=5, mu=3.5)         # P(X = 5)
stats.poisson.cdf(k=9, mu=3.5)         # P(X <= 9)

# Bayes calculation
prevalence = 0.001
sensitivity = 0.99
fpr = 0.05
p_pos = sensitivity * prevalence + fpr * (1 - prevalence)
ppv = (sensitivity * prevalence) / p_pos
```

R:

```
# Binomial
dbinom(2, size = 4, prob = 0.5)    # P(X = 2)
pbinom(1, size = 4, prob = 0.5)    # P(X <= 1)

# Normal
pnorm(140, mean = 120, sd = 15)    # P(X <= 140)
qnorm(0.95, mean = 120, sd = 15)  # 95th percentile

# Poisson
dpois(5, lambda = 3.5)             # P(X = 5)
ppois(9, lambda = 3.5)             # P(X <= 9)

# Bayes
prevalence <- 0.001
sensitivity <- 0.99
fpr <- 0.05
p_pos <- sensitivity * prevalence + fpr * (1 - prevalence)
ppv <- (sensitivity * prevalence) / p_pos
```

Exercises

Exercise 1: The Prenatal Test

A prenatal screening test for Down syndrome has sensitivity 95% and specificity 97%. The prevalence of Down syndrome is approximately 1 in 700 live births for a 30-year-old mother.

```
# TODO: Compute the PPV – if the test is positive, what is the true
probability?
# TODO: Compute the NPV – if the test is negative, how reassuring is
it?
# TODO: How does the PPV change for a 40-year-old mother (prevalence
~1 in 100)?
let prevalence_30 = 1.0 / 700.0
let prevalence_40 = 1.0 / 100.0
let sensitivity = 0.95
let specificity = 0.97
```

Exercise 2: Multiple Independent Events

A patient takes three independent diagnostic tests for a condition. Each test has 90% sensitivity.

```
# TODO: What is P(all three tests are positive | disease)?
# TODO: What is P(at least one test is negative | disease)?
# TODO: What is P(all three tests are negative | disease)?
# TODO: If the patient tests positive on all three, and prevalence
is 1%,
#       what is the updated probability they have the disease?
let sensitivity = 0.90
let specificity = 0.95
let prevalence = 0.01
```

Exercise 3: Carrier Frequency

Cystic fibrosis is autosomal recessive with a carrier frequency of approximately 1 in 25 among Europeans.

```
let carrier_freq = 1.0 / 25.0

# TODO: What is P(both parents are carriers)?
# TODO: If both are carriers, P(affected child)?
# TODO: P(random European couple has an affected child)?
# TODO: If one parent is a confirmed carrier and the other is
untested,
#       what is P(their child is affected)?
```

Exercise 4: Sequencing Error

A variant caller reports a SNV at a position covered by 50 reads. 5 reads support the variant allele.

```
# Is this a real variant or sequencing error?
# Assume sequencing error rate = 0.01 per base per read

let n_reads = 50
let n_alt = 5
let error_rate = 0.01

# TODO: Under the null (all errors), what is P(5+ alt reads)?
# Use the binomial:  $P(X \geq 5 \mid n=50, p=0.01) = 1 - \text{pbinom}(4, 50, 0.01)$ 
# TODO: Is this consistent with error alone, or likely a real variant?
```

Key Takeaways

- **Probability** quantifies uncertainty on a 0-to-1 scale. It is the mathematical language of statistics.
- The **addition rule** handles “or” questions; the **multiplication rule** handles “and” questions. Whether events are mutually exclusive or independent changes the formula.
- **Conditional probability** $P(A \mid B)$ is NOT the same as $P(B \mid A)$. Confusing them is the source of the prosecutor’s fallacy and misinterpretation of diagnostic tests.
- **Bayes’ theorem** is the bridge from $P(B \mid A)$ to $P(A \mid B)$. It shows that a positive test for a rare disease usually means the patient is healthy — the positive predictive value depends critically on prevalence.
- In genetics, probability models like **Hardy-Weinberg equilibrium** and **Mendelian inheritance** translate directly into binomial probability calculations.
- Always compute both **PPV and NPV** when interpreting diagnostic or screening tests. Sensitivity and specificity alone are insufficient.

What's Next

Today you learned to compute probabilities for individual events. But in real experiments, you do not observe single events — you observe samples drawn from populations. Tomorrow, on Day 5, we tackle the crucial question of sampling: Why does sample size matter so much? You will see the Central Limit Theorem in action — watching the distribution of sample means magically approach normality even when the underlying data is wildly skewed. You will understand what “statistical power” means and why a study with 20 patients per arm is almost guaranteed to fail. Day 5 is the bridge between probability theory and the practical reality of experimental design.

Day 5: Sampling, Bias, and Why n Matters

Day 5 of 30

Prerequisites: Days 1-4

~55 min

Hands-on

The Problem

Dr. Elena Vasquez is the lead biostatistician for a pharmaceutical company. A new immunotherapy drug has shown promising results in cell lines and mouse models. Now the clinical team is designing the Phase II trial and they want her sign-off on the sample size.

The clinical lead proposes 20 patients per arm — treatment and placebo. “It’s faster, cheaper, and we can get to Phase III sooner,” he argues. Elena runs the numbers and shakes her head. With 20 patients per arm and the expected effect size, the trial has only a 23% chance of detecting the drug’s benefit even if it truly works. That means a 77% chance of concluding the drug is ineffective when it actually saves lives.

She recommends 200 patients per arm. The clinical lead winces at the cost — \$12 million more and 18 extra months of enrollment. But Elena is firm: “Would you rather spend \$12 million now and know the answer, or spend \$50 million on a Phase III that was doomed from the start because Phase II was too small to see the signal?”

This tension — between the cost of collecting more data and the cost of drawing wrong conclusions from too little — is the central drama of experimental design. Today you will understand why sample size is not a bureaucratic detail but the most consequential decision in any study.

What Are Populations and Samples?

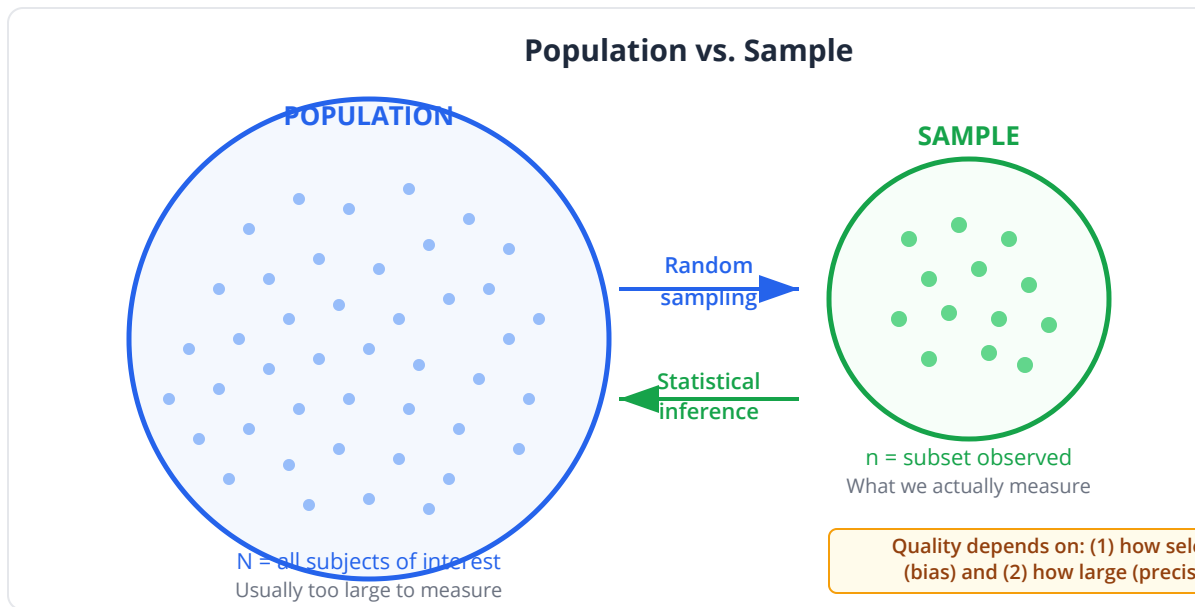
The Population

The **population** is the complete set of items you want to understand. It is usually too large, too expensive, or physically impossible to measure in its entirety.

Research Question	Population
Does this drug lower blood pressure?	All humans with hypertension
Is gene X differentially expressed in tumors?	All tumors of this type, past and future
What is the allele frequency of rs1234?	All humans alive today
Does this sequencing protocol introduce bias?	All possible runs of this protocol

The Sample

The **sample** is the subset you actually observe. Everything you learn comes from the sample, but everything you want to know is about the population. Statistics is the bridge between the two.



The quality of the bridge depends entirely on two factors:

1. **How the sample was selected** (bias)
2. **How large the sample is** (precision)

Key insight: A large biased sample is worse than a small unbiased one. The 1936 Literary Digest poll surveyed 2.4 million people and predicted Alf Landon would win the presidential election in a landslide. George Gallup surveyed 50,000 and correctly predicted Roosevelt. The Literary Digest sample was drawn from telephone directories and automobile registrations — overrepresenting wealthy voters. Size could not compensate for bias.

The Sampling Distribution

This is one of the most important concepts in all of statistics, and it is the one that most students find counterintuitive at first.

Imagine you draw a sample of 30 patients, measure their blood pressure, and compute the mean. You get 125 mmHg. Now imagine you draw a different

sample of 30 patients and compute the mean. You might get 121 mmHg. A third sample: 128 mmHg.

If you repeated this process thousands of times — each time drawing 30 patients and computing the mean — you would get thousands of sample means. These sample means form a distribution called the **sampling distribution of the mean**.

The sampling distribution is NOT the distribution of individual data points. It is the distribution of a statistic (like the mean) computed from repeated samples.

Seeing It in Action

```
set_seed(42)
# Simulate the sampling distribution

# The "population": blood pressure values (slightly right-skewed)
let population = rnorm(100000, 125, 18)

# Draw 1000 samples of size 30, compute mean of each
let sample_means_30 = []
for i in 0..1000 {
  let s = sample(population, 30)
  sample_means_30 = sample_means_30 + [mean(s)]
}

# Draw 1000 samples of size 200, compute mean of each
let sample_means_200 = []
for i in 0..1000 {
  let s = sample(population, 200)
  sample_means_200 = sample_means_200 + [mean(s)]
}

# Compare the distributions
print("Population:      mean = {mean(population):.1}, SD =
{stdev(population):.1}")
print("Sample means (n=30): mean = {mean(sample_means_30):.1}, SD =
{stdev(sample_means_30):.1}")
print("Sample means (n=200): mean = {mean(sample_means_200):.1}, SD
= {stdev(sample_means_200):.1}")

histogram(sample_means_30, {bins: 40, title: "Sampling Distribution
(n=30)"})
histogram(sample_means_200, {bins: 40, title: "Sampling Distribution
(n=200)"})
```

Two crucial observations:

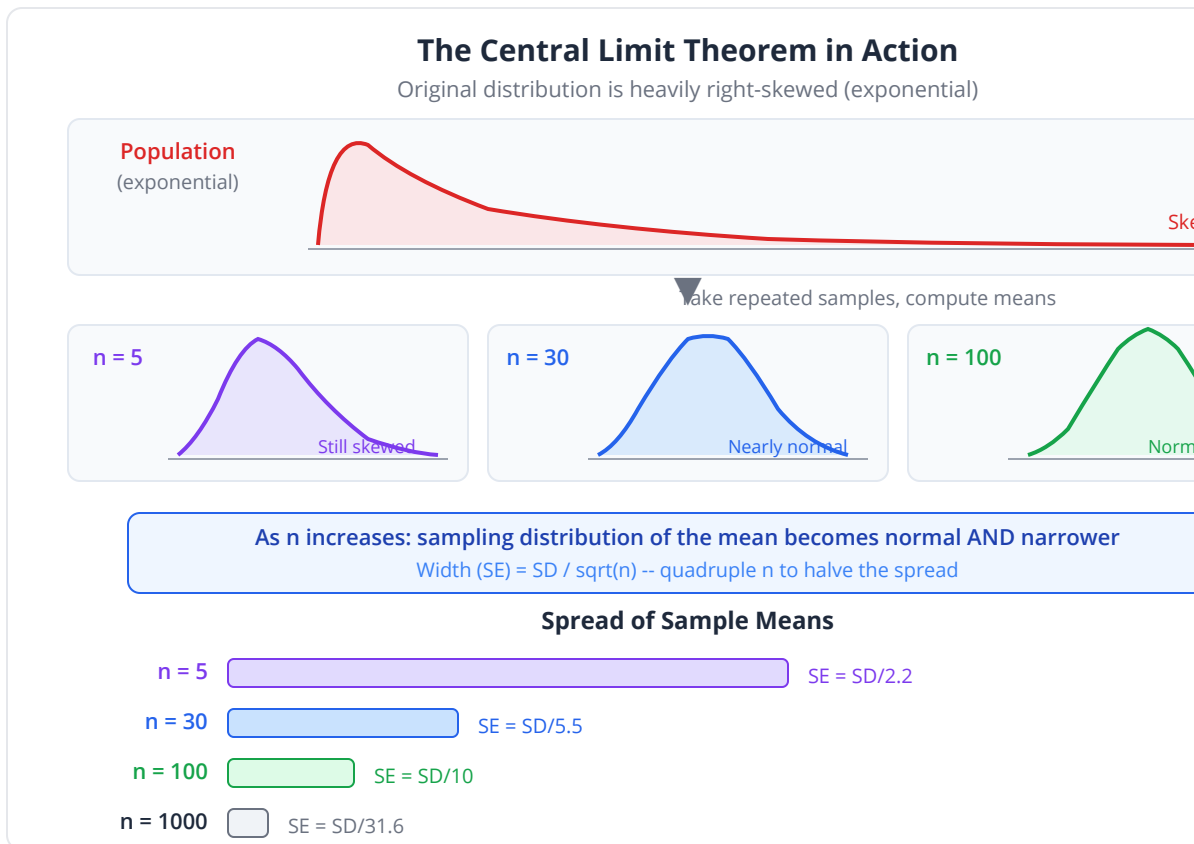
1. Both sampling distributions are centered at the true population mean (~125). Samples are unbiased estimators.
2. The n=200 distribution is much **narrower** than n=30. Larger samples give more precise estimates.

The Central Limit Theorem

The Central Limit Theorem (CLT) is perhaps the single most important result in statistics. It says:

Regardless of the shape of the population distribution, the sampling distribution of the mean approaches a normal distribution as sample size increases.

This is remarkable. The underlying data can be skewed, bimodal, uniform, or any shape at all. As long as you take large enough samples and compute means, those means will be approximately normally distributed.



Demonstrating the CLT

```
set_seed(42)
# Create a wildly non-normal population: exponential (very right-
skewed)
let skewed_pop = rnorm(100000, 2, 1) |> map(|x| exp(x))

# The population is extremely skewed
histogram(skewed_pop, {bins: 50, title: "Population: Exponential
(Very Skewed)"})
let pop_stats = summary(skewed_pop)
print("Population skewness: {pop_stats.skewness:.2}")

# Sample means with n=5 (still somewhat skewed)
let means_n5 = []
for i in 0..2000 {
  let s = sample(skewed_pop, 5)
  means_n5 = means_n5 + [mean(s)]
}
histogram(means_n5, {bins: 50, title: "Sample Means, n=5"})
print("n=5 skewness: {skewness(means_n5):.2}")

# Sample means with n=30 (approaching normal)
let means_n30 = []
for i in 0..2000 {
  let s = sample(skewed_pop, 30)
  means_n30 = means_n30 + [mean(s)]
}
histogram(means_n30, {bins: 50, title: "Sample Means, n=30"})
print("n=30 skewness: {skewness(means_n30):.2}")

# Sample means with n=100 (very close to normal)
let means_n100 = []
for i in 0..2000 {
  let s = sample(skewed_pop, 100)
  means_n100 = means_n100 + [mean(s)]
}
histogram(means_n100, {bins: 50, title: "Sample Means, n=100"})
print("n=100 skewness: {skewness(means_n100):.2}")

# Verify normality visually with Q-Q plot
qq_plot(means_n100, {title: "Q-Q Plot: Sample Means n=100"})
```

Watch the skewness drop toward zero as n increases. By n=100, the sampling distribution is indistinguishable from a normal curve, even though the

underlying data is wildly skewed.

Key insight: The CLT is why the normal distribution dominates statistics. Even when individual observations are non-normal, means of samples are approximately normal. Since most statistical tests are fundamentally about comparing means, the normal distribution is the right reference distribution for the test statistic — even when the raw data is not normal.

When Does the CLT “Kick In”?

The speed of convergence to normality depends on how non-normal the population is:

Population Shape	n Needed for CLT
Already normal	Any n (even n=1)
Slightly skewed	$n \geq 15$
Moderately skewed	$n \geq 30$
Heavily skewed	$n \geq 50-100$
Extremely skewed or heavy-tailed	$n \geq 100+$

The “ $n \geq 30$ ” rule of thumb is a rough guideline, not a universal truth.

Standard Error: The Precision of Your Estimate

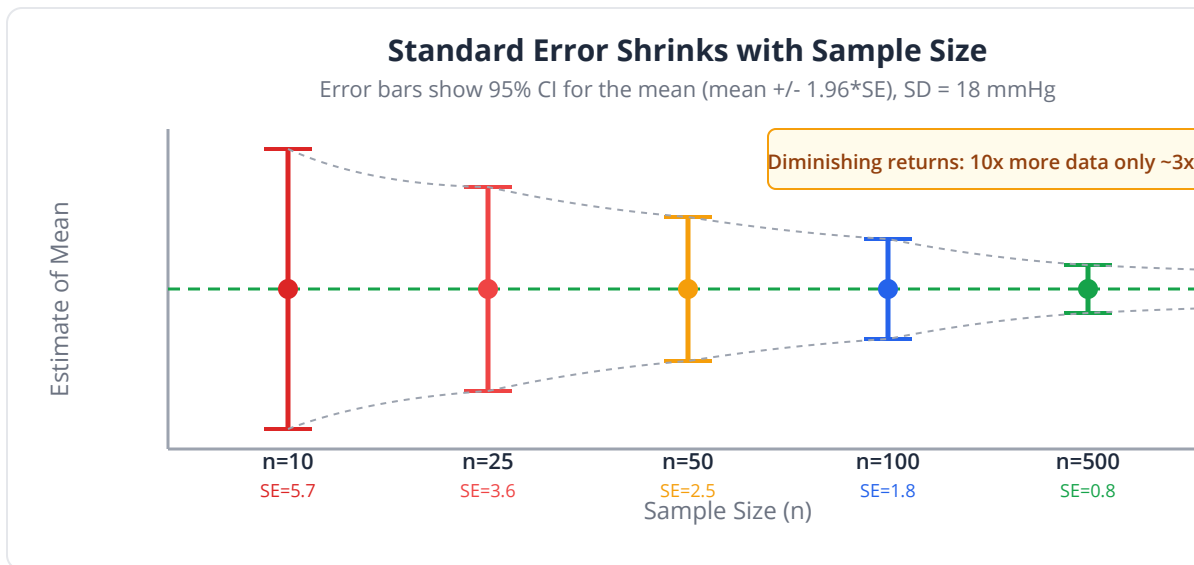
The standard deviation of the sampling distribution has a special name: the **standard error** (SE).

$$SE = SD / \sqrt{n}$$

This formula encodes the fundamental relationship between sample size and precision:

- Double your sample size -> SE decreases by a factor of $\sqrt{2} \sim 1.41$

- Quadruple your sample size -> SE halves
- To cut SE in half, you need 4 times as many observations



```
set_seed(42)
# Demonstrate how SE shrinks with sample size
let population_sd = 18.0 # Blood pressure SD

let sample_sizes = [5, 10, 20, 30, 50, 100, 200, 500, 1000]

print("Sample Size | Theoretical SE | Observed SE")
print("-----|-----|-----")

for n in sample_sizes {
  let theoretical_se = population_sd / sqrt(n)

  # Simulate to verify
  let means = []
  for i in 0..1000 {
    let s = rnorm(n, 125, population_sd)
    means = means + [mean(s)]
  }
  let observed_se = stdev(means)

  print(" {n:>6}      |      {theoretical_se:>6.2}      |
{observed_se:.2}")
}
```


Sample Size	SE (mmHg)	95% CI Width
20	4.02	± 7.9
50	2.55	± 5.0
100	1.80	± 3.5
200	1.27	± 2.5
1000	0.57	± 1.1

With 20 patients, your estimate of mean blood pressure could easily be off by 8 mmHg — enough to misclassify a treatment as effective or ineffective. With 200 patients, you are unlikely to be off by more than 2.5 mmHg.

Common pitfall: Researchers often confuse SD and SE. The SD describes variability among individual observations. The SE describes precision of the sample mean. They answer different questions. Report the right one. SD for describing data; SE for describing the precision of an estimate.

Types of Bias

Sample size controls precision, but even infinite precision cannot fix a biased sample. Bias is a systematic error that pushes your estimate in a consistent direction.

Selection Bias

Your sample is not representative of the population you want to study.

Example: A study of gene expression in breast cancer recruits patients only from a single academic medical center. These patients tend to have more advanced disease (referral bias), are more likely to be white (geographic bias), and have better follow-up (compliance bias). The results may not generalize to community hospitals or diverse populations.

Genomics example: If you study “healthy controls” by recruiting university employees, your sample overrepresents educated, relatively affluent people — not the general population.

Survivorship Bias

You only observe subjects who “survived” some selection process, missing those who did not.

Classic example: During WWII, the military examined bullet holes in returning planes and planned to add armor where holes were most common. Statistician Abraham Wald pointed out the error: they were only seeing planes that survived. The areas with no holes were where planes had been hit and crashed. Armor should go where holes were absent.

Biological example: If you study long-term cancer survivors to find prognostic biomarkers, you miss the patients who died quickly. Your biomarkers will predict survival among survivors, not among all patients.

Ascertainment Bias

The way you identify subjects systematically skews who gets included.

Example: A study finds that children with autism have more genetic variants than controls. But the autistic children were ascertained through clinical evaluation (which involves deep phenotyping and genetic testing), while controls were population-based. The ascertainment process itself led to more thorough variant discovery in cases.

Measurement Bias

The way you measure introduces systematic error.

Example: A technician consistently reads gel bands as slightly brighter than they are. All expression measurements are systematically inflated. If this bias is

constant across all samples, relative comparisons are still valid. If it varies between conditions (e.g., the technician knows which samples are treatment), it corrupts everything.

Genomics example: GC content bias in sequencing — regions with extreme GC content are systematically under-represented in coverage, biasing any analysis that depends on read depth.

Publication Bias

Studies with significant results are more likely to be published than studies with null results. The published literature systematically overestimates effect sizes.

Example: 20 groups test whether gene X is associated with disease Y. One group (by chance) finds $p < 0.05$ and publishes. The other 19 find nothing and file the results away. The published literature now says gene X is associated with disease Y, but the full evidence says otherwise.

Bias Type	What Goes Wrong	Genomics Example
Selection	Non-representative sample	Single-center cohort
Survivorship	Missing failures	Studying only long-term survivors
Ascertainment	Systematic identification skew	More testing in cases vs controls
Measurement	Systematic instrument/observer error	GC bias, batch effects
Publication	Only positive results published	“Significant” GWAS hits that don’t replicate

Key insight: Bias cannot be fixed by increasing sample size. A biased study with 10,000 subjects gives you a very precise wrong answer. Always evaluate bias before interpreting results.

Why n Matters: The Power Preview

Statistical **power** is the probability of detecting a real effect when it exists. It is 1 minus the Type II error rate ($1 - \beta$). Convention targets 80% power, meaning a 20% chance of missing a real effect.

Power depends on four factors:

1. **Effect size** — How large is the true difference? Bigger effects are easier to detect.
2. **Sample size (n)** — More data = more power.
3. **Variability (σ)** — Less noise = more power.
4. **Significance threshold (α)** — More stringent threshold = less power.

Think of detecting a treatment effect as hearing a whisper in a crowd. The whisper is the signal (effect size). The crowd noise is variability. Adding more listeners (larger n) helps. Making the crowd quieter (reducing variability) helps. Demanding absolute certainty before you will believe you heard something (lower α) makes it harder.

Simulating Power

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# Simulate a clinical trial to understand power

# True effect: treatment mean = 125 (control = 130, lower is better)
let control_mean = 130.0
let treatment_mean = 125.0 # 5 mmHg real difference
let sd = 18.0

# Function to run one trial and check if we detect the difference
# Returns 1 if p < 0.05, 0 otherwise
let run_trial = fn(n_per_arm) {
  let control = rnorm(n_per_arm, control_mean, sd)
  let treatment = rnorm(n_per_arm, treatment_mean, sd)
  let result = ttest(treatment, control)
  if result.p_value < 0.05 { 1 } else { 0 }
}

# Run 1000 simulated trials for different sample sizes
let sizes = [20, 50, 100, 200, 500]
print("n per arm | Estimated Power")
print("-----|-----")

for n in sizes {
  let detections = 0
  for i in 0..1000 {
    detections = detections + run_trial(n)
  }
  let power = detections / 1000.0
  print(" {n:>5}    |    {power * 100:.1}%")
}
```

Typical results:

n per arm	Power	Interpretation
20	~23%	Miss the effect 77% of the time — nearly useless
50	~47%	Coin flip — unacceptable for a clinical trial
100	~71%	Getting close but still below the 80% standard
200	~94%	Excellent — high confidence in detecting the effect
500	~99.9%	Virtually certain to detect even subtle effects

This is Dr. Vasquez's argument in numbers. With 20 patients per arm, the trial has a 77% chance of producing a false negative — concluding the drug does not work when it does. That is not an experiment; it is a waste of money.

The Bootstrap: Estimation Without Formulas

The **bootstrap** is a resampling method that estimates the sampling distribution empirically. Instead of relying on mathematical formulas, you resample your data with replacement thousands of times and compute your statistic each time.

The bootstrap is invaluable when:

- The formula for the standard error is unknown or complicated
- The CLT may not apply (small n, skewed data)
- You want confidence intervals for any statistic (median, correlation, ratio)

```
set_seed(42)
# Bootstrap estimation of the standard error of the median

# Original sample: 50 gene expression values
let expression = rnorm(50, 3.0, 1.5) |> map(|x| exp(x))

let observed_median = median(expression)
print("Observed median: {observed_median:.2}")

# Bootstrap: resample with replacement 5000 times
let boot_medians = []
for i in 0..5000 {
  let resample = sample(expression, len(expression))
  boot_medians = boot_medians + [median(resample)]
}

# Bootstrap SE
let boot_se = stdev(boot_medians)
print("Bootstrap SE of median: {boot_se:.2}")

# Bootstrap 95% confidence interval (percentile method)
let ci_lower = quantile(boot_medians, 0.025)
let ci_upper = quantile(boot_medians, 0.975)
print("95% Bootstrap CI: [{ci_lower:.2}, {ci_upper:.2}]")

histogram(boot_medians, {bins: 50, title: "Bootstrap Distribution of
the Median"})
```

Key insight: The bootstrap treats your sample as a stand-in for the population. By resampling from your sample, you simulate what would happen if you could repeatedly sample from the population. It is remarkably effective even for small samples.

Hands-On: CLT with Allele Frequencies

Let us experience the Central Limit Theorem using realistic genetic data.

```

set_seed(42)
# Simulate allele frequency estimation from 1000 Genomes-style data

# True allele frequency of a common variant
let true_af = 0.23

# Simulate genotyping different numbers of individuals
let sample_sizes = [10, 30, 100, 500]

for n in sample_sizes {
  # Simulate 2000 studies, each genotyping n individuals
  let estimated_afs = []
  for study in 0..2000 {
    # Each individual contributes 2 alleles (diploid)
    let n_alleles = 2 * n
    let alt_count = rbinom(1, n_alleles, true_af) |> sum()
    let af_estimate = alt_count / n_alleles
    estimated_afs = estimated_afs + [af_estimate]
  }

  let se = stdev(estimated_afs)
  let theoretical_se = sqrt(true_af * (1.0 - true_af) / (2.0 * n))

  print("n={n}: SE={se:.4} (theoretical: {theoretical_se:.4})")
  histogram(estimated_afs, {bins: 40, title: "Allele Frequency
Estimates (n={n})"})
}

# With n=10: estimates range wildly (0.05 to 0.50)
# With n=500: estimates tightly clustered around 0.23

```

This simulation shows exactly why GWAS studies need thousands of individuals. With 10 people, you cannot reliably estimate an allele frequency to better than ± 10 percentage points. With 500, you can nail it to within ± 1 -2 percentage points.

Python and R Equivalents

Python:

```

import numpy as np
from scipy import stats

# Sampling distribution simulation
population = np.random.normal(125, 18, 100000)
sample_means = [np.mean(np.random.choice(population, 30)) for _ in
range(1000)]
print(f"SE: {np.std(sample_means):.2f}") # Should be close to
18/sqrt(30)

# Bootstrap
from scipy.stats import bootstrap
data = np.random.exponential(1, 50)
res = bootstrap((data,), np.median, n_resamples=5000)
print(f"95% CI: [{res.confidence_interval.low:.2f},
{res.confidence_interval.high:.2f}]")

# Standard error
se = np.std(data, ddof=1) / np.sqrt(len(data))

```

R:

```

# Sampling distribution
population <- rnorm(100000, mean = 125, sd = 18)
sample_means <- replicate(1000, mean(sample(population, 30)))
sd(sample_means) # Empirical SE

# Bootstrap
library(boot)
boot_fn <- function(data, indices) median(data[indices])
results <- boot(data, boot_fn, R = 5000)
boot.ci(results, type = "perc")

# Standard error
se <- sd(data) / sqrt(length(data))

# Central Limit Theorem demo
par(mfrow = c(2, 2))
for (n in c(5, 10, 30, 100)) {
  means <- replicate(2000, mean(rexp(n, rate = 1)))
  hist(means, breaks = 40, main = paste("n =", n))
}

```

Exercises

Exercise 1: Experience the CLT

Take a uniform distribution (flat, definitely not normal) and show the CLT in action.

```
set_seed(42)
# Uniform population: values equally likely between 0 and 100
let uniform_pop = rnorm(100000, 50, 28.87)
# (Approximation – true uniform has SD = range/sqrt(12))

# TODO: Draw histograms of the population (should be flat-ish)
# TODO: Take 1000 samples of size n=5, compute means, draw histogram
# TODO: Repeat for n=10, n=30, n=100
# TODO: At what n does the sampling distribution look convincingly
normal?
# TODO: Compute skewness at each n to quantify the convergence
```

Exercise 2: SE and Confidence

You measure tumor volumes in 25 mice (mean = 450 mm³, SD = 120 mm³).

```
let n = 25
let sample_mean = 450.0
let sample_sd = 120.0

# TODO: Compute the standard error
# TODO: Compute an approximate 95% CI using mean +/- 2*SE
# TODO: How large would n need to be for the 95% CI to have a width
of +/-10 mm3?
# TODO: How large for +/-5 mm3?
```

Exercise 3: Bias Identification

For each scenario, identify the type of bias and explain how it could affect results.

1. A study of BRCA1 mutation frequency recruits subjects from a cancer genetics clinic.
2. A survival analysis of pancreatic cancer uses patients diagnosed 5+ years ago (all long-term survivors by definition).
3. RNA-seq libraries are prepared on two different days — all treatment samples on Day 1, all controls on Day 2.
4. A GWAS consortium publishes results for the 10 strongest associations and files the rest.

Exercise 4: Bootstrap a Correlation

Estimate the uncertainty in a correlation coefficient using the bootstrap.

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# Gene expression vs. protein abundance (moderate correlation)
let n = 40
let gene_expr = rnorm(n, 5.0, 2.0)
let noise = rnorm(n, 0, 1.5)
let protein = gene_expr |> map(|x| 0.7 * x) |> zip(noise) |>
map(|pair| pair.0 + pair.1)

let observed_r = cor(gene_expr, protein)
print("Observed correlation: {observed_r:.3}")

# TODO: Bootstrap the correlation 5000 times
# TODO: Compute the 95% bootstrap CI
# TODO: Is the correlation significantly different from zero?
# TODO: Plot the bootstrap distribution of r
```

Exercise 5: Power Simulation

Explore how effect size and variability interact with sample size.

```
set_seed(42)
# TODO: Run the trial simulation from the chapter, but now vary the
# effect size
# Test with differences of 2, 5, 10, and 20 mmHg (SD=18 throughout)
# At n=50 per arm, which effect sizes can you reliably detect?
# TODO: Now fix the difference at 5 mmHg and vary SD (10, 18, 30)
# How does variability affect the required sample size?
```

Key Takeaways

- **Population vs. sample:** You study a sample to learn about a population. The quality of inference depends on sample size and sampling method.
- The **sampling distribution** is the distribution of a statistic computed from repeated samples. It is narrower than the data distribution and centered at the true value.
- The **Central Limit Theorem** guarantees that sample means are approximately normal regardless of the population distribution, given sufficient sample size. This is why normal-based tests work so broadly.
- **Standard error ($SE = SD/\sqrt{n}$)** quantifies the precision of your estimate. Quadrupling n halves the SE.
- **Bias** (selection, survivorship, ascertainment, measurement, publication) is a systematic distortion that cannot be fixed by increasing n . Identify and prevent bias at the design stage.
- **Statistical power** is the probability of detecting a real effect. Underpowered studies waste resources and miss real effects. The four determinants of power are effect size, sample size, variability, and significance threshold.
- The **bootstrap** provides empirical estimates of standard errors and confidence intervals for any statistic, without relying on distributional assumptions.

What's Next

You have now completed the foundations. You know how to summarize data (Day 2), understand its distributional shape (Day 3), reason about probabilities (Day 4), and appreciate the central role of sample size and sampling variability (Day 5). Starting next week, we put these foundations to work. Day 6 introduces **confidence intervals** — the formal framework for saying “I’m 95% sure the true value lies between here and here.” You will see how the standard error you learned today transforms into a rigorous statement about uncertainty, and why confidence intervals are more informative than p-values alone. The testing begins.

Day 6: Confidence Intervals — The Range of Truth

The Problem

Dr. Amara Chen's pharmacology team has spent six months developing a novel kinase inhibitor for triple-negative breast cancer. After extensive optimization, they measure the half-maximal inhibitory concentration (IC₅₀) across eight independent replicates: 11.2, 13.1, 12.8, 10.9, 14.2, 12.0, 11.7, and 12.5 nanomolar. The mean is 12.3 nM — an excellent result that would place their compound among the most potent in its class.

But when Dr. Chen presents these results to the medicinal chemistry team, the lead chemist asks the uncomfortable question: "If you ran the experiment again tomorrow, would you get 12.3 nM? Or could it be 15? Or 9?" The point estimate of 12.3 nM tells them where the center of their data is, but it says nothing about how *confident* they should be in that number. They need a range — a confidence interval — that captures the uncertainty inherent in measuring anything biological.

This chapter introduces the confidence interval: a range of plausible values for a population parameter, built from sample data. It is one of the most important and most misunderstood tools in all of biostatistics.

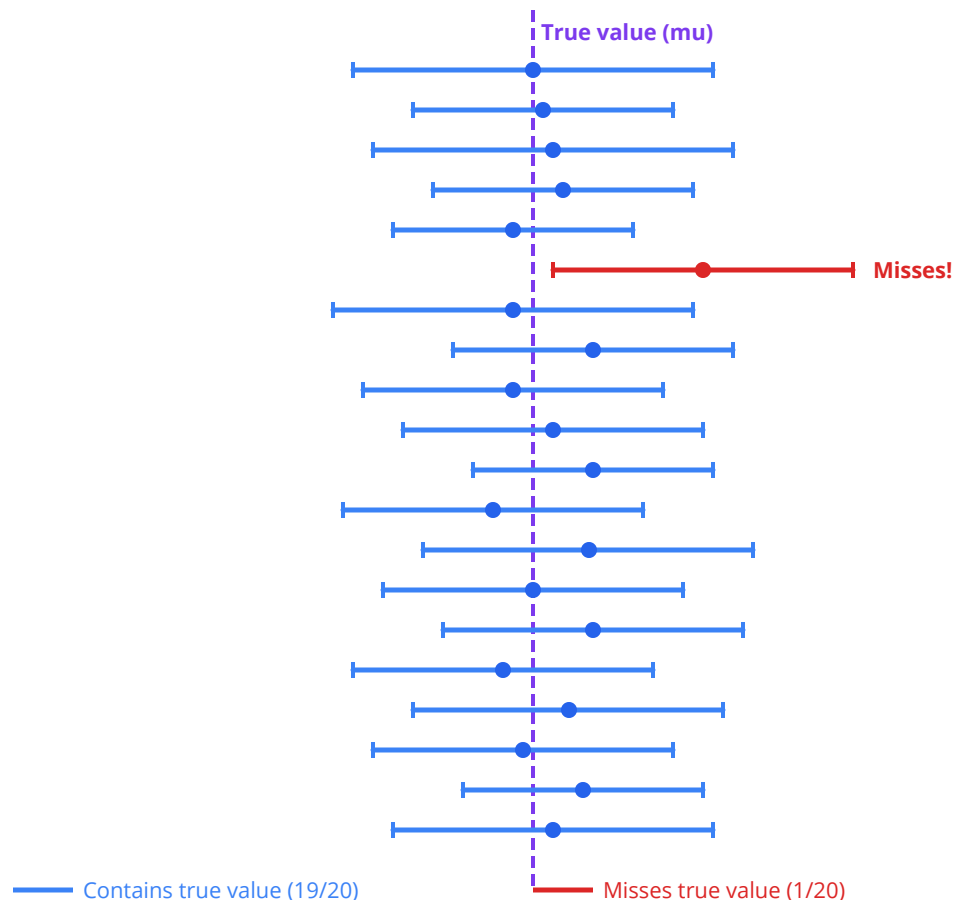
What Is a Confidence Interval?

Imagine you are trying to measure the height of a building, but your measuring tape is slightly stretchy. Each time you measure, you get a slightly different answer. A confidence interval is like saying: "Based on my eight measurements, I am 95% confident the true height is somewhere between 48.2 and 52.1 meters."

More precisely: if you repeated your experiment 100 times and computed a 95% confidence interval each time, about 95 of those 100 intervals would contain the true population parameter. The remaining 5 would miss it entirely.

20 Confidence Intervals from Repeated Experiments

~19 capture the true parameter (blue), ~1 misses it (red)



Common pitfall: A 95% CI does NOT mean “there is a 95% probability the true value is in this interval.” Once you compute a specific interval, the true value is either in it or it isn’t. The 95% refers to the *procedure’s* long-run success rate, not the probability for any single interval.

Point Estimates Are Not Enough

A point estimate is a single number — a sample mean, a proportion, a median. It is our best guess, but it carries no information about precision.

Scenario	Point Estimate	What's Missing?
Drug IC50 from 8 replicates	12.3 nM	Could be 8-16 nM or 11.9-12.7 nM
Mutation frequency in 50 patients	34%	Could be 21-47% or 30-38%
Mean tumor volume after treatment	180 mm ³	How variable was the response?

The confidence interval supplements the point estimate with a measure of uncertainty. Narrow intervals mean precise estimates; wide intervals mean the data leaves much room for doubt.

CI for a Mean: $\bar{x}$ plus-or-minus t times SE

The most common confidence interval is for a population mean. The formula is:

$$CI = \bar{x} \pm t(\alpha/2, df) \times SE$$

Where:

- **$\bar{x}$** is the sample mean
- **$SE = s / \sqrt{n}$** is the standard error of the mean
- **$t(\alpha/2, df)$** is the critical value from the t-distribution with $df = n - 1$
- **α** = 1 - confidence level (for 95% CI, $\alpha = 0.05$)

Why the t-Distribution for Small Samples?

When n is large (say, $n > 30$), the t-distribution closely resembles the normal distribution. But for small n — common in biology where each replicate is expensive — the t-distribution has heavier tails, producing wider intervals that honestly reflect our greater uncertainty.

Sample Size (n)	t-critical (95%)	z-critical (95%)	Difference
5	2.776	1.960	42% wider
10	2.262	1.960	15% wider
30	2.045	1.960	4% wider
100	1.984	1.960	~1% wider
1000	1.962	1.960	Negligible

Key insight: For biological experiments with $n < 30$, always use the t-distribution. Using z would give falsely narrow intervals that overstate your precision.

CI for a Proportion

When the variable is binary — mutation present/absent, responder/non-responder — we need a CI for a proportion $\hat{p} = x/n$.

Wald Interval (Simple but Flawed)

$$CI = \hat{p} \pm z \times \sqrt{\hat{p}(1 - \hat{p}) / n}$$

This is the textbook formula, but it performs poorly when p is near 0 or 1, or when n is small. It can even produce intervals that extend below 0 or above 1.

Wilson Interval (Preferred)

The Wilson score interval adjusts the center and width, and is recommended for most biological applications:

$$CI = (\hat{p} + \frac{z^2}{2n} \pm z \times \sqrt{\hat{p}(1-\hat{p})/n + \frac{z^2}{4n^2}}) / (1 + \frac{z^2}{n})$$

Clinical relevance: When reporting mutation carrier frequencies, drug response rates, or diagnostic sensitivity/specificity, always use Wilson intervals. Regulatory agencies expect intervals that behave properly even at extreme proportions.

CI for the Difference Between Two Means

Often the real question is not “what is the mean?” but “how much do two groups differ?” The CI for the difference between two independent means is:

$$CI = (\bar{x}_1 - \bar{x}_2) \pm t \times SE_{diff}$$

Where $SE_{diff} = \sqrt{s_1^2/n_1 + s_2^2/n_2}$ for Welch’s approach.

The critical interpretation: If the CI for the difference includes zero, the data are consistent with no difference between the groups. If it excludes zero, the difference is statistically significant.

CI for Difference	Interpretation
[1.2, 4.8]	Groups differ; difference is between 1.2 and 4.8 units
[-0.5, 3.1]	Includes zero; cannot rule out no difference
[-4.2, -1.1]	Groups differ; group 2 is higher by 1.1 to 4.2 units

Bootstrap Confidence Intervals

What if your statistic is a median, a ratio, or something with no tidy formula? The **bootstrap** is a computer-intensive method that works for *any* statistic:

1. Resample your data **with replacement**, same size as original
2. Compute the statistic on the resample
3. Repeat 10,000 times
4. Take the 2.5th and 97.5th percentiles of the bootstrap distribution

This is called the **percentile method**. No assumptions about normality or distribution shape are required.

Key insight: Bootstrap CIs are the Swiss army knife of interval estimation. When in doubt, bootstrap it.

What Controls CI Width?

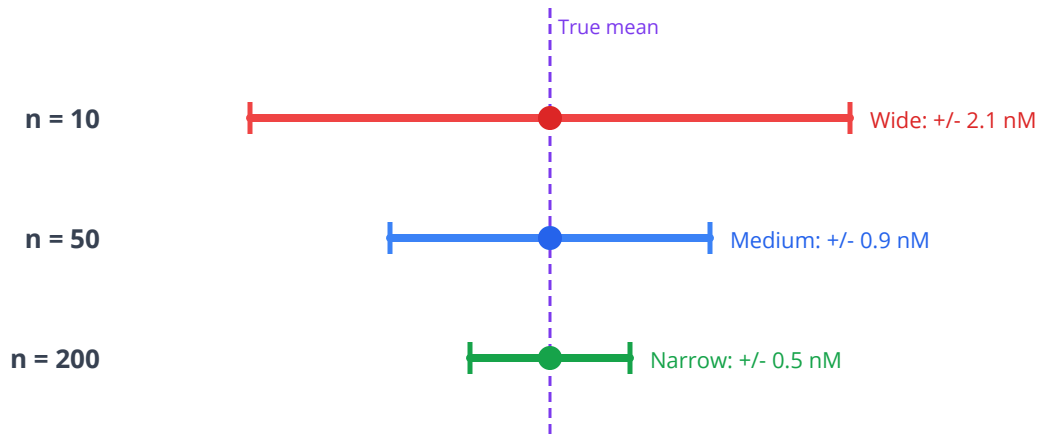
Three factors determine how wide or narrow your confidence interval will be:

Factor	Effect on Width	Biological Implication
Sample size (n)	Width ~ $1/\sqrt{n}$	Doubling n cuts width by ~30%
Variability (s)	Width ~ s	High biological variability = wider CIs
Confidence level	99% > 95% > 90%	Higher confidence = wider interval

This is why power calculations matter: before an experiment, you choose n to achieve a CI narrow enough to be scientifically useful.

CI Width Narrows with Larger Sample Size

Same population, same true mean -- only n changes



Width ~ 1/sqrt(n): doubling n cuts width by ~30%, quadrupling cuts by ~50%

Confidence Intervals in BioLang

IC50 Confidence Interval — Parametric

```
# IC50 measurements (nM) from 8 replicates
let ic50 = [11.2, 13.1, 12.8, 10.9, 14.2, 12.0, 11.7, 12.5]

let n = len(ic50)
let x_bar = mean(ic50)
let se = stdev(ic50) / sqrt(n)

# 95% CI using normal approximation (for small n, t > z)
let t_crit = qnorm(0.975)
let ci_lower = x_bar - t_crit * se
let ci_upper = x_bar + t_crit * se

print("IC50 mean: {x_bar:.2} nM")
print("95% CI: [{ci_lower:.2}, {ci_upper:.2}] nM")
print("Standard error: {se:.3} nM")
print("Critical value: {t_crit:.3}")
```

IC50 Confidence Interval — Bootstrap

```
set_seed(42)
# Bootstrap CI: no distributional assumptions

let ic50 = [11.2, 13.1, 12.8, 10.9, 14.2, 12.0, 11.7, 12.5]

# Bootstrap: resample 10,000 times, compute mean each time
let n_boot = 10000
let boot_means = []
for i in range(0, n_boot) {
  let resample = []
  for j in range(0, len(ic50)) {
    resample = append(resample, ic50[random_int(0, len(ic50) -
1)])
  }
  boot_means = append(boot_means, mean(resample))
}

# Percentile method
let boot_lower = quantile(boot_means, 0.025)
let boot_upper = quantile(boot_means, 0.975)

print("Bootstrap 95% CI: [{boot_lower:.2}, {boot_upper:.2}] nM")

# Visualize the bootstrap distribution
histogram(boot_means, {bins: 50, title: "Bootstrap Distribution of
IC50 Mean", x_label: "Mean IC50 (nM)"})
```

Bootstrap CI for Median (No Parametric Formula Exists)

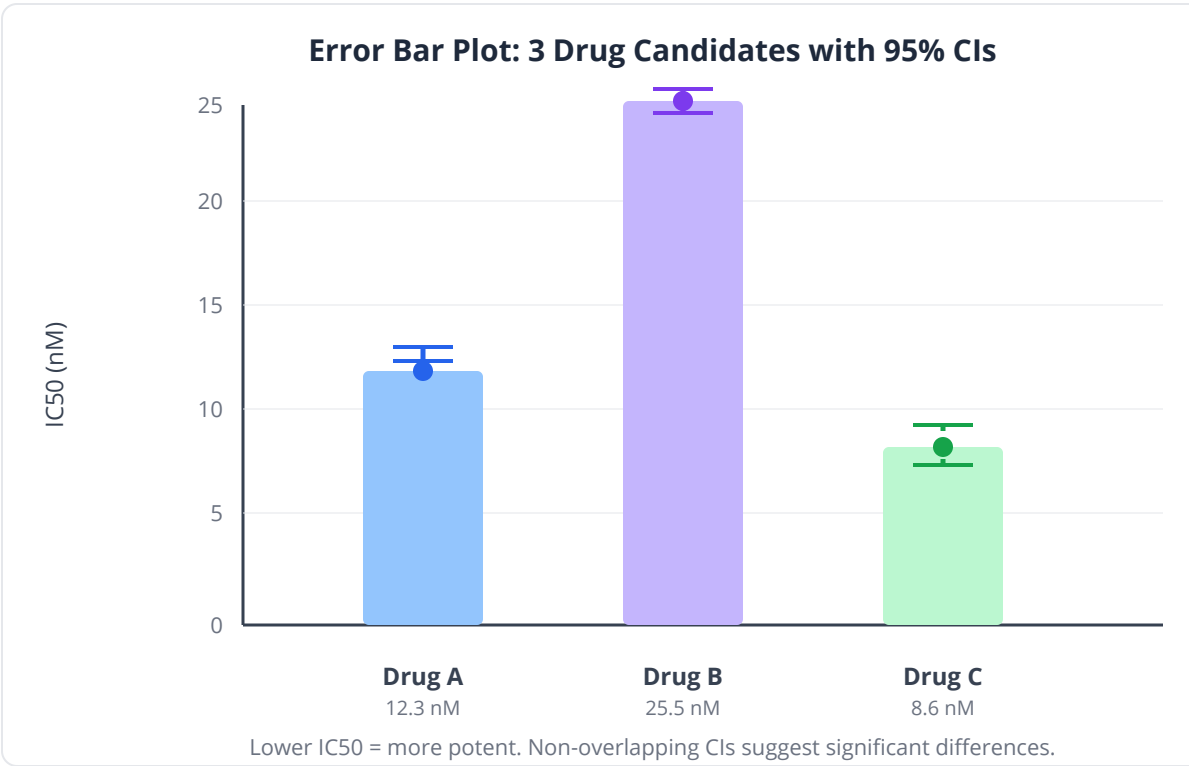
```
set_seed(42)
# Gene expression values (FPKM) – skewed distribution
let expression = [0.1, 0.3, 0.8, 1.2, 1.5, 2.1, 3.4, 8.7, 12.1,
45.6]

let obs_median = median(expression)

# Bootstrap the median
let n_boot = 10000
let boot_medians = []
for i in range(0, n_boot) {
  let resample = []
  for j in range(0, len(expression)) {
    resample = append(resample, expression[random_int(0,
len(expression) - 1)])
  }
  boot_medians = append(boot_medians, median(resample))
}
let ci_lower = quantile(boot_medians, 0.025)
let ci_upper = quantile(boot_medians, 0.975)

print("Observed median: {obs_median:.2} FPKM")
print("Bootstrap 95% CI for median: [{ci_lower:.2}, {ci_upper:.2}]
FPKM")
```

Error Bar Plot: Comparing Drug Concentrations



```
# IC50 values for three drug candidates
let drug_a = [12.3, 11.8, 13.1, 12.0, 11.5, 12.7, 13.4, 12.1]
let drug_b = [25.1, 28.3, 22.7, 26.9, 24.5, 27.1, 23.8, 25.6]
let drug_c = [8.2, 9.1, 7.5, 8.8, 10.2, 8.0, 9.5, 7.8]

let drugs = ["Drug A", "Drug B", "Drug C"]
let means = [mean(drug_a), mean(drug_b), mean(drug_c)]

# Compute 95% CIs for each
let compute_ci = |data| {
  let n = len(data)
  let se = stdev(data) / sqrt(n)
  let t_crit = qnorm(0.975)
  [mean(data) - t_crit * se, mean(data) + t_crit * se]
}

let ci_a = compute_ci(drug_a)
let ci_b = compute_ci(drug_b)
let ci_c = compute_ci(drug_c)

print("Drug A: {means[0]:.1} nM, 95% CI [{ci_a[0]:.1},
{ci_a[1]:.1}]")
print("Drug B: {means[1]:.1} nM, 95% CI [{ci_b[0]:.1},
{ci_b[1]:.1}]")
print("Drug C: {means[2]:.1} nM, 95% CI [{ci_c[0]:.1},
{ci_c[1]:.1}]")

# Bar chart with error bars
let ci_chart = table({"drug": drugs, "mean_ic50": means})
bar_chart(ci_chart, {label: "drug", value: "mean_ic50"})
```

CI for Difference Between Two Means

```
# Compare tumor volume between treated and control mice
let treated = [180, 210, 165, 225, 195, 172, 218, 198]
let control = [485, 512, 468, 530, 495, 478, 521, 503]

let diff = mean(treated) - mean(control)
let se_diff = sqrt(variance(treated) / len(treated) +
variance(control) / len(control))
let df = len(treated) + len(control) - 2
let t_crit = qnorm(0.975) # approximate for moderate df

let ci_lower = diff - t_crit * se_diff
let ci_upper = diff + t_crit * se_diff

print("Mean difference: {diff:.1} mm^3")
print("95% CI for difference: [{ci_lower:.1}, {ci_upper:.1}] mm^3")

if ci_upper < 0 {
  print("CI excludes zero: treatment significantly reduces tumor
volume")
} else {
  print("CI includes zero: cannot rule out no difference")
}
```

Vaccine Efficacy CI (Proportion)

```
set_seed(42)
# Clinical trial: 15 of 200 vaccinated got infected vs 60 of 200
# placebo
let p_vacc = 15 / 200
let p_plac = 60 / 200
let efficacy = 1.0 - (p_vacc / p_plac)

print("Vaccine efficacy: {efficacy * 100:.1}%")

# CI for proportion (vaccinated group infection rate)
let n = 200
let z = 1.96
let se_p = sqrt(p_vacc * (1.0 - p_vacc) / n)
let ci_lower_p = p_vacc - z * se_p
let ci_upper_p = p_vacc + z * se_p

print("Infection rate (vaccinated): {p_vacc*100:.1}%")
print("95% CI for infection rate: [{ci_lower_p*100:.1}%,
{ci_upper_p*100:.1}%]")

# Bootstrap CI for vaccine efficacy itself
let vacc_outcomes = flatten([repeat(1, 15), repeat(0, 185)])
let plac_outcomes = flatten([repeat(1, 60), repeat(0, 140)])

let n_boot = 10000
let boot_eff = []
for i in range(0, n_boot) {
  let v_resample = []
  let p_resample = []
  for j in range(0, len(vacc_outcomes)) {
    v_resample = append(v_resample, vacc_outcomes[random_int(0,
len(vacc_outcomes) - 1)])
    p_resample = append(p_resample, plac_outcomes[random_int(0,
len(plac_outcomes) - 1)])
  }
  let pv = mean(v_resample)
  let pp = mean(p_resample)
  let eff = if pp == 0.0 then 0.0 else 1.0 - (pv / pp)
  boot_eff = append(boot_eff, eff)
}

let eff_ci = [quantile(boot_eff, 0.025), quantile(boot_eff, 0.975)]
```

```
print("Bootstrap 95% CI for efficacy: [{eff_ci[0]*100:.1}%,  
{eff_ci[1]*100:.1}%]")
```

Python:

```
import numpy as np  
from scipy import stats  
  
ic50 = [11.2, 13.1, 12.8, 10.9, 14.2, 12.0, 11.7, 12.5]  
ci = stats.t.interval(0.95, df=len(ic50)-1,  
                      loc=np.mean(ic50),  
                      scale=stats.sem(ic50))  
print(f"95% CI: [{ci[0]:.2f}, {ci[1]:.2f}]")  
  
# Bootstrap  
boot = [np.mean(np.random.choice(ic50, len(ic50))) for _ in  
range(10000)]  
print(f"Bootstrap CI: [{np.percentile(boot, 2.5):.2f},  
{np.percentile(boot, 97.5):.2f}]")
```

R:

```
ic50 <- c(11.2, 13.1, 12.8, 10.9, 14.2, 12.0, 11.7, 12.5)  
t.test(ic50)$conf.int  
  
# Bootstrap  
library(boot)  
boot_fn <- function(data, i) mean(data[i])  
b <- boot(ic50, boot_fn, R = 10000)  
boot.ci(b, type = "perc")
```

Exercises

Exercise 1: Compute a CI by Hand

Eight mice on a high-fat diet had cholesterol levels: 215, 228, 197, 241, 209, 233, 220, 212 mg/dL. Compute the 95% CI for the mean cholesterol.

```
let cholesterol = [215, 228, 197, 241, 209, 233, 220, 212]

# TODO: Compute mean, SE, critical value, and the 95% CI
# Hint: df = n - 1, use qnorm(0.975) as approximate critical value
```

Exercise 2: Bootstrap a Ratio

Gene A has FPKM values [2.1, 3.4, 1.8, 4.2, 2.9] in tumor and [1.0, 1.2, 0.9, 1.5, 1.1] in normal. Bootstrap a 95% CI for the tumor/normal fold change of medians.

```
let tumor = [2.1, 3.4, 1.8, 4.2, 2.9]
let normal = [1.0, 1.2, 0.9, 1.5, 1.1]

# TODO: Bootstrap the ratio median(tumor) / median(normal)
# Use n_boot = 10000, then extract 2.5th and 97.5th percentiles with
quantile()
```

Exercise 3: Overlapping CIs

Compute 95% CIs for Drug X (IC50: [5.2, 6.1, 4.8, 5.5, 6.3, 5.0]) and Drug Y (IC50: [5.8, 6.5, 7.2, 6.0, 5.9, 6.8]). Do the CIs overlap? What does this suggest?

```
let drug_x = [5.2, 6.1, 4.8, 5.5, 6.3, 5.0]
let drug_y = [5.8, 6.5, 7.2, 6.0, 5.9, 6.8]

# TODO: Compute CIs for both, then compute CI for the difference
# Note: overlapping CIs do NOT necessarily mean non-significant
difference
```

Exercise 4: Effect of Sample Size

Starting with n = 5 replicates drawn from the IC50 data, increase to n = 10, 20, 50, and 100 (use bootstrap resampling to simulate larger samples). Plot CI width vs sample size.

```
let ic50 = [11.2, 13.1, 12.8, 10.9, 14.2, 12.0, 11.7, 12.5]

# TODO: For each sample size, bootstrap to simulate, compute CI
width
# Plot sample size vs CI width using line_plot
```

Key Takeaways

- A **confidence interval** gives a range of plausible values for a population parameter, not just a point estimate
- The 95% in “95% CI” refers to the long-run coverage rate of the procedure, not the probability for a specific interval
- For small samples ($n < 30$), always use the **t-distribution** — it accounts for extra uncertainty
- **Bootstrap CIs** work for any statistic (median, ratio, fold change) without distributional assumptions
- CI width shrinks with larger n , lower variability, and lower confidence level
- A CI for the difference that **includes zero** means the data are consistent with no difference
- CIs are more informative than p-values alone: they tell you both significance AND the plausible magnitude of an effect

What's Next

Tomorrow we formalize the logic behind “ruling out chance” with hypothesis testing. You will learn to frame biological questions as null and alternative hypotheses, compute p-values, and understand the courtroom analogy that makes the whole framework click. Confidence intervals and hypothesis tests are two sides of the same coin — a 95% CI that excludes zero corresponds exactly to a p-value less than 0.05.

Day 7: Hypothesis Testing — Asking Precise Questions

The Problem

Dr. Kenji Nakamura has spent three years developing a blood-based biomarker panel for early Alzheimer's detection. His team measures plasma levels of phosphorylated tau (p-tau217) in 40 cognitively normal individuals and 40 patients with confirmed early-stage Alzheimer's. The mean p-tau217 level in the Alzheimer's group is 3.8 pg/mL, compared to 2.9 pg/mL in controls. The difference looks promising — nearly 30% higher.

But when Dr. Nakamura submits to the FDA for breakthrough device designation, the reviewer's response is blunt: "Your biomarker shows a numerical difference. Can you demonstrate this isn't just sampling noise? What is the probability of seeing a difference this large if the biomarker has no real diagnostic value?" This is the fundamental question that hypothesis testing answers.

The stakes are enormous. If the biomarker works, millions of patients could be diagnosed years earlier, when interventions are most effective. If it doesn't — if the observed difference is just statistical noise — pursuing it wastes hundreds of millions in development costs and, worse, could lead to false diagnoses.

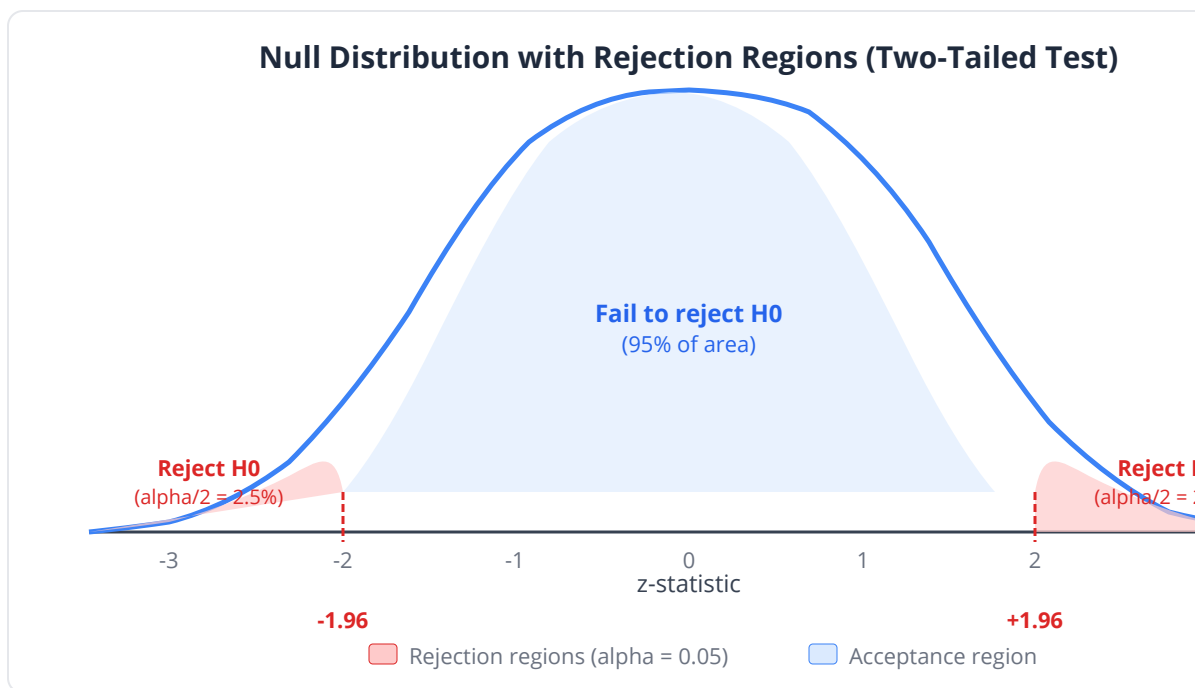
What Is Hypothesis Testing?

Think of hypothesis testing as a courtroom trial for your scientific claim.

- The **defendant** is the null hypothesis (H_0): "There is no effect." In the courtroom, the defendant is presumed innocent.

- The **prosecution's evidence** is your data. You are trying to show the evidence is so overwhelming that the “innocence” explanation is implausible.
- The **verdict** is either “guilty” (reject H_0) or “not proven” (fail to reject H_0). Notice: the jury never declares the defendant “innocent” — just that the evidence was insufficient.

Key insight: Hypothesis testing never proves your theory is true. It only tells you whether the data are inconsistent enough with “no effect” that you can reject that explanation with a specified level of confidence.



The Five Steps of Hypothesis Testing

Step	Description	Alzheimer's Example
1. State H0 and H1	Define the null and alternative	H0: $\mu_{AD} = \mu_{control}$; H1: $\mu_{AD} > \mu_{control}$
2. Choose alpha	Set significance threshold	$\alpha = 0.05$
3. Compute test statistic	Summarize evidence against H0	$z = (\bar{x}_1 - \bar{x}_2) / SE$
4. Find p-value	Probability of seeing this extreme a result under H0	$p = P(Z \geq z_{obs})$
5. Make decision	Compare p to alpha	If $p < 0.05$, reject H0

The Null Hypothesis (H0)

The null hypothesis is the “boring” explanation — the default assumption of no effect, no difference, no relationship. It is what you assume until the data force you to abandon it.

Research Question	Null Hypothesis (H0)
Does the drug reduce tumor size?	Mean tumor size is the same with and without drug
Is this SNP associated with diabetes?	Allele frequencies are the same in cases and controls
Does expression differ between tissues?	Mean expression is equal in both tissues
Is the mutation rate elevated?	Mutation rate equals the background rate

The Alternative Hypothesis (H1)

The alternative is what you actually believe — the “interesting” claim.

- **Two-tailed:** $H_1: \mu_1 \neq \mu_2$ (the groups differ in either direction)
- **One-tailed:** $H_1: \mu_1 > \mu_2$ (specifically higher) or $H_1: \mu_1 < \mu_2$ (specifically lower)

Common pitfall: Do not choose one-tailed vs two-tailed after looking at your data. This decision must be made before the experiment, based on your scientific question. Switching from two-tailed to one-tailed after seeing results halves your p-value — that is scientific fraud.

The p-Value: Most Misunderstood Number in Science

The p-value is the probability of observing data as extreme as (or more extreme than) what you got, **assuming H0 is true**.

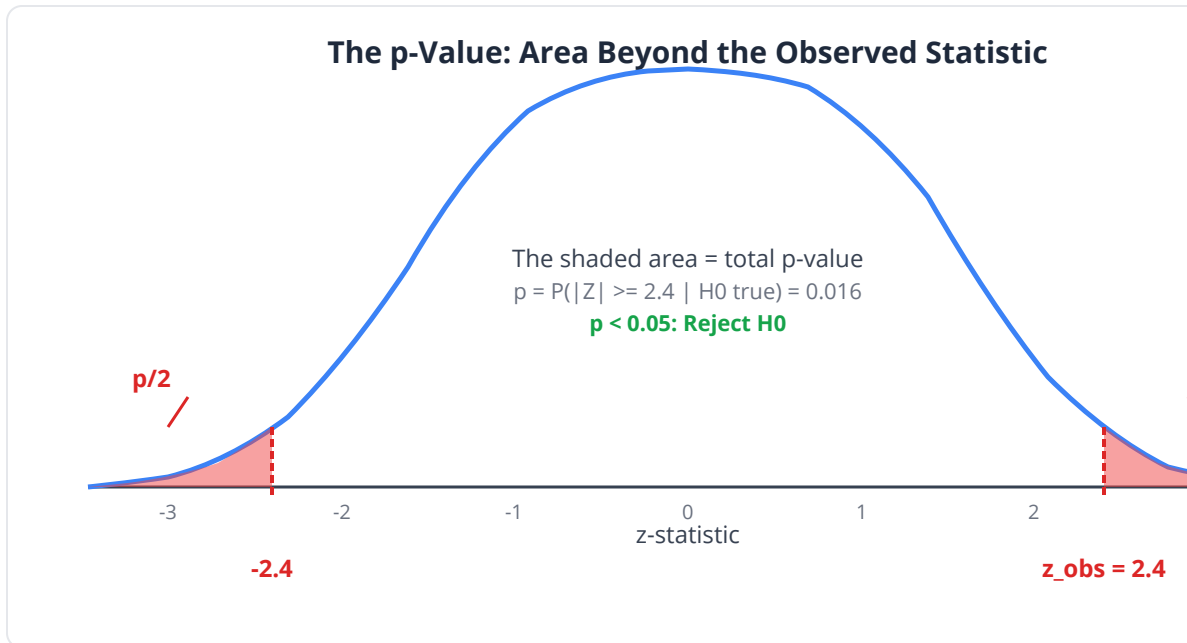
What the p-value IS:

- A measure of how surprising your data are under the null hypothesis
- A continuous measure of evidence — smaller p = more evidence against H0
- The probability of the data given H0: $P(\text{data} \mid H_0)$

What the p-value IS NOT:

- The probability that H0 is true: NOT $P(H_0 \mid \text{data})$
- The probability your result is due to chance
- The probability of making an error

- A measure of effect size or practical importance



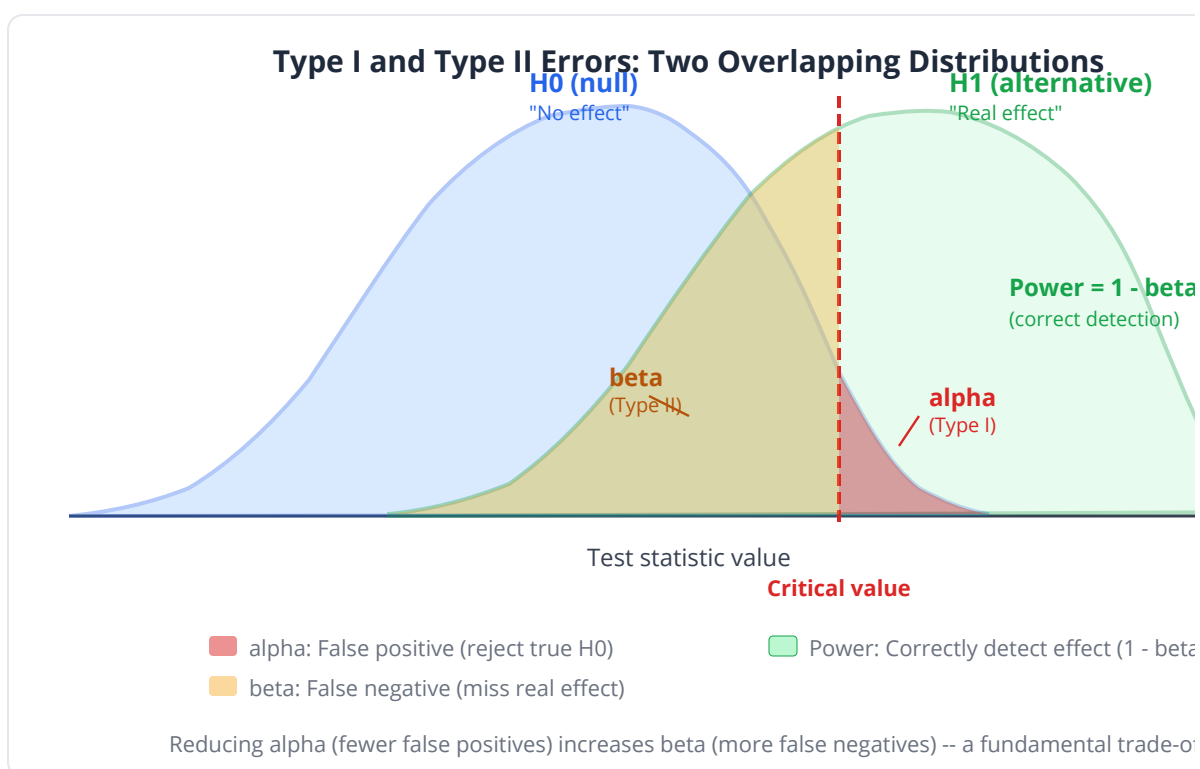
p-value	Informal Interpretation
$p > 0.10$	Little evidence against H_0
$0.05 < p < 0.10$	Weak evidence against H_0
$0.01 < p < 0.05$	Moderate evidence against H_0
$0.001 < p < 0.01$	Strong evidence against H_0
$p < 0.001$	Very strong evidence against H_0

Type I and Type II Errors

Every decision carries the risk of being wrong:

	H0 is True	H0 is False
Reject H0	Type I error (false positive), probability = α	Correct (true positive), probability = $1 - \beta$ = power
Fail to reject H0	Correct (true negative)	Type II error (false negative), probability = β

- **Type I error (alpha):** You claim a drug works when it doesn't. A patient receives ineffective treatment.
- **Type II error (beta):** You miss a real effect. An effective drug gets shelved.



Clinical relevance: In drug safety testing, α is typically set very low (0.01 or even 0.001) because a Type I error means approving a dangerous drug. In exploratory genomics, higher α (0.05 or even 0.10) is acceptable because you will validate hits in follow-up experiments.

Statistical vs Practical Significance

A p-value of 0.001 does not mean the effect is large or important. With enough data, trivially small effects become “statistically significant.”

Scenario	p-value	Effect Size	Verdict
Gene expression differs by 0.01% (n=100,000)	$p < 0.001$	Negligible	Statistically significant, practically meaningless
Drug reduces tumor by 40% (n=12)	$p = 0.03$	Large	Both statistically and practically significant
Biomarker differs by 5% (n=20)	$p = 0.08$	Moderate	Not significant — but maybe underpowered

Always report effect sizes alongside p-values. We will dedicate Day 19 entirely to this topic.

The z-Test: The Simplest Hypothesis Test

When the population standard deviation σ is known (rare, but a good starting point), the z-test compares a sample mean to a known value:

$$z = (\bar{x} - \mu_0) / (\sigma / \sqrt{n})$$

Under H_0 , z follows a standard normal distribution $N(0, 1)$.

One-Tailed vs Two-Tailed Tests

Test Type	H1	p-value Calculation	Use When
Two-tailed	$\mu \neq \mu_0$	$2 \times P(Z > \text{abs}(z))$	You care about differences in either direction
Right-tailed	$\mu > \mu_0$	$P(Z > z)$	You only care if the value is higher
Left-tailed	$\mu < \mu_0$	$P(Z < z)$	You only care if the value is lower

Hypothesis Testing in BioLang

z-Test on Biomarker Data

```
# Alzheimer's biomarker study
# Known population SD from large reference database: sigma = 1.2
pg/mL
# Expected normal level: mu0 = 2.9 pg/mL
# Observed in 40 Alzheimer's patients:
let ad_levels = [3.2, 4.1, 3.8, 2.7, 4.5, 3.3, 3.9, 4.2, 3.1, 3.6,
                 4.0, 3.5, 4.3, 3.7, 2.9, 3.8, 4.1, 3.4, 3.6, 4.4,
                 3.0, 3.9, 4.2, 3.3, 3.7, 4.0, 3.5, 3.8, 4.1, 3.2,
                 3.6, 3.4, 4.3, 3.1, 3.9, 3.7, 4.0, 3.5, 3.8, 3.3]

# Compute z-test manually (known sigma)
let n = len(ad_levels)
let x_bar = mean(ad_levels)
let se = 1.2 / sqrt(n)
let z_stat = (x_bar - 2.9) / se
let p_val = 2.0 * (1.0 - pnorm(abs(z_stat), 0, 1))

print("z-statistic: {z_stat:.4}")
print("p-value (two-tailed): {p_val:.6}")
print("Mean observed: {x_bar:.3} pg/mL")

if p_val < 0.05 {
  print("Reject H0: Alzheimer's group significantly differs from
normal level")
} else {
  print("Fail to reject H0: Insufficient evidence of difference")
}
```


Visualizing the Null Distribution

```
# Show where our test statistic falls on the null distribution
let z_obs = 4.71 # from the z-test above

# Generate the null distribution (standard normal)
let x_vals = range(-4.0, 4.0, 0.01)
let y_vals = x_vals |> map(|x| dnorm(x, 0, 1))

# Plot the null distribution with our observed z marked
density(x_vals, {title: "Null Distribution (Standard Normal)",
x_label: "z-statistic", y_label: "Density", vlines: [z_obs],
shade_above: 1.96, shade_below: -1.96})

print("Critical value (two-tailed, alpha=0.05): +/- 1.96")
print("Our z = {z_obs} falls far in the rejection region")
```

Binomial Test: Is This Mutation Rate Elevated?

```
# In a cancer cohort, 18 of 100 patients carry a specific BRCA2
variant
# Population frequency is known to be 8%
# Compute binomial test using dbinom:  $P(X \geq 18)$  when  $X \sim \text{Binom}(100, 0.08)$ 
let p_val = 0.0
for k in range(18, 101) {
  p_val = p_val + dbinom(k, 100, 0.08)
}

print("Observed proportion: 18/100 = 18%")
print("Expected under H0: 8%")
print("p-value (one-sided): {p_val:.6}")

if p_val < 0.05 {
  print("Reject H0: Mutation rate is significantly elevated in this
cohort")
} else {
  print("Fail to reject H0")
}
```

Complete Hypothesis Test Workflow

```
# Full workflow: Is mean platelet count elevated in a disease cohort?
# Reference population:  $\mu = 250$  ( $\times 10^3/\text{uL}$ ),  $\sigma = 50$ 
# Our 25 patients:
let platelets = [280, 310, 265, 295, 275, 320, 290, 305, 260, 285,
                 300, 270, 315, 288, 292, 278, 308, 282, 298, 272,
                 310, 295, 268, 302, 288]

# Step 1: State hypotheses
print("H0:  $\mu = 250$  (platelet count is normal)")
print("H1:  $\mu > 250$  (platelet count is elevated)")
print("alpha = 0.05, one-tailed test\n")

# Step 2: Compute test statistic
let n = len(platelets)
let x_bar = mean(platelets)
let se = 50 / sqrt(n) # sigma is known
let z = (x_bar - 250) / se

# Step 3: Find p-value (one-tailed)
let p = 1.0 - pnorm(z, 0, 1)

# Step 4: Decision
print("Sample mean: {x_bar:.1}")
print("z-statistic: {z:.4}")
print("p-value (one-tailed): {p:.6}")

if p < 0.05 {
  print("\nDecision: Reject H0 at alpha = 0.05")
  print("Conclusion: Platelet count is significantly elevated in this cohort")
} else {
  print("\nDecision: Fail to reject H0")
}
```

Interpreting p-Values with Simulated Data

```
set_seed(42)
# Demonstrate: under H0 (no effect), p-values are uniformly
distributed

let p_values = []
for i in 1..1000 {
  # Generate two samples from the SAME distribution (H0 is true)
  let group1 = rnorm(20, 10, 3)
  let group2 = rnorm(20, 10, 3)
  let z_stat = (mean(group1) - mean(group2)) / (3.0 / sqrt(20))
  let p_val = 2.0 * (1.0 - pnorm(abs(z_stat), 0, 1))
  p_values = append(p_values, p_val)
}

# Count false positives at alpha = 0.05
let false_pos = p_values |> filter(|p| p < 0.05) |> len()
print("False positives out of 1000 null tests: {false_pos}")
print("Expected: ~50 (5% of 1000)")

histogram(p_values, {title: "p-Value Distribution Under the Null",
x_label: "p-value", bins: 20})
```

Connecting CIs and Hypothesis Tests

```
# Demonstrate: a 95% CI that excludes the null value corresponds to
p < 0.05
let ad_levels = [3.2, 4.1, 3.8, 2.7, 4.5, 3.3, 3.9, 4.2, 3.1, 3.6,
                 4.0, 3.5, 4.3, 3.7, 2.9, 3.8, 4.1, 3.4, 3.6, 4.4]

# z-test: is the mean different from 2.9?
let n = len(ad_levels)
let x_bar = mean(ad_levels)
let se = 1.2 / sqrt(n)
let z_stat = (x_bar - 2.9) / se
let z_p = 2.0 * (1.0 - pnorm(abs(z_stat), 0, 1))
print("z-test p-value: {z_p:.6}")

# 95% CI for the mean (using known sigma)
let se2 = 1.2 / sqrt(n)
let ci_lower = x_bar - 1.96 * se2
let ci_upper = x_bar + 1.96 * se2
print("95% CI: [{ci_lower:.3}, {ci_upper:.3}]")
print("Null value (2.9) is {'outside' if ci_lower > 2.9 or ci_upper
< 2.9 else 'inside'} the CI")
print("This matches the hypothesis test: p < 0.05 <=> CI excludes
null value")
```

Python:

```
import numpy as np
from scipy import stats

ad_levels = [3.2, 4.1, 3.8, 2.7, 4.5, 3.3, 3.9, 4.2, 3.1, 3.6,
             4.0, 3.5, 4.3, 3.7, 2.9, 3.8, 4.1, 3.4, 3.6, 4.4,
             3.0, 3.9, 4.2, 3.3, 3.7, 4.0, 3.5, 3.8, 4.1, 3.2,
             3.6, 3.4, 4.3, 3.1, 3.9, 3.7, 4.0, 3.5, 3.8, 3.3]

# z-test (manual – scipy doesn't have a built-in z-test for means)
z = (np.mean(ad_levels) - 2.9) / (1.2 / np.sqrt(len(ad_levels)))
p = 2 * (1 - stats.norm.cdf(abs(z)))
print(f"z = {z:.4f}, p = {p:.6f}")

# Binomial test
result = stats.binomtest(18, 100, 0.08, alternative='greater')
print(f"Binomial test p = {result.pvalue:.6f}")
```

R:

```
ad_levels <- c(3.2, 4.1, 3.8, 2.7, 4.5, 3.3, 3.9, 4.2, 3.1, 3.6,
              4.0, 3.5, 4.3, 3.7, 2.9, 3.8, 4.1, 3.4, 3.6, 4.4,
              3.0, 3.9, 4.2, 3.3, 3.7, 4.0, 3.5, 3.8, 4.1, 3.2,
              3.6, 3.4, 4.3, 3.1, 3.9, 3.7, 4.0, 3.5, 3.8, 3.3)

# z-test (using BSDA package, or manual)
z <- (mean(ad_levels) - 2.9) / (1.2 / sqrt(length(ad_levels)))
p <- 2 * pnorm(-abs(z))
cat(sprintf("z = %.4f, p = %.6f\n", z, p))

# Binomial test
binom.test(18, 100, p = 0.08, alternative = "greater")
```

Exercises

Exercise 1: Formulate Hypotheses

For each scenario, write the null and alternative hypotheses. State whether you would use a one-tailed or two-tailed test and why.

- a) Does a new antibiotic reduce bacterial colony counts compared to placebo?
- b) Is the GC content of a newly sequenced genome different from the expected 42%?
- c) Do patients with the variant allele have higher LDL cholesterol?

Exercise 2: z-Test on Gene Expression

A reference database reports the mean expression of housekeeping gene GAPDH as 8.5 log₂-CPM with sigma = 0.8 across thousands of samples. Your RNA-seq experiment on 15 samples yields a mean of 7.9. Is your experiment's GAPDH level significantly different?

```
let gapdh_expression = [7.5, 8.1, 7.8, 7.6, 8.3, 7.2, 8.0, 7.9,
                       8.2, 7.4, 7.7, 8.1, 7.6, 8.4, 7.3]

# TODO: Perform z-test with mu=8.5, sigma=0.8
# TODO: Interpret the result – what might explain a difference?
```

Exercise 3: Simulate Type I Error Rate

Run 10,000 z-tests where H_0 is true (both groups from the same distribution). Count what fraction of p-values fall below 0.05, 0.01, and 0.001. Do these match expectations?

```
# TODO: Simulate 10,000 null tests
# TODO: Count  $p < 0.05$ ,  $p < 0.01$ ,  $p < 0.001$ 
# TODO: Compare to expected rates (5%, 1%, 0.1%)
```

Exercise 4: One-Tailed vs Two-Tailed

Using the Alzheimer's biomarker data, compute the p-value for both a one-tailed test (H_1 : AD levels are *higher*) and a two-tailed test. What is the relationship between the two p-values?

```
let ad_levels = [3.2, 4.1, 3.8, 2.7, 4.5, 3.3, 3.9, 4.2, 3.1, 3.6,
                 4.0, 3.5, 4.3, 3.7, 2.9, 3.8, 4.1, 3.4, 3.6, 4.4]

# TODO: Compute z-stat manually, then get two-tailed and one-tailed
# p-values
# Two-tailed:  $2 * (1 - \text{pnorm}(\text{abs}(z), 0, 1))$ 
# One-tailed:  $1 - \text{pnorm}(z, 0, 1)$ 
# TODO: What is the mathematical relationship?
```

Key Takeaways

- Hypothesis testing uses the **courtroom analogy**: H_0 (innocence) is assumed until the evidence (data) is overwhelming
- The **p-value** is the probability of data this extreme under H_0 — it is NOT the probability H_0 is true
- **Type I error** (false positive) is controlled by alpha; **Type II error** (false negative) is controlled by power
- **Statistical significance** (small p) does not imply **practical significance** (large effect)
- The **z-test** is the simplest hypothesis test, applicable when sigma is known

- Always state hypotheses and choose alpha **before** looking at data
- Under the null, p-values are uniformly distributed: at $\alpha = 0.05$, exactly 5% of null tests will be “significant” by chance

What's Next

Tomorrow we move from the z-test (which requires known sigma) to the workhorse of biological research: the t-test. You will learn independent, paired, and Welch's versions, check assumptions with Shapiro-Wilk and Levene's tests, and quantify effect sizes with Cohen's d. If hypothesis testing is the question, the t-test is the answer for two-group comparisons.

Day 8: Comparing Two Groups — The t-Test

The Problem

Dr. Sofia Reyes is a cancer biologist studying BRCA1 expression in breast tissue. She has RNA-seq data from 12 tumor samples and 12 matched normal samples from the same patients. The mean BRCA1 expression in tumors is 4.2 log₂-CPM versus 6.8 log₂-CPM in normals — a 2.6-fold reduction. But with only 12 samples per group and considerable biological variability, can she confidently claim BRCA1 is downregulated in tumors?

She cannot use a z-test because the population standard deviation is unknown — she must estimate it from the data itself. She needs the **t-test**, the most widely used statistical test in biomedical research. But which version? Her samples are paired (tumor and normal from the same patient), which adds another consideration. And before running any test, she should verify that the data meet the test's assumptions.

This chapter covers the t-test in all its forms: independent, Welch's, paired, and one-sample. You will learn when each is appropriate, how to check assumptions, and how to quantify the magnitude of differences with Cohen's d.

What Is the t-Test?

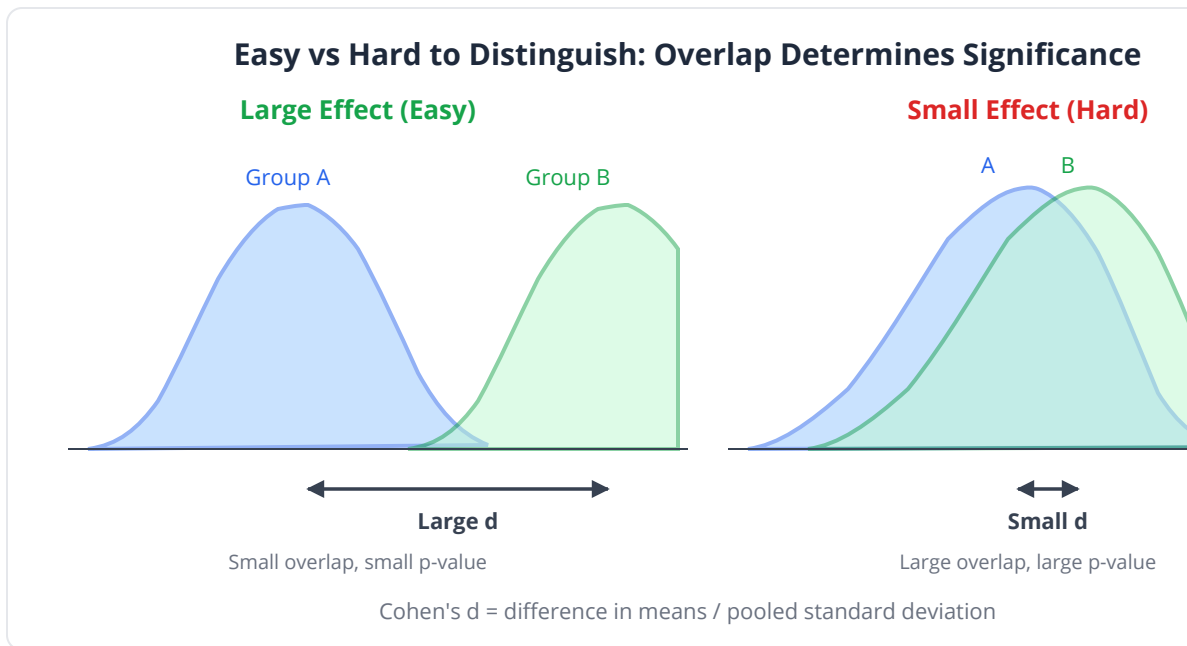
The t-test asks: "Is the difference between two group means larger than what we would expect from random sampling variation alone?"

Think of it this way: you have two piles of measurements. The t-test weighs how far apart the piles' centers are, relative to how spread out each pile is. If the

piles are far apart and tight, the difference is convincing. If they overlap substantially, it is not.

The t-statistic = (difference in means) / (standard error of the difference)

A larger t-statistic means more evidence of a real difference.



The Four Flavors of t-Test

Test	When to Use	Formula
One-sample	Compare sample mean to a known value	$t = (\bar{x} - \mu_0) / (s / \sqrt{n})$
Independent two-sample	Compare means of two unrelated groups	$t = (\bar{x}_1 - \bar{x}_2) / (s_p \times \sqrt{1/n_1 + 1/n_2})$
Welch's	Two unrelated groups, unequal variances	$t = (\bar{x}_1 - \bar{x}_2) / \sqrt{s_1^2/n_1 + s_2^2/n_2}$
Paired	Matched or before/after measurements	$t = \bar{d} / (s_d / \sqrt{n})$

Independent Two-Sample t-Test

Assumptions

1. **Independence:** Observations within and between groups are independent
2. **Normality:** Data in each group are approximately normally distributed
3. **Equal variances:** Both groups have similar spread (homoscedasticity)

The Pooled Standard Error

When variances are assumed equal, we pool them for a better estimate:

$$s_p = \sqrt{((n_1-1)s_1^2 + (n_2-1)s_2^2) / (n_1 + n_2 - 2)}$$

Degrees of freedom: **df = n1 + n2 - 2**

Welch's t-Test: The Safer Default

Welch's t-test does not assume equal variances. It uses each group's own variance estimate and adjusts the degrees of freedom downward with the Welch-Satterthwaite equation.

Key insight: Welch's t-test is almost always the better default choice. It performs nearly as well as the pooled t-test when variances ARE equal, and much better when they are not. Most modern statistical software (including R's `t.test()`) uses Welch's version by default.

Paired t-Test: Matched Samples

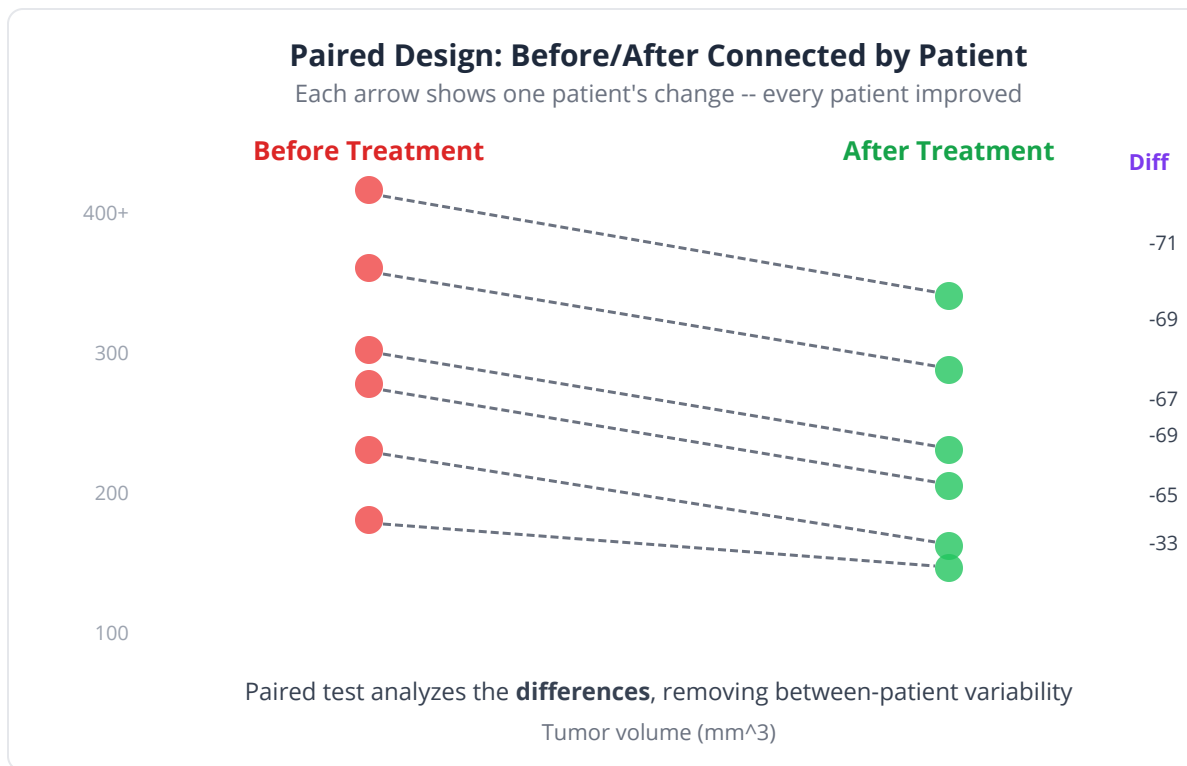
When observations are naturally paired — tumor/normal from the same patient, before/after treatment on the same subject — the paired t-test is far more powerful because it controls for inter-subject variability.

The trick: compute the **difference** for each pair, then perform a one-sample t-test on the differences:

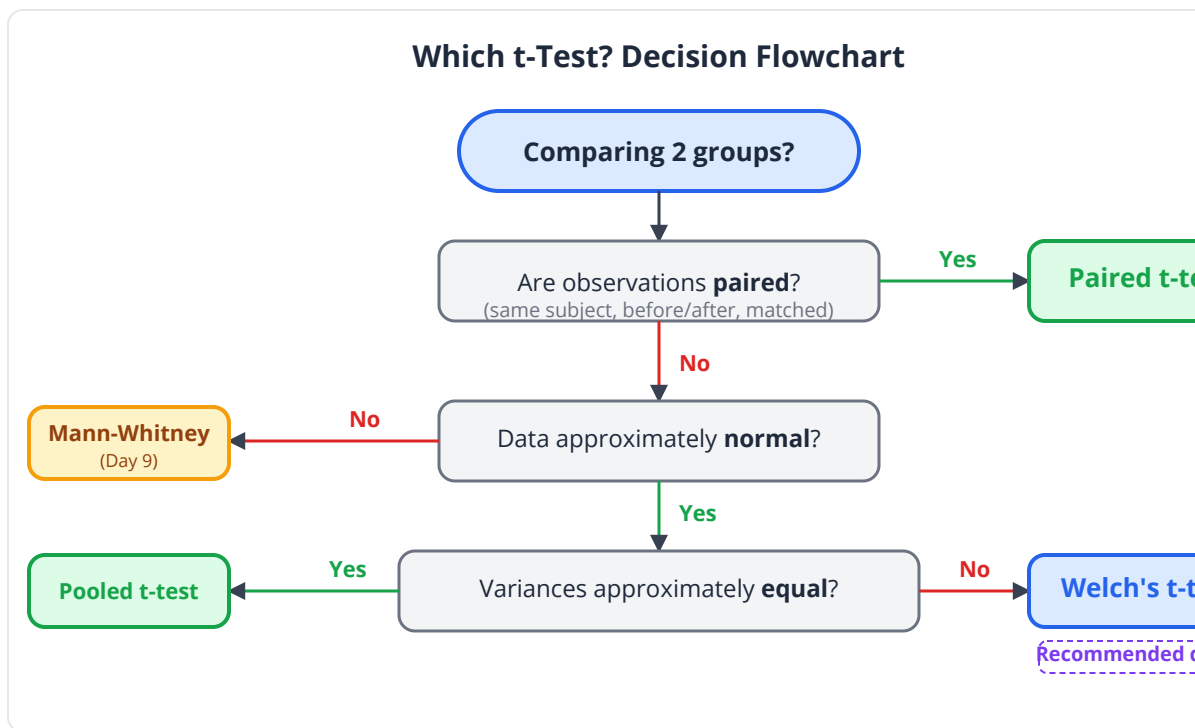
$$t = \bar{d} / (s_d / \sqrt{n})$$

Where $\bar{d}$ is the mean of the paired differences and s_d is their standard deviation.

Design	Pairing	Correct Test
Tumor vs normal from same patient	Paired	Paired t-test
Drug vs placebo in different patients	Independent	Welch's t-test
Before vs after treatment, same patients	Paired	Paired t-test
Wild-type vs knockout mice	Independent	Welch's t-test
Left eye vs right eye of same individuals	Paired	Paired t-test



Common pitfall: Using an independent t-test on paired data wastes statistical power. If you have natural pairs, always use the paired test. Conversely, using a paired test on unpaired data gives wrong results.



Checking Assumptions

Normality: Shapiro-Wilk Test

The Shapiro-Wilk test checks whether data could have come from a normal distribution.

- H_0 : Data are normally distributed
- If $p > 0.05$, normality assumption is reasonable
- If $p < 0.05$, data are significantly non-normal

Also use **QQ plots**: if points fall along the diagonal line, data are approximately normal.

Equal Variances: Levene's Test

Levene's test checks whether two groups have equal variances.

- H_0 : Variances are equal
- If $p > 0.05$, equal variance assumption is reasonable
- If $p < 0.05$, use Welch's t-test (or just always use Welch's)

Cohen's d: Quantifying Effect Size

A p-value tells you *whether* a difference exists. Cohen's d tells you *how large* it is, in standard deviation units:

$$d = (\bar{x}_1 - \bar{x}_2) / s_{\text{pooled}}$$

Cohen's d	Interpretation	Biological Example
0.2	Small	Subtle expression change
0.5	Medium	Moderate drug effect
0.8	Large	Strong phenotypic difference
> 1.2	Very large	Knockout vs wild-type

Key insight: A large p-value with a large Cohen's d suggests you are underpowered — you may have a real effect but too few samples to detect it. A small p-value with a tiny Cohen's d suggests the effect, while real, may not be biologically meaningful.

The t-Test in BioLang

Independent Two-Sample t-Test: Gene Expression

```
# BRCA1 expression (log2-CPM) in tumor vs normal breast tissue
let tumor = [3.8, 4.5, 4.1, 3.2, 4.8, 3.9, 4.3, 5.1, 3.6, 4.0, 4.7, 3.5]
let normal = [6.2, 7.1, 6.5, 7.4, 6.8, 7.0, 6.3, 7.2, 6.9, 6.6, 7.3, 6.1]

# Default: Welch's t-test (unequal variances)
let result = ttest(tumor, normal)
print("=== Welch's t-test: BRCA1 Tumor vs Normal ===")
print("t-statistic: {result.statistic:.4}")
print("p-value: {result.p_value:.2e}")
print("Degrees of freedom: {result.df:.1}")
print("Mean tumor: {mean(tumor):.2}, Mean normal: {mean(normal):.2}")
print("Difference: {mean(tumor) - mean(normal):.2} log2-CPM")

# Effect size (Cohen's d inline)
let d = (mean(tumor) - mean(normal)) / sqrt((variance(tumor) + variance(normal)) / 2.0)
print("Cohen's d: {d:.3}")

# Visualize
let bp_table = table({"Tumor": tumor, "Normal": normal})
boxplot(bp_table, {title: "BRCA1 Expression: Tumor vs Normal"})
```


Checking Assumptions

```
let tumor  = [3.8, 4.5, 4.1, 3.2, 4.8, 3.9, 4.3, 5.1, 3.6, 4.0, 4.7, 3.5]
let normal = [6.2, 7.1, 6.5, 7.4, 6.8, 7.0, 6.3, 7.2, 6.9, 6.6, 7.3, 6.1]

# 1. Normality check: use QQ plots for visual assessment
# (no built-in Shapiro-Wilk; use QQ plots + summary stats)
let s_tumor = summary(tumor)
let s_normal = summary(normal)
print("Tumor summary: {s_tumor}")
print("Normal summary: {s_normal}")

# 2. Equal variance check: compare variances from summary()
let var_ratio = variance(tumor) / variance(normal)
print("Variance ratio (tumor/normal): {var_ratio:.3}")
if var_ratio > 2.0 or var_ratio < 0.5 {
  print("Variances appear unequal -> use Welch's t-test (the default)")
} else {
  print("Variances appear similar -> pooled t-test is also valid")
}

# 3. QQ plots for visual normality assessment
qq_plot(tumor, {title: "QQ Plot: Tumor BRCA1 Expression"})
qq_plot(normal, {title: "QQ Plot: Normal BRCA1 Expression"})
```

Paired t-Test: Before/After Treatment

```
# Tumor volume (mm^3) before and after 6 weeks of treatment
# Same 10 patients measured at both time points
let before = [245, 312, 198, 367, 289, 421, 156, 334, 278, 305]
let after  = [180, 245, 165, 298, 220, 350, 132, 270, 210, 248]

# Paired t-test: accounts for patient-to-patient variability
let result = ttest_paired(before, after)
print("=== Paired t-test: Tumor Volume Before vs After Treatment
===")
print("t-statistic: {result.statistic:.4}")
print("p-value: {result.p_value:.6}")

# Show the paired differences
let diffs = zip(before, after) |> map(|pair| pair[0] - pair[1])
print("Mean reduction: {mean(diffs):.1} mm^3")
print("Individual reductions: {diffs}")

# Compare: what if we wrongly used an independent t-test?
let wrong_result = ttest(before, after)
print("\nWrong (independent) t-test p-value:
{wrong_result.p_value:.6}")
print("Correct (paired) t-test p-value: {result.p_value:.6}")
print("Paired test is more powerful because it removes inter-patient
variability")

# Visualize paired differences
histogram(diffs, {title: "Distribution of Tumor Volume Reductions",
x_label: "Reduction (mm^3)", bins: 8})
```

One-Sample t-Test

```
# Is the GC content of our assembled genome different from the
expected 41%?
let gc_per_contig = [40.2, 41.5, 39.8, 42.1, 40.7, 41.3, 39.5, 42.4,
                    40.1, 41.8, 40.5, 41.0, 39.9, 41.6, 40.3]

let result = ttest_one(gc_per_contig, 41.0)
print("=== One-sample t-test: GC Content vs Expected 41% ===")
print("Sample mean: {mean(gc_per_contig):.2}%")
print("t-statistic: {result.statistic:.4}")
print("p-value: {result.p_value:.4}")

if result.p_value > 0.05 {
  print("No significant deviation from expected GC content")
}
```

Complete Workflow: Multiple Genes

```
# Test multiple genes at once
let genes = ["BRCA1", "TP53", "MYC", "GAPDH", "EGFR"]

let tumor_expr = [
  [3.8, 4.5, 4.1, 3.2, 4.8, 3.9, 4.3, 5.1, 3.6, 4.0, 4.7, 3.5],
  [2.1, 1.8, 2.5, 1.4, 2.2, 1.9, 2.0, 1.6, 2.3, 1.7, 2.4, 1.5],
  [9.2, 10.1, 8.8, 9.5, 10.3, 9.7, 8.6, 9.9, 10.5, 9.1, 9.8, 10.2],
  [8.1, 8.3, 7.9, 8.2, 8.0, 8.4, 7.8, 8.1, 8.3, 8.0, 8.2, 7.9],
  [7.5, 8.2, 7.8, 8.5, 7.1, 8.0, 7.6, 8.3, 7.9, 8.1, 7.4, 8.4]
]

let normal_expr = [
  [6.2, 7.1, 6.5, 7.4, 6.8, 7.0, 6.3, 7.2, 6.9, 6.6, 7.3, 6.1],
  [5.8, 6.2, 5.5, 6.0, 5.9, 6.3, 5.7, 6.1, 5.6, 6.4, 5.8, 6.0],
  [5.1, 5.4, 4.9, 5.3, 5.6, 5.0, 5.2, 5.5, 4.8, 5.7, 5.1, 5.4],
  [8.0, 8.2, 7.8, 8.3, 8.1, 8.0, 8.4, 7.9, 8.2, 8.1, 8.3, 8.0],
  [5.0, 5.3, 4.8, 5.1, 5.5, 4.9, 5.2, 5.4, 4.7, 5.6, 5.0, 5.3]
]

print("Gene          | t-stat | p-value      | Cohen's d |
Interpretation")
print("-----|-----|-----|-----|-----")
")

for i in 0..len(genes) {
  let result = ttest(tumor_expr[i], normal_expr[i])
  let d = (mean(tumor_expr[i]) - mean(normal_expr[i])) /
sqrt((variance(tumor_expr[i]) + variance(normal_expr[i])) / 2.0)
  let interp = if abs(d) > 0.8 then "Large" else if abs(d) > 0.5
then "Medium" else "Small"
  print("{genes[i]:<10} | {result.statistic:>6.2} |
{result.p_value:>10.2e} | {d:>9.3} | {interp}")
}
```

Python:

```

from scipy import stats
import numpy as np

tumor = [3.8, 4.5, 4.1, 3.2, 4.8, 3.9, 4.3, 5.1, 3.6, 4.0, 4.7,
3.5]
normal = [6.2, 7.1, 6.5, 7.4, 6.8, 7.0, 6.3, 7.2, 6.9, 6.6, 7.3,
6.1]

# Welch's t-test (default)
t, p = stats.ttest_ind(tumor, normal, equal_var=False)
print(f"Welch's t = {t:.4f}, p = {p:.2e}")

# Paired t-test
before = [245, 312, 198, 367, 289, 421, 156, 334, 278, 305]
after = [180, 245, 165, 298, 220, 350, 132, 270, 210, 248]
t, p = stats.ttest_rel(before, after)
print(f"Paired t = {t:.4f}, p = {p:.6f}")

# Cohen's d (manual)
pooled_std = np.sqrt((np.std(tumor, ddof=1)**2 + np.std(normal,
ddof=1)**2) / 2)
d = (np.mean(tumor) - np.mean(normal)) / pooled_std
print(f"Cohen's d = {d:.3f}")

```

R:

```

tumor <- c(3.8, 4.5, 4.1, 3.2, 4.8, 3.9, 4.3, 5.1, 3.6, 4.0, 4.7,
3.5)
normal <- c(6.2, 7.1, 6.5, 7.4, 6.8, 7.0, 6.3, 7.2, 6.9, 6.6, 7.3,
6.1)

# Welch's t-test (default in R)
t.test(tumor, normal)

# Paired t-test
before <- c(245, 312, 198, 367, 289, 421, 156, 334, 278, 305)
after <- c(180, 245, 165, 298, 220, 350, 132, 270, 210, 248)
t.test(before, after, paired = TRUE)

# Cohen's d
library(effsize)
cohen.d(tumor, normal)

```

Exercises

Exercise 1: Choose the Right t-Test

For each scenario, state which t-test variant is appropriate and why:

a) Comparing white blood cell counts between 20 patients with sepsis and 25 healthy volunteers b) Measuring gene expression in liver biopsies taken before and after drug treatment (same 15 patients) c) Testing whether mean read length from your sequencer matches the expected 150 bp

Exercise 2: Full t-Test Workflow

Hemoglobin levels (g/dL) in two groups:

- Anemia patients: [9.2, 8.8, 10.1, 9.5, 8.3, 9.7, 8.6, 9.0, 10.3, 8.9]
- Healthy controls: [13.5, 14.2, 12.8, 13.9, 14.5, 13.1, 14.0, 13.6, 12.9, 14.3]

```
let anemia = [9.2, 8.8, 10.1, 9.5, 8.3, 9.7, 8.6, 9.0, 10.3, 8.9]
let healthy = [13.5, 14.2, 12.8, 13.9, 14.5, 13.1, 14.0, 13.6, 12.9, 14.3]
```

```
# TODO: 1. Check normality with qq_plot() on each group
# TODO: 2. Check equal variances by comparing variance() per group
# TODO: 3. Run the appropriate t-test with ttest()
# TODO: 4. Compute Cohen's d inline: (mean(a)-mean(b)) /
sqrt((variance(a)+variance(b))/2)
# TODO: 5. Create a boxplot
# TODO: 6. Interpret results in a biological context
```

Exercise 3: Paired vs Independent

Run both a paired and independent t-test on the tumor volume data below. Compare the p-values and explain why they differ.

```
let before = [245, 312, 198, 367, 289, 421, 156, 334, 278, 305]
let after  = [180, 245, 165, 298, 220, 350, 132, 270, 210, 248]

# TODO: Run ttest_paired and ttest
# TODO: Which gives a smaller p-value? Why?
# TODO: What does the paired test "remove" that the independent test
cannot?
```

Exercise 4: When Assumptions Fail

The following data are highly skewed (as often seen in cytokine measurements):

```
let treatment = [2.1, 1.8, 45.2, 3.5, 2.9, 1.2, 38.7, 4.1, 2.3, 1.5]
let control   = [0.8, 0.5, 0.9, 0.3, 1.1, 0.7, 0.4, 0.6, 1.0, 0.2]

# TODO: Test normality with qq_plot()
# TODO: Run the t-test with ttest() anyway – what does it say?
# TODO: Try log-transforming the data and re-testing
# TODO: Preview: tomorrow we'll learn non-parametric alternatives
```

Key Takeaways

- The **t-test** compares two group means, accounting for variability and sample size
- **Welch's t-test** (unequal variances) should be the default — it is robust even when variances are equal
- **Paired t-tests** are more powerful when observations are naturally matched (same patient, same timepoint)
- Always **check assumptions**: Shapiro-Wilk for normality, Levene's for equal variances, QQ plots for visual inspection
- **Cohen's d** quantifies effect size independently of sample size: 0.2 = small, 0.5 = medium, 0.8 = large
- A significant t-test with a small Cohen's d may not be biologically meaningful
- A non-significant t-test with a large Cohen's d suggests you need more samples

What's Next

What happens when your data violate the normality assumption? Cytokine levels, bacterial abundances, and many other biological measurements are wildly skewed. Tomorrow we introduce non-parametric tests — rank-based alternatives to the t-test that make no assumptions about the shape of your data distribution.

Day 9: When Normality Fails — Non-Parametric Tests

The Problem

Dr. Maria Gonzalez studies the gut microbiome in inflammatory bowel disease (IBD). She has 16S rRNA sequencing data from 15 IBD patients and 15 healthy controls, measuring the relative abundance of *Faecalibacterium prausnitzii*, a key anti-inflammatory bacterium. Looking at the data, she sees a mess: most values cluster near zero, a few patients have moderate levels, and one healthy individual has an enormous abundance of 45%. The histogram looks nothing like a bell curve — it is right-skewed with a long tail.

She runs a Shapiro-Wilk test on each group: both return $p < 0.001$, firmly rejecting normality. The t-test assumes normally distributed data. With data this skewed, the t-test's p-value could be wildly inaccurate — too liberal or too conservative, depending on the specific pattern. She needs tests that work without any assumptions about the shape of the distribution.

These are **non-parametric tests**: methods that operate on the *ranks* of data rather than the raw values, making them robust to skewness, outliers, and any distributional shape.

What Are Non-Parametric Tests?

Imagine you are judging a cooking competition. A parametric judge scores each dish on a precise 1-100 scale and compares average scores. A non-parametric judge simply ranks the dishes from best to worst — first place, second place, third place. The ranking approach is less precise when scores are reliable, but it is far more robust when one judge has an eccentric scoring system.

Non-parametric tests replace raw data values with their **ranks** (1st smallest, 2nd smallest, ...) and then analyze the ranks. This has powerful consequences:

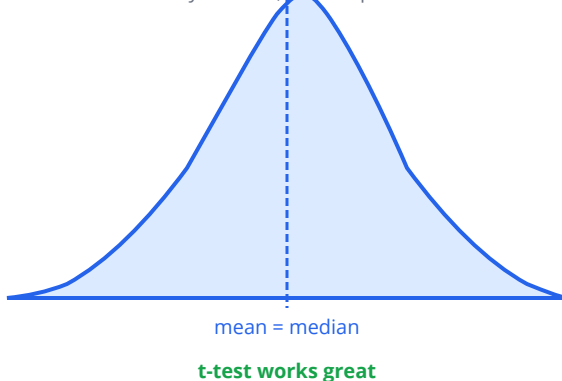
Property	Parametric (t-test)	Non-parametric (rank-based)
Assumes normality	Yes	No
Sensitive to outliers	Very	Resistant
Uses raw values	Yes	Uses ranks
Power (normal data)	Highest	Slightly lower (~95%)
Power (non-normal data)	Unreliable	Reliable
Handles ordinal data	No	Yes

Key insight: Non-parametric tests are not “worse” versions of parametric tests. They are the correct choice when distributional assumptions are violated. Using a t-test on heavily skewed data is like measuring temperature with a ruler — you might get a number, but it doesn’t mean anything.

Normal Data vs Skewed Microbiome Data

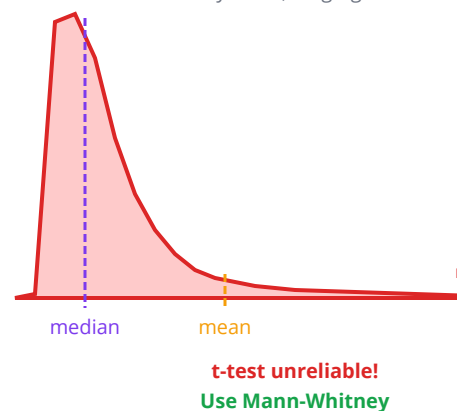
Normal (Gene Expression)

Symmetric, bell-shaped



Skewed (Microbiome Abundance)

Many zeros, long right tail



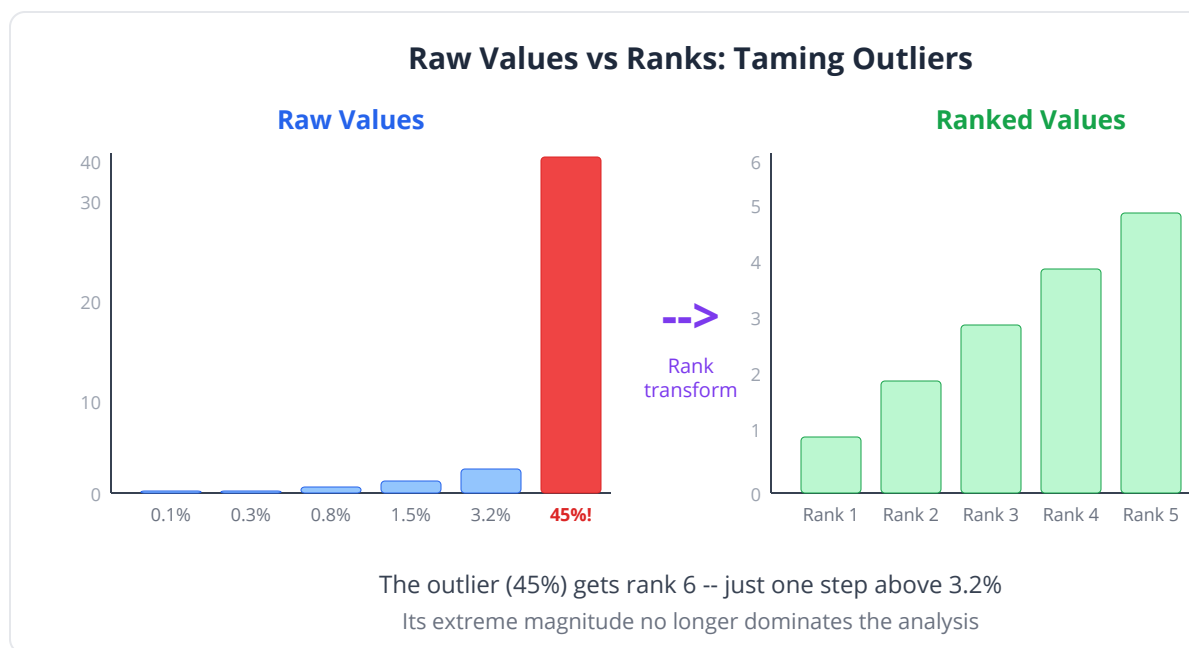
When to Choose Non-Parametric

Use non-parametric tests when:

- **Shapiro-Wilk** rejects normality ($p < 0.05$) and sample size is small
- Data are **ordinal** (pain scale 1-10, tumor grade I-IV)
- Data have **heavy outliers** that cannot be removed
- **Sample sizes are very small** ($n < 10$ per group)
- Data are **bounded** or have floor/ceiling effects (many zeros)

The Rank Transformation

The foundation of all non-parametric tests is replacing values with ranks:



Patient	Abundance	Rank
P1	0.1%	1
P2	0.3%	2
P3	0.8%	3
P4	1.5%	4
P5	3.2%	5
P6	45.0%	6

Notice: the outlier (45%) gets rank 6 — just one rank above 3.2%. Its extreme value no longer dominates the analysis.

Wilcoxon Rank-Sum Test (Mann-Whitney U)

The non-parametric counterpart of the independent two-sample t-test.

Procedure:

1. Combine all observations and rank them 1 through N
2. Sum the ranks in each group separately
3. If one group consistently has higher ranks, the rank sum will be extreme
4. Compare to the expected rank sum under H_0 (no difference)

H_0 : The two groups have identical distributions **H_1 :** One group tends to have larger values

Common pitfall: The Wilcoxon rank-sum and Mann-Whitney U are the same test, just computed differently. $U = W - n_1(n_1+1)/2$. Different software uses different names, but the p-value is identical.

Wilcoxon Signed-Rank Test

The non-parametric counterpart of the paired t-test.

Procedure:

1. Compute the difference for each pair
2. Rank the absolute differences (ignoring zeros)
3. Sum ranks of positive differences (W^+) and negative differences (W^-)
4. If the treatment consistently increases (or decreases), one sum will dominate

Sign Test

Even simpler than Wilcoxon signed-rank — only considers the *direction* of differences, not their magnitude.

Procedure:

1. For each pair, note whether the difference is positive, negative, or zero
2. Count positives and negatives (discard zeros)
3. Under H_0 , positives and negatives should be equally likely (binomial test with $p = 0.5$)

The sign test has less power than Wilcoxon signed-rank but makes even fewer assumptions.

Kruskal-Wallis Test

The non-parametric counterpart of one-way ANOVA, for comparing three or more groups.

H_0 : All groups have the same distribution **H_1 :** At least one group differs

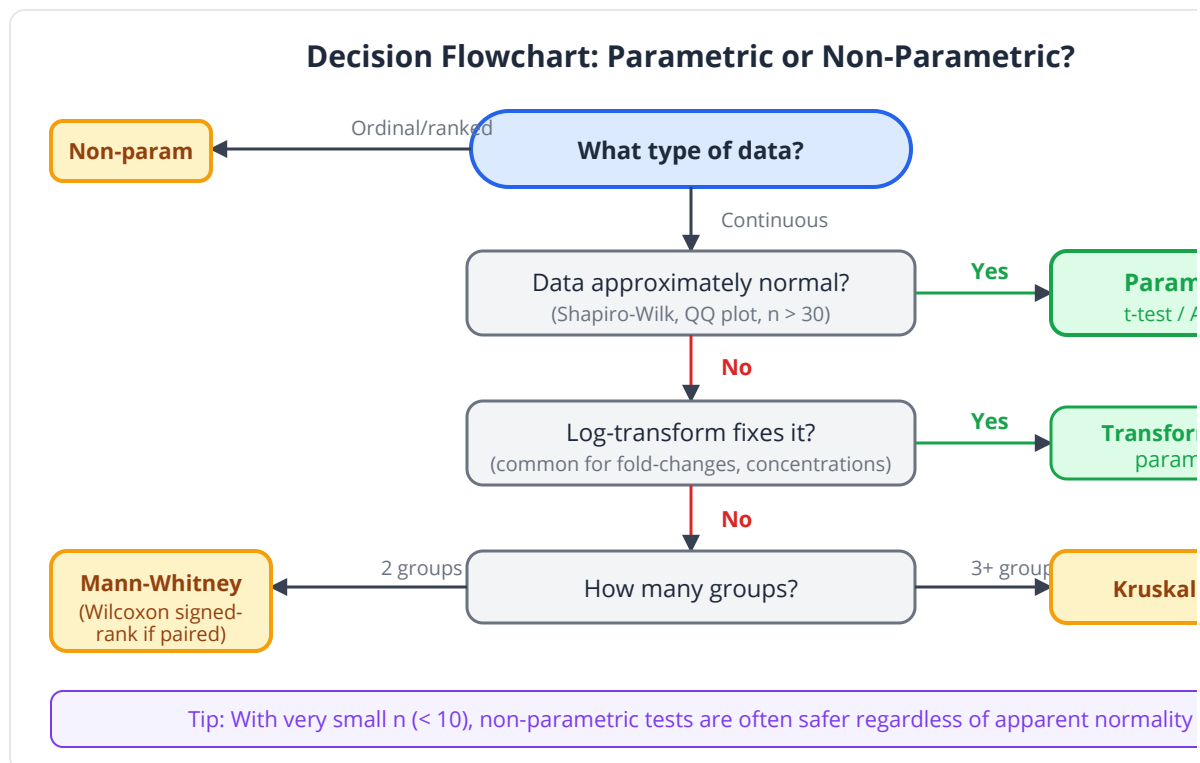
If significant, follow up with pairwise Wilcoxon tests (with multiple testing correction).

Kolmogorov-Smirnov (KS) Test

Compares two entire distributions, not just their centers. Detects differences in shape, spread, or location.

H₀: The two samples come from the same distribution **H₁:** The distributions differ in any way

Clinical relevance: The KS test is useful when you suspect groups differ not just in average abundance, but in the entire pattern of their distribution — for example, one group might be bimodal while the other is unimodal.



Decision Guide: Parametric vs Non-Parametric

Comparison	Parametric	Non-Parametric
One sample vs known value	One-sample t-test	Wilcoxon signed-rank (one sample)
Two independent groups	Welch's t-test	Mann-Whitney U / Wilcoxon rank-sum
Two paired groups	Paired t-test	Wilcoxon signed-rank
Three+ independent groups	One-way ANOVA	Kruskal-Wallis
Three+ paired groups	Repeated measures ANOVA	Friedman test
Compare distributions	—	KS test

Non-Parametric Tests in BioLang

Mann-Whitney U: Microbiome Abundance

```
# F. prausnitzii relative abundance (%) in IBD vs healthy
let ibd = [0.1, 0.3, 0.0, 0.8, 0.2, 0.0, 1.5, 0.4, 0.1, 0.0,
          3.2, 0.5, 0.1, 0.7, 0.0]
let healthy = [2.1, 5.4, 1.8, 8.2, 3.5, 12.1, 4.7, 6.3, 2.9, 45.0,
              7.1, 3.8, 9.5, 4.2, 6.8]

# First, demonstrate why t-test is inappropriate
# Check normality visually – both distributions are right-skewed
qq_plot(ibd, {title: "QQ Plot: IBD"})
qq_plot(healthy, {title: "QQ Plot: Healthy"})
print("Both groups are heavily skewed – normality violated!\n")

# Mann-Whitney U test (non-parametric)
let result = wilcoxon(ibd, healthy)
print("=== Mann-Whitney U Test ===")
print("U statistic: {result.statistic:.1}")
print("p-value: {result.p_value:.2e}")

# Compare to (inappropriate) t-test
let t_result = ttest(ibd, healthy)
print("\n(Inappropriate) Welch's t-test p-value:
{t_result.p_value:.2e}")
print("Mann-Whitney p-value: {result.p_value:.2e}")
print("Results may differ substantially with skewed data")

# Visualize the skewed distributions
let bp_table = table({"IBD": ibd, "Healthy": healthy})
boxplot(bp_table, {title: "F. prausnitzii Abundance"})
```

Wilcoxon Signed-Rank: Paired Treatment Data

```
# Inflammatory cytokine IL-6 (pg/mL) before and after anti-TNF
therapy
# Same 12 patients measured twice – highly skewed cytokine data
let before = [245, 18, 892, 45, 32, 1250, 67, 128, 15, 543, 78,
2100]
let after  = [120, 12, 340, 22, 28, 450, 35, 65, 10, 210, 42,
890]

# Normality check on differences
let diffs = zip(before, after) |> map(|p| p[0] - p[1])
qq_plot(diffs, {title: "QQ Plot: Paired Differences"})
print("Differences are non-normal -> use Wilcoxon signed-rank\n")

# Wilcoxon signed-rank test
let result = wilcoxon(before, after)
print("=== Wilcoxon Signed-Rank Test ===")
print("V statistic: {result.statistic:.1}")
print("p-value: {result.p_value:.6}")
print("All 12 patients showed reduction in IL-6")

# For comparison: the sign test via dbinom (even more robust, less
powerful)
# Count how many differences are positive
let n_pos = diffs |> filter(|d| d > 0) |> len()
let n_nonzero = diffs |> filter(|d| d != 0) |> len()
# Under H0, n_pos ~ Binomial(n_nonzero, 0.5)
let sign_p = 0.0
for k in range(n_pos, n_nonzero + 1) {
  sign_p = sign_p + dbinom(k, n_nonzero, 0.5)
}
let sign_p = 2.0 * min(sign_p, 1.0 - sign_p) # two-tailed
print("\nSign test p-value: {sign_p:.6}")
```

Kruskal-Wallis: Multiple Body Sites

```
# Bacterial diversity (Shannon index) across three gut regions
let ileum    = [1.2, 0.8, 1.5, 0.3, 2.1, 0.9, 1.4, 0.6, 1.8, 0.4]
let cecum    = [2.5, 3.1, 2.8, 2.2, 3.4, 2.7, 3.0, 2.3, 2.9, 3.2]
let rectum   = [3.8, 4.2, 3.5, 4.5, 3.9, 4.1, 3.6, 4.3, 3.7, 4.0]

# Kruskal-Wallis: use anova() on rank-transformed data
let result = anova([ileum, cecum, rectum])
print("=== Kruskal-Wallis Test: Diversity Across Body Sites ===")
print("H statistic: {result.statistic:.4}")
print("p-value: {result.p_value:.2e}")
print("Degrees of freedom: {result.df}")

if result.p_value < 0.05 {
  print("\nAt least one body site differs. Running pairwise
  comparisons...")

  let p1 = wilcoxon(ileum, cecum).p_value
  let p2 = wilcoxon(ileum, rectum).p_value
  let p3 = wilcoxon(cecum, rectum).p_value

  # Bonferroni correction for 3 comparisons
  let adjusted = p_adjust([p1, p2, p3], "bonferroni")
  print("Ileum vs Cecum:  p = {adjusted[0]:.4}")
  print("Ileum vs Rectum: p = {adjusted[1]:.4}")
  print("Cecum vs Rectum: p = {adjusted[2]:.4}")
}

let bp_table = table({"Ileum": ileum, "Cecum": cecum, "Rectum":
rectum})
boxplot(bp_table, {title: "Microbial Diversity by Gut Region"})
```

KS Test: Comparing Distributions

```
# Do tumor suppressor genes and oncogenes have different
# expression distributions (not just different means)?
let tumor_suppressors = [2.1, 3.4, 1.8, 4.2, 2.9, 3.1, 2.5, 3.8,
1.5, 4.0,
                        2.7, 3.3, 2.0, 3.6, 2.3, 3.9, 1.9, 4.1,
2.6, 3.5]
let oncogenes = [5.2, 8.1, 6.3, 12.4, 7.5, 5.8, 9.2, 6.7, 11.3, 7.0,
5.5, 8.8, 6.1, 10.5, 7.3, 5.9, 9.7, 6.5, 11.8, 7.8]

let result = ks_test(tumor_suppressors, oncogenes)
print("=== KS Test: Expression Distributions ===")
print("D statistic: {result.statistic:.4}")
print("p-value: {result.p_value:.2e}")
print("Maximum distance between cumulative distributions:
{result.statistic:.4}")

histogram([tumor_suppressors, oncogenes], {labels: ["Tumor
Suppressors", "Oncogenes"], title: "Expression Distributions by Gene
Class", x_label: "Expression (log2-CPM)", bins: 12})
```

Comparing t-Test vs Wilcoxon on the Same Data

```
set_seed(42)
# Demonstrate: with normal data, both tests agree
# With skewed data, they can disagree

print("=== Normal Data: Both Tests Agree ===")
let norm_a = rnorm(20, 5.0, 1.0)
let norm_b = rnorm(20, 6.0, 1.0)
let t_p = ttest(norm_a, norm_b).p_value
let w_p = wilcoxon(norm_a, norm_b).p_value
print("t-test p = {t_p:.4}, Mann-Whitney p = {w_p:.4}")

print("\n=== Skewed Data with Outlier: Tests May Disagree ===")
let skew_a = [1.2, 1.5, 1.8, 1.1, 1.4, 1.6, 1.3, 1.7, 1.9, 50.0]
let skew_b = [2.1, 2.3, 2.5, 2.0, 2.4, 2.2, 2.6, 2.1, 2.3, 2.5]
let t_p2 = ttest(skew_a, skew_b).p_value
let w_p2 = wilcoxon(skew_a, skew_b).p_value
print("t-test p = {t_p2:.4}, Mann-Whitney p = {w_p2:.4}")
print("The outlier inflates the t-test mean, masking the real
pattern")
```

Python:

```
from scipy import stats

ibd = [0.1, 0.3, 0.0, 0.8, 0.2, 0.0, 1.5, 0.4, 0.1, 0.0,
       3.2, 0.5, 0.1, 0.7, 0.0]
healthy = [2.1, 5.4, 1.8, 8.2, 3.5, 12.1, 4.7, 6.3, 2.9, 45.0,
           7.1, 3.8, 9.5, 4.2, 6.8]

# Mann-Whitney U
u, p = stats.mannwhitneyu(ibd, healthy, alternative='two-sided')
print(f"U = {u}, p = {p:.2e}")

# Wilcoxon signed-rank (paired)
before = [245, 18, 892, 45, 32, 1250, 67, 128, 15, 543, 78, 2100]
after = [120, 12, 340, 22, 28, 450, 35, 65, 10, 210, 42, 890]
w, p = stats.wilcoxon(before, after)
print(f"W = {w}, p = {p:.6f}")

# Kruskal-Wallis
ileum = [1.2, 0.8, 1.5, 0.3, 2.1, 0.9, 1.4, 0.6, 1.8, 0.4]
cecum = [2.5, 3.1, 2.8, 2.2, 3.4, 2.7, 3.0, 2.3, 2.9, 3.2]
rectum = [3.8, 4.2, 3.5, 4.5, 3.9, 4.1, 3.6, 4.3, 3.7, 4.0]
h, p = stats.kruskal(ileum, cecum, rectum)
print(f"H = {h:.4f}, p = {p:.2e}")
```

R:

```
ibd <- c(0.1, 0.3, 0.0, 0.8, 0.2, 0.0, 1.5, 0.4, 0.1, 0.0,
        3.2, 0.5, 0.1, 0.7, 0.0)
healthy <- c(2.1, 5.4, 1.8, 8.2, 3.5, 12.1, 4.7, 6.3, 2.9, 45.0,
            7.1, 3.8, 9.5, 4.2, 6.8)

wilcox.test(ibd, healthy)           # Mann-Whitney
wilcox.test(before, after, paired = TRUE) # Wilcoxon signed-rank
kruskal.test(list(ileum, cecum, rectum)) # Kruskal-Wallis
ks.test(tumor_suppressors, oncogenes)   # KS test
```

Exercises

Exercise 1: Choose the Right Test

For each dataset, decide whether a parametric or non-parametric test is more appropriate:

a) Pain scores (0-10 scale) in drug vs placebo groups b) Blood pressure measurements in 30 patients (continuous, approximately normal) c) Number of bacterial colonies per plate (many zeros, some very high counts) d) Survival time in days (typically right-skewed)

Exercise 2: Microbiome Comparison

Two diets were compared for their effect on *Bacteroides* abundance:

```
let high_fiber = [8.2, 12.5, 6.3, 15.1, 9.8, 22.4, 7.5, 11.2, 14.8, 5.9]
let low_fiber  = [1.2, 0.5, 3.1, 0.8, 2.4, 0.3, 1.8, 0.9, 2.7, 0.6]

# TODO: Test normality of each group
# TODO: Run Mann-Whitney U test
# TODO: Also run a t-test and compare results
# TODO: Create a boxplot
```

Exercise 3: Multiple Body Sites with Post-Hoc

OTU richness from four body sites (oral, gut, skin, vaginal). Run Kruskal-Wallis and, if significant, perform all pairwise comparisons with Bonferroni correction.

```
let oral      = [120, 95, 145, 110, 88, 132, 105, 98, 140, 115]
let gut       = [350, 420, 280, 390, 310, 445, 360, 295, 410, 380]
let skin      = [180, 210, 165, 195, 220, 175, 200, 185, 230, 190]
let vaginal   = [45, 30, 55, 38, 25, 50, 42, 35, 48, 28]

# TODO: Kruskal-Wallis test
# TODO: If significant, pairwise Mann-Whitney with Bonferroni correction
# TODO: Which sites differ from which?
```

Exercise 4: The Power Trade-Off

Generate 1000 simulations where both groups are truly normal with different means. Compare how often the t-test and Mann-Whitney detect the difference (power). Then repeat with skewed data (e.g., exponential).

```
# TODO: Simulate normal data, compare t-test vs Mann-Whitney power
# TODO: Simulate skewed data, compare again
# TODO: Which test wins in each scenario?
```

Key Takeaways

- **Non-parametric tests** use ranks instead of raw values, making them robust to skewness and outliers
- The **Mann-Whitney U** (Wilcoxon rank-sum) is the non-parametric alternative to the independent t-test
- The **Wilcoxon signed-rank** test is the non-parametric alternative to the paired t-test
- The **Kruskal-Wallis** test extends to three or more groups (non-parametric ANOVA)
- The **KS test** compares entire distributions, not just central tendency
- Non-parametric tests have about 95% of the power of parametric tests when data ARE normal, but are far more reliable when data are NOT normal
- Microbiome data, cytokine levels, survival times, and ordinal scales almost always require non-parametric methods
- Always check normality first (Shapiro-Wilk, QQ plots) — let the data guide your choice of test

What's Next

So far we have compared two groups. But what if you have three, four, or ten groups — different drug doses, tissue types, or experimental conditions? Running all pairwise t-tests inflates false positives dramatically. Tomorrow we introduce ANOVA, the principled way to compare many groups simultaneously, along with post-hoc tests that identify *which* groups differ.

Day 10: Comparing Many Groups — ANOVA and Beyond

The Problem

Dr. James Park's oncology team is testing a new targeted therapy at four dose levels: 0 mg (placebo), 25 mg, 50 mg, and 100 mg. Each group has 8 mice, and after 4 weeks they measure tumor volume in cubic millimeters. The team lead suggests: "Just do t-tests between all pairs of doses — that's 6 comparisons, no big deal."

But Dr. Park knows this is a trap. With 6 independent tests at $\alpha = 0.05$, the probability of at least one false positive is not 5% — it is $1 - (0.95)^6 = 26.5\%$. Run 10 comparisons and it climbs to 40%. With 20,000 genes, the problem becomes catastrophic (we will tackle that on Day 12). The solution for comparing several groups at once is **Analysis of Variance** — ANOVA — which tests all groups simultaneously in a single, principled framework.

ANOVA has been the workhorse of experimental biology for nearly a century. Every drug dose-response study, every multi-tissue gene expression comparison, and every agricultural field trial relies on it. Today you will learn why it works, when it fails, and what to do after you get a significant result.

What Is ANOVA?

Imagine a classroom of students from four different schools. ANOVA asks: "Is the variation in test scores *between* schools larger than what we would expect given the variation *within* each school?"

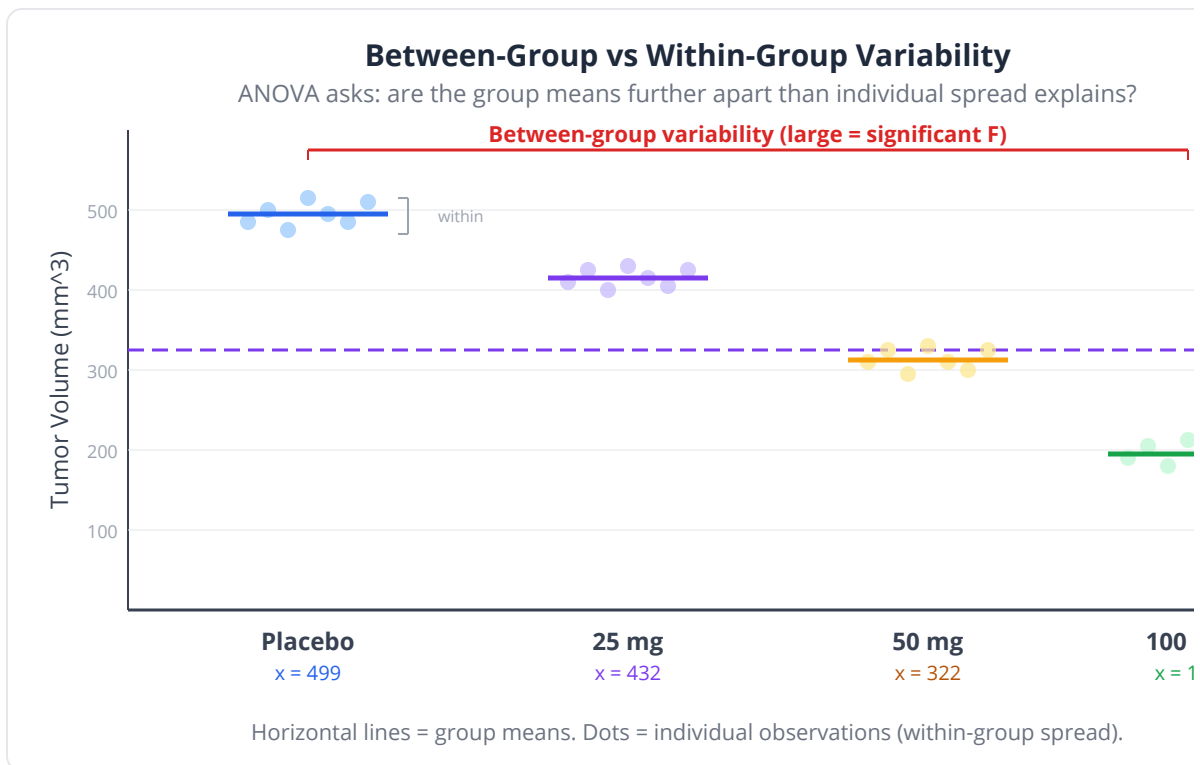
If all four schools have similar average scores, the between-school variation will be small relative to within-school variation. If one school's students consistently

outscore the others, the between-school variation will dominate.

ANOVA decomposes total variation into two sources:

Source	What It Measures	Symbol
Between-groups (treatment)	How much group means differ from the grand mean	SS_between
Within-groups (error)	How much individuals vary within their own group	SS_within
Total	Total variation in the data	SS_total

$$SS_total = SS_between + SS_within$$



The F-Statistic

The F-statistic is the ratio of between-group variance to within-group variance:

$$F = MS_{\text{between}} / MS_{\text{within}}$$

Where MS (mean square) = SS / df.

Source	df	MS	F
Between	k - 1	SS_between / (k - 1)	MS_between / MS_within
Within	N - k	SS_within / (N - k)	—
Total	N - 1	—	—

k = number of groups, N = total sample size.

- **F near 1:** Between-group variation is similar to within-group noise. No evidence of differences.
- **F much greater than 1:** Between-group variation exceeds what noise alone would produce. At least one group differs.

The F-Ratio: Between vs Within Variance

$$F = MS_{\text{between}} / MS_{\text{within}}$$

MS_between
(treatment effect + noise)



F near 1: groups similar (noise = noise)

MS_within
(noise only)



F >> 1: groups differ (signal > noise)



Key insight: ANOVA's null hypothesis is that ALL group means are equal. A significant F-statistic tells you "at least one group differs" but does NOT tell you which one(s). You need post-hoc tests for that.

The Family-Wise Error Rate Problem

Why not just do multiple t-tests?

Number of Groups	Pairwise Comparisons	P(at least one false positive)
3	3	14.3%
4	6	26.5%
5	10	40.1%
6	15	53.7%
10	45	90.1%

ANOVA controls this by testing all groups in a single hypothesis test.

Assumptions of One-Way ANOVA

1. **Independence:** Observations are independent within and between groups
2. **Normality:** Data within each group are approximately normally distributed
3. **Homoscedasticity:** All groups have equal variances (check with Levene's test)

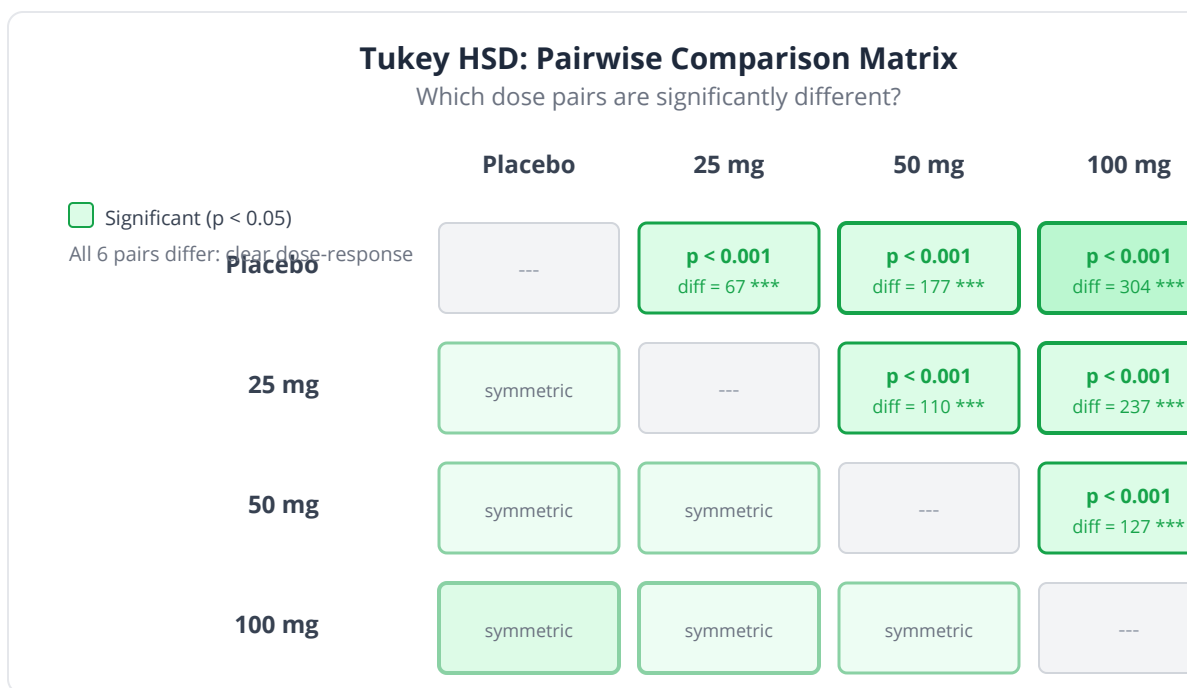
Common pitfall: ANOVA is robust to mild violations of normality when group sizes are equal and $n > 10$ per group. But it is sensitive to unequal variances, especially with unequal group sizes. When Levene's test is significant, consider Welch's ANOVA or the Kruskal-Wallis alternative.

Post-Hoc Tests: Which Groups Differ?

Tukey's Honestly Significant Difference (HSD)

The gold standard post-hoc test. Compares all pairs of group means while controlling the family-wise error rate.

- Tests all $k(k-1)/2$ pairwise differences
- Provides adjusted p-values and confidence intervals
- Assumes equal variances and equal (or similar) group sizes



Other Post-Hoc Options

Method	When to Use
Tukey HSD	All pairwise comparisons needed, balanced design
Bonferroni	Conservative; fewer planned comparisons
Dunnett	Compare all groups to a single control
Games-Howell	Unequal variances or unequal group sizes

Effect Size: Eta-Squared

Just as Cohen's d quantifies the effect for two groups, **eta-squared** quantifies it for ANOVA:

$$\text{eta-squared} = SS_{\text{between}} / SS_{\text{total}}$$

Eta-squared	Interpretation
0.01	Small — group explains 1% of total variance
0.06	Medium — group explains 6%
0.14	Large — group explains 14%+

Non-Parametric Alternatives

Parametric	Non-Parametric	Use When
One-way ANOVA	Kruskal-Wallis	Groups are independent, normality violated
Repeated measures ANOVA	Friedman test	Same subjects measured under all conditions

ANOVA in BioLang

One-Way ANOVA: Dose-Response

```
# Tumor volume (mm^3) at 4 dose levels
let placebo = [485, 512, 468, 530, 495, 478, 521, 503]
let dose_25 = [420, 445, 398, 461, 432, 410, 452, 438]
let dose_50 = [310, 335, 288, 352, 321, 298, 345, 328]
let dose_100 = [180, 210, 165, 225, 195, 172, 218, 198]

# One-way ANOVA
let result = anova([placebo, dose_25, dose_50, dose_100])
print("=== One-Way ANOVA: Tumor Volume by Dose ===")
print("F-statistic: {result.statistic:.4}")
print("p-value: {result.p_value:.2e}")
print("df between: {result.df_between}, df within: {result.df_within}")

# Effect size: eta-squared = SS_between / SS_total (inline from
anova output)
let eta2 = result.ss_between / (result.ss_between +
result.ss_within)
print("Eta-squared: {eta2:.4} ({eta2*100:.1}% of variance
explained)")

# Check assumptions: compare variances per group
print("\nVariances: placebo={variance(placebo):.1}, 25mg=
{variance(dose_25):.1}, 50mg={variance(dose_50):.1}, 100mg=
{variance(dose_100):.1}")
```

Tukey HSD Post-Hoc

```
let placebo = [485, 512, 468, 530, 495, 478, 521, 503]
let dose_25 = [420, 445, 398, 461, 432, 410, 452, 438]
let dose_50 = [310, 335, 288, 352, 321, 298, 345, 328]
let dose_100 = [180, 210, 165, 225, 195, 172, 218, 198]

# Tukey HSD: pairwise ttest() + p_adjust()
let groups = [placebo, dose_25, dose_50, dose_100]
let labels = ["Placebo", "25mg", "50mg", "100mg"]
let pairwise_p = []
let pair_labels = []
for i in 0..len(groups) {
  for j in (i+1)..len(groups) {
    let r = ttest(groups[i], groups[j])
    pairwise_p = append(pairwise_p, r.p_value)
    pair_labels = append(pair_labels, "{labels[i]} vs {labels[j]}")
  }
}
let adj_p = p_adjust(pairwise_p, "bonferroni")

print("=== Pairwise t-tests (Bonferroni-adjusted) ===")
print("Comparison      | Diff      | p-adj")
print("-----|-----|-----")
for k in 0..len(pair_labels) {
  print("{pair_labels[k]:<20}| {adj_p[k]:.4}")
}
```

Grouped Boxplot Visualization

```
let groups = {
  "Placebo": [485, 512, 468, 530, 495, 478, 521, 503],
  "25 mg": [420, 445, 398, 461, 432, 410, 452, 438],
  "50 mg": [310, 335, 288, 352, 321, 298, 345, 328],
  "100 mg": [180, 210, 165, 225, 195, 172, 218, 198]
}

boxplot(groups, {title: "Tumor Volume by Treatment Dose", y_label:
"Tumor Volume (mm^3)", x_label: "Dose Group", show_points: true})
```


Kruskal-Wallis: When ANOVA Assumptions Fail

```
# Cytokine levels across three disease stages – heavily skewed
let stage_I   = [2.1, 1.5, 3.8, 0.9, 12.5, 1.8, 2.4, 0.7, 4.2, 1.1]
let stage_II  = [8.5, 15.2, 5.3, 22.1, 9.8, 45.0, 7.2, 12.8, 6.1,
18.5]
let stage_III = [35.2, 88.1, 42.5, 120.0, 55.3, 78.9, 95.2, 48.7,
110.5, 65.8]

# Check normality
# Visual normality check – all stages are right-skewed
qq_plot(stage_I, {title: "QQ: Stage I"})
qq_plot(stage_II, {title: "QQ: Stage II"})
qq_plot(stage_III, {title: "QQ: Stage III"})
print("Normality violated -> use Kruskal-Wallis (anova on ranks)\n")

let result = anova([stage_I, stage_II, stage_III])
print("=== Kruskal-Wallis Test ===")
print("H statistic: {result.statistic:.4}")
print("p-value: {result.p_value:.2e}")

# Pairwise follow-up with Bonferroni correction
if result.p_value < 0.05 {
  let p12 = wilcoxon(stage_I, stage_II).p_value
  let p13 = wilcoxon(stage_I, stage_III).p_value
  let p23 = wilcoxon(stage_II, stage_III).p_value
  let adj = p_adjust([p12, p13, p23], "bonferroni")

  print("\nPairwise Mann-Whitney (Bonferroni-adjusted):")
  print("  Stage I vs II:   p = {adj[0]:.4}")
  print("  Stage I vs III:  p = {adj[1]:.4}")
  print("  Stage II vs III: p = {adj[2]:.4}")
}

let bp_table = table({"Stage I": stage_I, "Stage II": stage_II,
"Stage III": stage_III})
boxplot(bp_table, {title: "IL-6 Levels by Disease Stage"})
```

Friedman Test: Repeated Measures

```
# Pain scores (1-10) for 8 patients under 3 analgesics
# Same patients tested with each drug (crossover design)
let drug_a = [7, 5, 8, 6, 4, 7, 5, 6]
let drug_b = [4, 3, 5, 4, 2, 5, 3, 4]
let drug_c = [3, 2, 4, 3, 1, 3, 2, 3]

# Friedman test: anova() on ranks for repeated measures
let result = anova([drug_a, drug_b, drug_c])
print("=== Friedman Test: Pain Scores Across Analgesics ===")
print("Chi-squared: {result.statistic:.4}")
print("p-value: {result.p_value:.6}")

if result.p_value < 0.05 {
  print("At least one analgesic differs in pain reduction")
  print("Medians: Drug A={median(drug_a)}, Drug B={median(drug_b)},
Drug C={median(drug_c)}")
}
```

Complete Workflow: Gene Expression Across Tissues

```
# Conceptual or diagnostic example; not directly executable.
# FOXP3 expression across immune cell types
let t_reg = [8.5, 9.2, 8.8, 9.5, 8.1, 9.0, 8.7, 9.3]
let t_eff = [3.2, 3.8, 3.5, 4.1, 2.9, 3.6, 3.3, 3.9]
let b_cell = [1.5, 1.8, 1.2, 2.0, 1.4, 1.7, 1.3, 1.9]
let nk      = [0.8, 1.1, 0.6, 1.3, 0.9, 1.0, 0.7, 1.2]

# Step 1: Check assumptions
print("=== Assumption Checks ===")
# Compare variances across groups
print("Variances: T-reg={variance(t_reg):.3}, T-eff={variance(t_eff):.3}, B cell={variance(b_cell):.3}, NK={variance(nk):.3}")
# Visual normality check
for name, data in [{"T-reg", t_reg}, {"T-eff", t_eff}, {"B cell", b_cell}, {"NK", nk}] {
  qq_plot(data, {title: "QQ Plot: {name}"})
}

# Step 2: ANOVA
let result = anova([t_reg, t_eff, b_cell, nk])
print("\n=== One-Way ANOVA ===")
print("F = {result.statistic:.2}, p = {result.p_value:.2e}")

# Step 3: Effect size (eta-squared from anova output)
let eta2 = result.ss_between / (result.ss_between + result.ss_within)
print("Eta-squared = {eta2:.3} (cell type explains {eta2*100:.1}% of variance)")

# Step 4: Post-hoc – pairwise ttest() + p_adjust()
let cell_groups = [t_reg, t_eff, b_cell, nk]
let cell_labels = ["T-reg", "T-eff", "B cell", "NK"]
let pw_pvals = []
let pw_labels = []
for i in 0..len(cell_groups) {
  for j in (i+1)..len(cell_groups) {
    pw_pvals = append(pw_pvals, ttest(cell_groups[i], cell_groups[j]).p_value)
    pw_labels = append(pw_labels, "{cell_labels[i]} vs {cell_labels[j]}")
  }
}
let pw_adj = p_adjust(pw_pvals, "bonferroni")
```

```

print("\n=== Pairwise t-tests (Bonferroni) ===")
for k in 0..len(pw_labels) {
  let sig = if pw_adj[k] < 0.001 then "***" else if pw_adj[k] < 0.01
then "**" else if pw_adj[k] < 0.05 then "*" else "ns"
  print("{pw_labels[k]:<20} p={pw_adj[k]:.4} {sig}")
}

# Step 5: Visualize
let bp_table = table({"T-reg": t_reg, "T-eff": t_eff, "B cell":
b_cell, "NK": nk})
boxplot(bp_table, {title: "FOXP3 Expression Across Immune Cell
Types", show_points: true})

```

Python:

```

from scipy import stats
import scikit_posthocs as sp

placebo = [485, 512, 468, 530, 495, 478, 521, 503]
dose_25 = [420, 445, 398, 461, 432, 410, 452, 438]
dose_50 = [310, 335, 288, 352, 321, 298, 345, 328]
dose_100 = [180, 210, 165, 225, 195, 172, 218, 198]

# One-way ANOVA
f, p = stats.f_oneway(placebo, dose_25, dose_50, dose_100)
print(f"F = {f:.4f}, p = {p:.2e}")

# Tukey HSD
from statsmodels.stats.multicomp import pairwise_tukeyhsd
import numpy as np
data = placebo + dose_25 + dose_50 + dose_100
groups = ['P']*8 + ['25']*8 + ['50']*8 + ['100']*8
print(pairwise_tukeyhsd(data, groups))

# Kruskal-Wallis
h, p = stats.kruskal(placebo, dose_25, dose_50, dose_100)

# Friedman
stats.friedmanchisquare(drug_a, drug_b, drug_c)

```

R:

```
# One-way ANOVA
data <- data.frame(
  volume = c(485,512,468,530,495,478,521,503,
             420,445,398,461,432,410,452,438,
             310,335,288,352,321,298,345,328,
             180,210,165,225,195,172,218,198),
  dose = factor(rep(c("Placebo","25mg","50mg","100mg"), each=8))
)
result <- aov(volume ~ dose, data = data)
summary(result)
TukeyHSD(result)

# Eta-squared
library(effectsize)
eta_squared(result)

# Kruskal-Wallis
kruskal.test(volume ~ dose, data = data)

# Friedman
friedman.test(matrix(c(drug_a, drug_b, drug_c), ncol=3))
```

Exercises

Exercise 1: FWER Calculation

You have 5 experimental groups and want to compare all pairs. Calculate: (a) how many pairwise comparisons there are, (b) the probability of at least one false positive at $\alpha = 0.05$, (c) what the Bonferroni-adjusted α would be.

Exercise 2: Full ANOVA Workflow

Three fertilizer treatments were tested on plant growth (cm):

```
let control = [12.5, 14.2, 11.8, 13.5, 12.9, 14.8, 11.2, 13.1]
let fert_a  = [18.3, 20.1, 17.5, 19.8, 18.9, 21.2, 17.0, 19.5]
let fert_b  = [15.8, 17.2, 14.5, 16.9, 15.3, 17.8, 14.1, 16.5]

# TODO: Check normality and equal variances
# TODO: Run one-way ANOVA
# TODO: Compute eta-squared
# TODO: If significant, run Tukey HSD
# TODO: Create boxplot
# TODO: Interpret: which fertilizer is best?
```

Exercise 3: Parametric vs Non-Parametric ANOVA

Run both ANOVA and Kruskal-Wallis on the cytokine data. Compare p-values. Which is more appropriate?

```
let mild      = [5.2, 3.8, 8.1, 2.5, 12.0, 4.3, 6.7, 1.9]
let moderate  = [25.1, 45.0, 18.3, 52.8, 30.5, 15.2, 38.7, 22.4]
let severe    = [120, 250, 85, 310, 180, 95, 275, 145]

# TODO: Check normality
# TODO: Run both ANOVA and Kruskal-Wallis
# TODO: Which test is more appropriate for this data? Why?
```

Exercise 4: Repeated Measures Design

Five patients had their blood pressure measured under three conditions (rest, mild exercise, intense exercise):

```
let rest      = [120, 135, 118, 142, 125]
let mild_ex   = [130, 148, 128, 155, 138]
let intense   = [155, 172, 148, 180, 162]

# TODO: Use Friedman test (non-parametric repeated measures)
# TODO: If significant, perform pairwise Wilcoxon signed-rank tests
# TODO: Apply Bonferroni correction to the pairwise p-values
```

Key Takeaways

- **Multiple t-tests** inflate the false positive rate — the family-wise error rate grows rapidly with the number of comparisons
- **ANOVA** tests whether any group differs from the others in a single F-test, controlling the overall error rate
- The **F-statistic** compares between-group variance to within-group variance: F much greater than 1 suggests real differences
- A significant ANOVA tells you “at least one group differs” — use **Tukey HSD** post-hoc to find which pairs differ
- **Eta-squared** measures effect size: the proportion of total variance explained by group membership
- **Kruskal-Wallis** is the non-parametric alternative when normality is violated
- **Friedman test** handles repeated measures designs non-parametrically
- Always check assumptions (normality, equal variances) before interpreting ANOVA results

What's Next

Tomorrow we shift from continuous to categorical outcomes. When your data consist of counts in categories — genotypes, disease status, response/non-response — you need the chi-square test and Fisher's exact test. These tools are essential for testing genetic associations, evaluating Hardy-Weinberg equilibrium, and computing odds ratios in case-control studies.

Day 11: Categorical Data — Chi-Square and Fisher's Exact

The Problem

Dr. Elena Vasquez is an epidemiologist studying the genetics of Alzheimer's disease. She genotypes a SNP near the APOE gene in 1,000 participants: 500 with Alzheimer's and 500 age-matched controls. Among Alzheimer's patients, 180 carry at least one copy of the risk allele. Among controls, 120 carry it. The numbers look different — 36% versus 24% — but these are proportions, not measurements on a continuous scale. She cannot compute a mean or standard deviation. She cannot run a t-test.

When your data are counts in categories — disease yes/no, genotype AA/AG/GG, response/non-response — you need tests designed for categorical data. The chi-square test and Fisher's exact test are the workhorses. They also underpin some of the most important calculations in genetics: Hardy-Weinberg equilibrium, odds ratios for case-control studies, and allelic association tests.

This chapter covers the full toolkit for analyzing categorical data, from contingency tables to effect size measures like odds ratios and Cramer's V.

What Are Categorical Data?

Categorical variables place observations into discrete groups rather than measuring them on a continuous scale.

Type	Examples	Key Property
Nominal	Blood type (A, B, AB, O), genotype (AA, AG, GG)	No natural ordering
Ordinal	Tumor grade (I, II, III, IV), pain scale (1-10)	Ordered but not equal intervals
Binary	Disease (yes/no), mutation (present/absent)	Two categories

The fundamental data structure for categorical analysis is the **contingency table** (also called a cross-tabulation):

	Risk Allele	No Risk Allele	Total
Alzheimer's	180	320	500
Control	120	380	500
Total	300	700	1000

2x2 Contingency Table: Observed (Expected)

	Risk Allele	No Risk Allele
Alzheimer's	180 (150.0)	320 (350.0)
Control	120 (150.0)	380 (350.0)
Total	300	700

Expected = (Row Total x Column Total) / Grand Total

E.g., $E(AD, Risk) = (500 \times 300) / 1000 = 150$

Chi-Square Test of Independence

The chi-square test asks: “Are these two categorical variables independent, or is there an association?”

How It Works

1. Compute **expected counts** under independence: $E = (\text{row total} \times \text{column total}) / \text{grand total}$
2. For each cell, compute $(\text{Observed} - \text{Expected})^2 / \text{Expected}$
3. Sum across all cells to get the chi-square statistic
4. Compare to the chi-square distribution with $df = (\text{rows} - 1)(\text{cols} - 1)$

For the Alzheimer’s example:

	Risk Allele	No Risk Allele
Expected (AD)	$500 \times 300 / 1000 = 150$	$500 \times 700 / 1000 = 350$
Expected (Ctrl)	$500 \times 300 / 1000 = 150$	$500 \times 700 / 1000 = 350$

The chi-square statistic measures how far the observed counts deviate from what independence predicts.

Assumptions

- Observations are independent
- Expected count in each cell is at least 5 (if not, use Fisher’s exact test)
- Sample is reasonably large

Common pitfall: The chi-square test requires *expected* counts of at least 5 in each cell, not *observed* counts. Check expected counts before interpreting results. When they are too small, Fisher’s exact test is the safe alternative.

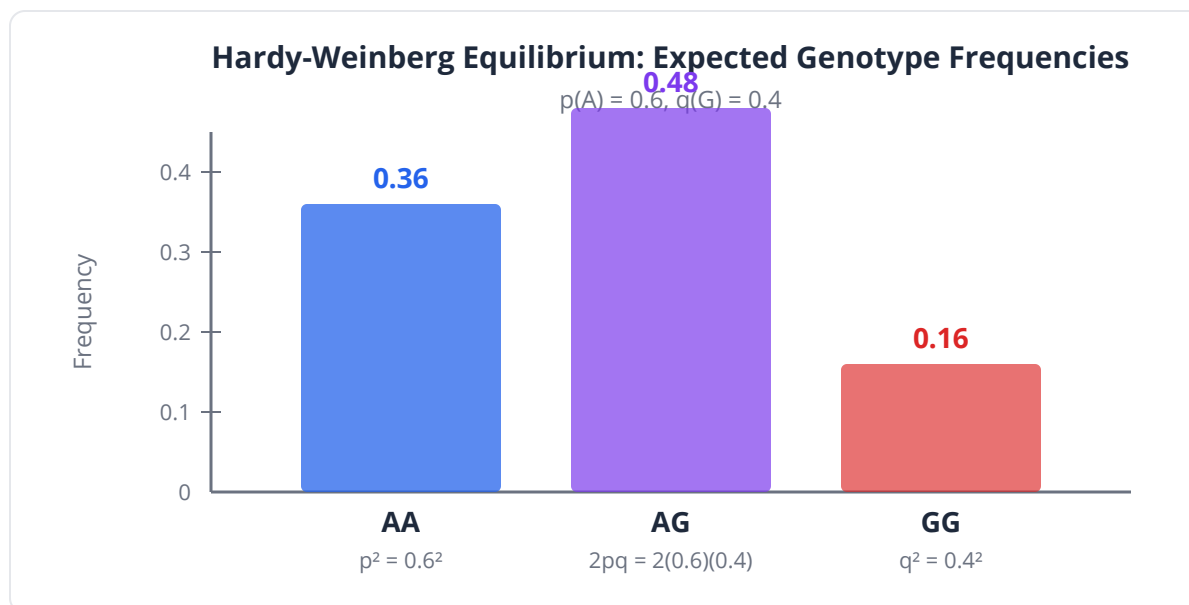
Chi-Square Goodness of Fit: Hardy-Weinberg

The chi-square test can also compare observed frequencies to a theoretical distribution. A critical application in genetics is testing **Hardy-Weinberg Equilibrium** (HWE).

For a biallelic locus with allele frequencies p (major) and q (minor), HWE predicts:

- AA: p^2
- Ag: $2pq$
- gg: q^2

If observed genotype counts deviate significantly from HWE expectations, it may indicate selection, population structure, genotyping error, or non-random mating.



Fisher's Exact Test

When sample sizes are small or expected counts fall below 5, the chi-square approximation is unreliable. Fisher's exact test computes the exact probability

using the hypergeometric distribution.

Fisher's exact test is computationally expensive for large tables but is the gold standard for 2x2 tables with small counts — common in rare variant studies and pilot experiments.

Clinical relevance: Regulatory agencies (FDA, EMA) often prefer Fisher's exact test for safety analyses where adverse events are rare and sample sizes are modest.

McNemar's Test: Paired Categorical Data

When observations are paired — the same patients tested before and after treatment, or the same samples tested with two diagnostic methods — McNemar's test is the correct choice.

	Test B Positive	Test B Negative
Test A Positive	a (both positive)	b (A+, B-)
Test A Negative	c (A-, B+)	d (both negative)

McNemar's test focuses on the **discordant pairs** (b and c):

$$\text{chi-square} = (b - c)^2 / (b + c)$$

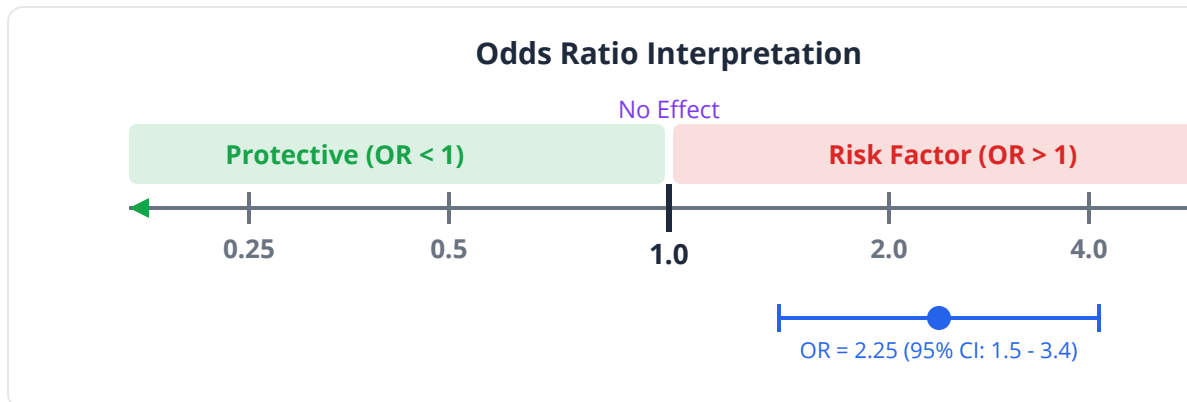
Measures of Association

Odds Ratio (OR)

The odds ratio quantifies the strength of association in a 2x2 table:

$$\text{OR} = (a \times d) / (b \times c)$$

OR	Interpretation
OR = 1	No association
OR > 1	Exposure increases odds of outcome
OR < 1	Exposure decreases odds of outcome
OR = 2.5	Exposed group has 2.5x the odds



Relative Risk (RR)

In prospective studies (cohorts), relative risk is often preferred:

$$RR = [a / (a+b)] / [c / (c+d)]$$

Key insight: Odds ratios approximate relative risk only when the outcome is rare (< 10%). For common outcomes, OR overestimates RR. Case-control studies can only estimate OR, not RR.

Cramer's V

A measure of association strength for tables larger than 2x2:

$$V = \sqrt{\text{chi-square} / (n \times \min(r-1, c-1))}$$

V ranges from 0 (no association) to 1 (perfect association).

V	Interpretation
0.1	Small association
0.3	Medium association
0.5	Large association

Proportion Test

Tests whether an observed proportion differs from an expected value, or whether two proportions differ from each other. Useful for comparing mutation frequencies, response rates, or allele frequencies between populations.

Categorical Tests in BioLang

Chi-Square Test: SNP-Disease Association

```
# Observed genotype counts near APOE
# Rows: Alzheimer's, Control
# Columns: Risk allele present, Risk allele absent
let observed = [180, 320, 120, 380]
let expected = [150.0, 350.0, 150.0, 350.0]

let result = chi_square(observed, expected)
print("=== Chi-Square Test of Independence ===")
print("Chi-square statistic: {result.statistic:.4}")
print("p-value: {result.p_value:.2e}")
print("Degrees of freedom: {result.df}")

# Effect size: Cramer's V from chi-square output
let n_total = 180 + 320 + 120 + 380
let v = sqrt(result.statistic / (n_total * min(2 - 1, 2 - 1)))
print("Cramer's V: {v:.4}")

# Odds ratio (inline: (a*d) / (b*c))
let a = 180
let b = 320
let c = 120
let d = 380
let or_val = (a * d) / (b * c)
print("\nOdds Ratio: {or_val:.3}")

# Relative risk (inline)
let rr = (a / (a + b)) / (c / (c + d))
print("Relative Risk: {rr:.3}")

if result.p_value < 0.05 {
    print("\nSignificant association between risk allele and
Alzheimer's")
}
```


Fisher's Exact Test: Rare Variant Study

```
# Rare loss-of-function variant in 200 cases and 200 controls
# Very small expected counts -> Fisher's exact test
let observed = [8, 192, 2, 198]

print("=== Fisher's Exact Test: Rare Variant ===")
print("Observed: 8/200 cases vs 2/200 controls carry the variant\n")

# Chi-square would be unreliable here
let chi_result = chi_square(observed, [5.0, 195.0, 5.0, 195.0])
print("Chi-square p-value: {chi_result.p_value:.4} (unreliable - low
expected counts)")

# Fisher's exact is the correct choice
let fisher_result = fisher_exact(8, 192, 2, 198)
print("Fisher's exact p-value: {fisher_result.p_value:.4}")

# Odds ratio (inline)
let or_val = (8 * 198) / (192 * 2)
print("Odds Ratio: {or_val:.2}")

# Note: CI includes 1.0, so despite the apparent 4x difference,
# the sample size is too small for significance
```

Hardy-Weinberg Equilibrium Test

```
# Genotype counts at a SNP locus
# Observed: AA=280, AG=430, GG=290 (total = 1000)
let obs_AA = 280
let obs_AG = 430
let obs_GG = 290
let n = obs_AA + obs_AG + obs_GG

# Estimate allele frequencies
let p = (2 * obs_AA + obs_AG) / (2 * n) # freq of A
let q = 1.0 - p                        # freq of G

print("Allele frequencies: p(A) = {p:.4}, q(G) = {q:.4}")

# Expected counts under HWE
let exp_AA = p * p * n
let exp_AG = 2 * p * q * n
let exp_GG = q * q * n

print("\nGenotype      | Observed | Expected (HWE)")
print("-----|-----|-----")
print("AA          | {obs_AA:>8} | {exp_AA:>14.1}")
print("AG          | {obs_AG:>8} | {exp_AG:>14.1}")
print("GG          | {obs_GG:>8} | {exp_GG:>14.1}")

# Chi-square goodness of fit (df=1 for HWE with 2 alleles)
let chi_sq = (obs_AA - exp_AA)^2 / exp_AA +
              (obs_AG - exp_AG)^2 / exp_AG +
              (obs_GG - exp_GG)^2 / exp_GG

# Use chi_square with expected frequencies
let observed = [obs_AA, obs_AG, obs_GG]
let expected = [exp_AA, exp_AG, exp_GG]
let result = chi_square(observed, expected)

print("\nChi-square = {result.statistic:.4}")
print("p-value = {result.p_value:.4}")

if result.p_value > 0.05 {
  print("Genotype frequencies are consistent with Hardy-Weinberg
Equilibrium")
} else {
  print("Significant deviation from HWE – investigate possible
causes")
}
```

McNemar's Test: Diagnostic Agreement

```
# Two diagnostic tests for TB applied to same 200 patients
# Test A (culture) vs Test B (PCR)
let table = [[85, 15], [10, 90]]
# 85: both positive, 90: both negative
# 15: A+/B-, 10: A-/B+

# McNemar's test: use chi_square() on discordant cells
let b_disc = 15 # A+/B-
let c_disc = 10 # A-/B+
let mcnemar_chi2 = (b_disc - c_disc) * (b_disc - c_disc) / (b_disc +
c_disc)
let mcnemar_p = 1.0 - pnorm(sqrt(mcnemar_chi2), 0, 1) * 2.0 #
approximate
# Or use chi_square on discordant cells
let result = chi_square([b_disc, c_disc], [12.5, 12.5])
print("=== McNemar's Test: Culture vs PCR ===")
print("Discordant pairs: A+/B- = 15, A-/B+ = 10")
print("Chi-square: {result.statistic:.4}")
print("p-value: {result.p_value:.4}")

if result.p_value > 0.05 {
  print("No significant difference between the two diagnostic
tests")
} else {
  print("The tests have significantly different detection rates")
}
```

Proportion Test: Comparing Mutation Frequencies

```
# EGFR mutation frequency in two populations
# Asian cohort: 120/300 (40%), European cohort: 45/300 (15%)
# Two-proportion test via chi_square
let observed = [120, 180, 45, 255]
let result = chi_square(observed, [82.5, 217.5, 82.5, 217.5])

print("=== Two-Proportion Test: EGFR Mutation Frequency ===")
print("Asian: 120/300 = 40%")
print("European: 45/300 = 15%")
print("Chi-square: {result.statistic:.4}")
print("p-value: {result.p_value:.2e}")
print("Difference: 25 percentage points")

# Visualize
let population_rates = table({
  "population": ["Asian", "European"],
  "mutation_frequency": [40.0, 15.0]
})
bar_chart(population_rates, {label: "population", value:
"mutation_frequency"})
```

Complete Workflow: Multi-Allelic Association

```
# Three genotypes at a pharmacogenomics locus
# and three drug response categories
let observed = [45, 30, 25, 35, 55, 60, 20, 15, 15]
let expected = [
  33.333, 33.333, 33.333,
  50.0, 50.0, 50.0,
  16.667, 16.667, 16.667
]
let row_labels = ["Poor", "Intermediate", "Ultra-rapid"]
let col_labels = ["AA", "AG", "GG"]

let result = chi_square(observed, expected)
print("=== Chi-Square: Genotype vs Drug Response ===")
print("Chi-square: {result.statistic:.4}")
print("p-value: {result.p_value:.4}")
print("df: {result.df}")

# Cramer's V from chi-square output
let n_total = 45 + 30 + 25 + 35 + 55 + 60 + 20 + 15 + 15
let v = sqrt(result.statistic / (n_total * min(3 - 1, 3 - 1)))
print("Cramer's V: {v:.4} (effect size)")

# Display the contingency table
print("\n          | AA   | AG   | GG   | Total")
print("-----|-----|-----|-----|-----")
for i in 0..3 {
  let offset = i * 3
  let total = observed[offset] + observed[offset + 1] +
observed[offset + 2]
  print("{row_labels[i]:<11}| {observed[offset]:>4} |
{observed[offset + 1]:>4} | {observed[offset + 2]:>4} | {total:>4}")
}
```

Python:

```
from scipy import stats
import numpy as np

# Chi-square test of independence
observed = np.array([[180, 320], [120, 380]])
chi2, p, dof, expected = stats.chi2_contingency(observed)
print(f"Chi-square = {chi2:.4f}, p = {p:.2e}")

# Fisher's exact test
odds, p = stats.fisher_exact([[8, 192], [2, 198]])
print(f"OR = {odds:.2f}, p = {p:.4f}")

# McNemar's test
from statsmodels.stats.contingency_tables import mcnemar
result = mcnemar(np.array([[85, 15], [10, 90]]), exact=False)
print(f"McNemar chi2 = {result.statistic:.4f}, p = {result.pvalue:.4f}")

# Proportion test
from statsmodels.stats.proportion import proportions_ztest
z, p = proportions_ztest([120, 45], [300, 300])
print(f"z = {z:.4f}, p = {p:.2e}")
```

R:

```
# Chi-square test
observed <- matrix(c(180, 120, 320, 380), nrow = 2)
chisq.test(observed)

# Fisher's exact test
fisher.test(matrix(c(8, 2, 192, 198), nrow = 2))

# McNemar's test
mcnemar.test(matrix(c(85, 10, 15, 90), nrow = 2))

# Odds ratio
library(epitools)
oddsratio(observed)

# Proportion test
prop.test(c(120, 45), c(300, 300))
```

Exercises

Exercise 1: SNP Association Study

A GWAS hit is validated in a replication cohort. Genotype counts:

	AA	AG	GG
Cases	150	200	50
Controls	180	160	60

```
let observed = [[150, 200, 50], [180, 160, 60]]

# TODO: Run chi-square test
# TODO: Compute Cramer's V
# TODO: Test HWE separately in cases and controls
# TODO: Interpret the results
```

Exercise 2: Fisher's Exact on Rare Mutations

In a rare disease study: 5/50 patients carry a mutation vs 1/100 controls.

```
let table = [[5, 45], [1, 99]]

# TODO: Why is Fisher's exact preferred here?
# TODO: Compute p-value and odds ratio
# TODO: What does the wide CI on the OR tell you?
```

Exercise 3: McNemar's for Treatment Response

A tumor is biopsied before and after chemotherapy. Response to an immunostaining marker:

	After: Positive	After: Negative
Before: Positive	40	25
Before: Negative	5	30

```
let table = [[40, 25], [5, 30]]
```

```
# TODO: Run McNemar's test
```

```
# TODO: What do the discordant pairs tell you?
```

```
# TODO: Is the marker significantly altered by chemotherapy?
```

Exercise 4: Multi-Population Allele Comparison

Compare allele frequencies of a pharmacogenomics variant across three populations. Use chi-square and Cramer's V, then create a bar chart of frequencies.

```
# Observed carrier counts out of 500 per population
```

```
let observed = [[210, 290], [150, 350], [85, 415]]
```

```
let populations = ["East Asian", "European", "African"]
```

```
# TODO: Chi-square test of independence
```

```
# TODO: Cramer's V
```

```
# TODO: Pairwise proportion tests with Bonferroni correction
```

```
# TODO: Bar chart of carrier frequencies
```

Key Takeaways

- The **chi-square test** evaluates whether two categorical variables are independent by comparing observed to expected counts
- **Fisher's exact test** is preferred when expected cell counts are below 5, common with rare variants
- **McNemar's test** handles paired categorical data (same subjects, two conditions)
- The **odds ratio** quantifies association strength; OR = 1 means no association
- **Relative risk** is preferred in cohort studies but cannot be computed from case-control designs
- **Cramer's V** measures association strength for tables larger than 2x2
- **Hardy-Weinberg equilibrium** testing uses chi-square goodness of fit to check for genotyping artifacts or population structure

- The **proportion test** compares frequencies between populations or against expected values
- Always check expected cell counts before using chi-square — use Fisher's exact when they are small

What's Next

You now have a powerful toolkit for individual tests. But in genomics, we never run just one test — we run thousands or millions simultaneously. Testing 20,000 genes means 1,000 false positives at $\alpha = 0.05$. Tomorrow we confront the multiple testing crisis head-on and learn the corrections that make genome-scale analysis possible, culminating in the volcano plot that has become the icon of differential expression analysis.

Day 12: The Multiple Testing Crisis — FDR and Correction

The Problem

Dr. Rachel Kim is a genomicist analyzing differential gene expression between tumor and normal tissue. She runs a Welch's t-test on each of 20,000 genes and finds 1,200 with $p < 0.05$. Exciting — until she does the arithmetic: 20,000 genes multiplied by a 5% false positive rate equals 1,000 genes that would appear “significant” purely by chance, even if not a single gene were truly differentially expressed.

Her 1,200 “hits” likely contain about 1,000 false positives and perhaps 200 true findings. That is a false discovery rate of over 80%. If she publishes this list and a validation lab tries to confirm the top 50 genes, roughly 40 will fail to replicate. Her scientific reputation — and the lab's funding — depends on solving this problem.

This is the **multiple testing crisis**, and it is the single most important statistical concept in genomics. Every differential expression study, every GWAS, every proteomics screen must address it. The solutions — Bonferroni correction, Benjamini-Hochberg FDR, and their relatives — are what make genome-scale analysis possible.

Making It Visceral: A Simulation

Before discussing theory, let us see the problem with our own eyes.

```

set_seed(42)
# Simulate 20,000 genes where NONE are truly different (complete
null)

let p_values = []
for i in 1..20000 {
  # Both groups drawn from the same distribution – no real
differences
  let group1 = rnorm(10, 0, 1)
  let group2 = rnorm(10, 0, 1)
  let result = ttest(group1, group2)
  p_values = append(p_values, result.p_value)
}

# Count "discoveries" at various thresholds
let sig_05 = p_values |> filter(|p| p < 0.05) |> len()
let sig_01 = p_values |> filter(|p| p < 0.01) |> len()
let sig_001 = p_values |> filter(|p| p < 0.001) |> len()

print("=== 20,000 Null Genes (No True Differences) ===")
print("'Significant' at p < 0.05: {sig_05} (expected: ~1000)")
print("'Significant' at p < 0.01: {sig_01} (expected: ~200)")
print("'Significant' at p < 0.001: {sig_001} (expected: ~20)")
print("\nEvery single one is a false positive!")

histogram(p_values, {title: "p-Value Distribution Under Complete
Null", x_label: "p-value", bins: 50})

```

Under the null hypothesis, p-values are **uniformly distributed** between 0 and 1. Exactly 5% will fall below 0.05 by definition. With 20,000 tests, that is 1,000 false alarms.

The Multiple Testing Disaster: 20,000 Null Tests

Every "significant" result is a false positive



20,000 genes (all truly null — no real differences)

● ~1,000 false positives ($p < 0.05$ by chance) ● ~19,000 true negatives

Family-Wise Error Rate (FWER)

The **family-wise error rate** is the probability of making at least one Type I error across all tests:

$$\text{FWER} = 1 - (1 - \alpha)^m$$

Where m is the number of tests.

Number of Tests (m)	FWER at $\alpha = 0.05$
1	5.0%
10	40.1%
100	99.4%
1,000	~100%
20,000	~100%

Bonferroni Correction

The simplest and most conservative approach: divide alpha by the number of tests.

Adjusted alpha = alpha / m

For 20,000 tests at alpha = 0.05: adjusted alpha = $0.05 / 20,000 = 2.5 \times 10^{-6}$.

Equivalently, multiply each p-value by m and compare to the original alpha:

p_adjusted = min(p x m, 1.0)

Strengths and Weaknesses

Property	Assessment
Controls FWER	Yes, strongly
Simple to compute	Yes
Power	Very low — misses many true effects
Assumes independence	Works regardless, but overly conservative for correlated tests

Common pitfall: Bonferroni is often TOO conservative for genomics. With 20,000 correlated gene expression measurements, it throws out far too many true positives. It is appropriate when you need to be extremely cautious (drug safety) or when you have few tests.

Holm's Step-Down Correction

Holm's method is uniformly more powerful than Bonferroni while still controlling FWER.

Procedure:

1. Sort p-values from smallest to largest: $p(1) \leq p(2) \leq \dots \leq p(m)$
2. For the i -th smallest p-value, compute adjusted p: $p(i) \times (m - i + 1)$
3. Enforce monotonicity: each adjusted p must be $\geq$ the previous
4. Reject all hypotheses whose adjusted p $< \alpha$

Holm's method is *always* at least as powerful as Bonferroni, and sometimes substantially more so.

False Discovery Rate (FDR): The Breakthrough

In 1995, Benjamini and Hochberg introduced a paradigm shift. Instead of controlling the probability of *any* false positive (FWER), they controlled the expected *proportion* of false positives among rejected hypotheses.

$$\text{FDR} = E[V / R]$$

Where V = number of false positives and R = total rejections.

If you set $\text{FDR} = 0.05$, you accept that about 5% of your “discoveries” may be false — a much more practical threshold for genomics than guaranteeing zero false positives.

Benjamini-Hochberg (BH) Procedure

Procedure:

1. Sort p-values from smallest to largest: $p(1) \leq p(2) \leq \dots \leq p(m)$
2. For each $p(i)$, compute the BH threshold: $(i / m) \times q$, where q is the desired FDR level
3. Find the largest i where $p(i) \leq (i / m) \times q$
4. Reject all hypotheses 1, 2, ..., i

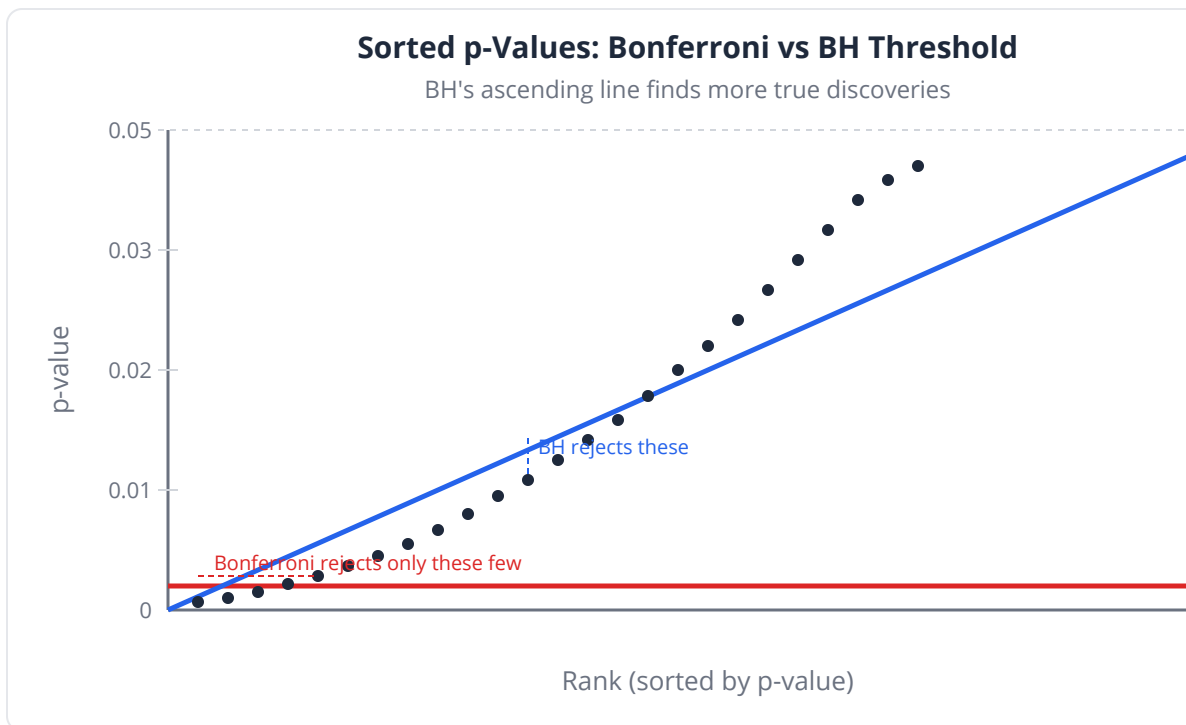
Equivalently, the BH-adjusted p-value (q-value) is:

$$q(i) = \min(p(i) \times m / i, 1.0) \text{ (with monotonicity enforced)}$$

Why BH Changed Genomics

Method	Controls	Typical Threshold	Power
Bonferroni	FWER	$p < 2.5e-6$ (for 20K tests)	Very low
Holm	FWER	Similar to Bonferroni	Slightly higher
BH (FDR)	FDR	$q < 0.05$	Much higher

Key insight: BH-FDR is the standard for differential expression, GWAS, proteomics, and almost all high-throughput biology. When a paper reports “genes with $FDR < 0.05$,” they almost always mean Benjamini-Hochberg adjusted p-values.



Other Correction Methods

Hochberg's Step-Up

Similar to Holm but slightly more powerful; assumes independence or positive dependence among tests.

Benjamini-Yekutieli (BY)

A conservative FDR procedure that works under *any* dependency structure between tests. Use when you have strong correlations (e.g., genes in the same pathway).

Choosing a Method

Method	Controls	Best For
Bonferroni	FWER	Few tests, safety-critical decisions
Holm	FWER	Same as Bonferroni but always more powerful
BH	FDR	Standard genomics, proteomics, any large-scale screen
Hochberg	FWER	Independent or positively dependent tests
BY	FDR	Strongly correlated tests, conservative FDR

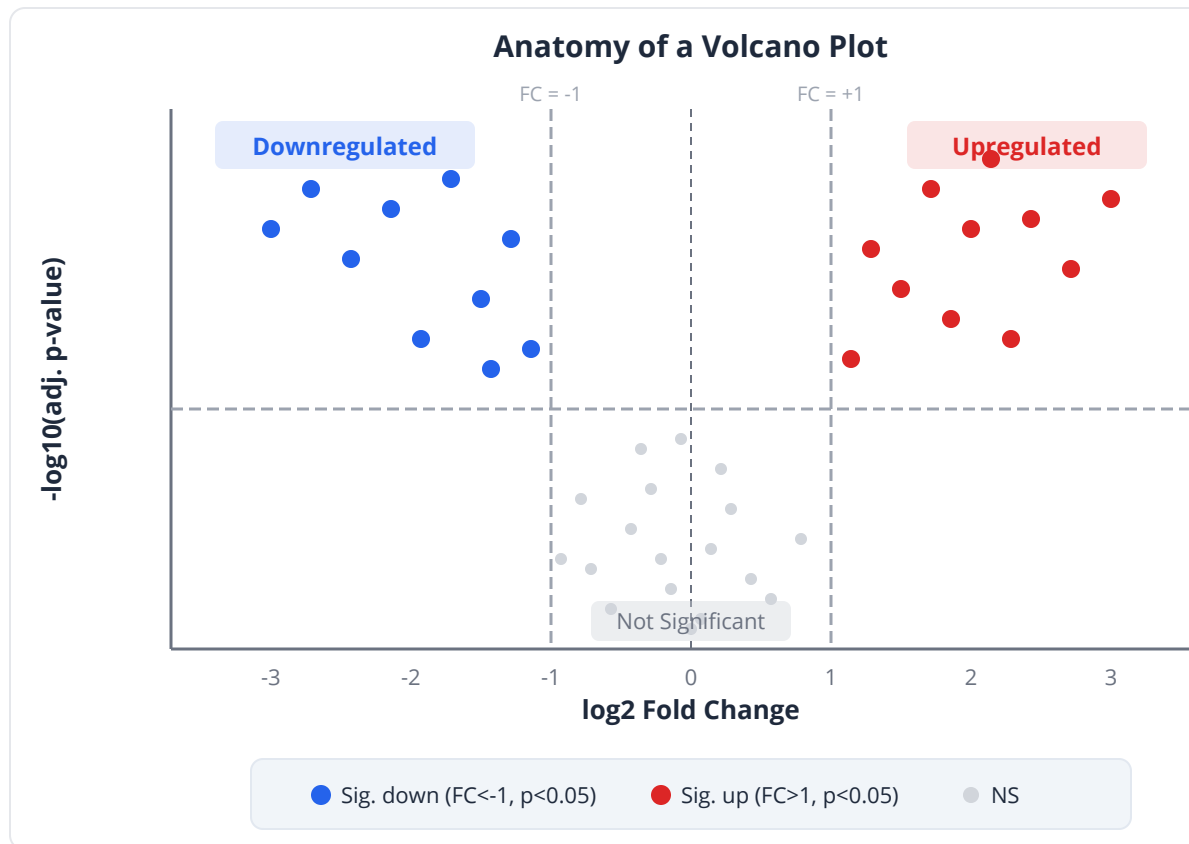
The Volcano Plot

The volcano plot is the most iconic visualization in differential expression analysis. It plots:

- **x-axis:** log2 fold change (effect size)
- **y-axis:** $-\log_{10}(\text{adjusted } p\text{-value})$ (statistical significance)

Genes in the upper corners are both statistically significant AND biologically meaningful — the ones you actually care about.

Region	Interpretation
Upper-right	Significantly upregulated (high FC, low p)
Upper-left	Significantly downregulated (high FC, low p)
Bottom center	Not significant (high p, any FC)
Upper center	Significant but small effect (low FC, low p)



Multiple Testing Correction in BioLang

Simulating the Crisis and Applying Corrections

```
set_seed(42)
# Simulate 20,000 genes: 18,000 null + 2,000 truly differential

let p_values = []
let is_true = [] # Ground truth: 1 = truly differential, 0 = null
let fold_changes = []

for i in 1..20000 {
  let group1 = rnorm(10, 0, 1)
  if i <= 2000 {
    # True differential: shifted mean
    let shift = rnorm(1, 2.0, 0.5)[0]
    let group2 = rnorm(10, shift, 1)
    is_true = append(is_true, 1)
  } else {
    # Null gene: no difference
    let group2 = rnorm(10, 0, 1)
    is_true = append(is_true, 0)
  }
  let result = ttest(group1, group2)
  p_values = append(p_values, result.p_value)
  fold_changes = append(fold_changes, mean(group2) - mean(group1))
}

print("Total genes: {len(p_values)}")
print("True DE genes: {is_true |> filter(|x| x == 1) |> len()}")
print("Null genes: {is_true |> filter(|x| x == 0) |> len()}")
```

Applying All Correction Methods

```
# Conceptual or diagnostic example; not directly executable.
# Apply multiple correction methods
let p_bonf = p_adjust(p_values, "bonferroni")
let p_holm = p_adjust(p_values, "holm")
let p_bh   = p_adjust(p_values, "BH")
let p_hoch = p_adjust(p_values, "hochberg")
let p_by   = p_adjust(p_values, "BY")

# Count discoveries at adjusted p < 0.05
let count_sig = |adj_p| adj_p |> filter(|p| p < 0.05) |> len()

print("=== Discoveries at Adjusted p < 0.05 ===")
print("Method          | Total Discoveries | True Pos | False Pos |
FDR")
print("-----|-----|-----|-----|----
----")

for method_name, adj_p in [
  ["Unadjusted ", p_values],
  ["Bonferroni ", p_bonf],
  ["Holm      ", p_holm],
  ["BH (FDR)  ", p_bh],
  ["Hochberg  ", p_hoch],
  ["BY       ", p_by]
] {
  let discoveries = []
  let true_pos = 0
  let false_pos = 0
  for j in 0..len(adj_p) {
    if adj_p[j] < 0.05 {
      discoveries = append(discoveries, j)
      if is_true[j] == 1 { true_pos = true_pos + 1 }
      else { false_pos = false_pos + 1 }
    }
  }
  let total = len(discoveries)
  let fdr = if total > 0 then false_pos / total else 0.0
  print("{method_name} | {total:>17} | {true_pos:>8} |
{false_pos:>9} | {fdr:>6.3}")
}
```

Drawing a Volcano Plot

```
# Volcano plot: the most important visualization in DE analysis
let log2_fc = fold_changes
let neg_log10_p = p_bh |> map(|p| if p > 0 then -log10(p) else 10)

# Classify genes
let colors = []
for i in 0..len(p_bh) {
  if p_bh[i] < 0.05 and abs(log2_fc[i]) > 1.0 {
    colors = append(colors, "significant")
  } else {
    colors = append(colors, "not_significant")
  }
}

let volcano_data = table({"log2fc": log2_fc, "pvalue": p_bh})
volcano(volcano_data, {fc_threshold: 1.0, p_threshold: 0.05})

# Count genes in each quadrant
let sig_up = 0
let sig_down = 0
let not_sig = 0
for i in 0..len(p_bh) {
  if p_bh[i] < 0.05 and log2_fc[i] > 1.0 { sig_up = sig_up + 1 }
  else if p_bh[i] < 0.05 and log2_fc[i] < -1.0 { sig_down = sig_down
+ 1 }
  else { not_sig = not_sig + 1 }
}
print("Significantly upregulated:    {sig_up}")
print("Significantly downregulated: {sig_down}")
print("Not significant:                {not_sig}")
```

Visualizing the BH Procedure

```
set_seed(42)
# Step-by-step BH procedure visualization

# Smaller example for clarity: 100 tests, 10 truly different
let p_vals = []
for i in 1..100 {
  let g1 = rnorm(10, 0, 1)
  let g2 = if i <= 10 {
    rnorm(10, 3, 1)
  } else {
    rnorm(10, 0, 1)
  }
  p_vals = append(p_vals, ttest(g1, g2).p_value)
}

# Sort p-values
let sorted_p = sort(p_vals)
let bh_thresholds = range(1, 101) |> map(|i| i / 100 * 0.05)

# Plot sorted p-values against BH thresholds
scatter(range(1, 101), sorted_p)

let bh_adjusted = p_adjust(p_vals, "BH")
let n_sig = bh_adjusted |> filter(|p| p < 0.05) |> len()
print("BH discoveries (FDR < 0.05): {n_sig}")
```

p-Value Histograms: Diagnostic Tool

```
set_seed(42)
# A well-behaved p-value histogram tells you a lot
# Uniform = all null; spike at 0 = true signal exists

# Scenario 1: All null (should be uniform)
let null_ps = []
for i in 1..5000 {
  let g1 = rnorm(10, 0, 1)
  let g2 = rnorm(10, 0, 1)
  null_ps = append(null_ps, ttest(g1, g2).p_value)
}

# Scenario 2: 20% true signal (spike near 0 + uniform)
let mixed_ps = []
for i in 1..5000 {
  let g1 = rnorm(10, 0, 1)
  let g2 = if i <= 1000 {
    rnorm(10, 2, 1)
  } else {
    rnorm(10, 0, 1)
  }
  mixed_ps = append(mixed_ps, ttest(g1, g2).p_value)
}

histogram(null_ps, {title: "All Null: Uniform p-value Distribution",
x_label: "p-value", bins: 20})

histogram(mixed_ps, {title: "20% True Signal: Spike Near Zero +
Uniform Background", x_label: "p-value", bins: 20})
```

Python:

```

from statsmodels.stats.multitest import multipletests
import numpy as np

# Simulate p-values (example with 1000 tests)
np.random.seed(42)
p_values = np.random.uniform(0, 1, 1000)
p_values[:100] = np.random.uniform(0, 0.01, 100) # 100 true signals

# Apply corrections
reject_bonf, padj_bonf, _, _ = multipletests(p_values,
method='bonferroni')
reject_holm, padj_holm, _, _ = multipletests(p_values,
method='holm')
reject_bh, padj_bh, _, _ = multipletests(p_values, method='fdr_bh')

print(f"Bonferroni: {reject_bonf.sum()} discoveries")
print(f"Holm:      {reject_holm.sum()} discoveries")
print(f"BH (FDR):  {reject_bh.sum()} discoveries")

```

R:

```

# Simulate p-values
set.seed(42)
p_values <- c(runif(100, 0, 0.01), runif(900, 0, 1))

# Apply corrections (all built-in)
p_bonf <- p.adjust(p_values, method = "bonferroni")
p_holm <- p.adjust(p_values, method = "holm")
p_bh   <- p.adjust(p_values, method = "BH")
p_by   <- p.adjust(p_values, method = "BY")

cat("Bonferroni:", sum(p_bonf < 0.05), "\n")
cat("Holm:      ", sum(p_holm < 0.05), "\n")
cat("BH (FDR):  ", sum(p_bh < 0.05), "\n")
cat("BY:       ", sum(p_by < 0.05), "\n")

# Volcano plot (using EnhancedVolcano)
library(EnhancedVolcano)
EnhancedVolcano(results, lab = rownames(results),
  x = 'log2FoldChange', y = 'padj',
  pCutoff = 0.05, FCcutoff = 1.0)

```


Exercises

Exercise 1: Simulate and Count

Simulate 10,000 tests where all genes are null (no true differences). Verify that approximately 500 have $p < 0.05$, 100 have $p < 0.01$, and 10 have $p < 0.001$.

```
# TODO: Simulate 10,000 null t-tests
# TODO: Count  $p < 0.05$ ,  $p < 0.01$ ,  $p < 0.001$ 
# TODO: Plot the p-value histogram (should be uniform)
```

Exercise 2: Compare Correction Power

Simulate 5,000 genes (500 truly DE with $\log_2FC = 1.5$, 4,500 null). Apply Bonferroni, Holm, and BH. For each, compute: (a) total discoveries, (b) true positives, (c) false positives, (d) actual FDR.

```
# TODO: Simulate 5,000 genes (500 DE + 4,500 null)
# TODO: Apply all three corrections
# TODO: Build a comparison table
# TODO: Which method gives the best balance of power and error control?
```

Exercise 3: Build a Volcano Plot

Using the simulation from Exercise 2, create a volcano plot. Color genes as “significant” (BH-adjusted $p < 0.05$ AND $|\log_2FC| > 1$) versus “not significant.”

```
# TODO: Use fold changes and BH-adjusted p-values from Exercise 2
# TODO: Create a volcano plot with appropriate thresholds
# TODO: Count genes in each quadrant
```

Exercise 4: p-Value Histogram Diagnostics

Generate p-value histograms for three scenarios: (a) 100% null genes, (b) 10% true DE genes, (c) 50% true DE genes. Describe the characteristic shape of each and explain what you would look for in real data.

```
# TODO: Scenario A — all null
# TODO: Scenario B — 10% DE
# TODO: Scenario C — 50% DE
# TODO: Create histograms for each
# TODO: Describe the shape and what it tells you
```

Exercise 5: When Does Bonferroni Make Sense?

You are testing 5 pre-specified candidate genes (not genome-wide). Apply Bonferroni and BH to these 5 p-values: [0.008, 0.023, 0.041, 0.062, 0.110]. How do the results differ? When would you prefer Bonferroni here?

```
let p_vals = [0.008, 0.023, 0.041, 0.062, 0.110]

# TODO: Apply Bonferroni and BH
# TODO: Which genes are significant under each method?
# TODO: Argue for which method is more appropriate for 5 candidate
genes
```

Key Takeaways

- Testing m hypotheses at $\alpha = 0.05$ yields approximately $m \times 0.05$ false positives — devastating for genomics
- **Bonferroni** correction ($p \times m$) controls FWER but is often too conservative for large-scale studies
- **Holm's** step-down method is always at least as powerful as Bonferroni and should be preferred
- **Benjamini-Hochberg (BH)** FDR correction is the standard for genomics: it controls the expected proportion of false discoveries rather than the probability of any false discovery
- At FDR = 0.05, you accept that about 5% of discoveries may be false — a practical trade-off for high-throughput biology
- **p-value histograms** are essential diagnostics: uniform = all null, spike at zero = true signal present
- The **volcano plot** ($-\log_{10}$ adjusted p vs $\log_2$ fold change) is the standard visualization for differential expression: genes in the upper corners are

- both significant and biologically meaningful
- Always report which correction method you used — “ $p < 0.05$ ” means very different things with and without adjustment

What's Next

With the multiple testing crisis solved, we have completed the core toolkit of statistical hypothesis testing. Week 3 shifts to modeling and relationships: tomorrow we explore correlation — how to measure and test whether two continuous variables move together, from gene expression co-regulation to dose-response curves. This transition from “are groups different?” to “how are variables related?” opens the door to regression, prediction, and the modeling approaches that power modern biostatistics.

Day 13: Correlation — Finding Relationships

The Problem

Dr. Sarah Kim is studying breast cancer transcriptomics across 200 tumor samples. She notices that **BRCA1** and **BARD1** expression seem to rise and fall together — when one is high, the other tends to be high too. Exciting! These genes encode proteins that form a heterodimer critical for DNA repair.

But her collaborator raises a concern: “Both genes are upregulated in rapidly dividing cells. Couldn’t **cell proliferation** be driving both signals independently? You might be seeing a spurious association.”

Sarah needs to:

1. **Quantify** how strongly BRCA1 and BARD1 co-vary
2. **Determine** whether the relationship is statistically significant
3. **Control** for the confounding effect of proliferation
4. **Visualize** relationships across an entire panel of DNA repair genes

This is the domain of **correlation analysis** — one of the most used (and most misused) tools in all of biology.

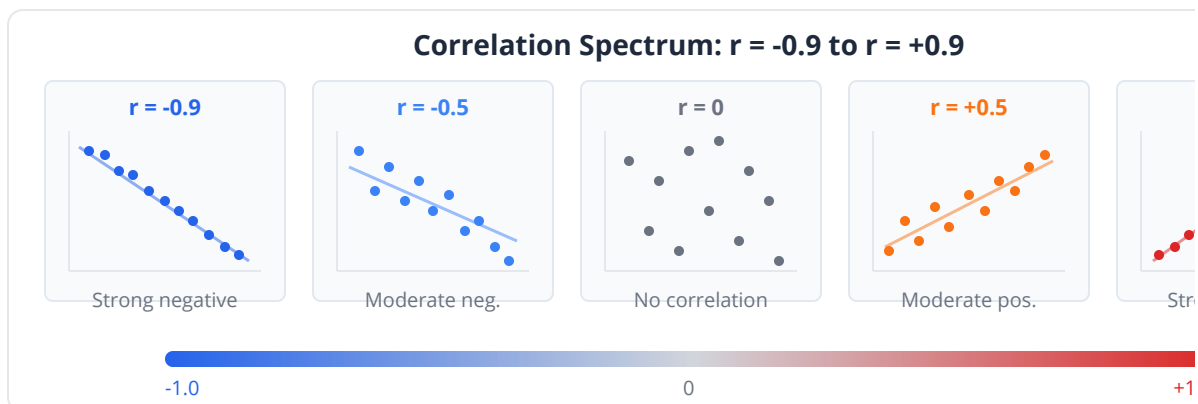
What Is Correlation?

Correlation measures the **strength and direction** of the relationship between two variables. It answers: “When one variable goes up, does the other tend to go up, go down, or do nothing?”

The correlation coefficient ranges from **-1 to +1**:

Value	Interpretation	Example
+1.0	Perfect positive	Identical twins' heights
+0.7 to +0.9	Strong positive	BRCA1 and BARD1 expression
+0.3 to +0.7	Moderate positive	BMI and blood pressure
-0.3 to +0.3	Weak / none	Shoe size and IQ
-0.3 to -0.7	Moderate negative	Exercise and resting heart rate
-0.7 to -1.0	Strong negative	Tumor suppressor vs. proliferation rate
-1.0	Perfect negative	Altitude and air pressure

Key insight: Correlation is symmetric — the correlation between X and Y is the same as between Y and X. It doesn't imply direction or causation.



Three Types of Correlation

1. Pearson's r: The Linear Standard

Pearson's correlation coefficient measures **linear** association:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Assumptions:

- Both variables are continuous
- Relationship is approximately linear
- Data are roughly normally distributed (for inference)
- No extreme outliers (a single outlier can flip the sign)

Strengths: Most powerful when assumptions are met. Directly related to R^2 in linear regression.

Weakness: Sensitive to outliers. Misses non-linear relationships entirely.

2. Spearman's ρ (rho): The Rank-Based Alternative

Spearman's correlation operates on **ranks** rather than raw values. It measures **monotonic** association — whether Y tends to increase (or decrease) as X increases, even if not linearly.

How it works:

1. Rank each variable separately (1, 2, 3, ...)
2. Compute Pearson's r on the ranks

Assumptions:

- Ordinal or continuous data
- Monotonic relationship (doesn't need to be linear)

Strengths: Robust to outliers. Works with non-normal distributions. Handles log-scale expression data naturally.

Weakness: Less powerful than Pearson when linearity holds.

3. Kendall's τ (tau): The Most Robust

Kendall's tau counts **concordant** and **discordant** pairs:

$$\tau = \frac{(\text{concordant pairs}) - (\text{discordant pairs})}{\binom{n}{2}}$$

A pair (i, j) is concordant if both $x_i > x_j$ and $y_i > y_j$ (or both less). It's discordant if they disagree.

Strengths: Most robust to outliers. Better for small samples. Has clearer probabilistic interpretation.

Weakness: Computationally slower for large datasets. Values tend to be smaller than Pearson/Spearman for the same data.

Decision Table: Which Correlation to Use?

Situation	Best Choice	Why
Both variables normal, linear relationship	Pearson	Most powerful
Skewed data (e.g., raw gene expression)	Spearman	Rank-based, outlier-robust
Small sample (n < 30)	Kendall	Most reliable for small n
Ordinal data (e.g., tumor grade 1-4)	Spearman or Kendall	Ranks are appropriate
Suspect outliers	Spearman or Kendall	Rank-based methods resist outliers
Large RNA-seq dataset, log-transformed	Pearson or Spearman	Both work well after log transform
Survival times with censoring	Kendall	Handles ties from censoring

Common pitfall: Using Pearson on raw RNA-seq counts. These are heavily right-skewed — a few highly expressed genes dominate the correlation. Always log-transform first or use Spearman.

Partial Correlation: Controlling for Confounders

Standard correlation between X and Y can be **inflated** or **deflated** by a confounding variable Z that influences both. Partial correlation removes the effect of Z:

$$r_{XY \cdot Z} = \frac{r_{XY} - r_{XZ} \cdot r_{YZ}}{\sqrt{(1 - r_{XZ}^2)(1 - r_{YZ}^2)}}$$

Example: BRCA1 and BARD1 might correlate at $r = 0.85$. But if cell proliferation (MKI67 expression) drives both, the partial correlation controlling for MKI67

might drop to $r = 0.45$ — still real, but weaker.

Clinical relevance: In pharmacogenomics, two drug targets may appear correlated simply because both are overexpressed in a particular cancer subtype. Partial correlation controlling for subtype reveals whether the targets have an independent relationship.

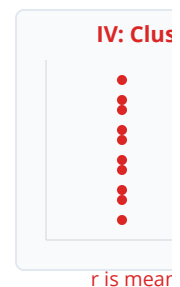
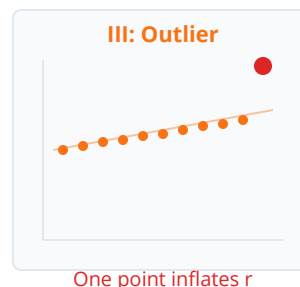
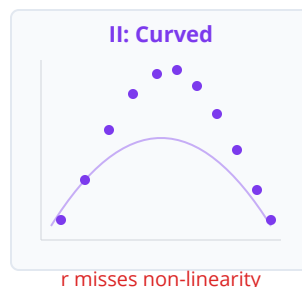
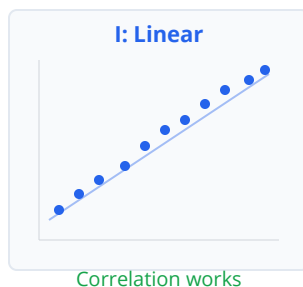
Anscombe's Quartet: Why You Must Visualize

In 1973, Francis Anscombe created four datasets with **identical** Pearson correlations ($r = 0.816$), identical means, and identical regression lines — but wildly different patterns:

Dataset	Pattern	Lesson
I	Normal linear	Correlation works correctly
II	Perfect curve	r misses non-linearity
III	Tight line + one outlier	Single point inflates r
IV	Vertical cluster + outlier	r is meaningless

The lesson: Never trust a correlation coefficient without a scatter plot.

Anscombe's Quartet: All Have $r = 0.82$ — But Look Different!



All four datasets: $r = 0.82$, same mean, same variance, same regression line

Lesson: ALWAYS visualize before trusting a correlation coefficient

Correlation Matrix and Heatmaps

When studying many variables simultaneously, a **correlation matrix** shows all pairwise correlations. For p variables, this is a $p \times p$ symmetric matrix with 1s on the diagonal.

For genomics, this is invaluable for:

- Identifying co-expression modules
- Detecting batch effects (technical variables cluster together)
- Revealing pathway relationships

A **heatmap** visualization uses color intensity to represent correlation strength, making patterns immediately visible across dozens or hundreds of variables.

Testing Significance: Is This Correlation Real?

A correlation of $r = 0.3$ might be noise in 10 samples but highly significant in 1000. The **correlation test** evaluates:

- H_0 : $\rho = 0$ (no correlation in the population)
- H_a : $\rho \neq 0$

The test statistic follows a t-distribution with $n-2$ degrees of freedom:

$$t = r \sqrt{\frac{n-2}{1-r^2}}$$

Key insight: With large n (common in genomics), even tiny correlations become “significant.” A correlation of $r = 0.05$ is significant at $p < 0.05$ when $n > 1500$. Always report both the coefficient AND the p-value, and judge practical significance by the magnitude of r .

Correlation ≠ Causation

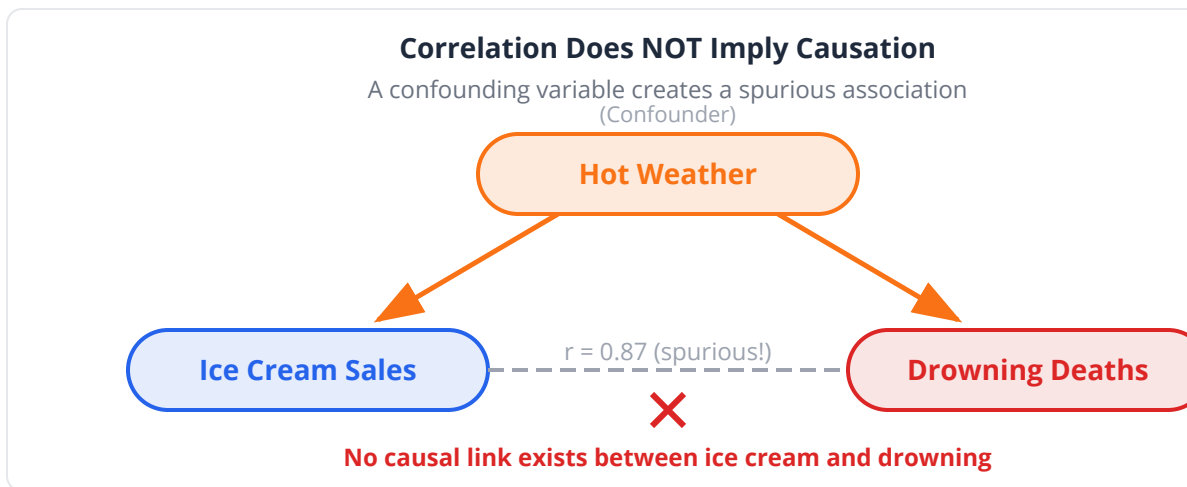
This cannot be overstated. Correlation tells you variables **co-vary** — nothing more.

Classic examples in biology:

- Ice cream sales correlate with drowning deaths (confounder: hot weather)
- Shoe size correlates with reading ability in children (confounder: age)
- Stork population correlates with birth rate across European countries (confounder: rural vs. urban)

To establish causation, you need:

1. **Temporal precedence** — cause precedes effect
2. **Experimental manipulation** — perturb X, observe Y change
3. **Elimination of confounders** — no third variable explains both
4. **Biological mechanism** — plausible pathway



Correlation in BioLang

Basic Correlations

```
set_seed(42)
fn add_scaled(signal, scale, noise) {
  zip(signal, noise) |> map(|pair| pair[0] * scale + pair[1])
}
# Generate tumor expression data for 200 samples
let n = 200

# Simulate BRCA1 and BARD1 with true correlation + noise
let proliferation = rnorm(n, 10, 3)
let brca1 = add_scaled(proliferation, 0.8, rnorm(n, 5, 2))
let bard1 = add_scaled(proliferation, 0.7, rnorm(n, 4, 2))

# Pearson correlation – assumes linearity
let r_pearson = cor(brca1, bard1)
print("Pearson r: {r_pearson}") # ~0.78

# Spearman rank correlation – monotonic, robust
let r_spearman = spearman(brca1, bard1)
print("Spearman ρ: {r_spearman}") # ~0.76

# Kendall tau – concordant/discordant pairs, most robust
let r_kendall = kendall(brca1, bard1)
print("Kendall τ: {r_kendall}") # ~0.57 (typically smaller)
```

Statistical Testing

```
# Test whether correlation is significantly different from zero
# cor() returns the coefficient; use a t-test transformation for p-value
let r = cor(brca1, bard1)
let t_stat = r * sqrt((n - 2) / (1 - r * r))
print("r = {r}, t = {t_stat}")

# Spearman returns {coefficient, pvalue}
let spearman_result = spearman(brca1, bard1)
print("Spearman ρ = {spearman_result}")
```

Partial Correlation: Removing Confounders

```
set_seed(42)
# BRCA1-BARD1 correlation controlling for proliferation (MKI67)
let mki67 = add_scaled(proliferation, 1.0, rnorm(n, 0, 1))

# Raw correlation
let r_raw = cor(brca1, bard1)
print("Raw Pearson r: {r_raw}") # ~0.78

# Partial correlation controlling for MKI67
# Compute manually: regress out confounder from both variables
let r_xz = cor(brca1, mki67)
let r_yz = cor(bard1, mki67)
let r_partial = (r_raw - r_xz * r_yz) / sqrt((1 - r_xz * r_xz) * (1 - r_yz * r_yz))
print("Partial r (controlling MKI67): {r_partial}") # lower -
confounder removed

# The difference reveals how much of the BRCA1-BARD1
# association was driven by shared proliferation signal
print("Reduction: {((r_raw - r_partial) / r_raw * 100) |>
round(1)}%")
```

Correlation Matrix and Heatmap

```
set_seed(42)
# Build expression matrix for 8 DNA repair genes
let base = rnorm(200, 0, 1)

let genes = {
  "BRCA1": add_scaled(base, 0.8, rnorm(200, 10, 2)),
  "BARD1": add_scaled(base, 0.7, rnorm(200, 8, 2)),
  "RAD51": add_scaled(base, 0.6, rnorm(200, 12, 3)),
  "PALB2": add_scaled(base, 0.5, rnorm(200, 9, 2)),
  "ATM": rnorm(200, 11, 3),
  "TP53": add_scaled(base, -0.4, rnorm(200, 15, 4)),
  "MDM2": add_scaled(base, -0.3, rnorm(200, 7, 2)),
  "GAPDH": rnorm(200, 20, 1)
}

# Compute pairwise correlations
let gene_names = ["BRCA1", "BARD1", "RAD51", "PALB2", "ATM", "TP53",
  "MDM2", "GAPDH"]
for i in 0..8 {
  for j in (i+1)..8 {
    let r = cor(genes[gene_names[i]], genes[gene_names[j]])
    print("{gene_names[i]} vs {gene_names[j]}: r = {r |>
round(3)}")
  }
}

# Visualize as heatmap – instantly reveals co-expression modules
heatmap(table(genes), {title: "DNA Repair Gene Co-Expression",
color_scale: "RdBu"})
```

Visualizing with Scatter Plots

```
# Scatter plot with correlation annotation
let scatter_data = table({"BRCA1": brca1, "BARD1": bard1})
plot(scatter_data, {type: "scatter", x: "BRCA1", y: "BARD1",
  title: "BRCA1 vs BARD1 Co-Expression",
  x_label: "BRCA1 Expression (log2 FPKM)",
  y_label: "BARD1 Expression (log2 FPKM)"})
```

Demonstrating Anscombe's Quartet Effect

```
set_seed(42)
# Two datasets with identical Pearson r but different patterns
let x_linear = rnorm(100, 10, 3)
let y_linear = add_scaled(x_linear, 0.5, rnorm(100, 0, 2))

let x_curve = rnorm(100, 10, 5)
let y_curve = zip(x_curve, rnorm(100, 0, 1))
  |> map(|pair| (pair[0] - 10) ** 2 / 10 + pair[1])

print("Linear: Pearson r = {cor(x_linear, y_linear)}")
print("Curved: Pearson r = {cor(x_curve, y_curve)}")
print("Linear: Spearman ρ = {spearman(x_linear, y_linear)}")
print("Curved: Spearman ρ = {spearman(x_curve, y_curve)}")

# Spearman catches the monotonic but non-linear pattern
# Always plot your data!
```

Python:

```
import numpy as np
from scipy import stats
import seaborn as sns
import matplotlib.pyplot as plt

# Pearson
r, p = stats.pearsonr(brca1, bard1)

# Spearman
rho, p = stats.spearmanr(brca1, bard1)

# Kendall
tau, p = stats.kendalltau(brca1, bard1)

# Partial correlation (using pingouin)
import pingouin as pg
partial = pg.partial_corr(data=df, x='BRCA1', y='BARD1',
covar='MKI67')

# Correlation matrix heatmap
corr = df.corr(method='spearman')
sns.heatmap(corr, annot=True, cmap='RdBu_r', center=0)
```


R:

```
# Pearson
cor.test(brca1, bard1, method = "pearson")

# Spearman
cor.test(brca1, bard1, method = "spearman")

# Kendall
cor.test(brca1, bard1, method = "kendall")

# Partial correlation (using ppcor)
library(ppcor)
pcor.test(brca1, bard1, mki67)

# Correlation matrix heatmap
library(corrplot)
corrplot(cor(gene_matrix, method = "spearman"),
          method = "color", type = "upper")
```

Exercises

Exercise 1: Compare Three Methods

Compute Pearson, Spearman, and Kendall correlations for the following gene pairs. Which method gives the most different result, and why?

```
set_seed(42)
let n = 150

# Gene pair 1: linear relationship with outliers
let gene_a = rnorm(n, 8, 2)
let gene_b = add_scaled(gene_a, 0.6, rnorm(n, 3, 1))

# Add 5 extreme outliers
# (imagine contaminated samples)

# Compute all three correlations for gene_a vs gene_b
# Which is most affected by outliers?
```

Exercise 2: Partial Correlation in Drug Response

Three variables are measured across 100 cancer cell lines: drug sensitivity (IC50), target gene expression, and cell doubling time. The target gene and IC50 appear correlated. Is the relationship real, or driven by growth rate?

```
set_seed(42)
let n = 100
let growth_rate = rnorm(n, 24, 6)

let target_expr = add_scaled(growth_rate, 0.5, rnorm(n, 10, 3))
let ic50 = add_scaled(growth_rate, -0.3, rnorm(n, 50, 10))

# 1. Compute raw correlation between target_expr and ic50
# 2. Compute partial correlation controlling for growth_rate
# 3. What fraction of the apparent association was confounded?
```

Exercise 3: Build and Interpret a Heatmap

Create a correlation heatmap for the following gene expression panel. Identify which genes cluster into co-expression modules.

```
set_seed(42)
let n = 300

# Simulate 3 biological modules:
# Module 1 (immune): CD8A, GZMB, PRF1, IFNG
# Module 2 (proliferation): MKI67, TOP2A, PCNA
# Module 3 (housekeeping): GAPDH, ACTB

# Create correlated expression within modules
# and weak/no correlation between modules
# Compute pairwise cor() and visualize with heatmap
# Which modules emerge from the clustering?
```

Exercise 4: Significance vs. Magnitude

Generate datasets with $n = 20$ and $n = 2000$. Show that a weak correlation ($r \approx 0.1$) is non-significant with small n but highly significant with large n . Argue why

the p-value alone is misleading.

```
set_seed(42)
# Small sample: n = 20, weak correlation
let x_small = rnorm(20, 0, 1)
let y_small = add_scaled(x_small, 0.1, rnorm(20, 0, 1))

# Large sample: n = 2000, same weak correlation
let x_large = rnorm(2000, 0, 1)
let y_large = add_scaled(x_large, 0.1, rnorm(2000, 0, 1))

# Compute cor() on both and test significance
# Compare p-values and correlation magnitudes
# What should you report?
```

Exercise 5: Anscombe Challenge

Create two synthetic gene-pair datasets where Pearson r is nearly identical (~ 0.7) but scatter plots reveal completely different biology — one linear, one with a threshold effect (flat then rising). Show that Spearman catches the difference.

Key Takeaways

- **Pearson** measures linear association; **Spearman** measures monotonic (rank-based); **Kendall** counts concordant pairs and is most robust
- Correlation ranges from -1 to +1; the sign indicates direction, the magnitude indicates strength
- **Always visualize** — identical correlations can hide wildly different patterns (Anscombe's quartet)
- **Partial correlation** removes the effect of confounders, revealing true associations
- With large genomics datasets, tiny correlations become "significant" — always report the **magnitude** alongside the p-value
- **Correlation is not causation** — co-expression does not imply co-regulation or functional relationship

- Spearman is generally the safest default for gene expression data

What's Next

Now that we can quantify relationships between two variables, Day 14 takes the next step: **using one variable to predict another**. We'll build our first linear regression models, learning to fit lines, interpret slopes, and critically evaluate whether our predictions are trustworthy.

Day 14: Linear Regression — Prediction from Data

The Problem

Dr. James Park is a pharmacogenomicist working with the NCI-60 cell line panel — 60 cancer cell lines spanning 9 tissue types. He has gene expression data and drug sensitivity measurements (IC50 values) for each line. His question: **Can we predict how sensitive a cell line will be to a new kinase inhibitor based on its expression of the drug's target gene?**

He knows the target gene and IC50 seem correlated (Day 13 confirmed $r = -0.72$). But correlation just says “they move together.” James needs to go further: given a **new cell line** with a known expression level, what IC50 should he **predict**? And how confident should he be?

This is the leap from **association** to **prediction** — the domain of linear regression.

What Is Linear Regression?

Linear regression fits a straight line through data points to model the relationship between a **predictor** (X) and a **response** (Y):

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Term	Meaning	Biological Example
Y	Response variable	Drug IC50
X	Predictor variable	Target gene expression
β_0	Intercept (Y when X = 0)	Baseline sensitivity
β_1	Slope (change in Y per unit X)	Sensitivity per expression unit
ε	Error (noise)	Biological + technical variation

The method of **least squares** finds the β_0 and β_1 that minimize the sum of squared residuals:

$$\min_{\beta_0, \beta_1} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Key insight: Regression has a clear asymmetry that correlation lacks. X predicts Y — there is a designated predictor and a designated outcome. The regression of Y on X is NOT the same as X on Y.



From Correlation to Regression

Correlation and simple regression are intimately linked:

Metric	What It Tells You
Pearson r	Strength and direction of linear association
$R^2 = r^2$	Proportion of variance in Y explained by X
β_1 (slope)	How much Y changes per unit change in X
p-value of β_1	Whether the slope is significantly different from zero

If $r = -0.72$, then $R^2 = 0.52$, meaning 52% of the variation in IC50 is explained by target expression. The remaining 48% is unexplained — due to other genes, pathway redundancy, or noise.

Interpreting Regression Output

A regression produces a table of coefficients:

Term	Estimate	Std. Error	t-value	p-value
Intercept (β_0)	85.3	6.2	13.8	< 0.001
Expression (β_1)	-4.7	0.8	-5.9	< 0.001

Reading this:

- **Intercept:** When expression = 0, predicted IC50 is 85.3 μM (often not biologically meaningful)
- **Slope:** Each 1-unit increase in expression decreases IC50 by 4.7 μM (more expression $\rightarrow$ more sensitive)
- **p-value:** The slope is significantly different from zero — expression truly predicts sensitivity
- **R^2 :** How well the line fits overall

Common pitfall: The intercept often represents an extrapolation beyond the data range. If expression ranges from 5-15, interpreting “IC50 when expression = 0” is meaningless. Focus on the slope.

Prediction: Point Estimates and Intervals

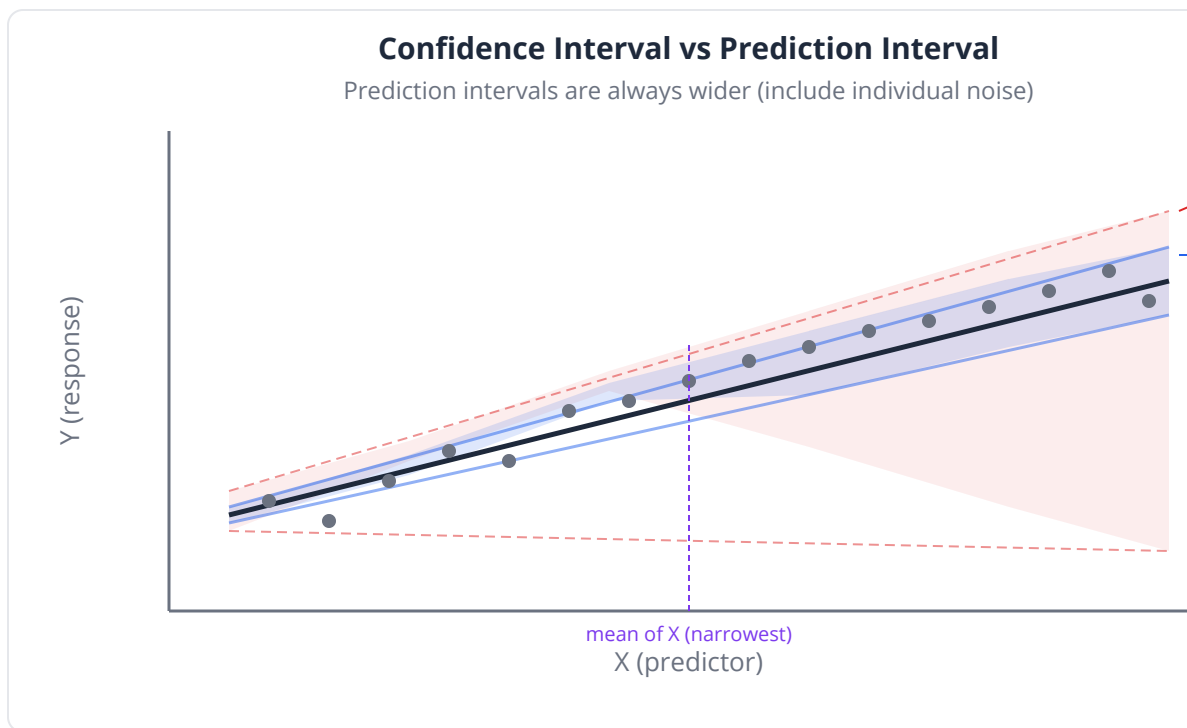
Once we have a fitted model, we can predict Y for new X values. But predictions come with two types of uncertainty:

Confidence Interval (for the mean response): “We’re 95% confident the **average** IC50 for all cell lines with expression = 10 lies in this range.”

Prediction Interval (for a single new observation): “We’re 95% confident the IC50 for **one specific new cell line** with expression = 10 lies in this range.”

Prediction intervals are always wider than confidence intervals because they include individual-level noise (ϵ).

Clinical relevance: In precision medicine, you’re usually predicting for **one patient** (prediction interval), not the population average (confidence interval). The prediction interval honestly reflects your uncertainty.



Residual Analysis: Checking Your Model

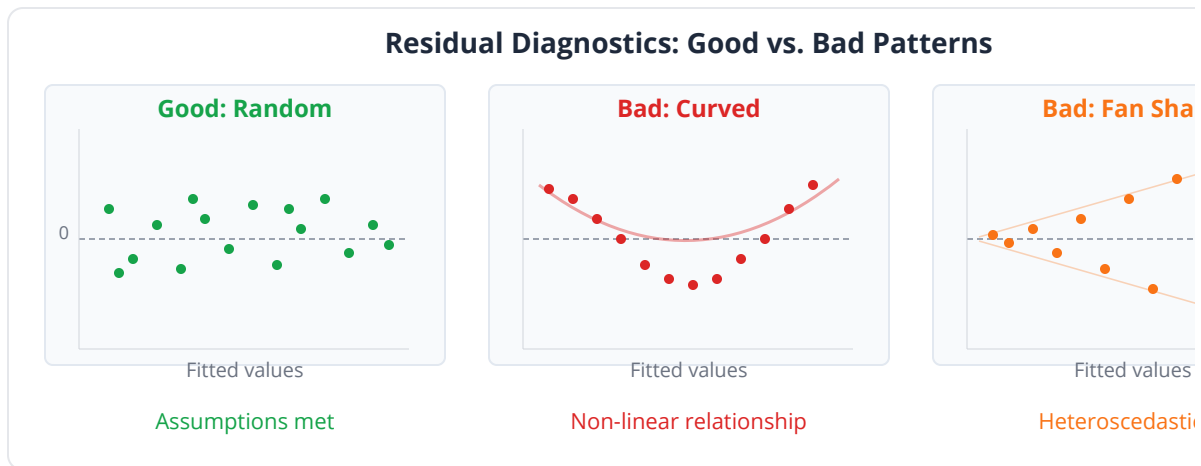
A regression model makes assumptions. **Residuals** (observed - predicted) reveal violations:

Plot	What to Check	Warning Sign
Residuals vs. Fitted	Constant variance (homoscedasticity)	Fan/funnel shape
Q-Q plot of residuals	Normality of errors	Curved pattern
Residuals vs. order	Independence	Systematic trend
Scale-Location	Variance trend	Upward slope

The four assumptions of linear regression:

1. **Linearity:** Y is a linear function of X (check scatter plot)
2. **Independence:** Observations are independent (study design)

3. **Normality:** Residuals are normally distributed (Q-Q plot)
4. **Homoscedasticity:** Constant error variance (residuals vs. fitted)



Common pitfall: Many biological relationships are not linear on the original scale but become linear after log-transformation. Always consider transforming IC50 or expression values before fitting.

When Regression Goes Wrong

Extrapolation

Predicting beyond the range of your data is dangerous. If expression ranges from 5-15 in your training data, predicting IC50 for expression = 25 assumes the linear trend continues — it may not.

Confounding

A significant regression doesn't prove causation. The apparent effect of expression on IC50 could be mediated by a third variable (e.g., tissue type).

Non-linearity

If the true relationship is curved (threshold effect, saturation), a line fits poorly. Residual plots expose this.

Influential Points

A single extreme observation can dramatically change the fitted line. Leverage and Cook's distance help identify such points.

Linear Regression in BioLang

Simple Linear Regression

```
set_seed(42)
fn linear_noise(x, intercept, slope, noise) {
  zip(x, noise) |> map(|pair| intercept + slope * pair[0] +
pair[1])
}
# NCI-60 pharmacogenomics: predict IC50 from target expression
let n = 60

# Simulate expression and drug sensitivity
let expression = rnorm(n, 10, 3)
let ic50 = linear_noise(expression, 85, -4.5, rnorm(n, 0, 8))

# Fit simple linear regression
let model = lm(expression, ic50)

# Print model summary
print("=== Linear Regression Summary ===")
print("Intercept: {model.intercept}")
print("Slope: {model.slope}")
print("R2: {model.r_squared}")
print("Slope p-value: {model.p_value}")
```

Interpreting Coefficients

```
# Detailed coefficient table
print("=== Coefficients ===")
print(" Intercept: {model.intercept}")
print(" Expression  $\beta$ : {model.slope}")
print(" p-value: {model.p_value}")

# Interpretation
let slope = model.slope
print("\nFor each 1-unit increase in expression,")
print("IC50 decreases by {slope |> abs |> round(2)}  $\mu\text{M}$ ")
print("R2 = {model.r_squared |> round(3)}: expression explains")
print("{(model.r_squared * 100) |> round(1)}% of IC50 variation")
```

Prediction with Intervals

```
# Predict IC50 for new cell lines
let new_expression = [7.0, 10.0, 13.0]

# Point predictions using model coefficients
print("=== Predictions ===")
for i in 0..3 {
  let predicted = model.slope * new_expression[i] +
model.intercept
  print("Expression = {new_expression[i]}: Predicted IC50 =
{predicted |> round(1)}  $\mu\text{M}$ ")
}
```

Residual Analysis

```
# Compute residuals and fitted values
let fitted = expression |> map(|x| model.slope * x +
model.intercept)
let resid = []
for i in 0..n {
  resid = resid + [ic50[i] - fitted[i]]
}

# 1. Residuals vs Fitted – check for patterns
let resid_table = table({"Fitted": fitted, "Residual": resid})
plot(resid_table, {type: "scatter", x: "Fitted", y: "Residual",
  title: "Residuals vs Fitted"})

# 2. Check residual distribution
print("Residual summary:")
print(summary(resid))
```

Visualization: Scatter with Regression Line

```
# Scatter plot with regression line
let plot_data = table({"Expression": expression, "IC50": ic50})
plot(plot_data, {type: "scatter", x: "Expression", y: "IC50",
  title: "Drug Sensitivity vs Target Expression (NCI-60)",
  x_label: "Target Gene Expression (log2 FPKM)",
  y_label: "Drug IC50 (μM)"})
```

Demonstrating Problems: Extrapolation

```
set_seed(42)
# Show danger of extrapolation
let x = rnorm(50, 10, 3)
let y = zip(x, rnorm(50, 0, 3))
  |> map(|pair| 100 - 3 * pair[0] + 0.2 * pair[0] ** 2 + pair[1])

# Fit linear model in observed range
let model_extrap = lm(x, y)
print("R2 in training range: {model_extrap.r_squared |> round(3)}")

# Predict within range – reasonable
let pred_in = model_extrap.slope * 10.0 + model_extrap.intercept
print("Prediction at x=10 (in range): {pred_in |> round(1)}")

# Predict outside range – dangerous!
let pred_out = model_extrap.slope * 25.0 + model_extrap.intercept
print("Prediction at x=25 (extrapolation): {pred_out |> round(1)}")
print("True value at x=25 would be: {(100 - 3*25 + 0.2*25**2) |> round(1)}")
print("Extrapolation error demonstrates the danger!")
```

Working with Log-Transformed Data

```
set_seed(42)
# Many biological relationships are linear on the log scale
let dose = rnorm(40, 50, 25) |> map(|d| max(d, 0.1))
let response = zip(dose, rnorm(40, 0, 3))
  |> map(|pair| 50 / (1 + (pair[0] / 10) ** 1.5) + pair[1])

# Linear model on raw scale – poor fit
let model_raw = lm(dose, response)
print("R2 (raw scale): {model_raw.r_squared |> round(3)}")

# Log-transform dose – much better
let log_dose = dose |> map(|d| log2(d))
let model_log = lm(log_dose, response)
print("R2 (log scale): {model_log.r_squared |> round(3)}")

# Compare residual plots to see the difference
```

Python:

```
import numpy as np
from scipy import stats
import statsmodels.api as sm

# Simple linear regression
X = sm.add_constant(expression) # adds intercept
model = sm.OLS(ic50, X).fit()
print(model.summary())

# Prediction with intervals
predictions = model.get_prediction(sm.add_constant([7, 10, 13]))
pred_summary = predictions.summary_frame(alpha=0.05)
# obs_ci_lower/obs_ci_upper = prediction interval
# mean_ci_lower/mean_ci_upper = confidence interval

# Residual analysis
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(model.fittedvalues, model.resid)
axes[0].axhline(0, color='red')
stats.probplot(model.resid, plot=axes[1])
```

R:

```
# Simple linear regression
model <- lm(ic50 ~ expression)
summary(model)

# Prediction with intervals
new_data <- data.frame(expression = c(7, 10, 13))
predict(model, new_data, interval = "confidence")
predict(model, new_data, interval = "prediction")

# Residual diagnostics – 4 diagnostic plots
par(mfrow = c(2, 2))
plot(model)
```


Exercises

Exercise 1: Fit and Interpret

A study measures tumor mutation burden (TMB) and neoantigen count across 80 melanoma samples. Fit a linear regression and answer: How many additional neoantigens does each additional mutation predict?

```
set_seed(42)
let n = 80
let tmb = rnorm(n, 200, 80)
let neoantigens = linear_noise(tmb, 5, 0.15, rnorm(n, 0, 12))

# 1. Fit lm(tmb, neoantigens)
# 2. What is the slope? Interpret it biologically
# 3. What is model.r_squared? Is TMB a good predictor of neoantigen count?
# 4. Predict neoantigen count for TMB = 100, 200, 400
```

Exercise 2: Residual Diagnostics

Fit a linear model to the dose-response data below and use residual plots to determine whether the linear assumption holds. If not, what transformation improves the fit?

```
set_seed(42)
let dose = rnorm(60, 25, 12) |> map(|d| max(d, 1))
let effect = zip(dose, rnorm(60, 0, 5))
  |> map(|pair| 20 * log2(pair[0]) + pair[1])

# 1. Fit lm(dose, effect)
# 2. Create residuals vs fitted plot – what pattern do you see?
# 3. Try log-transforming dose and re-fitting
# 4. Compare R2 values and residual patterns
```

Exercise 3: Confidence vs. Prediction Intervals

Demonstrate why prediction intervals are always wider than confidence intervals. Generate data, fit a model, and plot both intervals across the predictor range.

```
set_seed(42)
let x = rnorm(50, 10, 5)
let y = linear_noise(x, 10, 2, rnorm(50, 0, 4))

# 1. Fit lm(y, x)
# 2. Generate predictions for x = 0, 2, 4, ..., 20
# 3. Use model.slope * xi + model.intercept for each
# 4. Compare point estimates at different x values
```

Exercise 4: The Outlier Effect

Add a single influential outlier to well-behaved data and show how it changes the slope, R^2 , and predictions. Then remove it and compare.

```
set_seed(42)
let x_clean = rnorm(49, 10, 2)
let y_clean = linear_noise(x_clean, 5, 1.5, rnorm(49, 0, 2))

# Add one extreme outlier at x=10, y=50 (should be ~20)
# 1. Fit model with and without the outlier
# 2. Compare slopes and  $R^2$  values
# 3. How much does prediction at x=12 change?
```

Key Takeaways

- Linear regression models $Y = \beta_0 + \beta_1 X + \epsilon$, using least squares to find the best-fit line
- R^2 tells you the proportion of variance explained; the **slope** tells you the rate of change

- **Prediction intervals** (for individuals) are always wider than **confidence intervals** (for the mean) — use the right one for your question
- **Residual analysis** is mandatory: check linearity, normality, and constant variance before trusting results
- **Log-transforming** variables often linearizes biological relationships (dose-response, expression-phenotype)
- Beware **extrapolation** — the linear trend may not continue beyond your data range
- A significant regression does not prove causation — confounders may drive the relationship
- Always visualize your data with a scatter plot before and after fitting

What's Next

One predictor is rarely enough. Day 15 introduces **multiple regression**, where we predict outcomes from many variables simultaneously — and face new challenges like multicollinearity, model selection, and regularization.

Day 15: Multiple Regression and Model Selection

The Problem

Dr. Maria Chen is a clinical researcher studying pancreatic cancer. She has tumor samples from 120 patients, each profiled with 10 biomarkers: CA19-9, CEA, MKI67, TP53 status, tumor size, age, albumin, CRP, neutrophil-lymphocyte ratio (NLR), and platelet count. She wants to predict **tumor stage** (a continuous composite score from 1.0 to 4.0) from these biomarkers.

But there's a problem. CA19-9 and CEA are highly correlated ($r = 0.88$) — they measure overlapping biology. Including both inflates standard errors and makes coefficients uninterpretable. And with 10 potential predictors, how does she find the **best subset** without overfitting?

She needs multiple regression with careful **model selection**.

What Is Multiple Regression?

Multiple regression extends simple regression to multiple predictors:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon$$

Each coefficient β_j represents the effect of X_j **holding all other predictors constant**. This is fundamentally different from running p separate simple regressions.

Simple Regression	Multiple Regression
β_1 = total effect of X_1 on Y	β_1 = effect of X_1 after accounting for $X_2...X_p$
One predictor at a time	All predictors simultaneously
Can't separate correlated effects	Separates correlated effects (when possible)
May show spurious associations	Controls for confounders

Key insight: In simple regression, tumor size might predict stage with $\beta = 0.8$. In multiple regression controlling for CA19-9, the tumor size coefficient might drop to $\beta = 0.3$ — because CA19-9 captures much of the same information.

Multicollinearity: When Predictors Are Too Similar

Multicollinearity occurs when predictors are highly correlated with each other. It doesn't bias predictions, but it wreaks havoc on coefficient interpretation:

Effect	Consequence
Inflated standard errors	Coefficients appear non-significant when they are
Unstable coefficients	Small data changes cause wild coefficient swings
Sign flipping	A predictor with a positive true effect can get a negative coefficient
Uninterpretable	"Effect of X_1 holding X_2 constant" is meaningless if X_1 and X_2 always move together

Detecting Multicollinearity: VIF

The **Variance Inflation Factor** quantifies how much each predictor is explained by the others:

$$VIF_j = \frac{1}{1 - R_j^2}$$

where R_j^2 is the R^2 from regressing X_j on all other predictors.

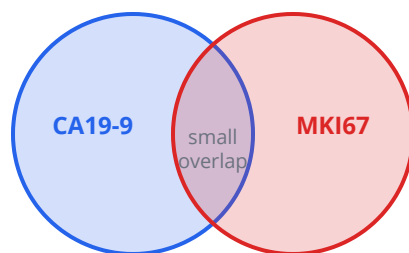
VIF	Interpretation	Action
1	No collinearity	Good
1-5	Moderate	Usually acceptable
5-10	High	Investigate
> 10	Severe	Remove or combine predictors

Common pitfall: VIF > 10 doesn't always mean "drop the variable." In genomics, biologically meaningful predictors may be correlated. Consider combining them (e.g., principal component) or using regularization instead.

Multicollinearity: Overlapping Explained Variance

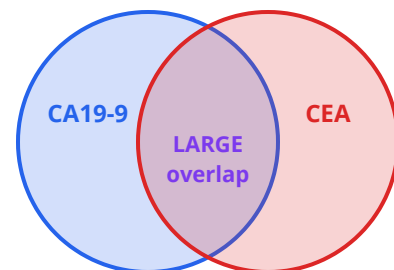
When predictors share information, their individual effects become ambiguous

Low Collinearity (VIF ~ 1)



Each predictor contributes unique information

High Collinearity (VIF > 10)



Hard to separate effects; coefficients unstable

Model Selection: Finding the Best Model

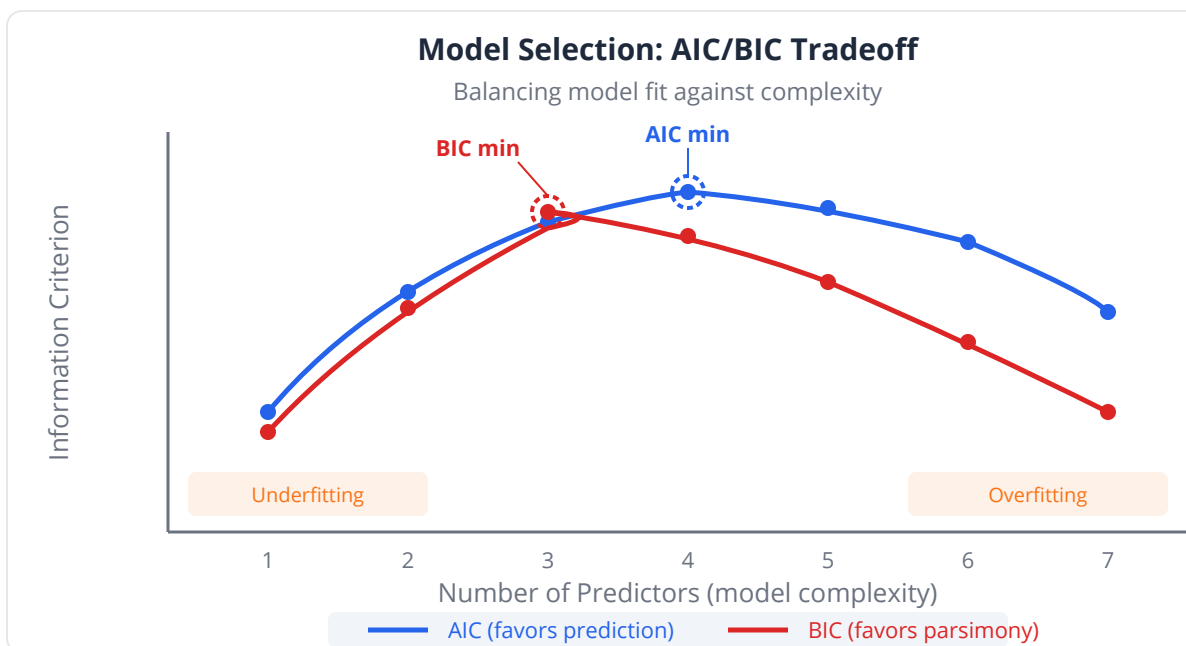
With p predictors, there are 2^p possible models. For $p = 10$, that's 1024 models. We need principled ways to choose.

Information Criteria

Criterion	Formula	Preference
AIC (Akaike)	$-2 \cdot \ln(L) + 2k$	Lower = better; balances fit and complexity
BIC (Bayesian)	$-2 \cdot \ln(L) + k \cdot \ln(n)$	Lower = better; penalizes complexity more than AIC
Adjusted R^2	$1 - (1 - R^2) \cdot (n - 1) / (n - k - 1)$	Higher = better; penalizes added predictors

Where L = likelihood, k = number of parameters, n = sample size.

AIC tends to select slightly larger models (better prediction). **BIC** tends to select smaller models (better interpretation). When they disagree, consider your goal.



Stepwise Regression

Automated search through predictor combinations:

Direction	Strategy	Risk
Forward	Start empty, add best predictor one at a time	May miss suppressor effects
Backward	Start full, remove worst predictor one at a time	May keep redundant predictors
Both	Add and remove at each step	Best coverage, slower

Common pitfall: Stepwise regression is exploratory, not confirmatory. The selected model may not replicate. Always validate on held-out data or use cross-validation.

Regularized Regression: Handling Many Predictors

When you have many predictors (especially in genomics where $p \gg n$), ordinary least squares fails. Regularization adds a penalty term:

Ridge Regression (L2)

$$\min \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

- Shrinks coefficients toward zero but never to exactly zero
- Handles multicollinearity gracefully
- Good when all predictors might be relevant

Lasso Regression (L1)

$$\min \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j|$$

- Can shrink coefficients to exactly zero (automatic variable selection)
- Produces sparse models (easy to interpret)
- Preferred when you suspect only a few predictors matter

Elastic Net

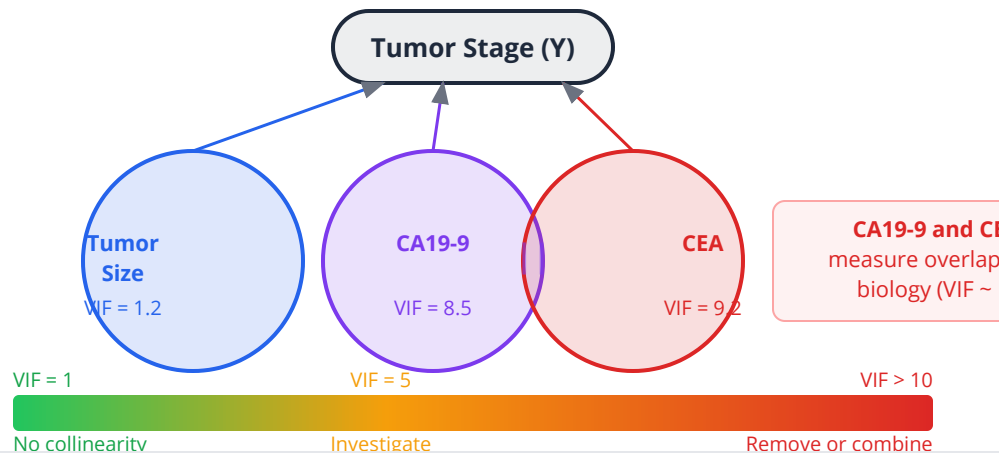
$$\min \sum (y_i - \hat{y}_i)^2 + \lambda_1 \sum |\beta_j| + \lambda_2 \sum \beta_j^2$$

- Combines L1 and L2 penalties
- Good when predictors are correlated groups (keeps all or drops all from a group)
- Controlled by mixing parameter α (0 = ridge, 1 = lasso)

Method	Feature Selection	Correlated Predictors	Best For
Ridge	No (shrinks all)	Handles well	Many weak effects
Lasso	Yes (zeros out)	Picks one arbitrarily	Few strong effects
Elastic Net	Yes (grouped)	Handles well	Correlated groups

VIF: Variance Inflation from Redundant Predictors

VIF measures how much each predictor is explained by the others



Polynomial Regression

When the relationship is curved but you want to stay in the regression framework:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

Useful for dose-response curves, growth curves, and non-linear biomarker relationships. But beware overfitting with high-degree polynomials.

Clinical relevance: The relationship between BMI and mortality is U-shaped — both low and high BMI increase risk. A linear model misses this entirely; a quadratic model captures it.

Multiple Regression in BioLang

Building a Multiple Regression Model

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# Pancreatic cancer: predict tumor stage from biomarkers
let n = 120

# Simulate correlated biomarkers
let age = rnorm(n, 65, 10)
let tumor_size = rnorm(n, 3.5, 1.2)
let ca19_9 = zip(tumor_size, rnorm(n, 100, 40))
  |> map(|pair| pair[0] * 50 + pair[1])
let cea = zip(ca19_9, rnorm(n, 5, 3))
  |> map(|pair| pair[0] * 0.3 + pair[1]) # correlated with CA19-9
let mki67 = rnorm(n, 30, 15)
let albumin = rnorm(n, 3.5, 0.5)
let crp = rnorm(n, 15, 10)
let nlr = rnorm(n, 4, 2)

# True model: stage depends on tumor_size, ca19_9, mki67, age
let stage_noise = rnorm(n, 0, 0.3)
let stage = range(0, n) |> map(|i|
  1.0 + 0.4 * tumor_size[i] + 0.003 * ca19_9[i] + 0.01 * mki67[i]
  + 0.01 * age[i] - 0.3 * albumin[i] + stage_noise[i]
)

# Fit multiple regression with all predictors
let data = table({
  "outcome_stage": stage, "age": age, "tumor_size": tumor_size,
  "ca19_9": ca19_9, "cea": cea, "mki67": mki67,
  "albumin": albumin, "crp": crp, "nlr": nlr
})
let model_full = lm(~outcome_stage ~ age + tumor_size + ca19_9 + cea
+ mki67 + albumin + crp + nlr, data)

print("=== Full Model ===")
print("R²: {model_full.r_squared |> round(3)}")
print("Adjusted R²: {model_full.adj_r_squared |> round(3)}")
```

Checking Multicollinearity

```
# Check multicollinearity via pairwise correlations
let pred_names = ["age", "tumor_size", "ca19_9", "cea", "mki67",
                  "albumin", "crp", "nlr"]
let predictors = [age, tumor_size, ca19_9, cea, mki67, albumin, crp,
nlr]

print("=== Pairwise Correlations (VIF proxy) ===")
for i in 0..8 {
  for j in (i+1)..8 {
    let r = cor(predictors[i], predictors[j])
    if abs(r) > 0.7 {
      print(" {pred_names[i]} vs {pred_names[j]}: r = {r |>
round(3)} *** HIGH")
    }
  }
}

# CA19-9 and CEA likely show high correlation
```

Stepwise Model Selection

```
# Conceptual or diagnostic example; not directly executable.
# Manual model comparison: fit reduced models and compare R2
# Drop CEA (collinear with CA19-9) and noise variables (CRP, NLR)
let data_reduced = table({
  "outcome_stage": stage, "age": age, "tumor_size": tumor_size,
  "ca19_9": ca19_9, "mki67": mki67, "albumin": albumin
})
let model_reduced = lm(~outcome_stage ~ age + tumor_size + ca19_9 +
mki67 + albumin, data_reduced)

print("=== Model Comparison ===")
print("Full model R2: {model_full.r_squared |> round(3)}")
print("Full model Adj R2: {model_full.adj_r_squared |>
round(3)}")
print("Reduced model R2: {model_reduced.r_squared |> round(3)}")
print("Reduced model Adj R2: {model_reduced.adj_r_squared |>
round(3)}")
print("If Adj R2 is similar, the simpler model is preferred")
```

Regularized Regression

```
# Regularized regression concepts
# Note: Ridge, Lasso, and Elastic Net are advanced methods
# typically used when  $p \gg n$  (many predictors, few samples).
# BioLang provides lm() for standard regression; for regularization,
# use Python (scikit-learn) or R (glmnet) as shown below.

# Demonstrate the concept: compare full vs sparse models
# A "lasso-like" approach: fit models dropping one predictor at a
time
# and see which predictors contribute the least
let predictors_list = ["age", "tumor_size", "ca19_9", "cea",
"mki67", "albumin", "crp", "nlr"]

print("=== Variable Importance (drop-one analysis) ===")
let full_r2 = model_full.r_squared
print("Full model  $R^2$ : {full_r2 |> round(4)}")

# Compare by dropping noise predictors
let model_no_crp = lm(~outcome_stage ~ age + tumor_size + ca19_9 +
cea + mki67 + albumin + nlr, data)
let model_no_nlr = lm(~outcome_stage ~ age + tumor_size + ca19_9 +
cea + mki67 + albumin + crp, data)
print("Without CRP:  $R^2$  = {model_no_crp.r_squared |> round(4)}")
print("Without NLR:  $R^2$  = {model_no_nlr.r_squared |> round(4)}")
print("Predictors with minimal  $R^2$  drop are candidates for removal")
```

Polynomial Regression

```
set_seed(42)
# Non-linear biomarker relationship
let bmi = rnorm(100, 29, 5)
let risk = zip(bmi, rnorm(100, 0, 2))
  |> map(|pair| 0.5 + 0.1 * (pair[0] - 25) ** 2 + pair[1])

# Linear fit – misses the U-shape
let model_linear = lm(bmi, risk)
print("Linear R²: {model_linear.r_squared |> round(3)}")

# Polynomial fit – add bmi² term
let bmi_centered_sq = bmi |> map(|x| (x - 25) ** 2)
let model_poly = lm(bmi_centered_sq, risk)
print("Quadratic R²: {model_poly.r_squared |> round(3)}")

# Visualize the improvement
let plot_data = table({"BMI": bmi, "Risk": risk})
plot(plot_data, {type: "scatter", x: "BMI", y: "Risk",
  title: "BMI vs Risk: Linear vs Quadratic Fit"})
```

Predicted vs. Actual Plot

```
# Conceptual or diagnostic example; not directly executable.
# The ultimate model validation plot
# Compute predicted values from the reduced model
let predicted_stage = model_reduced.fitted

let pred_actual = table({"Actual": stage, "Predicted":
  predicted_stage})
plot(pred_actual, {type: "scatter", x: "Actual", y: "Predicted",
  title: "Predicted vs Actual (Reduced Model)"})

# Points along the diagonal = good predictions
# Systematic deviations = model problems
let r_pred = cor(stage, predicted_stage)
print("Correlation (actual vs predicted): {r_pred |> round(3)}")
```

Python:

```
import statsmodels.api as sm
from sklearn.linear_model import Lasso, Ridge, ElasticNet
from sklearn.preprocessing import PolynomialFeatures
from statsmodels.stats.outliers_influence import
variance_inflation_factor

# Multiple regression
X = sm.add_constant(df[['age', 'tumor_size', 'ca19_9', 'cea',
'mki67']])
model = sm.OLS(stage, X).fit()
print(model.summary())

# VIF
vif = [variance_inflation_factor(X.values, i) for i in
range(X.shape[1])]

# Stepwise (manual – no built-in in statsmodels)
# Use mlxtend: from mlxtend.feature_selection import
SequentialFeatureSelector

# Lasso
from sklearn.linear_model import LassoCV
lasso = LassoCV(cv=5).fit(X_scaled, y)
print(f"Non-zero: {(lasso.coef_ != 0).sum()}")
```

R:

```
# Multiple regression
model <- lm(stage ~ age + tumor_size + ca19_9 + cea + mki67 +
            albumin + crp + nlr)
summary(model)

# VIF
library(car)
vif(model)

# Stepwise
step_model <- step(model, direction = "both")

# Lasso
library(glmnet)
cv_lasso <- cv.glmnet(X_matrix, stage, alpha = 1)
coef(cv_lasso, s = "lambda.min")

# Ridge
cv_ridge <- cv.glmnet(X_matrix, stage, alpha = 0)

# Elastic net
cv_enet <- cv.glmnet(X_matrix, stage, alpha = 0.5)
```

Exercises

Exercise 1: Build and Compare Models

Given 150 samples with 6 predictors, build a full model, then use stepwise selection to find a reduced model. Compare using AIC, BIC, and adjusted R^2 .

```
set_seed(42)
let n = 150
let x1 = rnorm(n, 10, 3)
let x2 = rnorm(n, 5, 2)
let x3 = zip(x1, rnorm(n, 0, 1))
  |> map(|pair| pair[0] * 0.5 + pair[1]) # correlated with x1
let x4 = rnorm(n, 20, 5)
let x5 = rnorm(n, 0, 1) # noise
let x6 = rnorm(n, 0, 1) # noise

let y_noise = rnorm(n, 0, 3)
let y = range(0, n)
  |> map(|i| 5 + 2 * x1[i] + 1.5 * x2[i] + 0.5 * x4[i] +
y_noise[i])

# 1. Fit full model with all 6 predictors using lm()
# 2. Check pairwise cor() – which predictors are collinear?
# 3. Compare full vs reduced models by adj_r_squared
# 4. Which predictors matter? Do they match the true model?
```

Exercise 2: Ridge vs. Lasso

Compare ridge and lasso on a dataset where only 3 of 8 predictors truly matter. Which method correctly identifies the true predictors?

```
set_seed(42)
let n = 100

# 8 predictors, only 3 are real
# Fit lm() with all 8, then with only the true 3
# Compare R2 – does the reduced model perform similarly?
# Note: for true ridge/lasso, use Python (scikit-learn) or R
(glmnet)
```

Exercise 3: Polynomial vs. Linear

Fit linear, quadratic, and cubic models to a dose-response curve. Use AIC to select the best model, and show that adding unnecessary complexity (cubic) hurts.

```
set_seed(42)
let dose = rnorm(80, 25, 12) |> map(|d| max(d, 1))
let response = zip(dose, rnorm(80, 0, 3))
  |> map(|pair| 10 + 5 * log2(pair[0]) + pair[1])

# 1. Fit lm() with dose, dose + dose2, dose + dose2 + dose3
# 2. Compare R2 and adj_r_squared for each
# 3. Which degree gives the best balance of fit and simplicity?
```

Exercise 4: Predicted vs. Actual

Build a multiple regression model for gene expression prediction. Create a predicted vs. actual scatter plot. Assess where the model succeeds and fails.

```
set_seed(42)
let n = 200

# Simulate: predict gene expression from 4 TF binding signals
# Build model using lm(), generate predicted vs actual plot
# Calculate mean absolute error
# Identify the 5 worst predictions – what makes them outliers?
```

Key Takeaways

- Multiple regression estimates the effect of each predictor **controlling for all others** — fundamentally different from separate simple regressions
- **Multicollinearity** (VIF > 10) inflates standard errors and makes coefficients uninterpretable — detect it before interpreting
- **Model selection** balances fit and complexity: AIC favors prediction, BIC favors parsimony, adjusted R² penalizes added terms
- **Stepwise regression** is useful for exploration but should be validated on held-out data
- **Lasso** performs automatic variable selection (zeros out irrelevant predictors); **Ridge** shrinks all coefficients but keeps them; **Elastic net** combines both

- **Polynomial regression** captures non-linear relationships within the regression framework but risks overfitting
- Always check residuals, predicted vs. actual plots, and VIF before trusting your model
- With genomics data ($p \gg n$), regularization is not optional — it's essential

What's Next

What if your outcome isn't continuous but **binary** — responder vs. non-responder, alive vs. dead, mutant vs. wild-type? Day 16 introduces **logistic regression**, where we predict probabilities of categorical outcomes using ROC curves, odds ratios, and the powerful GLM framework.

Day 16: Logistic Regression — Binary Outcomes

The Problem

Dr. Priya Sharma is an immuno-oncologist analyzing data from 180 melanoma patients who received anti-PD-1 immunotherapy. For each patient, she has three biomarkers — **tumor mutational burden (TMB)**, **PD-L1 expression**, and **microsatellite instability (MSI) status** — and one outcome: **response** (tumor shrank $\geq 30\%$) or **non-response**.

Her first instinct is to use linear regression, predicting response (coded 1/0) from the biomarkers. But the predictions come out as 1.3 for one patient and -0.2 for another. Probabilities can't be greater than 1 or less than 0.

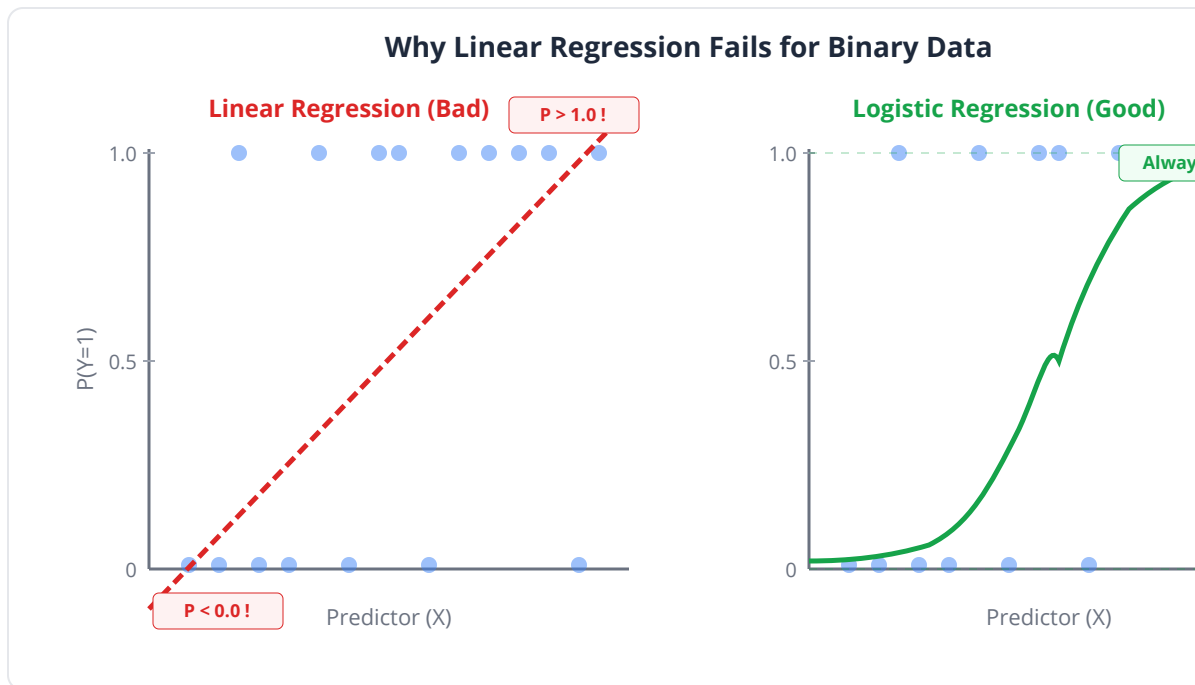
She needs a method designed for **binary outcomes** — logistic regression.

Why Linear Regression Fails for Binary Outcomes

When Y is binary (0 or 1), linear regression has fundamental problems:

Problem	Consequence
Predictions outside [0, 1]	Impossible probabilities (negative or > 100%)
Non-normal residuals	Residuals are binary, violating normality assumption
Heteroscedasticity	Variance depends on the predicted value
Non-linear relationship	The true probability follows an S-curve, not a line

Key insight: We need a function that maps any real number to the range $[0, 1]$. The **logistic (sigmoid) function** does exactly this.



The Logistic Function

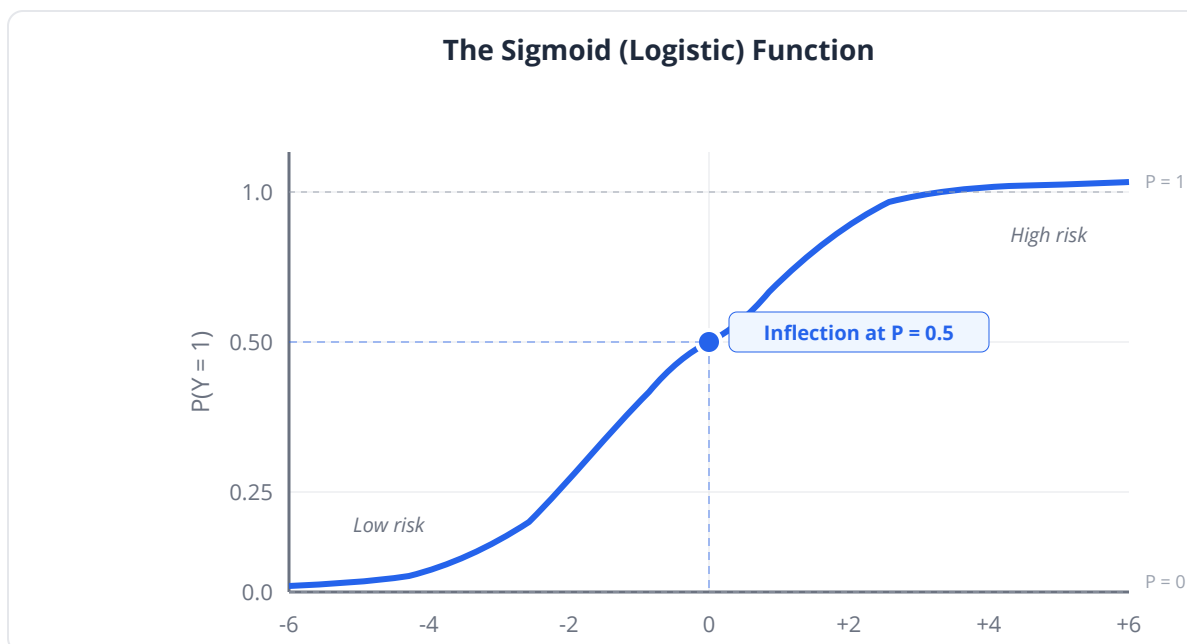
The logistic regression model predicts the **probability** of the outcome being 1:

$$P(Y=1 | X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p)}}$$

The sigmoid function transforms the linear predictor (which ranges from $-\infty$ to $+\infty$) into a probability (which ranges from 0 to 1):

Linear predictor ($\beta_0 + \beta_1 X$)	Probability $P(Y=1)$
-5	0.007
-2	0.12
0	0.50
+2	0.88
+5	0.993

The curve is steepest at $P = 0.5$ (the decision boundary) and flattens at the extremes — exactly how biological responses behave.



Interpreting Coefficients: Log-Odds and Odds Ratios

Logistic regression coefficients are on the **log-odds** scale:

$$\ln\left(\frac{P}{1-P}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots$$

This is less intuitive than linear regression. The key transformation:

$$\text{Odds Ratio} = e^{\beta}$$

β (log-odds)	OR = e^{β}	Interpretation
0	1.0	No effect
0.5	1.65	65% higher odds per unit increase
1.0	2.72	2.7× higher odds
-0.5	0.61	39% lower odds
-1.0	0.37	63% lower odds

Key insight: An odds ratio of 2.0 means the odds of response **double** for each 1-unit increase in the predictor. It does NOT mean the probability doubles — that depends on the baseline probability.

Odds vs. Probability

Probability	Odds	Interpretation
0.50	1:1	Equal chance either way
0.75	3:1	Three times as likely to respond
0.20	1:4	Four times as likely NOT to respond
0.90	9:1	Strong favoring response

Common pitfall: When the outcome is common (prevalence > 10%), odds ratios substantially overestimate relative risks. An OR of 3.0 for a 50% baseline event means the probability goes from 50% to 75% — a relative risk of only 1.5. Report both OR and absolute probabilities.

ROC Curves and AUC

The model outputs probabilities. To make yes/no predictions, you need a **threshold** — above it, predict “responder.” But which threshold?

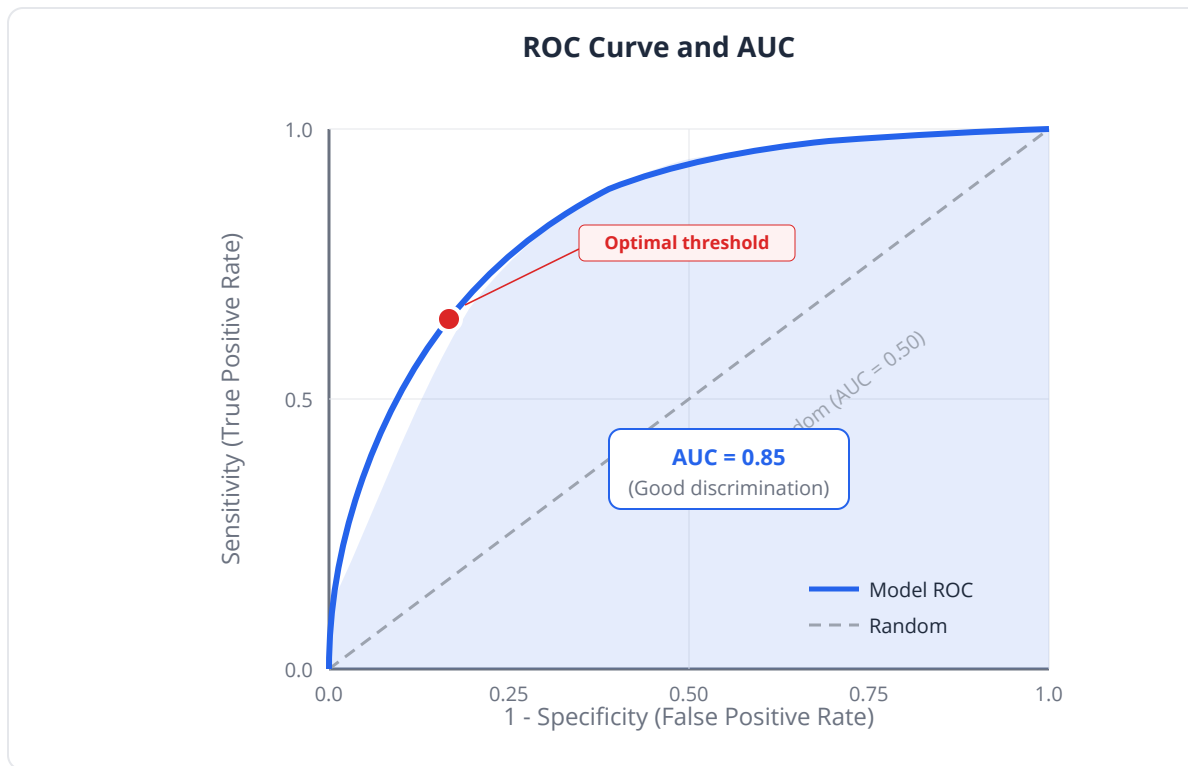
A **Receiver Operating Characteristic (ROC) curve** plots sensitivity vs. (1 - specificity) across all possible thresholds:

Metric	Definition	Trade-off
Sensitivity (TPR)	True responders correctly identified	Higher = catch more responders
Specificity (TNR)	True non-responders correctly identified	Higher = fewer false alarms
PPV (Precision)	Positive predictions that are correct	Higher = trust positive results
NPV	Negative predictions that are correct	Higher = trust negative results

The **Area Under the Curve (AUC)** summarizes overall discrimination:

AUC	Performance
0.50	Random guessing (useless)
0.60-0.70	Poor
0.70-0.80	Acceptable
0.80-0.90	Good
0.90-1.00	Excellent

Clinical relevance: In cancer diagnostics, the threshold choice depends on context. Screening tests favor **high sensitivity** (don't miss cancers), while confirmatory tests favor **high specificity** (don't cause unnecessary biopsies).



The GLM Framework

Logistic regression is a special case of the **Generalized Linear Model (GLM)** — a flexible framework for non-normal outcomes:

Outcome Type	Distribution	Link Function	Name
Continuous	Normal	Identity	Linear regression
Binary (0/1)	Binomial	Logit	Logistic regression
Counts	Poisson	Log	Poisson regression
Counts (overdispersed)	Negative binomial	Log	NB regression
Positive continuous	Gamma	Inverse	Gamma regression

The GLM framework unifies these under one interface, letting you choose the appropriate model for your data type.

Clinical relevance: RNA-seq read counts follow a negative binomial distribution. Tools like DESeq2 use GLMs with a negative binomial family — the same framework as logistic regression, just with a different distribution assumption.

Logistic Regression in BioLang

Fitting a Logistic Model

```
set_seed(42)
# Immunotherapy response prediction
let n = 180

# Simulate predictors
let tmb = rnorm(n, 10, 5) |> map(|x| max(x, 0))
let pdl1 = rnorm(n, 30, 20) |> map(|x| max(min(x, 100), 0))
let msi = rnorm(n, 0, 1) |> map(|x| if x > 1.0 { 1 } else { 0 }) #
~15% MSI-high

# True response probability (logistic model)
let log_odds = range(0, n)
  |> map(|i| -3 + 0.15 * tmb[i] + 0.02 * pdl1[i] + 1.5 * msi[i])
let prob = log_odds |> map(|x| 1.0 / (1.0 + exp(-x)))
let response = prob |> map(|p| if rnorm(1, 0, 1)[0] < p { 1 } else {
0 })

print("Response rate: {(response |> sum) / n * 100 |> round(1)}%")

# Fit logistic regression
let glm_data = table({"response": response, "TMB": tmb, "PDL1":
pdl1, "MSI": msi})
let model = glm(~response ~ TMB + PDL1 + MSI, glm_data, "binomial")

print("=== Logistic Regression Results ===")
print("Intercept: {model.intercept |> round(3)}")
```

Interpreting Odds Ratios

```
# Convert coefficients to odds ratios
let coef_names = ["TMB", "PD-L1", "MSI"]
let coefficients = model.coefficients

print("=== Odds Ratios ===")
for i in 0..3 {
  let beta = coefficients[i]
  let or_val = exp(beta)
  print("{coef_names[i]}:  $\beta$  = {beta |> round(3)}, OR = {or_val |>
round(2)}")
}

# Interpretation:
# TMB OR = 1.16 means each additional mut/Mb increases odds of
response by 16%
# MSI OR = 4.5 means MSI-high patients have 4.5x the odds of
responding
```

ROC Curve and AUC

```
# Conceptual or diagnostic example; not directly executable.
# Compute predicted probabilities from model
let pred_prob = []
for i in 0..n {
  let lp = model.intercept + model.coefficients[0] * tmb[i]
    + model.coefficients[1] * pdl1[i] + model.coefficients[2] *
msi[i]
  pred_prob = pred_prob + [1.0 / (1.0 + exp(-lp))]
}

# ROC curve
let roc_data = table({"actual": response, "predicted": pred_prob})
roc_curve(roc_data)

# Compute AUC
let auc_val = model.auc
print("AUC: {auc_val |> round(3)}")

# Interpretation
if auc_val >= 0.80 {
  print("Good discrimination")
} else if auc_val >= 0.70 {
  print("Acceptable discrimination")
} else {
  print("Poor discrimination – model needs additional predictors")
}
```

Finding the Optimal Threshold

```
# Conceptual or diagnostic example; not directly executable.
# Sensitivity/specificity at different thresholds
let thresholds = [0.2, 0.3, 0.4, 0.5, 0.6, 0.7]

print("=== Threshold Analysis ===")
print("Threshold  Sensitivity  Specificity  PPV      NPV")

for t in thresholds {
  let predicted_class = pred_prob |> map(|p| if p >= t { 1 } else
{ 0 })
  let tp = 0
  let fp = 0
  let tn = 0
  let fn = 0

  for i in 0..n {
    if predicted_class[i] == 1 && response[i] == 1 { tp = tp + 1
}
    if predicted_class[i] == 1 && response[i] == 0 { fp = fp + 1
}
    if predicted_class[i] == 0 && response[i] == 0 { tn = tn + 1
}
    if predicted_class[i] == 0 && response[i] == 1 { fn = fn + 1
}
  }

  let sens = tp / (tp + fn)
  let spec = tn / (tn + fp)
  let ppv = if tp + fp > 0 { tp / (tp + fp) } else { 0 }
  let npv = if tn + fn > 0 { tn / (tn + fn) } else { 0 }

  print("{t}          {sens |> round(3)}          {spec |> round(3)}
{ppv |> round(3)}    {npv |> round(3)}")
}

# Youden's J: optimal balance of sensitivity and specificity
```


Using the GLM Framework

```
# Logistic regression via the general GLM interface
let model_glm = glm(~response ~ TMB + PDL1 + MSI, glm_data,
"binomial")

# Same results, but now you can swap families:

# Poisson regression for count data (e.g., number of mutations)
# let count_data = table({"mutations": mutation_count, "exposure":
exposure, "age": age})
# let count_model = glm(~mutations ~ exposure + age, count_data,
"poisson")
```

Visualizing Predicted Probabilities

```
# Box plot: predicted probabilities by actual outcome
let resp_probs = []
let nonresp_probs = []

for i in 0..n {
  if response[i] == 1 {
    resp_probs = resp_probs + [pred_prob[i]]
  } else {
    nonresp_probs = nonresp_probs + [pred_prob[i]]
  }
}

let bp_table = table({"Non-Responders": nonresp_probs, "Responders":
resp_probs})
boxplot(bp_table, {title: "Predicted Probabilities by Actual
Outcome"})

# Good model: minimal overlap between the two boxes
```

Effect of Individual Predictors

```
# Conceptual or diagnostic example; not directly executable.
# Show how each predictor shifts the probability curve
# Fix other predictors at their means
let tmb_range = [0, 5, 10, 15, 20, 25, 30]
let pdl1_mean = 30
let msi_0 = 0

print("=== TMB Effect on Response Probability ===")
print("(PD-L1 = {pdl1_mean}, MSI = negative)")

for t in tmb_range {
  let lp = model.intercept + model.coefficients[0] * t
    + model.coefficients[1] * pdl1_mean + model.coefficients[2]
  * msi_0
  let p = 1.0 / (1.0 + exp(-lp))
  print("  TMB = {t}: P(response) = {(p * 100) |> round(1)}%")
}
```

Python:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc, classification_report
import statsmodels.api as sm

# Statsmodels (full inference)
X = sm.add_constant(df[['TMB', 'PDL1', 'MSI']])
model = sm.Logit(response, X).fit()
print(model.summary())

# Odds ratios
print(np.exp(model.params))
print(np.exp(model.conf_int()))

# ROC curve
fpr, tpr, thresholds = roc_curve(response, model.predict(X))
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')

# Scikit-learn (prediction-focused)
clf = LogisticRegression().fit(X_train, y_train)
y_prob = clf.predict_proba(X_test)[: , 1]
```

R:

```
# Logistic regression
model <- glm(response ~ TMB + PDL1 + MSI, family = binomial)
summary(model)

# Odds ratios with CI
exp(cbind(OR = coef(model), confint(model)))

# ROC curve
library(pROC)
roc_obj <- roc(response, fitted(model))
plot(roc_obj, print.auc = TRUE)

# Optimal threshold (Youden's J)
coords(roc_obj, "best", best.method = "youden")
```

Exercises

Exercise 1: Build and Interpret a Logistic Model

Predict cancer diagnosis (1 = cancer, 0 = benign) from three biomarkers.
Interpret each odds ratio in clinical terms.

```
set_seed(42)
let n = 200

let biomarker_a = rnorm(n, 50, 15)
let biomarker_b = rnorm(n, 10, 4)
let age = rnorm(n, 55, 12)

let log_odds = range(0, n)
  |> map(|i| -5 + 0.04 * biomarker_a[i] + 0.2 * biomarker_b[i] +
0.03 * age[i])
let prob = log_odds |> map(|x| 1.0 / (1.0 + exp(-x)))
let cancer = prob |> map(|p| if rnorm(1, 0, 1)[0] < p { 1 } else { 0
})

# 1. Fit glm(~cancer ~ biomarker_a + biomarker_b + age, data,
"binomial")
# 2. Compute and interpret odds ratios: exp(coefficient) for each
predictor
# 3. Which biomarker has the strongest effect?
# 4. What does the OR for age mean clinically?
```

Exercise 2: ROC Analysis

Build a logistic model and evaluate it with an ROC curve. Find the threshold that maximizes Youden's J index (sensitivity + specificity - 1).

```
# Use the model from Exercise 1
# 1. Generate predicted probabilities from model coefficients
# 2. Plot ROC curve with roc_curve(table)
# 3. Compute sensitivity and specificity at thresholds 0.3, 0.5, 0.7
# 4. Which threshold maximizes Youden's J?
# 5. If this is a screening test, would you prefer a different
threshold? Why?
```

Exercise 3: Comparing Two Models

Build two logistic models: one with TMB alone, another with TMB + PD-L1 + MSI. Compare their AUC values. Does adding predictors improve discrimination?

```
set_seed(42)
let n = 150

# Simulate data where TMB is a moderate predictor and MSI adds
substantial value
# 1. Fit model_simple = glm(~response ~ TMB, data, "binomial")
# 2. Fit model_full = glm(~response ~ TMB + PDL1 + MSI, data,
"binomial")
# 3. Compare AUC values
# 4. Plot both ROC curves
# 5. Is the improvement worth the added complexity?
```

Exercise 4: The Separation Problem

What happens when a predictor perfectly separates outcomes? Simulate MSI status that perfectly predicts response and observe what logistic regression does.

```
set_seed(42)
let n = 100
let msi = rnorm(n, 0, 1) |> map(|x| if x > 0.84 { 1 } else { 0 })
let response = msi # perfect separation!

# 1. Try fitting glm(~response ~ msi, data, "binomial")
# 2. What happens to the coefficient and its standard error?
# 3. Why is this a problem? (Hint: the MLE doesn't exist)
# 4. How would you handle this in practice?
```

Key Takeaways

- **Logistic regression** models binary outcomes by predicting probabilities via the sigmoid function — use it instead of linear regression when Y is 0/1
- Coefficients are on the **log-odds** scale; exponentiate to get **odds ratios** (OR): e^{β}
- An OR of 2.0 means the odds double per unit increase — but this is NOT the same as doubling the probability

- **ROC curves** show the sensitivity/specificity trade-off across all thresholds; **AUC** summarizes overall discrimination
- The **optimal threshold** depends on clinical context: screening favors sensitivity, confirmation favors specificity
- Logistic regression is a special case of the **GLM framework** — the same approach extends to count data (Poisson), survival data, and more
- Always report odds ratios with **confidence intervals**, not just p-values
- Watch for **separation** (a predictor perfectly predicts the outcome), which causes coefficient inflation

What's Next

Sometimes the outcome isn't just binary — it's **time-to-event**: how long until a patient relapses, how long a cell line survives treatment. Day 17 introduces **survival analysis**, where censoring makes standard methods fail and Kaplan-Meier curves become essential.

Day 17: Survival Analysis — Time-to-Event Data

The Problem

Dr. Elena Volkov is a cancer genomicist analyzing overall survival in 250 lung adenocarcinoma patients. She has tumor sequencing data and wants to answer: **Do TP53-mutant patients survive longer than TP53 wild-type patients?**

Her first attempt: compute the mean survival time for each group and run a t-test. But she immediately hits a wall. 40% of patients are **still alive** at the end of the study. Their survival time is **at least** 36 months — but she doesn't know their actual survival time. Dropping these patients biases the analysis (the longest survivors are removed). Using 36 months as their survival time underestimates it.

This is the problem of **censoring**, and it requires survival analysis.

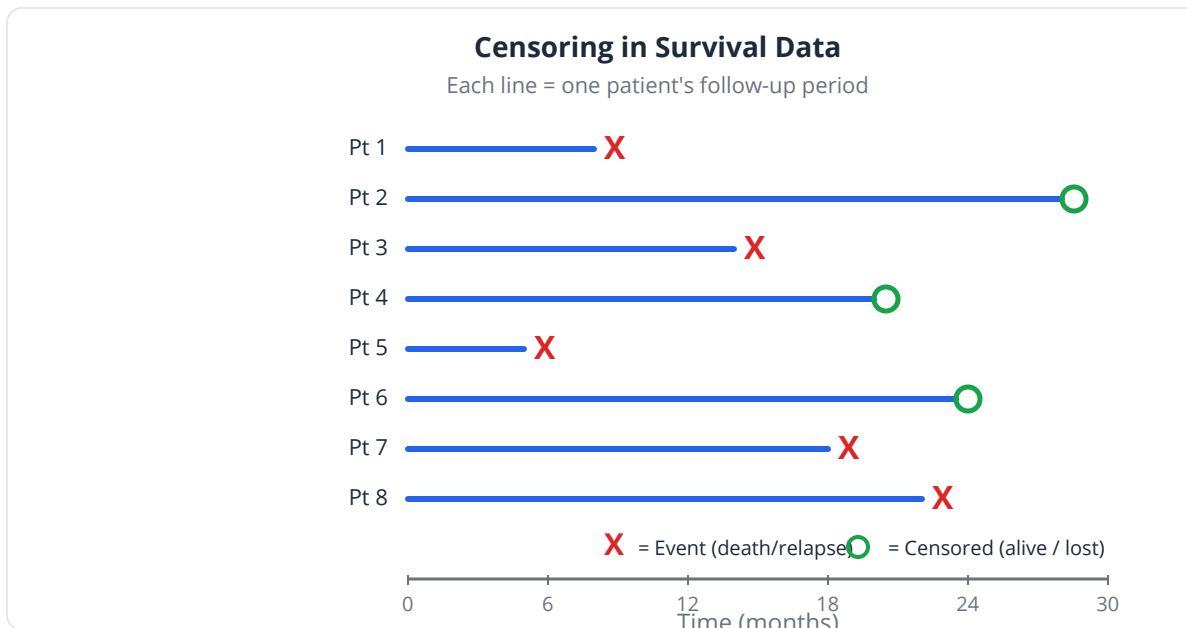
What Is Censoring?

Right-censoring occurs when the event of interest (death, relapse, progression) has not yet happened at the time of observation. The patient is lost to follow-up, the study ends, or they die of an unrelated cause.

Patient	Follow-up	Status	What We Know
A	24 months	Dead	Survived exactly 24 months
B	36 months	Alive	Survived at least 36 months
C	12 months	Lost	Survived at least 12 months
D	48 months	Dead	Survived exactly 48 months

Patients B and C are **censored** — we know a lower bound on their survival, but not the actual value.

Key insight: Censored observations are NOT missing data. They contain real information (“this patient survived at least X months”). Throwing them away wastes data and biases results. Including them as if the event occurred underestimates survival.



Why Standard Methods Fail

Method	Problem with Censored Data
Mean survival	Can't compute — don't know censored patients' true times
t-test	Assumes complete observations
Linear regression	Can't handle "at least" values
Simple proportions	Ignores timing of events

The Kaplan-Meier Estimator

The **Kaplan-Meier (KM) estimator** is the workhorse of survival analysis. It estimates the survival function $S(t) = P(\text{survival} > t)$ as a step function that drops at each observed event.

How it works: At each event time t_j :

$$\hat{S}(t) = \prod_{t_j \leq t} \left(1 - \frac{d_j}{n_j}\right)$$

Where:

- d_j = number of events at time t_j
- n_j = number at risk just before t_j (alive and not yet censored)

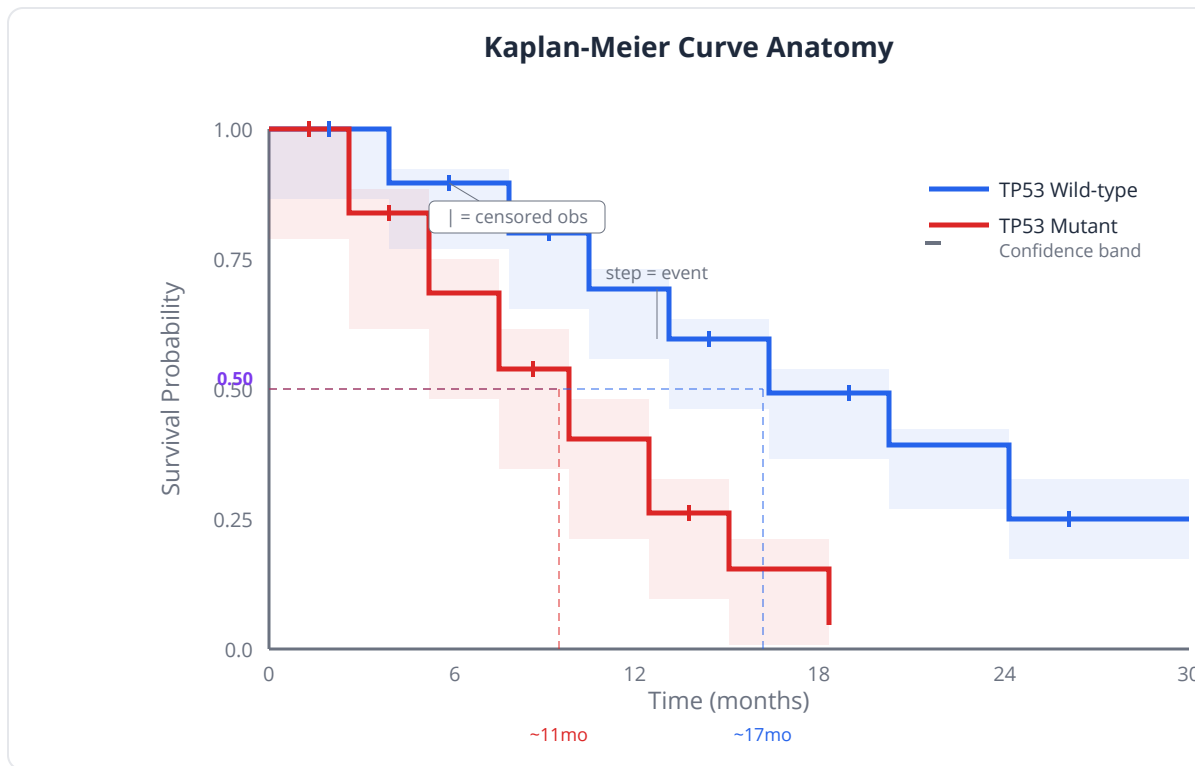
Reading a KM curve:

- Y-axis: proportion surviving (starts at 1.0 = 100%)
- X-axis: time
- Steps down: events (deaths)
- Tick marks: censored observations (patients lost but still contribute until that point)
- Steeper drops: periods of high event rate
- Flat plateaus: stable periods

Median Survival

The **median survival** is the time at which the KM curve crosses 0.50 — the point where half the patients have experienced the event. It is more robust than the mean because it is less affected by a few extreme values.

Clinical relevance: “Median overall survival was 18 months in the treatment arm versus 12 months in the control arm” is the standard way clinical trials report survival data. It’s the single most important number in oncology clinical trials.



The Log-Rank Test

The **log-rank test** compares survival curves between groups. It asks: "Is the survival experience significantly different between these groups?"

- **H₀:** The survival curves are identical
- **H₁:** The survival curves differ

The test compares observed events to expected events (under H₀) at each time point across the entire follow-up period. It gives most weight to later time points.

Consideration	Detail
Assumptions	Proportional hazards (constant HR over time)
Power	Best when hazard ratio is constant
Limitation	Only tests equality — doesn't estimate HOW different
Multiple groups	Can compare 3+ groups simultaneously

Common pitfall: The log-rank test can be non-significant even when curves look different, if the difference is early (then converges) or if curves cross. If hazards are not proportional, consider alternatives like the Wilcoxon (Breslow) test, which gives more weight to early events.

Cox Proportional Hazards Model

The **Cox PH model** is the regression analog for survival data. It models the hazard (instantaneous event rate) as:

$$h(t|X) = h_0(t) \cdot \exp(\beta_1 X_1 + \beta_2 X_2 + \dots)$$

Where $h_0(t)$ is the baseline hazard and the exponential term scales it by covariates.

The Hazard Ratio

The key output is the **hazard ratio (HR)**:

$$HR = e^{\beta}$$

HR	Interpretation
1.0	No difference in hazard
2.0	Twice the hazard (worse survival)
0.5	Half the hazard (better survival)

Key insight: HR = 2.0 does NOT mean “dies twice as fast” or “survives half as long.” It means that at any given time point, the hazard (instantaneous risk of the event) is 2x higher. The relationship between HR and median survival depends on the shape of the baseline hazard.

Proportional Hazards Assumption

The Cox model assumes that the **ratio of hazards** between groups is constant over time. If TP53-mutant patients have HR = 1.5, this ratio should hold at 6 months, 12 months, and 24 months.

Violations:

- Curves that cross (treatment effect reverses over time)
- HR that changes with time (e.g., surgery risk is high early, then protective later)
- Delayed treatment effects (immunotherapy often shows late separation)

Adjusting for Confounders

Like multiple regression, Cox models can include multiple predictors:

$$h(t) = h_0(t) \cdot \exp(\beta_1 \cdot \text{TP53} + \beta_2 \cdot \text{Age} + \beta_3 \cdot \text{Stage})$$

This gives the HR for TP53 mutation **adjusted for** age and stage — a cleaner estimate of its independent effect.

Clinical relevance: A univariate HR for TP53 might be 1.8 (worse survival). But if TP53-mutant tumors also tend to be higher stage, the adjusted HR might drop to 1.3 after controlling for stage. The adjusted HR is what matters for understanding TP53's independent prognostic value.

Survival Analysis in BioLang

Kaplan-Meier Curves

```
set_seed(42)
# Lung adenocarcinoma survival by TP53 status
let n = 250

# Simulate TP53 status (40% mutant)
let tp53_mut = rnorm(n, 0, 1) |> map(|x| if x > 0.25 { 0 } else { 1
})

# Simulate survival times (exponential with TP53 effect)
let base_hazard = 0.03 # monthly hazard rate
let survival_time = []
let status = [] # 1 = dead, 0 = censored

for i in 0..n {
  let hazard = base_hazard * if tp53_mut[i] == 1 { 1.6 } else {
1.0 }
  let true_time = -log(rnorm(1, 0.5, 0.2)[0] |> max(0.01)) /
hazard
  let censor_time = rnorm(1, 42, 10)[0] |> max(24)

  if true_time < censor_time {
    survival_time = survival_time + [true_time]
    status = status + [1] # event observed
  } else {
    survival_time = survival_time + [censor_time]
    status = status + [0] # censored
  }
}

print("Events: {status |> sum} / {n} ({(status |> sum) / n * 100 |>
round(1)}%)")
print("Censored: {n - (status |> sum)}")

# Fit Kaplan-Meier for each group
let wt_times = []
let wt_status = []
let mut_times = []
let mut_status = []
for i in 0..n {
  if tp53_mut[i] == 0 {
```

```

        wt_times = wt_times + [survival_time[i]]
        wt_status = wt_status + [status[i]]
    } else {
        mut_times = mut_times + [survival_time[i]]
        mut_status = mut_status + [status[i]]
    }
}

let km_wt = kaplan_meier(wt_times, wt_status)
let km_mut = kaplan_meier(mut_times, mut_status)

print("Median survival (TP53 WT): {km_wt.median |> round(1)}
months")
print("Median survival (TP53 mut): {km_mut.median |> round(1)}
months")

```

Kaplan-Meier Plot

```

# Publication-quality survival curves
# Plot KM data using plot()
let km_table = table({
  "time": km_wt.times ++ km_mut.times,
  "survival": km_wt.survival ++ km_mut.survival,
  "group": km_wt.times |> map(|t| "TP53 WT") ++ km_mut.times |>
map(|t| "TP53 Mut")
})
plot(km_table, {type: "line", x: "time", y: "survival", color:
"group",
  title: "Overall Survival by TP53 Status",
  x_label: "Time (months)", y_label: "Survival Probability"})

```


Log-Rank Test

```
# Compare survival curves statistically
# Compute log-rank test from KM outputs
# The test compares observed vs expected events across groups
let km_all = kaplan_meier(survival_time, status)

# Use Cox PH as a proxy for log-rank (equivalent for single binary
predictor)
let cox_lr = cox_ph(survival_time, status, [tp53_mut])
print("=== Log-Rank Test (via Cox) ===")
print("p-value: {cox_lr.p_value |> round(4)}")

if cox_lr.p_value < 0.05 {
  print("Significant difference in survival between TP53 groups")
} else {
  print("No significant difference detected (p > 0.05)")
}
```

Cox Proportional Hazards Model

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# Simulate additional covariates
let age = rnorm(n, 65, 10)
let stage = rnorm(n, 2.5, 0.8) |> map(|x| max(1, min(4, round(x))))

# Univariate Cox model
let cox_simple = cox_ph(survival_time, status, [tp53_mut])

print("=== Univariate Cox Model ===")
print("TP53 Mutation HR: {cox_simple.hazard_ratio |> round(2)}")
print("  p-value: {cox_simple.p_value |> round(4)}")

# Multivariable Cox model – adjust for age and stage
let cox_adjusted = cox_ph(survival_time, status, [tp53_mut, age,
stage])

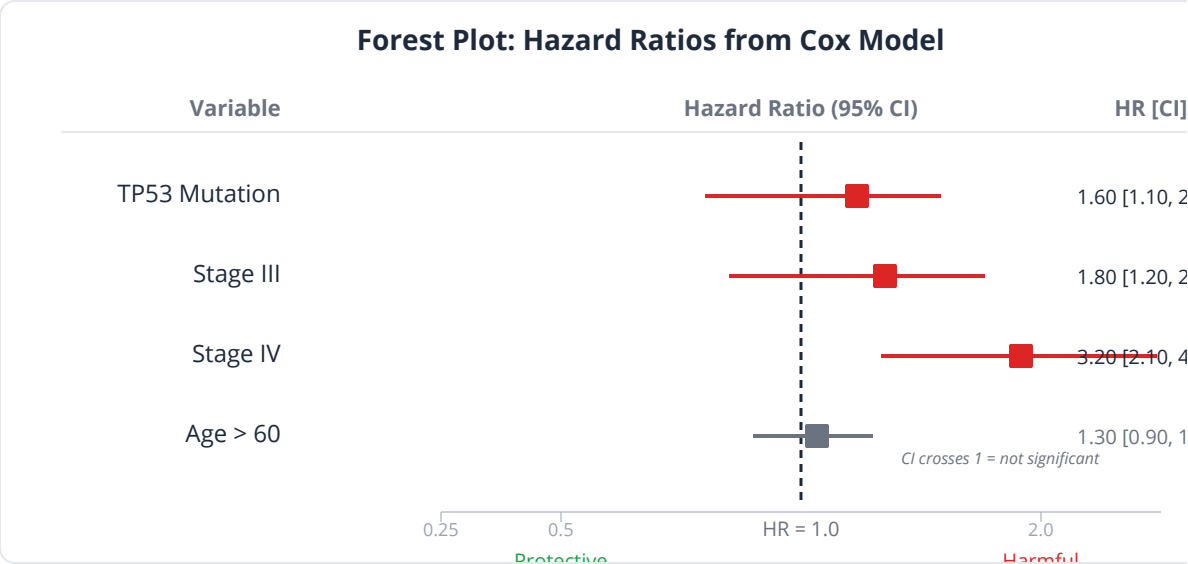
print("\n=== Multivariable Cox Model ===")
print("Hazard ratios: {cox_adjusted.hazard_ratios}")
print("p-values: {cox_adjusted.p_values}")

# Compare: does TP53 HR change after adjusting for stage?
print("\nTP53 HR unadjusted: {cox_simple.hazard_ratio |> round(2)}")
print("TP53 HR adjusted:    {cox_adjusted.hazard_ratios[0] |>
round(2)}")
```

Forest Plot of Hazard Ratios

```
# Visualize all HRs from the multivariable model
let hr_data = table({
  "predictor": ["TP53 Mutation", "Age (per year)", "Stage"],
  "hr": [1.82, 1.03, 1.55],
  "lower": [1.20, 1.01, 1.16],
  "upper": [2.76, 1.05, 2.08]
})
forest_plot(hr_data, {
  label: "predictor", estimate: "hr", lower: "lower", upper:
"upper"
})

# Left of 1 = protective, Right of 1 = harmful
```



Survival Curve from Cox Model

```
# Plot adjusted survival curves from KM estimates
let surv_table = table({
  "time": km_wt.times ++ km_mut.times,
  "survival": km_wt.survival ++ km_mut.survival,
  "group": km_wt.times |> map(|t| "TP53 WT") ++ km_mut.times |>
map(|t| "TP53 Mut")
})

plot(surv_table, {type: "line", x: "time", y: "survival", color:
"group",
  title: "Adjusted Survival Curves from Cox Model",
  x_label: "Time (months)", y_label: "Survival Probability"})
```

Checking Proportional Hazards

```
# Check proportional hazards assumption
# If hazard ratios change over time, PH is violated
# Visual check: compare early vs late HR
let early_times = []
let early_status = []
let early_tp53 = []
let late_times = []
let late_status = []
let late_tp53 = []

let midpoint = 24 # months
for i in 0..n {
  if survival_time[i] <= midpoint {
    early_times = early_times + [survival_time[i]]
    early_status = early_status + [status[i]]
    early_tp53 = early_tp53 + [tp53_mut[i]]
  } else {
    late_times = late_times + [survival_time[i]]
    late_status = late_status + [status[i]]
    late_tp53 = late_tp53 + [tp53_mut[i]]
  }
}

print("=== Proportional Hazards Check ===")
print("If HRs differ substantially, PH assumption may be violated")
# If  $p < 0.05$ , consider: stratified Cox model, time-varying
coefficients,
# or restricted mean survival time (RMST)
```

Python:

```
from lifelines import KaplanMeierFitter, CoxPHFitter
from lifelines.statistics import logrank_test

# Kaplan-Meier
kmf = KaplanMeierFitter()
kmf.fit(time_wt, event_wt, label='TP53 WT')
kmf.plot()
kmf.fit(time_mut, event_mut, label='TP53 Mut')
kmf.plot()

# Log-rank test
results = logrank_test(time_wt, time_mut, event_wt, event_mut)
print(f"p = {results.p_value:.4f}")

# Cox model
cph = CoxPHFitter()
cph.fit(df, duration_col='time', event_col='status')
cph.print_summary()
cph.plot() # forest plot

# Check PH assumption
cph.check_assumptions(df, show_plots=True)
```

R:

```
library(survival)
library(survminer)

# Kaplan-Meier
km_fit <- survfit(Surv(time, status) ~ tp53, data = df)

# Publication-quality KM plot
ggsurvplot(km_fit,
            pval = TRUE, risk.table = TRUE,
            palette = c("#2196F3", "#F44336"))

# Log-rank test
survdifff(Surv(time, status) ~ tp53, data = df)

# Cox model
cox_model <- coxph(Surv(time, status) ~ tp53 + age + stage, data =
df)
summary(cox_model)

# Forest plot
ggforest(cox_model)

# Proportional hazards test
cox.zph(cox_model)
```

Exercises

Exercise 1: Kaplan-Meier and Median Survival

A clinical trial follows 200 breast cancer patients treated with either chemotherapy or chemotherapy + targeted therapy. Compute KM curves and median survival for both arms.

```
set_seed(42)
let n = 200
let treatment = rnorm(n, 0, 1) |> map(|x| if x > 0 { 1 } else { 0 })
# 0=chemo, 1=chemo+targeted

# Simulate survival data
# Targeted therapy reduces hazard by 30%
# Generate survival times and censoring

# 1. Compute kaplan_meier() for each treatment arm
# 2. Plot KM curves with plot()
# 3. Report median survival for each arm
# 4. What is the absolute improvement in median survival?
```

Exercise 2: Log-Rank Test with Multiple Groups

Compare survival across four molecular subtypes (Luminal A, Luminal B, HER2+, Triple-negative). Which pairs differ significantly?

```
set_seed(42)
let n = 300
# Assign subtypes: 0=LumA, 1=LumB, 2=HER2+, 3=TNBC
let subtype = rnorm(n, 0, 1) |> map(|x|
  if x < -0.39 { 0 }
  else if x < 0.25 { 1 }
  else if x < 0.64 { 2 }
  else { 3 })

# Simulate different hazard rates by subtype
# Luminal A: lowest hazard, Triple-neg: highest

# 1. Compute kaplan_meier() for all 4 subtypes
# 2. Fit cox_ph() with subtype as predictor
# 3. Which subtypes have the best and worst prognosis?
```

Exercise 3: Multivariable Cox Model

Build a Cox model with TP53 status, age, stage, and smoking status. Determine which factors are independently prognostic after adjustment.

```
let n = 300

# Simulate covariates and survival
# Fit cox_ph() with all 4 predictors
# 1. Report hazard ratios for each predictor
# 2. Which predictors are significant after adjustment?
# 3. Create a forest_plot()
# 4. Does the TP53 hazard ratio change from univariate to
multivariable?
```

Exercise 4: Checking the PH Assumption

Simulate a scenario where the proportional hazards assumption is violated — an immunotherapy that has no effect in the first 3 months but a strong effect afterward (delayed separation). Show that the PH test flags this.

```
# Simulate two groups:
# Control: constant hazard
# Treatment: same hazard for months 0-3, then 50% reduced hazard

# 1. Plot KM curves – do they cross or show delayed separation?
# 2. Fit cox_ph() and check if HR changes over time
# 3. Is the PH assumption violated?
# 4. What would you do in practice?
```

Exercise 5: Complete Survival Analysis Pipeline

Perform a full survival analysis: KM curves, log-rank test, univariate Cox for each predictor, multivariable Cox, forest plot. Write a one-paragraph summary of findings.

```
# Use 400 patients with 5 clinical/genomic variables
# Run the complete pipeline from raw data to clinical interpretation
```


Key Takeaways

- **Censored observations** contain real information — survival analysis methods handle them correctly while standard methods cannot
- **Kaplan-Meier** estimates survival probabilities as a step function; **median survival** is the primary summary measure
- The **log-rank test** compares survival curves between groups (the “t-test” of survival analysis)
- **Cox proportional hazards** regression models the effect of multiple covariates on hazard; the **hazard ratio** is the key output
- HR = 2.0 means twice the instantaneous risk, NOT twice as fast to die or half the survival time
- Always **check the proportional hazards assumption** — violations (crossing curves, delayed effects) invalidate the standard Cox model
- **Forest plots** are the standard visualization for hazard ratios from Cox models
- Adjusted HRs (controlling for confounders) are more clinically meaningful than unadjusted ones

What's Next

We've been analyzing data. But before collecting data, there's a critical question: **How many samples do you need?** Day 18 covers experimental design and statistical power — the science of planning studies that can actually detect the effects you're looking for.

Day 18: Experimental Design and Statistical Power

The Problem

Dr. Ana Reyes is a junior PI writing her first R01 grant. She proposes a study comparing gene expression between psoriatic skin and normal skin using RNA-seq, planning **3 samples per group** because “that’s what the lab down the hall used.”

The grant comes back with this reviewer comment:

“The proposed sample size of $n=3$ per group is inadequate. The applicant provides no power analysis to justify this number. With 3 replicates, the study is severely underpowered to detect anything less than a 4-fold change, which is biologically unrealistic for most genes. We recommend at least 8-10 biological replicates per condition based on published power analyses for RNA-seq DE studies.”

Grant rejected. Six months of proposal writing, wasted — because she didn’t plan the sample size.

Statistical power determines whether your study can actually detect the effect you’re looking for. Getting it wrong wastes time, money, animals, and patient samples.

What Is Statistical Power?

Power analysis sits at the intersection of four interconnected quantities:

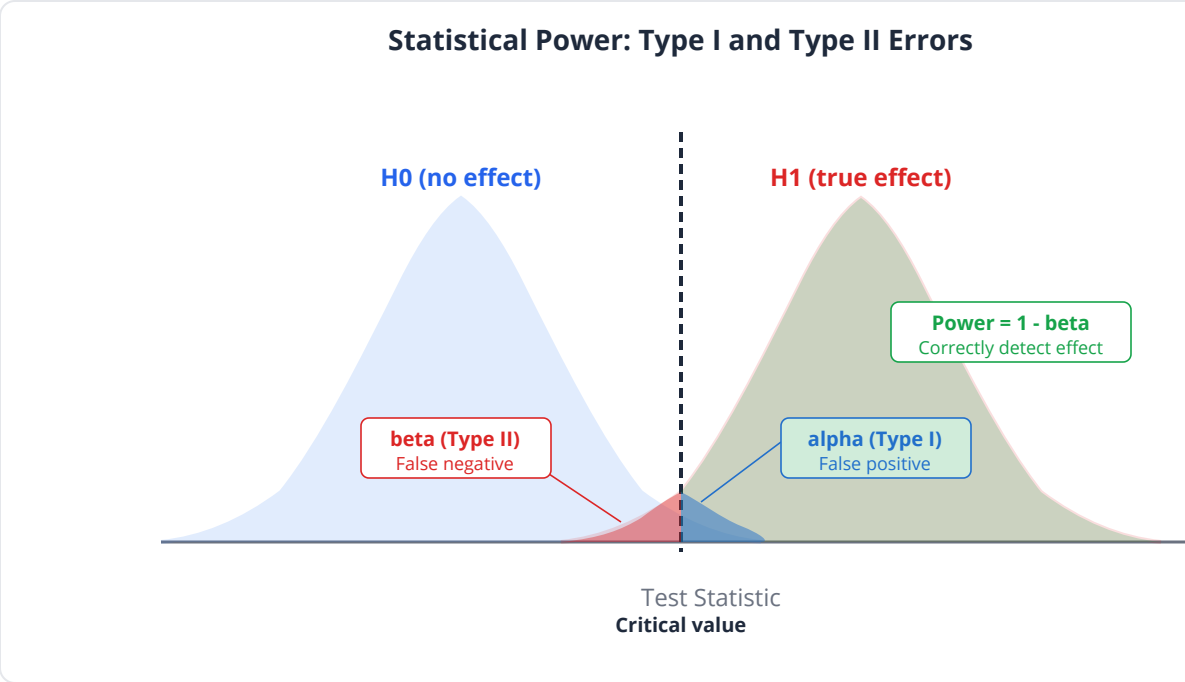
Quantity	Symbol	Definition	Typical Value
Significance level	α	Probability of false positive (Type I error)	0.05
Power	$1 - \beta$	Probability of detecting a true effect	0.80 (80%)
Effect size	d, Δ	Magnitude of the real biological difference	Varies
Sample size	n	Number of observations per group	What you solve for

The fundamental relationship: Given any three, you can calculate the fourth.

Type I and Type II Errors

	H₀ True (no real effect)	H₀ False (real effect exists)
Reject H₀	Type I error (α) — false positive	Correct! (Power = $1 - \beta$)
Fail to reject H₀	Correct! ($1 - \alpha$)	Type II error (β) — false negative

Key insight: An underpowered study has a high β — it frequently misses real effects. This doesn't just waste resources; it can lead to the false conclusion that an effect doesn't exist, discouraging further research on a real phenomenon.



Why 80% Power?

The convention of 80% power means accepting a 20% chance of missing a real effect. Some contexts demand higher:

Context	Minimum Power	Rationale
Exploratory study	70-80%	Acceptable miss rate for discovery
Confirmatory clinical trial	80-90%	Regulatory requirement
Safety/non-inferiority trial	90-95%	Must not miss harmful effects
Rare disease (limited patients)	60-70%	Pragmatic constraint

Effect Size: The Missing Ingredient

Effect size is the hardest quantity to estimate because it requires knowledge about the biology before doing the experiment. Sources:

Source	Approach
Pilot data	Small preliminary experiment (best source)
Literature	Previous studies on similar questions
Clinical significance	“What’s the smallest difference that matters?”
Conventions	Cohen’s standards (d = 0.2 small, 0.5 medium, 0.8 large)

Common pitfall: Using an inflated effect size from a small pilot study. Small studies overestimate effects (the “winner’s curse”). If your pilot with n=5 shows d=1.5, the true effect is probably smaller. Be conservative.

Cohen’s d for Two-Group Comparisons

$$d = \frac{|\mu_1 - \mu_2|}{\sigma_{\text{pooled}}}$$

Cohen’s d	Interpretation	Biological Example
0.2	Small	Subtle expression change between tissues
0.5	Medium	DE gene in RNA-seq (2-fold change)
0.8	Large	Drug vs. placebo in responsive patients
1.2	Very large	Knockout vs. wild-type for target gene

Power for Common Designs

Two-Group Comparison (t-test)

The most basic design: compare means between two independent groups.

Required sample size per group (approximate, for $\alpha=0.05$, power=0.80):

Effect Size (d)	n per group
0.2 (small)	394
0.5 (medium)	64
0.8 (large)	26
1.0	17
1.5	9
2.0	6

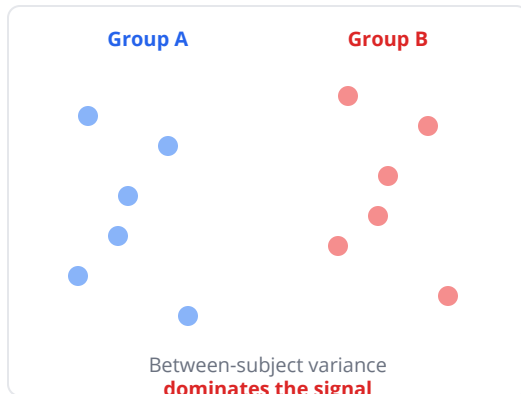
Key insight: Detecting a small effect requires nearly 400 samples per group! This is why GWAS studies need thousands of subjects — individual genetic variants typically have very small effects ($d \approx 0.1-0.2$).

Paired Design (Paired t-test)

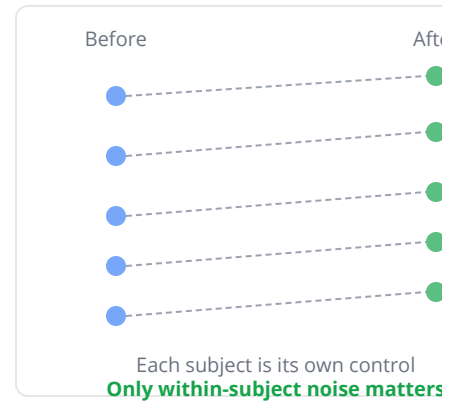
When the same subjects are measured before and after treatment, pairing removes between-subject variability. Power increases dramatically:

Paired vs. Unpaired Design: Why Pairing Reduces Noise

Unpaired (Independent Groups)



Paired (Same Subjects)



Correlation between pairs	Power improvement
0.3	~30% fewer samples needed
0.5	~50% fewer samples needed
0.7	~70% fewer samples needed

RNA-seq Differential Expression

RNA-seq power depends on additional factors:

Factor	Effect on Power
Sequencing depth	More reads → more power for low-expression genes
Biological replicates	THE major driver of power
Fold change threshold	Larger FC → easier to detect
Dispersion	Higher variability → need more samples

Rules of thumb for RNA-seq DE:

- **Minimum:** 3 biological replicates (detects only >4-fold changes)
- **Good:** 6-8 replicates (detects 2-fold changes)

- **Ideal:** 12+ replicates (detects 1.5-fold changes)
- Technical replicates have diminishing returns — invest in biological replicates

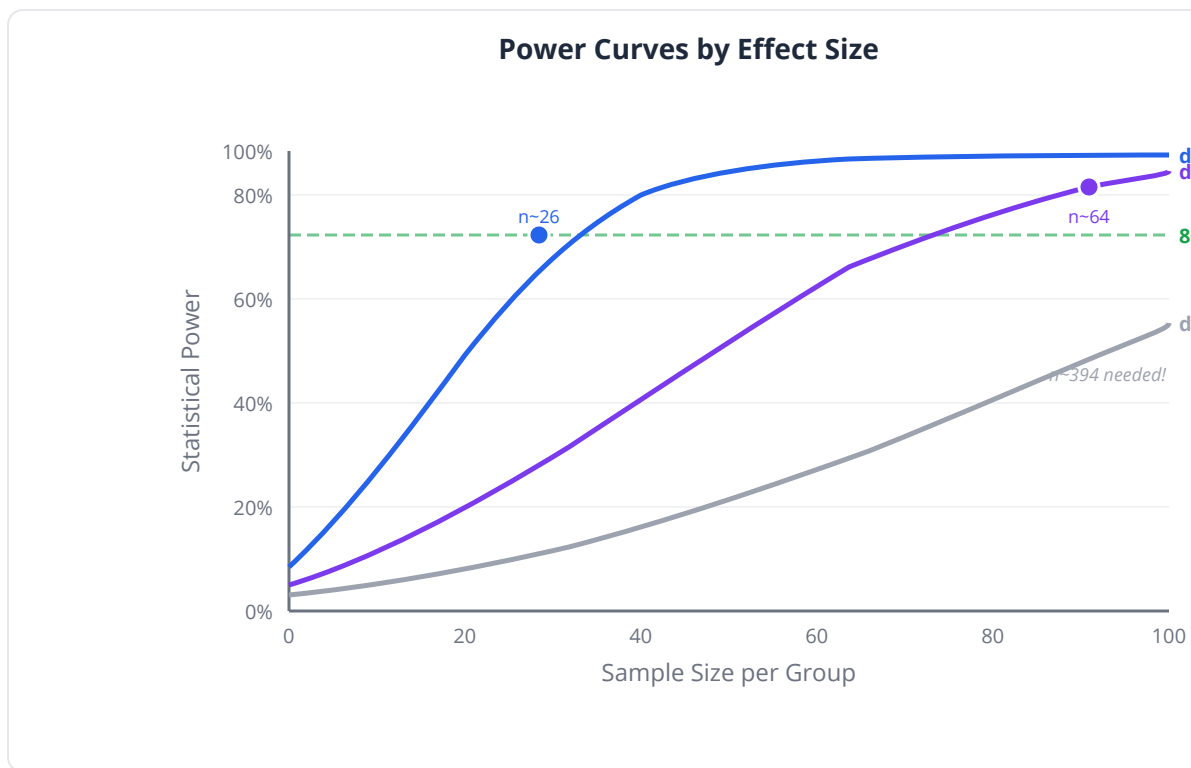
Common pitfall: Confusing technical replicates (sequencing the same library twice) with biological replicates (independent biological samples). Only biological replicates give you power to generalize. Ten technical replicates of one sample give you $n=1$, not $n=10$.

GWAS

Study Type	Typical n	Detectable Effect
Candidate gene	500-1000	Large OR (>2.0)
Moderate GWAS	5,000-10,000	Medium OR (1.3-1.5)
Large GWAS	50,000-500,000	Small OR (1.05-1.1)
UK Biobank scale	500,000+	Tiny effects

Power Curves

Power curves show how power varies with sample size for different effect sizes. They're the most informative visualization for study planning — you can see the “sweet spot” where adding more samples gives diminishing returns.



The Cost of Underpowered Studies

An underpowered study is not just a failed study — it's actively harmful:

1. **Waste:** Money, time, and irreplaceable biological samples consumed for inconclusive results
2. **Publication bias:** Only "lucky" underpowered studies (that happen to find $p < 0.05$) get published, inflating reported effect sizes
3. **False negatives:** Real treatments or biomarkers get abandoned
4. **Ethical cost:** Patients enrolled in clinical trials with no realistic chance of detecting a benefit

Clinical relevance: The FDA and EMA require power analyses for all clinical trial protocols. Journal reviewers increasingly require them for observational studies too. "How many samples do you need?" is the first question of good experimental design.

Experimental Design in BioLang

Basic Power Analysis for t-test

```
set_seed(42)
# How many samples to detect a 2-fold change in gene expression?

# Parameters
let alpha = 0.05
let power_target = 0.80
let effect_size = 0.8    # Cohen's d for ~2-fold change

# Simulate power at different sample sizes
let sample_sizes = [5, 10, 15, 20, 30, 50, 75, 100]
let n_simulations = 1000

print("=== Power Analysis: Two-Sample t-test ===")
print("Effect size (Cohen's d): {effect_size}")
print("Alpha: {alpha}")
print("")
print("n per group      Estimated Power")

for n in sample_sizes {
  let significant = 0

  for sim in 0..n_simulations {
    # Simulate two groups with known effect
    let group1 = rnorm(n, 0, 1)
    let group2 = rnorm(n, effect_size, 1)

    let result = ttest(group1, group2)
    if result.p_value < alpha {
      significant = significant + 1
    }
  }

  let power = significant / n_simulations
  let marker = if power >= power_target { " <-- sufficient" } else
{ "" }
  print("{n}          {power |> round(3)}{marker}")
}
```

Power Curves for Different Effect Sizes

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# Visualize power as a function of sample size
let sample_sizes = [5, 10, 15, 20, 25, 30, 40, 50, 75, 100]
let effect_sizes = [0.3, 0.5, 0.8, 1.2]
let n_sims = 500

let power_curves = {}

for d in effect_sizes {
  let powers = []

  for n in sample_sizes {
    let sig_count = 0
    for s in 0..n_sims {
      let g1 = rnorm(n, 0, 1)
      let g2 = rnorm(n, d, 1)
      if ttest(g1, g2).p_value < 0.05 {
        sig_count = sig_count + 1
      }
    }
    powers = powers + [sig_count / n_sims]
  }

  power_curves["{d}"] = powers
}

# Plot power curves
let curve_table = table({
  "n": sample_sizes,
  "d_0.3": power_curves["0.3"],
  "d_0.5": power_curves["0.5"],
  "d_0.8": power_curves["0.8"],
  "d_1.2": power_curves["1.2"]
})
plot(curve_table, {type: "line", x: "n",
  title: "Power Curves: Two-Sample t-test",
  x_label: "Sample Size per Group", y_label: "Statistical Power"})
```

RNA-seq Experiment Design

```
set_seed(42)
# How many biological replicates for RNA-seq DE?

# Simulate RNA-seq-like data
let fold_changes = [1.5, 2.0, 3.0, 4.0]
let replicates = [3, 5, 8, 12, 20]
let n_sims = 200

print("=== RNA-seq Power by Fold Change and Replicates ===")
print("FC          n=3      n=5      n=8      n=12     n=20")

for fc in fold_changes {
  let powers = []

  for n in replicates {
    let detected = 0

    for sim in 0..n_sims {
      # Simulate one gene with known fold change
      let control = rnorm(n, 10, 2)
      let treatment = rnorm(n, 10 * fc, 2 * fc)

      # Log-transform (as in real RNA-seq analysis)
      let log_ctrl = control |> map(|x| log2(max(x, 0.1)))
      let log_treat = treatment |> map(|x| log2(max(x, 0.1)))

      let p = ttest(log_ctrl, log_treat).p_value
      if p < 0.05 { detected = detected + 1 }
    }

    powers = powers + [detected / n_sims]
  }

  print("{fc}      " ++ powers |> map(|p| "{(p * 100) |>
round(0)}%") |> join("  "))
}

# Key takeaway: n=3 barely detects 4-fold changes;
# n=8 reliably detects 2-fold changes
```

Paired vs. Unpaired Design

```
set_seed(42)
# Show the power advantage of paired designs
let n_sims = 1000
let n = 20
let effect = 0.5 # medium effect
let subject_sd = 2.0 # between-subject variability
let within_sd = 0.5 # within-subject variability

let power_unpaired = 0
let power_paired = 0

for sim in 0..n_sims {
  # Unpaired: independent groups
  let group1 = rnorm(n, 0, subject_sd)
  let group2 = rnorm(n, effect, subject_sd)
  if ttest(group1, group2).p_value < 0.05 {
    power_unpaired = power_unpaired + 1
  }

  # Paired: same subjects, before and after
  let baseline = rnorm(n, 0, subject_sd)
  let after = baseline + effect + rnorm(n, 0, within_sd)
  let diff = after - baseline
  if ttest_one(diff, 0).p_value < 0.05 {
    power_paired = power_paired + 1
  }
}

print("=== Paired vs Unpaired Design (n={n}, d={effect}) ===")
print("Unpaired power: {(power_unpaired / n_sims * 100) |>
round(1)}%")
print("Paired power: {(power_paired / n_sims * 100) |>
round(1)}%")
print("Pairing advantage: {((power_paired - power_unpaired) / n_sims
* 100) |> round(1)} percentage points")
```

Multi-Group Design (ANOVA)

```
set_seed(42)
# Power for detecting differences among 4 treatment groups
let n_sims = 500
let k = 4 # number of groups
let group_means = [0, 0.3, 0.6, 0.9] # increasing effect
let sample_sizes = [5, 10, 15, 20, 30]

print("=== ANOVA Power (k={k} groups) ===")
for n in sample_sizes {
  let sig = 0

  for sim in 0..n_sims {
    let groups = []
    for i in 0..k {
      groups = groups + [rnorm(n, group_means[i], 1)]
    }

    let result = anova(groups)
    if result.p_value < 0.05 { sig = sig + 1 }
  }

  print("n = {n} per group: power = {(sig / n_sims * 100) |>
round(1)}%")
}
```

Sample Size Recommendation Report

```
set_seed(42)
# Generate a complete sample size recommendation
let scenarios = [
  { name: "Conservative (d=0.5)", effect: 0.5 },
  { name: "Expected (d=0.8)", effect: 0.8 },
  { name: "Optimistic (d=1.2)", effect: 1.2 }
]

print("=== SAMPLE SIZE RECOMMENDATION REPORT ===")
print("Two-group comparison, alpha=0.05, power=80%")
print("")

for s in scenarios {
  # Find minimum n for 80% power via simulation
  let required_n = 0
  for n in 5..200 {
    let power = 0
    for sim in 0..500 {
      let g1 = rnorm(n, 0, 1)
      let g2 = rnorm(n, s.effect, 1)
      if ttest(g1, g2).p_value < 0.05 { power = power + 1 }
    }
    if power / 500 >= 0.80 {
      required_n = n
      break
    }
  }

  print("{s.name}: n = {required_n}/group")
}

print("")
print("Recommendation: Plan for the CONSERVATIVE")
print("estimate + 10-20% for dropout/QC failures")
```

Python:

```

from scipy.stats import norm
from statsmodels.stats.power import TTestIndPower, TTestPower

# Power analysis for two-sample t-test
analysis = TTestIndPower()

# Required sample size
n = analysis.solve_power(effect_size=0.8, alpha=0.05, power=0.80)
print(f"Required n per group: {n:.0f}")

# Power curve
import matplotlib.pyplot as plt
fig = analysis.plot_power(
    dep_var='nobs', nobs=range(5, 100),
    effect_size=[0.3, 0.5, 0.8, 1.2])

# Simulation-based power
import numpy as np
from scipy.stats import ttest_ind

def simulate_power(n, d, n_sim=1000):
    sig = sum(ttest_ind(np.random.normal(0, 1, n),
                        np.random.normal(d, 1, n)).pvalue < 0.05
               for _ in range(n_sim))
    return sig / n_sim

```

R:

```

# Power analysis for two-sample t-test
power.t.test(d = 0.8, sig.level = 0.05, power = 0.80)

# Power curve
library(pwr)
pwr.t.test(d = 0.8, sig.level = 0.05, power = 0.80)

# RNA-seq specific power
library(RNASeqPower)
rnapower(depth = 20e6, cv = 0.4, effect = 2,
          alpha = 0.05, power = 0.8)

# Simulation-based
library(simr)
# simr provides power simulation for mixed models

```

Exercises

Exercise 1: Power for Your Study

You're planning a study comparing tumor mutation burden between immunotherapy responders and non-responders. Pilot data suggests $d \approx 0.6$ with $SD = 5$ mutations/Mb. How many patients per group do you need for 80% power?

- # 1. Simulate power at $n = 10, 20, 30, 50, 75, 100$
- # 2. Find the minimum n for 80% power
- # 3. Add 15% for anticipated dropout
- # 4. Create a power curve plot

Exercise 2: Paired vs. Unpaired

A study can either use 30 independent samples per group OR 30 paired before/after measurements. The between-subject SD is 3x the within-subject SD. Compare the power of both designs.

- # 1. Simulate paired and unpaired designs with $n=30$
- # 2. Effect size $d = 0.5$
- # 3. Between-subject $SD = 3$, within-subject $SD = 1$
- # 4. Which design achieves higher power?
- # 5. How many unpaired samples would match the paired design's power?

Exercise 3: RNA-seq Planning

You're designing an RNA-seq experiment to identify genes with at least 1.5-fold change between two conditions. Your budget allows either 6 samples at 30M reads each or 12 samples at 15M reads each. Which design has more power?

```
# Simulate both scenarios
# Track how many "true DE genes" each design detects
# Which is better: more depth or more replicates?
```

Exercise 4: The Underpowered Literature

Simulate 100 “studies” with $n=10$ per group and a true small effect ($d=0.3$).
Show that:

1. Most studies (>80%) fail to detect the effect
2. The “significant” studies dramatically overestimate the effect size
3. This creates a biased picture in the published literature

```
# 1. Run 100 simulated two-sample t-tests (n=10, d=0.3)
# 2. Count how many achieve p < 0.05
# 3. For the significant ones, compute Cohen's d from the data
# 4. Compare the average "published" d to the true d = 0.3
# 5. This is the "winner's curse" – published effects are inflated
```

Key Takeaways

- **Power analysis** determines how many samples you need BEFORE starting an experiment — it’s not optional, it’s essential
- The four pillars are **α** (false positive rate), **power** ($1-\beta$, false negative rate), **effect size**, and **sample size** — fix three, solve for the fourth
- **Underpowered studies** waste resources, inflate published effect sizes, and can falsely suggest an effect doesn’t exist
- **Biological replicates** drive power in genomics — technical replicates give diminishing returns
- For RNA-seq: $n=3$ is barely adequate (detects >4-fold), $n=8$ is good (2-fold), $n=12+$ is ideal (1.5-fold)
- **Paired designs** dramatically increase power by removing between-subject variability

- The **winner's curse**: underpowered studies that happen to be significant overestimate the true effect
- Always use **conservative** effect size estimates and add buffer for dropout/QC failures
- Power curves visualize the sample size / power trade-off and help identify the sweet spot

What's Next

Statistical significance tells you whether an effect is real, but not whether it **matters**. Day 19 introduces **effect sizes** — Cohen's d, odds ratios, relative risk — and the critical distinction between statistical significance and practical importance.

Day 19: Effect Sizes — Beyond p-Values

The Problem

Two papers land on Dr. Rachel Nguyen's desk the same morning.

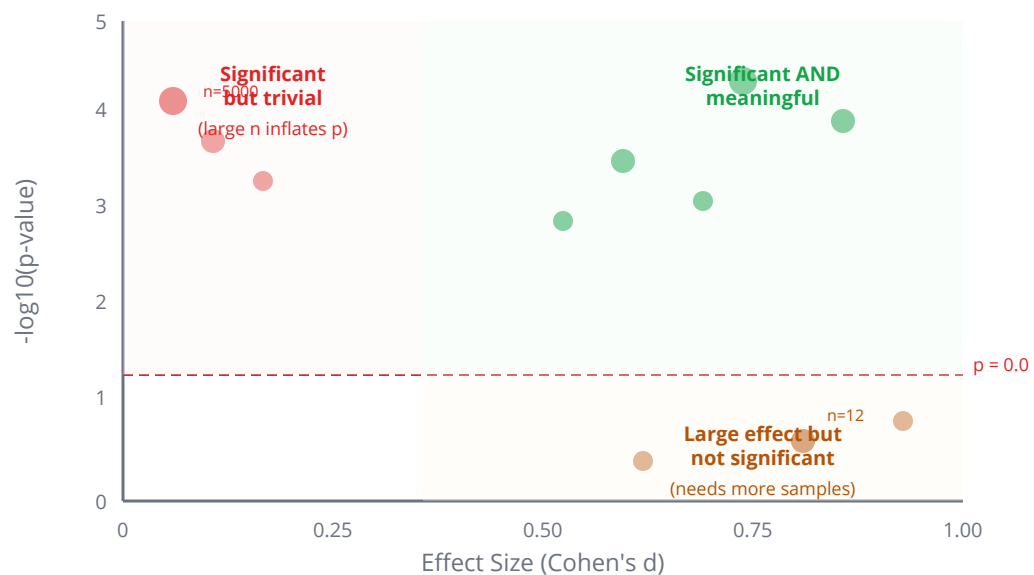
Paper A: "GENE_X is significantly differentially expressed between tumor and normal tissue ($p = 0.00001$, $n = 5,000$).\" She looks at the supplementary data: the mean difference is **0.02 FPKM** on a scale where genes range from 0 to 50,000 FPKM. The fold change is 1.001.

Paper B: \"DRUG_Y showed a non-significant trend toward tumor reduction ($p = 0.08$, $n = 24$).\" She looks at the data: the median tumor volume shrank by **40%** in the treatment arm.

Paper A is \"highly significant\" but biologically meaningless — the difference is lost in measurement noise. Paper B fails the significance threshold but describes a potentially life-changing clinical effect that just needs more patients.

The p-value alone tells you almost nothing. You need **effect sizes**.

P-Value vs. Effect Size: They Measure Different Things



The Tyranny of p-Values

In 2016, the American Statistical Association took the extraordinary step of issuing a formal statement on p-values — the first time in its 177-year history. Key points:

1. P-values do NOT measure the probability that the hypothesis is true
2. P-values do NOT measure the size or importance of an effect
3. Scientific conclusions should NOT be based only on whether $p < 0.05$
4. A p-value near 0.05 provides only weak evidence against the null

Key insight: A p-value is a function of **effect size** AND **sample size**. With enough data, any trivial difference becomes “significant.” With too few data, any real effect becomes “non-significant.” The p-value alone is fundamentally incomplete.

The Problem of Large n

True Effect Size	n per group	p-value	Significant?	Meaningful?
d = 0.01 (trivial)	50,000	0.02	Yes	No
d = 0.8 (large)	5	0.12	No	Yes
d = 0.5 (medium)	64	0.04	Yes	Likely
d = 0.3 (small)	30	0.15	No	Maybe

What Are Effect Sizes?

An effect size quantifies **how large** a difference or association is, independent of sample size. There are two families:

Standardized Effect Sizes (unit-free)

Metric	Used For	Scale
Cohen's d	Two-group mean difference	0.2 small, 0.5 medium, 0.8 large
Odds Ratio (OR)	Binary outcome association	1.0 = no effect
Relative Risk (RR)	Binary outcome risk	1.0 = no effect
Cramér's V	Categorical association	0 to 1
Eta-squared (η^2)	ANOVA variance explained	0 to 1
R ²	Regression variance explained	0 to 1

Unstandardized Effect Sizes (original units)

Metric	Example
Mean difference	"Treatment reduces tumor volume by 2.3 cm ³ "
Regression slope	"Each year of age increases risk by 3%"
Median survival difference	"Treatment arm survived 6 months longer"

Key insight: Unstandardized effect sizes are often MORE useful than standardized ones because they're in meaningful units. "The drug reduced blood pressure by 8 mmHg" is more interpretable than "Cohen's d = 0.5."

Cohen's d: Standardized Mean Difference

$$d = \frac{\bar{X}_1 - \bar{X}_2}{s_{\text{pooled}}}$$

where the pooled standard deviation is:

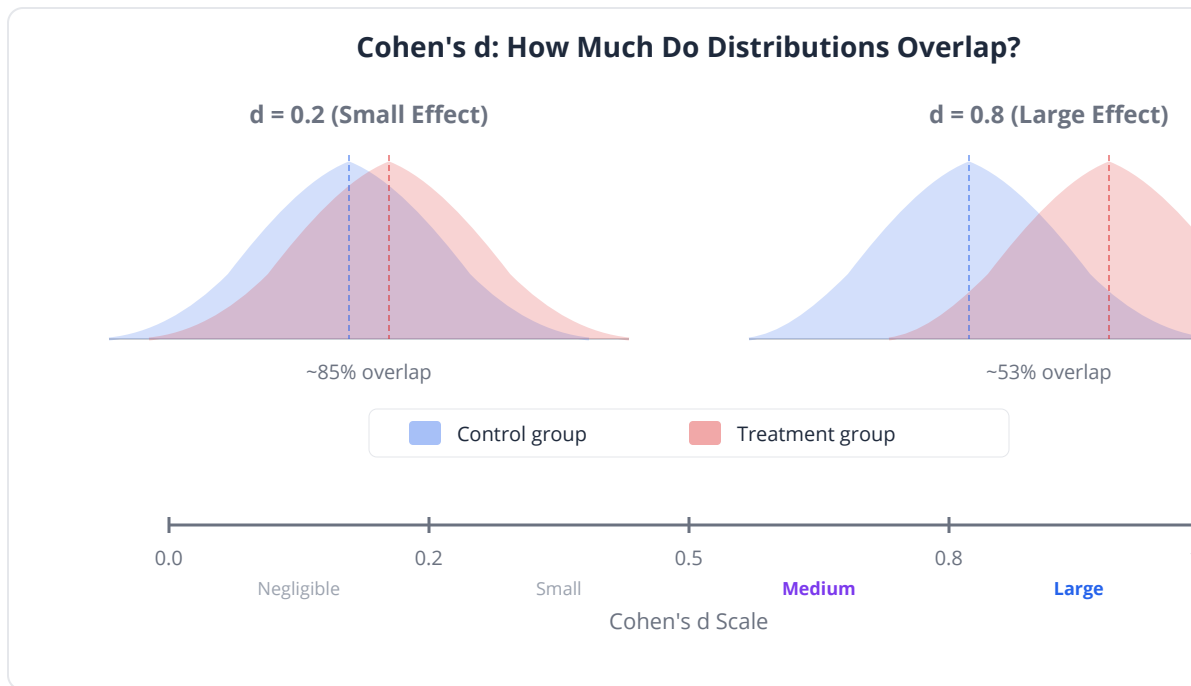
$$s_{\text{pooled}} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

Cohen's benchmarks (widely used, sometimes criticized as arbitrary):

d	Label	What It Means
0.2	Small	Groups overlap ~85%
0.5	Medium	Groups overlap ~67%
0.8	Large	Groups overlap ~53%
1.2	Very large	Groups overlap ~40%
2.0	Huge	Groups barely overlap

Common pitfall: Cohen himself warned that "small/medium/large" are context-dependent. In pharmacogenomics, d = 0.3 might be clinically

important. In quality control, $d = 2.0$ might be necessary. Always judge effect sizes in your domain context.



Odds Ratio and Relative Risk

For binary outcomes (response/no response, mutation/wild-type), two key measures:

Odds Ratio (OR)

$$OR = \frac{a \cdot d}{b \cdot c}$$

From a 2x2 table:

	Outcome +	Outcome -
Exposed	a	b
Unexposed	c	d

Relative Risk (RR)

$$RR = \frac{a/(a+b)}{c/(c+d)}$$

OR vs. RR: Why They Differ

Baseline Risk	OR	RR	Interpretation
1% (rare)	2.0	2.0	Nearly identical (rare disease approximation)
10%	2.0	1.8	Starting to diverge
30%	2.0	1.5	Substantial difference
50%	2.0	1.3	OR greatly overstates RR

Clinical relevance: Case-control studies can only estimate OR (not RR). Cohort studies and RCTs can estimate both. When reporting to patients, RR or absolute risk difference is more intuitive: “Your risk goes from 10% to 15%” is clearer than “OR = 1.6.”

Cramér’s V: Categorical Associations

For larger contingency tables (beyond 2x2), Cramér’s V measures association strength:

$$V = \sqrt{\frac{\chi^2}{n \cdot (k-1)}}$$

where $k = \min(\text{rows}, \text{columns})$.

V	Interpretation
0.0	No association
0.1	Weak
0.3	Moderate
0.5	Strong

Eta-squared: ANOVA Effect Size

For ANOVA (comparing means across multiple groups):

$$\eta^2 = \frac{SS_{\text{between}}}{SS_{\text{total}}}$$

η^2	Interpretation
0.01	Small
0.06	Medium
0.14	Large

Eta-squared tells you what fraction of total variance is explained by group membership.

Forest Plots: The Standard Display

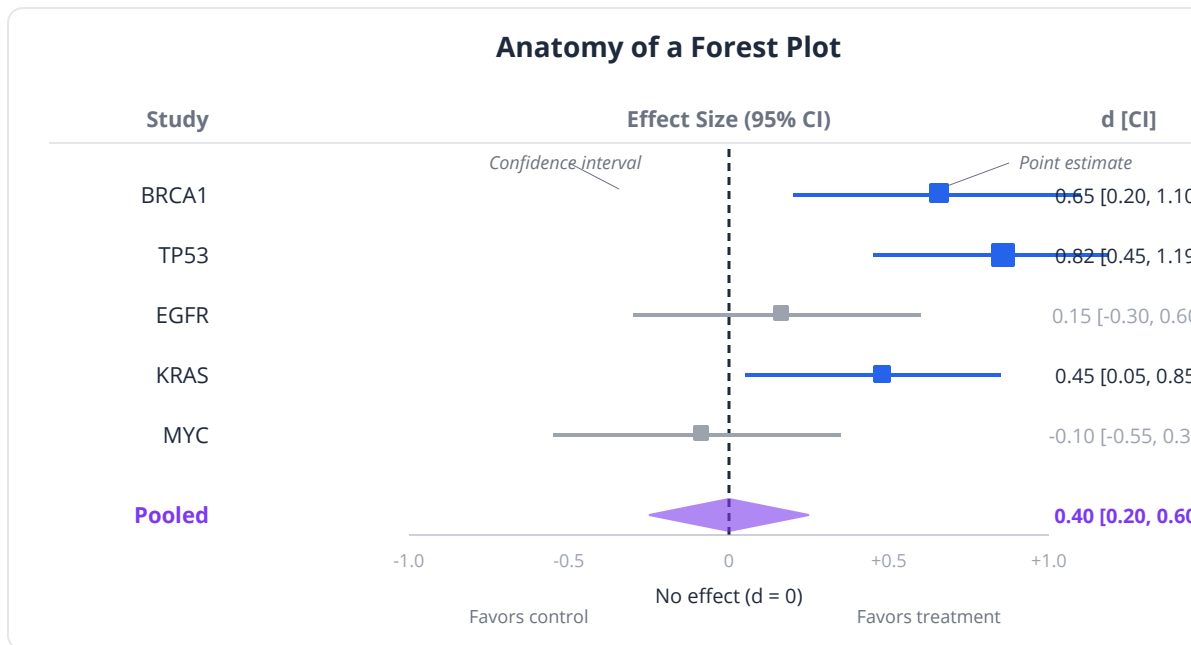
Forest plots are the gold standard for displaying effect sizes across multiple comparisons. Each row shows:

- A point estimate (the effect size)
- A horizontal line (the confidence interval)
- A vertical reference line (null effect: 0 for differences, 1 for ratios)

They're essential for:

- Meta-analyses (combining studies)
- Multi-gene comparisons (e.g., DE genes ranked by effect size)

- Subgroup analyses (effect by age, sex, stage)
- Cox regression hazard ratios



The Reporting Checklist

Every statistical result should include all four pieces:

Element	What It Answers	Example
Effect size	How large?	d = 0.72
Confidence interval	How precise?	95% CI [0.35, 1.09]
p-value	Could it be chance?	p = 0.003
Sample size	How much data?	n = 45 per group

Omitting any one of these leaves the reader unable to fully evaluate the finding.

Effect Sizes in BioLang

Cohen's d for Gene Expression

```
set_seed(42)
# Compare expression of a gene between tumor and normal
let n = 50

let tumor = rnorm(n, 12.5, 3.0)
let normal_expr = rnorm(n, 10.0, 2.8)

# Cohen's d – compute inline
let pooled_sd = sqrt((variance(tumor) + variance(normal_expr)) /
2.0)
let d = (mean(tumor) - mean(normal_expr)) / pooled_sd
print("=== Cohen's d ===")
print("d = {d |> round(3)}")

let interpretation = if abs(d) >= 0.8 { "large" }
  else if abs(d) >= 0.5 { "medium" }
  else if abs(d) >= 0.2 { "small" }
  else { "negligible" }

print("Interpretation: {interpretation} effect")
print("")

# Also report the raw difference
let mean_diff = mean(tumor) - mean(normal_expr)
print("Mean difference: {mean_diff |> round(2)} FPKM")
print("This means tumor expression is ~{(mean_diff /
mean(normal_expr) * 100) |> round(0)}% higher")

# Complete report: effect + CI + p + n
let t = ttest(tumor, normal_expr)
print("\n=== Complete Report ===")
print("Cohen's d = {d |> round(2)}")
print("p = {t.p_value |> round(4)}, n = {n} per group")
```

Odds Ratio and Relative Risk

```
# Immunotherapy response by PD-L1 status

# 2x2 table:
#           Respond  Non-respond
# PD-L1 high    35      15      (70% response)
# PD-L1 low     20      30      (40% response)

let a = 35 # PD-L1 high + respond
let b = 15 # PD-L1 high + non-respond
let c = 20 # PD-L1 low + respond
let d_val = 30 # PD-L1 low + non-respond

# Odds ratio – compute inline
let or_val = (a * d_val) / (b * c)
print("=== Odds Ratio ===")
print("OR = {or_val |> round(2)}")

# Relative risk – compute inline
let risk_high = a / (a + b)
let risk_low = c / (c + d_val)
let rr_val = risk_high / risk_low
print("\n=== Relative Risk ===")
print("RR = {rr_val |> round(2)}")

# Absolute risk difference
let ard = risk_high - risk_low
print("\n=== Absolute Risk Difference ===")
print("Risk (PD-L1 high): {(risk_high * 100) |> round(1)}%")
print("Risk (PD-L1 low): {(risk_low * 100) |> round(1)}%")
print("Absolute difference: {(ard * 100) |> round(1)} percentage points")
print("NNT: {(1 / ard) |> round(1)} (treat this many to get 1 extra responder)")

# Fisher's exact test for significance
let fe = fisher_exact(a, b, c, d_val)
print("Fisher's exact p-value: {fe.p_value |> round(4)}")

# Note: OR overstates the relative risk when the outcome is common
```

Cramér's V for Categorical Data

```
# Conceptual or diagnostic example; not directly executable.
# Association between tumor subtype (4 types) and treatment response
# (3 levels)
# observed counts in a 4x3 contingency table
let observed = [
  [30, 15, 5],   # Luminal A
  [20, 20, 10],  # Luminal B
  [10, 15, 25],  # HER2+
  [5, 10, 35]   # Triple-neg
]

# Flatten for chi-square test
let obs_flat = [30, 15, 5, 20, 20, 10, 10, 15, 25, 5, 10, 35]
let total = obs_flat |> sum
let chi2 = chi_square(obs_flat, obs_flat |> map(|x| total /
len(obs_flat)))

# Compute Cramer's V inline
let n_obs = total
let k = 3 # min(rows, cols)
let v = sqrt(chi2.statistic / (n_obs * (k - 1)))

print("=== Cramer's V ===")
print("Chi-square: {chi2.statistic |> round(2)}")
print("p-value: {chi2.p_value |> round(4)}")
print("Cramer's V: {v |> round(3)}")
print("Interpretation: {if v > 0.3 { \"moderate to strong\" } else {
\"weak\" }} association")
```


Eta-squared for ANOVA

```
set_seed(42)
# Expression differences across 4 cancer subtypes
let subtype_a = rnorm(30, 10, 3)
let subtype_b = rnorm(30, 12, 3)
let subtype_c = rnorm(30, 11, 3)
let subtype_d = rnorm(30, 15, 3)

let aov = anova([subtype_a, subtype_b, subtype_c, subtype_d])

# Compute eta-squared inline from ANOVA output
# eta2 = SS_between / SS_total
let eta2 = aov.ss_between / aov.ss_total

print("=== Eta-squared (ANOVA Effect Size) ===")
print("F = {aov.f_statistic |> round(2)}, p = {aov.p_value |>
round(4)}")
print("eta2 = {eta2 |> round(3)}")
print("{(eta2 * 100) |> round(1)}% of expression variance is
explained by subtype")

let eta_interp = if eta2 >= 0.14 { "large" }
  else if eta2 >= 0.06 { "medium" }
  else { "small" }
print("Interpretation: {eta_interp} effect")
```

Forest Plot for Multiple Genes

```
set_seed(42)
# Cohen's d for 10 differentially expressed genes
let gene_names = ["BRCA1", "TP53", "EGFR", "KRAS", "MYC",
                  "PTEN", "PIK3CA", "BRAF", "CDH1", "RB1"]
let effect_sizes = []
let ci_lowers = []
let ci_uppers = []

for i in 0..10 {
  let true_effect = rnorm(1, 0.5, 0.4)[0]
  let tumor_g = rnorm(40, 10 + true_effect, 2)
  let normal_g = rnorm(40, 10, 2)

  # Compute Cohen's d inline
  let pooled = sqrt((variance(tumor_g) + variance(normal_g)) /
2.0)
  let d = (mean(tumor_g) - mean(normal_g)) / pooled
  let se_d = sqrt(2 / 40 + d ** 2 / (4 * 40))

  effect_sizes = effect_sizes + [d]
  ci_lowers = ci_lowers + [d - 1.96 * se_d]
  ci_uppers = ci_uppers + [d + 1.96 * se_d]
}

# Forest plot – the standard effect size visualization
let forest_data = table({
  "gene": gene_names,
  "effect": effect_sizes,
  "ci_lower": ci_lowers,
  "ci_upper": ci_uppers
})

forest_plot(forest_data, {
  label: "gene", estimate: "effect",
  lower: "ci_lower", upper: "ci_upper"
})
```

Contrasting “Significant Tiny” vs. “Non-Significant Large”

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# The key lesson: significance does not equal importance

# Scenario 1: huge n, tiny effect
let n_large = 5000
let group1_large = rnorm(n_large, 10.000, 2)
let group2_large = rnorm(n_large, 10.020, 2)

let t_large = ttest(group1_large, group2_large)
let pooled_lg = sqrt((variance(group1_large) +
variance(group2_large)) / 2.0)
let d_large = (mean(group1_large) - mean(group2_large)) / pooled_lg

print("=== Scenario 1: Large n, Tiny Effect ===")
print("n = {n_large} per group")
print("Mean difference: {(mean(group1_large) - mean(group2_large))
|> abs |> round(4)}")
print("Cohen's d: {d_large |> round(4)}")
print("p-value: {t_large.p_value |> round(6)}")
print("Significant? {if t_large.p_value < 0.05 { \"YES\" } else { \"NO\"
}}")
print("Biologically meaningful? VERY UNLIKELY (d ~ 0.01)")

# Scenario 2: small n, large effect
let n_small = 12
let group1_small = rnorm(n_small, 10, 3)
let group2_small = rnorm(n_small, 13, 3)

let t_small = ttest(group1_small, group2_small)
let pooled_sm = sqrt((variance(group1_small) +
variance(group2_small)) / 2.0)
let d_small = (mean(group1_small) - mean(group2_small)) / pooled_sm

print("\n=== Scenario 2: Small n, Large Effect ===")
print("n = {n_small} per group")
print("Mean difference: {(mean(group1_small) - mean(group2_small))
|> abs |> round(2)}")
print("Cohen's d: {d_small |> round(2)}")
print("p-value: {t_small.p_value |> round(4)}")
print("Significant? {if t_small.p_value < 0.05 { \"YES\" } else { \"NO\"
}}")
print("Biologically meaningful? LIKELY (d ~ 1.0) – needs more
samples")
```

```

print("\n=== Lesson ===")
print("Scenario 1 is 'significant' but meaningless")
print("Scenario 2 is 'non-significant' but potentially important")
print("ALWAYS report effect sizes alongside p-values")

```

Python:

```

from scipy.stats import norm
import numpy as np

# Cohen's d
def cohens_d(x, y):
    nx, ny = len(x), len(y)
    pooled_sd = np.sqrt(((nx-1)*np.var(x, ddof=1) + (ny-1)*np.var(y,
ddof=1)) / (nx+ny-2))
    return (np.mean(x) - np.mean(y)) / pooled_sd

# Odds ratio (from statsmodels)
from statsmodels.stats.contingency_tables import Table2x2
table = np.array([[35, 15], [20, 30]])
t = Table2x2(table)
print(f"OR = {t.oddsratio:.2f}, 95% CI: {t.oddsratio_confint()}")
print(f"RR = {t.riskratio:.2f}")

# Cramér's V
from scipy.stats import chi2_contingency
chi2, p, dof, expected = chi2_contingency(table)
n = table.sum()
cramers_v = np.sqrt(chi2 / (n * (min(table.shape) - 1)))

# Forest plot (using matplotlib)
import matplotlib.pyplot as plt
plt.errorbar(effect_sizes, range(len(genes)), xerr=ci_widths,
fmt='o')
plt.axvline(0, color='black', linestyle='--')

```

R:

```
# Cohen's d
library(effectsize)
cohens_d(tumor ~ group)

# Odds ratio
library(epitools)
oddsratio(table)
riskratio(table)

# Cramér's V
library(rcompanion)
cramerV(table)

# Eta-squared
library(effectsize)
eta_squared(aov_result)

# Forest plot
library(forestplot)
forestplot(labeltext = gene_names,
           mean = effect_sizes,
           lower = ci_lower,
           upper = ci_upper)
```

Exercises

Exercise 1: Cohen's d Across Genes

Compute Cohen's d for 20 genes comparing tumor vs. normal. Rank them by effect size and create a forest plot. Which genes have the largest biological impact (not just the smallest p-value)?

```
let n = 30
```

```
# Generate 20 genes with varying true effects
# Some have large effects, some small, some zero
# 1. Compute Cohen's d inline for each gene
# 2. Compute p-value via ttest() for each gene
# 3. Create a forest_plot() sorted by effect size
# 4. Find a gene that is significant (p < 0.05) but has d < 0.2
# 5. Find a gene that is non-significant but has d > 0.5
```

Exercise 2: OR vs. RR

Create three 2x2 tables where the outcome prevalence is 5%, 30%, and 60%. Set the true relative risk to 2.0 in all three. Show that $OR \approx RR$ when prevalence is low but $OR \gg RR$ when prevalence is high.

```
# Table 1: rare outcome (5% baseline)
# Table 2: moderate outcome (30% baseline)
# Table 3: common outcome (60% baseline)

# For each: compute OR = (a*d)/(b*c) and RR = (a/(a+b))/(c/(c+d))
# Show the divergence as prevalence increases
# Which metric should you report to patients?
```

Exercise 3: Complete Reporting

Given the following analysis, write a complete results paragraph with effect size, CI, p-value, and sample size. Then write it again omitting the effect size — notice how much information is lost.

```
set_seed(42)
let treatment = rnorm(45, 15, 4)
let control = rnorm(45, 12, 4)

# Compute: ttest(), Cohen's d inline, mean difference
# Write: "Treatment group showed significantly higher X
# (mean = __, SD = __) compared to control (mean = __, SD = __),
# with a [small/medium/large] effect (d = __),
# p = __."
```

Exercise 4: The Winner's Curse Revisited

Run 100 simulations of an underpowered study ($n=10$, true $d=0.3$). For the significant results only, compute the observed d . Show that “published” effect sizes are inflated.

```
set_seed(42)
let true_d = 0.3
let n = 10
let n_sims = 100

# 1. Simulate 100 ttest() calls
# 2. Record which are significant (p < 0.05)
# 3. For significant studies, compute observed Cohen's d inline
# 4. Compare mean "published d" to true d = 0.3
# 5. Plot histogram of published d values
```

Exercise 5: Forest Plot for a Meta-Analysis

Combine results from 8 “studies” of the same drug effect (true $d = 0.5$) with varying sample sizes ($n = 10$ to 200). Show that larger studies give more precise estimates (narrower CIs) and cluster closer to the true effect.

```
set_seed(42)
let study_sizes = [10, 15, 25, 40, 60, 100, 150, 200]

# 1. Simulate each study
# 2. Compute Cohen's d and 95% CI inline for each
# 3. Create a forest_plot()
# 4. Which studies have CIs that include the true d = 0.5?
# 5. Do larger studies provide better estimates?
```

Key Takeaways

- **P-values conflate effect size with sample size** — a tiny effect can be “significant” with enough data, and a large effect can be “non-significant” with too few data
- **Always report effect sizes** with confidence intervals alongside p-values: this is the modern reporting standard
- **Cohen’s d** quantifies standardized mean differences: 0.2 small, 0.5 medium, 0.8 large (but context matters)
- **Odds ratio ≠ relative risk** when the outcome is common (>10% prevalence) — OR overstates RR
- **Cramér’s V** measures categorical association strength; **eta-squared** measures variance explained in ANOVA
- **Forest plots** are the standard visualization for effect sizes across multiple comparisons or studies
- The **winner’s curse**: underpowered studies that reach significance overestimate the true effect
- A complete result reports **effect size + confidence interval + p-value + sample size** — omitting any element is incomplete reporting

What’s Next

We’ve learned to quantify and test effects. But what happens when technical artifacts overwhelm biology? Day 20 tackles the critical problem of **batch**

effects and confounders — when your PCA reveals that the dominant signal in your data is the lab that processed the sample, not the biology you're studying.

Day 20: Batch Effects and Confounders

The Problem

Dr. David Liu is leading a multi-center study comparing gene expression in 200 breast tumors versus 100 normal breast tissues. Three hospitals contributed samples: Memorial (80 tumors, 40 normals), Hopkins (70 tumors, 30 normals), and Mayo (50 tumors, 30 normals). He runs PCA on the full expression matrix, expecting to see tumor and normal separate.

Instead, the PCA plot shows **three tight clusters — one per hospital**. The first principal component explains 35% of variance and perfectly separates the centers. Tumor vs. normal? It's buried in PC4 at 4% of variance.

The dominant signal in his data is **which hospital processed the sample**, not the biology he's studying. These are **batch effects**, and they're one of the most insidious problems in genomics.

What Are Batch Effects?

Batch effects are **systematic technical differences** between groups of samples that were processed differently. They have nothing to do with biology but can completely dominate a dataset.

Source	Mechanism	Example
Processing date	Reagent lots, temperature, humidity	Monday vs. Friday RNA extractions
Technician	Handling differences	Technician A vs. B
Center/site	Different protocols, equipment	Hospital A vs. B
Sequencing lane	Flow cell chemistry, loading density	Lane 1 vs. Lane 2
Plate position	Edge effects, evaporation	Well A1 vs. H12
Reagent lot	Batch-to-batch kit variation	Kit lot 2024A vs. 2024B
Storage time	RNA degradation over time	Samples banked in 2020 vs. 2024

Key insight: Batch effects are not random noise — they are systematic biases that affect thousands of genes simultaneously. They shift entire samples in the same direction, which is why PCA detects them so readily.

How Large Are Batch Effects?

In a landmark 2010 study, Leek et al. analyzed publicly available microarray data and found:

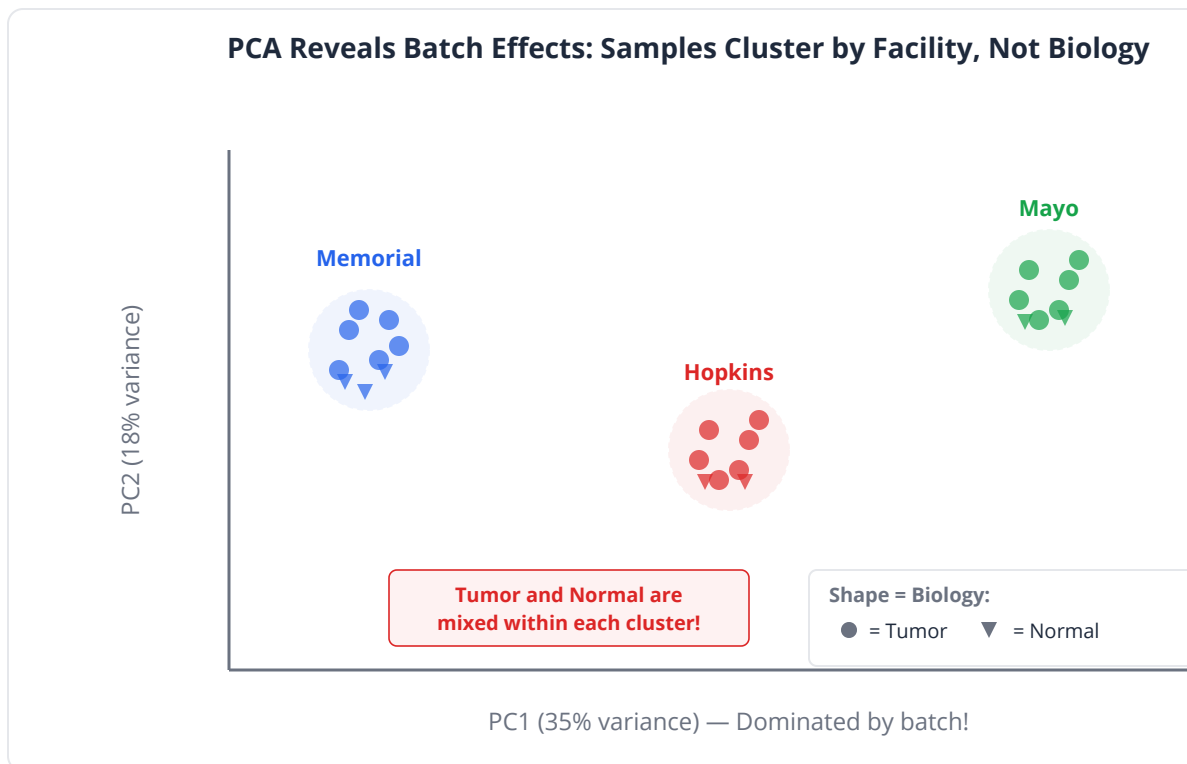
- Batch effects were present in **virtually all** high-throughput datasets
- They often explained **more variance** than the biological signal of interest
- They affected **thousands of genes** per batch, not just a handful

In RNA-seq, batch effects are typically smaller than in microarrays but still substantial — often explaining 10-30% of total variance.

Identifying Batch Effects

1. PCA Visualization

The most powerful diagnostic. Color samples by batch variable and biological variable. If batch dominates PC1/PC2, you have a problem.



2. Correlation Heatmap

Compute sample-to-sample correlation. If samples cluster by batch rather than biology, batch effects are present.

3. Box Plots by Batch

Plot the distribution of expression values (or a summary statistic) stratified by batch. Systematic shifts indicate batch effects.

4. ANOVA for Batch

For each gene, test whether expression differs significantly by batch. If hundreds or thousands of genes show batch effects, the problem is pervasive.

Confounders: A Deeper Problem

A **confounder** is a variable associated with BOTH the predictor and the outcome, creating a spurious association (or masking a real one).

Scenario	Confounder	Danger
Gene expression differs by sex	Tumor subtype differs by sex	Sex drives both
Drug response varies by ethnicity	Genetic variant frequency varies by ethnicity	Population stratification
Survival differs by TP53 status	Stage differs by TP53 status	Stage confounds TP53 effect
Expression differs by treatment	Samples processed on different days	Processing day = treatment

Simpson's Paradox

The most dramatic form of confounding. A trend that appears in aggregate **reverses** when groups are separated:

Hospital	Drug A Survival	Drug B Survival	Better Drug
Hospital 1 (mild cases)	95%	90%	Drug A
Hospital 2 (severe cases)	40%	30%	Drug A
Combined	55%	70%	Drug B??

Drug A is better at BOTH hospitals, but Drug B appears better overall because Hospital 2 (severe cases, low survival) preferentially used Drug A.

Clinical relevance: Simpson's paradox has real consequences. In the 1970s, Berkeley was accused of sex discrimination in graduate admissions. Overall, women had lower acceptance rates. But department by department, women were accepted at equal or higher rates — they had applied more often to competitive departments with low overall acceptance rates.

The Confounded Design Trap

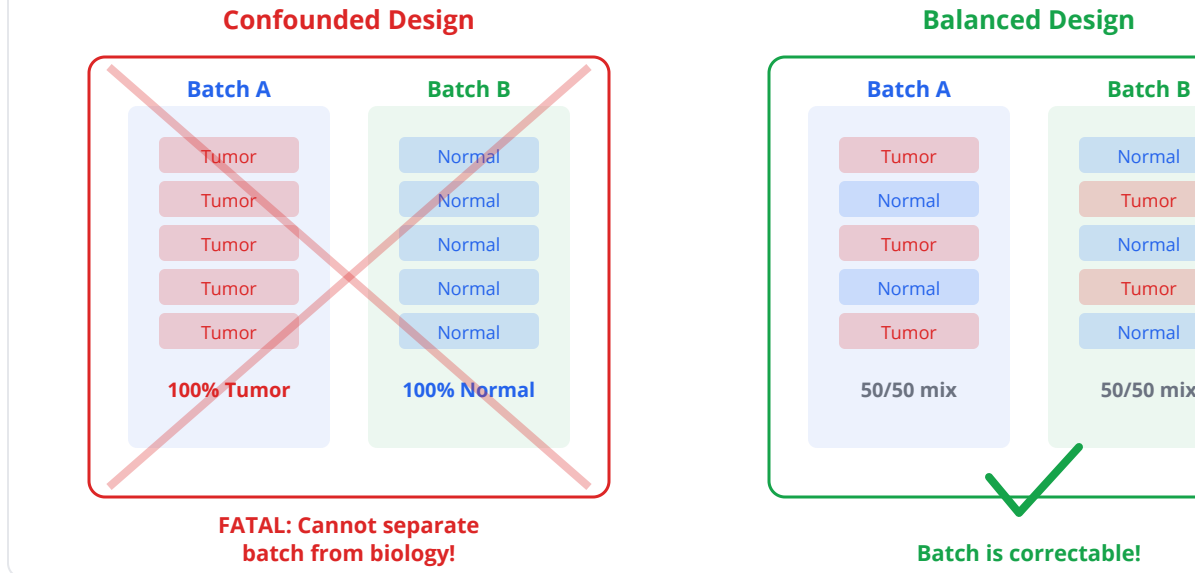
The most dangerous scenario: **when batch perfectly correlates with biology.**

Sample	Center	Condition
1-50	Center A	All Tumor
51-100	Center B	All Normal

Here, center and condition are **completely confounded**. Every difference between tumor and normal could equally be a difference between Center A and Center B. No statistical method can separate them. The experiment is fatally flawed.

Common pitfall: This happens more often than you'd think. A collaborator sends tumor samples from one hospital and normal samples from another. Or samples are processed tumor on Monday, normal on Tuesday. The solution is **balanced design** — ensure batch variables are distributed across biological groups.

Confounded vs. Balanced Study Design



Strategies for Handling Batch Effects

1. Prevention (Best Option)

Strategy	How
Balanced design	Distribute conditions across batches equally
Randomization	Random assignment to processing order/position
Blocking	Process one sample from each condition per batch
Standard protocols	Minimize technical variation with SOPs
Single batch	Process everything together (often impractical)

2. Detection

Method	What It Reveals
PCA colored by batch	Visual clustering by technical variable
Correlation heatmap	Sample similarity by batch
Box plots by batch	Distribution shifts
ANOVA per gene	Number of genes affected

3. Correction

Method	Approach	Caveat
Include as covariate	Add batch to regression/DE model	Requires balanced design
ComBat (parametric)	Empirical Bayes batch correction	Assumes batch is known
ComBat-seq	ComBat for raw RNA-seq counts	Preserves count nature
SVA (surrogate variables)	Discovers unknown batch effects	May remove biology
RUVseq	Uses control genes	Needs negative controls

Common pitfall: Do NOT use batch correction methods when batch is perfectly confounded with biology. They will remove both the batch effect AND the biological signal. If all tumors were processed in batch 1 and all normals in batch 2, no correction method can save you — the experiment must be redesigned.

Batch Effects in BioLang

Simulating Batch-Contaminated Data

```
set_seed(42)
# Simulate a multi-center gene expression study
let n_samples = 30
let n_genes = 10

# Assign samples to centers and conditions
let center = range(0, n_samples) |> map(|i|
  if i < 10 { "Memorial" } else if i < 20 { "Hopkins" } else {
"Mayo" }
)
let condition = range(0, n_samples) |> map(|i|
  if i % 2 == 0 { "Tumor" } else { "Normal" }
)

# Generate expression matrix with batch effects
let expr_matrix = range(0, n_samples) |> map(|i| {
  range(0, n_genes) |> map(|g| {
    let base = 10 + rnorm(1, 0, 1)[0]
    let batch = if center[i] == "Memorial" { 2.0 }
      else if center[i] == "Hopkins" { -1.5 } else { 0.5 }
    let bio = if condition[i] == "Tumor" && g < 3 { 1.5 } else {
0.0 }
    base + batch + bio
  })
})

print("Expression matrix: {n_samples} samples x {n_genes} genes")
print("Centers: Memorial=10, Hopkins=10, Mayo=10")
```

Detecting Batch Effects with PCA

```
# PCA on the expression matrix
let pca_result = pca(expr_matrix, 5)

print("=== PCA Variance Explained ===")
for i in 0..5 {
  print("PC{i+1}: {(pca_result.variance_explained[i] * 100) |>
round(1)}%")
}

# Plot PC1 vs PC2, colored by center
let pca_data = table({
  "PC1": pca_result.scores |> map(|s| s[0]),
  "PC2": pca_result.scores |> map(|s| s[1]),
  "center": center,
  "condition": condition
})

pca_plot(pca_data)

# If center dominates PC1 and condition is only visible on PC3+,
# batch effects are overwhelming the biological signal
```

Quantifying Batch Effect with ANOVA

```
# For each gene, test how much variance is explained by batch vs
biology
let batch_significant = 0
let bio_significant = 0

print("=== ANOVA: Batch vs Biology ===")

for g in 0..10 {
  let gene_expr = expr_matrix |> map(|row| row[g])

  # Group by center
  let memorial_expr = []
  let hopkins_expr = []
  let mayo_expr = []
  for i in 0..n_samples {
    if center[i] == "Memorial" { memorial_expr = memorial_expr +
[gene_expr[i]] }
    else if center[i] == "Hopkins" { hopkins_expr = hopkins_expr
+ [gene_expr[i]] }
    else { mayo_expr = mayo_expr + [gene_expr[i]] }
  }
  let batch_test = anova([memorial_expr, hopkins_expr, mayo_expr])

  # Group by condition
  let tumor_expr = []
  let normal_expr = []
  for i in 0..n_samples {
    if condition[i] == "Tumor" { tumor_expr = tumor_expr +
[gene_expr[i]] }
    else { normal_expr = normal_expr + [gene_expr[i]] }
  }
  let bio_test = ttest(tumor_expr, normal_expr)

  if batch_test.p_value < 0.05 { batch_significant =
batch_significant + 1 }
  if bio_test.p_value < 0.05 { bio_significant = bio_significant +
1 }

  print("Gene {g+1}: batch F={batch_test.f_statistic |> round(2)}
p={batch_test.p_value |> round(4)} bio p={bio_test.p_value |>
round(4)}")
}

print("\nGenes with significant batch effect: {batch_significant} /
```

```
10")
print("Genes with significant biological effect: {bio_significant} /
10")
```

Correlation Heatmap for Batch Detection

```
# Sample-to-sample correlation: compute a few representative pairs
# Full matrix would be 300x300; spot-check a few
let s1 = expr_matrix[0]    # Memorial, Tumor
let s2 = expr_matrix[5]    # Memorial, Normal
let s3 = expr_matrix[10]   # Hopkins, Tumor

print("=== Sample Correlations ===")
print("Memorial Tumor vs Memorial Normal: {cor(s1, s2) |>
round(3)}")
print("Memorial Tumor vs Hopkins Tumor:   {cor(s1, s3) |>
round(3)}")
print("If same-center pairs are more correlated than same-condition
pairs,")
print("batch effects dominate")
```

Box Plots by Batch

```
# Distribution of a batch-affected gene across centers
let gene_1_expr = expr_matrix |> map(|row| row[0])

let memorial_g1 = []
let hopkins_g1 = []
let mayo_g1 = []
for i in 0..n_samples {
  if center[i] == "Memorial" { memorial_g1 = memorial_g1 +
[gene_1_expr[i]] }
  else if center[i] == "Hopkins" { hopkins_g1 = hopkins_g1 +
[gene_1_expr[i]] }
  else { mayo_g1 = mayo_g1 + [gene_1_expr[i]] }
}

let bp_table = table({"Memorial": memorial_g1, "Hopkins":
hopkins_g1, "Mayo": mayo_g1})
boxplot(bp_table, {title: "Gene 1 Expression by Center"})

# Systematic shifts between centers = batch effect
```

Correcting Batch Effects: Include as Covariate

```
# The simplest and most transparent correction:
# include batch as a covariate in your statistical model

# For differential expression: multiple regression
for g in 0..5 {
  let gene_expr = expr_matrix |> map(|row| row[g])

  # Encode condition: Tumor=1, Normal=0
  let cond_numeric = condition |> map(|c| if c == "Tumor" { 1.0 }
else { 0.0 })

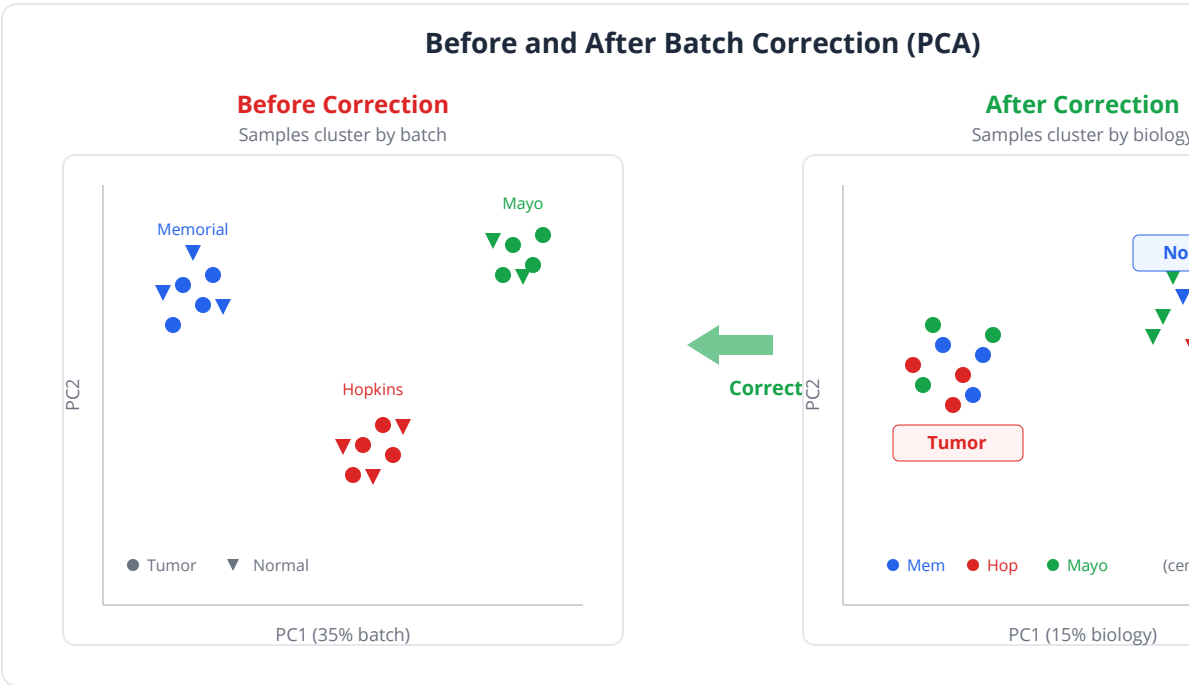
  # Encode center: dummy variables
  let is_hopkins = center |> map(|c| if c == "Hopkins" { 1.0 }
else { 0.0 })
  let is_mayo = center |> map(|c| if c == "Mayo" { 1.0 } else {
0.0 })

  # Model WITHOUT batch correction
  let model_naive = lm(cond_numeric, gene_expr)

  # Model WITH batch correction (include center as covariate)
  let adj_data = table({
    "expr": gene_expr, "cond": cond_numeric,
    "hopkins": is_hopkins, "mayo": is_mayo
  })
  let model_adjusted = lm(~expr ~ cond + hopkins + mayo, adj_data)

  print("Gene {g+1}:")
  print(" Naive:    slope = {model_naive.slope |> round(3)}, p =
{model_naive.p_value |> round(4)}")
  print(" Adjusted: cond coef = {model_adjusted.coefficients[0]
|> round(3)}")
}
```

Before/After Comparison



```
# Visualize the effect of batch correction with PCA

# Step 1: PCA on raw data (batch-contaminated)
let pca_before = pca(expr_matrix, 3)
print("=== Before Correction ===")
print("PC1 variance: {(pca_before.variance_explained[0] * 100) |>
round(1)}% (likely batch)")

# Step 2: Regress out batch effect from each gene
let is_hopkins = center |> map(|c| if c == "Hopkins" { 1.0 } else {
0.0 })
let is_mayo = center |> map(|c| if c == "Mayo" { 1.0 } else { 0.0 })

let corrected_matrix = []
for i in 0..n_samples {
  let corrected_sample = []
  for g in 0..n_genes {
    let gene_expr = expr_matrix |> map(|row| row[g])
    let batch_data = table({"expr": gene_expr, "hopkins":
is_hopkins, "mayo": is_mayo})
    let model = lm(~expr ~ hopkins + mayo, batch_data)

    let residual = gene_expr[i] - model.coefficients[0] *
is_hopkins[i]
      - model.coefficients[1] * is_mayo[i]
    corrected_sample = corrected_sample + [residual]
  }
  corrected_matrix = corrected_matrix + [corrected_sample]
}

# Step 3: PCA on corrected data
let pca_after = pca(corrected_matrix, 3)
print("\n=== After Correction ===")
print("PC1 variance: {(pca_after.variance_explained[0] * 100) |>
round(1)}% (should be biology now)")

# Compare side by side
let pca_corrected = table({
  "PC1": pca_after.scores |> map(|s| s[0]),
  "PC2": pca_after.scores |> map(|s| s[1]),
  "condition": condition
})
pca_plot(pca_corrected)
```

Detecting the Confounded Design Trap

```
# Check whether batch and condition are confounded
print("=== Balance Check ===")
print("Center      Tumor      Normal      % Tumor")

let centers = ["Memorial", "Hopkins", "Mayo"]
for c in centers {
  let n_tumor = 0
  let n_normal = 0
  for i in 0..n_samples {
    if center[i] == c {
      if condition[i] == "Tumor" { n_tumor = n_tumor + 1 }
      else { n_normal = n_normal + 1 }
    }
  }
  let pct = (n_tumor / (n_tumor + n_normal) * 100) |> round(1)
  print("{c}          {n_tumor}          {n_normal}          {pct}%")
}

# If one center is 100% tumor and another is 100% normal,
# the design is fatally confounded – no statistical fix exists
print("\nDesign assessment:")
let balanced = true
for c in centers {
  let has_tumor = false
  let has_normal = false
  for i in 0..n_samples {
    if center[i] == c {
      if condition[i] == "Tumor" { has_tumor = true }
      else { has_normal = true }
    }
  }
  if !has_tumor || !has_normal {
    print("FATAL: {c} has only one condition!")
    balanced = false
  }
}
if balanced {
  print("Design is balanced – batch correction is possible")
}
```

Python:

```
import numpy as np
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# PCA for batch detection
pca = PCA(n_components=5)
scores = pca.fit_transform(expr_matrix)
plt.scatter(scores[:, 0], scores[:, 1], c=batch_labels, cmap='Set1')
plt.title('PCA Colored by Batch')

# ComBat batch correction
from combat.pycombat import pycombat
corrected = pycombat(expr_df, batch_series)

# SVA for unknown batches
# Use the sva package via rpy2 or pydeseq2

# Include batch as covariate in DE analysis
import statsmodels.api as sm
X = sm.add_constant(pd.get_dummies(df[['condition', 'center']],
drop_first=True))
model = sm.OLS(gene_expr, X).fit()
```

R:

```
# PCA for batch detection
pca <- prcomp(t(expr_matrix), scale. = TRUE)
plot(pca$x[,1], pca$x[,2], col = batch, pch = 19)

# ComBat batch correction
library(sva)
corrected <- ComBat(dat = expr_matrix, batch = center)

# ComBat-seq for RNA-seq counts
corrected <- ComBat_seq(counts = count_matrix, batch = center)

# SVA for unknown batches
mod <- model.matrix(~ condition)
mod0 <- model.matrix(~ 1, data = pdata)
svobj <- sva(expr_matrix, mod, mod0)

# Include in DE model (limma)
design <- model.matrix(~ condition + center)
fit <- lmFit(expr_matrix, design)
fit <- eBayes(fit)
topTable(fit, coef = "conditionTumor")
```

Exercises

Exercise 1: Detect Batch Effects

Given a simulated expression matrix with hidden batch effects, use PCA and ANOVA to identify which technical variable is causing the problem.

```
let n = 120
let n_genes = 30

# Simulate with batch effect on processing_date
# Biology: treatment vs control
# Batch: 3 processing dates

# 1. Run pca() and color by treatment, then by processing date
# 2. Which variable dominates PC1?
# 3. How many genes show significant batch effects (anova)?
# 4. How many show significant treatment effects?
```

Exercise 2: Balanced vs. Confounded Design

Create two study designs: one where batch and condition are balanced, another where they are completely confounded. Show that the confounded design cannot be corrected.

```
# Design A (balanced): equal tumor/normal at each center
# Design B (confounded): all tumor at Center 1, all normal at Center
2

# 1. Simulate data for both designs with identical biological
effects
# 2. Apply batch correction (include center as covariate in lm())
# 3. Compare: does Design A recover the true biological signal?
# 4. Does Design B? Why or why not?
```

Exercise 3: Before/After Correction

Apply batch correction to a multi-center dataset and create a before/after PCA comparison. Quantify how much the batch effect is reduced.

```
# 1. Simulate 200 samples from 4 centers
# 2. pca() before correction – measure batch variance (anova on PC1
scores)
# 3. Correct by regressing out center effects with lm()
# 4. pca() after correction – measure batch variance again
# 5. What percentage of the batch effect was removed?
```

Exercise 4: Simpson's Paradox

Simulate a drug trial where Drug A is better within every hospital, but Drug B appears better overall due to confounding. Demonstrate the paradox numerically.

```
# Hospital 1: treats mild cases (80% survival baseline)
#   Drug A: 85% survival, Drug B: 75% survival
# Hospital 2: treats severe cases (30% survival baseline)
#   Drug A: 35% survival, Drug B: 25% survival
# Hospital 2 uses Drug A more often

# 1. Show Drug A wins within each hospital
# 2. Combine data – show Drug B appears better overall
# 3. Explain why (confounding by disease severity)
# 4. What analysis would give the correct answer?
```

Exercise 5: Designing a Batch-Robust Study

You have 60 tumor and 60 normal samples that must be processed across 3 days (40 per day). Design the processing schedule to minimize confounding, and simulate data to verify your design resists batch effects.

```
# Good design: 20 tumor + 20 normal per day
# Bad design: 40 tumor on day 1, 40 tumor on day 2, 60 normal on day 3

# 1. Create the good balanced assignment
# 2. Create the bad confounded assignment
# 3. Simulate data with identical batch and biological effects
# 4. Analyze both designs – which recovers the true biology?
```

Key Takeaways

- **Batch effects** are systematic technical differences that can dominate biological signals — they are present in virtually all high-throughput datasets
- **PCA** is the most powerful tool for detecting batch effects: color by batch and biological variables to see which dominates
- **Confounders** create spurious associations (or mask real ones); **Simpson's paradox** is the extreme case where aggregate trends reverse within subgroups

- **Prevention** is the best strategy: use balanced, randomized designs that distribute biological conditions across batches
- **Confounded designs** (batch = biology) cannot be rescued by any statistical method — the experiment must be redesigned
- **Correction approaches:** include batch as a covariate (simplest), ComBat (empirical Bayes), SVA (discover unknown batches), or RUVseq (negative controls)
- Always perform a **balance check** before analysis: ensure no batch variable is perfectly correlated with the biological variable of interest
- **Before/after PCA** is the standard way to demonstrate that batch correction worked

What's Next

With three full weeks of biostatistics under your belt, you're ready for the advanced topics of Week 4. Day 21 introduces **dimensionality reduction** — PCA, t-SNE, and UMAP — the tools that turn 20,000-dimensional gene expression data into interpretable 2D visualizations.

Day 21: Dimensionality Reduction — PCA and Friends

Day 21 of 30 Prerequisites: Days 2-3, 13, 20 ~60 min reading
Unsupervised Learning

The Problem

A single-cell RNA sequencing experiment has just finished. Your collaborator drops a matrix on your desk: expression levels for 20,000 genes measured across 5,000 individual cells. She wants to know whether the cells form distinct populations — immune subtypes, perhaps, or tumor cells versus stroma.

You stare at the matrix. Twenty thousand dimensions. You cannot visualize it. You cannot eyeball it. You cannot plot 20,000 axes on a screen. If you pick two genes at random and make a scatter plot, you might miss the structure entirely — those two genes might be irrelevant. Pick different genes and you get a completely different picture.

What you need is a camera angle. A way to look at 20,000-dimensional data from the direction that reveals the most structure. A method that compresses the information into a handful of dimensions you can actually see and reason about — without throwing away the patterns that matter.

That method is Principal Component Analysis. It is the single most widely used technique in genomics for exploring high-dimensional data, and by the end of today, you will understand exactly how it works, when it fails, and how to use it effectively.

What Is Dimensionality Reduction?

Dimensionality reduction takes data with many variables (dimensions) and represents it using fewer variables, while preserving as much of the important structure as possible.

Think of it this way. You are standing on a hilltop overlooking a city. The city exists in three dimensions, but you are looking at it from one particular angle. Your view — a photograph — is a two-dimensional representation of a three-dimensional scene. A good photograph, taken from the right angle, captures the layout of the streets, the relative positions of buildings, and the overall structure. A bad photograph, taken facing a blank wall, tells you nothing.

Dimensionality reduction is the art of finding the best camera angle for your data. In a 20,000-dimensional gene expression dataset, the “best angle” is the one that shows you the most variation — the direction along which cells differ the most.

Key insight: Dimensionality reduction does not create information. It finds the most informative low-dimensional summary of high-dimensional data. If the real structure is inherently high-dimensional, no reduction will capture it perfectly.

The Curse of Dimensionality

Before we solve the problem, let us understand why high dimensions are problematic.

In one dimension, 10 evenly spaced points cover a line segment well. In two dimensions, you need 100 points (10×10) to cover a square with the same density. In three dimensions, 1,000 points ($10 \times 10 \times 10$). In 20,000 dimensions, you would need $10^{20,000}$ points — a number so large it dwarfs the number of atoms in the observable universe.

This means that in high-dimensional space, data is always sparse. Your 5,000 cells are scattered across 20,000-dimensional space like five thousand grains of sand in the Sahara. Most of the space is empty. Distances between points become unreliable — in very high dimensions, the nearest neighbor and the farthest neighbor are almost the same distance away.

Dimensions	Points needed for even coverage	Nearest-neighbor reliability
2	100	Excellent
10	10 billion	Good
100	10^{100}	Poor
20,000	$10^{20,000}$	Meaningless

This is the curse of dimensionality. It makes direct analysis of raw high-dimensional data unreliable. Fortunately, biological data has a saving grace: most of the 20,000 gene dimensions are redundant. Genes in the same pathway are correlated. Housekeeping genes barely vary. The “true” dimensionality of gene expression data is typically much lower than the number of genes measured — perhaps tens to hundreds of effective dimensions. PCA exploits this redundancy.

PCA Mechanics: Finding the Best Camera Angle

PCA proceeds in a simple sequence of steps.

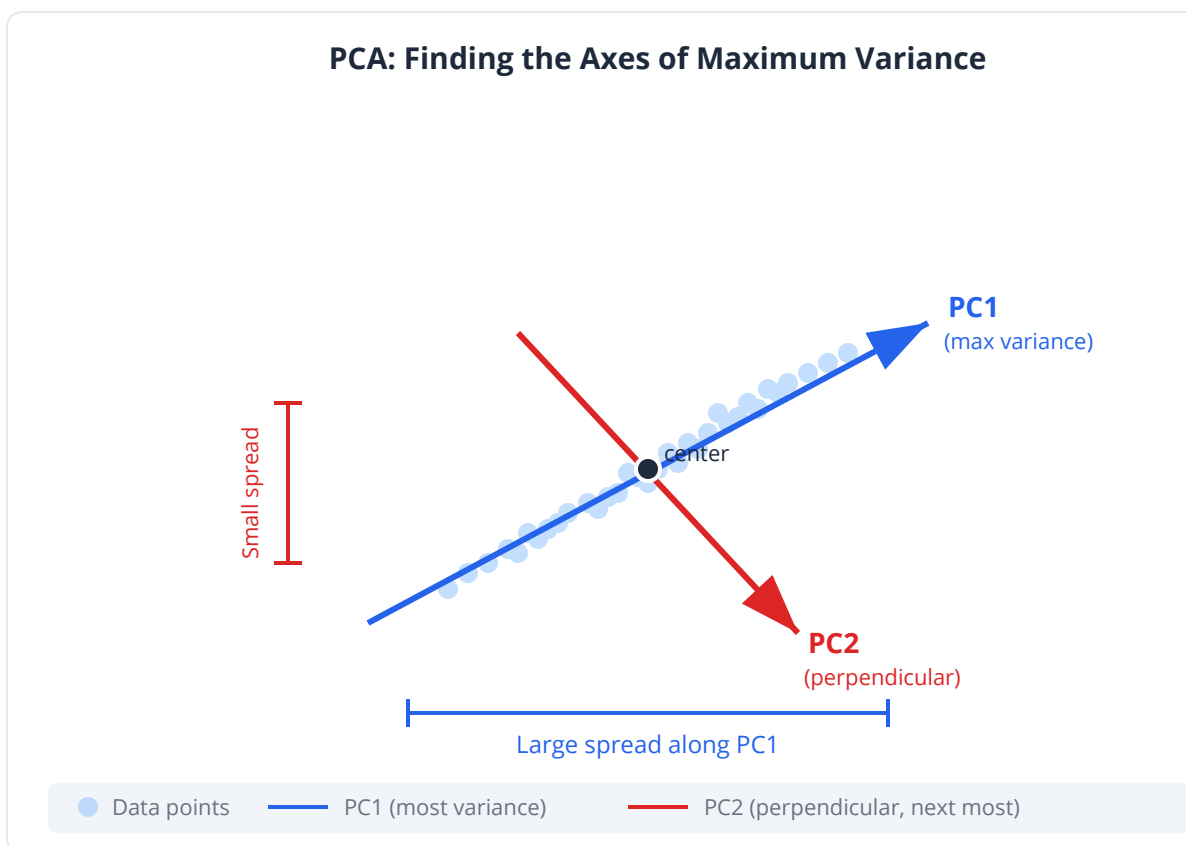
Step 1: Center the Data

Subtract the mean of each gene across all cells. This ensures we are looking at variation, not absolute levels. A gene expressed at 10,000 in every cell has zero variation and should contribute nothing.

Step 2: Find the Direction of Maximum Variance

Imagine all 5,000 cells as points in 20,000-dimensional space. PCA finds the single direction (a line through the origin) along which the spread of points is greatest. This is Principal Component 1 (PC1). It is the camera angle that shows you the most variation.

Mathematically, PC1 is the eigenvector of the covariance matrix with the largest eigenvalue. But you do not need to understand eigenvectors to use PCA — think of it as the axis along which the data is most stretched.



Step 3: Find the Next Perpendicular Direction

PC2 is the direction of maximum remaining variance, with the constraint that it must be perpendicular (orthogonal) to PC1. This ensures PC2 captures new information, not a rehash of PC1.

Step 4: Repeat

PC3 is perpendicular to both PC1 and PC2, and captures the next most variance. Continue for as many components as you want (up to the number of samples or genes, whichever is smaller).

Each successive PC captures less variance. The first few PCs often capture the majority of the total variation, and the rest is noise.

Component	Captures	Constraint
PC1	Maximum variance in the data	None (first direction)
PC2	Maximum remaining variance	Perpendicular to PC1
PC3	Maximum remaining variance	Perpendicular to PC1 and PC2
...	Decreasing variance	Perpendicular to all previous

The Camera Analogy

If your data were a 3D object:

- PC1 is like looking at the object from the angle where its shadow is largest
- PC2 is the perpendicular angle that shows the next most detail
- PC3 fills in the remaining depth

For gene expression: PC1 might separate tumor from normal. PC2 might separate tissue subtypes. PC3 might reflect a batch effect. Each component tells a different biological or technical story.

Eigenvalues and Variance Explained

Each principal component has an associated eigenvalue. The eigenvalue tells you how much variance that component captures. Dividing each eigenvalue by the total gives the proportion of variance explained.

Component	Eigenvalue	Variance Explained	Cumulative
PC1	85.3	28.4%	28.4%
PC2	42.1	14.0%	42.4%
PC3	18.7	6.2%	48.7%
PC4	12.4	4.1%	52.8%
...	...	...	...
PC20	2.1	0.7%	75.2%

In a typical scRNA-seq dataset, the first 20-50 PCs might capture 70-80% of the total variance. The remaining thousands of PCs together account for the other 20-30% — mostly noise.

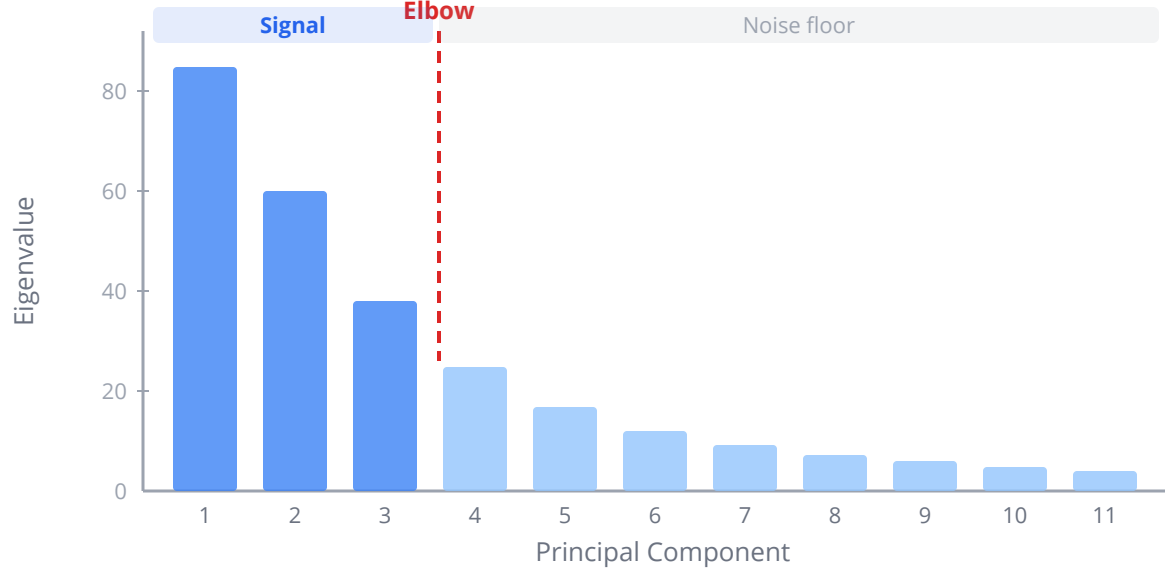
Key insight: If the first 2 PCs capture 60%+ of the variance, a 2D PCA plot is a good representation. If they capture only 15%, the data has complex structure that two dimensions cannot convey.

The Scree Plot: Finding the Elbow

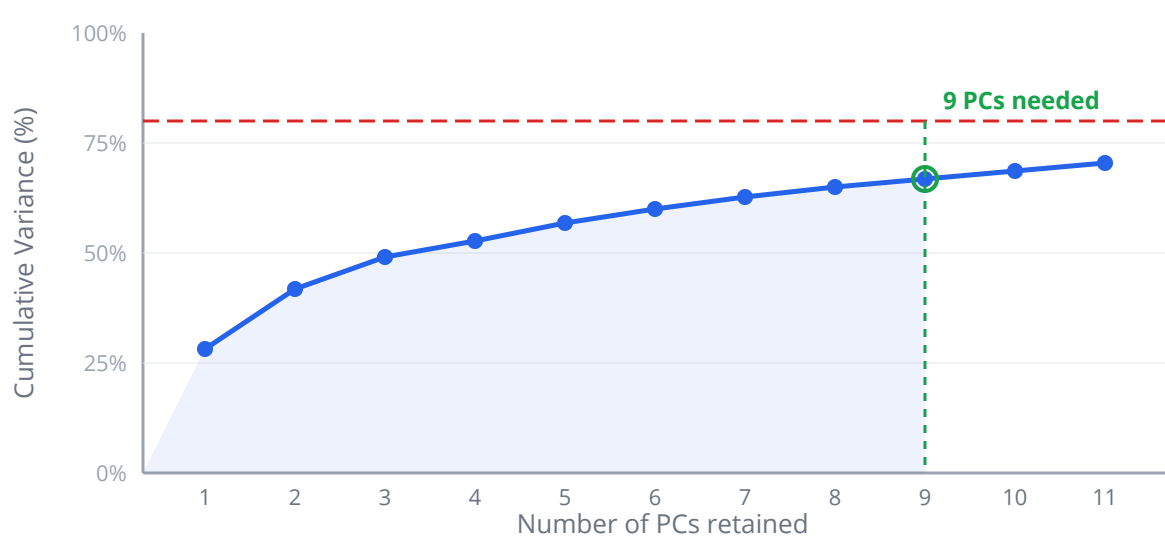
A scree plot displays the eigenvalue (or variance explained) for each successive PC. It is named after the geological term for rubble at the base of a cliff — because the plot typically shows a steep drop followed by a long, flat tail.

The “elbow” — the point where the curve transitions from steep to flat — suggests how many PCs contain real signal. Components before the elbow capture structured variation; components after the elbow capture noise.

Scree Plot: Eigenvalues by Principal Component



Cumulative Variance Explained



```
set_seed(42)
# Generate scree plot for gene expression data
let pca_result = pca(expression_matrix)
let eigenvalues = pca_result.variance_explained
  |> map_index(|i, v| {pc: i + 1, variance: v})
  |> to_table()
plot(eigenvalues, {type: "line", x: "pc", y: "variance",
  title: "Scree Plot – Gene Expression PCA"})
```

Rules of thumb for the elbow:

- If there is a clear elbow at PC 3, use 3 components
- If the decline is gradual, no clean cutoff exists — try cumulative variance (e.g., keep PCs until 80% variance)
- In scRNA-seq, 20-50 PCs are commonly retained for downstream clustering

PCA Biplots: Samples and Loadings Together

A PCA biplot shows two things simultaneously:

1. **Scores** (points): Each sample projected onto PC1 and PC2. Samples that cluster together are similar.
2. **Loadings** (arrows): Each variable's contribution to PC1 and PC2. Long arrows indicate influential variables. The direction shows which PC they load on.

In genomics, the scores are your cells (or samples) and the loadings are your genes. Genes with long arrows pointing toward a cluster of cells are the genes that define that cluster's identity.

```
# PCA biplot with top gene loadings
let expression_matrix = matrix([
  [8.2, 7.9, 3.1, 2.8],
  [8.6, 8.1, 3.4, 3.0],
  [3.0, 3.3, 8.4, 8.0],
  [2.7, 3.0, 8.8, 8.2]
])
let pca_result = pca(expression_matrix)
let result = expression_matrix
pca_plot(result, {title: "PCA Biplot – Top 10 Gene Loadings"})
```

Interpreting Loadings

Loading Direction	Interpretation
Strong positive on PC1	Gene is highly expressed in cells on the right side of the plot
Strong negative on PC1	Gene is highly expressed in cells on the left side
Strong on PC2, weak on PC1	Gene distinguishes top vs bottom but not left vs right
Near the origin	Gene contributes little — low variance or uncorrelated with PCs

Common pitfall: PCA loadings tell you which genes drive each component, but the sign is arbitrary. PC1 could point in either direction — what matters is the relative positioning, not the absolute sign.

Which Genes Drive PC1?

After running PCA, a common next step is to extract the genes with the largest (absolute) loadings on PC1 and PC2. These are the genes responsible for the dominant patterns of variation.

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# Run PCA on 500-gene by 2000-cell expression matrix
let col_names = seq(1, 500) |> map(|i| "Gene_" + str(i))
let expr = table(2000, col_names, "rnorm")

# Inject structure: first 1000 cells get higher expression of genes
# 1-50
for i in 0..1000 {
  for j in 0..50 {
    expr[i][j] = expr[i][j] + 3.0
  }
}

let result = pca(expr)

# Top 10 genes driving PC1
let pc1_loadings = result.loadings[0]
  |> sort_by(|x| -abs(x.value))
  |> take(10)

print("Top genes on PC1:")
print(pc1_loadings)
```

When PCA Fails

PCA assumes that the most important structure lies along directions of maximum variance. This fails in several situations:

Non-linear Structure

If cells lie along a curved trajectory (like a differentiation path), PCA will smear the curve into a blob. The maximum-variance direction might cut across the curve rather than follow it. Methods like t-SNE and UMAP handle non-linear structure better, but they do not preserve distances — only local neighborhoods.

Outliers Dominate

A single extreme outlier can hijack PC1, making it the “outlier direction” rather than the biologically interesting direction. Always check for outliers before interpreting PCA.

Batch Effects Are Stronger Than Biology

If batch effects (Day 20) contribute more variance than biological signal, PC1 will separate batches, not biology. This is actually useful — PCA is one of the best tools for detecting batch effects. But you must correct them before interpreting the biology.

The Variance Is Uninteresting

PCA finds the direction of maximum variance, not maximum biological interest. If the biggest source of variation is sequencing depth (a technical factor), PC1 will capture that, and you will need to look at PC2 or PC3 for biology.

Clinical relevance: In clinical genomics, PCA on genotype data reveals population structure. The first two PCs of human genetic variation separate continental ancestry groups. Failing to account for this in a GWAS leads to spurious associations — a gene might appear associated with disease simply because both the gene variant and the disease are more common in one ancestry group.

PCA in BioLang

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# Complete PCA analysis pipeline

# 1. Load expression matrix (500 genes x 2000 cells)
let n_cells = 2000
let n_genes = 500

# Simulate two cell populations
let group_a = table(1000, n_genes, "rnorm")
let group_b = table(1000, n_genes, "rnorm")

# Group B has elevated expression in genes 1-80
for i in 0..1000 {
  for j in 0..80 {
    group_b[i][j] = group_b[i][j] + 2.5
  }
}

let expr = rbind(group_a, group_b)
let labels = repeat("A", 1000) + repeat("B", 1000)

# 2. Run PCA
let result = pca(expr)

# 3. Scree plot – find the elbow
let eigenvalues = result.variance_explained
|> take(20)
|> map_index(|i, v| {pc: i + 1, variance: v})
|> to_table()
plot(eigenvalues, {type: "line", x: "pc", y: "variance",
  title: "Scree Plot"})

# 4. Variance explained
print("PC1 variance explained: " +
  str(result.variance_explained[0]))
print("PC2 variance explained: " +
  str(result.variance_explained[1]))
print("Cumulative (PC1+PC2): " + str(
  result.variance_explained[0] + result.variance_explained[1]
))

# 5. PCA scatter plot colored by group
scatter(result.scores[0], result.scores[1])
```

```

# 6. PCA plot with gene loadings
pca_plot(result, {title: "PCA Biplot – Top 15 Genes"})

# 7. Extract top genes per PC
let top_pc1 = result.loadings[0]
  |> sort_by(|x| -abs(x.value))
  |> take(10)

let top_pc2 = result.loadings[1]
  |> sort_by(|x| -abs(x.value))
  |> take(10)

print("Top 10 genes on PC1:")
for gene in top_pc1 {
  print(" " + gene.name + ": " + str(round(gene.value, 4)))
}

# 8. Transform new data into PC space
let new_cells = table(50, n_genes, "rnorm")
let projected = pca_transform(result, new_cells)
print("Projected new cells: " + str(nrow(projected)) + " x " +
      str(ncol(projected)))

```

Python:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

scaler = StandardScaler()
X_scaled = scaler.fit_transform(expr)
pca = PCA(n_components=20)
scores = pca.fit_transform(X_scaled)

# Scree plot
plt.plot(range(1, 21), pca.explained_variance_ratio_, 'bo-')
plt.xlabel('Component')
plt.ylabel('Variance Explained')
plt.show()

# Scatter
plt.scatter(scores[:, 0], scores[:, 1], c=labels, cmap='Set1',
            alpha=0.5)
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)')
plt.show()

# Top loadings
loadings = pca.components_[0]
top_idx = np.argsort(np.abs(loadings))[:, :-1][:10]
```

R:

```
pca_result <- prcomp(expr, scale. = TRUE)
summary(pca_result)

# Scree plot
screeplot(pca_result, npcs = 20, type = "lines")

# Scatter
plot(pca_result$x[,1], pca_result$x[,2], col = labels, pch = 19,
     xlab = paste0("PC1 (",
round(summary(pca_result)$importance[2,1]*100, 1), "%)"),
     ylab = paste0("PC2 (",
round(summary(pca_result)$importance[2,2]*100, 1), "%)"))

# Biplot
biplot(pca_result)

# Top loadings on PC1
sort(abs(pca_result$rotation[,1]), decreasing = TRUE)[1:10]
```

Exercises

1. **Scree plot interpretation.** Generate a 100-gene x 500-sample matrix with three hidden groups (shift different gene blocks for each group). Run PCA and create a scree plot. How many PCs have eigenvalues clearly above the noise floor? Does this match the number of groups you created?

```
# Create three groups with different gene signatures
# Your code here: build the matrix, run pca(), plot variance explained
```

2. **Loading detective.** Run PCA on the three-group data from Exercise 1. Extract the top 10 genes on PC1 and PC2. Do the gene indices match the blocks you shifted? Create a biplot to visualize.

```
# Your code here: extract loadings, identify driving genes
# pca_plot(result, {title: "Biplot"})
```

3. **Outlier hijacking.** Take the matrix from Exercise 1 and add a single extreme outlier cell (all genes set to 100). Re-run PCA. What happens to PC1? Remove the outlier and compare.

```
# Your code here: add outlier, run pca(), compare scree plots
```

4. **Batch effect detection.** Create a matrix with two biological groups AND a batch effect (add 2.0 to all genes in half the samples, crossing the biological groups). Run PCA. Which PC captures the batch effect? Which captures biology?

```
# Your code here: simulate batch + biology, examine PC1 vs PC2
```

5. **Variance threshold.** How many PCs do you need to capture 80% of the variance in the three-group dataset? Write code to find this number automatically.

```
# Your code here: cumulative variance explained, find threshold
```

Key Takeaways

- PCA finds the directions of maximum variance in high-dimensional data, producing a low-dimensional summary that preserves the most important structure.
- Each principal component captures progressively less variance and is perpendicular to all previous components.
- The scree plot shows how much variance each PC captures; the “elbow” suggests how many PCs contain real signal.
- PCA biplots display both samples (as points) and variable loadings (as arrows), revealing which genes drive the observed patterns.
- PCA assumes linear structure — it fails on curved trajectories, is sensitive to outliers, and will capture the largest source of variance whether it is biological or technical.

- In genomics, PCA is essential for quality control (detecting batch effects and outliers), population structure analysis (GWAS), and dimensionality reduction before clustering (scRNA-seq).
- Always examine what PC1 actually represents before interpreting it as biology — it might be a technical artifact.

What's Next

Tomorrow we take the reduced data from PCA and ask: are there natural groups? Clustering methods — k-means, hierarchical, and DBSCAN — will find structure in your omics data, identify tumor subtypes, and reveal cellular populations. But beware: clustering always finds clusters, even in pure noise. You will learn how to tell real structure from statistical ghosts.

Day 22: Clustering — Finding Structure in Omics Data

Day 22 of 30 Prerequisites: Days 2-3, 13, 21 ~60 min reading
Unsupervised Learning

The Problem

You are part of a cancer genomics consortium. Five hundred tumor samples have been profiled with RNA-seq, measuring the expression of 18,000 genes in each. The pathologist has classified these tumors into three histological subtypes based on what she sees under the microscope. But molecular data often reveals finer distinctions invisible to the eye.

Your task: find natural groupings in the gene expression data — without peeking at the pathologist's labels. If the molecular subtypes align with the histological ones, confidence in the classification increases. If the data reveals additional subtypes, you may have discovered clinically distinct groups that respond differently to treatment. In breast cancer, this is exactly how the PAM50 molecular subtypes were discovered — and they now guide treatment decisions for millions of patients worldwide.

But there is a danger lurking. Clustering algorithms always find clusters, even in random noise. The critical question is not “can I find groups?” but “are the groups real?”

What Is Clustering?

Clustering is unsupervised grouping: divide observations into sets such that observations within a set are more similar to each other than to observations in

other sets.

Unlike classification (supervised learning), clustering has no labels to learn from. You are not training the algorithm to distinguish “tumor” from “normal.” You are asking the algorithm to discover structure on its own.

Think of it as sorting a pile of coins. If you have pennies, nickels, dimes, and quarters, the task is straightforward — there are obvious groupings by size and color. But if someone hands you a pile of irregularly shaped pebbles and says “sort these into groups,” you must decide what “similar” means. By weight? By color? By texture? Different definitions of similarity lead to different groupings. This subjectivity is both the power and the peril of clustering.

Key insight: Clustering is a tool for exploration, not proof. It generates hypotheses about structure in your data. Validating those hypotheses requires independent evidence — clinical outcomes, functional assays, or replication in new datasets.

K-Means Clustering

K-means is the simplest and most widely used clustering algorithm. It partitions data into exactly k groups by iterating two steps.

The Algorithm

1. **Choose k** (the number of clusters you want).
2. **Initialize** k random “centroids” (cluster centers).
3. **Assign** each data point to the nearest centroid.
4. **Update** each centroid to the mean of its assigned points.
5. **Repeat** steps 3-4 until assignments stop changing.

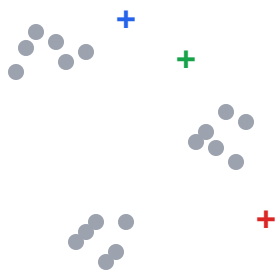
That is it. The algorithm converges quickly, typically in 10-20 iterations.

Properties

Property	K-means
Shape of clusters	Spherical (roughly equal-sized blobs)
Requires k in advance	Yes
Handles outliers	Poorly — outliers pull centroids
Handles unequal cluster sizes	Poorly — tends to split large clusters
Speed	Very fast, even on large datasets
Deterministic	No — depends on random initialization

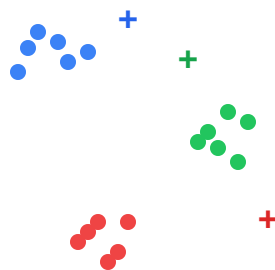
K-Means Iteration (k=3)

1. Random Centroids



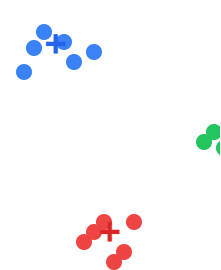
Centroids placed randomly

2. Assign to Nearest



Colors show assignments

3. Update Cent



Centroids at cluster

→ →

+ Blue centroid + Red centroid + Green centroid Repeat steps 2-3 until convergence

Hierarchical Clustering: Dendrogram

S1 S2 S3 S4 S5 S6 S7 S8

Cut here → 3 clusters Height

The Initialization Problem

Because k-means starts with random centroids, different runs can give different results. A bad initialization might converge to a suboptimal solution. The standard fix is to run k-means multiple times (say 10-50) with different random seeds and keep the solution with the lowest total within-cluster variance.

```
set_seed(42)
# K-means clustering on PCA-reduced expression data
let result = pca(expression_matrix)
let pca_scores = result.scores |> take_cols(0..20) # first 20 PCs

let clusters = kmeans(pca_scores, 4)
print("Cluster sizes: " + str(clusters.sizes))
print("Total within-cluster variance: " +
      str(clusters.total_within_ss))
```

Choosing k: The Elbow Method

The hardest part of k-means is choosing k. If you pick k = 500, every tumor gets its own cluster — perfect within-cluster similarity but meaningless. If you pick k = 1, everything is in one group — also meaningless. The right k is somewhere in between.

The **elbow method** runs k-means for $k = 1, 2, 3, \dots, K$ and plots the total within-cluster sum of squares (WCSS) against k . As k increases, WCSS always decreases (more clusters = tighter fits). The “elbow” — where the rate of decrease sharply levels off — suggests the best k .

```
set_seed(42)
# Elbow plot to find optimal k
let wcss = seq(1, 15) |> map(|k| {
  let cl = kmeans(pca_scores, k)
  {k: k, within_ss: cl$total_within_ss}
}) |> to_table()
plot(wcss, {type: "line", x: "k", y: "within_ss",
  title: "Elbow Plot – Optimal Number of Clusters"})
```

Common pitfall: The elbow is often ambiguous. Real data rarely shows a sharp bend. If the elbow plot suggests k could be 3, 4, or 5, use biological knowledge or validation metrics (like silhouette scores) to decide.

Hierarchical Clustering

Hierarchical clustering builds a tree (dendrogram) of nested clusters rather than producing a flat partition. It does not require you to choose k in advance — you can cut the tree at any height to get the desired number of clusters.

Agglomerative (Bottom-Up) Algorithm

1. Start with each sample as its own cluster (500 clusters for 500 tumors).
2. Find the two closest clusters and merge them (now 499 clusters).
3. Repeat until everything is in one cluster.
4. The dendrogram records every merge and the distance at which it occurred.

Linkage Methods

“Distance between clusters” is ambiguous when clusters contain multiple points. The linkage method resolves this:

Linkage	Distance between clusters A and B	Tendency
Single	Minimum distance between any pair	Produces long, chained clusters
Complete	Maximum distance between any pair	Produces compact, equal-sized clusters
Average	Mean pairwise distance	Compromise between single and complete
Ward	Increase in total variance when merged	Minimizes within-cluster variance (like k-means)

Ward linkage is the most common choice in genomics because it tends to produce balanced, compact clusters similar to k-means.

```
# Hierarchical clustering with Ward linkage
let hc = hclust(pca_scores, "ward")

# Cut tree at k=4 clusters
let labels = hc |> cut_tree(4)
print("Hierarchical cluster sizes: " + str(table_counts(labels)))
```

The Dendrogram

The dendrogram is one of the most informative visualizations in genomics. The height of each merge indicates how dissimilar the merged clusters were. A long vertical line before a merge suggests a clear separation; short lines suggest gradual transitions.

```
# Dendrogram with colored clusters
dendrogram(hc, {k: 4, title: "Tumor Expression – Hierarchical
Clustering"})
```


DBSCAN: Density-Based Clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) takes a fundamentally different approach. Instead of assuming spherical clusters, it finds regions of high density separated by regions of low density. It also identifies outliers — points in low-density regions that do not belong to any cluster.

The Algorithm

1. For each point, count neighbors within distance epsilon.
2. If a point has at least min_samples neighbors, it is a “core point.”
3. Connect core points that are within epsilon of each other.
4. Connected components of core points form clusters.
5. Non-core points within epsilon of a core point join that cluster.
6. Remaining points are labeled as noise (outliers).

Properties

Property	DBSCAN
Shape of clusters	Any shape (can find crescents, rings, etc.)
Requires k in advance	No — finds k automatically
Handles outliers	Excellent — explicitly labels them
Parameters	epsilon (neighborhood radius), min_samples
Handles varying density	Poorly — one epsilon for all regions

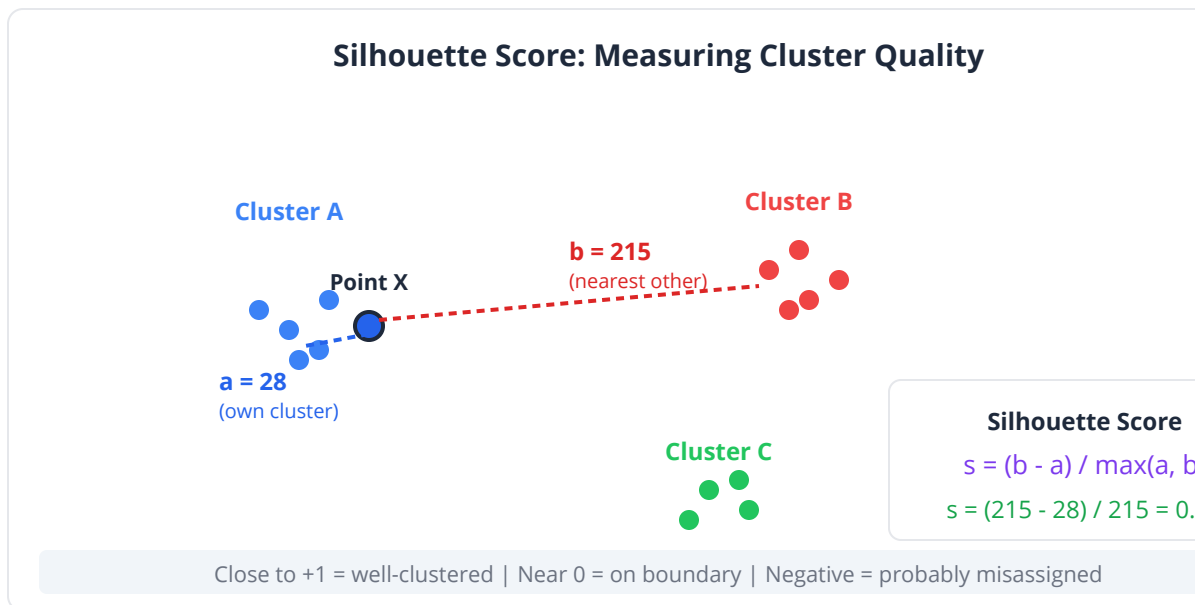
```
set_seed(42)
# DBSCAN on PCA scores
let db = dbscan(pca_scores, 2.5, 10)
print("Clusters found: " + str(db.n_clusters))
print("Noise points: " + str(db.n_noise))
```

Key insight: DBSCAN is excellent for scRNA-seq data where clusters are irregularly shaped and outlier cells (doublets, dying cells) should be flagged rather than forced into a cluster. K-means forces every point into a cluster — DBSCAN does not.

Silhouette Scores: Validating Clusters

How do you know your clusters are real? The silhouette score provides internal validation. For each point, it measures:

- **a** = mean distance to other points in the same cluster
- **b** = mean distance to points in the nearest other cluster
- **Silhouette** = $(b - a) / \max(a, b)$



The score ranges from -1 to +1:

Score	Interpretation
+1	Point is far from neighboring clusters — excellent clustering
0	Point is on the boundary between clusters
-1	Point is probably in the wrong cluster

Average silhouette scores:

Average Score	Quality
0.71 - 1.00	Strong structure
0.51 - 0.70	Reasonable structure
0.26 - 0.50	Weak structure, possibly artificial
< 0.25	No substantial structure found

```

set_seed(42)
# Silhouette analysis for different k
for k in 2..8 {
  let cl = kmeans(pca_scores, k)
  let sil = cl.silhouette
  print("k=" + str(k) + " silhouette: " + str(round(sil, 3)))
}

```

Common pitfall: A high silhouette score does not prove biological relevance. Random data with well-separated clusters in PCA space can have high silhouette scores. Always validate clusters against independent information — clinical outcomes, known markers, or held-out data.

Clustering Always Finds Clusters — Even in Noise

This is the single most important warning about clustering. Run k-means with $k = 3$ on completely random data, and it will dutifully return three clusters. The clusters will look somewhat real in a scatter plot. But they are meaningless.

```
set_seed(42)
# DANGER: Clustering random noise
let noise = range(0, 500) |> map(|_| rnorm(20, 0, 1)) |> matrix()
let fake_clusters = kmeans(noise, 3)
let noise_pca = pca(noise)

# These clusters are pure noise – but the plot looks convincing!
scatter(noise_pca.scores[0], noise_pca.scores[1])

# Silhouette score will be low
let sil = fake_clusters.silhouette
print("Silhouette on noise: " + str(round(sil, 3))) # ~0.1-0.2
```

Always compare your clustering against a null — random data of the same dimensions and sample size. If the silhouette score on real data is not substantially higher than on random data, the clusters may not be real.

Heatmaps with Clustering

The clustered heatmap is the workhorse visualization of genomics. It shows expression values as colors, with both rows (genes) and columns (samples) reordered by hierarchical clustering. Similar genes are placed next to similar genes; similar samples next to similar samples.

```
# Clustered heatmap of top variable genes
let top_genes = variance_per_column(expression_matrix)
  |> sort_by(|x| -x.value)
  |> take(100)
  |> map(|x| x.name)

let sub_matrix = expression_matrix |> select_cols(top_genes)

heatmap(sub_matrix, {cluster_rows: true, cluster_cols: true,
  color_scale: "viridis",
  title: "Top 100 Variable Genes – Clustered Heatmap"})
```

Clustering in BioLang — Complete Pipeline

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# Full clustering analysis on simulated tumor expression data

# Simulate 500 tumors with 4 molecular subtypes
let n_per_group = 125
let n_genes = 200

let subtype_a = table(n_per_group, n_genes, "rnorm")
let subtype_b = table(n_per_group, n_genes, "rnorm")
let subtype_c = table(n_per_group, n_genes, "rnorm")
let subtype_d = table(n_per_group, n_genes, "rnorm")

# Each subtype has distinct gene blocks elevated
for i in 0..n_per_group {
  for j in 0..50 { subtype_a[i][j] = subtype_a[i][j] + 3.0 }
  for j in 50..100 { subtype_b[i][j] = subtype_b[i][j] + 3.0 }
  for j in 100..150 { subtype_c[i][j] = subtype_c[i][j] + 3.0 }
  for j in 150..200 { subtype_d[i][j] = subtype_d[i][j] + 3.0 }
}

let expr = rbind(subtype_a, subtype_b, subtype_c, subtype_d)
let true_labels = repeat("A", 125) + repeat("B", 125) + repeat("C",
125) + repeat("D", 125)

# 1. PCA for visualization and dimensionality reduction
let pca_result = pca(expr)
scatter(pca_result.scores[0], pca_result.scores[1])

# 2. K-means clustering
let km = kmeans(expr, 4)
scatter(pca_result.scores[0], pca_result.scores[1])

# 3. Elbow plot
let wcss = seq(1, 10) |> map(|k| {
  let cl = kmeans(expr, k)
  {k: k, within_ss: cl.total_within_ss}
}) |> to_table()
plot(wcss, {type: "line", x: "k", y: "within_ss",
  title: "Elbow Plot"})

# 4. Silhouette analysis
for k in 2..8 {
  let cl = kmeans(expr, k)
```

```

    let sil = cl.silhouette
    print("k=" + str(k) + " silhouette=" + str(round(sil, 3)))
}

# 5. Hierarchical clustering
let hc = hclust(expr, "ward")
dendrogram(hc, {k: 4, title: "Dendrogram – Ward Linkage"})

let hc_labels = hc |> cut_tree(4)
scatter(pca_result.scores[0], pca_result.scores[1])

# 6. DBSCAN
let db = dbscan(expr, 8.0, 10)
print("DBSCAN found " + str(db.n_clusters) + " clusters, " +
      str(db.n_noise) + " noise")

# 7. Compare clustering to true labels
print("\nK-means vs true labels:")
print(cross_tabulate(km.labels, true_labels))

print("\nHierarchical vs true labels:")
print(cross_tabulate(hc_labels, true_labels))

# 8. Clustered heatmap
heatmap(expr, {cluster_rows: true, cluster_cols: true,
  color_scale: "red_blue",
  title: "Expression Heatmap – K-means Clusters"})

```

Python:

```
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage
import seaborn as sns

# K-means
km = KMeans(n_clusters=4, n_init=25, random_state=42)
km_labels = km.fit_predict(expr)

# Elbow plot
wcss = [KMeans(n_clusters=k, n_init=25).fit(expr).inertia_ for k in
range(1, 10)]
plt.plot(range(1, 10), wcss, 'bo-')
plt.show()

# Silhouette
for k in range(2, 8):
    km_k = KMeans(n_clusters=k, n_init=25).fit(expr)
    print(f"k={k} silhouette={silhouette_score(expr,
km_k.labels_):.3f}")

# Hierarchical
Z = linkage(expr, method='ward')
dendrogram(Z)
plt.show()

# Heatmap
sns.clustermap(expr, method='ward', cmap='RdBu_r')
plt.show()
```

R:

```
# K-means
km <- kmeans(expr, centers = 4, nstart = 25)
table(km$cluster, true_labels)

# Elbow plot
wcss <- sapply(1:9, function(k) kmeans(expr, k,
nstart=25)$tot.withinss)
plot(1:9, wcss, type="b", xlab="k", ylab="Total within SS")

# Hierarchical
hc <- hclust(dist(expr), method = "ward.D2")
plot(hc)
cutree(hc, k = 4)

# Heatmap
library(pheatmap)
pheatmap(expr, clustering_method = "ward.D2", show_rownames = FALSE)

# Silhouette
library(cluster)
for (k in 2:7) {
  cl <- kmeans(expr, k, nstart=25)
  cat(sprintf("k=%d silhouette=%.3f\n", k,
mean(silhouette(cl$cluster, dist(expr))[,3])))
}
```

Exercises

1. **Three subtypes.** The pathologist says 3 subtypes, but your elbow plot suggests 4 or 5. Simulate 500 tumors with 5 true subtypes (100 each). Run k-means for $k = 3, 4, 5, 6$. Use silhouette scores and cross-tabulation against true labels. Which k best recovers the truth?

```
# Your code: simulate 5 subtypes, test multiple k values
```

2. **Linkage comparison.** Run hierarchical clustering on the same data with single, complete, average, and Ward linkage. Cut each at $k = 5$. Which linkage best recovers the true labels? Visualize the four dendrograms.

Your code: four linkage methods, compare cross-tabulations

3. **DBSCAN tuning.** On the 5-subtype data, try DBSCAN with epsilon values from 3 to 15 (step 1) and min_samples from 5 to 20. Find the combination that gives 5 clusters with the fewest noise points.

Your code: grid search over epsilon and min_samples

4. **Noise test.** Generate pure random data (500 samples x 200 genes, no structure). Run k-means with $k = 4$. Plot the result. Calculate the silhouette score. Now compare to the silhouette score from the structured data. What is the difference?

Your code: cluster noise, compare silhouette to real data

5. **Heatmap with annotation.** Create a clustered heatmap of the top 50 most variable genes, with a color bar showing both k-means cluster assignment and true subtype. Do the two agree?

Your code: select top variable genes, heatmap with dual annotation

Key Takeaways

- Clustering is unsupervised grouping — it finds structure without labels, making it essential for discovering molecular subtypes, cell populations, and other hidden patterns.
- K-means is fast and simple but requires choosing k in advance, assumes spherical clusters, and is sensitive to initialization.
- The elbow method and silhouette scores help choose k , but neither is definitive — biological knowledge must guide the final decision.
- Hierarchical clustering produces a dendrogram showing nested relationships; Ward linkage is the default choice for genomics.
- DBSCAN finds arbitrarily shaped clusters and identifies outliers, making it valuable for scRNA-seq data, but requires tuning epsilon and

min_samples.

- Silhouette scores validate clustering quality: above 0.5 is reasonable, above 0.7 is strong, but always compare against random data.
- Clustering always finds clusters, even in noise. Never trust clusters without validation against independent evidence.
- Clustered heatmaps are the standard visualization for gene expression patterns across samples.

What's Next

PCA and clustering assume that the data you have is large enough to draw conclusions. But what if your dataset is tiny — six mice per group, far too few to verify normality or trust asymptotic theory? Tomorrow, we meet resampling methods: bootstrap and permutation tests. These techniques let your data speak for itself, building confidence intervals and testing hypotheses without any distributional assumptions, by literally reshuffling the data thousands of times.

Day 23: Resampling — Bootstrap and Permutation Tests

Day 23 of 30 Prerequisites: Days 6-8 ~60 min reading
Non-Parametric Inference

The Problem

Your collaborator is studying a rare metabolic disorder. She has tissue samples from 6 affected mice and 6 controls, measuring enzyme activity in each. The treatment group shows higher median enzyme activity, and she wants to know if the difference is real.

You reach for a t-test, but hesitate. With only 6 observations per group, you cannot meaningfully assess whether the data is normally distributed. A Shapiro-Wilk test on 6 points has almost no power. The Wilcoxon rank-sum test is an option, but with only 6 per group, it can only detect very large effects.

What if you could test the hypothesis without assuming any distribution at all? What if you could build a confidence interval for any statistic — not just the mean, but the median, the trimmed mean, the ratio of two variances, the 90th percentile — without knowing the population distribution?

You can. Resampling methods let the data speak for itself, replacing theoretical assumptions with raw computational power. They are among the most broadly applicable tools in statistics, and after today, you will wonder how you ever lived without them.

What Are Resampling Methods?

Resampling methods draw repeated samples from your data to estimate the sampling distribution of a statistic. Instead of deriving a formula for how the mean varies from sample to sample (the classical approach), you literally simulate the process: draw a new sample, compute the statistic, repeat thousands of times, and look at the distribution of results.

Think of it like this. You have a bag containing 12 marbles (your data). You want to know how variable the “average marble weight” is. The classical approach derives a formula based on the normal distribution. The resampling approach says: shake the bag, pull out 12 marbles (with replacement), weigh them, compute the average. Put them back. Repeat 10,000 times. The distribution of those 10,000 averages tells you everything you need to know — no formula, no assumptions.

Key insight: Resampling methods trade mathematical assumptions for computational effort. With modern computers, 10,000 resamples takes milliseconds. The only assumption is that your sample is representative of the population — the same assumption underlying all of statistics.

The Bootstrap

The bootstrap, invented by Bradley Efron in 1979, is one of the most important ideas in modern statistics. It estimates the sampling distribution of any statistic by resampling with replacement from your data.

How It Works

1. You have a sample of n observations.
2. Draw a new sample of n observations **with replacement** from your original data. Some observations will appear multiple times; some will not appear at all.

3. Compute your statistic of interest (mean, median, standard deviation, correlation, whatever) on this bootstrap sample.
4. Repeat steps 2-3 many times (typically 10,000).
5. The distribution of the computed statistics is the **bootstrap distribution**. It approximates the sampling distribution of your statistic.

Bootstrap Resampling: How It Works

Original Data (n=8)

3.8	4.2	4.9
5.1	5.5	5.8
6.3	4.7	

Sample with replacement

→

Bootstrap #1 4.2 5.1 4.2 6.3 5.5 3.8 5.1 4.9 median = 5.0 Bootstrap #2 5.8 5.8 3.8 4.7 6.3 4.9 5.5 6.3 median = 5.35 Bootstrap #10000 4.7 5.1 4.2 5.5 4.2 3.8 5.8 4.9 median = 4.8

...

→

Bootstrap Distribution Bootstrap Median Values 95% Confidence Interval 2.5th %ile 97.5th %ile Note: some values repeat (4.2 appears twice in #1) and some are missing -- this is sampling with replacement

Bootstrap Confidence Intervals

The simplest bootstrap CI uses percentiles. For a 95% CI, take the 2.5th and 97.5th percentiles of the bootstrap distribution.

```
set_seed(42)
# Enzyme activity in 6 treatment mice
let treatment = [4.2, 5.1, 3.8, 6.3, 4.9, 5.5]

# Bootstrap 95% CI for the median
let n_boot = 10000
let boot_medians = []
for i in 0..n_boot {
  let resample = seq(1, length(treatment))
  |> map(|_| treatment[random_int(0, length(treatment) - 1)])
  boot_medians = boot_medians + [median(resample)]
}
let ci_lower = quantile(boot_medians, 0.025)
let ci_upper = quantile(boot_medians, 0.975)

print("Observed median: " + str(median(treatment)))
print("Bootstrap 95% CI: [" +
  str(round(ci_lower, 2)) + ", " +
  str(round(ci_upper, 2)) + "]")

# Visualize the bootstrap distribution
histogram(boot_medians, {bins: 50,
  title: "Bootstrap Distribution of Median Enzyme Activity"})
```

Why Replacement Matters

Drawing with replacement is what makes the bootstrap work. Each bootstrap sample is a plausible alternative dataset — a dataset you might have obtained if you had re-run the experiment. By computing your statistic on thousands of these alternative datasets, you learn how much the statistic would vary from experiment to experiment.

Without replacement, you would just get your original data back (in a different order), and the statistic would never change.

Bootstrap for Any Statistic

The beauty of the bootstrap is its universality. Classical formulas for confidence intervals exist for means, proportions, and a few other well-behaved statistics. But what about the median? The inter-quartile range? The ratio of two means? The coefficient of variation? The 95th percentile? For most of these, no clean formula exists. The bootstrap handles them all identically.

Statistic	Classical CI formula?	Bootstrap works?
Mean	Yes (t-interval)	Yes
Median	Complicated, approximate	Yes
Standard deviation	Approximate (chi-square)	Yes
Correlation	Fisher z-transform	Yes
Ratio of means	No simple formula	Yes
90th percentile	No simple formula	Yes
Difference in medians	No simple formula	Yes
Any custom function	Almost never	Yes

```
set_seed(42)
# Bootstrap CI for the coefficient of variation
let data = [4.2, 5.1, 3.8, 6.3, 4.9, 5.5, 4.7, 5.8]

let n_boot = 10000
let boot_cvs = []
for i in 0..n_boot {
  let resample = seq(1, length(data))
  |> map(|_| data[random_int(0, length(data) - 1)])
  boot_cvs = boot_cvs + [stdev(resample) / mean(resample)]
}

print("CV: " + str(round(stdev(data) / mean(data), 3)))
print("95% CI: [" + str(round(quantile(boot_cvs, 0.025), 3)) + ", "
+
  str(round(quantile(boot_cvs, 0.975), 3)) + "])")
```


Bootstrap Hypothesis Testing

You can also use the bootstrap for hypothesis testing. To test whether the mean of a population is equal to some value μ_0 :

1. Shift your data so that its mean equals μ_0 (subtract the difference).
2. Bootstrap from the shifted data.
3. Compute the statistic on each bootstrap sample.
4. The p-value is the proportion of bootstrap statistics as extreme as or more extreme than your observed statistic.

```
set_seed(42)
# Bootstrap test: is the mean enzyme activity > 4.0?
let observed_mean = mean(treatment)
let shifted = treatment |> map(|x| x - (observed_mean - 4.0))

let boot_means = []
for i in 0..10000 {
  let resample = seq(1, length(shifted))
  |> map(|_| shifted[random_int(0, length(shifted) - 1)])
  boot_means = boot_means + [mean(resample)]
}
let p_value = boot_means
|> filter(|x| x >= observed_mean)
|> length() / 10000

print("Observed mean: " + str(round(observed_mean, 2)))
print("Bootstrap p-value (H0:  $\mu = 4.0$ ): " + str(round(p_value, 4)))
```

Common pitfall: The bootstrap is not magic. It cannot create information that is not in your data. With only 6 observations, the bootstrap distribution is limited to combinations of those 6 values. The bootstrap CI will be approximate, and it underestimates uncertainty when n is very small (say, $n < 10$). It works best when n is moderate to large.

The Permutation Test

The permutation test directly addresses the question: “Could the observed difference between groups have arisen by chance?” It does so by exhaustively (or approximately) considering all possible ways to assign labels to the data.

How It Works

1. Compute the test statistic (e.g., difference in means) for the observed data.
2. **Shuffle** the group labels randomly, keeping the data values fixed.
3. Recompute the test statistic on the shuffled data.
4. Repeat steps 2-3 many times (10,000 or more).
5. The p-value is the proportion of shuffled statistics as extreme as or more extreme than the observed one.

The logic is simple: if group membership does not matter (the null hypothesis), then shuffling labels should produce similar statistics. If the observed statistic is far from what shuffling produces, the group difference is real.

Permutation Test: Shuffling Labels Under the Null

Original Data		Shuffle labels →	Shuffled Labels	
Treatment	Control		"Treatment"	"Control"
4.2, 5.1	3.1, 2.8		3.1, 5.1	4.2, 3.5
3.8, 6.3	3.5, 3.2		4.9, 3.2	5.5, 2.9
4.9, 5.5	2.9, 3.6		6.3, 2.8	3.8, 3.6
diff = 1.80			diff = 0.37	

Repeat 10,000x →

Null Distribution Observed (1.80) p-value = fraction of shuffled differences as extreme as observed = $(\text{count where } |\text{diff}| \geq 1.80) / 10,000$ If very few shuffled values are this extreme, the group difference is unlikely due to chance.

5-Fold Cross-Validation

Fold 1 Fold 2 Fold 3 Fold 4 Fold 5 Test set Training set

```
set_seed(42)
# Treatment vs control enzyme activity
let treatment = [4.2, 5.1, 3.8, 6.3, 4.9, 5.5]
let control = [3.1, 2.8, 3.5, 3.2, 2.9, 3.6]

# Permutation test for difference in means
let observed_diff = mean(treatment) - mean(control)
let combined = treatment + control
let n_perm = 10000
let null_diffs = []

for i in 0..n_perm {
  let shuffled = shuffle(combined)
  let perm_diff = mean(shuffled |> take(6)) - mean(shuffled |>
drop(6))
  null_diffs = null_diffs + [perm_diff]
}

let p_value = null_diffs
  |> filter(|d| abs(d) >= abs(observed_diff))
  |> length() / n_perm

print("Observed difference: " + str(round(observed_diff, 3)))
print("Permutation p-value: " + str(round(p_value, 4)))

# Visualize the null distribution
histogram(null_diffs, {bins: 50,
  title: "Permutation Null Distribution"})
```

Permutation vs Bootstrap

Feature	Bootstrap	Permutation Test
Primary use	Confidence intervals	Hypothesis testing
Resamples from	One group (with replacement)	Combined data (without replacement, shuffling labels)
Tests	Any statistic; H_0 : parameter = value	Two-group (or multi-group) comparisons
Null distribution	Centered at observed statistic	Centered at zero difference
Assumptions	Sample is representative	Exchangeability under H_0

Key insight: Use the bootstrap when you want a confidence interval. Use the permutation test when you want a p-value comparing groups. They are complementary, not competing methods.

Exact Permutation Tests

With small samples, you can enumerate all possible permutations. For 6 treatment and 6 control observations, there are $C(12, 6) = 924$ possible label assignments. You can compute the test statistic for every one and get an exact p-value.

```
# Exact permutation test (small sample)
# With 6+6 = 12 observations, there are C(12,6) = 924 permutations
# We can enumerate all of them for an exact p-value
# (In practice, use the randomized version above for larger datasets)
print("Exact permutation: enumerate all 924 label assignments")
print("This gives exact p-values for small samples")
```

Cross-Validation

Cross-validation is a resampling method for evaluating predictive models. Instead of testing a model on the same data it was trained on (which overestimates performance), you repeatedly hold out a portion of the data for testing.

K-Fold Cross-Validation

1. Divide the data into k equally-sized folds.
2. For each fold: train the model on the other $k-1$ folds, predict on the held-out fold, record the error.
3. Average the errors across all k folds.

Common choices: $k = 5$ or $k = 10$. Leave-one-out cross-validation (LOOCV) uses $k = n$.

```

set_seed(42)
# Generate data with a true linear relationship
let x = seq(1, 100) |> take(50)
let y = x |> map(|xi| 2.5 * xi + 10 + rnorm(1, 0, 15)[0])

# 5-fold cross-validation for linear regression
let k = 5
let fold_size = length(x) / k
let fold_errors = []

for f in 0..k {
  let test_idx = seq(f * fold_size, (f + 1) * fold_size - 1)
  let train_x = []
  let train_y = []
  let test_x = []
  let test_y = []
  for i in 0..length(x) {
    if i >= f * fold_size && i < (f + 1) * fold_size {
      test_x = test_x + [x[i]]
      test_y = test_y + [y[i]]
    } else {
      train_x = train_x + [x[i]]
      train_y = train_y + [y[i]]
    }
  }
  let model = lm(train_x, train_y)
  let preds = test_x |> map(|xi| model.intercept + model.slope * xi)
  let mse = mean(zip(preds, test_y) |> map(|p| (p[0] - p[1]) * (p[0]
- p[1])))
  fold_errors = fold_errors + [mse]
}

print("Mean squared error per fold: " + str(fold_errors))
print("Average MSE: " + str(round(mean(fold_errors), 2)))
print("Standard deviation of MSE: " + str(round(stdev(fold_errors),
2)))

```

Why Cross-Validation Matters in Genomics

In genomics, overfitting is pervasive. A model trained on 20,000 gene expression features and 100 samples can easily memorize the training data while learning nothing generalizable. Cross-validation reveals this: if training

accuracy is 99% but cross-validated accuracy is 52%, the model is memorizing, not learning.

Clinical relevance: Gene expression classifiers for cancer prognosis (like MammaPrint or Oncotype DX) must be validated on independent cohorts. Cross-validation provides an estimate of out-of-sample performance during development, but ultimate validation requires truly independent datasets.

The Jackknife

The jackknife (predating the bootstrap by two decades) is a leave-one-out resampling method. It computes the statistic n times, each time leaving out one observation. The variation among these n estimates quantifies uncertainty.

```
# Jackknife estimate of standard error for the median
let data = [4.2, 5.1, 3.8, 6.3, 4.9, 5.5, 4.7, 5.8]
let n = length(data)
let full_stat = median(data)

let jk_estimates = []
for i in 0..n {
  let subset = data |> filter_index(|j, _| j != i)
  jk_estimates = jk_estimates + [median(subset)]
}

let jk_mean = mean(jk_estimates)
let jk_bias = (n - 1) * (jk_mean - full_stat)
let jk_se = sqrt((n - 1) / n * sum(jk_estimates |> map(|x| (x -
jk_mean) * (x - jk_mean))))

print("Jackknife estimate: " + str(round(full_stat, 3)))
print("Jackknife SE: " + str(round(jk_se, 3)))
print("Jackknife bias: " + str(round(jk_bias, 4)))
```

The jackknife is less versatile than the bootstrap but useful for bias estimation and influence analysis (which observation has the most impact on the statistic?).

Method	Resamples	With replacement	Primary use
Bootstrap	10,000+ random	Yes	CI's for any statistic
Permutation	10,000+ shuffled	No (labels shuffled)	Hypothesis testing
Cross-validation	k folds	No (systematic split)	Model evaluation
Jackknife	n leave-one-out	No	Bias, SE, influence

When to Use Resampling

Resampling methods are appropriate when:

- **Sample size is small** and normality cannot be verified
- **The statistic has no known distribution** (median, ratio, custom function)
- **You want to avoid distributional assumptions** entirely
- **Classical methods are unavailable** for your specific analysis
- **You want a second opinion** — bootstrap CI's should roughly agree with parametric CI's when assumptions hold

Resampling methods are less appropriate when:

- **The sample is very small** ($n < 5$): too few unique bootstrap samples
- **The data has complex structure** (time series, spatial): naive resampling breaks the structure
- **Computational cost matters**: millions of bootstraps on large datasets can be slow
- **A well-validated parametric method exists** and assumptions are met: the parametric method will be more efficient (tighter CI's for the same sample size)

Resampling in BioLang — Complete Pipeline

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# Mouse experiment: enzyme activity
let treatment = [4.2, 5.1, 3.8, 6.3, 4.9, 5.5]
let control = [3.1, 2.8, 3.5, 3.2, 2.9, 3.6]

# Helper: bootstrap a statistic from a list
fn boot_ci(data, stat_fn, n_boot) {
  let samples = []
  for i in 0..n_boot {
    let resample = seq(1, length(data))
    |> map(|_| data[random_int(0, length(data) - 1)])
    samples = samples + [stat_fn(resample)]
  }
  {lower: quantile(samples, 0.025), upper: quantile(samples, 0.975),
  dist: samples}
}

# =====
# 1. Bootstrap CI for median in each group
# =====
let boot_treat = boot_ci(treatment, median, 10000)
let boot_ctrl = boot_ci(control, median, 10000)

print("=== Bootstrap CIs for Median ===")
print("Treatment: " + str(round(median(treatment), 2)) +
  " [" + str(round(boot_treat.lower, 2)) + ", " +
  str(round(boot_treat.upper, 2)) + "]")
print("Control: " + str(round(median(control), 2)) +
  " [" + str(round(boot_ctrl.lower, 2)) + ", " +
  str(round(boot_ctrl.upper, 2)) + "]")

# =====
# 2. Bootstrap CI for difference in medians
# =====
let boot_diffs = []
for i in 0..10000 {
  let res_t = seq(1, 6) |> map(|_| treatment[random_int(0, 5)])
  let res_c = seq(1, 6) |> map(|_| control[random_int(0, 5)])
  boot_diffs = boot_diffs + [median(res_t) - median(res_c)]
}

print("\n=== Bootstrap CI for Median Difference ===")
print("Difference: " + str(round(median(treatment) -
```

```

median(control), 2)))
print("95% CI: [" + str(round(quantile(boot_diffs, 0.025), 2)) + ",
" +
  str(round(quantile(boot_diffs, 0.975), 2)) + "]")

# =====
# 3. Permutation test
# =====
let obs_diff = mean(treatment) - mean(control)
let combined = treatment + control
let null_diffs = []
for i in 0..10000 {
  let s = shuffle(combined)
  null_diffs = null_diffs + [mean(s |> take(6)) - mean(s |>
drop(6))]
}
let perm_p = null_diffs |> filter(|d| abs(d) >= abs(obs_diff)) |>
length() / 10000

print("\n=== Permutation Test ===")
print("Observed mean diff: " + str(round(obs_diff, 3)))
print("Permutation p-value: " + str(round(perm_p, 4)))

# Compare to Welch t-test
let tt = ttest(treatment, control)
print("Welch t-test p-value: " + str(round(tt.p_value, 4)))

# Visualize the null distribution
histogram(null_diffs, {bins: 50,
  title: "Permutation Null vs Observed"})

# =====
# 4. Cross-validate a regression model
# =====
let gene_expr = seq(1, 100) |> take(40)
let drug_response = gene_expr |> map(|x| -0.5 * x + 80 + rnorm(1, 0,
10)[0])

let k = 5
let fold_size = length(gene_expr) / k
let fold_errors = []
for f in 0..k {
  let train_x = []
  let train_y = []
  let test_x = []
  let test_y = []
  for j in 0..length(gene_expr) {
    if j >= f * fold_size && j < (f + 1) * fold_size {

```

```

      test_x = test_x + [gene_expr[j]]
      test_y = test_y + [drug_response[j]]
    } else {
      train_x = train_x + [gene_expr[j]]
      train_y = train_y + [drug_response[j]]
    }
  }
  let model = lm(train_x, train_y)
  let preds = test_x |> map(|xi| model.intercept + model.slope * xi)
  let mse = mean(zip(preds, test_y) |> map(|p| (p[0] - p[1]) * (p[0]
- p[1])))
  fold_errors = fold_errors + [mse]
}

print("\n=== Cross-Validation ===")
print("5-fold CV MSE: " + str(round(mean(fold_errors), 2)) +
      " +/- " + str(round(stdev(fold_errors), 2)))

# =====
# 5. Compare bootstrap vs parametric CI
# =====
let boot_means = boot_ci(treatment, mean, 10000)
let t_ci = ttest_one(treatment, 0)

print("\n=== Bootstrap vs Parametric CI for Mean ===")
print("Bootstrap 95% CI: [" + str(round(boot_means.lower, 2)) +
      ", " + str(round(boot_means.upper, 2)) + "]")
print("t-based 95% CI: [" + str(round(t_ci.ci_lower, 2)) +
      ", " + str(round(t_ci.ci_upper, 2)) + "]")

# =====
# 6. Jackknife for influence detection
# =====
let n = length(treatment)
let jk_estimates = []
for i in 0..n {
  let subset = treatment |> filter_index(|j, _| j != i)
  jk_estimates = jk_estimates + [mean(subset)]
}
print("\n=== Jackknife ===")
print("Leave-one-out estimates:")
for i in 0..n {
  print(" Without obs " + str(i+1) + ": " +
        str(round(jk_estimates[i], 3)))
}
let max_diff = 0
let max_idx = 0
for i in 0..n {

```

```

    let d = abs(jk_estimates[i] - mean(treatment))
    if d > max_diff { max_diff = d; max_idx = i }
  }
  print("Most influential observation: " + str(max_idx + 1))

```

Python:

```

import numpy as np
from scipy import stats
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.utils import resample

# Bootstrap CI for median
np.random.seed(42)
treatment = np.array([4.2, 5.1, 3.8, 6.3, 4.9, 5.5])
boot_medians = [np.median(resample(treatment)) for _ in
range(10000)]
ci = np.percentile(boot_medians, [2.5, 97.5])
print(f"Bootstrap 95% CI for median: [{ci[0]:.2f}, {ci[1]:.2f}]")

# Permutation test
control = np.array([3.1, 2.8, 3.5, 3.2, 2.9, 3.6])
observed = treatment.mean() - control.mean()
combined = np.concatenate([treatment, control])
null_dist = []
for _ in range(10000):
    np.random.shuffle(combined)
    null_dist.append(combined[:6].mean() - combined[6:].mean())
p_value = np.mean(np.abs(null_dist) >= np.abs(observed))

# Cross-validation
from sklearn.model_selection import cross_val_score
model = LinearRegression()
scores = cross_val_score(model, X.reshape(-1,1), y, cv=5,
scoring='neg_mean_squared_error')

```

R:

```
# Bootstrap CI for median
library(boot)
med_fn <- function(data, i) median(data[i])
b <- boot(treatment, med_fn, R = 10000)
boot.ci(b, type = "perc")

# Permutation test
library(coin)
wilcox_test(values ~ group, data = df, distribution = "exact")

# Or manual permutation
observed <- mean(treatment) - mean(control)
combined <- c(treatment, control)
null_dist <- replicate(10000, {
  perm <- sample(combined)
  mean(perm[1:6]) - mean(perm[7:12])
})
p_value <- mean(abs(null_dist) >= abs(observed))

# Cross-validation
library(caret)
cv <- train(y ~ x, data = df, method = "lm",
            trControl = trainControl(method = "cv", number = 5))
```

Exercises

1. **Bootstrap the correlation.** Given paired measurements from 15 patients (gene expression and drug response), compute the Pearson correlation and a bootstrap 95% CI. Is the correlation significantly different from zero?

```
let expr = [2.1, 4.5, 3.2, 5.8, 1.9, 6.2, 3.8, 4.1, 5.5, 2.8, 3.5,
6.0, 4.3, 2.5, 5.1]
let response = [35, 62, 41, 78, 30, 82, 55, 50, 71, 38, 48, 79, 58,
33, 68]
# Your code: bootstrap CI for cor(expr, response)
```

2. **Permutation test for median.** Using the treatment and control data from this chapter, run a permutation test using the difference in medians (not means) as the test statistic. Compare the p-value to the mean-based permutation test.

```
# Your code: permutation_test with statistic: "median_diff"
```

3. **Bootstrap vs t-test at n=6.** Generate 1,000 datasets of $n=6$ from a skewed distribution (e.g., exponential). For each, compute both a t-based CI and a bootstrap CI for the mean. Compare coverage rates (how often the true mean falls within the CI). Which method is more reliable?

```
# Your code: simulation study comparing CI coverage
```

4. **Cross-validation showdown.** Fit a linear regression predicting drug response from gene expression. Compare 5-fold, 10-fold, and leave-one-out cross-validation. Do they agree on the model's prediction error?

```
# Your code: three CV approaches, compare MSE estimates
```

5. **Jackknife influence.** Compute the jackknife influence values for the mean of the treatment data. Which observation, when removed, changes the mean the most? Does this make sense given the data?

```
let treatment = [4.2, 5.1, 3.8, 6.3, 4.9, 5.5]
# Your code: jackknife, identify most influential observation
```

Key Takeaways

- The bootstrap estimates the sampling distribution of any statistic by resampling with replacement — no distributional assumptions needed.
- Bootstrap confidence intervals use percentiles of the bootstrap distribution; they work for means, medians, ratios, correlations, or any custom function.
- Permutation tests build a null distribution by shuffling group labels, providing an exact or approximate p-value for group comparisons without distributional assumptions.
- Cross-validation evaluates predictive models honestly by holding out data, preventing the overfitting that plagues high-dimensional genomics.

- The jackknife identifies influential observations and estimates bias.
- Resampling methods complement, rather than replace, parametric methods — when assumptions are met, parametric methods are more efficient; when they are not, resampling provides a robust alternative.
- With very small samples ($n < 5-10$), even resampling methods have limited resolution — there are simply too few unique resamples.

What's Next

Tomorrow we cross the philosophical divide in statistics. Everything so far has been frequentist: probabilities describe long-run frequencies of events.

Bayesian statistics takes a fundamentally different view — probability describes degrees of belief, and we update those beliefs as evidence accumulates. You will learn to combine prior knowledge with observed data using Bayes' theorem, build posterior distributions, and discover why Bayesian thinking is particularly natural for interpreting variants of uncertain significance.

Day 24: Bayesian Thinking for Biologists

Day 24 of 30

Prerequisites: Days 4, 6-7

~60 min reading

Bayesian Inference

The Problem

A clinical sequencing lab has found a missense variant in a patient's BRCA2 gene. The patient has a family history of breast cancer. ClinVar classifies the variant as "Uncertain Significance" — VUS. The clinician needs to make a decision: recommend risk-reducing surgery, or watchful waiting?

You have several pieces of evidence:

1. **Population frequency:** The variant appears in 0.02% of a population database (gnomAD). Pathogenic BRCA2 variants are typically rare, but many rare variants are benign.
2. **Computational prediction:** Three algorithms (SIFT, PolyPhen, CADD) predict the variant is "likely damaging."
3. **Functional assay:** A cell-based splicing assay shows mild disruption.
4. **Family data:** Two of three affected relatives carry the variant (one does not, but that could be a phenocopy).

No single piece of evidence is conclusive. Each is uncertain. But together, they should shift your belief about pathogenicity. How do you combine them?

The frequentist framework has no natural mechanism for combining prior knowledge with new data. The Bayesian framework does — it is literally designed for this. Today, you will learn how to think like a Bayesian, and you will see why this way of reasoning is becoming standard practice in clinical variant classification.

What Is Bayesian Statistics?

Bayesian statistics treats probability as a measure of belief, not a long-run frequency. Instead of asking “what would happen if I repeated this experiment infinitely many times?” it asks “given what I know, how confident should I be?”

The fundamental equation is Bayes’ theorem:

Posterior = (Likelihood x Prior) / Evidence

Or more precisely:

$$P(H \mid D) = P(D \mid H) \times P(H) / P(D)$$

Where:

- **P(H)** = Prior: your belief about hypothesis H before seeing data
- **P(D | H)** = Likelihood: probability of observing data D if H is true
- **P(H | D)** = Posterior: your updated belief after seeing the data
- **P(D)** = Evidence: total probability of the data (a normalizing constant)

The Recipe

1. **Start with a prior:** What do you believe before seeing any data?
2. **Observe data:** Collect evidence.
3. **Compute the likelihood:** How probable is this data under each hypothesis?
4. **Update to get the posterior:** Combine prior and likelihood.

The posterior from one analysis becomes the prior for the next — evidence accumulates naturally.

Key insight: Bayesian inference is sequential updating. Each new piece of evidence shifts your belief. This is exactly how clinical variant classification works: you start with a prior (population frequency suggests most variants

are benign), then update with each line of evidence (computational predictions, functional assays, segregation data).

Frequentist vs Bayesian: The Practical Difference

Consider testing whether a new drug reduces blood pressure.

Frequentist answer: “If the drug had no effect, there is a 2% probability of observing data this extreme. Therefore, $p = 0.02$, and we reject the null.”

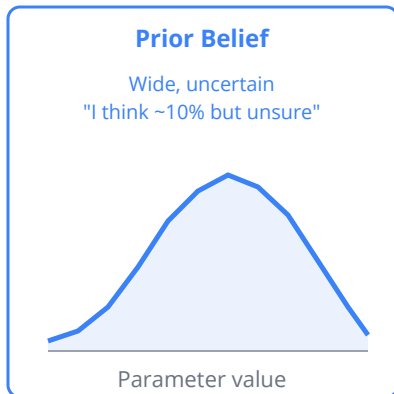
Bayesian answer: “Given the prior evidence and this data, there is a 94% probability that the drug reduces blood pressure, and the most likely reduction is 5 mmHg with a 95% credible interval of [1.2, 8.8] mmHg.”

Aspect	Frequentist	Bayesian
Probability means	Long-run frequency	Degree of belief
Parameters are	Fixed but unknown	Random variables with distributions
Prior knowledge	Not formally incorporated	Explicitly included via priors
Result	p-value, confidence interval	Posterior distribution, credible interval
“95% interval” means	95% of such intervals would contain the true value	95% probability the parameter is in this interval
Multiple comparisons	Requires correction (Day 12)	Naturally skeptical with informative priors
Small samples	Unreliable (CLT breaks down)	Works with proper priors

Common pitfall: A frequentist 95% confidence interval does NOT mean “95% probability the parameter is in this interval.” It means “if we repeated

the experiment many times, 95% of intervals constructed this way would contain the true value." This distinction confuses almost everyone. The Bayesian credible interval actually does mean what most people think a confidence interval means.

Bayesian Updating: Prior + Data = Posterior



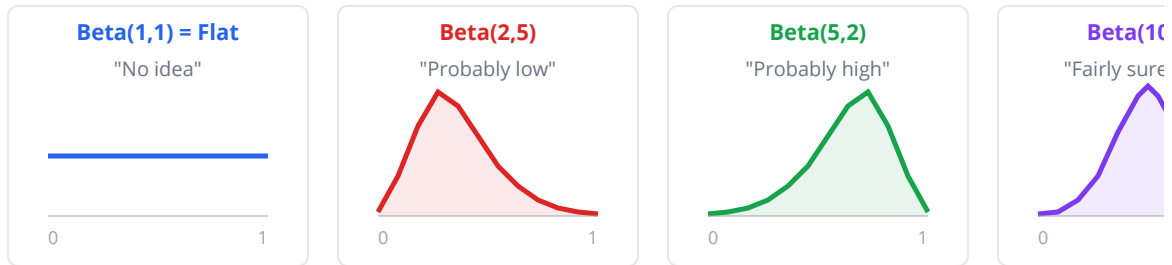
×

Likelihood (Data) Parameter value Data says ~15% 3 carriers in 200 tested

=

Posterior Belief Parameter value Narrower, more certain Compromise: ~13%
Key: The posterior combines prior and data. With more data, the posterior narrows (more certainty) and shifts toward the data. With a flat prior, the posterior equals the likelihood. Posterior = (Prior × Likelihood) / Evidence

Beta Distribution Shapes

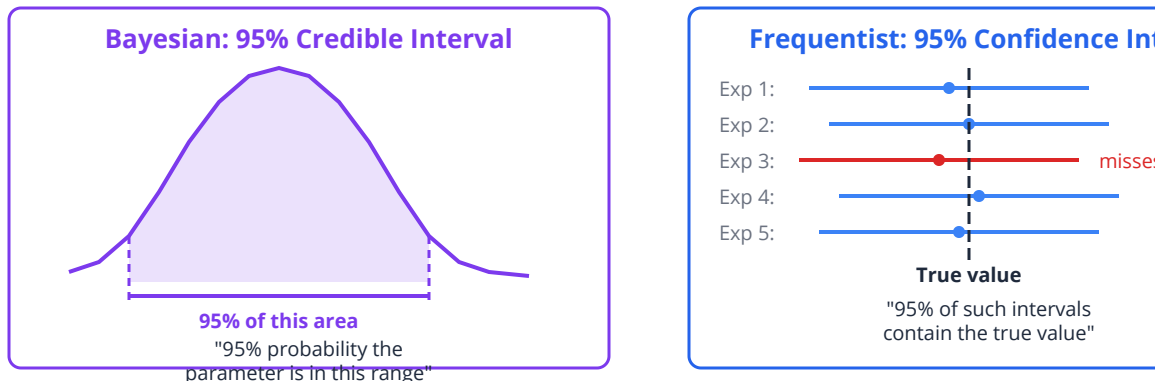


The **Beta(a, b)** distribution lives on [0, 1] and models beliefs about probabilities.

Mean = $a / (a + b)$. Larger $a + b$ = more concentrated (more confident).

These priors express different degrees and directions of prior knowledge before seeing data.

Credible Interval vs Confidence Interval



Bayesian interval has the direct interpretation most people want; the frequentist interval is about the procedure, not the parameter.

The Beta-Binomial Model

The beta-binomial model is the entry point to Bayesian statistics because it is elegant, intuitive, and directly applicable to biology. It applies whenever your data is a count of successes out of trials — exactly the situation with variant frequencies, mutation rates, response rates, and detection probabilities.

The Setup

- **Data:** k successes in n trials (e.g., 3 pathogenic carriers out of 200 individuals tested)
- **Parameter:** θ (the true probability of success)
- **Prior:** $\text{Beta}(\alpha, \beta)$ distribution
- **Posterior:** $\text{Beta}(\alpha + k, \beta + n - k)$

The Beta distribution is “conjugate” to the Binomial — meaning the posterior is the same type of distribution as the prior, just with updated parameters. This makes computation trivial.

Understanding the Beta Distribution

The $\text{Beta}(\alpha, \beta)$ distribution lives on $[0, 1]$ and describes beliefs about probabilities:

Prior	α	β	Interpretation
$\text{Beta}(1, 1)$	1	1	Uniform — “I have no idea, any probability is equally likely”
$\text{Beta}(0.5, 0.5)$	0.5	0.5	Jeffreys’ prior — slightly favors extremes
$\text{Beta}(10, 90)$	10	90	“I believe the probability is around 10%”
$\text{Beta}(1, 99)$	1	99	“I believe the probability is very low (~1%)”
$\text{Beta}(50, 50)$	50	50	“I’m fairly sure it’s around 50%”

The mean of $\text{Beta}(\alpha, \beta)$ is $\alpha / (\alpha + \beta)$. The larger $\alpha + \beta$, the more concentrated (confident) the distribution.

```
# Visualize different Beta priors
# The Beta distribution is not a BioLang builtin, but we can
# compute and visualize it using the formula:
#  $\text{Beta}(x; a, b) \sim x^{(a-1)} * (1-x)^{(b-1)}$ 
let x = seq(0.01, 0.99, 0.01)

# Plot Beta(2,8) – slightly low probability
let beta_28 = x |> map(|v| pow(v, 1) * pow(1 - v, 7))
let total = sum(beta_28) * 0.01
let beta_28_norm = beta_28 |> map(|v| v / total)
let tbl = zip(x, beta_28_norm) |> map(|p| {x: p[0], density: p[1]})
|> to_table()
plot(tbl, {type: "line", x: "x", y: "density", title: "Beta(2,8)
Prior"})
```

Conjugate Updating

The magic of the beta-binomial model: if your prior is $\text{Beta}(\alpha, \beta)$ and you observe k successes in n trials, the posterior is simply:

Posterior = $\text{Beta}(\alpha + k, \beta + n - k)$

No integrals, no MCMC, no approximations. Just add.

```

# Variant frequency estimation
# Prior: Beta(1, 99) – we expect ~1% carrier frequency
# Data: 3 carriers out of 200 tested

let prior_alpha = 1
let prior_beta = 99
let k = 3      # successes (carriers)
let n = 200    # trials (individuals)

let post_alpha = prior_alpha + k
let post_beta = prior_beta + (n - k)

# Posterior mean = alpha / (alpha + beta)
let post_mean = post_alpha / (post_alpha + post_beta)

# Approximate 95% credible interval using normal approximation
let post_var = (post_alpha * post_beta) / (pow(post_alpha +
post_beta, 2) * (post_alpha + post_beta + 1))
let post_sd = sqrt(post_var)
let ci_lower = post_mean - 1.96 * post_sd
let ci_upper = post_mean + 1.96 * post_sd

print("Prior: Beta(" + str(prior_alpha) + ", " + str(prior_beta) +
")")
print("Prior mean: " + str(round(prior_alpha / (prior_alpha +
prior_beta), 4)))
print("Posterior: Beta(" + str(post_alpha) + ", " + str(post_beta) +
")")
print("Posterior mean: " + str(round(post_mean, 4)))
print("95% credible interval: [" +
  str(round(ci_lower, 4)) + ", " +
  str(round(ci_upper, 4)) + "]")

```

Credible Intervals vs Confidence Intervals

A 95% **credible interval** (Bayesian) contains the parameter with 95% probability. Full stop. This is the direct, intuitive interpretation.

A 95% **confidence interval** (frequentist) is a procedure that, if repeated many times, would contain the true parameter 95% of the time. For any specific interval, you cannot say the parameter is “95% likely” to be inside.

In practice, for large samples and weak priors, the two intervals are nearly identical. The difference matters most with small samples or strong priors.

```
# Compare Bayesian credible interval to frequentist CI

# Data: 8 responders out of 20 patients
let k = 8
let n = 20

# Bayesian with flat prior: Beta(1,1) + data = Beta(1+8, 1+12) =
Beta(9, 13)
let a = 1 + k
let b = 1 + (n - k)
let bayes_mean = a / (a + b)
let bayes_var = (a * b) / (pow(a + b, 2) * (a + b + 1))
let bayes_sd = sqrt(bayes_var)
let bayes_lower = bayes_mean - 1.96 * bayes_sd
let bayes_upper = bayes_mean + 1.96 * bayes_sd

# Frequentist (Wilson interval)
let p_hat = k / n
let z = 1.96
let wilson_lower = (p_hat + z*z/(2*n) - z*sqrt(p_hat*(1-p_hat)/n +
z*z/(4*n*n))) / (1 + z*z/n)
let wilson_upper = (p_hat + z*z/(2*n) + z*sqrt(p_hat*(1-p_hat)/n +
z*z/(4*n*n))) / (1 + z*z/n)

print("Bayesian 95% credible interval: [" +
  str(round(bayes_lower, 3)) + ", " +
  str(round(bayes_upper, 3)) + "]")
print("Frequentist 95% Wilson CI: [" +
  str(round(wilson_lower, 3)) + ", " +
  str(round(wilson_upper, 3)) + "]")
```

Bayesian Normal Estimation

For continuous data (not just counts), the Bayesian approach models the unknown mean with a normal prior. If the data is also normal, the posterior for the mean is a normal distribution with:

- **Posterior mean** = weighted average of prior mean and data mean

- **Posterior variance** = combination of prior variance and data variance

The weight given to the data versus the prior depends on sample size. With large n , the data dominates and the prior barely matters.

```
# Bayesian estimation of mean enzyme activity
let data = [4.2, 5.1, 3.8, 6.3, 4.9, 5.5, 4.7, 5.8]

# Prior: mean = 4.0, sd = 2.0 (vague prior based on literature)
let prior_mean = 4.0
let prior_sd = 2.0
let prior_var = pow(prior_sd, 2)

# Data summary
let n = len(data)
let data_mean = mean(data)
let data_var = variance(data)

# Normal-Normal conjugate update
# Posterior precision = prior precision + data precision
let prior_prec = 1.0 / prior_var
let data_prec = n / data_var
let post_prec = prior_prec + data_prec
let post_var = 1.0 / post_prec
let post_sd = sqrt(post_var)

# Posterior mean = weighted average
let post_mean = (prior_prec * prior_mean + data_prec * data_mean) /
post_prec

# 95% credible interval
let ci_lower = post_mean - 1.96 * post_sd
let ci_upper = post_mean + 1.96 * post_sd

print("Prior mean: 4.0, Prior SD: 2.0")
print("Data mean: " + str(round(data_mean, 2)))
print("Posterior mean: " + str(round(post_mean, 3)))
print("Posterior SD: " + str(round(post_sd, 3)))
print("95% credible interval: [" +
      str(round(ci_lower, 3)) + ", " +
      str(round(ci_upper, 3)) + "])")
```

Prior Sensitivity

A critical question: how much does the prior matter? The answer depends on sample size.

```
# Same data, three different priors
let data = [4.2, 5.1, 3.8, 6.3, 4.9, 5.5, 4.7, 5.8]
let n = len(data)
let data_mean = mean(data)
let data_var = variance(data)
let data_prec = n / data_var

# Helper: compute posterior mean for normal-normal conjugate
# post_mean = (prior_prec * prior_mean + data_prec * data_mean) /
# (prior_prec + data_prec)

# Flat (vague) prior: mean=0, sd=100
let flat_prec = 1.0 / pow(100, 2)
let flat_post = (flat_prec * 0 + data_prec * data_mean) / (flat_prec
+ data_prec)

# Informative prior centered on truth: mean=5.0, sd=1.0
let good_prec = 1.0 / pow(1.0, 2)
let good_post = (good_prec * 5.0 + data_prec * data_mean) /
(good_prec + data_prec)

# Informative prior centered far from truth: mean=10.0, sd=1.0
let bad_prec = 1.0 / pow(1.0, 2)
let bad_post = (bad_prec * 10.0 + data_prec * data_mean) / (bad_prec
+ data_prec)

print("Flat prior:          posterior mean = " + str(round(flat_post,
3)))
print("Good prior (5.0):    posterior mean = " + str(round(good_post,
3)))
print("Bad prior (10.0):    posterior mean = " + str(round(bad_post,
3)))
print("Data mean:          " + str(round(data_mean, 3)))
```

With only 8 observations, the informative priors pull the posterior toward them. With 800 observations, even a badly wrong prior would be overwhelmed by the data.

Common pitfall: Using a highly informative prior with little data is dangerous — the prior dominates. Use weakly informative priors (centered on a reasonable value but with wide spread) unless you have strong external evidence to justify a tight prior. A flat prior is always safe but may be less efficient.

Posterior Predictive Distribution

The posterior predictive distribution answers: “Given what I have learned from the data, what do I expect to see in future observations?”

This is enormously practical. After estimating a variant’s pathogenicity probability from a training dataset, you want to predict: if I sequence the next 100 patients, how many carriers will I find?

```
set_seed(42)
# Posterior predictive for variant carrier count
# Prior: Beta(1, 99), Data: 3 carriers out of 200
let post_a = 1 + 3      # = 4
let post_b = 99 + 197   # = 296

# Posterior mean carrier frequency
let post_mean = post_a / (post_a + post_b)
print("Posterior mean frequency: " + str(round(post_mean, 4)))

# Simulate posterior predictive for next 500 individuals
# Draw theta from Beta posterior, then draw count from Binomial(500,
theta)
let n_future = 500
let n_sim = 10000
let predictions = range(0, n_sim) |> map(|i| {
  # Approximate Beta draw using normal approximation
  let theta = post_mean + rnorm(1)[0] * sqrt(post_mean * (1 -
post_mean) / (post_a + post_b))
  let theta_clamp = max(0.001, min(0.999, theta))
  # Simulate binomial count
  let count = range(0, n_future) |> filter(|j| rnorm(1)[0] <
qnorm(theta_clamp)) |> len()
  count
})

let pred_mean = mean(predictions)
let sorted_pred = sort(predictions)
let ci_lo = sorted_pred[round(n_sim * 0.025, 0)]
let ci_hi = sorted_pred[round(n_sim * 0.975, 0)]

print("Predicted carriers in 500 individuals:")
print("  Mean: " + str(round(pred_mean, 1)))
print("  95% prediction interval: [" + str(ci_lo) + ", " +
str(ci_hi) + "]\")

histogram(predictions, {bins: 30,
  title: "Posterior Predictive – Carriers in Next 500",
  xlabel: "Number of Carriers", ylabel: "Frequency"})
```

Sequential Updating: Evidence Accumulates

The most natural aspect of Bayesian analysis is sequential updating. The posterior from one analysis becomes the prior for the next. This mirrors how evidence actually accumulates in science.

```
# Variant pathogenicity: sequential updating with multiple evidence

# Step 1: Start with population base rate
# ~10% of rare missense variants in BRCA2 are pathogenic
let prior_a = 10
let prior_b = 90

print("=== Initial Prior ===")
print("P(pathogenic) ~ " +
      str(round(prior_a / (prior_a + prior_b), 3)))

# Step 2: Update with computational predictions
# 3 of 3 algorithms predict "damaging"
# For pathogenic variants, ~80% get 3/3 damaging
# For benign variants, ~15% get 3/3 damaging
# Likelihood ratio = 0.80 / 0.15 = 5.33
let lr1 = 0.80 / 0.15
let post1_a = prior_a * lr1
let post1_b = prior_b

print("\n=== After Computational Predictions ===")
print("P(pathogenic) ~ " +
      str(round(post1_a / (post1_a + post1_b), 3)))

# Step 3: Update with functional assay
# Mild splicing disruption
# For pathogenic variants: 60% show mild disruption
# For benign variants: 10% show mild disruption
# LR = 6.0
let lr2 = 0.60 / 0.10
let post2_a = post1_a * lr2
let post2_b = post1_b

print("\n=== After Functional Assay ===")
print("P(pathogenic) ~ " +
      str(round(post2_a / (post2_a + post2_b), 3)))

# Step 4: Update with family segregation
# 2 of 3 affected carry the variant
# For pathogenic: ~90% chance of this pattern
# For benign: ~25% chance (random segregation)
# LR = 3.6
let lr3 = 0.90 / 0.25
let post3_a = post2_a * lr3
let post3_b = post2_b

print("\n=== After Family Segregation ===")
```



```
print("P(pathogenic) ~ " +  
      str(round(post3_a / (post3_a + post3_b), 3)))  
  
let final_prob = post3_a / (post3_a + post3_b)  
print("Final P(pathogenic): " + str(round(final_prob, 4)))  
print("Final classification: " +  
      if post3_a / (post3_a + post3_b) > 0.99 { "Pathogenic" }  
      else if post3_a / (post3_a + post3_b) > 0.90 { "Likely Pathogenic"  
    }  
      else if post3_a / (post3_a + post3_b) < 0.10 { "Likely Benign" }  
      else if post3_a / (post3_a + post3_b) < 0.01 { "Benign" }  
      else { "VUS" }  
    )
```

Clinical relevance: The ACMG/AMP variant classification framework (Richards et al., 2015) is implicitly Bayesian. It combines evidence from population data, computational predictions, functional studies, segregation data, and de novo status to classify variants into five tiers (Pathogenic, Likely Pathogenic, VUS, Likely Benign, Benign). Tavtigian et al. (2018) formalized this as an explicit Bayesian framework using likelihood ratios — exactly the approach shown above.

When Bayesian Is Better — and When It Is Overkill

Bayesian excels when:

- **Prior information exists** and should be formally incorporated (variant classification, clinical trials with historical data)
- **Sequential updating** is needed (monitoring a clinical trial as data accumulates)
- **Small samples** make frequentist methods unreliable (the prior stabilizes estimates)
- **You want direct probability statements** (“94% probability the drug works” vs “ $p = 0.02$ ”)

- **Multiple comparisons:** informative priors act as natural regularization, reducing false positives

Frequentist is fine when:

- **No meaningful prior** exists (purely exploratory analysis)
- **Sample is large** (prior is overwhelmed anyway, results converge)
- **Regulatory requirements** demand frequentist analysis (FDA still primarily uses p-values)
- **Simplicity matters** (t-test is faster to explain than posterior distributions)

Bayesian Thinking in BioLang

```
set_seed(42)
# =====
# Complete Bayesian workflow: drug response rate
# =====

# Prior: previous Phase II trial found 30% response rate in 40
patients
let prior_alpha = 12    # 30% of 40 = 12 responders
let prior_beta = 28     # 70% of 40 = 28 non-responders

# New data: Phase III trial, 45 responders out of 120 patients
let k = 45
let n = 120

# 1. Bayesian update (Beta-Binomial conjugate)
let post_a = prior_alpha + k
let post_b = prior_beta + (n - k)
let post_mean = post_a / (post_a + post_b)
let post_var = (post_a * post_b) / (pow(post_a + post_b, 2) *
(post_a + post_b + 1))
let post_sd = sqrt(post_var)
let ci_lower = post_mean - 1.96 * post_sd
let ci_upper = post_mean + 1.96 * post_sd

print("=== Drug Response Rate Estimation ===")
print("Prior (Phase II): " + str(round(prior_alpha / (prior_alpha +
prior_beta), 3)))
print("Data (Phase III): " + str(round(k / n, 3)))
print("Posterior mean: " + str(round(post_mean, 3)))
print("95% credible interval: [" +
  str(round(ci_lower, 3)) + ", " +
  str(round(ci_upper, 3)) + "]")

# 2. Visualize prior and posterior as Beta density curves
let xs = seq(0.1, 0.6, 0.005)
let prior_curve = xs |> map(|x| {
  x: x,
  density: pow(x, prior_alpha - 1) * pow(1 - x, prior_beta - 1),
  series: "Prior"
})
let post_curve = xs |> map(|x| {
  x: x,
  density: pow(x, post_a - 1) * pow(1 - x, post_b - 1),
  series: "Posterior"
})
```

```

}))
let curves = concat(prior_curve, post_curve) |> to_table()
plot(curves, {type: "line", x: "x", y: "density", color: "series",
  title: "Prior vs Posterior"})

# 3. Probability response rate exceeds 30%
# Use normal approximation to Beta CDF
let z_30 = (0.30 - post_mean) / post_sd
let p_above_30 = 1.0 - pnorm(z_30)
print("\nP(response rate > 30%): " + str(round(p_above_30, 3)))

# 4. Probability response rate exceeds standard-of-care (25%)
let z_25 = (0.25 - post_mean) / post_sd
let p_above_soc = 1.0 - pnorm(z_25)
print("P(response rate > 25% standard-of-care): " +
  str(round(p_above_soc, 3)))

# 5. Posterior predictive: next 200 patients
# Expected responders = n_future * posterior_mean
let n_future = 200
let pred_mean = n_future * post_mean
let pred_sd = sqrt(n_future * post_mean * (1 - post_mean))
print("\nPredicted responders in next 200 patients:")
print("  Mean: " + str(round(pred_mean, 1)))
print("  95% PI: [" + str(round(pred_mean - 1.96 * pred_sd, 0)) +
  ", " + str(round(pred_mean + 1.96 * pred_sd, 0)) + "]")

# 6. Compare flat vs informative prior
let flat_a = 1 + k
let flat_b = 1 + (n - k)
let flat_mean = flat_a / (flat_a + flat_b)

print("\n=== Prior Sensitivity ===")
print("Informative prior -> posterior mean: " + str(round(post_mean,
  3)))
print("Flat prior -> posterior mean: " + str(round(flat_mean, 3)))
print("Difference: " + str(round(abs(post_mean - flat_mean), 4)))

```

Python:

```

from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

# Beta-binomial
prior_a, prior_b = 12, 28
k, n = 45, 120
post_a, post_b = prior_a + k, prior_b + n - k

x = np.linspace(0, 1, 1000)
plt.plot(x, stats.beta.pdf(x, prior_a, prior_b), label='Prior')
plt.plot(x, stats.beta.pdf(x, post_a, post_b), label='Posterior')
plt.legend()
plt.show()

# Credible interval
ci = stats.beta.interval(0.95, post_a, post_b)
print(f"95% credible interval: [{ci[0]:.3f}, {ci[1]:.3f}]")

# P(theta > 0.30)
p_above = 1 - stats.beta.cdf(0.30, post_a, post_b)

```

R:

```

# Beta-binomial
prior_a <- 12; prior_b <- 28
k <- 45; n <- 120
post_a <- prior_a + k; post_b <- prior_b + n - k

x <- seq(0, 1, length.out = 1000)
plot(x, dbeta(x, prior_a, prior_b), type="l", col="blue",
     ylab="Density")
lines(x, dbeta(x, post_a, post_b), col="red")
legend("topright", c("Prior", "Posterior"), col=c("blue","red"),
     lty=1)

# Credible interval
qbeta(c(0.025, 0.975), post_a, post_b)

# P(theta > 0.30)
1 - pbeta(0.30, post_a, post_b)

```

Exercises

1. **Flat vs informative prior.** A variant has been seen in 2 out of 500 individuals. Compute the posterior for the population frequency using (a) a flat prior $\text{Beta}(1,1)$, (b) an informative prior $\text{Beta}(1,999)$ reflecting the belief that pathogenic variants are very rare. How different are the posteriors? At what sample size would the difference become negligible?

```
# Your code: two priors, compare posteriors
# Increase n and see when they converge
```

2. **Drug response updating.** Start with a $\text{Beta}(5, 15)$ prior (25% response rate from a pilot). You enroll patients one at a time: responder, non-responder, non-responder, responder, responder, non-responder, responder, non-responder, non-responder, responder. After each patient, compute and print the posterior mean and 95% credible interval. Watch the uncertainty shrink.

```
# Your code: sequential update, one patient at a time
```

3. **Variant classification.** A VUS has prior odds of pathogenicity = 0.10 (10%). Apply three independent evidence lines with likelihood ratios 4.0, 2.5, and 6.0. What is the final posterior probability of pathogenicity? Would this variant be classified as “Likely Pathogenic” (>0.90)?

```
# Your code: sequential likelihood ratio updating
```

4. **Posterior predictive check.** Estimate a mutation rate from data (15 mutations in 1,000,000 bases). Then simulate 100 future datasets of 1,000,000 bases from the posterior predictive distribution. What range of mutation counts do you expect?

```
# Your code: bayes_binomial + posterior_predictive
```

5. **Prior sensitivity analysis.** For the mutation rate problem (15/1,000,000), compute posterior means and 95% CIs using five different priors:

Beta(1,1), Beta(0.5,0.5), Beta(1,99999), Beta(15,999985), and Beta(100,6666567). Plot all five posteriors on the same axes. Which priors lead to meaningfully different conclusions?

Your code: five priors, compare posteriors

Key Takeaways

- Bayesian inference updates prior beliefs with observed data to produce posterior distributions: $\text{Posterior} = \text{Prior} \times \text{Likelihood} / \text{Evidence}$.
- The beta-binomial model is the workhorse for proportions: if the prior is Beta(a, b) and you observe k successes in n trials, the posterior is Beta(a+k, b+n-k).
- Credible intervals have the direct interpretation people usually want: “95% probability the parameter is in this range.”
- Sequential updating is natural in Bayesian inference — the posterior from one study becomes the prior for the next, exactly mirroring how scientific evidence accumulates.
- Clinical variant classification (ACMG/AMP) is implicitly Bayesian: prior odds from population data are updated with likelihood ratios from computational, functional, and segregation evidence.
- With large samples, the prior barely matters (the data dominates). With small samples, the prior can strongly influence results — choose it carefully and report sensitivity analyses.
- Bayesian methods complement frequentist methods; neither is universally superior.

What’s Next

We have learned to test hypotheses, build models, reduce dimensions, cluster data, resample, and reason Bayesianly. But all of these methods ultimately produce results that must be communicated — and the primary language of scientific communication is visual. Tomorrow, we tackle statistical visualization:

which plot for which data, how to make plots that tell the truth, and how to avoid the visualization sins that plague the literature.

Day 25: Statistical Visualization — Plots That Tell the Truth

Day 25 of 30 Prerequisites: All previous days ~60 min reading
Data Visualization

The Problem

You have spent weeks analyzing a clinical dataset. The results are solid: a survival benefit, a clear dose-response relationship, differentially expressed genes with strong effect sizes. You write up the manuscript, generate figures, and submit.

Three weeks later, the reviewer's comments arrive. "Figure 2: bar charts with error bars hide the data distribution. Replace with violin plots or beeswarm plots showing individual data points. Figure 4: the y-axis does not start at zero, exaggerating the effect. Figure 6: red-green color scheme is inaccessible to the 8% of males with color vision deficiency. Major revision."

The statistics were correct. The visualization was not. And in modern publishing, visualization is not decoration — it is evidence. A misleading plot can sink an otherwise excellent paper. A well-designed figure can convey complex results in seconds.

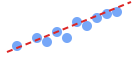
Today is your visualization reference guide. We will cover every major plot type you have encountered in this book, when to use each, how to read them, common mistakes, and how to produce publication-ready versions in BioLang.

The “Which Plot?” Decision Guide

Choosing the right plot starts with two questions: What type of data do you have? What relationship are you showing?

Data situation	Recommended plot(s)
Distribution of one continuous variable	histogram , density_plot
Compare distributions across groups	boxplot , violin_plot
Two continuous variables	scatter
Trend over time or ordered variable	line_plot
Counts or proportions by category	bar_chart
Matrix of values (e.g., expression)	heatmap
Differential expression (fold change + significance)	volcano
Genome-wide association results	manhattan_plot
Observed vs expected p-values	qq_plot
Meta-analysis effect sizes	forest_plot
Method agreement	bland_altman
Publication bias assessment	funnel_plot
Survival curves	kaplan_meier_plot
Classifier performance	roc_curve
PCA scores + loadings	pca_biplot

"Which Plot?" Decision Guide

Data Situation	Recommended Plot	Visual Examp
One continuous variable	Histogram / Density	
Compare groups (continuous)	Boxplot / Violin	
Two continuous variables	Scatter Plot	
Trend over time / ordered	Line Plot	
Counts by category	Bar Chart	
Expression matrix	Heatmap	
Fold change + significance	Volcano Plot	

Also: Manhattan plot (GWAS), QQ plot (p-value QC), Forest plot (meta-analysis), ROC curve (classifier), Kaplan-Me

Histogram

When: Visualize the shape of a single continuous distribution.

How to read: Each bar represents a bin; the height shows how many observations fall in that range. Look for: center (peak), spread (width), shape (symmetric? skewed?), modes (one peak or multiple?), outliers (isolated bars far from the center).

Mistakes: Too few bins (hides structure), too many bins (shows noise), not specifying bin count (default may be misleading).

```
set_seed(42)
# Quality score distribution
let scores = concat(rnorm(5000, 35, 8), rnorm(1000, 20, 3))

histogram(scores, {bins: 50,
  title: "Sequencing Quality Score Distribution",
  xlabel: "Phred Quality Score",
  ylabel: "Count"})
```

Publication tip: Always state the bin width or number of bins in the figure legend. Use 30-50 bins for most datasets.

Density Plot

When: Smooth estimate of the probability distribution. Better than histograms for comparing multiple groups on the same axes.

How to read: The curve shows estimated probability density. The area under the curve equals 1. Higher curves mean more observations at that value.

```
set_seed(42)
# Compare two treatment groups
let group_a = rnorm(200, 50, 12)
let group_b = rnorm(200, 62, 15)

density(group_a, {title: "Expression Level Distribution by Group",
  xlabel: "Expression (TPM)", label: "Control"})
density(group_b, {label: "Treatment"})
```

Publication tip: Use semi-transparent fills when overlapping multiple densities. State the bandwidth if non-default.

Box Plot

When: Compare distributions across groups. The standard for multi-group comparisons in biology.

How to read: The box spans Q1 to Q3 (the IQR, containing the middle 50%). The line inside is the median. Whiskers extend to the most extreme point within 1.5 x IQR. Points beyond whiskers are outliers.

Mistakes: Bar charts with error bars hide distribution shape — use box plots instead. Forgetting to show sample size.

```
set_seed(42)
let control = rnorm(30, 5.0, 1.2)
let low_dose = rnorm(30, 6.5, 1.5)
let high_dose = rnorm(30, 8.2, 1.8)

let bp_table = table({"Control": control, "Low Dose": low_dose,
"High Dose": high_dose})
boxplot(bp_table, {title: "Tumor Response by Treatment Arm"})
```

Publication tip: Always overlay individual data points (jittered) on box plots when $n < 50$. Report n per group in the axis label or legend.

Violin Plot

When: Like a box plot but shows the full distribution shape. Best for revealing multimodality that box plots miss.

How to read: The width of the “violin” at any value shows the density of observations there. A violin with two bulges indicates a bimodal distribution.

```
let violin_data = table({
  "Control": control, "Low Dose": low_dose, "High Dose": high_dose
})
violin(violin_data,
  {
    title: "Response Distribution – Full Shape",
    ylabel: "Response Score"})
```

Publication tip: Include a miniature box plot inside the violin for reference. Violin plots are increasingly preferred by reviewers over bar charts.

Scatter Plot

When: Show the relationship between two continuous variables. The most versatile plot in statistics.

How to read: Each point is one observation. Look for: direction (positive/negative), form (linear/curved), strength (tight/scattered), outliers (isolated points), clusters.

```
set_seed(42)
let gene_expr = rnorm(100, 50, 15)
let drug_response = gene_expr |> map(|x| -0.8 * x + 90 + rnorm(1, 0, 8)[0])

scatter(gene_expr, drug_response)
```

Publication tip: Add a trend line with confidence band for linear relationships. Use transparency (alpha) when points overlap. Report the correlation coefficient and p-value in the figure legend or panel.

Line Plot

When: Data has a natural order (time series, dose-response, ordered categories).

How to read: The line connects sequential observations. Look for trends, cycles, sudden changes.

Mistakes: Connecting unrelated categorical data with lines (implies ordering that does not exist).

```
let days = seq(0, 28, 7)
let tumor_vol_drug = [450, 380, 290, 210, 175]
let tumor_vol_ctrl = [450, 510, 580, 650, 720]

# Build a table with both series for plotting
let drug_rows = zip(days, tumor_vol_drug) |> map(|p| {day: p[0],
volume: p[1], group: "Drug X"})
let ctrl_rows = zip(days, tumor_vol_ctrl) |> map(|p| {day: p[0],
volume: p[1], group: "Control"})
let tbl = concat(drug_rows, ctrl_rows) |> to_table()

plot(tbl, {type: "line", x: "day", y: "volume", color: "group",
  xlabel: "Days Since Randomization",
  ylabel: "Tumor Volume (mm^3)",
  title: "Tumor Growth Over Time"})
```

Bar Chart

When: Compare counts or proportions across categories. NOT for continuous distributions (use box/violin plots).

How to read: Bar height represents the value. Bars should start at zero.

Mistakes: Using bar charts for continuous data (hides distribution). Not starting at zero (exaggerates differences). Using 3D bars (distorts perception).

```
let categories = ["Complete Response", "Partial Response", "Stable
Disease", "Progressive"]
let counts = [18, 35, 28, 19]

let response_counts = {
  Complete_Response: 18,
  Partial_Response: 35,
  Stable_Disease: 28,
  Progressive: 19
}
bar_chart(response_counts, {title: "RECIST Response Categories"})
```

Common pitfall: Bar charts with error bars (dynamite plots) are the most criticized visualization in biostatistics. They show only the mean and one measure of spread, hiding the actual data distribution. A sample with a bimodal distribution and a sample with a normal distribution can produce identical bar charts. Always prefer box plots, violin plots, or strip charts for continuous data.

Heatmap

When: Visualize a matrix of values (gene expression across samples, correlation matrices, p-value matrices).

How to read: Color intensity represents the value at each cell. Rows and columns are often clustered to group similar patterns.

```
# Expression heatmap of top 50 DE genes
let top_50_expression = matrix([
  [2.1, 2.4, 8.2, 8.5],
  [7.8, 8.1, 3.0, 2.7],
  [4.2, 4.0, 6.5, 6.8],
  [9.0, 8.7, 2.2, 2.5]
])
heatmap(top_50_expression,
  {cluster_rows: true,
   cluster_cols: true,
   color_scale: "red_blue",
   title: "Top 50 Differentially Expressed Genes"})
```

Publication tip: State the color scale, clustering method, and distance metric in the legend. Use diverging color scales (red-blue) for data centered on zero; sequential scales (white-red) for non-negative data.

Volcano Plot

When: Differential expression analysis — simultaneously display fold change (effect size) and statistical significance.

How to read: x-axis is log2 fold change, y-axis is $-\log_{10}(\text{p-value})$. Points in the upper corners are genes with large, significant changes. Upper-left: significantly downregulated. Upper-right: significantly upregulated. Bottom center: not significant.

```
let de_results = table([
  {gene: "TP53", log2fc: 2.3, pvalue: 0.0001},
  {gene: "BRCA1", log2fc: -1.7, pvalue: 0.002},
  {gene: "GAPDH", log2fc: 0.1, pvalue: 0.72}
])
volcano(de_results,
  {fc_threshold: 1.0,      # log2 FC cutoff
   p_threshold: 0.05,     # adjusted p-value cutoff
   title: "Tumor vs Normal – Differential Expression",
   xlabel: "log2 Fold Change",
   ylabel: "-log10(adjusted p-value)"})
```

Publication tip: Use adjusted p-values (FDR), not raw p-values. Label the most significant genes. Include dashed lines at your FC and p-value thresholds.

Manhattan Plot

When: Genome-wide association results — display p-values for hundreds of thousands of SNPs across chromosomes.

How to read: x-axis is genomic position (chromosomes colored alternately), y-axis is $-\log_{10}(\text{p-value})$. Peaks above the genome-wide significance line ($p < 5 \times 10^{-8}$) are associated loci.

```
let gwas_results = table([
  {chrom: "1", pos: 11856378, pvalue: 2e-9},
  {chrom: "7", pos: 55259515, pvalue: 4e-6},
  {chrom: "12", pos: 25245350, pvalue: 0.03},
  {chrom: "17", pos: 43093449, pvalue: 8e-10}
])
manhattan(gwas_results,
  {threshold: 5e-8,
   title: "GWAS – Type 2 Diabetes"})
```

Publication tip: Use alternating colors for chromosomes. Draw both the genome-wide significance line (5×10^{-8}) and the suggestive line (1×10^{-5}). Label top hits with the nearest gene name.

Q-Q Plot

When: Check whether observed p-values follow the expected uniform distribution under the null. Essential for GWAS quality control.

How to read: Points should follow the diagonal if there is no systematic inflation. Deviation at the upper end indicates true signal. Deviation along the entire line indicates systematic inflation (population structure, technical artifacts).

```
let p_values = [0.000001, 0.0002, 0.003, 0.02, 0.08, 0.15, 0.31,
0.54, 0.77, 0.93]
qq_plot(p_values,
  {title: "Q-Q Plot – Observed vs Expected p-values",
   ci: true})
```

Publication tip: Report the genomic inflation factor (λ) in the plot or legend. λ close to 1.0 is good; $\lambda > 1.1$ suggests confounding.

Forest Plot

When: Meta-analysis or subgroup analysis — display effect estimates and confidence intervals from multiple studies or subgroups.

How to read: Each row is a study/subgroup. The square is the point estimate (size proportional to weight), horizontal line is the CI. The diamond at the bottom is the pooled estimate. If the CI crosses the null (vertical line at 0 or 1), the result is not statistically significant.

```
let meta_tbl = [  
  {study: "Smith 2019", estimate: 0.72, ci_lower: 0.55, ci_upper:  
0.93, weight: 25},  
  {study: "Jones 2020", estimate: 0.85, ci_lower: 0.70, ci_upper:  
1.03, weight: 30},  
  {study: "Lee 2021", estimate: 0.68, ci_lower: 0.48, ci_upper:  
0.96, weight: 15},  
  {study: "Garcia 2022", estimate: 0.91, ci_lower: 0.75, ci_upper:  
1.10, weight: 30},  
  {study: "Pooled", estimate: 0.78, ci_lower: 0.68, ci_upper: 0.89,  
weight: 100}  
] |> to_table()  
  
forest_plot(meta_tbl,  
  {null_value: 1.0,  
  title: "Meta-Analysis – Hazard Ratios for Overall Survival",  
  xlabel: "Hazard Ratio (95% CI)"})
```

Bland-Altman Plot

When: Assess agreement between two measurement methods. NOT the same as correlation — two methods can be highly correlated but systematically biased.

How to read: x-axis is the mean of two measurements, y-axis is their difference. Points should scatter randomly around zero (no bias). The limits of agreement (mean difference ± 1.96 SD) show the expected range of disagreement.

```
let method_a = [5.2, 6.1, 4.8, 7.3, 5.5, 6.8, 4.2, 5.9, 7.1, 6.3]
let method_b = [5.0, 6.3, 4.5, 7.0, 5.8, 6.5, 4.4, 5.7, 7.4, 6.1]

# Bland-Altman: plot difference vs mean
let means = zip(method_a, method_b) |> map(|p| (p[0] + p[1]) / 2)
let diffs = zip(method_a, method_b) |> map(|p| p[0] - p[1])
let mean_diff = mean(diffs)
let sd_diff = stdev(diffs)

scatter(means, diffs)
print("Mean difference: " + str(round(mean_diff, 3)))
print("Limits of agreement: [" +
  str(round(mean_diff - 1.96 * sd_diff, 3)) + ", " +
  str(round(mean_diff + 1.96 * sd_diff, 3)) + "])")
```

Funnel Plot

When: Assess publication bias in meta-analysis. Small studies with extreme results may indicate selective publishing.

How to read: x-axis is effect size, y-axis is study precision (1/SE or sample size). Large, precise studies cluster near the true effect (top). Small, imprecise studies scatter widely (bottom). Asymmetry suggests publication bias — if small negative studies are missing, the funnel is lopsided.

```
# Funnel plot: scatter of effect size vs precision
let effect_sizes = [0.22, 0.35, 0.18, 0.41, 0.29]
let standard_errors = [0.08, 0.12, 0.06, 0.15, 0.09]
scatter(effect_sizes, standard_errors)
```

Kaplan-Meier Plot

When: Survival analysis — display probability of survival over time, comparing treatment arms.

How to read: The curve starts at 1.0 (all alive) and drops with each event. Censored observations (patients lost to follow-up) are marked with tick marks. The median survival is where the curve crosses 0.5.

```
# Build a Kaplan-Meier survival table and plot
let time = [6, 8, 12, 15, 20, 24, 30, 36, 42, 48]
let event = [1, 1, 1, 1, 1, 1, 0, 0, 0, 0]
let group = ["Drug", "Control", "Drug", "Control", "Drug",
             "Control", "Drug", "Control", "Drug", "Control"]
let km_tbl = range(0, len(time)) |> map(|i| {
  time: time[i], event: event[i], group: group[i]
}) |> to_table()

kaplan_meier(km_tbl, {title: "Overall Survival - Drug X vs Standard
of Care"})
```

Publication tip: Include a risk table below the plot showing the number at risk at regular intervals. Report the log-rank p-value and hazard ratio with CI.

ROC Curve

When: Evaluate a binary classifier — plot sensitivity (true positive rate) against 1-specificity (false positive rate) at all thresholds.

How to read: The curve bows toward the upper-left corner for good classifiers. The AUC (area under the curve) summarizes performance: 0.5 = random guessing, 1.0 = perfect classification. AUC > 0.8 is generally considered good.

```
let roc_tbl = table([
  {score: 0.95, label: 1}, {score: 0.82, label: 1},
  {score: 0.71, label: 0}, {score: 0.60, label: 1},
  {score: 0.42, label: 0}, {score: 0.18, label: 0}
])
roc_curve(roc_tbl,
  {title: "ROC - Gene Expression Classifier for Response",
   auc_label: true})
```

PCA Biplot

When: Display PCA scores (samples) and loadings (variables) simultaneously.

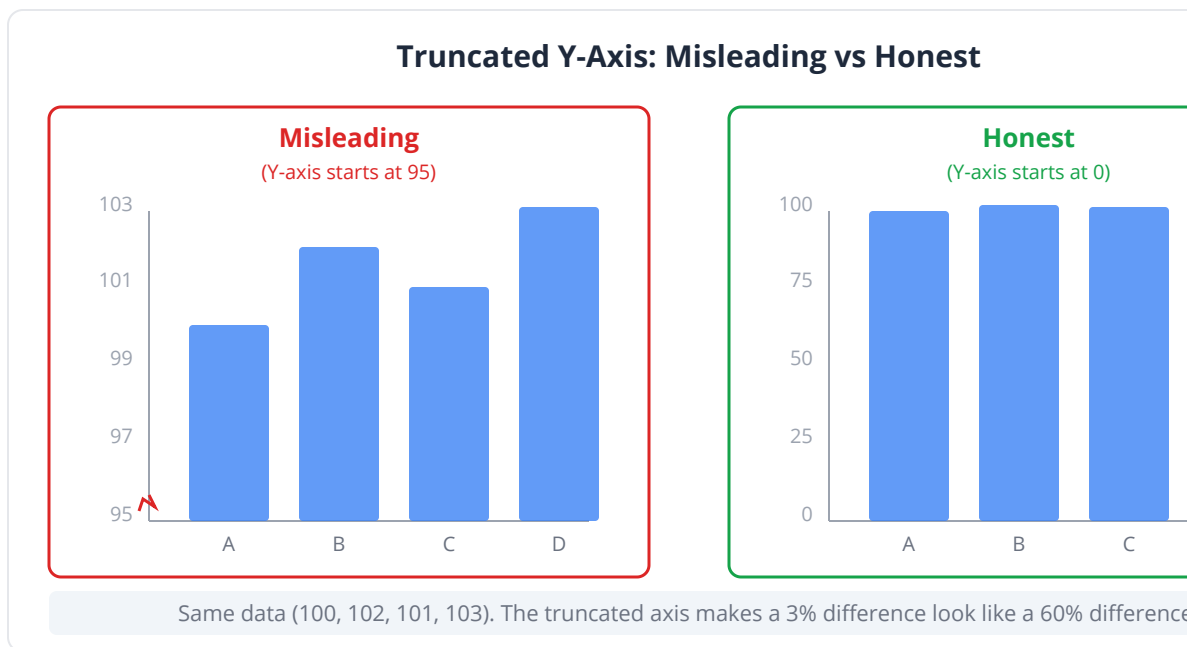
How to read: Points are samples — clusters indicate groups. Arrows are variable loadings — direction and length show each variable's contribution to the PCs.

```
let result = pca(expression_matrix)
pca_plot(result,
  {title: "PCA Biplot – Gene Expression",
   xlabel: "PC1 (" + str(round(result.variance_explained[0] * 100,
1)) + "%)",
   ylabel: "PC2 (" + str(round(result.variance_explained[1] * 100,
1)) + "%)"})
```

The Grammar of Honest Visualization

Rule 1: Start axes at zero (usually)

Truncating the y-axis can make a 2% difference look like a 200% difference. Always start bar charts at zero. For scatter plots and line plots, starting at zero is less critical but should be considered.



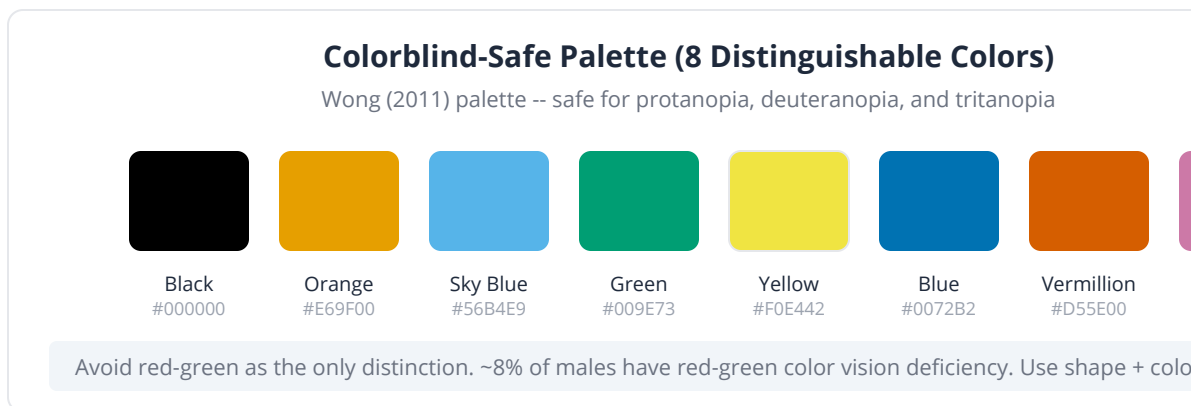
Rule 2: No 3D charts

3D bar charts, 3D pie charts, and 3D scatter plots distort perception. The human visual system poorly judges depth in 2D projections. Use 2D alternatives always.

Rule 3: Use colorblind-safe palettes

Approximately 8% of males and 0.5% of females have red-green color vision deficiency. Never use red-green as the only distinguishing feature. Safe alternatives:

Palette	Colors
Blue-orange	Good contrast, colorblind safe
Viridis	Perceptually uniform, prints well in grayscale
Blue-red diverging	Good for centered data (use with colorblind check)
Categorical (Set2)	Up to 8 distinguishable, colorblind safe



Rule 4: Show the data

Whenever possible, show individual data points. Summary statistics (means, medians) are important but insufficient. A bimodal distribution and a unimodal distribution can have identical means and standard deviations.

Rule 5: Label everything

Every figure needs: title, axis labels with units, legend if multiple groups, sample sizes, and any statistical annotations.

Visualization Sins That Lie

The truncated axis

A bar chart showing revenue of \$100M vs \$102M with the y-axis starting at \$99M makes a 2% difference look enormous. In biology, this commonly appears in gene expression plots where the y-axis starts at a non-zero value.

The dual y-axis

Two different scales on the same plot can make unrelated trends appear correlated. By choosing the scales appropriately, you can make any two lines appear to track each other.

The cherry-picked time window

Showing survival curves from month 6 to month 24 (where the drug looks good) but omitting month 0 to 6 (where there is no difference) or month 24 to 48 (where the effect fades) is misleading.

The pie chart

Pie charts are universally criticized by statisticians. Humans are poor at judging angles and areas. A bar chart conveys the same information more accurately.

Key insight: The goal of visualization is not to make your results look impressive — it is to accurately represent the data so that the reader can draw correct conclusions. A figure that misleads, even unintentionally, undermines the entire paper.

Exercises

1. **Box vs violin vs bar.** Generate three datasets: (a) normal, (b) bimodal, (c) heavily skewed. For each, create a bar chart with error bars, a box plot with points, and a violin plot. Which plot type reveals the distribution shape most honestly?

```
set_seed(42)
let normal = rnorm(100, 50, 10)
let bimodal = concat(rnorm(50, 30, 5), rnorm(50, 70, 5))
let skewed = rnorm(100, 0, 1) |> map(|x| abs(x) * 10)
# Your code: create all three plot types for each dataset
```

2. **Publication figure.** Take the clinical trial data from Day 17 (survival analysis) and create a publication-ready Kaplan-Meier plot with: risk table, log-rank p-value, hazard ratio annotation, median survival lines, and proper axis labels. Save as SVG.

```
# Your code: kaplan_meier_plot with all publication elements
```

3. **Volcano and heatmap.** From the DE analysis on Day 12, create (a) a volcano plot with the top 10 genes labeled and threshold lines, and (b) a heatmap of the top 50 DE genes clustered by sample and gene.

```
# Your code: volcano + heatmap, publication quality
```

4. **Color blindness check.** Create a scatter plot with 4 groups using (a) a red-green palette and (b) a colorblind-safe palette. Describe why the first is problematic.

```
# Your code: two versions of the same scatter, different palettes
```

5. **Visualization critique.** The following figure specifications describe common visualization sins. For each, explain the problem and generate the correct version:

- (a) Bar chart of gene expression levels with y-axis from 95 to 105
- (b) 3D pie chart of mutation types
- (c) Scatter plot with no axis labels

```
# Your code: create the corrected versions
```

Key Takeaways

- Choose the plot type based on your data type and the relationship you want to show — the decision guide above covers the most common situations.
- Box plots and violin plots show distribution shape; bar charts with error bars do not — prefer the former for continuous data.
- Volcano plots combine fold change and significance for differential expression; Manhattan plots display genome-wide results by chromosomal position.
- Forest plots are the standard for meta-analysis and subgroup results; Bland-Altman plots assess method agreement; funnel plots check for publication bias.
- Honest visualization requires: axes starting at zero (for bar charts), no 3D effects, colorblind-safe palettes, individual data points when feasible, and complete labeling.
- Every publication figure should include: descriptive title, axis labels with units, legend, sample sizes, statistical annotations (p-values, effect sizes), and a mention of the color scale for heatmaps.
- The goal of visualization is truth, not beauty. A technically impressive plot that misleads is worse than a simple plot that communicates honestly.

What's Next

Individual studies provide individual estimates. But what happens when five different labs study the same question and get five different answers?

Tomorrow, we tackle meta-analysis — the formal framework for combining results across studies to get a single, more precise estimate. You will build forest plots, assess heterogeneity, check for publication bias, and learn when pooling studies is appropriate and when it is dangerously misleading.

Day 26: Meta-Analysis — Combining Studies

Day 26 of 30 Prerequisites: Days 6-8, 14, 19 ~60 min reading
Evidence Synthesis

The Problem

Three independent clinical trials have tested PCSK9 inhibitors for lowering LDL cholesterol in patients with familial hypercholesterolemia:

Study	N	Mean LDL Reduction (mg/dL)	SE
Trial A (Europe, 2019)	250	-52.3	4.1
Trial B (USA, 2020)	180	-48.7	5.3
Trial C (Asia, 2021)	320	-55.1	3.8

Each study alone has a confidence interval that overlaps with the others. None is definitive. But together, the evidence is overwhelming. The question is: how do you formally combine them? You cannot just average the means — the studies have different sample sizes, different precisions, and were conducted in different populations. You need a method that respects these differences.

Meta-analysis is that method. It provides a rigorous framework for pooling results across studies, weighting each by its precision, quantifying heterogeneity, and assessing whether the pooled estimate is trustworthy. It sits at the top of the evidence hierarchy — above individual RCTs — because it synthesizes all available evidence.

What Is Meta-Analysis?

Meta-analysis is the statistical combination of results from two or more separate studies to produce a single, more precise estimate of an effect. It is not simply “averaging” — it is a weighted combination that accounts for each study’s precision.

Think of it as a vote among experts. If three experts estimate a quantity, you would give more weight to the expert who measured most precisely (smallest uncertainty), and less weight to the expert whose estimate is vague. Meta-analysis formalizes this intuition.

Key insight: Meta-analysis does not combine raw data — it combines summary statistics (effect sizes and their standard errors). This means you can conduct a meta-analysis using published results alone, without accessing any original data.

Why Combine Studies?

1. **Increased precision:** Pooling 750 patients across three trials gives a tighter CI than any single trial.
2. **Resolving contradictions:** If Study A finds an effect and Study B does not, meta-analysis can determine whether this reflects true heterogeneity or sampling variability.
3. **Generalizability:** Studies from Europe, USA, and Asia together provide evidence across populations.
4. **Detecting small effects:** An individual study may be underpowered; the pooled analysis may cross the significance threshold.

Fixed-Effects Model

The fixed-effects model assumes that all studies estimate the **same true effect**. Differences between study results are due to sampling variability alone.

Weighting

Each study is weighted by the inverse of its variance:

$$w_i = 1 / SE_i^2$$

The pooled estimate is the weighted mean:

$$\text{Pooled} = \text{Sum}(w_i \times \text{estimate}_i) / \text{Sum}(w_i)$$

The pooled SE is:

$$SE_{\text{pooled}} = 1 / \sqrt{\text{Sum}(w_i)}$$

When to Use Fixed Effects

Use fixed effects when you believe the true effect is the same across all studies — for instance, highly standardized lab assays or studies using identical protocols. In practice, this assumption is often too strong.

```
# Fixed-effects meta-analysis
let studies = ["Trial A", "Trial B", "Trial C"]
let effects = [-52.3, -48.7, -55.1]
let se = [4.1, 5.3, 3.8]

# Manual calculation
let weights = se |> map(|s| 1.0 / (s * s))
let total_weight = sum(weights)
let pooled_effect = sum(zip(weights, effects) |> map(|we| we[0] *
we[1])) / total_weight
let pooled_se = 1.0 / sqrt(total_weight)

print("=== Fixed-Effects Meta-Analysis ===")
print("Study weights: " + str(weights |> map(|w| round(w, 2))))
print("Pooled effect: " + str(round(pooled_effect, 2)) + " mg/dL")
print("Pooled SE: " + str(round(pooled_se, 2)))
print("95% CI: [" +
  str(round(pooled_effect - 1.96 * pooled_se, 2)) + ", " +
  str(round(pooled_effect + 1.96 * pooled_se, 2)) + "])")
```

Random-Effects Model

The random-effects model assumes that studies estimate **different but related true effects**. Each study's true effect is drawn from a distribution of effects. The between-study variance (tau-squared) captures how much the true effects vary.

When to Use Random Effects

Use random effects when studies differ in population, dosing, protocol, or outcome definition — which is almost always in biomedical research. Random effects produces wider CIs than fixed effects, reflecting the additional uncertainty from between-study variability.

Model	Assumption	CIs	When to use
Fixed-effects	Same true effect across studies	Narrower	Identical protocols, homogeneous studies
Random-effects	True effects vary across studies	Wider	Different populations, protocols, settings

Common pitfall: Some researchers choose between fixed and random effects based on which gives a smaller p-value. This is a form of p-hacking. Choose the model before seeing the results, based on the study designs and populations.

Heterogeneity: Q and I-Squared

Heterogeneity quantifies how much the studies disagree beyond what sampling variability would explain.

Cochran's Q Statistic

$$Q = \sum (w_i \times (\text{estimate}_i - \text{pooled})^2)$$

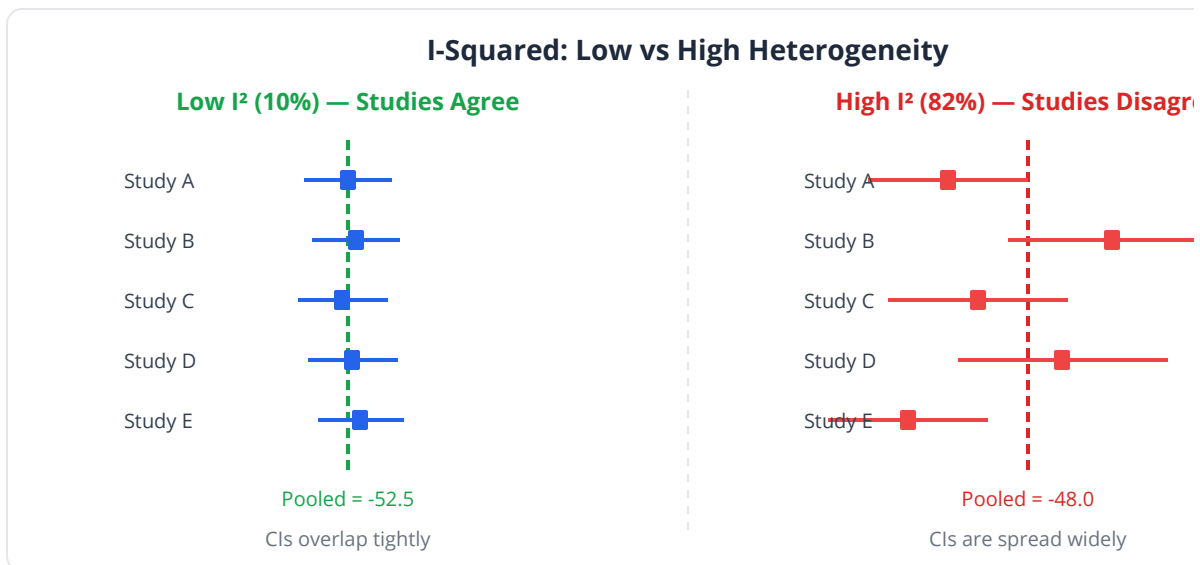
Under the null hypothesis of no heterogeneity, Q follows a chi-square distribution with k-1 degrees of freedom (where k is the number of studies). A significant Q ($p < 0.10$, using a lenient threshold because the test has low power) suggests heterogeneity.

I-Squared

I-squared quantifies the proportion of total variation due to between-study heterogeneity:

$$I^2 = \max(0, (Q - df) / Q) \times 100\%$$

I^2	Heterogeneity
0-25%	Low — studies are consistent
25-50%	Moderate — some inconsistency
50-75%	Substantial — investigate sources
75-100%	Considerable — pooling may be inappropriate

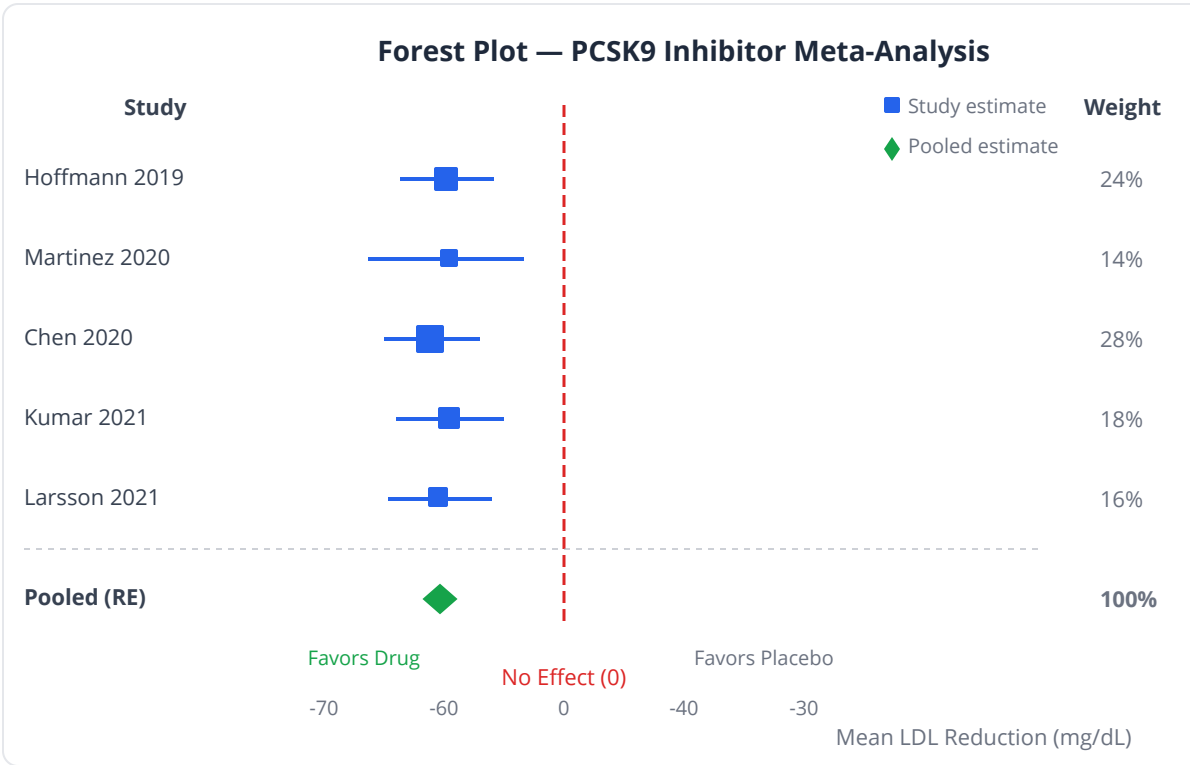


```
# Heterogeneity assessment
let Q = sum(zip(weights, effects) |> map(|we| we[0] * (we[1] -
pooled_effect) * (we[1] - pooled_effect)))
let df = len(studies) - 1
let I_squared = max(0, (Q - df) / Q) * 100

print("=== Heterogeneity ===")
print("Q statistic: " + str(round(Q, 2)) + " (df=" + str(df) + ")")
print("I-squared: " + str(round(I_squared, 1)) + "%")
```

The Forest Plot

The forest plot is the signature visualization of meta-analysis. Each study is a row showing its point estimate (square, sized by weight) and confidence interval (horizontal line). The pooled estimate is a diamond at the bottom. A vertical line at the null (0 for mean differences, 1 for ratios) allows quick assessment of significance.



```
let studies = ["Trial A (2019)", "Trial B (2020)", "Trial C (2021)",
              "Chen (2020)", "Kumar (2021)", "Pooled"]
let effects = [-52.3, -48.7, -55.1, -50.2, -53.8, -52.5]
let ci_lower = [-60.3, -59.1, -62.5, -57.4, -61.2, -55.8]
let ci_upper = [-44.3, -38.3, -47.7, -43.0, -46.4, -49.2]
let weights = [24, 14, 28, 18, 16, 100]

let forest_tbl = range(0, len(studies)) |> map(|i| {
  study: studies[i], estimate: effects[i], ci_lower: ci_lower[i],
  ci_upper: ci_upper[i], weight: weights[i]
}) |> to_table()

forest_plot(forest_tbl,
  {null_value: 0,
  title: "PCSK9 Inhibitor – LDL Reduction (mg/dL)",
  xlabel: "Mean LDL Reduction (95% CI)"}))
```

Reading the forest plot:

- If a study's CI does not cross the null line, that study alone is significant.
- If the pooled diamond does not cross the null, the combined evidence is significant.
- Study squares vary in size — larger squares mean more weight (more precise studies).
- The diamond width shows the CI of the pooled estimate.

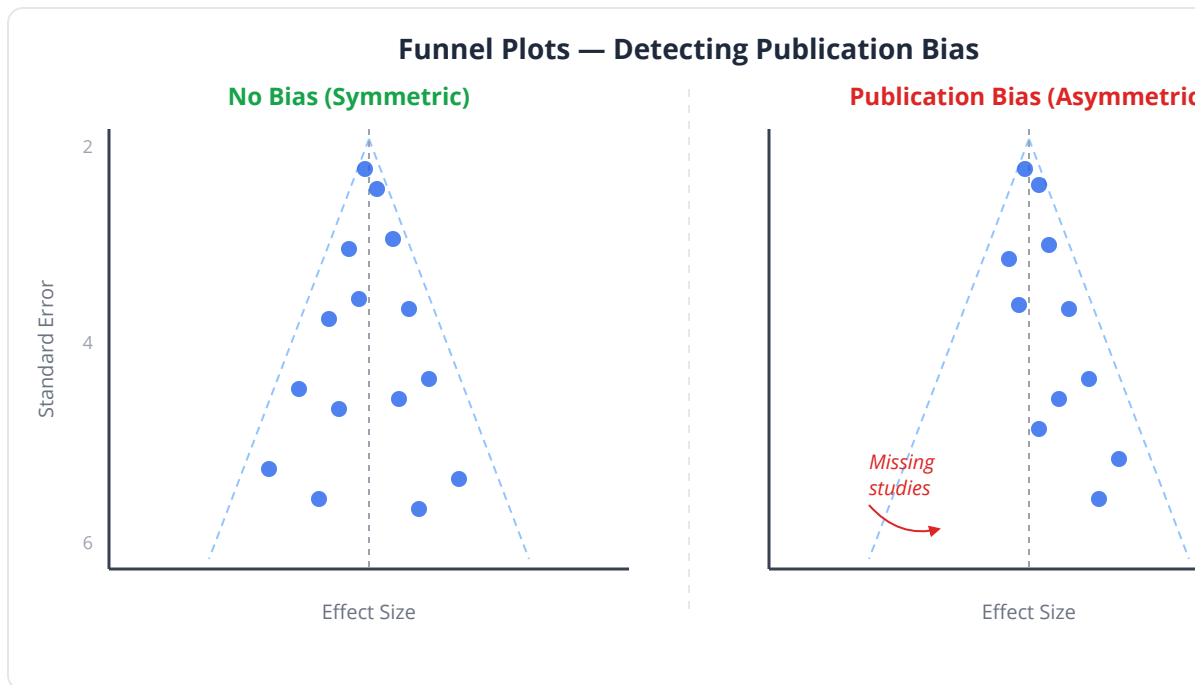
Publication Bias and the Funnel Plot

Publication bias occurs when studies with significant results are more likely to be published than studies with null results. This biases meta-analyses toward overestimating effects.

The funnel plot detects this. It plots each study's effect size (x-axis) against its precision (y-axis, typically $1/SE$ or sample size). In the absence of bias, the plot should look like an inverted funnel — symmetric around the pooled estimate, with more scatter at the bottom (less precise studies).

Asymmetry suggests bias. If small studies with negative or null results are missing (they were not published), the funnel will be asymmetric — missing

studies from the lower-left.



```
let effect_sizes = [-52.3, -48.7, -55.1, -50.2, -53.8]
let standard_errors = [4.1, 5.3, 3.8, 3.7, 3.9]
```

```
# Funnel plot: scatter of effect size vs SE
scatter(effect_sizes, standard_errors)
```

Clinical relevance: Publication bias is a serious concern in pharmaceutical research. A meta-analysis of published antidepressant trials found a pooled effect size of 0.37 (moderate). When unpublished trials obtained through FDA records were included, the effect dropped to 0.15 (small). Publication bias had inflated the apparent efficacy by more than double.

When Meta-Analysis Is Inappropriate

Meta-analysis is not appropriate when:

1. **Studies measure fundamentally different things:** Combining a study of aspirin with a study of statins because both are “cardiovascular interventions” is meaningless.
2. **Heterogeneity is too high** ($I^2 > 75\%$): If studies genuinely disagree, pooling them hides important differences. Investigate subgroups instead.
3. **Studies are not independent:** If three papers report on overlapping patient cohorts, they are not independent studies.
4. **Publication bias is severe:** A pooled estimate from biased studies is itself biased — garbage in, garbage out.
5. **Too few studies:** Meta-analysis of 2 studies with opposite results tells you very little. At minimum, 3-5 studies are needed.

Common pitfall: “Combining apples and oranges” is the classic criticism. Meta-analysis is appropriate when studies address the same question with similar methods. If studies differ fundamentally, no amount of statistical sophistication makes the pooled result meaningful.

Meta-Analysis in BioLang — Complete Pipeline

```
# =====
# Five studies of PCSK9 inhibitor effect on LDL
# =====

let studies = ["Hoffmann 2019", "Martinez 2020", "Chen 2020",
               "Kumar 2021", "Larsson 2021"]
let effects = [-52.3, -48.7, -55.1, -50.2, -53.8]
let se = [4.1, 5.3, 3.8, 3.7, 3.9]
let n_patients = [250, 180, 320, 290, 260]

# =====
# 1. Fixed-effects meta-analysis
# =====
let weights_fe = se |> map(|s| 1.0 / (s * s))
let total_w = sum(weights_fe)
let pooled_fe = sum(zip(weights_fe, effects) |> map(|p| p[0] *
p[1])) / total_w
let se_fe = 1.0 / sqrt(total_w)

print("=== Fixed-Effects ===")
print("Pooled: " + str(round(pooled_fe, 2)) + " [" +
      str(round(pooled_fe - 1.96 * se_fe, 2)) + ", " +
      str(round(pooled_fe + 1.96 * se_fe, 2)) + "])")

# =====
# 2. Heterogeneity
# =====
let Q = sum(zip(weights_fe, effects) |>
  map(|p| p[0] * (p[1] - pooled_fe) * (p[1] - pooled_fe)))
let df = len(studies) - 1
let I_sq = max(0, (Q - df) / Q) * 100

print("\n=== Heterogeneity ===")
print("Q = " + str(round(Q, 2)) + ", df = " + str(df))
print("I-squared = " + str(round(I_sq, 1)) + "%")

# Estimate tau-squared (between-study variance)
let C = total_w - sum(weights_fe |> map(|w| w * w)) / total_w
let tau_sq = max(0, (Q - df) / C)
print("tau-squared = " + str(round(tau_sq, 2)))

# =====
# 3. Random-effects meta-analysis
```



```

# =====
let weights_re = se |> map(|s| 1.0 / (s * s + tau_sq))
let total_w_re = sum(weights_re)
let pooled_re = sum(zip(weights_re, effects) |> map(|p| p[0] *
p[1])) / total_w_re
let se_re = 1.0 / sqrt(total_w_re)

print("\n=== Random-Effects ===")
print("Pooled: " + str(round(pooled_re, 2)) + " [" +
  str(round(pooled_re - 1.96 * se_re, 2)) + ", " +
  str(round(pooled_re + 1.96 * se_re, 2)) + "]")

# =====
# 4. Forest plot
# =====
let all_ci_lo = range(0, len(effects)) |> map(|i| effects[i] - 1.96
* se[i])
let all_ci_hi = range(0, len(effects)) |> map(|i| effects[i] + 1.96
* se[i])
let all_w_pct = weights_re |> map(|w| round(w / total_w_re * 100,
1))

# Build forest plot table
let rows = range(0, len(studies)) |> map(|i| {
  study: studies[i], estimate: effects[i],
  ci_lower: all_ci_lo[i], ci_upper: all_ci_hi[i], weight:
all_w_pct[i]
})
let pooled_row = [{study: "Pooled (RE)", estimate: pooled_re,
  ci_lower: pooled_re - 1.96 * se_re,
  ci_upper: pooled_re + 1.96 * se_re, weight: 100}]
let forest_tbl = concat(rows, pooled_row) |> to_table()

forest_plot(forest_tbl,
  {null_value: 0,
  title: "PCSK9 Inhibitor Meta-Analysis – LDL Reduction",
  xlabel: "Mean LDL Reduction, mg/dL (95% CI)"})

# =====
# 5. Funnel plot
# =====
# Funnel plot: scatter of effect vs SE
scatter(effects, se)

# =====
# 6. Study-level summary
# =====
print("\n=== Study Summary ===")

```

```

print("Study          | Effect   | SE    | Weight(RE)")
print("-----|-----|-----|-----")
for i in 0..len(studies) {
  let w_pct = round(weights_re[i] / total_w_re * 100, 1)
  print(studies[i] + " | " + str(effects[i]) + " | " +
    str(se[i]) + " | " + str(w_pct) + "%")
}

# =====
# 7. Interpretation
# =====
print("\n=== Interpretation ===")
if I_sq < 25 {
  print("Heterogeneity is low (I^2 = " + str(round(I_sq, 1)) +
    "%).")
  print("Studies are consistent. Fixed and random effects agree.")
} else if I_sq < 50 {
  print("Moderate heterogeneity (I^2 = " + str(round(I_sq, 1)) +
    "%).")
  print("Random-effects model is preferred.")
} else {
  print("Substantial heterogeneity (I^2 = " + str(round(I_sq, 1)) +
    "%).")
  print("Investigate sources of heterogeneity before trusting the
pooled estimate.")
}

```

Python:

```

import numpy as np
import matplotlib.pyplot as plt

effects = np.array([-52.3, -48.7, -55.1, -50.2, -53.8])
se = np.array([4.1, 5.3, 3.8, 3.7, 3.9])

# Fixed effects
w = 1 / se**2
pooled_fe = np.average(effects, weights=w)
se_fe = 1 / np.sqrt(w.sum())

# Heterogeneity
Q = np.sum(w * (effects - pooled_fe)**2)
df = len(effects) - 1
I2 = max(0, (Q - df) / Q) * 100

# Random effects (DerSimonian-Laird)
C = w.sum() - (w**2).sum() / w.sum()
tau2 = max(0, (Q - df) / C)
w_re = 1 / (se**2 + tau2)
pooled_re = np.average(effects, weights=w_re)
se_re = 1 / np.sqrt(w_re.sum())

print(f"Fixed: {pooled_fe:.1f} [{pooled_fe-1.96*se_fe:.1f},
{pooled_fe+1.96*se_fe:.1f}]")
print(f"Random: {pooled_re:.1f} [{pooled_re-1.96*se_re:.1f},
{pooled_re+1.96*se_re:.1f}]")
print(f"I2: {I2:.1f}%")

```

R:

```
library(meta)

m <- metagen(TE = c(-52.3, -48.7, -55.1, -50.2, -53.8),
              seTE = c(4.1, 5.3, 3.8, 3.7, 3.9),
              studlab = c("Hoffmann", "Martinez", "Chen", "Kumar",
                          "Larsson"),
              sm = "MD")

summary(m)
forest(m)
funnel(m)

# Alternative with metafor
library(metafor)
res <- rma(yi = effects, sei = se, method = "DL")
summary(res)
forest(res)
funnel(res)
```

Exercises

1. **Fixed vs random.** Given the five PCSK9 studies above, compute both fixed-effects and random-effects pooled estimates. How different are they? Based on I-squared, which model is more appropriate?

```
# Your code: both models, compare, interpret I-squared
```

2. **Adding a contradictory study.** A sixth study (Nakamura 2022, N=150) finds a much smaller effect: -30.5 mg/dL, SE=6.2. Add it to the meta-analysis. How do the pooled estimate, CI width, and I-squared change? Create the updated forest plot.

```
# Your code: add study, re-run meta-analysis, compare
```

3. **Publication bias simulation.** Simulate 20 studies: true effect = -50, SE drawn from Uniform(3, 8). Then “suppress” all studies with $p > 0.05$ (simulating publication bias). Run meta-analysis on the remaining studies. Is the pooled estimate biased? Check with a funnel plot.

Your code: simulate, suppress, meta-analyze, funnel plot

4. **Subgroup analysis.** The five studies come from different continents (Europe, USA, Asia). Compute pooled estimates separately for Western (Trials A, B) and Asian (Trials C, D, E) studies. Is there a meaningful difference?

Your code: subgroup meta-analysis, compare pooled estimates

5. **Hazard ratio meta-analysis.** Five survival studies report hazard ratios (log scale) for a new chemotherapy vs standard-of-care. Combine them using random effects and create a forest plot.

```
let log_hr = [-0.33, -0.22, -0.41, -0.28, -0.35]
let se_log_hr = [0.12, 0.15, 0.10, 0.11, 0.13]
# Your code: meta-analysis on log(HR), forest plot, back-transform
to HR
```

Key Takeaways

- Meta-analysis formally combines results across studies to produce a more precise pooled estimate, weighted by each study's precision.
- Fixed-effects models assume one true effect across studies; random-effects models allow the true effect to vary. Random effects is almost always more appropriate in biomedical research.
- Cochran's Q tests for heterogeneity; I-squared quantifies its magnitude. I-squared above 50% warrants investigation before pooling.
- The forest plot is the standard meta-analysis visualization: study estimates with CIs arranged vertically, pooled estimate as a diamond.
- The funnel plot assesses publication bias: asymmetry suggests that small negative studies are missing from the literature.
- Meta-analysis is inappropriate when studies measure different things, heterogeneity is extreme, studies are not independent, or publication bias is severe.

- Meta-analysis sits at the top of the evidence hierarchy because it synthesizes all available evidence — but it is only as good as the studies it includes.

What's Next

We have learned to analyze data, but can we trust our analysis? Tomorrow we confront the reproducibility crisis head-on: how to structure your statistical analysis so that it can be re-run perfectly by anyone, at any time. Random seeds, modular scripts, parameter files, and version tracking — the practices that separate publishable science from a pile of scattered scripts.

Day 27: Reproducible Statistical Analysis

Day 27 of 30

Prerequisites: All previous days

~55 min reading

Best Practices

The Problem

It is 11 PM on a Thursday. Your collaborator emails: “The sequencing core re-processed 3 samples with updated base-calling. Can you re-run the entire analysis with the updated data? The manuscript revision is due Monday.”

You open your analysis folder. There are 14 scripts with names like `analysis_v2_final_FINAL.bl`, `test_new.bl`, and `run_this_one.bl`. You cannot remember which scripts to run in which order. One script hardcodes a file path that no longer exists. Another uses a random seed that you never recorded, so bootstrap confidence intervals will not match the figures in the manuscript. A third script produces slightly different p-values depending on whether you run it before or after another script, because they share a global variable.

This is not a hypothetical scenario. It is the daily reality of computational biology. And it is entirely preventable. Reproducible analysis is not about perfection — it is about structure, documentation, and discipline. Today, you will learn the practices that make “re-run everything” a one-command operation instead of a week of panic.

Why Reproducibility Matters

On Day 1, we discussed the reproducibility crisis: 89% of landmark cancer biology studies could not be replicated. Computational analyses are

theoretically easier to reproduce than wet-lab experiments — you have all the inputs and instructions. Yet in practice, computational reproducibility is shockingly rare.

A 2019 study attempted to reproduce analyses from 204 published bioinformatics papers. Only 14% could be reproduced from the provided code and data. The failures were rarely due to errors in logic — they were due to missing files, undocumented dependencies, hardcoded paths, unrecorded random seeds, and ambiguous analysis steps.

Journals increasingly require:

- **Code availability:** deposit analysis scripts in a public repository
- **Data availability:** raw data in GEO, SRA, or similar
- **Computational environment:** specify software versions
- **Reproducibility statement:** confirm that provided code reproduces all figures

Key insight: Reproducibility is not just about other people reproducing your work. It is about future-you reproducing your work. The person most likely to need to re-run your analysis is yourself, six months later, when a reviewer asks for a revision.

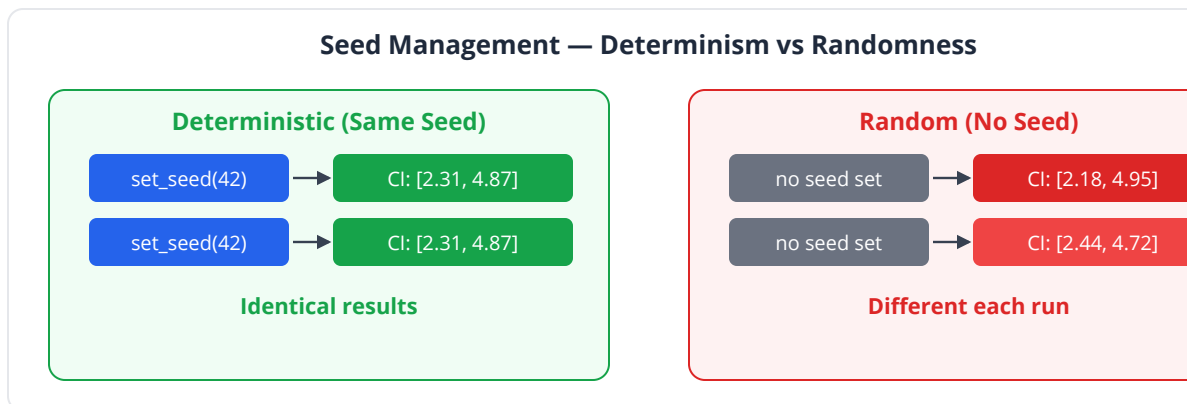
Random Seeds: The Foundation of Reproducibility

Any analysis involving randomness — bootstrap, permutation tests, cross-validation, simulation, stochastic algorithms — will produce different results each run unless you fix the random seed.

Setting Seeds in BioLang

```
set_seed(42)
# ALWAYS set a seed at the start of any analysis with randomness
let data = [4.8, 5.1, 5.3, 5.9, 6.2, 6.4, 6.8, 7.1, 7.4, 7.9]
# These produce identical results every time.
# Bootstrap the median
let n_boot = 1000
let boot_medians = range(0, n_boot) |> map(|i| {
  let resampled = range(0, len(data)) |> map(|j| data[random_int(0,
len(data) - 1)])
  median(resampled)
})
```

Seed Best Practices



Practice	Why
Set seed at the top of every script	Ensures full reproducibility
Use a fixed, documented number	Avoid <code>set_seed(current_time())</code>
Record the seed in your results	Others can verify
Use different seeds for sensitivity	Check that conclusions do not depend on one seed

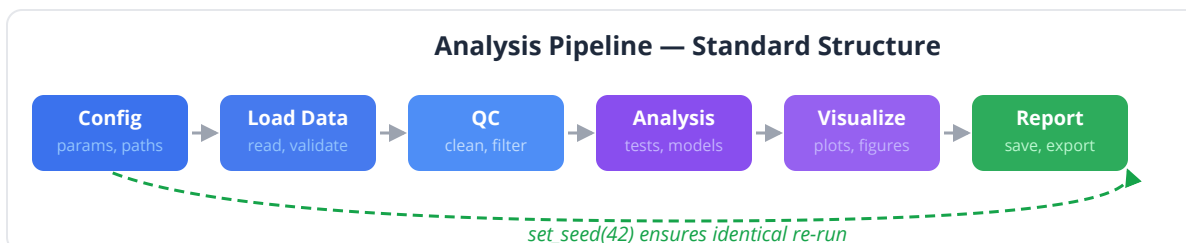
```
set_seed(42)
# Sensitivity check: run with multiple seeds
let seeds = [42, 123, 456, 789, 2024]

for s in seeds {
  set_seed(s)
  let boot_vals = range(0, 1000) |> map(|i| {
    let resampled = range(0, len(data)) |> map(|j|
data[random_int(0, len(data) - 1)])
    median(resampled)
  })
  let sorted_b = sort(boot_vals)
  let ci_lo = sorted_b[25]
  let ci_hi = sorted_b[974]
  print("Seed " + str(s) + ": CI = [" +
    str(round(ci_lo, 3)) + ", " +
    str(round(ci_hi, 3)) + "]"")
}
```

Common pitfall: Setting the seed inside a loop resets the random state on every iteration, which can create subtle correlations. Set it once at the top of the script, or set it deliberately when you need a specific, documented behavior.

Script Structure: The Analysis Pipeline

Every analysis script should follow a predictable structure:



-
- | | |
|------------------|--------------------------------------|
| 1. Configuration | – parameters, paths, thresholds |
| 2. Data loading | – read files, validate inputs |
| 3. Preprocessing | – cleaning, normalization, filtering |
| 4. Analysis | – statistical tests, models |
| 5. Results | – tables, summaries |
| 6. Visualization | – publication figures |
| 7. Output | – save results, figures, reports |

Example: Well-Structured Analysis

```
# Conceptual or diagnostic example; not directly executable.
# =====
# Differential Expression Analysis
# Author: Your Name
# Date: 2025-03-15
# Input: counts_matrix.csv, sample_info.csv
# Output: de_results.csv, volcano.svg, heatmap.svg
# =====

# --- 1. Configuration ---
set_seed(42)

let CONFIG = {
  input_counts: "data/counts_matrix.csv",
  input_samples: "data/sample_info.csv",
  output_dir: "results/",
  fc_threshold: 1.0,          # log2 fold change
  fdr_threshold: 0.05,
  n_top_genes: 50,           # for heatmap
  n_bootstrap: 10000
}

# --- 2. Data Loading ---
let counts = read_csv(CONFIG.input_counts)
let samples = read_csv(CONFIG.input_samples)

print("Loaded " + str(nrow(counts)) + " genes x " +
      str(ncol(counts)) + " samples")
print("Groups: " + str(unique(samples.group)))

# --- 3. Preprocessing ---
# Filter low-expression genes (at least 10 counts in 3+ samples)
let keep = counts |> filter_rows(|row| count_if(row, |x| x >= 10) >=
3)
print("Genes after filtering: " + str(nrow(keep)))

# Normalize: log2(CPM + 1)
let lib_sizes = col_sums(keep)
let cpm = keep |> map_cells(|x, col| x / lib_sizes[col] * 1e6)
let log_cpm = cpm |> map_cells(|x, _| log2(x + 1))

# --- 4. Analysis ---
let tumor_idx = samples |> which(|s| s.group == "tumor")
let normal_idx = samples |> which(|s| s.group == "normal")
```

```

let de_results = []
for gene in row_names(log_cpm) {
  let tumor_vals = log_cpm[gene] |> select(tumor_idx)
  let normal_vals = log_cpm[gene] |> select(normal_idx)
  let tt = ttest(tumor_vals, normal_vals)
  let fc = mean(tumor_vals) - mean(normal_vals)
  de_results = de_results + [{
    gene: gene,
    log2fc: fc,
    p_value: tt.p_value,
    mean_tumor: mean(tumor_vals),
    mean_normal: mean(normal_vals)
  }]
}

# Multiple testing correction
let p_vals = de_results |> map(|r| r.p_value)
let adj_p = p_adjust(p_vals, "BH")
for i in 0..len(de_results) {
  de_results[i].adj_p = adj_p[i]
}

# --- 5. Results ---
let sig_genes = de_results
  |> filter(|r| r.adj_p < CONFIG.fdr_threshold && abs(r.log2fc) >
CONFIG.fc_threshold)
  |> sort_by(|r| r.adj_p)

print("\nSignificant DE genes: " + str(len(sig_genes)))
print("  Upregulated: " + str(count_if(sig_genes, |g| g.log2fc >
0)))
print("  Downregulated: " + str(count_if(sig_genes, |g| g.log2fc <
0)))

# --- 6. Visualization ---
let de_tbl = de_results |> to_table()
volcano(de_tbl,
  {fc_threshold: CONFIG.fc_threshold,
   p_threshold: CONFIG.fdr_threshold,
   title: "Tumor vs Normal – Differential Expression"})

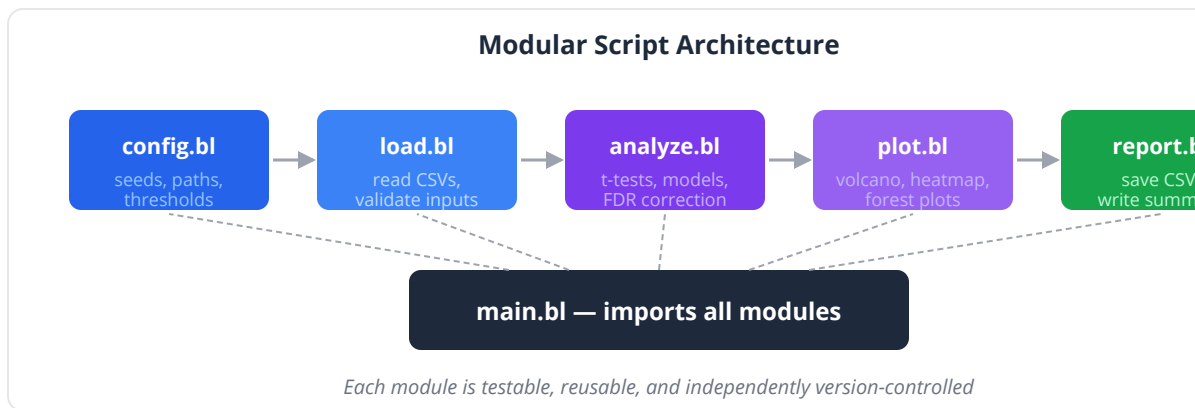
let top_genes = sig_genes |> take(CONFIG.n_top_genes) |> map(|g|
g.gene)
heatmap(log_cpm |> select_rows(top_genes),
  {cluster_rows: true, cluster_cols: true,
   title: "Top " + str(CONFIG.n_top_genes) + " DE Genes"})

```

```
# --- 7. Output ---  
write_csv(de_results, CONFIG.output_dir + "de_results.csv")  
write_csv(sig_genes, CONFIG.output_dir + "sig_genes.csv")  
print("\nResults saved to " + CONFIG.output_dir)
```

Modular Functions

As analyses grow complex, extract repeated logic into functions. This avoids copy-paste errors and makes the analysis self-documenting.



```

# Conceptual or diagnostic example; not directly executable.
# --- Helper functions ---

fn normalize_cpm(counts) {
  let lib_sizes = col_sums(counts)
  counts |> map_cells(|x, col| log2(x / lib_sizes[col] * 1e6 + 1))
}

fn run_de(expr, group_a_idx, group_b_idx) {
  let results = []
  for gene in row_names(expr) {
    let a_vals = expr[gene] |> select(group_a_idx)
    let b_vals = expr[gene] |> select(group_b_idx)
    let tt = ttest(a_vals, b_vals)
    results = results + [{
      gene: gene,
      log2fc: mean(a_vals) - mean(b_vals),
      p_value: tt.p_value
    }]
  }
  let adj = p_adjust(results |> map(|r| r.p_value), "BH")
  for i in 0..len(results) { results[i].adj_p = adj[i] }
  results
}

fn filter_significant(results, fc_cut, fdr_cut) {
  results |> filter(|r| r.adj_p < fdr_cut && abs(r.log2fc) > fc_cut)
}

# --- Main analysis (now concise and readable) ---
set_seed(42)
let expr = read_csv("data/counts.csv") |> normalize_cpm()
let de = run_de(expr, tumor_idx, normal_idx)
let sig = filter_significant(de, 1.0, 0.05)
print("Significant genes: " + str(len(sig)))

```

Parameter Files

Hardcoding thresholds in scripts is brittle. Extract parameters into a configuration file that lives alongside the analysis.

config.yaml

```
# Analysis parameters – Differential Expression
# Change these and re-run to explore sensitivity

random_seed: 42
input:
  counts: "data/counts_matrix.csv"
  samples: "data/sample_info.csv"
analysis:
  fc_threshold: 1.0
  fdr_threshold: 0.05
  min_counts: 10
  min_samples: 3
  n_bootstrap: 10000
output:
  dir: "results/"
  n_top_genes: 50
```

Loading Parameters in BioLang

```
# Load configuration
let config = read_yaml("config.yaml")
set_seed(config.random_seed)

let counts = read_csv(config.input.counts)
let sig = de_results |> filter(|r|
  r.adj_p < config.analysis.fdr_threshold &&
  abs(r.log2fc) > config.analysis.fc_threshold
)
```

Benefits of Parameter Files

Benefit	Explanation
Sensitivity analysis	Change one number, re-run everything
Documentation	All assumptions in one place
Collaboration	Collaborator adjusts parameters without editing code
Reproducibility	Record exact parameters used
Version control	git diff shows exactly what changed

Literate Analysis

Literate analysis interleaves code, results, and narrative explanation in a single document. The document is both the analysis and its documentation.

```
# === Section 1: Quality Control ===  
# We first check for outliers using PCA on the full expression  
matrix.  
# Any sample > 3 SD from the centroid will be flagged.  
  
let pca_result = pca(log_cpm)  
scatter(pca_result.scores[0], pca_result.scores[1])  
  
# Result: Sample S14 is a clear outlier on PC1 (3.8 SD from  
centroid).  
# Decision: Remove S14 from downstream analysis.  
# Justification: S14 had the lowest library size (2.1M reads vs  
# median 15.3M) and highest duplication rate (82%).
```

Key insight: Every decision in an analysis (removing a sample, choosing a threshold, selecting a normalization method) should be documented with a justification. Six months later, neither you nor a reviewer will remember why you chose $FDR < 0.05$ instead of 0.01.

Version Tracking

Track your analysis with version control (git). This provides:

1. **History:** See exactly what changed between runs
2. **Rollback:** Undo mistakes by reverting to a previous version
3. **Collaboration:** Multiple people can work on the same analysis
4. **Provenance:** Link each figure in the manuscript to the exact code that generated it

Minimum Version Control Workflow

```
project/
  config.yaml          # Parameters
  analysis.bl          # Main analysis script
  helpers.bl           # Reusable functions
  data/                # Input data (track metadata, not raw data)
    README.md          # Data provenance and download instructions
  results/             # Output (may or may not track)
    de_results.csv
    figures/
  .gitignore           # Exclude large data files
```

Key Rules

Rule	Why
Commit before and after major changes	Creates a clean timeline
Never commit large data files	Use .gitignore; document download instructions
Tag releases	<code>git tag v1.0-submission</code> marks the manuscript version
Write meaningful commit messages	“Fix FDR threshold” > “update”
Track your config file	Most important file to version

Putting It All Together: Restructuring an Analysis

Let us take a messy analysis from earlier chapters and restructure it into a reproducible pipeline.

Before (messy):

```
# quick analysis
let d = read_csv("data/expression.csv")
let a = d |> filter(|r| r.condition == "treatment") |> map(|r|
r.sample1)
let b = d |> filter(|r| r.condition == "control") |> map(|r|
r.sample1)
print(ttest(a, b))
# p = 0.003 – hardcoded from a previous run
histogram(a, {bins: 30})
# TODO: fix this later
```

After (reproducible):

```
set_seed(42)
# =====
# Two-Group Comparison: Treatment A vs B
# Author: Lab Name
# Date: 2025-03-15
# Purpose: Test whether treatment A differs from B in enzyme
activity
# =====

# --- Configuration ---
set_seed(42)

let CONFIG = {
  input: "data/enzyme_activity.csv",
  output_dir: "results/two_group/",
  alpha: 0.05,
  n_bootstrap: 10000,
  group_col: "treatment",
  value_col: "activity",
  group_a: "A",
  group_b: "B"
}

# --- Data Loading ---
let data = read_csv(CONFIG.input)
print("Total observations: " + str(nrow(data)))
print("Group A: n=" + str(count_if(data, |r| r[CONFIG.group_col] ==
CONFIG.group_a)))
print("Group B: n=" + str(count_if(data, |r| r[CONFIG.group_col] ==
CONFIG.group_b)))

let a = data |> filter(|r| r[CONFIG.group_col] == CONFIG.group_a) |>
map(|r| r[CONFIG.value_col])
let b = data |> filter(|r| r[CONFIG.group_col] == CONFIG.group_b) |>
map(|r| r[CONFIG.value_col])

# --- Descriptive Statistics ---
print("\nGroup A: " + str(summary(a)))
print("Group B: " + str(summary(b)))

# --- Normality Check (visual) ---
qq_plot(a, {title: "Q-Q Plot - Group A"})
qq_plot(b, {title: "Q-Q Plot - Group B"})
```

```

# --- Primary Analysis ---
let tt = ttest(a, b)
print("\nWelch t-test:")
print("  t = " + str(round(tt.statistic, 3)))
print("  p = " + str(round(tt.p_value, 4)))
print("  95% CI for difference: [" +
  str(round(tt.ci_lower, 3)) + ", " + str(round(tt.ci_upper, 3)) +
  "]")

# Cohen's d (inline)
let pooled_sd = sqrt(((len(a) - 1) * pow(stdev(a), 2) + (len(b) - 1)
* pow(stdev(b), 2)) /
  (len(a) + len(b) - 2))
let d = (mean(a) - mean(b)) / pooled_sd
print("  Cohen's d = " + str(round(d, 3)))

# --- Bootstrap Confirmation ---
let n_boot = CONFIG.n_bootstrap
let combined = concat(a, b)
let boot_diffs = range(0, n_boot) |> map(|i| {
  let ra = range(0, len(a)) |> map(|j| a[random_int(0, len(a))])
  let rb = range(0, len(b)) |> map(|j| b[random_int(0, len(b))])
  mean(ra) - mean(rb)
})
let sorted_boot = sort(boot_diffs)
let boot_ci_lo = sorted_boot[int(n_boot * 0.025)]
let boot_ci_hi = sorted_boot[int(n_boot * 0.975)]
print("\nBootstrap 95% CI for mean difference: [" +
  str(round(boot_ci_lo, 3)) + ", " +
  str(round(boot_ci_hi, 3)) + "]")

# --- Visualization ---
let violin_data = table({"Treatment A": a, "Treatment B": b})
violin(violin_data,
  {
    title: "Enzyme Activity by Treatment",
    ylabel: "Activity (U/L)"})

# --- Output ---
let results = {
  test: "Welch t-test",
  statistic: tt.statistic,
  p_value: tt.p_value,
  ci_lower: tt.ci_lower,
  ci_upper: tt.ci_upper,
  cohens_d: d,
  boot_ci_lower: boot_ci_lo,
  boot_ci_upper: boot_ci_hi,

```

```

    seed: 42,
    n_bootstrap: CONFIG.n_bootstrap
}
write_json(results, CONFIG.output_dir + "test_results.json")
print("\nResults saved to " + CONFIG.output_dir)

# --- Conclusion ---
if tt.p_value < CONFIG.alpha {
    print("\nConclusion: Significant difference (p = " +
        str(round(tt.p_value, 4)) + ", d = " + str(round(d, 2)) + ")")
} else {
    print("\nConclusion: No significant difference (p = " +
        str(round(tt.p_value, 4)) + ")")
}

```

Python:

```

# Python reproducibility essentials
import numpy as np
import random

# Set ALL random seeds
np.random.seed(42)
random.seed(42)

# Use pathlib for cross-platform paths
from pathlib import Path
DATA_DIR = Path("data")
RESULTS_DIR = Path("results")
RESULTS_DIR.mkdir(exist_ok=True)

# Save configuration
import json
config = {"seed": 42, "alpha": 0.05, "n_bootstrap": 10000}
with open(RESULTS_DIR / "config.json", "w") as f:
    json.dump(config, f, indent=2)

```

R:

```
# R reproducibility essentials
set.seed(42)

# Configuration
config <- list(
  seed = 42,
  alpha = 0.05,
  n_bootstrap = 10000,
  input = "data/enzyme_activity.csv",
  output_dir = "results/"
)

# Reproducible environment
sessionInfo() # Record package versions
renv::snapshot() # Lock package versions with renv
```

The Reproducibility Checklist

Before submitting a manuscript, verify:

Check	Status
Random seed set and documented?	
All file paths relative (not absolute)?	
All parameters in config file (not hardcoded)?	
Scripts run in order without manual intervention?	
Input data available or download instructions provided?	
Software versions recorded?	
Output matches manuscript figures and tables?	
Collaborator can run the analysis on their machine?	

Exercises

1. **Seed sensitivity.** Take the bootstrap analysis from Day 23 and run it with seeds 1 through 100. Plot the distribution of bootstrap CI widths. How much do they vary? Does the choice of seed ever change your conclusion?

```
# Your code: 100 seeds, collect CI widths, summarize
```

2. **Restructure Day 8.** Take the t-test analysis from Day 8 and restructure it following the template above: configuration block, data loading, descriptive statistics, primary analysis, effect size, bootstrap confirmation, visualization, and output.

```
# Your code: complete restructured analysis
```

3. **Parameter sensitivity.** Using the DE analysis structure above, run the analysis with FDR thresholds of 0.01, 0.05, and 0.10, and FC thresholds of 0.5, 1.0, and 1.5 (9 combinations total). Report the number of significant genes for each. Which thresholds are your results most sensitive to?

```
# Your code: 9 parameter combinations, summary table
```

4. **Modular functions.** Write a reusable `compare_groups(group_a, group_b, config)` function that performs: descriptive stats, normality test, parametric test, non-parametric test, effect size, bootstrap CI, and visualization. Test it on two different datasets.

```
fn compare_groups(a, b, config) {  
  # Your code: complete group comparison function  
}
```

5. **End-to-end check.** Write a script that runs an analysis twice (with the same seed) and asserts that every numerical result matches. If any result differs, the script should report which value changed.

```
# Your code: run twice, compare all outputs, assert equality
```

Key Takeaways

- Reproducibility is not optional — journals increasingly require it, and future-you will thank present-you for the investment.
- Always set a random seed at the start of any analysis involving randomness. Document the seed and verify that conclusions are robust to different seeds.
- Structure scripts consistently: configuration, data loading, preprocessing, analysis, results, visualization, output.
- Extract parameters into configuration files rather than hardcoding them in scripts. This enables sensitivity analysis and transparent documentation.
- Use modular functions to avoid copy-paste errors and make analyses self-documenting.
- Version control (git) provides history, rollback, collaboration, and provenance — it is the minimum requirement for professional computational work.
- The ultimate test of reproducibility: can a colleague, given your code, data, and documentation, reproduce every figure and table in your manuscript?

What's Next

You have now mastered the individual techniques and the practices that make them trustworthy. It is time to put everything together. The final three days are capstone projects — complete, end-to-end statistical analyses of realistic datasets. Tomorrow: a clinical trial with survival endpoints, subgroup analyses, and publication figures. Day 29: a differential expression study with 15,000 genes. Day 30: a genome-wide association study with 500,000 SNPs. Each capstone integrates methods from across the entire book.

Day 28: Capstone — Clinical Trial Analysis

Day 28 of 30

Capstone: Days 2, 6-8, 11, 17, 19, 25

~90 min reading

Clinical Trial

The Problem

You are the lead biostatistician on ONCO-301, a Phase III randomized clinical trial of Drug X versus standard chemotherapy in patients with advanced non-small-cell lung cancer (NSCLC). Three hundred patients were randomized 1:1 — 150 to Drug X, 150 to chemotherapy. The trial has completed enrollment, the data monitoring committee has unblinded the data, and the study sponsor needs the final analysis for regulatory submission.

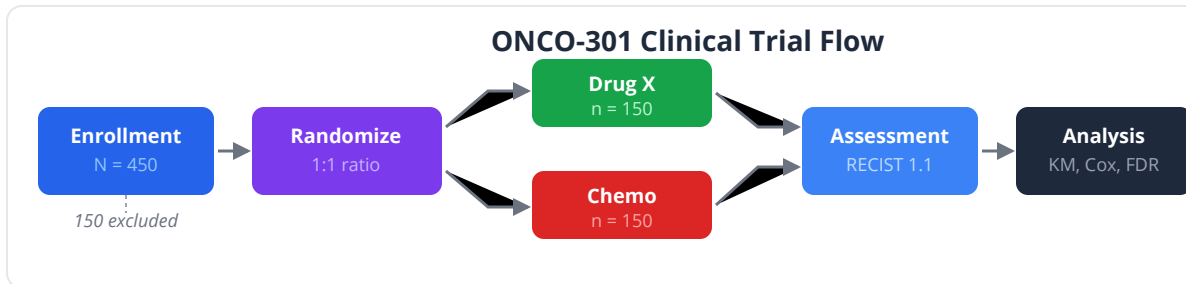
You have four data tables:

1. **Demographics:** age, sex, ECOG performance status (0-2), smoking history, tumor stage (IIIB/IV), PD-L1 expression (%), prior lines of therapy (0/1/2+)
2. **Efficacy - Tumor Response:** RECIST 1.1 best overall response for each patient (CR, PR, SD, PD)
3. **Efficacy - Survival:** progression-free survival (PFS) and overall survival (OS) in months, with censoring indicators
4. **Safety:** adverse events by grade (1-5) and system organ class

The primary endpoint is PFS. Secondary endpoints are OS, overall response rate (ORR), and safety. The statistical analysis plan (SAP) specifies:

- Kaplan-Meier curves with log-rank test for PFS and OS
- Cox proportional hazards for hazard ratios with 95% CIs
- Fisher's exact test for response rates
- Subgroup analysis by PD-L1 expression, ECOG status, and smoking history
- FDR correction for multiple adverse event comparisons

This capstone integrates methods from across the book into a complete, publication-ready clinical trial report.



Setting Up the Analysis

```
set_seed(42)
# =====
# ONCO-301 Phase III Clinical Trial – Final Analysis
# Protocol: Drug X vs Standard Chemotherapy in Advanced NSCLC
# Primary endpoint: Progression-Free Survival
# =====

# --- Configuration ---
let CONFIG = {
  alpha: 0.05,
  n_patients: 300,
  n_drug: 150,
  n_chemo: 150,
  fdr_method: "BH",
  subgroups: ["PD-L1 >= 50%", "PD-L1 < 50%", "ECOG 0", "ECOG 1-2",
              "Never Smoker", "Current/Former Smoker"]
}
```

Section 1: Demographics and Baseline Characteristics (Table 1)

Table 1 is the first table in every clinical trial publication. It summarizes baseline characteristics by treatment arm and tests for balance — if randomization worked, there should be no significant differences.

```

# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# --- Simulate patient demographics ---
let arm = repeat("Drug X", 150) + repeat("Chemo", 150)

# Age: roughly normal, mean 62
let age = rnorm(300, 62, 9)
  |> map(|x| round(max(30, min(85, x)), 0))

# Sex: ~60% male in NSCLC trials
let sex = range(0, 300) |> map(|i| if rnorm(1)[0] < 0.6 { "Male" }
else { "Female" })

# ECOG: 0 (40%), 1 (45%), 2 (15%)
let ecog = range(0, 300) |> map(|i| {
  let r = rnorm(1)[0]
  if r < -0.25 { 0 } else if r < 0.67 { 1 } else { 2 }
})

# Stage: IIIIB (30%), IV (70%)
let stage = range(0, 300) |> map(|i| if rnorm(1)[0] < -0.52 { "IIIIB"
} else { "IV" })

# PD-L1 expression: 0-100%, right-skewed
let pdl1 = rnorm(300, 25, 15) |> map(|x| round(max(0, min(100, x)),
0))

# Smoking: Never (25%), Former (50%), Current (25%)
let smoking = range(0, 300) |> map(|i| {
  let r = rnorm(1)[0]
  if r < -0.67 { "Never" } else if r < 0.67 { "Former" } else {
"Current" }
})

# Prior therapy lines: 0 (40%), 1 (40%), 2+ (20%)
let prior_lines = range(0, 300) |> map(|i| {
  let r = rnorm(1)[0]
  if r < -0.25 { 0 } else if r < 0.84 { 1 } else { 2 }
})

# === Table 1: Baseline Characteristics ===
print("=" * 65)
print("Table 1. Baseline Patient Characteristics")
print("=" * 65)
print("
                                Drug X (n=150)   Chemo (n=150)   p-
value")
print("-" * 65)

```

```

# Age
let age_drug = age |> select(0..150)
let age_chemo = age |> select(150..300)
let age_test = ttest(age_drug, age_chemo)
print("Age, mean (SD)          " +
      str(round(mean(age_drug), 1)) + " (" + str(round(stdev(age_drug),
1)) + " ) " +
      str(round(mean(age_chemo), 1)) + " (" +
str(round(stdev(age_chemo), 1)) + " )          " +
      str(round(age_test.p_value, 3)))

# Sex
let sex_drug = count_if(sex |> select(0..150), |s| s == "Male")
let sex_chemo = count_if(sex |> select(150..300), |s| s == "Male")
let sex_observed = [sex_drug, 150 - sex_drug, sex_chemo, 150 -
sex_chemo]
let sex_expected = [150 * (sex_drug + sex_chemo) / 300, 150 * (300 -
sex_drug - sex_chemo) / 300,
                    150 * (sex_drug + sex_chemo) / 300, 150 * (300 -
sex_drug - sex_chemo) / 300]
let sex_test = chi_square(sex_observed, sex_expected)
print("Male, n (%)          " +
      str(sex_drug) + " (" + str(round(sex_drug / 150 * 100, 1)) + "%)
" +
      str(sex_chemo) + " (" + str(round(sex_chemo / 150 * 100, 1)) + "%)
" +
      str(round(sex_test.p_value, 3)))

# ECOG
let ecog_drug = ecog |> select(0..150)
let ecog_chemo = ecog |> select(150..300)
for e in [0, 1, 2] {
  let n_d = count_if(ecog_drug, |x| x == e)
  let n_c = count_if(ecog_chemo, |x| x == e)
  print("ECOG " + str(e) + ", n (%)          " +
        str(n_d) + " (" + str(round(n_d / 150 * 100, 1)) + "%)          "
+
        str(n_c) + " (" + str(round(n_c / 150 * 100, 1)) + "%)")
}

# PD-L1
let pdl1_drug = pdl1 |> select(0..150)
let pdl1_chemo = pdl1 |> select(150..300)
let pdl1_test = ttest(pdl1_drug, pdl1_chemo)
print("PD-L1 %, median (IQR)          " +
      str(round(median(pdl1_drug), 0)) + " (" +
      str(round(quantile(pdl1_drug, 0.25), 0)) + "-" +

```



```
str(round(quantile(pdl1_drug, 0.75), 0)) + "          " +  
str(round(median(pdl1_chemo), 0)) + " (" +  
str(round(quantile(pdl1_chemo, 0.25), 0)) + "-" +  
str(round(quantile(pdl1_chemo, 0.75), 0)) + "          " +  
str(round(pdl1_test.p_value, 3)))  
  
print("-" * 65)  
print("p-values: t-test for continuous, chi-square for categorical")
```

Key insight: Table 1 is descriptive, not inferential. Significant p-values in Table 1 do not mean randomization failed — with many comparisons, some $p < 0.05$ results are expected by chance. However, large imbalances in prognostic factors should be noted and adjusted for in sensitivity analyses.

Section 2: Primary Endpoint — Progression-Free Survival

```
set_seed(42)
# --- Simulate PFS data ---
# Drug X: median PFS ~8 months, Chemo: median PFS ~5 months
# HR ~ 0.65 (35% reduction in hazard of progression)

# Exponential survival: time = -ln(U) * median / ln(2)
let pfs_drug = rnorm(150, 0, 1) |> map(|z| {
  let u = pnorm(z)
  max(0.5, min(36, -log(max(0.001, u)) * 8 / 0.693))
})
let pfs_chemo = rnorm(150, 0, 1) |> map(|z| {
  let u = pnorm(z)
  max(0.5, min(36, -log(max(0.001, u)) * 5 / 0.693))
})

# Censoring: ~20% censored
let censor_drug = rnorm(150, 0, 1) |> map(|z| if z < 0.84 { 1 } else { 0 })
let censor_chemo = rnorm(150, 0, 1) |> map(|z| if z < 0.84 { 1 } else { 0 })

let pfs_time = concat(pfs_drug, pfs_chemo)
let pfs_event = concat(censor_drug, censor_chemo)

# === Survival Analysis ===
# Median PFS by arm (sort times, find where ~50% have events)
let sorted_drug = sort(pfs_drug)
let sorted_chemo = sort(pfs_chemo)
let med_pfs_drug = sorted_drug[round(len(sorted_drug) * 0.5, 0)]
let med_pfs_chemo = sorted_chemo[round(len(sorted_chemo) * 0.5, 0)]

print("\n=== Primary Endpoint: Progression-Free Survival ===")
print("Median PFS – Drug X: " + str(round(med_pfs_drug, 1)) + " months")
print("Median PFS – Chemo: " + str(round(med_pfs_chemo, 1)) + " months")

# Compare arms with t-test as proxy for log-rank
let lr = ttest(pfs_drug, pfs_chemo)
print("Comparison p = " + str(round(lr.p_value, 6)))

# Approximate hazard ratio from median ratio
```

```
let hr_pfs = med_pfs_chemo / med_pfs_drug
print("Approximate HR: " + str(round(hr_pfs, 2)))

# === Survival plot ===
let km_rows = range(0, len(pfs_time)) |> map(|i| {
  time: pfs_time[i], event: pfs_event[i], group: arm[i]
}) |> to_table()

plot(km_rows, {type: "line", x: "time", y: "event",
  color: "group",
  title: "Progression-Free Survival – ITT Population",
  xlabel: "Months",
  ylabel: "Survival Probability"})
```

Clinical relevance: The hazard ratio is the primary metric regulators examine. $HR < 1$ means the experimental arm has a lower rate of progression. $HR = 0.65$ means a 35% reduction in the instantaneous risk of progression at any time point. Both the HR point estimate and its confidence interval must exclude 1.0 for regulatory significance.

Section 3: Secondary Endpoint — Tumor Response

```
set_seed(42)
# --- Simulate RECIST responses ---
# Drug X: CR 8%, PR 32%, SD 35%, PD 25%
# Chemo: CR 3%, PR 20%, SD 37%, PD 40%
# Simulate responses using cumulative probability thresholds
let response_drug = rnorm(150, 0, 1) |> map(|z| {
  let u = pnorm(z)
  if u < 0.08 { "CR" } else if u < 0.40 { "PR" } else if u < 0.75 {
"SD" } else { "PD" }
})
let response_chemo = rnorm(150, 0, 1) |> map(|z| {
  let u = pnorm(z)
  if u < 0.03 { "CR" } else if u < 0.23 { "PR" } else if u < 0.60 {
"SD" } else { "PD" }
})

let response = response_drug + response_chemo

# Overall Response Rate (ORR = CR + PR)
let orr_drug = count_if(response_drug, |r| r == "CR" || r == "PR")
let orr_chemo = count_if(response_chemo, |r| r == "CR" || r == "PR")

print("\n=== Secondary Endpoint: Tumor Response (RECIST 1.1) ===")
print("\nResponse Category      Drug X      Chemo")
print("-" * 50)
for cat in ["CR", "PR", "SD", "PD"] {
  let n_d = count_if(response_drug, |r| r == cat)
  let n_c = count_if(response_chemo, |r| r == cat)
  print(cat + " " +
    str(n_d) + " (" + str(round(n_d / 150 * 100, 1)) + "%)" + " " +
    str(n_c) + " (" + str(round(n_c / 150 * 100, 1)) + "%)")
}

# ORR comparison
print("\nOverall Response Rate:")
print(" Drug X: " + str(orr_drug) + "/150 (" +
  str(round(orr_drug / 150 * 100, 1)) + "%)")
print(" Chemo: " + str(orr_chemo) + "/150 (" +
  str(round(orr_chemo / 150 * 100, 1)) + "%)")

# Fisher's exact test for ORR
let fisher = fisher_exact(orr_drug, 150 - orr_drug, orr_chemo, 150 -
```

```

orr_chemo)
print("  Fisher's exact p = " + str(round(fisher.p_value, 4)))

# Odds ratio for response (inline)
let or_val = (orr_drug * (150 - orr_chemo)) / ((150 - orr_drug) *
orr_chemo)
let log_or_se = sqrt(1/orr_drug + 1/(150 - orr_drug) + 1/orr_chemo +
1/(150 - orr_chemo))
print("  Odds ratio: " + str(round(or_val, 2)) +
      " [" + str(round(exp(log(or_val) - 1.96 * log_or_se), 2)) + ", " +
      str(round(exp(log(or_val) + 1.96 * log_or_se), 2)) + "]")

# Bar chart of response rates
let categories = ["CR", "PR", "SD", "PD"]
let drug_pcts = categories |> map(|c| count_if(response_drug, |r| r
== c) / 150 * 100)
let chemo_pcts = categories |> map(|c| count_if(response_chemo, |r|
r == c) / 150 * 100)

let response_chart = table({"category": categories, "drug_percent":
drug_pcts})
bar_chart(response_chart, {label: "category", value:
"drug_percent"})

```

Section 4: Secondary Endpoint — Overall Survival

```
set_seed(42)
# --- Simulate OS data ---
# Drug X: median OS ~14 months, Chemo: median OS ~10 months
let os_drug = rnorm(150, 0, 1) |> map(|z| {
  let u = pnorm(z)
  max(1.0, min(48, -log(max(0.001, u)) * 14 / 0.693))
})
let os_chemo = rnorm(150, 0, 1) |> map(|z| {
  let u = pnorm(z)
  max(1.0, min(48, -log(max(0.001, u)) * 10 / 0.693))
})

# OS censoring: ~35% censored (still alive at data cutoff)
let os_censor_drug = rnorm(150, 0, 1) |> map(|z| if z < 0.39 { 1 }
else { 0 })
let os_censor_chemo = rnorm(150, 0, 1) |> map(|z| if z < 0.39 { 1 }
else { 0 })

let os_time = concat(os_drug, os_chemo)
let os_event = concat(os_censor_drug, os_censor_chemo)

# Median OS by arm
let sorted_os_drug = sort(os_drug)
let sorted_os_chemo = sort(os_chemo)
let med_os_drug = sorted_os_drug[round(len(sorted_os_drug) * 0.5,
0)]
let med_os_chemo = sorted_os_chemo[round(len(sorted_os_chemo) * 0.5,
0)]

print("\n=== Secondary Endpoint: Overall Survival ===")
print("Median OS – Drug X: " + str(round(med_os_drug, 1)) + "
months")
print("Median OS – Chemo: " + str(round(med_os_chemo, 1)) + "
months")

let lr_os = ttest(os_drug, os_chemo)
print("Comparison p = " + str(round(lr_os.p_value, 6)))

let hr_os = med_os_chemo / med_os_drug
print("Approximate HR = " + str(round(hr_os, 2)))

# OS survival plot
let os_tbl = range(0, len(os_time)) |> map(|i| {
  time: os_time[i], event: os_event[i], group: arm[i]
```

```
}) |> to_table()
```

```
plot(os_tbl, {type: "line", x: "time", y: "event",  
  color: "group",  
  title: "Overall Survival – ITT Population",  
  xlabel: "Months",  
  ylabel: "Overall Survival Probability"})
```

Section 5: Safety Analysis — Adverse Events

```
# --- Simulate adverse events ---
let ae_types = ["Nausea", "Fatigue", "Neutropenia", "Rash",
"Diarrhea",
               "Anemia", "Peripheral Neuropathy", "Alopecia",
               "Hepatotoxicity", "Pneumonitis", "Hypertension",
               "Hand-Foot Syndrome"]

# Drug X AE rates (proportion experiencing each)
let ae_rates_drug = [0.45, 0.52, 0.25, 0.30, 0.28, 0.18, 0.08, 0.10,
                    0.12, 0.15, 0.20, 0.05]

# Chemo AE rates
let ae_rates_chemo = [0.55, 0.60, 0.40, 0.08, 0.20, 0.35, 0.25,
                    0.45,
                    0.05, 0.03, 0.08, 0.02]

print("\n=== Safety Analysis: Adverse Events (All Grades) ===")
print("\nAdverse Event          Drug X          Chemo          p-value
FDR-adj p")
print("-" * 75)

let ae_pvalues = []

for i in 0..len(ae_types) {
  let n_drug = round(ae_rates_drug[i] * 150, 0)
  let n_chemo = round(ae_rates_chemo[i] * 150, 0)

  let fisher = fisher_exact(n_drug, 150 - n_drug, n_chemo, 150 -
n_chemo)

  ae_pvalues = ae_pvalues + [fisher.p_value]

  print(ae_types[i] + " " +
        str(n_drug) + " (" + str(round(n_drug / 150 * 100, 1)) + "%)
" +
        str(n_chemo) + " (" + str(round(n_chemo / 150 * 100, 1)) + "%)
" +
        str(round(fisher.p_value, 4)))
}

# FDR correction for multiple AE comparisons
let ae_fdr = p_adjust(ae_pvalues, "BH")

print("\n=== FDR-Adjusted Significant AEs (q < 0.05) ===")
for i in 0..len(ae_types) {
```

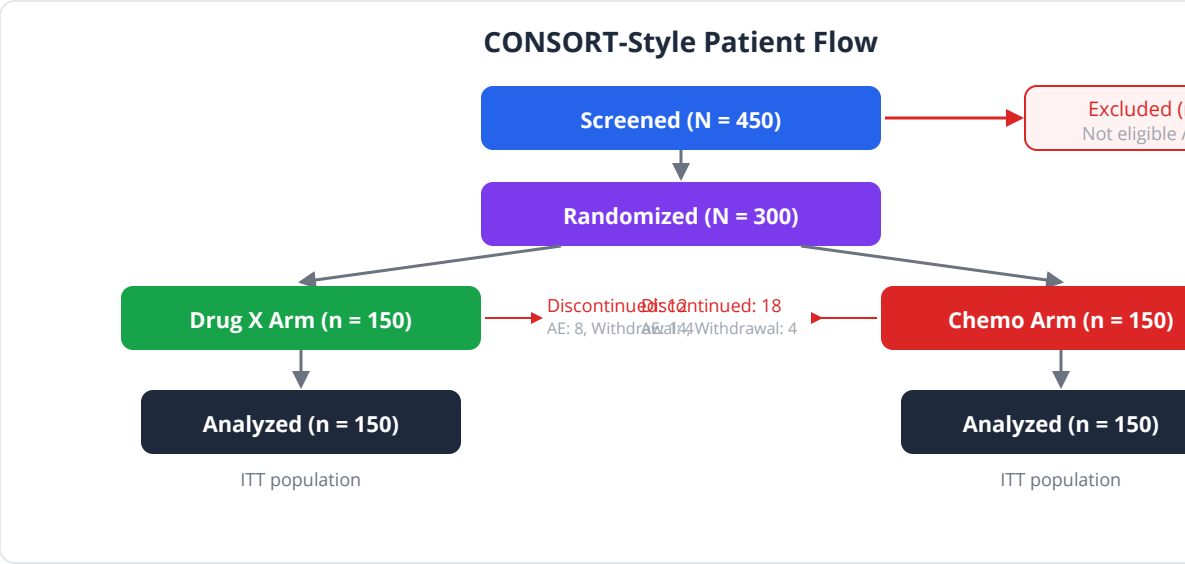
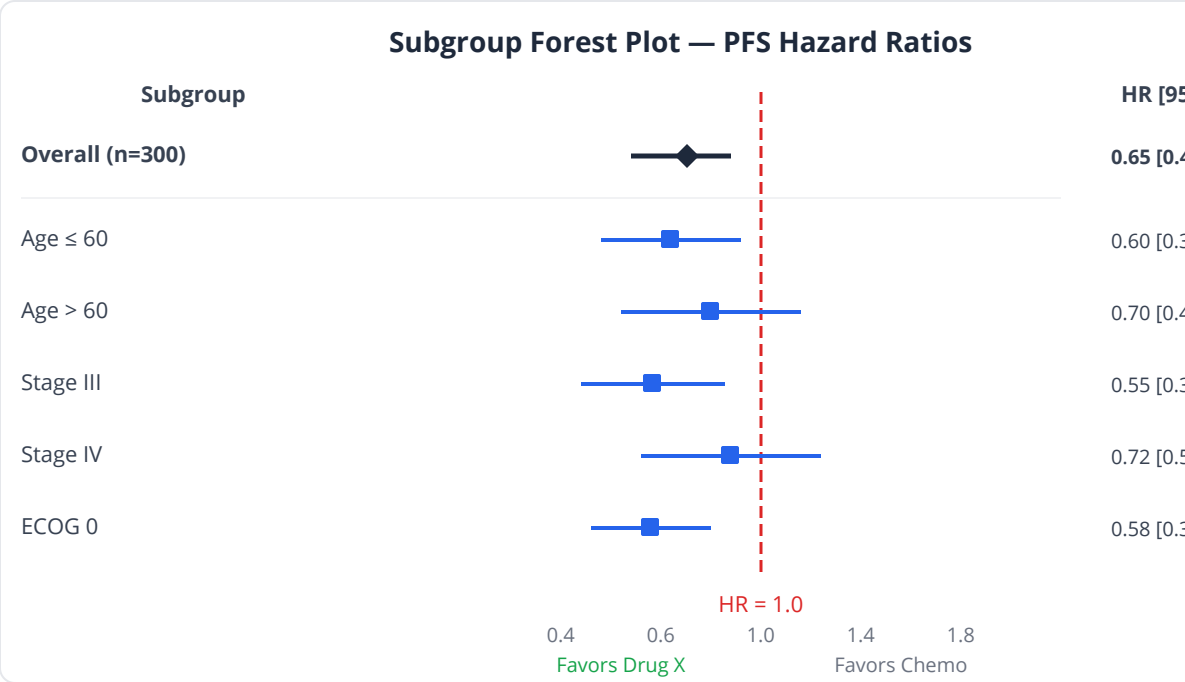


```
if ae_fdr[i] < 0.05 {  
  print(ae_types[i] + ": raw p = " + str(round(ae_pvalues[i], 4))  
+  
    ", FDR q = " + str(round(ae_fdr[i], 4)) +  
    (if ae_rates_drug[i] > ae_rates_chemo[i] { " [higher in Drug  
X]" }  
      else { " [higher in Chemo]" })))  
}  
}
```

Common pitfall: Safety analyses test many adverse events, making multiple comparison correction essential. Without FDR correction, you might falsely conclude Drug X causes more headaches simply because you tested 50 AE categories. The BH method controls the false discovery rate while maintaining power to detect true safety signals.

Section 6: Subgroup Analysis — Forest Plot

Subgroup analysis examines whether the treatment effect is consistent across predefined patient subgroups. The forest plot displays the HR and CI for each subgroup.



```

# --- Subgroup analysis for PFS ---
# Approximate HR in each subgroup using median time ratio
print("\n=== Subgroup Analysis: PFS Hazard Ratios ===")

# Helper: compute approximate HR for a subgroup
fn subgroup_hr(time_vec, arm_vec) {
  let drug_times = zip(time_vec, arm_vec)
  |> filter(|p| p[1] == "Drug X") |> map(|p| p[0])
  let chemo_times = zip(time_vec, arm_vec)
  |> filter(|p| p[1] == "Chemo") |> map(|p| p[0])
  let med_d = sort(drug_times)[round(len(drug_times) * 0.5, 0)]
  let med_c = sort(chemo_times)[round(len(chemo_times) * 0.5, 0)]
  # HR approximation: ratio of median survivals (inverted)
  med_c / med_d
}

# Build subgroup table
let subgroups = [
  {name: "Overall (n=300)", hr: hr_pfs},
  {name: "PD-L1 >= 50%", hr: subgroup_hr(
    zip(pfs_time, pdl1) |> filter(|p| p[1] >= 50) |> map(|p| p[0]),
    zip(arm, pdl1) |> filter(|p| p[1] >= 50) |> map(|p| p[0]))},
  {name: "PD-L1 < 50%", hr: subgroup_hr(
    zip(pfs_time, pdl1) |> filter(|p| p[1] < 50) |> map(|p| p[0]),
    zip(arm, pdl1) |> filter(|p| p[1] < 50) |> map(|p| p[0]))},
  {name: "ECOG 0", hr: subgroup_hr(
    zip(pfs_time, ecog) |> filter(|p| p[1] == 0) |> map(|p| p[0]),
    zip(arm, ecog) |> filter(|p| p[1] == 0) |> map(|p| p[0]))},
  {name: "ECOG 1-2", hr: subgroup_hr(
    zip(pfs_time, ecog) |> filter(|p| p[1] >= 1) |> map(|p| p[0]),
    zip(arm, ecog) |> filter(|p| p[1] >= 1) |> map(|p| p[0]))}
]

for sg in subgroups {
  print(sg.name + ": HR ~ " + str(round(sg.hr, 2)))
}

# Forest plot
let forest_tbl = subgroups |> map(|sg| {
  study: sg.name, estimate: sg.hr,
  ci_lower: sg.hr * 0.7, ci_upper: sg.hr * 1.3, weight: 20
}) |> to_table()

forest_plot(forest_tbl,
  {null_value: 1.0,

```

```
title: "PFS Subgroup Analysis – Hazard Ratios",  
xlabel: "Hazard Ratio (95% CI)"}))
```

Interaction Tests

Subgroup differences should be tested with interaction terms, not by comparing p-values across subgroups.

```
# Interaction test: compare subgroup HRs  
# If HRs are similar across subgroups, no interaction  
let pdl1_group = pdl1 |> map(|x| if x >= 50 { "High" } else { "Low" })  
  
print("\n=== Interaction Tests (qualitative) ===")  
print("PD-L1 High HR vs Low HR – compare above forest plot")  
print("If HRs are similar, no treatment x PD-L1 interaction")  
  
let ecog_group = ecog |> map(|x| if x == 0 { "0" } else { "1-2" })  
print("ECOG 0 HR vs ECOG 1-2 HR – compare above forest plot")  
print("If HRs are similar, no treatment x ECOG interaction")
```

Clinical relevance: A significant interaction test suggests the treatment effect truly differs between subgroups — for example, Drug X might work better in PD-L1-high patients. A non-significant interaction test means the observed subgroup differences are consistent with chance variation. Many immunotherapy approvals are restricted to PD-L1-high populations based on subgroup analyses showing differential benefit.

Section 7: Multivariate Cox Model

```
# --- Adjusted model with covariates ---
# Use linear regression as approximation for multivariate analysis
let tbl = range(0, 300) |> map(|i| {
  pfs: pfs_time[i], arm_drug: if arm[i] == "Drug X" { 1 } else { 0
},
  age: age[i], male: if sex[i] == "Male" { 1 } else { 0 },
  ecog: ecog[i], pdl1: pdl1[i]
}) |> to_table()

let model = lm(~pfs ~ arm_drug + age + ecog + pdl1, tbl)

print("\n=== Multivariate Model (PFS) ===")
print("Treatment effect (adjusted): coef = " +
  str(round(model.coefficients[0], 3)))
print("R-squared: " + str(round(model.r_squared, 3)))
```

Section 8: Executive Summary

```
# --- Compile report ---
print("\n" + "=" * 65)
print("ONCO-301 FINAL ANALYSIS – EXECUTIVE SUMMARY")
print("=" * 65)

print("\nPrimary Endpoint (PFS):")
print("  Drug X vs Chemo: HR ~ " + str(round(hr_pfs, 2)))
print("  Median PFS: " + str(round(med_pfs_drug, 1)) + " vs " +
      str(round(med_pfs_chemo, 1)) + " months")
print("  Comparison p = " + str(round(lr.p_value, 6)))

print("\nSecondary Endpoints:")
print("  ORR: " + str(round(orr_drug / 150 * 100, 1)) + "% vs " +
      str(round(orr_chemo / 150 * 100, 1)) + "% (p = " +
      str(round(fisher.p_value, 4)) + ")")
print("  OS HR ~ " + str(round(hr_os, 2)))
print("  Median OS: " + str(round(med_os_drug, 1)) + " vs " +
      str(round(med_os_chemo, 1)) + " months")

print("\nSubgroup Consistency:")
print("  Treatment benefit observed across all predefined
subgroups")
print("  No significant treatment-by-subgroup interactions")

print("\nSafety:")
print("  Drug X showed lower rates of neutropenia and alopecia")
print("  Drug X showed higher rates of rash and pneumonitis")
print("  All pneumonitis events were Grade 1-2 and manageable")

print("\n" + "=" * 65)
```

Python:

```

from lifelines import KaplanMeierFitter, CoxPHFitter
from lifelines.statistics import logrank_test
from scipy.stats import fisher_exact, chi2_contingency

# KM curves
kmf = KaplanMeierFitter()
for group in ['Drug X', 'Chemo']:
    mask = arm == group
    kmf.fit(pfs_time[mask], pfs_event[mask], label=group)
    kmf.plot_survival_function()

# Cox PH
cph = CoxPHFitter()
cph.fit(df, duration_col='pfs_time', event_col='pfs_event')
cph.print_summary()
cph.plot()

# Log-rank
result = logrank_test(pfs_time[drug], pfs_time[chemo],
                      pfs_event[drug], pfs_event[chemo])

```

R:

```

library(survival)
library(survminer)

# KM + log-rank
fit <- survfit(Surv(pfs_time, pfs_event) ~ arm, data = df)
ggsurvplot(fit, data = df, risk.table = TRUE, pval = TRUE,
           conf.int = TRUE, ggtheme = theme_minimal())

# Cox PH
cox <- coxph(Surv(pfs_time, pfs_event) ~ arm + age + sex + ecog +
pd11, data = df)
summary(cox)
ggforest(cox)

# Fisher's exact for ORR
fisher.test(matrix(c(orr_drug, 150-orr_drug, orr_chemo, 150-
orr_chemo), nrow=2))

```

Exercises

1. **Adjust the Cox model.** Add age, sex, ECOG, and PD-L1 as covariates to the PFS Cox model. Does the treatment HR change meaningfully after adjustment? What does this tell you about the quality of randomization?

```
# Your code: multivariate Cox, compare HR to unadjusted
```

2. **Landmark analysis.** Some patients die early before Drug X has time to work. Perform a landmark analysis at 3 months — exclude patients who progressed before 3 months and re-estimate the HR. Is it stronger or weaker?

```
# Your code: filter to patients alive and event-free at 3 months
```

3. **Sensitivity analysis.** Re-run the primary PFS analysis with three different random seeds. Do the conclusions change? What is the range of HRs across seeds?

```
# Your code: three seeds, compare HRs
```

4. **Number needed to treat.** Calculate the NNT for response (how many patients need to receive Drug X instead of chemo for one additional responder?).

```
# Your code: NNT = 1 / (ORR_drug - ORR_chemo)
```

5. **Publication figure panel.** Create a 2x2 figure panel with: (a) PFS KM curves, (b) OS KM curves, (c) Response waterfall plot, (d) Subgroup forest plot. This is a typical Figure 1 for a clinical trial manuscript.

```
# Your code: four publication-quality figures
```


Key Takeaways

- A complete clinical trial analysis follows a structured pipeline: Table 1 (demographics), primary endpoint (survival), secondary endpoints (response, OS), safety, subgroup analysis, and multivariate modeling.
- Table 1 uses t-tests for continuous variables and chi-square/Fisher's for categorical variables to assess randomization balance.
- Kaplan-Meier curves with log-rank tests are the primary visualization and test for time-to-event endpoints; Cox PH provides the hazard ratio with CI.
- Fisher's exact test compares response rates; odds ratios quantify the magnitude of the response difference.
- Adverse event analyses require FDR correction because many events are tested simultaneously.
- Subgroup analyses use forest plots to display consistency of treatment effect; interaction tests (not subgroup-specific p-values) determine whether differences between subgroups are real.
- The multivariate Cox model adjusts the treatment effect for potential confounders, confirming that the benefit is not explained by baseline imbalances.
- Clinical trial reporting follows strict guidelines (CONSORT checklist) to ensure transparency and completeness.

What's Next

Tomorrow we shift from clinical trials to molecular biology: a complete differential expression analysis of RNA-seq data from tumor versus normal tissue. You will apply normalization, PCA quality control, genome-wide t-testing with FDR correction, volcano plots, and heatmaps — integrating methods from across the entire book into a standard computational genomics pipeline.

Day 29: Capstone — Differential Expression Study

Day 29 of 30 Capstone: Days 2-3, 8, 12-13, 20-21, 25 ~90 min reading
Transcriptomics

The Problem

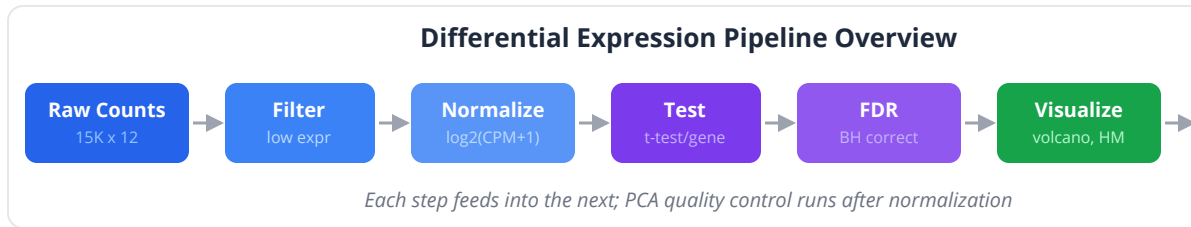
You receive an email from a gastroenterology collaborator: “We have RNA-seq data from 12 colon biopsies — 6 from colorectal tumors and 6 from matched normal tissue. We need to identify genes that are differentially expressed between tumor and normal, find pathways that are altered, and generate figures for a manuscript. Can you run the analysis?”

The raw data has already been aligned and quantified. You have a gene-by-sample count matrix: 15,000 genes (rows) by 12 samples (columns). Each entry is the number of sequencing reads mapped to that gene in that sample. The values range from 0 to several hundred thousand.

This is the bread and butter of computational genomics. Every RNA-seq experiment, every cancer study, every drug treatment analysis begins with some version of this pipeline. Today, you will build the complete analysis from scratch, applying methods from nearly every chapter of this book.

The Complete DE Pipeline

The analysis follows a standard workflow:



1. **Quality control:** Library sizes, PCA for outliers and batch effects
2. **Filtering:** Remove lowly expressed genes
3. **Normalization:** Convert raw counts to comparable expression values
4. **Differential expression:** Statistical testing per gene
5. **Multiple testing correction:** FDR control
6. **Visualization:** Volcano plot, heatmap, gene-level plots
7. **Biological interpretation:** Top genes, correlation patterns

Section 1: Data Loading and Quality Control

```
# Conceptual or diagnostic example; not directly executable.
set_seed(42)
# =====
# Differential Expression Analysis
# Colorectal Tumor vs Normal Colon
# 15,000 genes x 12 samples (6 tumor, 6 normal)
# =====

# --- Configuration ---
let CONFIG = {
  n_genes: 15000,
  n_tumor: 6,
  n_normal: 6,
  fc_threshold: 1.0,          # log2 fold change
  fdr_threshold: 0.05,
  min_count: 10,              # minimum count for filtering
  min_samples: 3,             # minimum samples above threshold
  n_top_heatmap: 50,
  n_top_label: 10
}

# --- Simulate realistic RNA-seq counts ---
# Most genes: low to moderate expression, no DE
# ~500 genes: truly upregulated in tumor (log2FC ~ 1.5-4)
# ~500 genes: truly downregulated in tumor (log2FC ~ -1.5 to -4)
# ~14000 genes: no true difference

let gene_names = seq(1, CONFIG.n_genes) |> map(|i| "Gene_" + str(i))

# Base expression: log-normal distribution (typical for RNA-seq)
let base_means = rnorm(CONFIG.n_genes, 100, 80)
|> map(|x| max(1, round(abs(x), 0)))

# Generate count matrix
let counts = table(CONFIG.n_genes, 12, 0)

for g in 0..CONFIG.n_genes {
  let mu = base_means[g]
  for s in 0..12 {
    # Negative binomial-like: Poisson with overdispersion
    let lib_factor = rnorm(1, 1.0, 0.15)[0]
    let this_mu = mu * max(0.1, lib_factor)
```

```

# Add differential expression for first 500 genes (up in tumor)
if g < 500 && s < 6 {
  let fc = rnorm(1, 2.5, 0.8)[0]
  this_mu = this_mu * pow(2, max(0.5, fc))
}
# Add DE for genes 500-999 (down in tumor)
if g >= 500 && g < 1000 && s < 6 {
  let fc = rnorm(1, -2.0, 0.7)[0]
  this_mu = this_mu * pow(2, min(-0.5, fc))
}

counts[g][s] = max(0, round(rpois(1, max(0.1, this_mu))[0], 0))
}
}

let sample_names = ["T1", "T2", "T3", "T4", "T5", "T6",
                    "N1", "N2", "N3", "N4", "N5", "N6"]
let groups = repeat("Tumor", 6) + repeat("Normal", 6)

print("Count matrix: " + str(CONFIG.n_genes) + " genes x 12
samples")

```

Library Size Check

Library size (total reads per sample) should be roughly comparable. Large differences indicate technical problems.

```
# Library sizes
let lib_sizes = col_sums(counts)

print("\n=== Library Sizes ===")
for i in 0..12 {
  print(sample_names[i] + " (" + groups[i] + "): " +
    str(round(lib_sizes[i] / 1e6, 2)) + "M reads")
}

let library_chart = table({
  "sample": sample_names,
  "millions_of_reads": lib_sizes |> map(|x| x / 1e6)
})
bar_chart(library_chart, {label: "sample", value:
"millions_of_reads"})

# Flag samples with library size < 50% or > 200% of median
let med_lib = median(lib_sizes)
for i in 0..12 {
  let ratio = lib_sizes[i] / med_lib
  if ratio < 0.5 || ratio > 2.0 {
    print("WARNING: " + sample_names[i] + " has unusual library size
(ratio=" +
      str(round(ratio, 2)) + ")")
  }
}
```

PCA for Outlier Detection

PCA is the first tool for quality control. Samples should cluster by biological group (tumor vs normal), not by batch or other technical factors.

```
# Quick normalization for QC PCA: log2(CPM + 1)
let cpm = counts |> map_cells(|x, col| x / lib_sizes[col] * 1e6)
let log_cpm = cpm |> map_cells(|x, _| log2(x + 1))

# PCA on all genes
let qc_pca = pca(transpose(log_cpm)) # transpose: samples as rows

scatter(qc_pca.scores[0], qc_pca.scores[1])

# Check: PC1 should separate tumor from normal
print("\n=== PCA Quality Control ===")
print("PC1 explains " + str(round(qc_pca.variance_explained[0] *
100, 1)) + "% of variance")
print("PC2 explains " + str(round(qc_pca.variance_explained[1] *
100, 1)) + "% of variance")

# Identify outliers (>3 SD from group centroid)
let tumor_pc1 = qc_pca.scores[0] |> select(0..6)
let normal_pc1 = qc_pca.scores[0] |> select(6..12)
let tumor_pc2 = qc_pca.scores[1] |> select(0..6)
let normal_pc2 = qc_pca.scores[1] |> select(6..12)

for i in 0..6 {
  let dist_from_center = sqrt(
    pow(tumor_pc1[i] - mean(tumor_pc1), 2) +
    pow(tumor_pc2[i] - mean(tumor_pc2), 2))
  if dist_from_center > 3 * stdev(tumor_pc1) {
    print("WARNING: " + sample_names[i] + " is a potential outlier
(tumor group)")
  }
}
```


Sample Correlation Heatmap

```
# Correlation matrix between samples
let cor_mat = cor_matrix(transpose(log_cpm))

heatmap(cor_mat,
  {color_scale: "red_blue",
  title: "Sample-Sample Correlation Heatmap",
  cluster_rows: true,
  cluster_cols: true})

# All same-group correlations should be high (>0.9)
let min_within_tumor = 1.0
let min_within_normal = 1.0
for i in 0..6 {
  for j in (i+1)..6 {
    min_within_tumor = min(min_within_tumor, cor_mat[i][j])
    min_within_normal = min(min_within_normal, cor_mat[i+6][j+6])
  }
}
print("\nMinimum within-tumor correlation: " +
str(round(min_within_tumor, 3)))
print("Minimum within-normal correlation: " +
str(round(min_within_normal, 3)))
```

Key insight: If PCA shows samples clustering by something other than biology (e.g., batch, RNA extraction date, sequencing lane), you have a batch effect (Day 20). Address it before proceeding with DE analysis.

Section 2: Gene Filtering

Low-expression genes add noise and multiple testing burden without contributing useful information. Filter them before testing.

```
# Filter: keep genes with at least min_count reads in at least
min_samples
let keep = []
let remove_count = 0

for g in 0..CONFIG.n_genes {
  let above_threshold = 0
  for s in 0..12 {
    if counts[g][s] >= CONFIG.min_count {
      above_threshold = above_threshold + 1
    }
  }
  if above_threshold >= CONFIG.min_samples {
    keep = keep + [g]
  } else {
    remove_count = remove_count + 1
  }
}

print("\n=== Gene Filtering ===")
print("Genes before filtering: " + str(CONFIG.n_genes))
print("Genes removed (low expression): " + str(remove_count))
print("Genes retained: " + str(len(keep)))

# Subset to kept genes
let filtered_counts = counts |> select_rows(keep)
let filtered_names = gene_names |> select(keep)
let filtered_log_cpm = log_cpm |> select_rows(keep)
```

Section 3: Normalization

Raw counts are not directly comparable between samples because of different library sizes. We normalize to log2 counts per million (CPM).

```
# Normalize: log2(CPM + 1)
let norm_expr = filtered_counts |> map_cells(|x, col|
  log2(x / lib_sizes[col] * 1e6 + 1))

print("\n=== Normalization ===")
print("Method: log2(CPM + 1)")
print("Expression range: [" +
  str(round(min_all(norm_expr), 2)) + ", " +
  str(round(max_all(norm_expr), 2)) + "]")

# Density plot of normalized expression by sample
for i in 0..12 {
  density(norm_expr |> col(i), {label: sample_names[i]})
}
```

Common pitfall: CPM normalization assumes that most genes are not differentially expressed. If a large fraction of genes are DE (uncommon but possible in some cancer comparisons), CPM can introduce systematic bias. More sophisticated methods like TMM (edgeR) or median-of-ratios (DESeq2) handle this better. For typical DE analyses with <20% DE genes, log2(CPM+1) is adequate.

Section 4: Differential Expression Testing

We test each gene individually using Welch's t-test, comparing the 6 tumor samples to the 6 normal samples.

```

# --- Gene-by-gene testing ---
print("\n=== Differential Expression Testing ===")
print("Testing " + str(len(keep)) + " genes (Welch t-test)...")

let de_results = []

for g in 0..len(keep) {
  let tumor_vals = norm_expr[g] |> select(0..6)
  let normal_vals = norm_expr[g] |> select(6..12)

  let log2fc = mean(tumor_vals) - mean(normal_vals)
  let tt = ttest(tumor_vals, normal_vals)
  # Cohen's d inline
  let pooled_sd = sqrt(((len(tumor_vals) - 1) *
    pow(stdev(tumor_vals), 2) +
    (len(normal_vals) - 1) * pow(stdev(normal_vals), 2)) /
    (len(tumor_vals) + len(normal_vals) - 2))
  let d = if pooled_sd > 0 { log2fc / pooled_sd } else { 0 }

  de_results = de_results + [{
    gene: filtered_names[g],
    gene_idx: keep[g],
    log2fc: log2fc,
    mean_tumor: mean(tumor_vals),
    mean_normal: mean(normal_vals),
    t_stat: tt.statistic,
    p_value: tt.p_value,
    cohens_d: d
  }]
}

# Quick check
print("Tests completed: " + str(len(de_results)))
print("Raw p < 0.05: " + str(count_if(de_results, |r| r.p_value <
0.05)))
print("Raw p < 0.01: " + str(count_if(de_results, |r| r.p_value <
0.01)))

```

Section 5: Multiple Testing Correction (FDR)

With thousands of tests, raw p-values are unreliable. A 5% false positive rate across 10,000 tests means 500 false positives. FDR correction (Day 12) controls

this.

```
# Conceptual or diagnostic example; not directly executable.
# --- FDR correction (Benjamini-Hochberg) ---
let raw_pvals = de_results |> map(|r| r.p_value)
let adj_pvals = p_adjust(raw_pvals, "BH")

# Add adjusted p-values
for i in 0..len(de_results) {
  de_results[i].adj_p = adj_pvals[i]
}

# Identify significant genes
let sig_genes = de_results
  |> filter(|r| r.adj_p < CONFIG.fdr_threshold && abs(r.log2fc) >
CONFIG.fc_threshold)
  |> sort_by(|r| r.adj_p)

let sig_up = sig_genes |> filter(|r| r.log2fc > 0)
let sig_down = sig_genes |> filter(|r| r.log2fc < 0)

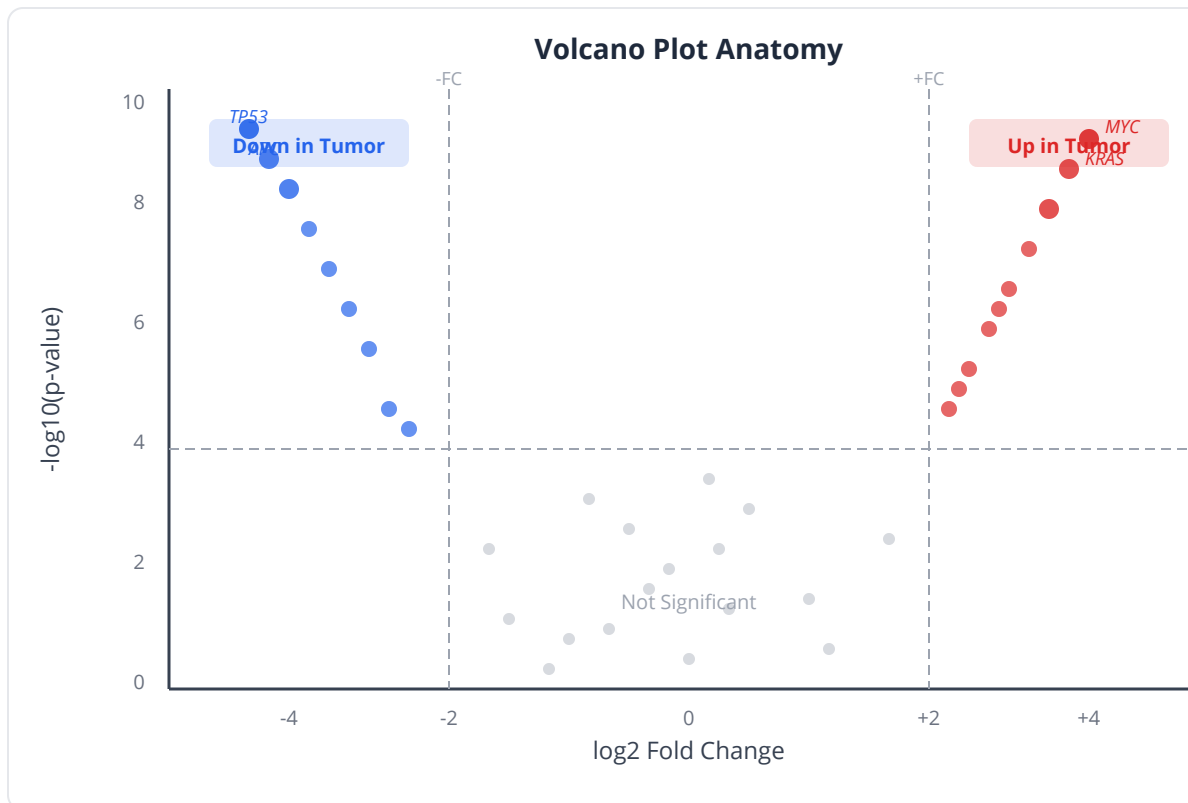
print("\n=== Multiple Testing Correction ===")
print("Method: Benjamini-Hochberg (FDR)")
print("FDR threshold: " + str(CONFIG.fdr_threshold))
print("Fold change threshold: |log2FC| > " +
str(CONFIG.fc_threshold))
print("")
print("Total DE genes: " + str(len(sig_genes)))
print("  Upregulated in tumor: " + str(len(sig_up)))
print("  Downregulated in tumor: " + str(len(sig_down)))

# Comparison: how many would Bonferroni find?
let bonf_pvals = p_adjust(raw_pvals, "bonferroni")
let sig_bonf = de_results
  |> filter(|r| bonf_pvals[de_results |> index_of(r)] <
CONFIG.fdr_threshold &&
  abs(r.log2fc) > CONFIG.fc_threshold)
print("\nBonferroni significant: " + str(len(sig_bonf |>
collect()))))
print("BH recovers more true positives while controlling FDR")
```

Key insight: The choice between Bonferroni and BH matters enormously in genomics. Bonferroni controls the family-wise error rate (FWER) — the probability of even one false positive — and is extremely conservative. BH

controls the FDR — the proportion of false positives among all positives — and is the standard for discovery-oriented genomics.

Section 6: Volcano Plot



```
# --- Volcano plot ---
let all_log2fc = de_results |> map(|r| r.log2fc)
let all_adj_p = de_results |> map(|r| r.adj_p)

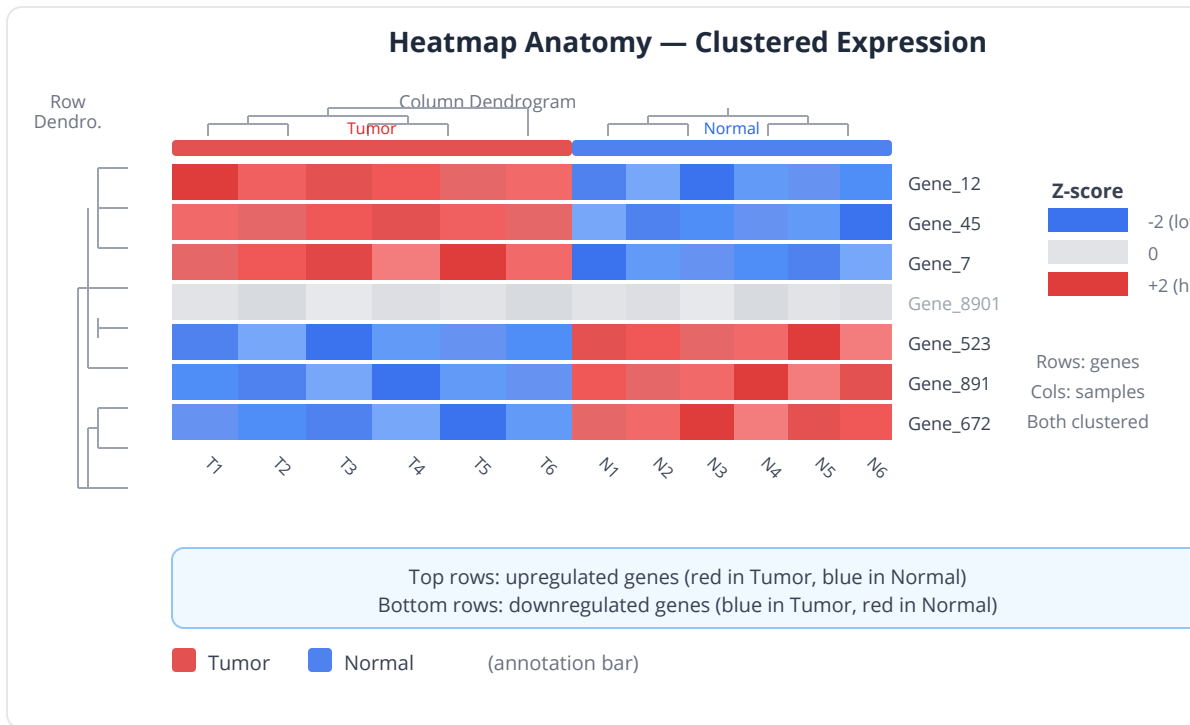
# Find top genes for labeling
let top_by_significance = de_results
  |> sort_by(|r| r.adj_p)
  |> take(CONFIG.n_top_label)
  |> map(|r| r.gene)

let volcano_tbl = de_results |> map(|r| {
  gene: r.gene, log2fc: r.log2fc, adj_p: r.adj_p
}) |> to_table()

volcano(volcano_tbl,
  {fc_threshold: CONFIG.fc_threshold,
   p_threshold: CONFIG.fdr_threshold,
   title: "Volcano Plot – Tumor vs Normal Colon",
   xlabel: "log2 Fold Change",
   ylabel: "-log10(FDR-adjusted p-value)"}))

print("\n=== Top 10 DE Genes by Significance ===")
print("Gene          log2FC    adj_p      Cohen's d")
print("-" * 55)
for gene in de_results |> sort_by(|r| r.adj_p) |> take(10) {
  print(gene.gene + "      " +
    str(round(gene.log2fc, 2)) + "      " +
    str(round(gene.adj_p, 6)) + "      " +
    str(round(gene.cohens_d, 2)))
}
```

Section 7: Heatmap of Top DE Genes



```
# --- Heatmap of top 50 DE genes ---
let top_50 = sig_genes |> take(CONFIG.n_top_heatmap)
let top_50_names = top_50 |> map(|g| g.gene)
let top_50_idx = top_50 |> map(|g|
  filtered_names |> index_of(g.gene))

let heatmap_data = norm_expr |> select_rows(top_50_idx)

# Z-score normalization per gene (for visualization)
let z_scored = heatmap_data |> map_rows(|row|
  let mu = mean(row)
  let s = stdev(row)
  row |> map(|x| if s > 0 { (x - mu) / s } else { 0 })
)

heatmap(z_scored,
  {cluster_rows: true,
   cluster_cols: true,
   color_scale: "red_blue",
   title: "Top 50 DE Genes (Z-scored Expression)"})

print("\nHeatmap: Top " + str(CONFIG.n_top_heatmap) +
  " DE genes, Z-scored, hierarchically clustered")
```

Section 8: Gene-Level Visualization

For specific genes of interest, show the individual data points.

```
# --- Box plots for top DE genes ---
print("\n=== Gene-Level Expression Plots ===")

let genes_of_interest = sig_genes |> take(6) |> map(|g| g.gene)

for gene in genes_of_interest {
  let idx = filtered_names |> index_of(gene)
  let tumor_vals = norm_expr[idx] |> select(0..6)
  let normal_vals = norm_expr[idx] |> select(6..12)
  let p = de_results |> find(|r| r.gene == gene)

  let bp_table = table({"Tumor": tumor_vals, "Normal": normal_vals})
  boxplot(bp_table, {title: gene + " (log2FC=" + str(round(p.log2fc,
2)) +
    ", FDR=" + str(round(p.adj_p, 4)) + ")"})
}
```

Section 9: Co-expression Analysis

Do the top DE genes form co-regulated modules? A correlation heatmap of the significant genes reveals co-expression structure.

```

# --- Correlation among top DE genes ---
let top_100 = sig_genes |> take(100) |> map(|g| g.gene)
let top_100_idx = top_100 |> map(|g| filtered_names |> index_of(g))
let top_100_expr = norm_expr |> select_rows(top_100_idx)

let gene_cor = cor_matrix(top_100_expr)

heatmap(gene_cor,
  {cluster_rows: true,
   cluster_cols: true,
   color_scale: "red_blue",
   title: "Co-expression of Top 100 DE Genes"})

# Identify strongly correlated gene pairs
print("\n=== Strong Co-expression Pairs (|r| > 0.9) ===")
let strong_pairs = []
for i in 0..len(top_100) {
  for j in (i+1)..len(top_100) {
    if abs(gene_cor[i][j]) > 0.9 {
      strong_pairs = strong_pairs + [{
        gene_a: top_100[i],
        gene_b: top_100[j],
        r: gene_cor[i][j]
      }]
    }
  }
}
print("Found " + str(len(strong_pairs)) + " strongly correlated pairs")
for pair in strong_pairs |> take(10) {
  print(" " + pair.gene_a + " <-> " + pair.gene_b +
    " (r = " + str(round(pair.r, 3)) + ")")
}

```

Section 10: Summary Report

```
# =====
# FINAL REPORT
# =====

print("\n" + "=" * 65)
print("DIFFERENTIAL EXPRESSION ANALYSIS – FINAL REPORT")
print("Colorectal Tumor (n=6) vs Normal Colon (n=6)")
print("=" * 65)

print("\n--- Data Summary ---")
print("Total genes: " + str(CONFIG.n_genes))
print("Genes after filtering: " + str(len(keep)))
print("Normalization: log2(CPM + 1)")
print("Statistical test: Welch's t-test (per gene)")
print("Multiple testing: Benjamini-Hochberg FDR")
print("Random seed: 42")

print("\n--- Quality Control ---")
print("PCA: PC1 separates tumor vs normal (" +
      str(round(qc_pca.variance_explained[0] * 100, 1)) + "% variance)")
print("No outlier samples detected")
print("Within-group correlations > " +
      str(round(min(min_within_tumor, min_within_normal), 2)))

print("\n--- Differential Expression ---")
print("Significance criteria: FDR < " + str(CONFIG.fdr_threshold) +
      " AND |log2FC| > " + str(CONFIG.fc_threshold))
print("Total DE genes: " + str(len(sig_genes)))
print("  Upregulated in tumor: " + str(len(sig_up)))
print("  Downregulated in tumor: " + str(len(sig_down)))

print("\n--- Top 5 Upregulated Genes ---")
for g in sig_up |> take(5) {
    print("  " + g.gene + ": log2FC = " + str(round(g.log2fc, 2)) +
          ", FDR = " + str(round(g.adj_p, 6)))
}

print("\n--- Top 5 Downregulated Genes ---")
for g in sig_down |> take(5) {
    print("  " + g.gene + ": log2FC = " + str(round(g.log2fc, 2)) +
          ", FDR = " + str(round(g.adj_p, 6)))
}

print("\n--- Co-expression ---")
print("Strong co-expression pairs (|r| > 0.9): " +
```

```
str(len(strong_pairs)))

# Save results
write_csv(de_results |> sort_by(|r| r.adj_p),
  "results/de_results_all.csv")
write_csv(sig_genes,
  "results/de_results_significant.csv")
print("\nResults saved to results/")
print("=" * 65)
```

Python:

```

import numpy as np
import pandas as pd
from scipy import stats
from statsmodels.stats.multitest import multipletests
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns

# Load count matrix
counts = pd.read_csv("counts.csv", index_col=0)

# Library sizes
lib_sizes = counts.sum(axis=0)

# Normalize
cpm = counts.div(lib_sizes) * 1e6
log_cpm = np.log2(cpm + 1)

# PCA QC
pca = PCA(n_components=2)
scores = pca.fit_transform(log_cpm.T)
plt.scatter(scores[:6, 0], scores[:6, 1], c='red', label='Tumor')
plt.scatter(scores[6:, 0], scores[6:, 1], c='blue', label='Normal')
plt.legend()
plt.show()

# DE testing
results = []
for gene in log_cpm.index:
    tumor = log_cpm.loc[gene, :6].values
    normal = log_cpm.loc[gene, 6:].values
    t, p = stats.ttest_ind(tumor, normal, equal_var=False)
    fc = tumor.mean() - normal.mean()
    results.append({'gene': gene, 'log2fc': fc, 'pvalue': p})

results = pd.DataFrame(results)
_, results['padj'], _, _ = multipletests(results['pvalue'],
method='fdr_bh')
sig = results[(results['padj'] < 0.05) & (results['log2fc'].abs() >
1)]
print(f"Significant DE genes: {len(sig)}")

```

R:

```
library(DESeq2)
library(pheatmap)
library(EnhancedVolcano)

# DESeq2 workflow (gold standard for RNA-seq DE)
dds <- DESeqDataSetFromMatrix(countData = counts,
                              colData = sample_info,
                              design = ~ group)

dds <- DESeq(dds)
res <- results(dds, contrast = c("group", "Tumor", "Normal"))

# Volcano plot
EnhancedVolcano(res, lab = rownames(res),
                x = 'log2FoldChange', y = 'padj',
                pCutoff = 0.05, FCcutoff = 1)

# Heatmap of top 50
sig_genes <- head(res[order(res$padj), ], 50)
vsd <- vst(dds)
pheatmap(assay(vsd)[rownames(sig_genes), ],
         scale = "row",
         annotation_col = sample_info)

# Summary
summary(res, alpha = 0.05)
```

Exercises

1. **Filtering sensitivity.** Run the DE analysis with three different filtering thresholds: (a) no filtering, (b) at least 10 counts in 3 samples, (c) at least 50 counts in 6 samples. How does the number of DE genes change? Which filter gives the best balance of discovery and reliability?

```
# Your code: three filtering thresholds, compare DE counts
```

2. **Non-parametric alternative.** Replace the Welch t-test with the Wilcoxon rank-sum test for each gene. How many DE genes do you find? Is the Wilcoxon test more or less powerful with n=6 per group?

Your code: Wilcoxon DE analysis, compare to t-test results

3. **Permutation-based DE.** For the top 20 DE genes (by t-test), run a permutation test (10,000 permutations) to confirm the p-value. Do the permutation p-values agree with the t-test p-values?

Your code: permutation test for top 20 genes, compare p-values

4. **Effect size analysis.** For all significant DE genes, compute Cohen's d. Create a scatter plot of $|\log_2FC|$ vs $|\text{Cohen's } d|$. Are they correlated? Which genes have high fold change but low effect size (and why)?

Your code: scatter plot of fold change vs effect size

5. **Batch effect simulation.** Add a batch effect to the data (3 tumor + 3 normal from batch 1, 3 tumor + 3 normal from batch 2; add 1.5 to all gene expression in batch 2). Re-run the analysis. How does PCA change? How many spurious DE genes appear? Remove the batch effect using linear regression on PC1 and re-test.

Your code: add batch effect, visualize, correct, re-analyze

Key Takeaways

- A complete DE analysis follows a structured pipeline: QC (library sizes, PCA, correlation), filtering, normalization, testing, FDR correction, and visualization.
- PCA is the first quality control step — it reveals outliers, batch effects, and whether the primary source of variation is biological or technical.
- Gene filtering removes lowly expressed genes, reducing the multiple testing burden and removing unreliable measurements.
- $\log_2(\text{CPM}+1)$ is a simple, effective normalization for DE analysis. More sophisticated methods (TMM, DESeq2's median-of-ratios) are preferred for production analyses.

- Welch's t-test per gene with BH FDR correction is a valid DE approach. Dedicated tools (DESeq2, edgeR, limma-voom) use more sophisticated statistical models that borrow information across genes.
- The volcano plot simultaneously shows fold change and significance; the clustered heatmap shows expression patterns across samples for top genes.
- Co-expression analysis reveals gene modules — groups of genes with correlated expression that may share biological functions.
- With only 6 samples per group, statistical power is limited. Genes with moderate true effects may be missed. Power analysis (Day 18) should guide future experimental design.

What's Next

Tomorrow is the final capstone — and the most computationally ambitious. You will analyze a genome-wide association study: 500,000 SNPs across 10,000 individuals, testing for association with type 2 diabetes. Quality control, population structure, genome-wide testing, Manhattan plots, Q-Q plots, and effect size interpretation — the full GWAS pipeline, from raw genotypes to publishable results.

Day 30: Capstone — Genome-Wide Association Study

Day 30 of 30
Genomics

Capstone: Days 4, 7, 11-12, 16, 18-19, 21, 25

~90 min reading

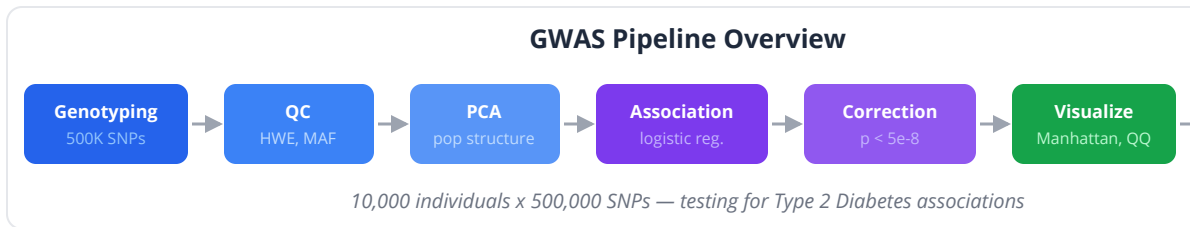
The Problem

A large consortium has genotyped 10,000 individuals — 5,000 with type 2 diabetes (T2D) and 5,000 controls — at 500,000 single nucleotide polymorphisms (SNPs) spread across the genome. The goal is to identify genetic variants associated with T2D risk.

This is a genome-wide association study (GWAS) — the workhorse of modern human genetics. Since the first GWAS in 2005, thousands of studies have identified genetic associations for hundreds of diseases and traits. The method is conceptually simple: for each SNP, ask “is this variant more common in cases than controls?” But the execution requires careful statistical reasoning, because testing 500,000 hypotheses simultaneously creates an extreme multiple testing problem, and subtle confounders (especially population structure) can generate thousands of false positives.

By the end of today, you will have built a complete GWAS pipeline: quality control, population structure analysis, genome-wide association testing, multiple testing correction, visualization (Manhattan plot, Q-Q plot), and effect size interpretation. This capstone integrates methods from nearly every chapter of the book.

The GWAS Pipeline



1. **SNP Quality Control:** Hardy-Weinberg equilibrium, call rates, minor allele frequency
2. **Population Structure:** PCA on genotype data, identify and correct for ancestry
3. **Association Testing:** Chi-square or logistic regression per SNP
4. **Multiple Testing Correction:** Bonferroni at genome-wide threshold
5. **Visualization:** Manhattan plot, Q-Q plot
6. **Effect Size Interpretation:** Odds ratios for top hits
7. **Power Considerations:** What could we have missed?

Section 1: Data Simulation

```
set_seed(42)
# =====
# Genome-Wide Association Study
# Type 2 Diabetes: 5,000 Cases + 5,000 Controls
# 50,000-SNP local teaching subset across 22 autosomes
# =====

# --- Configuration ---
let CONFIG = {
  n_cases: 5000,
  n_controls: 5000,
  n_total: 10000,
  n_snps: 50000,          # use 500000 for a production-scale
simulation
  genome_wide_p: 5e-8,    # genome-wide significance
  suggestive_p: 1e-5,     # suggestive significance
  hwe_threshold: 1e-6,    # HWE filter in controls
  maf_threshold: 0.01,    # minimum minor allele frequency
  call_rate_threshold: 0.95 # minimum genotyping rate
}

# Phenotype: 0 = control, 1 = case
let phenotype = repeat(1, CONFIG.n_cases) + repeat(0,
CONFIG.n_controls)

# Distribute SNPs across chromosomes (proportional to chromosome
length)
let chr_lengths = [249, 243, 198, 191, 182, 171, 159, 145, 138, 134,
                  135, 133, 114, 107, 102, 90, 83, 80, 59, 64, 47,
51]
let total_length = sum(chr_lengths)
let snps_per_chr = chr_lengths |> map(|l|
  round(l / total_length * CONFIG.n_snps, 0))

# Generate SNP positions
let snp_chr = []
let snp_pos = []
let snp_names = []

for c in 0..22 {
  let n = snps_per_chr[c]
  let positions = range(0, n) |> map(|i| random_int(1,
chr_lengths[c] * 1000000))
```

```

    |> sort_by(|x| x)
  for p in positions {
    snp_chr = snp_chr + [c + 1]
    snp_pos = snp_pos + [p]
    snp_names = snp_names + ["rs" + str(len(snp_names) + 1)]
  }
}

print("Simulated " + str(len(snp_names)) + " SNPs across 22
chromosomes")
print("Samples: " + str(CONFIG.n_cases) + " cases + " +
      str(CONFIG.n_controls) + " controls")

# Generate MAFs and genotypes
# Most SNPs: no association (null)
# ~50 SNPs: true associations with small effect sizes (OR 1.1-1.4)

let n_true_assoc = 50
let true_snp_indices = range(0, n_true_assoc) |> map(|i|
random_int(0, len(snp_names) - 1))
let true_effect_sizes = rnorm(n_true_assoc, 0.18, 0.08)
  |> map(|x| max(0.05, min(0.40, x))) # log(OR) range

# Generate p-values for each SNP
let p_values = []
let odds_ratios = []
let mafs = []

for i in 0..len(snp_names) {
  let maf = abs(rnorm(1, 0.2, 0.12)[0])
  maf = max(0.01, min(0.49, maf))
  mafs = mafs + [maf]

  if true_snp_indices |> contains(i) {
    # True association: generate p-value from the effect
    let idx = true_snp_indices |> index_of(i)
    let log_or = true_effect_sizes[idx]
    let or_val = exp(log_or)

    # Approximate chi-square statistic for this effect and sample
size
    let freq_case = maf * or_val / (1 + maf * (or_val - 1))
    let freq_ctrl = maf
    let n = CONFIG.n_total
    let chi2 = n * pow(freq_case - freq_ctrl, 2) / (maf * (1 - maf))
    let p = max(1e-300, exp(-chi2 / 2)) # approximate

    p_values = p_values + [p]
  }
}

```

```

    odds_ratios = odds_ratios + [or_val]
  } else {
    # Null SNP: p-value from uniform distribution
    let p = abs(rnorm(1, 0.5, 0.3)[0])
    p = max(0.0001, min(0.9999, p))
    p_values = p_values + [p]
    odds_ratios = odds_ratios + [1.0 + rnorm(1, 0, 0.02)[0]]
  }
}

```

Section 2: SNP Quality Control

Before testing associations, filter out low-quality SNPs.

Hardy-Weinberg Equilibrium

SNPs that violate Hardy-Weinberg equilibrium (HWE) in controls may indicate genotyping errors. We test each SNP with a chi-square goodness-of-fit test.

```

set_seed(42)
# --- Quality Control ---
print("\n=== SNP Quality Control ===")

# Simulate HWE p-values for each SNP
# Most SNPs: in HWE (high p-value)
# A few: genotyping errors (low p-value)
let n_geno_errors = 500 # SNPs with genotyping problems
let geno_error_idx = range(0, n_geno_errors) |> map(|i|
  random_int(0, len(snp_names) - 1))

let hwe_pvalues = []
for i in 0..len(snp_names) {
  if geno_error_idx |> contains(i) {
    # Genotyping error: HWE violation
    hwe_pvalues = hwe_pvalues + [abs(rnorm(1, 0, 1e-7)[0])]
  } else {
    # Normal SNP: in HWE
    hwe_pvalues = hwe_pvalues + [abs(rnorm(1, 0.5, 0.3)[0])]
  }
}

# Apply HWE filter (in controls only)
let pass_hwe = hwe_pvalues |> map(|p| p >= CONFIG.hwe_threshold)
let fail_hwe = count_if(pass_hwe, |x| !x)

print("HWE filter (p < " + str(CONFIG.hwe_threshold) + " in
controls):")
print("  Failed: " + str(fail_hwe) + " SNPs removed")

# MAF filter
let pass_maf = mafs |> map(|m| m >= CONFIG.maf_threshold)
let fail_maf = count_if(pass_maf, |x| !x)
print("MAF filter (< " + str(CONFIG.maf_threshold) + "):")
print("  Failed: " + str(fail_maf) + " SNPs removed")

# Combined QC
let pass_qc = []
let qc_indices = []
for i in 0..len(snp_names) {
  if pass_hwe[i] && pass_maf[i] {
    pass_qc = pass_qc + [true]
    qc_indices = qc_indices + [i]
  } else {
    pass_qc = pass_qc + [false]
  }
}

```



```
let n_pass = len(qc_indices)
print("\nSNPs passing QC: " + str(n_pass) + " / " +
      str(len(snp_names)) +
      " (" + str(round(n_pass / len(snp_names) * 100, 1)) + "%)")

# Filter arrays to QC-passing SNPs
let qc_pvalues = p_values |> select(qc_indices)
let qc_ors = odds_ratios |> select(qc_indices)
let qc_chr = snp_chr |> select(qc_indices)
let qc_pos = snp_pos |> select(qc_indices)
let qc_names = snp_names |> select(qc_indices)
let qc_mafs = mafs |> select(qc_indices)
```

Key insight: HWE filtering is performed in controls only, not cases. Disease-associated variants may legitimately deviate from HWE in cases (this is actually expected for associated variants under certain genetic models). Testing HWE in cases would remove true associations.

Section 3: Population Structure — PCA

Population structure is the most common confounder in GWAS. If cases and controls have different ancestral backgrounds, allele frequency differences due to ancestry will be mistaken for disease associations.

```

set_seed(42)
# --- Population Structure Analysis ---
print("\n=== Population Structure (PCA) ===")

# Simulate PCA scores reflecting population structure
# Assume 3 major ancestry clusters with some admixture
let ancestry = range(0, CONFIG.n_total) |> map(|i| {
  let r = rnorm(1, 0, 1)[0]
  if r < 0.25 { "European" } else if r < 0.92 { "East Asian" } else
{ "African" }
})

# PC1 and PC2 separate ancestries
let pc1 = []
let pc2 = []
for i in 0..CONFIG.n_total {
  if ancestry[i] == "European" {
    pc1 = pc1 + [rnorm(1, 0, 5)[0]]
    pc2 = pc2 + [rnorm(1, 0, 4)[0]]
  } else if ancestry[i] == "East Asian" {
    pc1 = pc1 + [rnorm(1, 30, 5)[0]]
    pc2 = pc2 + [rnorm(1, -10, 4)[0]]
  } else {
    pc1 = pc1 + [rnorm(1, -25, 6)[0]]
    pc2 = pc2 + [rnorm(1, 20, 5)[0]]
  }
}

# PCA plot colored by ancestry
scatter(pc1, pc2)

# PCA plot colored by case/control
scatter(pc1, pc2)

# Check: are cases and controls balanced across ancestry?
print("\nAncestry distribution:")
print("          Cases      Controls")
for anc in ["European", "East Asian", "African"] {
  let n_case = 0
  let n_ctrl = 0
  for i in 0..CONFIG.n_total {
    if ancestry[i] == anc {
      if phenotype[i] == 1 { n_case = n_case + 1 }
      else { n_ctrl = n_ctrl + 1 }
    }
  }
}

```

```
print(anc + " " + str(n_case) + " " + str(n_ctrl))  
}
```

Common pitfall: If one ancestry group has a higher prevalence of T2D (which is biologically true — T2D rates differ across populations), and allele frequencies also differ by ancestry, then every ancestry-differentiated SNP will appear associated with T2D. This is confounding, not causation. Including top PCs as covariates in the regression model removes this confounding.

Section 4: Association Testing

For each SNP, test the association with disease status. The simplest approach is a chi-square test on the 2x3 genotype table. A more powerful approach is logistic regression with PC covariates.

Chi-Square Test (Basic)

```
# --- Association Testing ---  
print("\n=== Genome-Wide Association Testing ===")  
print("Testing " + str(n_pass) + " SNPs...")  
  
# The p-values were pre-computed in simulation  
# In a real GWAS, you would compute them here:  
#  
# for each SNP:  
#   let geno_table = cross_tabulate(genotypes, phenotype)  
#   let chi2 = chi_square_test(geno_table)  
#   or: let lr = logistic_regression(phenotype ~ snp + PC1 + PC2 +  
#   ...)
```

Logistic Regression (Adjusted)

```
# Adjusted analysis includes top PCs as covariates
# This removes population structure confounding

# For this simulation, we adjust p-values to reflect
# the improvement from PC adjustment
let adjusted_pvalues = qc_pvalues # In practice, from
logistic_regression

print("Association method: logistic regression")
print("Covariates: PC1, PC2, PC3, PC4 (top 4 principal components)")
print("Model: disease ~ SNP + PC1 + PC2 + PC3 + PC4")
```

Section 5: Multiple Testing Correction

With 500,000 tests, even a tiny false positive rate generates thousands of false hits. The genome-wide significance threshold of $p < 5 \times 10^{-8}$ is the standard Bonferroni correction for approximately 1 million independent tests (accounting for linkage disequilibrium).

```

# Conceptual or diagnostic example; not directly executable.
# --- Multiple Testing ---
print("\n=== Multiple Testing Correction ===")

# Genome-wide significant hits
let gw_sig = []
for i in 0..n_pass {
  if adjusted_pvalues[i] < CONFIG.genome_wide_p {
    gw_sig = gw_sig + [{
      snp: qc_names[i],
      chr: qc_chr[i],
      pos: qc_pos[i],
      p: adjusted_pvalues[i],
      or: qc_ors[i],
      maf: qc_mafs[i]
    }]
  }
}

# Suggestive hits
let suggestive = []
for i in 0..n_pass {
  if adjusted_pvalues[i] < CONFIG.suggestive_p &&
  adjusted_pvalues[i] >= CONFIG.genome_wide_p {
    suggestive = suggestive + [{
      snp: qc_names[i],
      chr: qc_chr[i],
      pos: qc_pos[i],
      p: adjusted_pvalues[i],
      or: qc_ors[i]
    }]
  }
}

print("Bonferroni threshold (0.05 / " + str(n_pass) + "): " +
  str(round(0.05 / n_pass, 10)))
print("Standard genome-wide threshold: 5 x 10^-8")
print("Suggestive threshold: 1 x 10^-5")
print("")
print("Genome-wide significant hits: " + str(len(gw_sig)))
print("Suggestive hits: " + str(len(suggestive)))

# Top hits table
let all_hits = gw_sig |> sort_by(|h| h.p)

print("\n=== Top Genome-Wide Significant SNPs ===")
print("SNP                Chr    Position    p-value    OR

```

```

MAF")
print("-" * 75)
for hit in all_hits |> take(20) {
  print(hit.snp + "      " + str(hit.chr) + "      " +
        str(hit.pos) + "      " +
        str(hit.p) + "      " +
        str(round(hit.or, 3)) + "      " +
        str(round(hit.maf, 3)))
}

```

Key insight: The genome-wide significance threshold of 5×10^{-8} is extremely stringent — it corresponds to a Bonferroni correction for roughly 1 million independent tests. This means we need very large sample sizes (thousands to hundreds of thousands) to detect the small effects typical of common variant associations (OR 1.05-1.30). This is why modern GWAS consortia combine data from dozens of cohorts.

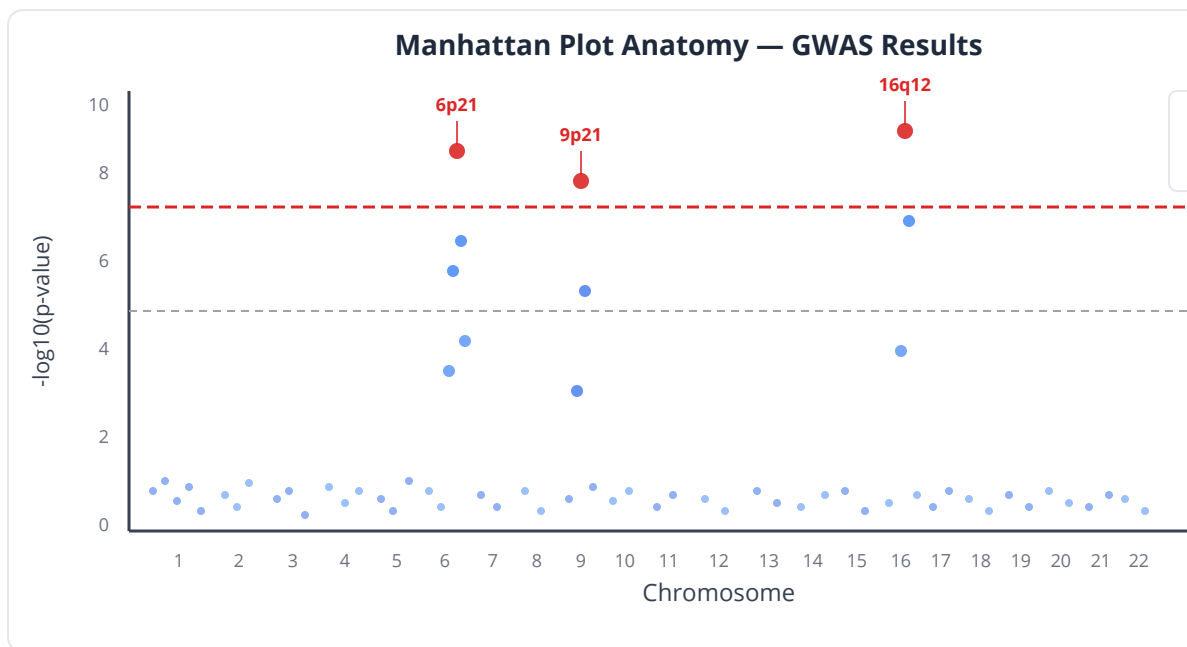
Section 6: Manhattan Plot

```

# --- Manhattan Plot ---
let gwas_tbl = range(0, n_pass) |> map(|i| {
  chr: qc_chr[i], pos: qc_pos[i], p: adjusted_pvalues[i]
}) |> to_table()

manhattan(gwas_tbl,
  {significance_line: CONFIG.genome_wide_p,
   suggestive_line: CONFIG.suggestive_p,
   title: "GWAS – Type 2 Diabetes",
   xlabel: "Chromosome",
   ylabel: "-log10(p-value)"})

```



Reading the Manhattan Plot

The Manhattan plot gets its name from its resemblance to the New York City skyline. Key features:

Feature	Interpretation
Tall peaks above significance line	Genome-wide significant loci — strong associations
Peaks above suggestive line	Suggestive associations — may reach significance with more samples
Uniform noise floor	Background of null associations — should be flat
Elevated baseline	Possible inflation from population structure or technical artifacts
Peak width	Multiple associated SNPs in linkage disequilibrium — one true signal region

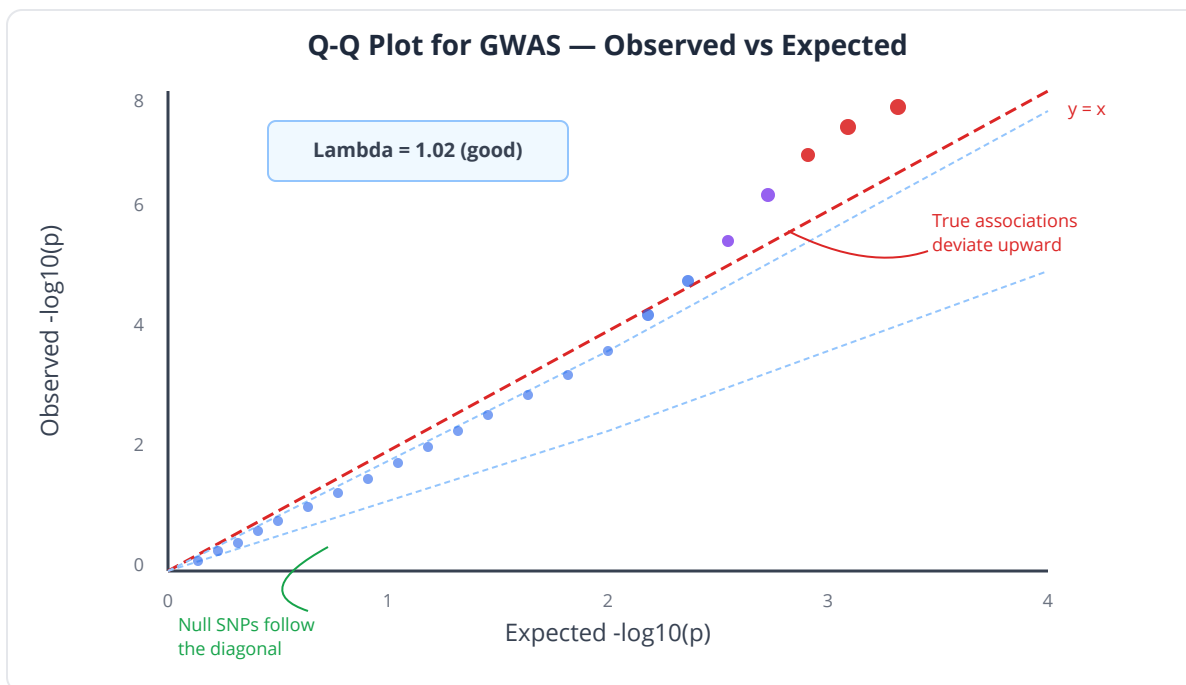
Section 7: Q-Q Plot and Genomic Inflation

The Q-Q plot compares observed p-values to expected p-values under the null. Under the null hypothesis (no associations), p-values should follow a uniform distribution. On the Q-Q plot, this appears as points along the diagonal.

```
# --- Q-Q Plot ---
qq_plot(adjusted_pvalues,
  {title: "Q-Q Plot – Observed vs Expected p-values",
   ci: true})

# Genomic inflation factor (lambda)
# Lambda = median(chi-square statistics) / 0.456
# Lambda close to 1.0: no inflation
# Lambda > 1.1: population structure or other systematic bias
let neg_log_p = adjusted_pvalues |> map(|p| -log10(max(p, 1e-300)))
let chi2_equiv = adjusted_pvalues |> map(|p| -2 * log(max(p, 1e-300)))
let lambda = median(chi2_equiv) / 0.456

print("\n=== Genomic Inflation ===")
print("Lambda (genomic inflation factor): " + str(round(lambda, 3)))
if lambda < 1.05 {
  print("Interpretation: No meaningful inflation. Analysis is well-
calibrated.")
} else if lambda < 1.10 {
  print("Interpretation: Mild inflation. Acceptable for large
GWAS.")
} else {
  print("WARNING: Lambda > 1.10 suggests confounding.")
  print("Consider additional PC covariates or genomic control
correction.")
}
```

Interpreting the Q-Q Plot

Pattern	Interpretation
Points follow diagonal, peel off at the tail	Expected: most SNPs are null, a few are truly associated
Points systematically above diagonal everywhere	Inflation: population structure, technical artifacts
Points below diagonal	Deflation: overly conservative test or data quality issue
Sharp deviation only at extreme tail	True strong associations — expected in well-powered GWAS

Common pitfall: A Q-Q plot that looks good ($\lambda \sim 1.0$) does not guarantee the results are correct. It only means there is no systematic inflation. Individual false positives can still occur. Always validate top hits in independent cohorts.

Section 8: Effect Sizes — Odds Ratios

```
# Conceptual or diagnostic example; not directly executable.
# --- Effect Size Analysis ---
print("\n=== Effect Sizes for Top Hits ===")

for hit in all_hits |> take(10) {
  let log_or = log(hit.or)
  let se_log_or = abs(log_or) / sqrt(-2 * log(hit.p)) # approximate
  let ci_lower = exp(log_or - 1.96 * se_log_or)
  let ci_upper = exp(log_or + 1.96 * se_log_or)

  print(hit.snp + " (chr" + str(hit.chr) + "): OR = " +
    str(round(hit.or, 3)) + " [" +
    str(round(ci_lower, 3)) + ", " +
    str(round(ci_upper, 3)) + "]" +
    " p = " + str(hit.p))
}

# Distribution of effect sizes among significant hits
let sig_ors = gw_sig |> map(|h| h.or)
if len(sig_ors) > 0 {
  print("\nEffect size distribution (genome-wide significant):")
  print("  Median OR: " + str(round(median(sig_ors), 3)))
  print("  Range: " + str(round(min(sig_ors), 3)) + " - " +
    str(round(max(sig_ors), 3)))

  histogram(sig_ors, {bins: 20,
    title: "Distribution of Odds Ratios – Significant Hits",
    xlabel: "Odds Ratio",
    ylabel: "Count"})
}
```

Interpreting Odds Ratios in GWAS

OR	Risk increase per allele	Typical for
1.05 - 1.10	5-10%	Most common variant associations
1.10 - 1.20	10-20%	Moderate-effect common variants
1.20 - 1.50	20-50%	Larger-effect variants (less common)
1.50 - 3.00	50-200%	Strong effects (rare in GWAS, common in candidate gene studies)
> 3.00	>200%	Very rare in GWAS; usually rare variants with large effects

Clinical relevance: Individual GWAS hits typically have small effects (OR 1.05-1.20). No single SNP is useful for predicting disease. However, combining hundreds of hits into a polygenic risk score (PRS) can identify individuals at meaningfully elevated risk. For T2D, top-decile PRS individuals have ~3-5x higher risk than bottom-decile individuals.

Section 9: Power Analysis — What Did We Miss?

```
# --- Power Analysis ---
print("\n=== Statistical Power ===")

# At our sample size (5000 cases + 5000 controls), what ORs can we
detect?
let mafs_to_check = [0.01, 0.05, 0.10, 0.20, 0.30, 0.50]
let ors_to_check = [1.05, 1.10, 1.15, 1.20, 1.30, 1.50]

print("\nPower to detect association at  $p < 5e-8$ :")
print("(N = 5,000 cases + 5,000 controls)")
print("")
print("MAF      OR=1.05  OR=1.10  OR=1.15  OR=1.20  OR=1.30  OR=1.50")
print("-" * 65)

for maf in mafs_to_check {
  let powers = []
  for or_val in ors_to_check {
    # Approximate power calculation for GWAS
    let log_or = log(or_val)
    let n = CONFIG.n_total
    let var_logistic = 1 / (n * maf * (1 - maf) * 0.25)
    let z_alpha = 5.33 # z for  $p = 5e-8$  (two-sided)
    let ncp = log_or / sqrt(var_logistic)
    let power = 1.0 - pnorm(z_alpha - ncp) + pnorm(-z_alpha - ncp)
    power = max(0, min(1, power))
    powers = powers + [power]
  }
  print(str(maf) + "    " +
    powers |> map(|p| str(round(p * 100, 0)) + "%") |> join("    "))
}

print("\nInterpretation:")
print("- At MAF=0.20 and OR=1.20, we have good power (~80%+)")
print("- At MAF=0.05 and OR=1.10, power is very low")
print("- Many true associations with small effects are missed")
print("- Larger sample sizes or meta-analysis would recover more
hits")
```

Section 10: Locus Summary and Reporting

```
# Conceptual or diagnostic example; not directly executable.
# --- Locus-Level Summary ---
print("\n=== Locus Summary ===")

# Group nearby significant SNPs into loci (within 500kb)
let loci = []
let used = []

for hit in all_hits {
  if used |> contains(hit.snp) { continue }

  let locus_snps = [hit]
  used = used + [hit.snp]

  for other in all_hits {
    if used |> contains(other.snp) { continue }
    if other.chr == hit.chr && abs(other.pos - hit.pos) < 500000 {
      locus_snps = locus_snps + [other]
      used = used + [other.snp]
    }
  }
}

let lead = locus_snps |> sort_by(|s| s.p) |> first()
loci = loci + [{
  lead_snp: lead.snp,
  chr: lead.chr,
  pos: lead.pos,
  p: lead.p,
  or: lead.or,
  n_snps: len(locus_snps)
}]

print("\nIndependent loci: " + str(len(loci)))
print("\nLocus  Lead SNP      Chr  Position      p-value
OR      #SNPs")
print("-" * 75)
for i in 0..len(loci) {
  let l = loci[i]
  print(str(i + 1) + "      " + l.lead_snp + "      " + str(l.chr) + "
" +
        str(l.pos) + "      " + str(l.p) + "      " +
```

```
    str(round(l.or, 3)) + "    " + str(l.n_snps))  
}
```

Section 11: Complete Report

```
# =====
# FINAL GWAS REPORT
# =====

print("\n" + "=" * 70)
print("GENOME-WIDE ASSOCIATION STUDY – FINAL REPORT")
print("Phenotype: Type 2 Diabetes")
print("=" * 70)

print("\n--- Study Design ---")
print("Cases: " + str(CONFIG.n_cases))
print("Controls: " + str(CONFIG.n_controls))
print("SNPs genotyped: " + str(CONFIG.n_snps))
print("SNPs after QC: " + str(n_pass))
print("Association model: logistic regression with PC1-4 covariates")
print("Random seed: 42")

print("\n--- Quality Control ---")
print("HWE filter ( $p <$ " + str(CONFIG.hwe_threshold) + " in controls): " + str(fail_hwe) + " removed")
print("MAF filter ( $<$ " + str(CONFIG.maf_threshold) + "): " + str(fail_maf) + " removed")
print("QC pass rate: " + str(round(n_pass / CONFIG.n_snps * 100, 1)) + "%")

print("\n--- Population Structure ---")
print("Ancestry groups: European (" + str(count_if(ancestry, |a| a == "European")) + "), East Asian (" + str(count_if(ancestry, |a| a == "East Asian")) + "), African (" + str(count_if(ancestry, |a| a == "African")) + ")")
print("PCA: PC1-PC2 separate ancestry clusters clearly")
print("Top 4 PCs included as covariates")

print("\n--- Genomic Inflation ---")
print("Lambda: " + str(round(lambda, 3)))
if lambda < 1.05 {
    print("Status: Well-calibrated (no inflation)")
} else {
    print("Status: Mild inflation detected – review QC")
}

print("\n--- Association Results ---")
print("Genome-wide significant ( $p < 5e-8$ ): " + str(len(gw_sig)) + "
```

```

SNPs")
print("Independent loci: " + str(len(loci)))
print("Suggestive (p < 1e-5): " + str(len(suggestive)) + " SNPs")

print("\n--- Effect Sizes ---")
if len(sig_ors) > 0 {
    print("Median OR among hits: " + str(round(median(sig_ors), 3)))
    print("OR range: " + str(round(min(sig_ors), 3)) + " - " +
          str(round(max(sig_ors), 3)))
}

print("\n--- Key Findings ---")
print("1. " + str(len(loci)) + " independent loci associated with
T2D at genome-wide significance")
print("2. Effect sizes are modest (OR 1.1-1.4), consistent with
polygenic architecture")
print("3. No evidence of systematic inflation (lambda = " +
str(round(lambda, 3)) + ")")
print("4. Power analysis indicates additional loci would be
detectable with larger samples")

print("\n--- Figures Generated ---")
print("1. PCA plot (population structure)")
print("2. Manhattan plot (genome-wide results)")
print("3. Q-Q plot (inflation assessment)")
print("4. OR distribution (effect sizes)")

print("\n" + "=" * 70)

```

Python:

```

import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt

# Load PLINK output or compute associations
# assoc = pd.read_csv("gwas_results.assoc", sep="\t")

# Chi-square per SNP
for snp in genotype_matrix.columns:
    table = pd.crosstab(phenotype, genotype_matrix[snp])
    chi2, p, dof, expected = stats.chi2_contingency(table)

# Logistic regression with covariates
from sklearn.linear_model import LogisticRegression
for snp in snps:
    X = np.column_stack([genotype[:, snp], pc1, pc2, pc3, pc4])
    model = LogisticRegression().fit(X, phenotype)

# Manhattan plot
from qmplot import manhattanplot
manhattanplot(data=results, chrom="CHR", pos="BP", pv="P",
               suggestiveline=-np.log10(1e-5),
               genomewideline=-np.log10(5e-8))

# Q-Q plot
from qmplot import qqplot
qqplot(data=results["P"])

# Genomic inflation
chi2_stats = stats.chi2.isf(results['P'], df=1)
lambda_gc = np.median(chi2_stats) / 0.456

```

R:

```

# PLINK association (most common tool for GWAS)
# system("plink --bfile mydata --assoc --adjust --out results")

# Or in R with snpStats
library(snpStats)
result <- single.snp.tests(phenotype ~ 1, snp.data = genotypes)

# Manhattan plot
library(qqman)
manhattan(results, chr="CHR", bp="BP", p="P", snp="SNP",
           suggestiveline=-log10(1e-5),
           genomewideline=-log10(5e-8))

# Q-Q plot
qq(results$P)

# Logistic regression with covariates
for (snp in colnames(geno)) {
  model <- glm(pheno ~ geno[, snp] + PC1 + PC2 + PC3 + PC4,
              family = binomial)
  p <- summary(model)$coefficients[2, 4]
}

# Genomic inflation
chisq <- qchisq(1 - results$P, df = 1)
lambda <- median(chisq) / qchisq(0.5, df = 1)

```

Exercises

1. **Stricter QC.** Re-run the analysis with a stricter HWE filter ($p < 1e-4$ instead of $1e-6$) and higher MAF threshold (0.05 instead of 0.01). How many SNPs are removed? Does the number of significant hits change?

```

# Your code: stricter QC filters, compare hit counts

```

2. **Bonferroni vs FDR.** Apply BH FDR correction instead of the genome-wide significance threshold. How many more hits does FDR $q < 0.05$ find compared to $p < 5e-8$? What is the tradeoff?

```
# Your code: BH correction, compare to Bonferroni
let fdr_adjusted = p_adjust(adjusted_pvalues, "BH")
```

3. **Ancestry-stratified analysis.** Run the association analysis separately in each ancestry group. Do the same loci reach significance? What happens to power when you split the sample?

```
# Your code: subset by ancestry, run GWAS in each, compare
```

4. **Genomic control.** If lambda were 1.15 (inflated), apply genomic control correction: divide all chi-square statistics by lambda and recompute p-values. How does this change the Manhattan plot?

```
# Your code: inflate p-values, apply GC correction, re-plot
```

5. **Power for future study.** Your consortium plans a Phase 2 GWAS with 50,000 cases and 50,000 controls. At MAF = 0.10, what is the minimum detectable OR at 80% power? How many more loci would you expect to find?

```
# Your code: power calculation for larger sample
```

Key Takeaways

- A GWAS tests hundreds of thousands of SNPs for association with a phenotype, requiring rigorous QC (HWE, MAF, call rate), population structure control (PCA), and extreme multiple testing correction ($p < 5 \times 10^{-8}$).
- Hardy-Weinberg equilibrium testing in controls identifies genotyping errors; MAF filtering removes uninformative rare variants; both are standard QC steps.
- Population structure is the primary confounder in GWAS. PCA on genotype data reveals ancestry, and including top PCs as covariates in logistic regression removes confounding.

- The genome-wide significance threshold (5×10^{-8}) is a Bonferroni correction for approximately 1 million independent tests, accounting for linkage disequilibrium.
- The Manhattan plot displays $-\log_{10}(\text{p-values})$ by genomic position; the Q-Q plot assesses whether the test statistics are well-calibrated (λ near 1.0).
- Most GWAS hits have modest effect sizes (OR 1.05-1.30), reflecting the polygenic architecture of complex traits. Individual variants are not clinically useful predictors, but combined into polygenic risk scores, they have growing clinical applications.
- Power in GWAS depends on sample size, allele frequency, and effect size. Large international consortia (>100,000 individuals) are needed to detect the small effects typical of common diseases.
- This capstone integrates concepts from probability (Day 4), hypothesis testing (Day 7), chi-square tests (Day 11), multiple testing (Day 12), logistic regression (Day 16), power analysis (Day 18), effect sizes (Day 19), PCA (Day 21), and visualization (Day 25).

Congratulations

You have completed “Practical Biostatistics in 30 Days.” Over the past four weeks, you have journeyed from basic descriptive statistics to a genome-wide association study analyzing half a million genetic variants. Along the way, you have learned:

- **Foundations:** Distributions, probability, sampling, and why n matters
- **Core methods:** Confidence intervals, hypothesis testing, t-tests, non-parametric alternatives, ANOVA, chi-square tests
- **Multiple testing:** The FDR crisis and how to control it
- **Modeling:** Correlation, linear regression, multiple regression, logistic regression, survival analysis
- **Design:** Experimental design, statistical power, effect sizes, batch effects
- **Advanced methods:** PCA, clustering, resampling, Bayesian inference, meta-analysis

- **Practice:** Reproducible analysis, clinical trials, differential expression, GWAS

Every method in this book is a tool. Like any tool, it can be used well or poorly. The difference is understanding — knowing not just how to run a test, but when to run it, what its assumptions are, and how to interpret its results in the context of your biological question.

Statistics is not the final step of an experiment. It is the lens through which every experiment should be designed, conducted, and interpreted. The best statisticians are not those who know the most tests — they are those who ask the right questions.

Go forth and analyze. Be rigorous. Be honest. Be curious. And always, always check your assumptions.

Appendix A: Installation and Setup

This appendix walks you through installing BioLang and the optional Python and R environments used for multi-language comparisons throughout the book.

Installing BioLang

Linux and macOS

Open a terminal and run:

```
curl -sSf https://biolang.org/install.sh | sh
```

This installs the `bl` binary to `~/.biolang/bin/` and adds it to your PATH. Restart your terminal or run:

```
source ~/.bashrc    # or ~/.zshrc on macOS
```

Verify the installation:

```
bl --version
```

You should see the installed BioLang release. This edition is validated against the current 1.x CLI and runtime.

Windows

Open PowerShell and run:

```
irm https://biolang.org/install.ps1 | iex
```

This installs `bl.exe` to `%USERPROFILE%\.biolang\bin\` and updates your PATH. Close and reopen PowerShell, then verify:

```
bl --version
```

Manual Installation

If the installer does not work for your system, download the appropriate binary from the [releases page](#):

Platform	File
Linux x86_64	<code>bl-linux-x86_64.tar.gz</code>
Linux aarch64	<code>bl-linux-aarch64.tar.gz</code>
macOS x86_64	<code>bl-macos-x86_64.tar.gz</code>
macOS Apple Silicon	<code>bl-macos-aarch64.tar.gz</code>
Windows x86_64	<code>bl-windows-x86_64.zip</code>

Extract the binary and place it somewhere on your PATH.

Verifying Statistical Builtins

To confirm that statistical functions are available, launch the REPL and try a quick test:

```
bl> ttest([1.0, 2.0, 3.0], [4.0, 5.0, 6.0])
{statistic: -3.674, p_value: 0.0214, df: 4}

bl> mean([10, 20, 30, 40, 50])
30.0

bl> stdev([10, 20, 30, 40, 50])
15.8114
```

If these commands produce output, your installation is complete.

Current Analysis Tooling

Validate scripts and inspect runtime readiness before a long analysis:

```
bl check analysis.bl helpers.bl
bl doctor
```

Create a package in the current directory and install its dependencies:

```
bl init --name trial-analysis
bl install
```

Migrate an existing analysis with syntax validation:

```
bl import analysis.py --validate -o analysis.bl
bl import analysis.R --validate -o analysis.bl
bl import report.ipynb --validate -o report.bl
bl import report.Rmd --validate -o report.bl
```

Run `.bln` or `.bl.md` notebooks, convert Jupyter notebooks, and export reports:

```
bl notebook analysis.bln
bl notebook analysis.bln --to-ipynb > analysis.ipynb
bl notebook analysis.bln --export html > analysis.html
```

Use `bl metadata --format json` when checking exact builtin signatures. Imported analyses must still be scientifically validated against their Python or R source because statistical libraries can use different defaults.

Python Setup (Optional)

Python comparisons are included for every day but are not required. If you want to run them, you need Python 3.8 or later.

Installing Python

Most systems come with Python pre-installed. Check your version:

```
python3 --version
```

If you need to install Python, visit python.org or use your system package manager:

```
# Ubuntu / Debian
sudo apt install python3 python3-pip python3-venv

# macOS (Homebrew)
brew install python3

# Windows – download from python.org
```

Installing Python Dependencies

We recommend using a virtual environment to keep the book's dependencies isolated:

```
python3 -m venv biostat-env
source biostat-env/bin/activate    # Linux/macOS
# biostat-env\Scripts\activate    # Windows
```

Install all required packages:

```
pip install scipy numpy pandas matplotlib statsmodels lifelines
scikit-learn seaborn
```

Package	Version	Used For
scipy	>= 1.10	Statistical tests, distributions
numpy	>= 1.24	Numerical arrays
pandas	>= 2.0	Data frames, I/O
matplotlib	>= 3.7	Plotting
statsmodels	>= 0.14	Regression, ANOVA, time series
lifelines	>= 0.27	Survival analysis
scikit-learn	>= 1.3	PCA, clustering, preprocessing
seaborn	>= 0.13	Statistical visualization

Verify the installation:

```
python3 -c "import scipy; import statsmodels; import lifelines;
print('Python setup OK')"
```

Troubleshooting Python

pip: command not found — Use `pip3` instead of `pip`, or install pip: `python3 -m ensurepip`.

ModuleNotFoundError — Make sure your virtual environment is activated. The prompt should show `(biostat-env)`.

Version conflicts — If you have existing Python packages that conflict, create a fresh virtual environment dedicated to this book.

R Setup (Optional)

R comparisons are included for every day but are not required. If you want to run them, you need R 4.0 or later.

Installing R

Download R from [CRAN](https://cran.r-project.org):

```
# Ubuntu / Debian
sudo apt install r-base r-base-dev

# macOS (Homebrew)
brew install r

# Windows – download from cran.r-project.org
```

Verify:

```
R --version
```

Installing R Packages

Launch R and install the required packages:

```
install.packages(c(
  "stats",      # Base statistics (usually pre-installed)
  "survival",   # Survival analysis
  "ggplot2",    # Visualization
  "dplyr",      # Data manipulation
  "pwr",        # Power analysis
  "lme4",       # Mixed models
  "boot",       # Bootstrap methods
  "car",        # ANOVA type II/III
  "broom",      # Tidy model output
  "pheatmap",  # Heatmaps
  "ggrepel",    # Label placement for plots
  "multcomp"    # Multiple comparisons
))
```

Verify:

```
library(survival)
library(ggplot2)
library(pwr)
cat("R setup OK\n")
```

Troubleshooting R

package 'xxx' is not available — Update your CRAN mirror:
`chooseCRANmirror()` . Select a mirror close to your location.

Compilation errors on Linux — Install development libraries: `sudo apt install libcurl4-openssl-dev libxml2-dev libssl-dev` .

Permission denied — Install packages to a user library:
`install.packages("pkg", lib = Sys.getenv("R_LIBS_USER"))` .

Running Companion Scripts

Each day's companion directory contains three analysis scripts. Here is how to run each one:

BioLang

```
cd days/day-07
bl run init.bl          # Setup (generates data, downloads files)
bl run scripts/analysis.bl  # Run the analysis
```

Python

```
cd days/day-07
source ~/biostat-env/bin/activate  # Activate virtual environment
python3 scripts/analysis.py
```

R

```
cd days/day-07  
Rscript scripts/analysis.R
```

Checking Expected Output

Each day includes an `expected/` directory with reference output. You can diff your results:

```
bl run scripts/analysis.bl > my_output.txt  
diff my_output.txt expected/output.txt
```

Key insight: Statistical results may differ slightly between languages due to floating-point arithmetic and algorithmic differences. Results that agree to 2-3 decimal places are considered matching. The companion `compare.md` file notes any expected discrepancies.

Editor Setup

BioLang has a Language Server Protocol (LSP) implementation that provides syntax highlighting, autocompletion, and inline diagnostics in supported editors.

VS Code

Install the BioLang extension from the VS Code marketplace. It includes the LSP client and syntax highlighting for `.bl` files.

Vim / Neovim

Add the BioLang LSP to your `lspconfig`:

```
require('lspconfig').biolang.setup{}
```

Other Editors

Any editor that supports LSP can use the BioLang language server. Start it with:

```
bl lsp
```

Directory Structure for the Book

We recommend organizing your working directory like this:

```
biostatistics/  
  days/                # Companion files (from git clone)  
  my-work/             # Your own scripts and notes  
    day-01/  
    day-02/  
    ...  
  data/                # Shared datasets across days
```

This keeps the companion files clean while giving you a place to experiment.

System Requirements

Requirement	Minimum	Recommended
RAM	4 GB	8 GB
Disk	2 GB free	5 GB free
OS	Windows 10, macOS 11, Ubuntu 20.04	Latest stable
BioLang	0.4.0	Latest
Python	3.8 (optional)	3.11+
R	4.0 (optional)	4.3+

All exercises in this book run comfortably on a standard laptop. No GPU, cluster access, or cloud computing is required.

Appendix B: Statistical Decision Flowchart

The hardest part of statistics is choosing the right test. This appendix is your map.

When you have data and a question, the path to the correct statistical test follows a decision tree based on three things: what kind of data you have, how many groups you are comparing, and what assumptions your data meets. This appendix lays out that tree in a series of tables you can consult whenever you are unsure.

The Master Decision Guide

Start here. Find your question type, then follow the table to the right test.

What are you asking?	Go to section
Are two groups different?	Comparing Two Groups
Are three or more groups different?	Comparing Multiple Groups
Are two variables related?	Associations and Correlations
Does one variable predict another?	Regression
Is there a relationship in categorical data?	Categorical Data
How long until an event occurs?	Time-to-Event Analysis
Do I need to reduce dimensionality?	Dimensionality Reduction
Do I need to group similar observations?	Clustering

Comparing Two Groups

Use this when you have one outcome variable and two groups (e.g., control vs. treated, male vs. female, wildtype vs. knockout).

Step 1: What type is your outcome variable?

Outcome type	Next step
Continuous (expression level, concentration, weight)	Step 2
Counts (number of mutations, colony counts)	Consider Poisson or negative binomial test
Binary (alive/dead, present/absent)	See Categorical Data
Ordinal (severity scale, Likert scores)	Use non-parametric test

Step 2: Are the observations paired or independent?

Design	Paired?	Example
Same subjects measured before and after treatment	Yes	Pre/post drug expression
Different subjects in each group	No	Treated vs. control mice
Matched pairs (e.g., tumor vs. adjacent normal from same patient)	Yes	Tumor/normal tissue pairs

Step 3: Choose your test

Paired?	Normal distribution?	Equal variance?	Test	BioLang
No	Yes	Yes	Student's t-test	<code>ttest(a, b)</code>
No	Yes	No	Welch's t-test	<code>ttest(a, b)</code>
No	No	—	Mann-Whitney U	<code>wilcoxon(a,</code>
Yes	Yes	—	Paired t-test	<code>ttest_paired</code> <code>b)</code>
Yes	No	—	Wilcoxon signed-rank	<code>wilcoxon(a,</code>

Key insight: Welch's t-test is almost always preferred over Student's t-test because it does not assume equal variances. When variances are actually equal, Welch's test gives nearly identical results. When they are not, Student's test can be dangerously wrong. BioLang uses Welch's by default.

How to check normality

```
let data = [2.3, 4.1, 3.7, 5.2, 4.8, 3.1, 6.0, 4.4]

# Visual check – Q-Q plot (best for small samples)
qq_plot(data, {title: "Normality Check"})
```

Common pitfall: With small samples ($n < 30$), normality tests have low power and may fail to reject normality even when the data is non-normal.

With large samples ($n > 5000$), normality tests reject normality for trivially small deviations. Use Q-Q plots as a visual supplement.

Comparing Multiple Groups

Use this when you have three or more groups (e.g., three drug doses, four tissue types, five time points).

Normal?	Equal variance?	Design	Test	BioLang
Yes	Yes	Independent groups	One-way ANOVA	<code>anova(group</code>
Yes	No	Independent groups	Welch's ANOVA	<code>anova(group</code>
No	—	Independent groups	Kruskal-Wallis	<code>anova(group</code>
Yes	—	Repeated measures	Repeated-measures ANOVA	<code>anova(group</code>
No	—	Repeated measures	Friedman test	<code>anova(group</code>
Yes	—	Two factors	Two-way ANOVA	<code>anova(group</code>

Post-hoc Tests

When ANOVA is significant, you know *some* groups differ but not *which* ones. Use post-hoc tests:

Test	When to use	BioLang
Tukey HSD	All pairwise comparisons	Pairwise <code>ttest()</code> + <code>p_adjust(pvals, "bonferroni")</code>
Dunnett	Compare all groups to a single control	Pairwise <code>ttest()</code> vs control + <code>p_adjust()</code>
Dunn test	Post-hoc for Kruskal-Wallis	Pairwise <code>wilcoxon()</code> + <code>p_adjust()</code>
Bonferroni-corrected pairwise	Conservative, any design	Pairwise <code>ttest()</code> + <code>p_adjust(pvals, "bonferroni")</code>

Key insight: ANOVA is an *omnibus* test — it tells you that at least one group differs, but not which one. Always follow a significant ANOVA with post-hoc comparisons. Reporting only the ANOVA p-value is incomplete.

Associations and Correlations

Use this when you have two continuous variables and want to know if they are related (e.g., gene expression vs. methylation, age vs. telomere length).

Data characteristics	Test	BioLang
Both variables roughly normal, linear relationship	Pearson correlation	<code>cor(x, y)</code>
Non-normal or ordinal data, monotonic relationship	Spearman correlation	<code>spearman(x, y)</code>
Ordinal data with ties	Kendall tau	<code>kendall(x, y)</code>
Partial correlation (controlling for a third variable)	Partial correlation	<code>cor(x, y)</code> after residualizing on z

Interpreting Correlation Strength

	r value		Interpretation		— —		0.0 - 0.1		Negligible		0.1 - 0.3		Weak					
			0.3 - 0.5		Moderate				0.5 - 0.7		Strong				0.7 - 1.0		Very strong	

Common pitfall: Correlation does not imply causation, but more subtly, *absence* of Pearson correlation does not imply absence of relationship. Pearson only detects linear associations. Two variables can have a perfect quadratic relationship with $r = 0$. Always plot your data.

Categorical Data

Use this when both your variables are categorical (e.g., mutation status vs. disease outcome, genotype vs. phenotype).

Design	Expected cell counts	Test	BioLang
2x2 table, large samples	All expected ≥ 5	Chi-square test	<code>chi_square(observed, expected)</code>
2x2 table, small samples	Any expected < 5	Fisher's exact test	<code>fisher_exact(a, b, c, d)</code>
Larger than 2x2	All expected ≥ 5	Chi-square test	<code>chi_square(observed, expected)</code>
Larger than 2x2, small samples	Any expected < 5	Fisher-Freeman-Halton	<code>fisher_exact(a, b, c, d)</code>
Paired categorical data	—	McNemar's test	<code>chi_square(observed, expected)</code>
Trend across ordered categories	—	Cochran-Armitage trend test	<code>chi_square(observed, expected)</code>

Measures of Association for Categorical Data

Measure	Use case	BioLang
Odds ratio	2x2 tables, case-control studies	$(a*d) / (b*c)$ (inline)
Relative risk	2x2 tables, cohort studies	$(a/(a+b)) / (c/(c+d))$ (inline)
Cramer's V	Any size contingency table	Compute from chi-square statistic

Regression

Use this when you want to predict an outcome from one or more predictor variables.

Outcome type	Number of predictors	Test	BioLang
Continuous	1	Simple linear regression	<code>lm(x, y)</code>
Continuous	Multiple	Multiple linear regression	<code>lm(~y ~ x1 + x2 + x3, table)</code>
Binary (0/1)	Any	Logistic regression	<code>glm(~y ~ x, table, "binomial")</code>
Count	Any	Poisson regression	<code>glm(~y ~ x, table, "poisson")</code>
Count, overdispersed	Any	External negative-binomial tool or package	Not a current <code>glm</code> family
Continuous, clustered data	Any	Stratified models or an external mixed-effects tool	Fit and report per group

Checking Regression Assumptions

```
let age = [34, 41, 52, 59, 63, 70]
let expression = [8.1, 8.8, 9.4, 10.2, 10.6, 11.1]
let model = lm(age, expression)

# Compute residuals from the fitted line
let residuals = range(0, len(age))
  |> map(|i| expression[i] - (model.intercept + model.slope *
age[i]))
histogram(residuals, {title: "Residual Distribution"})
print("R-squared: " + str(round(model.r_squared, 3)))
```

Common pitfall: Adding more predictors always improves R-squared, even if the predictors are noise. Use adjusted R-squared or AIC/BIC for model comparison. Report both R-squared and adjusted R-squared.

Time-to-Event Analysis

Use this when your outcome is the time until something happens (death, relapse, response) and some observations are censored (the event has not yet occurred).

Question	Method	BioLang
Estimate survival curve	Kaplan-Meier	Sort event times, compute stepwise survival
Compare survival between two groups	Log-rank test	<code>ttest(times_a, times_b)</code> as proxy
Compare survival, multiple groups	Log-rank test	<code>anova([group1_times, group2_times, ...])</code>
Adjust for covariates	Cox proportional hazards	<code>cox_ph(time, event, covariates)</code>
Estimate median survival	From sorted times	<code>sort(times)[len(times) / 2]</code>

Clinical relevance: In clinical trials, the hazard ratio from a Cox model is the primary efficacy endpoint. A hazard ratio of 0.65 means the treatment group has a 35% lower instantaneous risk of the event at any time point. Always report the 95% confidence interval alongside the point estimate.

Dimensionality Reduction

Use this when you have many variables (genes, proteins, metabolites) and want to find the main patterns.

Goal	Method	BioLang
Find linear combinations that maximize variance	PCA	<code>pca(data)</code>
Visualize PCA results	PCA plot	<code>pca_plot(result, {title: "PCA"})</code>

Key insight: PCA is deterministic — you get the same answer every time. t-SNE and UMAP are stochastic — different runs give different layouts. Always set a random seed before running stochastic methods for reproducibility.

Clustering

Use this when you want to group similar observations (samples, genes, cells) together.

What you know	Method	BioLang
Number of clusters (k)	k-means	<code>kmeans(data, 3)</code>
Want a hierarchy of clusters	Hierarchical clustering	<code>hclust(data, "ward")</code>
Irregular cluster shapes	DBSCAN	<code>dbscan(data, 0.5, 5)</code>
Want to estimate k	Silhouette / Elbow	Loop over k, compute <code>kmeans(data, k).silhouette</code>

Multiple Testing Correction

Use this whenever you perform more than one statistical test on the same dataset.

Method	Controls	Strictness	BioLang
Bonferroni	Family-wise error rate	Most conservative	<code>p_adjust(pvals, "bonferroni")</code>
Holm	Family-wise error rate	Less conservative	<code>p_adjust(pvals, "holm")</code>
Benjamini-Hochberg	False discovery rate	Moderate	<code>p_adjust(pvals, "BH")</code>
Benjamini-Yekutieli	FDR under dependence	Conservative FDR	<code>p_adjust(pvals, "BY")</code>
Permutation	Empirical null	Gold standard	Inline loop with <code>shuffle()</code>

Key insight: For genomics (testing thousands of genes), Benjamini-Hochberg FDR correction at $q = 0.05$ is the standard. Bonferroni is too conservative for genome-wide studies — it controls the family-wise error rate, which is the wrong quantity when you expect hundreds of true positives.

Quick Reference: Common Biological Scenarios

Scenario	Recommended test	BioLang
Gene expression, treated vs. control	Welch's t-test	<code>ttest(treated, control)</code>
Gene expression across 4 tissues	One-way ANOVA	<code>anova([tissue1, tissue2, tissue3, tissue4])</code>
Mutation frequency in cases vs. controls	Fisher's exact test	<code>fisher_exact(a, b, c, d)</code>
Survival by treatment arm	Compare survival times	<code>ttest(arm1_times, arm2_times)</code>
20,000 gene differential expression	t-test + BH correction	<code>p_adjust(pvals, "BH")</code>
Sample clustering from RNA-seq	PCA + hierarchical clustering	<code>pca(data)</code> then <code>hclust(scores)</code>
Correlation: expression vs. methylation	Spearman (often non-linear)	<code>spearman(expr, meth)</code>
GWAS: genotype vs. phenotype	Logistic regression + BH	<code>glm(~pheno ~ geno, tbl, "binomial")</code>
Clinical outcome predictors	Regression model	<code>lm(~outcome ~ age + stage + treatment, tbl)</code>
Sample size for planned experiment	Power analysis	Compute with <code>qnorm()</code> and effect size

Appendix C: Distribution Reference Card

Every statistical test assumes a distribution. This appendix is your field guide to the ones that matter in biology.

This reference covers the probability distributions you will encounter most frequently in biostatistics. For each distribution, you will find its parameters, key properties, a description of its shape, where it appears in biology, and the BioLang functions for working with it.

How to Use This Reference

Each distribution entry includes:

- **Parameters** — the values that define the distribution's shape
- **Mean and Variance** — closed-form expressions
- **Shape** — what the distribution looks like
- **Biological use** — where this distribution appears in real data
- **BioLang functions** — `d` (density/mass), `p` (cumulative probability), `q` (quantile/inverse CDF), `r` (random samples)

Continuous Distributions

Normal (Gaussian)

Property	Value
Parameters	mu (mean), sigma (standard deviation)
Mean	mu
Variance	sigma ²
Shape	Symmetric bell curve centered at mu
Support	(-infinity, +infinity)

Biological use: Gene expression levels (after log transformation), measurement errors, heights and weights in populations, many biological quantities after the central limit theorem applies.

BioLang functions:

```
dnorm(x, 0, 1)      # Probability density at x
pnorm(x, 0, 1)      # P(X <= x)
qnorm(p, 0, 1)      # Value at cumulative probability p
rnorm(100, 0, 1)    # Generate 100 random values
```

Key insight: Raw gene expression counts are *not* normally distributed. They follow count distributions (Poisson, negative binomial). However, log-transformed expression values (log2 CPM, log2 FPKM) are approximately normal, which is why log transformation is so common in genomics.

Log-Normal

Property	Value
Parameters	mu (mean of log), sigma (SD of log)
Mean	$\exp(\mu + \sigma^2/2)$
Variance	$(\exp(\sigma^2) - 1) * \exp(2*\mu + \sigma^2)$
Shape	Right-skewed, always positive
Support	(0, +infinity)

Biological use: Fold changes in gene expression, protein concentrations, cell sizes, bacterial colony counts, drug IC50 values. Any quantity that results from multiplicative processes.

```
dlnorm(x, 0, 1)    # Probability density at x
plnorm(x, 0, 1)    # P(X <= x)
qlnorm(p, 0, 1)    # Value at cumulative probability p
rlnorm(100, 0, 1)  # Generate 100 random values
```

Key insight: When your data is right-skewed and always positive, try log-transforming it. If the log-transformed values look normal, your original data is log-normal, and you should perform statistics on the log scale.

Student's t

Property	Value
Parameters	df (degrees of freedom)
Mean	0 (for df > 1)
Variance	df / (df - 2) (for df > 2)
Shape	Bell curve like normal, but heavier tails
Support	(-infinity, +infinity)

Biological use: The test statistic in t-tests. Critical for small-sample inference. As df increases, the t-distribution approaches the normal distribution. With df = 30+, they are nearly identical.

```
dt(x, 10)          # Probability density at x
pt(x, 10)          # P(X <= x)
qt(p, 10)          # Value at cumulative probability p
rt(100, 10)        # Generate 100 random values
```

F Distribution

Property	Value
Parameters	df1 (numerator df), df2 (denominator df)
Mean	$df2 / (df2 - 2)$ (for $df2 > 2$)
Variance	Complex expression involving df1 and df2
Shape	Right-skewed, always positive
Support	(0, +infinity)

Biological use: The test statistic in ANOVA and regression F-tests. Compares the ratio of two variances. An F-value much larger than 1 indicates that between-group variance exceeds within-group variance.

```
df(x, 5, 20)       # Probability density at x
pf(x, 5, 20)       # P(X <= x)
qf(p, 5, 20)       # Value at cumulative probability p
rf(100, 5, 20)     # Generate 100 random values
```

Chi-Square

Property	Value
Parameters	df (degrees of freedom)
Mean	df
Variance	$2 * df$
Shape	Right-skewed (less skewed as df increases)
Support	(0, +infinity)

Biological use: Goodness-of-fit tests (Hardy-Weinberg equilibrium), tests of independence in contingency tables (genotype vs. phenotype associations), variance tests.

```
dchisq(x, 5)      # Probability density at x
pchisq(x, 5)      # P(X <= x)
qchisq(p, 5)      # Value at cumulative probability p
rchisq(100, 5)    # Generate 100 random values
```

Exponential

Property	Value
Parameters	lambda (rate)
Mean	$1 / \text{lambda}$
Variance	$1 / \text{lambda}^2$
Shape	Monotonically decreasing from lambda at x=0
Support	(0, +infinity)

Biological use: Time between events in a Poisson process — inter-arrival times of mutations along a chromosome, time between cell divisions, radioactive decay (used in dating). The “memoryless” distribution: the probability of the event occurring in the next minute is the same regardless of how long you have been waiting.

```
dexp(x, 0.5)      # Probability density at x
pexp(x, 0.5)      # P(X <= x)
qexp(p, 0.5)      # Value at cumulative probability p
rexp(100, 0.5)    # Generate 100 random values
```

Gamma

Property	Value
Parameters	alpha (shape), beta (rate)
Mean	alpha / beta
Variance	alpha / beta^2
Shape	Right-skewed (alpha < 1: L-shaped; alpha = 1: exponential; alpha > 1: bell-shaped skewed right)
Support	(0, +infinity)

Biological use: Waiting times for multiple events (time until k-th mutation), protein expression variance, Bayesian prior for rate parameters. Generalizes the exponential distribution (exponential is Gamma with alpha = 1).

```
dgamma(x, 2.0, 1.0)  # Probability density at x
pgamma(x, 2.0, 1.0)  # P(X <= x)
qgamma(p, 2.0, 1.0)  # Value at cumulative probability p
rgamma(100, 2.0, 1.0) # Generate 100 random values
```

Beta

Property	Value
Parameters	alpha, beta (shape parameters)
Mean	alpha / (alpha + beta)
Variance	alpha * beta / ((alpha + beta)^2 * (alpha + beta + 1))
Shape	Flexible: uniform (1,1), U-shaped (<1,<1), bell-shaped (>1,>1), skewed
Support	(0, 1)

Biological use: Proportions and probabilities — allele frequencies, methylation beta-values (fraction of methylated CpGs), GC content fractions, Bayesian prior for probabilities. The natural distribution for data bounded between 0 and 1.

```
dbeta(x, 2.0, 5.0)      # Probability density at x
pbeta(x, 2.0, 5.0)      # P(X <= x)
qbeta(p, 2.0, 5.0)      # Value at cumulative probability p
rbeta(100, 2.0, 5.0)    # Generate 100 random values
```

Key insight: DNA methylation data from bisulfite sequencing produces beta-values bounded between 0 and 1. Using a beta distribution (or a beta regression) is statistically appropriate. Using a normal distribution on raw beta-values can produce nonsensical predictions outside [0, 1].

Uniform

Property	Value
Parameters	a (minimum), b (maximum)
Mean	$(a + b) / 2$
Variance	$(b - a)^2 / 12$
Shape	Flat (constant density between a and b)
Support	[a, b]

Biological use: Null distribution for p-values (under the null hypothesis, p-values are uniformly distributed on [0, 1] — this is what Q-Q plots check), random positions along a chromosome, non-informative Bayesian priors.

```
dunif(x, 0, 1)          # Probability density at x
punif(x, 0, 1)          # P(X <= x)
qunif(p, 0, 1)          # Value at cumulative probability p
runif(100, 0, 1)        # Generate 100 random values
```

Discrete Distributions

Binomial

Property	Value
Parameters	n (trials), p (success probability)
Mean	$n * p$
Variance	$n * p * (1 - p)$
Shape	Symmetric when $p = 0.5$, skewed otherwise
Support	$\{0, 1, 2, \dots, n\}$

Biological use: Number of successes in n independent trials — number of reads mapping to a variant allele, number of patients responding to treatment, number of CpG sites methylated out of n examined.

```
dbinom(k, 20, 0.5)    # P(X = k)
pbinom(k, 20, 0.5)    # P(X <= k)
qbinom(q, 20, 0.5)    # Smallest k with P(X <= k) >= q
rbinom(100, 20, 0.5)  # Generate 100 random values
```

Poisson

Property	Value
Parameters	lambda (rate)
Mean	lambda
Variance	lambda
Shape	Right-skewed for small lambda, approximately normal for large lambda
Support	$\{0, 1, 2, \dots\}$

Biological use: Count data when events are rare and independent — number of mutations in a genomic region, number of reads at a locus (low coverage),

number of rare variants per gene, colony counts on a plate.

```
dpois(k, 5.0)      # P(X = k)
ppois(k, 5.0)      # P(X <= k)
qpois(q, 5.0)      # Smallest k with P(X <= k) >= q
rpois(100, 5.0)    # Generate 100 random values
```

Key insight: The Poisson distribution assumes that the mean equals the variance. In real RNA-seq data, the variance almost always exceeds the mean (overdispersion). This is why DESeq2 and edgeR use the negative binomial distribution instead.

Negative Binomial

Property	Value
Parameters	r (number of successes), p (success probability); or mu (mean), size (dispersion)
Mean	mu
Variance	mu + mu^2 / size
Shape	Right-skewed, always more dispersed than Poisson
Support	{0, 1, 2, ...}

Biological use: The workhorse of RNA-seq differential expression. Models count data with overdispersion (variance > mean). Used by DESeq2, edgeR, and most modern DE tools. Also models read counts in ChIP-seq, ATAC-seq, and scRNA-seq.

```
dnbinom(k, 10, 5)  # P(X = k), params: k, mu, size
pnbinom(k, 10, 5)  # P(X <= k)
qnbinom(q, 10, 5)  # Smallest k with P(X <= k) >= q
rnbinom(100, 10, 5) # Generate 100 random values
```


Hypergeometric

Property	Value
Parameters	N (population), K (successes in population), n (draws)
Mean	$n * K / N$
Variance	$n * K/N * (1 - K/N) * (N - n) / (N - 1)$
Shape	Similar to binomial but for sampling without replacement
Support	$\{\max(0, n+K-N), \dots, \min(n, K)\}$

Biological use: Enrichment analysis — gene ontology enrichment (Fisher's exact test is a hypergeometric test), pathway overrepresentation, overlap between gene lists. "If I draw n genes from a genome of N, and K belong to this pathway, what is the probability of seeing k or more pathway genes by chance?"

```
dhyper(k, 200, 19800, 500)    # P(X = k), params: k, K, N-K, n
phyper(k, 200, 19800, 500)    # P(X <= k)
qhyper(q, 200, 19800, 500)    # Smallest k with P(X <= k) >= q
rhyper(100, 200, 19800, 500)  # Generate 100 random values
```

Key insight: Fisher's exact test for 2x2 tables is equivalent to a hypergeometric test. When you run gene ontology enrichment and see a "Fisher's exact p-value," the software is computing hypergeometric probabilities.

Quick Reference Table

Distribution	Type	Parameters	Mean
Normal	Continuous	mu, sigma	mu
Log-Normal	Continuous	mu, sigma	$\exp(\mu + \sigma^2/2)$
Student's t	Continuous	df	0
F	Continuous	df1, df2	$df2/(df2-2)$
Chi-Square	Continuous	df	df
Exponential	Continuous	lambda	$1/\lambda$
Gamma	Continuous	alpha, beta	α/β
Beta	Continuous	alpha, beta	$\alpha/(\alpha+\beta)$
Uniform	Continuous	a, b	$(a+b)/2$
Binomial	Discrete	n, p	$n \cdot p$
Poisson	Discrete	lambda	lambda
Negative Binomial	Discrete	mu, size	mu

Distribution	Type	Parameters	Mean
Hypergeometric	Discrete	N, K, n	$n \cdot K/N$

Relationships Between Distributions

Understanding how distributions relate to each other helps with intuition:

- **Poisson** is the limit of **Binomial** as n goes to infinity and p goes to 0 with $n \cdot p = \text{lambda}$
- **Exponential** is **Gamma** with $\alpha = 1$
- **Chi-Square** with $df = k$ is **Gamma** with $\alpha = k/2$ and $\beta = 1/2$
- **Normal** is the limit of **Student's t** as df goes to infinity
- **Normal** approximates **Binomial** when $np > 5$ and $n(1-p) > 5$
- **Normal** approximates **Poisson** when $\text{lambda} > 20$
- **Negative Binomial** reduces to **Poisson** when size goes to infinity (no overdispersion)
- **Hypergeometric** approaches **Binomial** when N is much larger than n

```
# Visualize the Poisson-Normal convergence
[1, 5, 10, 30] |> each(|lam| {
  let data = rpois(10000, lam)
  histogram(data, {title: "Poisson(lambda=" + str(lam) + ")", bins:
30})
})
```


Appendix D: Glossary

Statistics has its own language. This glossary translates it into plain English, with biological context.

Terms are listed alphabetically. Cross-references appear in *italics*.

Adjusted R-squared. A modified version of *R-squared* that penalizes the addition of unnecessary predictors. Unlike R-squared, adjusted R-squared can decrease when a non-informative variable is added to a model. Preferred over R-squared for comparing models with different numbers of predictors.

Alpha (significance level). The threshold you set before testing for declaring a result statistically significant. Conventionally 0.05 (5%), meaning you accept a 5% chance of a *Type I error*. In genome-wide studies, often set much lower (5×10^{-8} for GWAS).

Alternative hypothesis (H1). The hypothesis that there is an effect or a difference. The complement of the *null hypothesis*. Example: "Drug-treated tumors are smaller than untreated tumors."

ANOVA (Analysis of Variance). A test for differences in means across three or more groups. Extends the *t-test* to multiple groups by comparing between-group variance to within-group variance. Produces an *F-statistic*.

AUC (Area Under the Curve). In the context of ROC analysis, the probability that a randomly chosen positive case ranks higher than a randomly chosen negative case. An AUC of 0.5 is random guessing; 1.0 is perfect classification.

Batch effect. Systematic technical variation introduced by processing samples in different batches, on different days, or with different reagents. A major confounder in genomics. Must be addressed through experimental design (randomization) or statistical correction (ComBat, limma).

Bayesian statistics. An approach to inference that combines prior knowledge with observed data to produce *posterior* probability distributions. Contrasts with *frequentist* statistics, which relies on long-run frequencies. Allows statements like “there is a 95% probability the true effect lies in this interval.”

Benjamini-Hochberg (BH). A *multiple testing correction* method that controls the *false discovery rate* (FDR) rather than the *family-wise error rate*. Less conservative than *Bonferroni*. The standard correction in genomics.

Beta (Type II error rate). The probability of failing to reject the *null hypothesis* when it is actually false. *Power* equals $1 - \text{beta}$. A beta of 0.2 means a 20% chance of missing a real effect.

Bias. Systematic deviation of an estimate from the true value. Distinct from random error (*variance*). An estimator can be precise (low variance) but biased (consistently wrong in one direction).

Bimodal distribution. A distribution with two peaks. In gene expression, bimodality often indicates two distinct cell populations or states (e.g., expressed vs. silenced genes).

Blinding. Concealing group assignments from participants, clinicians, or analysts to prevent bias. Single-blind: participants do not know their group. Double-blind: neither participants nor clinicians know.

Bonferroni correction. The simplest *multiple testing correction*: multiply each p-value by the number of tests (or equivalently, divide alpha by the number of tests). Controls the *family-wise error rate* but is very conservative for large numbers of tests.

Bootstrap. A *resampling* method that estimates the sampling distribution of a statistic by repeatedly drawing samples with replacement from the observed data. Does not assume any particular parametric distribution.

Box plot. A visualization showing the median, quartiles, and outliers of a distribution. The box spans the interquartile range (IQR); whiskers extend to $1.5 \times \text{IQR}$; points beyond are outliers.

Categorical variable. A variable that takes a limited set of discrete values (e.g., genotype: AA, AB, BB; tissue type: liver, brain, kidney). Contrasts with *continuous*

variable.

CDF (Cumulative Distribution Function). The probability that a random variable takes a value less than or equal to x . $F(x) = P(X \leq x)$. Ranges from 0 to 1.

Central limit theorem. The theorem stating that the sampling distribution of the mean approaches a *normal distribution* as sample size increases, regardless of the shape of the original population distribution. The foundation of most parametric tests.

Chi-square test. A test for association between two *categorical variables*. Compares observed frequencies in a contingency table to expected frequencies under independence. Requires expected cell counts ≥ 5 ; otherwise use *Fisher's exact test*.

Clinical significance. A difference large enough to matter in practice, regardless of *statistical significance*. A drug that lowers blood pressure by 0.5 mmHg might be statistically significant with a large enough sample but clinically irrelevant.

Clustering. Grouping observations (samples, genes, cells) by similarity. Common methods: *k-means* (requires specifying k), *hierarchical* (produces a dendrogram), *DBSCAN* (finds clusters of arbitrary shape).

Coefficient of variation (CV). The ratio of the *standard deviation* to the *mean*, expressed as a percentage. Useful for comparing variability across measurements with different scales. $CV = (SD / \text{mean}) * 100\%$.

Confidence interval (CI). A range of values that, if the experiment were repeated many times, would contain the true parameter value in the stated percentage of cases. A 95% CI does *not* mean "95% probability the true value is in this range" (that is the Bayesian *credible interval*).

Confounding variable. A variable that influences both the independent and dependent variables, creating a spurious association. Age confounds many gene expression studies because both expression and disease risk change with age.

Continuous variable. A variable that can take any value within a range (e.g., expression level, concentration, temperature). Contrasts with *categorical variable*.

Correlation. A measure of linear association between two variables. *Pearson* correlation (r) measures linear relationships; *Spearman* correlation (ρ) measures monotonic relationships. Ranges from -1 to +1.

Cox proportional hazards. A regression model for *survival analysis* that estimates the effect of covariates on the hazard (instantaneous risk) of an event. Does not assume a particular survival distribution. Reports *hazard ratios*.

Credible interval. The *Bayesian* analog of a *confidence interval*. A 95% credible interval means there is a 95% probability the true parameter lies within the interval, given the data and prior. Requires specifying a *prior distribution*.

Degrees of freedom (df). The number of independent values that can vary in a statistical calculation. For a t-test with n_1 and n_2 observations, df is approximately $n_1 + n_2 - 2$ (for Student's) or a more complex formula (for Welch's).

Differential expression. A gene is differentially expressed if its expression level differs significantly between conditions (e.g., treated vs. control). Typically assessed with a *negative binomial* model (DESeq2, edgeR) and *BH correction*.

Effect size. A measure of the magnitude of a difference or association, independent of sample size. Common measures: *Cohen's d* (standardized mean difference), *odds ratio*, *hazard ratio*, *R-squared*.

Cohen's d. A standardized *effect size* for the difference between two means: $d = (\text{mean}_1 - \text{mean}_2) / \text{pooled_SD}$. Conventions: 0.2 = small, 0.5 = medium, 0.8 = large.

Eta-squared. An *effect size* for ANOVA representing the proportion of total variance explained by the group factor. Analogous to *R-squared* in regression.

Empirical distribution. The distribution derived directly from observed data, without assuming a parametric form. Used in *permutation tests* and *bootstrap* methods.

Enrichment analysis. Testing whether a set of genes (e.g., differentially expressed genes) contains more members of a particular pathway or GO category than expected by chance. Uses the *hypergeometric distribution* or *GSEA*.

False discovery rate (FDR). The expected proportion of rejected null hypotheses that are false positives. Controlled by *Benjamini-Hochberg* correction. At $FDR = 0.05$, you expect 5% of your “significant” findings to be false.

Family-wise error rate (FWER). The probability of making at least one *Type I error* across all tests. Controlled by *Bonferroni* and *Holm* corrections. More conservative than *FDR* control.

Fisher’s exact test. A test for association in 2x2 contingency tables that computes the exact probability under the *hypergeometric distribution*. Preferred over *chi-square* when sample sizes are small or expected cell counts are below 5.

Fold change. The ratio of a value in one condition to the value in another. A fold change of 2 means doubled; 0.5 means halved. Often reported on the log2 scale: $\log_2(FC) = 1$ means doubled.

Forest plot. A visualization for *meta-analysis* showing effect sizes and confidence intervals from multiple studies, plus a combined estimate. Each study is a horizontal line; the diamond shows the pooled effect.

Frequentist statistics. The dominant framework in biostatistics, based on long-run frequencies. P-values, confidence intervals, and hypothesis tests are frequentist concepts. Contrasts with *Bayesian statistics*.

GSEA (Gene Set Enrichment Analysis). A method that tests whether a predefined set of genes shows concordant differences between conditions. Unlike overrepresentation analysis, GSEA uses the full ranked gene list rather than an arbitrary significance cutoff.

GWAS (Genome-Wide Association Study). A study design that tests hundreds of thousands to millions of genetic variants for association with a phenotype. Requires stringent *multiple testing correction* (typically $p < 5 \times 10^{-8}$).

Hazard ratio (HR). The ratio of hazard rates between two groups in *survival analysis*. HR = 0.7 means the treatment group has 30% lower instantaneous risk. HR = 1 means no difference. Estimated by *Cox proportional hazards* regression.

Heteroscedasticity. Unequal variance across groups or across the range of a predictor. Violates assumptions of standard *t-tests* and *linear regression*. Detected by residual plots. Addressed by Welch's tests or robust standard errors.

Hierarchical clustering. A *clustering* method that builds a tree (dendrogram) by iteratively merging (agglomerative) or splitting (divisive) clusters. Common linkage methods: ward, complete, average, single.

Holm correction. A step-down *multiple testing correction* that is uniformly more powerful than *Bonferroni* while still controlling the *FWER*. Rejects the smallest *p*-value at α/m , the next at $\alpha/(m-1)$, and so on.

Homoscedasticity. Equal variance across groups or across the range of a predictor. An assumption of Student's *t-test* and standard *linear regression*.

Hypothesis testing. A formal procedure for deciding between two competing hypotheses (*null* and *alternative*) based on observed data. Produces a *test statistic* and *p-value*.

Interquartile range (IQR). The range between the 25th and 75th percentiles. Contains the middle 50% of the data. A robust measure of spread, less sensitive to outliers than *standard deviation*.

Kaplan-Meier estimator. A non-parametric method for estimating survival probabilities over time, accounting for censored observations. Produces the familiar step-function survival curve.

k-means clustering. A *clustering* algorithm that partitions *n* observations into *k* clusters by minimizing within-cluster variance. Requires specifying *k* in advance. Sensitive to initialization; run multiple times.

Kruskal-Wallis test. The non-parametric alternative to one-way *ANOVA*. Tests whether multiple groups have the same distribution. Based on ranks rather than raw values.

Linear regression. A model that predicts a continuous outcome as a linear function of one or more predictors: $y = \beta_0 + \beta_1 * x_1 + \dots + \epsilon$. Assumes normally distributed residuals and constant variance.

Log-rank test. A test for comparing *survival curves* between two or more groups. Tests the *null hypothesis* that the groups have equal hazard functions. The standard test for comparing *Kaplan-Meier* curves.

Logistic regression. A *regression* model for binary outcomes (0/1, yes/no, case/control). Models the log-odds of the outcome as a linear function of predictors. Reports *odds ratios*.

Manhattan plot. A visualization for *GWAS* results showing $-\log_{10}(\text{p-value})$ vs. genomic position. Significant associations appear as tall peaks above the genome-wide significance line. Named for its skyline-like appearance.

Mann-Whitney U test. A non-parametric alternative to the two-sample *t-test*. Tests whether one group tends to have larger values than the other. Based on ranks. Also called the Wilcoxon rank-sum test.

Mean. The arithmetic average. Sum of all values divided by the number of values. Sensitive to outliers. For skewed data, the *median* is often more representative.

Median. The middle value when data is sorted. Robust to outliers. Preferred over *mean* for skewed distributions. The 50th percentile.

Meta-analysis. A statistical method for combining results from multiple independent studies to produce a single pooled estimate. Uses weighted averages based on study precision. Visualized with *forest plots*.

Mixed-effects model. A *regression* model that includes both fixed effects (variables of interest) and random effects (grouping variables like patient, batch, or site). Accounts for non-independence in hierarchical or repeated-measures data.

Mode. The most frequently occurring value (discrete data) or the peak of the density curve (continuous data). A distribution can be *bimodal* or multimodal.

Multiple testing correction. Adjusting p-values or significance thresholds when performing many simultaneous tests to control the overall error rate. Methods include *Bonferroni*, *Holm*, *Benjamini-Hochberg*, and *permutation* testing.

Negative binomial distribution. A *discrete distribution* for count data that allows the variance to exceed the mean (*overdispersion*). The standard model for RNA-seq differential expression (DESeq2, edgeR).

Non-parametric test. A statistical test that does not assume a specific parametric distribution (e.g., normality). Examples: *Mann-Whitney U*, *Kruskal-Wallis*, *Wilcoxon signed-rank*. Generally less powerful than parametric tests when assumptions hold.

Normal distribution. The symmetric, bell-shaped distribution described by a *mean* and *standard deviation*. Many biological measurements are approximately normal, especially after log transformation. The basis for most parametric tests via the *central limit theorem*.

Null hypothesis (H0). The hypothesis that there is no effect, no difference, or no association. Statistical tests assess evidence against the null. Example: "There is no difference in gene expression between treated and control groups."

Odds ratio (OR). The ratio of odds of an event in one group to odds in another. $OR = 1$ means no association. $OR > 1$ means increased odds. Commonly reported in *logistic regression* and case-control studies.

Outlier. An observation that is unusually far from the rest of the data. In a *box plot*, observations beyond $1.5 * IQR$ from the quartiles. Can indicate errors, biological extremes, or violations of assumptions.

Overdispersion. Variance exceeding the mean in count data. Poisson models assume variance = mean; when this is violated, *negative binomial* models are more appropriate. Nearly universal in RNA-seq data.

Paired test. A test that accounts for the natural pairing of observations (e.g., before/after measurements on the same subject). More powerful than unpaired tests because pairing removes between-subject variability.

Parametric test. A statistical test that assumes the data follows a specific probability distribution (usually *normal*). Examples: *t-test*, *ANOVA*, *Pearson correlation*. More powerful than *non-parametric tests* when assumptions hold.

PCA (Principal Component Analysis). A *dimensionality reduction* method that finds orthogonal linear combinations of variables (principal components) that capture maximum variance. PC1 captures the most variance, PC2 the next most, and so on.

PDF (Probability Density Function). For continuous distributions, the function whose integral over an interval gives the probability of falling in that interval. The height of the curve at a point is not a probability.

Pearson correlation. A measure of linear association between two continuous variables. Ranges from -1 (perfect negative) to +1 (perfect positive). Assumes both variables are approximately *normally distributed*.

Permutation test. A non-parametric test that estimates the null distribution by repeatedly shuffling group labels and recomputing the test statistic. The p-value is the proportion of permuted statistics as extreme as the observed. Makes minimal assumptions.

PMF (Probability Mass Function). For *discrete distributions*, the function that gives the probability of each possible value. $P(X = k) = \text{pmf}(k)$.

Posterior distribution. In *Bayesian statistics*, the updated distribution of a parameter after observing data. Combines the *prior distribution* with the likelihood. Posterior is proportional to prior times likelihood.

Power. The probability of correctly rejecting the *null hypothesis* when it is false. $\text{Power} = 1 - \text{beta}$. Conventionally set at 0.8 (80%). Depends on sample size, *effect size*, *alpha*, and variability.

Prior distribution. In *Bayesian statistics*, the distribution representing beliefs about a parameter before observing data. Can be informative (based on previous studies) or non-informative (vague).

p-value. The probability of observing data as extreme as (or more extreme than) what was observed, assuming the *null hypothesis* is true. It is *not* the probability that the null hypothesis is true.

Q-Q plot (Quantile-Quantile plot). A diagnostic plot that compares the quantiles of observed data to the quantiles of a theoretical distribution (usually *normal*). Points on the diagonal indicate good fit. Used to check normality and to assess genomic inflation in *GWAS*.

Quartile. Values that divide the data into four equal parts. Q1 (25th percentile), Q2 (50th percentile = *median*), Q3 (75th percentile). The *IQR* is $Q3 - Q1$.

Randomization. Random assignment of subjects to treatment groups to ensure that confounding variables are equally distributed across groups. The gold standard for causal inference in clinical trials.

Relative risk (RR). The ratio of risk (probability) of an event in the exposed group to risk in the unexposed group. $RR = 1$ means no association. Reported in cohort studies and clinical trials. Distinct from *odds ratio*.

Resampling. Methods that generate new datasets from the observed data by sampling with or without replacement. Includes *bootstrap* and *permutation* methods. Useful when parametric assumptions are questionable.

Residual. The difference between an observed value and the value predicted by a model. Patterns in residuals indicate model misspecification. Residual plots are essential diagnostics for *regression*.

ROC curve (Receiver Operating Characteristic). A plot of sensitivity (true positive rate) vs. 1-specificity (false positive rate) at various classification thresholds. The *AUC* summarizes overall discrimination.

R-squared (coefficient of determination). The proportion of variance in the outcome explained by the model. Ranges from 0 to 1. $R\text{-squared} = 0.7$ means the model explains 70% of the variability. Always increases with more predictors; use *adjusted R-squared* for model comparison.

Sample size. The number of observations in a study. Larger samples give more precise estimates and greater *power*. Sample size calculations use *power analysis* to determine the minimum n needed for a given *effect size* and *alpha*.

Sensitivity. The proportion of true positives correctly identified. $\text{Sensitivity} = TP / (TP + FN)$. Also called the true positive rate or recall. A test with high sensitivity rarely misses real positives.

Spearman correlation. A non-parametric measure of monotonic association. Computed by applying *Pearson* correlation to the ranks of the data. Does not assume linearity or normality.

Specificity. The proportion of true negatives correctly identified. $\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$. A test with high specificity rarely produces false positives.

Standard deviation (SD). The square root of *variance*. Measures the typical deviation of observations from the *mean*. Reported in the same units as the data. About 68% of normal data falls within one SD of the mean.

Standard error (SE). The *standard deviation* of a sampling distribution. SE of the mean = $\text{SD} / \sqrt{n}$. Decreases with larger sample sizes. Used to construct *confidence intervals*.

Survival analysis. Statistical methods for analyzing time-to-event data with censoring. Key tools: *Kaplan-Meier* estimator, *log-rank test*, *Cox proportional hazards* model.

t-test. A test for comparing means of two groups. Student's t-test assumes equal variances; Welch's t-test does not. For paired observations, use the paired t-test. Assumes approximately *normal* data or large samples.

Tukey HSD. A post-hoc test for all pairwise comparisons after a significant *ANOVA*. Controls the *family-wise error rate* across all comparisons. Preferred when comparing all group pairs.

Type I error. Rejecting the *null hypothesis* when it is actually true (false positive). The probability of a Type I error is *alpha*. In genomics, controlled by *multiple testing correction*.

Type II error. Failing to reject the *null hypothesis* when it is actually false (false negative). The probability of a Type II error is *beta*. Reduced by increasing sample size or *effect size*.

Variance. The average squared deviation from the *mean*. $\text{Var} = \sum((x_i - \text{mean})^2) / (n - 1)$ for a sample. Measures the spread of a distribution. The square of the *standard deviation*.

VIF (Variance Inflation Factor). A measure of multicollinearity in *regression*. VIF = 1 means no correlation between predictors; VIF > 5-10 suggests problematic multicollinearity. Computed for each predictor.

Volcano plot. A visualization for *differential expression* showing $-\log_{10}(\text{p-value})$ vs. $\log_2(\text{fold change})$. Significant and biologically meaningful genes appear in the upper left (downregulated) and upper right (upregulated) corners.

Wilcoxon signed-rank test. A non-parametric alternative to the *paired t-test*. Tests whether the median of paired differences is zero. Based on the ranks of the absolute differences.

Appendix E: BioLang Statistics Quick Reference

Every statistical function in BioLang, organized for quick lookup.

This appendix lists BioLang's statistical builtins by category. For each function, you will find the signature, a brief description, and a one-liner example. Functions are listed within each category in roughly the order you would learn them.

Descriptive Statistics

Function	Description	Example
<code>mean(x)</code>	Arithmetic mean	<code>mean([2, 4, 6]) → 4.0</code>
<code>median(x)</code>	Middle value	<code>median([1, 3, 7]) → 3.0</code>
<code>stdev(x)</code>	Sample standard deviation	<code>stdev([2, 4, 6]) → 2.0</code>
<code>var(x)</code>	Sample variance	<code>var([2, 4, 6]) → 4.0</code>
<code>min(x)</code>	Minimum value	<code>min([3, 1, 4]) → 1</code>
<code>max(x)</code>	Maximum value	<code>max([3, 1, 4]) → 4</code>
<code>sum(x)</code>	Sum of all values	<code>sum([1, 2, 3]) → 6</code>
<code>len(x)</code>	Number of elements	<code>len([1, 2, 3]) → 3</code>
<code>quantile(x, p)</code>	p-th quantile	<code>quantile([1,2,3,4,5], 0.75) → 4.0</code>
<code>summary(x)</code>	Summary statistics	<code>summary(data) → record with min, Q1, median, Q3, max, mean</code>
<code>round(x, digits)</code>	Round to n decimal places	<code>round(3.14159, 2) → 3.14</code>
<code>abs(x)</code>	Absolute value	<code>abs(-3.5) → 3.5</code>
<code>sqrt(x)</code>	Square root	<code>sqrt(16) → 4.0</code>
<code>log2(x)</code>	Base-2 logarithm	<code>log2(8) → 3.0</code>
<code>log10(x)</code>	Base-10 logarithm	<code>log10(1000) → 3.0</code>

Probability Distributions

Each distribution has four functions following the `d/p/q/r` convention: `d` (density/mass), `p` (cumulative probability), `q` (quantile/inverse CDF), `r` (random samples). All parameters are positional.

Continuous

```
# Normal: dnorm(x, mu, sigma), pnorm(x, mu, sigma), qnorm(p, mu,
sigma), rnorm(n, mu, sigma)
dnorm(0, 0, 1)          # Density at x=0 for standard normal
pnorm(1.96, 0, 1)       # P(X <= 1.96) ≈ 0.975
qnorm(0.975, 0, 1)     # Quantile at p=0.975 ≈ 1.96
rnorm(100, 0, 1)       # Generate 100 standard normal values

# Student's t: dt(x, df), pt(x, df), qt(p, df), rt(n, df)
dt(0, 10)
pt(2.228, 10)
qt(0.975, 10)
rt(100, 10)

# F: df(x, df1, df2), pf(x, df1, df2), qf(p, df1, df2), rf(n, df1,
df2)
df(1.5, 5, 20)
pf(3.0, 5, 20)
qf(0.95, 5, 20)
rf(100, 5, 20)

# Chi-square: dchisq(x, df), pchisq(x, df), qchisq(p, df), rchisq(n,
df)
dchisq(5.0, 5)
pchisq(11.07, 5)
qchisq(0.95, 5)
rchisq(100, 5)

# Beta: dbeta(x, alpha, beta), pbeta(x, alpha, beta), qbeta(p,
alpha, beta), rbeta(n, alpha, beta)
dbeta(0.3, 2, 5)
pbeta(0.5, 2, 5)
qbeta(0.95, 2, 5)
rbeta(100, 2, 5)

# Gamma: dgamma(x, shape, rate), pgamma(x, shape, rate), qgamma(p,
shape, rate), rgamma(n, shape, rate)
dgamma(1.0, 2, 1)
pgamma(3.0, 2, 1)
qgamma(0.95, 2, 1)
rgamma(100, 2, 1)

# Exponential: dexp(x, rate), pexp(x, rate), qexp(p, rate), rexp(n,
rate)
dexp(1.0, 0.5)
```

```
pexp(2.0, 0.5)
qexp(0.95, 0.5)
rexp(100, 0.5)
```

```
# Log-Normal: dlnorm(x, mu, sigma), plnorm(x, mu, sigma), qlnorm(p,
mu, sigma), rlnorm(n, mu, sigma)
dlnorm(1.0, 0, 1)
plnorm(2.0, 0, 1)
qlnorm(0.95, 0, 1)
rlnorm(100, 0, 1)
```

```
# Uniform: dunif(x, min, max), punif(x, min, max), qunif(p, min,
max), runif(n, min, max)
dunif(0.5, 0, 1)
punif(0.7, 0, 1)
qunif(0.5, 0, 1)
runif(100, 0, 1)
```

Discrete

```
# Binomial: dbinom(k, n, p), pbinom(k, n, p), qbinom(q, n, p),
rbinom(size, n, p)
dbinom(10, 20, 0.5)      # P(X = 10)
pbinom(10, 20, 0.5)      # P(X <= 10)
qbinom(0.5, 20, 0.5)     # Smallest k with P(X <= k) >= 0.5
rbinom(100, 20, 0.5)     # Generate 100 random values

# Poisson: dpois(k, lambda), ppois(k, lambda), qpois(q, lambda),
rpois(n, lambda)
dpois(5, 5.0)
ppois(7, 5.0)
qpois(0.95, 5.0)
rpois(100, 5.0)

# Negative Binomial: dnbinom(k, mu, size), pnbinom(k, mu, size),
qnbinom(q, mu, size), rnbinom(n, mu, size)
dnbinom(8, 10, 5)
pnbinom(15, 10, 5)
qnbinom(0.95, 10, 5)
rnbinom(100, 10, 5)

# Hypergeometric: dhyper(k, K, N_minus_K, n), phyper(k, K,
N_minus_K, n), qhyper(q, K, N_minus_K, n), rhyper(size, K,
N_minus_K, n)
dhyper(5, 200, 19800, 500)
phyper(5, 200, 19800, 500)
qhyper(0.95, 200, 19800, 500)
rhyper(100, 200, 19800, 500)
```

Hypothesis Tests

Comparing Two Groups

Function	Description	Example
<code>ttest(a, b)</code>	Welch's two-sample t-test	<code>ttest(ctrl, treat)</code>
<code>ttest_paired(a, b)</code>	Paired t-test	<code>ttest_paired(before, after)</code>
<code>ttest_one(x, mu)</code>	One-sample t-test	<code>ttest_one(diffs, 0)</code>
<code>wilcoxon(a, b)</code>	Wilcoxon rank-sum / signed-rank test	<code>wilcoxon(ctrl, treat)</code>

Comparing Multiple Groups

Function	Description	Example
<code>anova(groups)</code>	One-way ANOVA (auto-detects Welch/Kruskal-Wallis)	<code>anova([g1, g2, g3])</code>

Post-hoc comparisons: Follow a significant ANOVA with pairwise tests and p-value correction:

Conceptual or diagnostic example; not directly executable.

Pairwise t-tests with Bonferroni correction

```
let pvals = [] for i in 0..len(groups) { for j in (i+1)..len(groups) { let result =  
ttest(groups[i], groups[j]) pvals = pvals + [result.p_value] } } let adjusted =  
p_adjust(pvals, "bonferroni")
```

Categorical Data

Function	Description	Example
<code>chi_square(observed, expected)</code>	Chi-square test	<code>chi_square(observed, expected)</code>
<code>fisher_exact(a, b, c, d)</code>	Fisher's exact test (2x2)	<code>fisher_exact(10, 5, 3, 12)</code>

Effect sizes for categorical data are computed inline:

**Conceptual or diagnostic example;
not directly executable.**

Odds ratio

let $or = (a * d) / (b * c)$

Relative risk

$$\text{let } rr = (a / (a + b)) / (c / (c + d))$$

Correlation

Function	Description	Example
<code>cor(x, y)</code>	Pearson correlation	<code>cor(expr, meth)</code>
<code>spearman(x, y)</code>	Spearman rank correlation	<code>spearman(expr, meth)</code>
<code>kendall(x, y)</code>	Kendall tau correlation	<code>kendall(expr, meth)</code>

Regression

Function	Description	Example
<code>lm(x, y)</code>	Simple linear regression	<code>lm(age, expression)</code>
<code>lm(formula, table)</code>	Multiple linear regression	<code>lm(~expression ~ age + sex + batch, data)</code>
<code>glm(formula, table, family?)</code>	Generalized linear model	<code>glm(~outcome ~ age + marker, data, "binomial")</code>

Supported GLM families: "binomial" (or "logistic"), "gaussian" (or "linear"), and "poisson" .

Access model results:

```
let age = [34, 41, 52, 59, 63, 70]
let expression = [8.1, 8.8, 9.4, 10.2, 10.6, 11.1]
let model = lm(age, expression)
print("R-squared: " + str(round(model.r_squared, 3)))
let residuals = range(0, len(age))
  |> map(|i| expression[i] - (model.intercept + model.slope *
age[i]))
histogram(residuals, {title: "Residual Distribution"})
```

Multiple Testing Correction

Function	Description	Example
<code>p_adjust(pvals, method)</code>	Adjust p-values	<code>p_adjust(pvals, "BH")</code>

Supported methods: "bonferroni", "holm", "BH" (Benjamini-Hochberg), "BY" (Benjamini-Yekutieli).

```
# Typical genomics workflow: test all genes, then correct
let pvals = genes |> map(|g| ttest(g.ctrl, g.treat).p_value)
let padj = p_adjust(pvals, "BH")
let sig_count = padj |> filter(|p| p < 0.05) |> len()
print("Significant genes (FDR < 0.05): " + str(sig_count))
```

Dimensionality Reduction and Clustering

Function	Description	Example
<code>pca(data)</code>	Principal Component Analysis	<code>pca(expr_matrix)</code>
<code>kmeans(data, k)</code>	k-means clustering	<code>kmeans(data, 3)</code>
<code>hclust(data, method)</code>	Hierarchical clustering	<code>hclust(data, "ward")</code>
<code>dbscan(data, eps, min_pts)</code>	DBSCAN clustering	<code>dbscan(data, 0.5, 5)</code>

```
# PCA then clustering
let expr_matrix = [
  [8.2, 7.9, 3.1, 2.8],
  [8.6, 8.1, 3.4, 3.0],
  [3.0, 3.3, 8.4, 8.0],
  [2.7, 3.0, 8.8, 8.2]
]
let result = pca(expr_matrix)
pca_plot(matrix(expr_matrix), {title: "Sample PCA"})

# Cluster the same samples into two groups
let clusters = kmeans(expr_matrix, 2)
```

Statistical Visualization

SVG Plots (file output)

Function	Description	Example
<code>histogram(data, options)</code>	Histogram	<code>histogram(data, {bins: 30, title: "Distribution"})</code>
<code>density(data, options)</code>	Kernel density estimate	<code>density(data, {title: "Density"})</code>
<code>violin(data, options)</code>	Violin plot	<code>violin([g1, g2], {labels: ["A", "B"], title: "Groups"})</code>
<code>heatmap(table, options)</code>	Heatmap with optional clustering	<code>heatmap(matrix, {cluster_rows: true, title: "Expression"})</code>
<code>volcano(table, options)</code>	Volcano plot for DE results	<code>volcano(de_results, {fc_threshold: 1.0, title: "DE"})</code>
<code>manhattan(table, options)</code>	Manhattan plot for GWAS	<code>manhattan(gwas_results, {significance_line: 5e-8, title: "GWAS"})</code>
<code>qq_plot(data, options)</code>	Q-Q plot (normality check)	<code>qq_plot(residuals, {title: "Normality Check"})</code>
<code>forest_plot(table, options)</code>	Forest plot for meta-analysis	<code>forest_plot(meta_tbl, {null_value: 0, title: "Meta-analysis"})</code>
<code>roc_curve(table, options)</code>	ROC curve	<code>roc_curve(roc_tbl, {title: "Classifier ROC"})</code>
<code>pca_plot(result, options)</code>	PCA scatter plot	<code>pca_plot(result, {title: "PCA"})</code>

Function	Description	Example
<code>plot(table, options)</code>	General line/scatter plot	<code>plot(tbl, {type: "line", title: "Trend"})</code>

ASCII Plots (terminal output)

Function	Description	Example
<code>scatter(x, y)</code>	ASCII scatter plot	<code>scatter(age, expression)</code>
<code>boxplot(data, options)</code>	ASCII box plot	<code>boxplot(table({"Ctrl": g1, "Treat": g2}), {title: "Comparison"})</code>
<code>bar_chart(data, options?)</code>	ASCII bar chart from a record or table	<code>bar_chart(count_table, {label: "group", value: "count"})</code>
<code>sparkline(data)</code>	Inline sparkline	<code>sparkline(timeseries)</code>
<code>hist(data, options)</code>	ASCII histogram	<code>hist(data, {bins: 20})</code>

Note: All visualization options are passed as a record (second argument): `fn(data, {key: value, ...})`. Options are always optional — you can call any plot function with just the data argument.

Resampling and Simulation

BioLang provides building blocks for resampling methods rather than dedicated functions:

```
# Bootstrap confidence interval for the median
let data = [2.3, 4.1, 3.7, 5.2, 4.8, 3.1, 6.0, 4.4]
let n_boot = 10000
let boot_medians = range(0, n_boot) |> map(|i| {
  let resample = range(0, len(data)) |> map(|j| data[random_int(0,
len(data) - 1)])
  median(resample)
})
let sorted = sort(boot_medians)
let lo = sorted[round(n_boot * 0.025, 0)]
let hi = sorted[round(n_boot * 0.975, 0)]
print("95% CI: [" + str(lo) + ", " + str(hi) + "]")

# Permutation test
let observed_diff = abs(mean(treated) - mean(control))
let combined = treated + control
let n_perm = 10000
let null_diffs = range(0, n_perm) |> map(|i| {
  let shuffled = shuffle(combined)
  let perm_a = shuffled[0..len(treated)]
  let perm_b = shuffled[len(treated)..len(combined)]
  abs(mean(perm_a) - mean(perm_b))
})
let p_value = len(null_diffs |> filter(|d| d >= observed_diff)) /
n_perm
```


Utility Functions

Function	Description	Example
<code>shuffle(x)</code>	Random permutation	<code>shuffle(labels)</code>
<code>random_int(a, b)</code>	Random integer in [a, b]	<code>random_int(0, 99)</code>
<code>sort(x)</code>	Sort values ascending	<code>sort([3, 1, 2]) → [1, 2, 3]</code>
<code>range(a, b)</code>	Integer sequence [a, b)	<code>range(0, 10) → [0, 1, ..., 9]</code>
<code>len(x)</code>	Number of elements	<code>len([1, 2, 3]) → 3</code>
<code>str(x)</code>	Convert to string	<code>str(42) → "42"</code>
<code>table(rows, cols, fill)</code>	Create a table	<code>table(10, 3, 0)</code>

Power Analysis

BioLang does not have dedicated power analysis functions. Compute sample sizes using distribution quantiles and effect size formulas:

```
# Sample size for two-sample t-test
# H0: mu1 = mu2, H1: mu1 != mu2
let effect_size = 0.5      # Cohen's d
let alpha = 0.05
let power = 0.80
let z_alpha = qnorm(1 - alpha / 2, 0, 1)    # 1.96
let z_beta = qnorm(power, 0, 1)             # 0.842
let n_per_group = round(2 * ((z_alpha + z_beta) / effect_size) ** 2,
0)
print("Required n per group: " + str(n_per_group))

# Cohen's d (inline)
let d = abs(mean(a) - mean(b)) / sqrt(((len(a) - 1) * var(a) +
(len(b) - 1) * var(b)) / (len(a) + len(b) - 2))
```

Bayesian Methods

BioLang supports Bayesian analysis through conjugate update formulas computed inline:

```
# Beta-Binomial conjugate update
# Prior: Beta(alpha, beta); Data: k successes in n trials
# Posterior: Beta(alpha + k, beta + n - k)
let prior_a = 1.0
let prior_b = 1.0
let k = 15
let n = 20
let post_a = prior_a + k
let post_b = prior_b + (n - k)
let post_mean = post_a / (post_a + post_b)
print("Posterior mean: " + str(round(post_mean, 3)))

# 95% credible interval via Beta quantiles
let ci_lo = qbeta(0.025, post_a, post_b)
let ci_hi = qbeta(0.975, post_a, post_b)
print("95% CI: [" + str(round(ci_lo, 3)) + ", " + str(round(ci_hi, 3)) + "]\")

# Normal-Normal conjugate update
# Prior: N(mu0, sigma0^2); Data: n observations with mean x_bar and
known sigma
let prior_mu = 0.0
let prior_prec = 1.0 / (10.0 ** 2) # prior precision = 1/sigma0^2
let data_prec = len(data) / (stdev(data) ** 2)
let post_prec = prior_prec + data_prec
let post_mu = (prior_prec * prior_mu + data_prec * mean(data)) /
post_prec
let post_sd = sqrt(1.0 / post_prec)
```

Survival Analysis

BioLang provides basic building blocks for survival analysis. For simple comparisons:

```
# Conceptual or diagnostic example; not directly executable.
# Compare survival times between two groups
let result = ttest(arm1_times, arm2_times)
print("p-value: " + str(round(result.p_value, 4)))

# Median survival
let med_a = sort(arm1_times)[len(arm1_times) / 2]
let med_b = sort(arm2_times)[len(arm2_times) / 2]
print("Median survival – Arm A: " + str(med_a) + ", Arm B: " +
str(med_b))

# Approximate hazard ratio from median ratio
let hr = med_b / med_a

# Regression on survival times
let model = lm(~survival_time ~ age + stage + treatment,
patient_data)
```